ALI EDUCATIONAL

Complete system reference | Source snapshot: 7 November 2026

Features, architecture, data model, build and test evidence, approved specifications and complete text-source appendix.

This report documents the current project found on disk. Source configuration: version 1.0.12 (code 12), Room schema 8, minimum Android API 26, compile/target API 36, JVM target 17. Installed phone version and current device behavior were not inspected in this task.

Evidence levels: SOURCE PRESENT means code exists; STORED RESULT means an existing test report was found; SPECIFICATION means intended behavior, not proof of implementation. No app tests or build were rerun for this documentation task.

Source location: C:\Users\Musid\.gemini\antigravity\scratch\AliEducational

Text source appendix: 87 files. Includes Kotlin implementation/tests, Android XML and selected build configuration. Binary assets, generated build internals, dependency code, signing keys, keystore.properties and local.properties are intentionally not reproduced. This is a reference PDF, not a buildable source checkout.

Reading guide

Part I: architecture and current feature map. Part II: database and code inventory. Part III: existing build/test evidence and limitations. Part IV: detailed approved handoffs. Part V: line-numbered implementation, resource and test code. PDF bookmarks identify major sections and each source file.

Part I - System architecture

The project uses Kotlin and Jetpack Compose/Material 3 for Android screens, Navigation Compose for routes, ViewModels and coroutine Flow state, Hilt dependency injection, Room/SQLite persistence and DataStore preferences. Apache POI handles spreadsheets; CameraX and ZXing support barcode scanning. Android biometric APIs protect entry. A native RemoteViews-style launcher widget has separate XML resources and a provider.

```
Android launcher / MainActivity / slim widget
    -> authentication and route selection
    -> Compose screens and navigation
    -> AliViewModel / MainViewModel
    -> AppRepository and domain calculations
    -> Room DAOs -> SQLite entities and migrations
    -> DataStore preferences / managed attachment files

Service boundaries:
  ReceiptService -> customer PDF / sharing
  ImportExportService + CashTrackerExcelService -> spreadsheets
  BackupRestoreService -> archive / staging / restore
  ShopRecordsExcelGenerator -> readable workbook
  TwoPhoneSyncService -> local network communication
```

This diagram describes architectural responsibilities inferred from the source organization. Individual screens/services may access data directly; the complete source appendix is authoritative for exact call paths.

Inventory

Products, barcodes, receipt of stock, removals, imports, cost calculations and stock history. Inspect ui/stock, CostCalculator and repository.

Billing

Cart, customer selection, payment, held bills, receipt, history and barcode analyzer/screens are present under ui/billing.

Accounts

Customer/supplier account UI and payment persistence are present. Shared financial identity and allocation behavior require transaction-level verification.

Cash and expenses

CashTrackerScreen, ExpensesScreen, cash categories and shop expense records support financial entry/reporting. Do not equate all inflows with revenue or outflows with expenses.

Pigmy

PigmySavingsScreen and scheme/deposit/withdrawal entities are present. Savings are transfers, not sales/profit.

Reports

ProfitScreen and CostCalculator contain reporting/calculation code. Exact definitions and shared-expense allocation must be checked against approved requirements.

Backup

BackupRestoreService and ShopRecordsExcelGenerator exist alongside backup UI. Their presence does not certify that every setting/file is recoverable.

Widget

AliSlimBarWidgetProvider and widget_slim_bar.xml exist with dedicated cash-in/out, bill and stock drawable resources.

Appearance and access

Theme/color/type resources, setup/settings screens and BiometricAuthManager are present. Actual light-mode contrast and lock lifecycle need device evidence.

Two-phone operation

TwoPhoneSyncService uses ServerSocket/Socket and HttpURLConnection imports. Do not describe it as verified authenticated encrypted synchronization merely from its presence; transport and role enforcement require review.

Financial relationships and required invariants

A product purchase receives stock and creates supplier liability only for unpaid consideration. Later supplier settlement changes cash/bank and payable, not stock again.

A completed customer bill records revenue and saved cost; unpaid value becomes customer due. Later repayment settles due and is not new revenue.

Product gross profit = net product sales minus associated saved COGS. Service revenue and recorded operating expenses must remain explicitly classified.

Cash balance, profit, customer dues, supplier dues, Pigmy savings and expected stock profit are distinct measures. Each must reconcile independently.

Refunds/reversals need links to original records, correct effective dates and exactly-once effects. Backup restore must not replay historic postings.

Part II - Data model and source inventory

Entity declarations found in Entities.kt:

ProductEntity, ProductBarcodeEntity, StockMovementEntity, SupplierEntity, CustomerEntity, PurchaseEntity, PurchaseLineEntity, SaleEntity, SaleLineEntity, AccountPaymentEntity, ShopExpenseEntity, CashCategoryEntity, ImportBatchEntity, HeldBillEntity, PigmySchemeEntity, PigmyDepositEntity, PigmyWithdrawalEntity, ProductGiveawayEntity, ServiceDefinitionEntity, SyncEventEntity

The complete field definitions, nullability, defaults, indexes and Room annotations are reproduced in the source appendix. The database declaration includes 20 entities and schema migration code. SQL relationships must be distinguished from application-only links; do not infer enforced foreign keys from field names alone.

app\src\main\AndroidManifest.xml | 67 lines | SHA-256
00b2219b14092d3e1509cfdc6bfdd097cd9430433e8e3ffb203a55d4fcc473c3

app\src\main\java\com\alieducational\app\AliEducationalApp.kt | 12 lines | SHA-256
5c180c9d8690566420fc373b9d8bf0d1eeea19af18eea53a943115f26a36f116

app\src\main\java\com\alieducational\app\data\local\AppDatabase.kt | 323 lines | SHA-256
0fe842e922f4ce7fa5aafcae136e3b2aa94e49d9dc878e497bd7971b84ea58e2

app\src\main\java\com\alieducational\app\data\local\dao\Daos.kt | 573 lines | SHA-256
fe3054c94f2f05995a30180533d2f99ae2023d70d8d0234e6ea9029874c602aa

app\src\main\java\com\alieducational\app\data\local\entity\Entities.kt | 329 lines | SHA-256
fe52d4d6087ee8516347240d9ec491e60b93b327381cfbe0b3a1e6188d4dd50a

app\src\main\java\com\alieducational\app\data\preferences\AppPreferences.kt | 209 lines | SHA-256
a9bfb3df4fe7558ae56bac4c0d1565a30ceb9a587159a6648960c755e2aa886d

app\src\main\java\com\alieducational\app\data\repository\AppRepository.kt | 1505 lines | SHA-256
3c92056936b062185e680a743e6675ed71fcc7028216bacc64db20fd88fd0779

app\src\main\java\com\alieducational\app\data\service\BackupRestoreService.kt | 510 lines | SHA-256
7e6ccb2d0fc75663a91a2c7c657c750c827a8342f4cc955cf51e7b025fffd8b2

app\src\main\java\com\alieducational\app\data\service\CashTrackerExcelService.kt | 709 lines | SHA-256
832024c71f79ee8e6c49978bc9c06c7015afd5a745d58f68d6efc53ead4cc2a8

app\src\main\java\com\alieducational\app\data\service\ImportExportService.kt | 548 lines | SHA-256
5c9ccb5d1991aa084893db62f3851be033c35597cb4e604ff68108427675cd9b

app\src\main\java\com\alieducational\app\data\service\ReceiptService.kt | 756 lines | SHA-256
a1d197732d9794b237d2678d007c3bc2a3a1851e9b3816efaad74440be9562cc

app\src\main\java\com\alieducational\app\data\service\ShopRecordsExcelGenerator.kt | 506 lines | SHA-256
baae704ee6ad384892541bc0c7d97886611992696d5b43ea203fac37adec772b

app\src\main\java\com\alieducational\app\data\service\TwoPhoneSyncService.kt | 480 lines | SHA-256
e01e9de2cc23da5b9283ba73a15c55547cb14b454c9a1a49effd2afe65a9464b

app\src\main\java\com\alieducational\app\di\DatabaseModule.kt | 108 lines | SHA-256
148fa5f62e62ca8e77ac5e0adad3816759b92267e10a5587ab22c4d33b8bef45

app\src\main\java\com\alieducational\app\domain\calculation\CostCalculator.kt | 273 lines | SHA-256
af46dd51a9187de408cb8e86358793fa99c069c99b70f2938033e63bd8c1181d

app\src\main\java\com\alieducational\app\MainActivity.kt | 182 lines | SHA-256
915044f97fcb1d5067fb58e32de19173c339d709a6e64fd48094128d82fc03c2

app\src\main\java\com\alieducational\app\MainViewModel.kt | 43 lines | SHA-256
d96f69be5170d498f9a4b142194ce051bc1493400fddf29594d51c2af5ae103a

app\src\main\java\com\alieducational\app\ui\accounts\AccountsScreen.kt | 1037 lines | SHA-256
5adee83aef71be95344e7b5066bef7ae589b26b93b3e40d99934b572add69854

app\src\main\java\com\alieducational\app\ui\AliViewModel.kt | 1464 lines | SHA-256
2482bb849afe796b595541916a7fe5474e2986aff7f4cb79dc8dca240cf2477d

app\src\main\java\com\alieducational\app\ui\auth\UnlockScreen.kt | 155 lines | SHA-256
5d9873fdfb6c383b74167d5d4e4a1ba3311381af2a02a733d789c1ca9684a0f4

app\src\main\java\com\alieducational\app\ui\billing\BarcodeAnalyzer.kt | 169 lines | SHA-256
e72ad48e4460e1c4de337c1482142ffc37745f957c435bbc205bed0d0dcbeb62

app\src\main\java\com\alieducational\app\ui\billing\BarcodeScannerScreen.kt | 475 lines | SHA-256
6042c69f27cc977fa12069cf302ecab539f4193b593d33d886f977de69dd4d72

app\src\main\java\com\alieducational\app\ui\billing\BillHistoryScreen.kt | 758 lines | SHA-256
3c5ffd6ea2d58188ba465b550c8390f89e9b1cb93798075b964950eb8973bc92

app\src\main\java\com\alieducational\app\ui\billing\BillingScreen.kt | 918 lines | SHA-256
e141a4cbf242a7fa248ca219660d3f8993a431c0f64b7e916e71cad1923b1a03

app\src\main\java\com\alieducational\app\ui\billing\BillReceiptScreen.kt | 890 lines | SHA-256
bf097f072d0881a9850fb80e3c743ef66a36790e34007e43a679e6d3d37357cc

app\src\main\java\com\alieducational\app\ui\billing\CustomerSelectionDialog.kt | 346 lines | SHA-256
98fe7cb97ec2420817b87a48e1b4d371bc6a0c9f9fdb1d3f5b27cc3643745d50

app\src\main\java\com\alieducational\app\ui\billing\HeldBillsDialog.kt | 247 lines | SHA-256
78af544fc929016a8eaa94ad83a4f7bd6e1903ad9c8c4ac844bb0bd881cf5b03

app\src\main\java\com\alieducational\app\ui\billing\PaymentScreen.kt | 733 lines | SHA-256
1fd98fc238ace3f54800a224db67ccf086f0fd5051af4f77a5171ed0856258b0

app\src\main\java\com\alieducational\app\ui\billing\ReviewBillScreen.kt | 800 lines | SHA-256
3f9b8d9ce231d98d4de6cabd5fb90cd8358edf10e9f03dc4ffc70496846bd036

app\src\main\java\com\alieducational\app\ui\home\HomeScreen.kt | 504 lines | SHA-256
5d5b8f5c348b7f3af998e91155925160b0556eca5adf569588f62da7c45956ae

app\src\main\java\com\alieducational\app\ui\more\BackupRestoreScreen.kt | 687 lines | SHA-256
e2938e22c61d130efe843a8e4fedcca1ba6c4142700825f4d06885f5816936c8

app\src\main\java\com\alieducational\app\ui\more\ExpensesScreen.kt | 346 lines | SHA-256
0ff468230cea57fef96709a019a9e71eac50d64fcb74d67284814091f6e9be09

app\src\main\java\com\alieducational\app\ui\more\MoreScreen.kt | 202 lines | SHA-256
05ae9a5a554098a9a393bf102eb420fa1f9b798d9d0bdb9a5c4441e74bea3da6

app\src\main\java\com\alieducational\app\ui\navigation\AppNavigation.kt | 86 lines | SHA-256
42e1a5e7aa209d7a40650cbe8b71597e1e25796c27adc10c8943b93720460400

app\src\main\java\com\alieducational\app\ui\navigation\BottomNavBar.kt | 69 lines | SHA-256
a25584707f5394f7e56a03b8b8e0a36d30d7e439f6466be9fffa1c33ac55bef5

app\src\main\java\com\alieducational\app\ui\navigation\NavHost.kt | 320 lines | SHA-256
b6e564c88e8df5b0d0341017b2c2b0a3a8e831d704d7ea043611763f817bd05f

app\src\main\java\com\alieducational\app\ui\pigmy\PigmySavingsScreen.kt | 1779 lines | SHA-256
0963ff01d082b866ac6c78cd461776310bc1062aa8181b260307a6253e689397

app\src\main\java\com\alieducational\app\ui\reports\CashTrackerScreen.kt | 1992 lines | SHA-256
007ad43e10856b5729f15b891413a4c934ca3838077d5010c3854fd64e0c0c9c

app\src\main\java\com\alieducational\app\ui\reports\ProfitScreen.kt | 1060 lines | SHA-256
80773e831d87a9e846961d02423553407312f352fb97cfffbed29f1c2a1bc21c

app\src\main\java\com\alieducational\app\ui\sales\RecordDailyTotalScreen.kt | 237 lines | SHA-256
63a1f9175ddaf4055cc09df64a81d8921ac12688a67dd2f4fc0b148d48fe340d

app\src\main\java\com\alieducational\app\ui\sales\RecordSaleScreen.kt | 690 lines | SHA-256
068e4786f3d74cd8c434163e7f1e5f8540aa815876a5524f20af0beb407a1f79

app\src\main\java\com\alieducational\app\ui\settings\SettingsScreen.kt | 440 lines | SHA-256
733adf52c58fc484aaa64d44171dc93f51121b3c11db0ec1640b050de629e8ca

app\src\main\java\com\alieducational\app\ui\setup\SetupScreen.kt | 215 lines | SHA-256
270c8f9468729b00e92094755f0e2b785dff03fcbd6bce670eddb76d8e77bb0e

app\src\main\java\com\alieducational\app\ui\stock\ImportCatalogScreen.kt | 670 lines | SHA-256
0996b6b1210a372609c3b5a72270f12b365aad4a2f066b12fff37041d4008e27

app\src\main\java\com\alieducational\app\ui\stock\ReceiveStockScreen.kt | 562 lines | SHA-256
ee2407cde1b4cf7ad3578636ce974303f6a6c87e3019c684125a5ba777732169

app\src\main\java\com\alieducational\app\ui\stock\RemoveStockScreen.kt | 423 lines | SHA-256
80c1ada23efaf38d0af83fc4a8718cfd7e691ff0f4ad6cd2058c9e18c4da856e

app\src\main\java\com\alieducational\app\ui\stock\StockScreen.kt | 998 lines | SHA-256
ccd12b439d38e6e5ea75a8869c6fb52f726fe829036fc2f87235c8b0ed0c7bad

app\src\main\java\com\alieducational\app\ui\theme\Color.kt | 29 lines | SHA-256
44aae02f986b563bdd971206f537249d060c59fb5f2cc9caa9806c8a6ae7cd69

app\src\main\java\com\alieducational\app\ui\theme\Theme.kt | 86 lines | SHA-256
b449c34e3038e8a9f016fb51c56e4f957082cb7b583d743d8126134dad281348

app\src\main\java\com\alieducational\app\ui\theme\Type.kt | 103 lines | SHA-256
9da4dbad299064762dfc67bace8fb9ac272c4d49f381cb5c80225adde5510844

app\src\main\java\com\alieducational\app\util\AttachmentManager.kt | 167 lines | SHA-256
772e03e7137a096e2a537390d82c0897f16f35b2412d9e77f597285f8b10ac94

app\src\main\java\com\alieducational\app\util\BiometricAuthManager.kt | 62 lines | SHA-256
78891462ce418adc3d786139df813b444ba8d2bb4a008b5610fd6ccfb0fc1abc

app\src\main\java\com\alieducational\app\widget\AliSlimBarWidgetProvider.kt | 124 lines | SHA-256
197947403e926b26dd0e6ea1999820201bbebf37fb7ff88b6070a2f281e49682

app\src\main\res\drawable\bg_widget_action_item.xml | 15 lines | SHA-256
860dc1492afaa6db49f5f7b7c645b040e62323f8ec09c4e6b86bd851dec73677

app\src\main\res\drawable\bg_widget_slim_bar.xml | 7 lines | SHA-256
fb56ec50fef31b01c5f96f869c5d2bccd8c10087063d3f0b96a117b3aa2055d3

app\src\main\res\drawable\ic_pigmy_coin.xml | 24 lines | SHA-256
aab6ceae54b46e5a220909afa1364db77fab92ef265b1612b312eebef4a4915f

app\src\main\res\drawable\ic_widget_add_stock.xml | 26 lines | SHA-256
57c1cfa482e3c4d2f9ca8c548f17d156bed6a06d1dd5b8872652489628803766

app\src\main\res\drawable\ic_widget_cash_in.xml | 14 lines | SHA-256
cc2d73e31c66895e76c4329d9155f634446707d1db36b7b6c4789a6e4040caf9

app\src\main\res\drawable\ic_widget_cash_out.xml | 14 lines | SHA-256
f65e7c67592db5df28625b625b0bca198372bebeeb156b9de0fa60161a97e05c

app\src\main\res\drawable\ic_widget_new_bill.xml | 22 lines | SHA-256
4f8452d127fe1d649039f80ffba6d815b015fb1e3473b91d7789a00b9d11d151

app\src\main\res\layout\widget_slim_bar.xml | 215 lines | SHA-256
2f96f983f8c88556146e7063be26a1bceb1cdf3236755393018a2786e4e3d8df

app\src\main\res\mipmap-anydpi-v26\ic_launcher.xml | 6 lines | SHA-256
678da6560d837e905ba0c82a15a11918714729de283138721e85ccc1eb1ee9ff

app\src\main\res\mipmap-anydpi-v26\ic_launcher_round.xml | 6 lines | SHA-256
678da6560d837e905ba0c82a15a11918714729de283138721e85ccc1eb1ee9ff

app\src\main\res\values\colors.xml | 4 lines | SHA-256
66ecb6c2fb642327d808de16e65ebe2b91df68d5dad3baf7df8eacfc6fa8a163

app\src\main\res\values\strings.xml | 218 lines | SHA-256
6abfd901de816b7414aea611fd713e6af7caa505070e7790f15b99a442ed3cd8

app\src\main\res\values\themes.xml | 7 lines | SHA-256
2563d183d01f329828936e20b7219d549e309551c89bba617b3e58343046e854

app\src\main\res\values-ur\strings.xml | 210 lines | SHA-256
ac9639cdbd7ebe902f23709b73a354fa139cade144f5be43cc73a2a96c9b2585

app\src\main\res\xml\ali_slim_widget_info.xml | 14 lines | SHA-256
ae5c1bb9628c8ac062f1c36c13ad4098fad88d562b86d25d489a94c7977753b4

app\src\main\res\xml\data_extraction_rules.xml | 15 lines | SHA-256
1ba9df670bd48d0fed4eda3cea25831a5058641be0f3a6ba2fd8d9490a2e64cb

app\src\main\res\xml\file_paths.xml | 7 lines | SHA-256
7d7a15aa5b337ee19e9a7f75ccc243312e42c8ff6b9cd99e793ad9ee5f762d21

app\src\test\java\com\alieducational\app\data\MigrationSqlTest.kt | 51 lines | SHA-256
25ea5407ef155569d5871cafeaee3ea4594468f8482f4dd6736924ba8661ea1f

app\src\test\java\com\alieducational\app\data\service\BackupPackageAndReconciliationTest.kt | 300 lines | SHA-256
bd60d613449f0eadb06c27c5365954f75c30b1c4ab4528cf9f0108867bff1abc

app\src\test\java\com\alieducational\app\domain\calculation\CostCalculatorTest.kt | 160 lines | SHA-256
2a2d9cfa3d9281c40fc5cc9bfb67d5cbc44ffc877ee91e8753f07705c48c2d43

app\src\test\java\com\alieducational\app\service\CashTrackerExcelServiceTest.kt | 119 lines | SHA-256
033a69a1b31c0c6dc021859a0bc3a0b2c0eff6b1930547fe1a03ae21713c7ad5

app\src\test\java\com\alieducational\app\service\FullSyntheticFixtureLifecycleTest.kt | 276 lines | SHA-256
9d68a850f1321c02ea272363e56041cd11706f0988b19a2b5024f9312e2f224b

app\src\test\java\com\alieducational\app\service\GoldenScenarioAndSection25Test.kt | 254 lines | SHA-256
ee0c2a8f68d3b47572043ac10c195d0e6d7df4921331a4ef116020c212414ffe

app\src\test\java\com\alieducational\app\service\ImportExportRobustnessTest.kt | 140 lines | SHA-256
4ef5ebb794733db08a8edfe12aa5f650d39f45222a93aa7f226a615c020f88e9

app\src\test\java\com\alieducational\app\service\NewFeaturesReconciliationTest.kt | 118 lines | SHA-256
9bda22cf4b142dbfc085b778ffd6db458850cebf42254bbb64388afe445a8c9b

app\src\test\java\com\alieducational\app\service\PigmySchemesAndReceiptComprehensiveTest.kt | 292 lines | SHA-256
881ad1abacc134c0735810c8c43631ffaec2f37678e4f774b97a62b10c84549e

app\src\test\java\com\alieducational\app\service\PigmyWithdrawalsAndCategoriesTest.kt | 410 lines | SHA-256
446b818373de5ed0a02c12f5d6205b7fa9f3db7826f7d2e31bcb411e31c6c829

app\src\test\java\com\alieducational\app\service\ReceiptServiceTest.kt | 157 lines | SHA-256
d5e54e5b8af51824ba5811ccb14fdcad83da2f13c1b62a2ce7c182baa4757fb3

app\src\test\java\com\alieducational\app\widget\AliSlimBarWidgetTest.kt | 64 lines | SHA-256
f70200bfd85e6d9ba7943730f408742d6448811135ab5a2b3832cd85f35a4271

app\build.gradle.kts | 158 lines | SHA-256 4f53acac103479f003bdfdb6cfd5d3edf03e97811ccd3b3038332c3081c2bc9d

build.gradle.kts | 9 lines | SHA-256 16b77743ad539774669c5e4c2a56d7ae879fc2da3a91905fcb47f50a5b177c28

settings.gradle.kts | 18 lines | SHA-256 c862dba813dcad0e3543761ff4e9a6d9757676463368dc613206ec2bc3f9724b

gradle\libs.versions.toml | 100 lines | SHA-256 3d1aeaa7af6d9569d971a3a80ae1201be61056d10e3731ca884267fd93507e87

app\proguard-rules.pro | 56 lines | SHA-256 14cf8dd49fff02df644d4b40a36b8292954ca7e3ac9fc1f8459892f634902716

Part III - Existing build and test evidence

app\build\outputs\apk\debug\app-debug.apk | 32,333,778 bytes | SHA-256
b24ec6e756b2a41f60737c7d24da298e1c3976a94b4cb8c18918367c8874442c

app\build\outputs\apk\release\app-release.apk | 10,912,366 bytes | SHA-256
e966fbca2ba6e298fcf7066f569c4f65a2ce45292f7aefecf472bacef41a35ca

APK presence and hash establish artifact identity only. This report does not establish that either APK matches every current source file or is installed on the shop phone.

app\build\test-results\testDebugUnitTest\TEST-com.alieducational.app.data.MigrationSqlTest.xml: tests=1, failures=0, errors=0, skipped=0, recorded timestamp=2026-11-01T17:24:55

app\build\test-results\testDebugUnitTest\TEST-com.alieducational.app.data.service.BackupPackageAndReconciliationTest.xml: tests=3, failures=0, errors=0, skipped=0, recorded timestamp=2026-11-01T17:24:55

app\build\test-results\testDebugUnitTest\TEST-com.alieducational.app.domain.calculation.CostCalculatorTest.xml: tests=9, failures=0, errors=0, skipped=0, recorded timestamp=2026-11-01T17:24:56

app\build\test-results\testDebugUnitTest\TEST-com.alieducational.app.service.CashTrackerExcelServiceTest.xml: tests=2, failures=0, errors=0, skipped=0, recorded timestamp=2026-11-01T17:24:56

app\build\test-results\testDebugUnitTest\TEST-com.alieducational.app.service.FullSyntheticFixtureLifecycleTest.xml: tests=4, failures=0, errors=0, skipped=0, recorded timestamp=2026-11-01T17:24:57

app\build\test-results\testDebugUnitTest\TEST-com.alieducational.app.service.GoldenScenarioAndSection25Test.xml: tests=3, failures=0, errors=0, skipped=0, recorded timestamp=2026-11-01T17:24:57

app\build\test-results\testDebugUnitTest\TEST-com.alieducational.app.service.ImportExportRobustnessTest.xml: tests=5, failures=0, errors=0, skipped=0, recorded timestamp=2026-11-01T17:24:57

app\build\test-results\testDebugUnitTest\TEST-com.alieducational.app.service.NewFeaturesReconciliationTest.xml: tests=4, failures=0, errors=0, skipped=0, recorded timestamp=2026-11-01T17:24:57

app\build\test-results\testDebugUnitTest\TEST-com.alieducational.app.service.PigmySchemesAndReceiptComprehensiveTest.xml: tests=3, failures=0, errors=0, skipped=0, recorded timestamp=2026-11-01T17:24:57

app\build\test-results\testDebugUnitTest\TEST-com.alieducational.app.service.PigmyWithdrawalsAndCategoriesTest.xml: tests=8, failures=0, errors=0, skipped=0, recorded timestamp=2026-11-01T17:24:57

app\build\test-results\testDebugUnitTest\TEST-com.alieducational.app.service.ReceiptServiceTest.xml: tests=3, failures=0, errors=0, skipped=0, recorded timestamp=2026-11-01T17:24:57

app\build\test-results\testDebugUnitTest\TEST-com.alieducational.app.widget.AliSlimBarWidgetTest.xml: tests=3, failures=0, errors=0, skipped=0, recorded timestamp=2026-11-01T17:24:57

app\build\test-results\testReleaseUnitTest\TEST-com.alieducational.app.data.MigrationSqlTest.xml: tests=1, failures=0, errors=0, skipped=0, recorded timestamp=2026-11-01T14:58:24

app\build\test-results\testReleaseUnitTest\TEST-com.alieducational.app.domain.calculation.CostCalculatorTest.xml: tests=9, failures=0, errors=0, skipped=0, recorded timestamp=2026-11-01T14:58:24

app\build\test-results\testReleaseUnitTest\TEST-com.alieducational.app.service.CashTrackerExcelServiceTest.xml: tests=2, failures=0, errors=0, skipped=0, recorded timestamp=2026-11-01T14:58:24

app\build\test-results\testReleaseUnitTest\TEST-com.alieducational.app.service.FullSyntheticFixtureLifecycleTest.xml: tests=4, failures=0, errors=0, skipped=0, recorded timestamp=2026-11-01T14:58:25

app\build\test-results\testReleaseUnitTest\TEST-com.alieducational.app.service.GoldenScenarioAndSection25Test.xml: tests=3, failures=0, errors=0, skipped=0, recorded timestamp=2026-11-01T14:58:25

app\build\test-results\testReleaseUnitTest\TEST-com.alieducational.app.service.ImportExportRobustnessTest.xml: tests=5, failures=0, errors=0, skipped=0, recorded timestamp=2026-11-01T14:58:25

app\build\test-results\testReleaseUnitTest\TEST-com.alieducational.app.service.NewFeaturesReconciliationTest.xml: tests=4, failures=0, errors=0, skipped=0, recorded timestamp=2026-11-01T14:58:25

app\build\test-results\testReleaseUnitTest\TEST-com.alieducational.app.service.PigmySchemesAndReceiptComprehensiveTest.xml: tests=3, failures=0, errors=0, skipped=0, recorded timestamp=2026-11-01T14:58:25

app\build\test-results\testReleaseUnitTest\TEST-com.alieducational.app.service.PigmyWithdrawalsAndCategoriesTest.xml: tests=8, failures=0, errors=0, skipped=0, recorded timestamp=2026-11-01T14:58:25

app\build\test-results\testReleaseUnitTest\TEST-com.alieducational.app.service.ReceiptServiceTest.xml: tests=3, failures=0, errors=0, skipped=0, recorded timestamp=2026-11-01T14:58:25

app\build\test-results\testReleaseUnitTest\TEST-com.alieducational.app.widget.AliSlimBarWidgetTest.xml: tests=3, failures=0, errors=0, skipped=0, recorded timestamp=2026-11-01T14:58:25

Stored reports may be stale or from a different build. Unit tests do not prove actual phone rendering, Files/Drive behavior, printing, restore completeness, biometric behavior or two-device lifecycle. Test source is included below for examining what was actually asserted.

Known evidence limitations and release checks

No production device was connected or updated during this report. No claim is made that all approved requirements have been implemented correctly.

The earlier supplied backup had a database, manifest and one photo; ZIP integrity passed, but database integrity/complete restoration was not established.

The user reported broken Home Bill History navigation, weak light-mode contrast, Accounts placement problems and widget mismatch on 1 November. Current source may include fixes; fresh device evidence is needed to close those reports.

Before release, reconcile the approved synthetic golden scenario, restore an externally saved backup on an isolated installation, verify signing/migration compatibility, and compare exact candidate APK identity.

Part IV - Detailed approved specifications

The following handoffs preserve detailed decisions, workflows, examples and acceptance tests. Their imperative language describes requested work. It does not establish implementation or test completion. Later explicit decisions override earlier proposals.

ANTIGRAVITY_LAYOUT_GRAPHS_SUPPLIERS_AND_SHOP_EXPENSES.md

Ali Educational: layout fixes, graphs, suppliers and shared Shop Expenses

Date: 1 November 2026

1. Purpose and status

This is a detailed implementation handoff for the existing Android app. It specifies requested changes and acceptance tests; it does not claim that code has been changed, an APK rebuilt, or tests passed.

Apply changes to the current app without resetting existing records or undoing completed features. The current source location discovered during review is `C:/Users/Musid/.gemini/antigravity/scratch/AliEducational`. Verify its current branch/build before editing. The older `android` folder in the document workspace is not the current app. Do not replace the current project with an earlier review copy.

Inputs: the supplied Pigmy Savings, Sales & Profit, Accounts and Add Stock & Product screenshots, the earlier consolidated feature review, and the user's latest answers. Screenshots establish visible layout problems, not complete behavior or database correctness.

Explicitly confirmed expense decisions

1. A Cash Out recorded as Shop Expense appears in Shop Expenses automatically, using one shared expense record. It must never be counted twice.

2. Only an explicitly linked expense refund recorded as Cash In reduces expenses. Ordinary Cash In does not reduce expenses.

3. Service costs, including paper, ink and printer repairs, belong in the same expense system with Products / Services / Shared classification.

The layout, graph and supplier details below specify the recommended implementation requested for this handoff. Do not invent historical amounts, supplier matches or expense classifications when migrating uncertain data.

2. Scope and priorities

Deliver:

- Responsive placement fixes for Pigmy, Sales & Profit, Accounts and stock entry.

- Readable profit summaries with consistent time filters and useful graphs.

- One supplier directory shared by Accounts and stock receiving.

- One expense system shared by Cash In/Out, Shop Expenses and service reporting.

- Linked expense refunds, synchronized edits/cancellations, attachments and accurate totals.

- Nondestructive migration, backup coverage and evidence from isolated tests.

Preserve English interface labels, Urdu/mixed-script product names, offline use, Owner/Billing restrictions, existing receipt changes, expandable Pigmy schemes and interest-free withdrawals. Do not add cloud services or automatic uploads for these changes.

Prioritize financial consistency and data preservation before cosmetic refinements. Do not change stored amounts merely to make dashboard values look better.

3. Common layout rules

- Preserve the charcoal/gold visual identity and optional light theme.

- Use consistent screen padding, card spacing and aligned text baselines; approximately 16 dp screen padding and 12-16 dp gaps are starting values, not rigid constraints.

- Respect status/navigation bars and keyboard insets. Remove redundant top spacers rather than removing required safe-area padding.

- Keep primary controls at least 48 dp in touch size.

- Never split a button word such as Withdraw across lines. Reflow controls or stack them instead of shrinking text to an unreadable size.

- Do not truncate important monetary amounts. Use readable Indian rupee formatting with grouping and two decimal places, including negative amounts.

- Use content-driven card heights. Long names and larger fonts must not collide with icons, totals or adjacent cards.

- At narrow widths or larger font sizes, convert two-column fields/cards into a single column.

- Keep scrolling content clear of fixed bottom actions and navigation. Keyboard opening must not hide the focused field or make Save unreachable.

- Provide accessible names for icons and distinguish selected, disabled and error states using text/shape as well as color.

4. Pigmy Savings placement fixes

Observed problems

- Withdraw wraps as “Withdra” and “w”.

- Deposit, Withdraw and an unexplained history count compete for one row.

- Several management icons crowd each scheme heading.

- Summary and scheme metadata need more consistent alignment.

Screen structure

1. Compact app bar with Back, Pigmy Savings and Add scheme; move Add scheme below the title if needed at larger text sizes.

2. Summary card showing Tracked savings balance as the main value.

3. Deposits, Withdrawals and Today's deposits as clearly labelled secondary metrics. Stack/reflow rather than squeeze.

4. Pigmy schemes heading and Active / All filter.

5. Vertically scrollable scheme cards.

Scheme card structure

```text

Scheme name More

Tracked balance

₹45.00

Deposits ₹115.00

Withdrawals ₹70.00

Usual deposit ₹100.00

[ + Deposit ] [ - Withdraw ]
```

[View history (4) v]

```

- Deposit and Withdraw have equal visual emphasis, width and height; stack at accessibility sizes if necessary.

- View history occupies its own row and explains the displayed count. Count the history entries actually available under the stated filter; do not show an unexplained number.

- Put Edit scheme, confirmed opening balance management and Archive/reactivate in a labelled overflow menu as applicable to the existing implementation. Preserve all existing permitted actions.

- Long scheme names wrap without pushing actions outside the card.

- Disabled withdrawal remains readable; explain insufficient tracked savings when relevant. Do not weaken withdrawal validation.

- Keep balance, deposit and withdrawal semantics unchanged: savings transfers do not affect sales or operating profit.

# 5. Sales & Profit layout and meanings

## Date controls

Replace the clipped chip strip with a compact period selector supporting Today, This Week, This Month, All Time and Custom dates. Show the actual selected date range below it. Preserve the app's documented week/timezone convention and apply it consistently.

Use the same range for summary cards, charts, detail lists and exports. Current stock projections must explicitly say “Current stock” because they are not historical period sales.

## Recommended Products tab order

1. Products / Services tabs.

2. Period selector and visible date range.

3. Prominent Gross profit card with a secondary, explicitly labelled gross margin.

4. Product sales and Cost of goods sold cards.

5. Recorded product expenses, linked refunds and result after product expenses.

6. Trend graph.

7. Expense breakdown graph.

8. Expected gross profit from current stock card, separated from realized results.

Use a full-width amount below the title instead of forcing title, amount and percentage onto one row. Render a loss with a clear minus sign and label, not only red color.

## Screenshot reconciliation

| Displayed component | Screenshot value |

|---|---:|

| Product sales | ₹145.78 |

| Cost of goods sold | ₹108.50 |

| Gross profit | ₹37.28 |

| Displayed expenses | ₹12,235.00 |

| Gross profit minus displayed expenses | -₹12,197.72 |

The arithmetic is consistent. The screenshot does not establish whether all ₹12,235 should be attributed to products. Inspect the contributing entries before changing the label or classification. Never silently remove legitimate expenses to improve the displayed result.
```

Calculation rules

- Net product sales = product revenue after allocated discounts and linked returns/reversals.

- Product gross profit = net product sales minus the associated historical product cost snapshots.

- Gross margin = gross profit divided by net product sales, multiplied by 100. If sales are zero or negative, show Not applicable rather than an infinite or misleading percentage.

- Net recorded product expenses = active Products-classified expenses minus their active linked refunds under the selected effective-date filter.

- Result after product expenses = product gross profit minus net recorded product expenses.

- Label this result according to what it actually includes. Show that Shared expenses are excluded; do not call a partial calculation complete shop net profit.

- Shared expenses remain separately visible and are counted once in any combined overview. Do not arbitrarily allocate them to Products or Services.

- Services shows net service revenue, recorded service expenses, linked service-expense refunds and revenue minus net recorded service expenses. This is not a claim of exact service profit.

- Missing sale costs must produce an incomplete/known-cost warning. Never treat missing costs as zero to claim exact profit.

- Customer repayments are collections, not additional sales. Supplier settlements and Pigmy transfers are not ordinary operating expenses.

- Current stock potential uses current sellable stock and known costs/prices; disclose missing costs and problematic negative stock.

6. Graph requirements

A. Sales and gross profit over time

- Place below the summary, using actual persisted transactions.

- Show separate labelled series for net product sales and product gross profit, with currency axes and a visible zero baseline.

- Use daily buckets for short ranges and weekly/monthly buckets for longer ranges. State the grouping. For Today, a single daily point is acceptable; do not fabricate hourly detail.

- Include all buckets in the range, including verified zero-activity days. Distinguish missing/incomplete data from zero.

- Negative profit must extend below zero. Do not clip losses.

- Tapping a bucket shows period, sales, COGS and gross profit, with access to contributing records when feasible.

- Sum of displayed buckets must reconcile to the selected-range cards, using integer paise or equivalent exact monetary calculation.

B. Expense breakdown

- Use readable horizontal bars by expense category.

- State whether the view shows Products, Services, Shared or All. On the Products tab default to Products and disclose that scope.

- Display expense outflows and linked refunds in the detail; the headline category amount is net recorded expense.

- Support negative net values for periods with refunds of earlier expenses. Do not force such values to zero.

- Tap a category to open the same filtered expense list used by Shop Expenses.

- Avoid pie charts for signed expense/refund data.

General behavior

- No decorative sample data in production. Empty periods show “No transactions in this period”.

- Use readable legend labels, accessible contrast and a text summary/table alternative.

- Graphs work offline and respect Owner access. Do not send shop data to an external chart service.

- Large histories must not freeze navigation; aggregate efficiently using the same financial rules as cards.

- Edits, refunds, cancellations and date changes refresh both graphs and totals consistently.

7. One shared supplier directory

User experience

Keep Add supplier in Accounts. Replace free-text Supplier Name in stock receiving with a full-width searchable selector:

```text
Supplier
[ Search or select supplier v ]
+ Add new supplier
```

- Both entry points read/write the same supplier directory with stable supplier IDs.

- Add new supplier opens the same creation form and returns with the new supplier selected.

- Preserve all entered product, quantity, price, invoice and attachment values while creating the supplier.

- Require a supplier name; keep optional contact details optional unless an existing requirement says otherwise.

- Normalize case and whitespace for duplicate warnings. Do not automatically merge two businesses solely because their names match.

- Renaming a supplier preserves all transaction links. Archive instead of deleting referenced suppliers.

- A supplier with a zero balance remains visible under All suppliers.

- A product can be received from different suppliers over time. Store the actual supplier on each receiving/purchase transaction. An optional preferred supplier is only a convenience, not a rewrite of history.

Accounts placement

- Use clear Suppliers / Customers terminology, with Amount we owe and Amount customers owe as summary labels.

- Keep the amount summary separate from the Add supplier button; use the app's primary gold treatment rather than implying that supplier creation itself is a loss.

- Add search and All / Outstanding filters.

- Place an explanatory empty state beneath the controls. Distinguish “No suppliers yet” from “No outstanding supplier balances”.

- Supplier detail shows purchases, payments, adjustments/reversals and outstanding amount with effective dates and links to source records.

Purchases and dues

Creating or selecting a supplier does not create a liability. Recording receipt of purchased stock captures invoice amount, paid amount, payment source and unpaid amount using the existing purchase model.

Example: receive ₹1,000 of stock, pay ₹600, and show ₹400 due. A fully paid ₹1,000 purchase appears in history with ₹0 due. Paying the remaining ₹400 later settles the liability; it must not create another inventory purchase or operating expense.

- Credit purchases require an identified supplier.

- Preserve existing workflows for opening inventory and product-only edits; do not invent a new purchase or debt for them.

- Adding stock, editing product details and correcting counted stock remain separate operations.

- Transport may belong to landed inventory cost, but its payee can differ from the supplier. Do not automatically add every transport charge to supplier dues or also count capitalized transport as an operating expense.

- Supplier invoice/payment save operations must be atomic and safe against repeated taps.

Existing supplier-name migration

Inventory may already contain supplier names with no directory link. Produce a reviewable migration mapping. Automatically reuse a link only where identity is unambiguous under verified existing data rules. Keep ambiguous/unmatched names visible for Owner review. Preserve old text for provenance. Creating directory entries from old names must not invent opening balances, purchases or payments.

8. Add Stock & Product form layout

Use a full-screen form or a sufficiently spacious adaptive sheet with a scrollable body and keyboard-safe actions.

Recommended sections:

1. Product details: name, optional barcode/scanner, category, brand.

2. Stock: received quantity/unit and low-stock threshold.

3. Pricing: cost per piece, selling price per piece, optional wholesale price and clearly defined margin.

4. Purchase details: supplier selector, invoice number, effective date, payment/dues information appropriate to the existing flow.

5. Transport and cost summary.

6. Save and Cancel.

- Supplier, product name and long labels occupy full-width rows.

- Keep currency/unit labels above values rather than wrapping placeholders such as “Wholesale (₹/pc)” inside a narrow box.

- Verify whether the existing Margin field means gross margin or markup; label and calculate consistently. Do not change persisted prices by assuming the two are equivalent.

- Preserve existing weighted-average costing, exact transport allocation and historical cost snapshots.

- Show validation beside the relevant field and retain values after validation failures.

9. Cash In/Out and Shop Expenses: one shared system

Core rule

A Shop Expense entered through Cash Out is the same economic expense shown in Shop Expenses and financial reports. Do not create one expense for each screen.

The database may use a canonical expense plus a linked cash/bank movement if needed. That is acceptable only with stable links, atomic writes and one reporting source of truth. Two related records must not become two independent expenses.

Cash Out form

Keep transaction purpose separate from category:

```text

Transaction purpose: Shop expense

Category: Electricity

Applies to: Shared

Amount: ₹500.00
```

Paid from: Shop cash

Effective date and time: ...

Notes: ...

Optional photo: ...

```

- Shop expense purpose explicitly opts the transaction into expense reporting.

- Categories describe the expense, such as Rent, Electricity, Paper or Repairs. Category text alone does not determine accounting treatment.

- Applies to must be Products, Services or Shared. Do not silently assign all existing or new expenses to Products.

- For Services, retain an optional specific-service target where supported; broader service costs remain distinguishable from expenses shared across the whole shop.

- Preserve the existing ability to add a category mid-form without losing input.

- Record actual funding source: cash-funded entries reduce physical shop cash; bank-funded entries affect bank records, not physical cash.

- Saving creates one expense and its one applicable payment movement, then refreshes Shop Expenses automatically.

## Shop Expenses entry point

- The existing Shop Expenses shortcut opens the shared list.

- Add expense from this screen uses the same validation and save operation as Shop Expense in Cash Out.

- A paid expense created here appears in the relevant Cash In/Out view with its payment source identified.

- If the current app supports unpaid/accrued expense records, preserve and label that behavior; do not fabricate a cash payment for an unpaid expense. These new cash-entry flows represent actual payments.

## Shop Expenses screen

Display:

1. Period selector and actual date range.

2. Gross recorded expenses.

3. Linked expense refunds received in the period.

4. Net recorded expenses = expenses minus refunds.

5. Products / Services / Shared breakdown.

6. Category breakdown graph and transaction list.

Filters: date, category, classification, payment source and active/cancelled status as appropriate. Default financial totals exclude cancelled entries; history retains them visibly.

Each list row shows amount, expense/refund direction, category, classification, effective date, source/destination, note preview and photo indicator. Detail opens the canonical record and related refund/payment history.

All-filter totals must equal the sum of Products, Services and Shared, plus an explicitly shown Unclassified legacy bucket if unresolved old records exist. Never hide uncategorized or unclassified history.

## 10. Cash In expense refunds

## Explicit linked flow

Cash In gains or uses an explicit Expense refund purpose. The user selects the original expense, then enters refund amount, receipt destination, effective date/time, optional note/reference and optional photo.
```

- Offer Record refund from expense detail, preselecting the original expense.

- Link by stable expense ID, not matching descriptions.

- Show original amount, prior active refunds and remaining refundable amount.

- Permit partial and multiple refunds up to the remaining refundable amount. Reject zero/negative amounts and excess refunds.

- The refund inherits the original expense's category and classification for reporting, including its specific service target if present.

- A refund received into cash increases physical cash once; one received into a bank does not increase physical cash.

- A refund reduces recorded expenses, not product sales or service revenue.

- Ordinary sales collections, customer repayments, savings withdrawals, owner contributions and unrelated Cash In never reduce Shop Expenses.

Date behavior

Use the refund's actual effective receipt date. Do not silently rewrite the original expense date or restate an earlier period.

Example: an October ₹1,000 expense refunded by ₹200 in November produces October expense ₹1,000, November expense refund ₹200, and combined net expense ₹800. November net expense may be negative if no other expenses occur; display it honestly and explain the earlier-period refund.

Refund versus cancellation

- Refund means a real payment was received back and creates an incoming movement.

- Cancellation corrects an erroneous entry and reverses that entry's recorded effects with audit history. It must not pretend that a real refund occurred.

- Prevent cancellation or amount reduction of an expense while active refunds would become invalid. Explain the linked records and require their appropriate correction first. Do not silently delete refund history.

- Changing the original expense's classification updates the reporting attribution of its linked refunds consistently, without creating another money movement. Retain audit history.

11. Editing, consistency and attachments

- Opening the same expense from Cash In/Out or Shop Expenses must show the same identity and current values.

- Editing amount updates the existing payment effect by the correct difference; it does not add another full payment.

- Changing payment source reverses the prior source effect and applies the new one atomically.

- Changing effective date moves the same entry between report periods without duplication.

- Editing category, classification, notes or photo alone does not change cash.

- Cancellations reverse committed effects once and remain visible in audit history.

- Repeated taps, process restarts, retry and applicable sync must preserve exactly-once financial effects.

- Photos use durable managed storage. Both screens reference the same attachment; avoid independent copies that can diverge.

- Photo replacement failure keeps the previous valid photo and entered form values. Do not silently save without requested evidence.

- Backups include actual photo bytes, IDs, expense/refund links, payment links, category/classification and effective/audit timestamps.

- Owner financial data and evidence must remain inaccessible to restricted Billing roles, including through sync, exports and deep links.

12. Expenses that must not be double-counted

| Event | Shop Expenses treatment |

|---|---|

| Rent/electricity paid as Shop expense | Include once in chosen classification |

| Service paper/ink paid as Shop expense | Include once under Services |

| Explicit refund linked to an expense | Reduce that expense classification on refund date |

| Pigmy deposit or withdrawal | Savings transfer; exclude |

| Customer repayment | Debt collection; exclude |

| Supplier payment settling stock purchase | Liability settlement; exclude from operating expenses |

| Stock purchase and capitalized delivery | Inventory cost; do not also post as operating expense |

| Product sale return/refund | Sales reversal; not an expense refund |

| Transfer between cash and bank | Transfer; exclude |

| Photo/category edit | No financial effect |

Do not add together an old service-expense table and its new canonical replacement. Define one query/source of truth for each report. If legacy migration is uncertain, show review status instead of deleting or guessing.

13. Migration, imports, exports and recovery

- Inspect the existing schema and transaction paths before choosing implementation details. Stable IDs and links are requirements; a particular schema layout is not mandated.

- Back up before migration and test upgrades from supported old schemas in isolation.

- Reconcile existing Shop Expenses, service expenses and cash entries. Merge/link only demonstrably identical economic records; amount/date/category similarity alone is insufficient evidence.

- Preserve original IDs where possible, or retain an explicit stable mapping with provenance.

- Preserve supplier balances, cash totals, stock quantities and historical costs unless a separately reviewed correction is required.

- Do not infer expense type from arbitrary words in historical Notes. Preserve unresolved legacy classification visibly.

- Existing supported spreadsheet import flows should preview purpose/classification/refund links where available. Older files lacking these fields remain reviewable; do not manufacture links.

- Export selected filters only, including empty results. Include category, classification, purpose, original expense/refund ID where appropriate and attachment presence.

- A spreadsheet is not a complete photo backup. Do not export inaccessible local paths as if they were portable evidence.

- Restore a complete test backup into a separate installation and verify photos, IDs, links, supplier dues and expense totals. Restoring records must not replay their cash effects.

14. Acceptance tests

Use synthetic data in a disposable app/database. Record expected and actual results, not only a ticked checklist.

Layout and graphs

| ID | Scenario | Expected |

|---|---|---|

| L01 | Pigmy at narrow phone width | Deposit/Withdraw readable; history on separate row |

| L02 | Large font and long scheme name | No clipped actions, overlapping amounts or split button words |

| L03 | Stock form with keyboard open | Focused field and Save accessible; input retained |

| L04 | Large currency values | Full amounts readable with grouping; no collision |

| L05 | Report period selection | No clipped label; range visible; cards/graphs/list agree |

| L06 | Zero sales / no activity | Honest empty state; no fabricated chart or undefined margin |

| L07 | Loss and refund-heavy period | Negative values shown below zero and in summaries |

| L08 | Graph bucket totals | Exactly reconcile to underlying selected-range totals |

| L09 | Missing cost or pending sync | Incomplete result disclosed, never shown as verified complete |

| L10 | Light/dark themes and accessibility | Readable labels, contrast, focus order and touch targets |

Suppliers

| ID | Scenario | Expected |

|---|---|---|

| S01 | Add supplier in Accounts | Selectable immediately in stock receiving |

| S02 | Add supplier during stock entry | Appears in Accounts; entered stock form values survive |

| S03 | Receive ₹1,000, pay ₹600 | Stock added once; supplier due ₹400; paid source reduced ₹600 |

| S04 | Pay remaining ₹400 | Due zero; no extra stock purchase or operating expense |

| S05 | Fully paid purchase | Supplier retained in All; excluded from Outstanding |

| S06 | Rename/archive supplier | History and links intact; outstanding balances remain accessible |

| S07 | Similar legacy supplier names | Ambiguous identities reviewed, not silently merged |

| S08 | Edit product name only | No new stock, debt or cash movement |

| S09 | Same product from another supplier | Each receipt retains its actual supplier |

| S10 | Repeated save/restart | One purchase, one payment and correct remaining liability |

Shared expenses and refunds

For E01-E06 start from an isolated cash balance of ₹5,000 with no expenses/refunds. Keep all events in one test period unless stated otherwise.

| ID | Action | Expected |

|---|---|---|

| E01 | Cash Out ₹500 electricity, Shared | Cash ₹4,500; one expense; gross/net expenses ₹500 |

| E02 | Add ₹200 paper, Services through Shop Expenses | Cash ₹4,300; gross/net expenses ₹700; service expenses ₹200 |

| E03 | Cash In ₹100 refund linked to electricity | Cash ₹4,400; gross ₹700; refunds ₹100; net ₹600; Shared net ₹400 |

| E04 | Ordinary Cash In ₹300 | Cash ₹4,700; net expenses remain ₹600 |

| E05 | Edit paper expense ₹200 to ₹250 | Cash ₹4,650; gross ₹750; refunds ₹100; net ₹650 |

| E06 | Cancel paper expense with no linked refund | Cash ₹4,900; gross ₹500; refunds ₹100; net ₹400; history retained |

| E07 | Bank-funded expense | Bank effect once; physical cash unchanged; expense included once |

| E08 | Bank-received linked expense refund | Bank increases once; cash unchanged; expense reduced once |

| E09 | Multiple partial refunds | Cumulative amount validated against original expense |

| E10 | Refund above remaining amount | Rejected without financial writes |

| E11 | Cancel/edit expense below active refunded amount | Clear linked-record validation; no orphan refund or hidden deletion |

| E12 | Refund of prior-period expense | Original period unchanged; refund period shows reduction; combined net reconciles |

| E13 | Change expense date/category/classification | Same identity; filters refresh; no duplicate cash effect |

| E14 | Replace/remove photo | Financial values unchanged; both screens show same evidence state |

| E15 | Retry/double tap/interrupted save | One committed expense/refund and one applicable movement |

| E16 | Record Pigmy/supplier settlement/customer repayment | Excluded from Shop Expenses |

| E17 | Products/Services/Shared filters | Breakdown sums to All, including disclosed unresolved legacy bucket |

| E18 | Restore backup elsewhere | Same IDs/totals/links; photos open; cash movements not replayed |

| E19 | Restricted Billing access | Owner expenses/photos blocked across UI, data and applicable sync |

| E20 | Export empty selected period | Empty result; never silently exports all records |

| E21 | Migration with suspected duplicate legacy expense | No automatic double count or destructive guessing; review evidence retained |

| E22 | Create category during expense entry | Amount/date/classification/photo preserved; new category selected |

Profit regression

- Product sales ₹1,000 and saved COGS ₹600 give gross profit ₹400.

- Products expenses ₹100 and linked Products refund ₹20 give net product expenses ₹80 and result after product expenses ₹320.

- Shared expense ₹50 stays separately visible; it is not silently subtracted from both Products and Services.

- Service expense appears once in the service view and once as the same record in the All expenses view, never twice in a combined total.

- Editing current catalog cost does not rewrite the ₹600 historical COGS.

- Credit repayments do not add revenue; savings transfers do not change profit.

15. Delivery and release evidence

Provide:

1. Changed-file summary and concise explanation of canonical supplier/expense links.

2. Before/after screenshots for all four affected screens at normal and larger text sizes.

3. Graph screenshots with positive, negative and empty periods and numerical reconciliation.

4. Expense/refund and supplier purchase/payment expected-versus-actual results.

5. Migration and isolated backup/restore evidence, including restored photos.

6. Passed, failed and unverified tests, with device/build details. A successful compile does not prove Android UI or accounting behavior.

7. Candidate APK identity/version/hash and known limitations if a build is produced.

Do not uninstall the live app, clear its storage or use destructive migration. Production installation is a separate action requiring explicit user approval. This document requests implementation preparation and verification, not authorization to overwrite live shop data.

16. Approved addition: Bill History linked to customer Accounts

The user approved a dedicated Bill History screen and explicitly requires it to link to Accounts, where the app shows how many customers owe the shop and the total amount owed. These are shared financial records, not independent balances maintained by each screen.

Navigation and contents

- Add a clearly labelled History action in Billing and a Today's bills shortcut on Home.

- Show completed bill count, net billed amount, payments applied to the selected bills and remaining bill dues. Show cancellations and refunds separately. Held/draft bills do not count as completed sales or customer debt.

- Filter by Today, This Week, This Month, All Time or Custom dates, payment status, products/services and originating device where available.

- Search by receipt number, customer name or phone number. Search results do not change the underlying records.

- Each row shows persisted receipt number, original date/time, customer or Walk-in, amount, payment status and device where applicable.

- Bill detail shows item/service snapshots, quantities, prices, discounts, tendered/applied payment and change where relevant, outstanding amount, later repayments, returns/refunds and cancellation history.

- Reopen, print, save PDF and share the persisted bill without creating another sale, payment or stock movement.

One customer identity and one source of truth

- Link credit bills to the same stable customer ID used by Accounts. Do not match customer debt by display name or phone text alone.

- Use the same customer selector and Add customer flow in Billing and Accounts. Preserve the active bill when adding a customer. Warn about possible duplicates without automatically merging identities.

- A sale with any unpaid amount requires an identified customer. Do not allow unassigned Walk-in debt.

- Accounts lists all customers, with an Outstanding filter. Customers with zero due remain accessible in All customers and retain their history.

- Display Customers who owe as the count of distinct customers with a positive reconciled amount due, not the count of unpaid bills.

- Display Total customers owe as the sum of positive reconciled customer balances. Show customer credits/advances separately; do not let one customer's credit hide another customer's debt.

- Derive both screens from the same canonical bills, payments, allocations and linked adjustments. Do not add cached bill dues to an already aggregated customer balance.

- Preserve reviewed opening balances and any valid legacy non-bill debt separately with provenance. Do not invent historical bills to represent an opening balance.

Cross-navigation

- Customer Accounts detail includes outstanding bills, settled bill history, opening/legacy balance items, repayments and linked adjustments.

- Tapping a bill in Accounts opens that same Bill History detail.

- Tapping the customer or View customer account from Bill History opens that customer's account.

- Accounts summary is the current outstanding position unless explicitly offering a documented as-of view. A Bill History date filter selects bills by their original bill date; it does not silently limit the customer's overall account balance.

- Clearly label Remaining due on selected bills separately from Total customer balance, which may include older bills and opening debt.

Payments and allocations

- Saving a completed credit/part-paid bill atomically records the sale, initial applied payments, customer debt and existing required stock effects once.

- Record a repayment from either Accounts or bill detail using the same payment command and stable payment ID. A repayment updates the cash/bank destination and customer/bill balances once; it is not another sale or new revenue.

- Bill-detail repayment preselects that bill. From Accounts, offer allocation to outstanding bills and separately recorded opening debt, with an explicit preview. An oldest-first suggestion is acceptable, but the user must see the allocation before saving.

- One payment may settle multiple bills. Preserve allocation amounts and require their sum to equal the amount applied; never credit the full payment independently to every bill.

- If overpayment/advances are already supported, keep the excess as an explicit customer credit with clear confirmation. Otherwise reject amounts above the amount being settled; do not silently create negative bill dues.

- Keep payment receipt date separate from original bill date. Payments applied to bills selected by bill date and collections received during a date range are different summaries and must be labelled separately.

- Edits, cancelled payments, returns and bill reversals recalculate linked balances atomically with audit history. Account for later repayments before reversing a bill; any resulting refund/customer credit must use an explicit supported resolution rather than orphaning payments or silently deleting money history.

- Customer rename/archive preserves all links. Outstanding customers remain visible and counted even when archived.

Worked example

1. Customer A has a ₹1,000 bill, pays ₹600 and owes ₹400. Bill History shows that bill due ₹400; Accounts shows one customer owing ₹400.

2. Customer A receives another unpaid bill for ₹200. Accounts still shows one customer owing, now ₹600; Bill History shows two outstanding bills.

3. Customer B has an unpaid bill for ₹300. Accounts shows two customers owing a combined ₹900.

4. Customer A repays ₹500, explicitly allocated ₹400 to the first bill and ₹100 to the second. The first bill is paid, the second has ₹100 due, and Accounts shows A ₹100 plus B ₹300: two customers owing ₹400 total.

5. Customer A pays the remaining ₹100. Accounts shows one customer owing ₹300. No repayment changes completed bill count, original sales revenue or stock.

Permissions, synchronization and recovery

- Owner sees all accepted bills. Billing-device history defaults to its own permitted bills; customer-account access and repayment actions must follow existing role permissions. Do not grant new broad financial access through a history link.

- Offline accepted sales and repayments use stable IDs and durable event handling. Retry must not duplicate debt, collections or bills.

- Disclose pending/unknown synchronization and incomplete Owner totals. Do not claim an offline device has zero pending bills without evidence.

- Backups and restores preserve customer IDs, bill IDs, original receipt numbers, payment IDs, allocations, opening balances, reversals and audit history. Restore must not repost payments or debt.

Acceptance tests

| ID | Scenario | Expected |

|---|---|---|

| H01 | Complete the worked example above | Counts and balances match every step in both screens |

| H02 | One customer has several unpaid bills | Customer counted once; each bill retains correct due |

| H03 | Attempt unpaid Walk-in sale | Customer selection required before committing credit |

| H04 | Repay through Accounts, then open bill | Same repayment/allocations visible; due updated once |

| H05 | Repay through bill detail, then open Accounts | Same payment visible; customer total refreshed once |

| H06 | Change Bill History date filter | Selected-bill totals change; overall customer balance remains correctly labelled |

| H07 | Later-period repayment of earlier bill | Collection appears on payment date; original revenue/date unchanged |

| H08 | Opening balance plus bill debt | Both retained separately; no fabricated bills or double-counted debt |

| H09 | Reprint/share/reopen or retry save/sync | No extra bill, stock deduction, payment or debt |

| H10 | Return/cancel bill with later repayments | Explicit valid refund/credit handling; no orphan allocations |

| H11 | Rename/archive customer | History and outstanding totals retained |

| H12 | Held/cancelled bills | Excluded from active completed-sale/debt totals according to status; audit history retained |

| H13 | Customer credit and another customer's debt | Credit shown separately; positive debt total not understated |

| H14 | Restore isolated backup | Same bills, customer counts, balances and allocations without replay |

| H15 | Restricted device follows account link | Unauthorized data/actions remain blocked |

Include Bill History and linked Accounts screenshots, payment-allocation reconciliation and these test results in the release handoff. This section adds requirements; it does not certify implementation.

17. Latest navigation decision and unified Accounts approach

Bill History placement - supersedes the earlier Home shortcut proposal

The user requests Bill History in the position currently occupied by the Settings shortcut below Cash In/Out. Replace that Home shortcut with Bill History, opening the full register with its period filters. Do not add a second Today's bills shortcut in another Home position as previously proposed. Keep a History action within Billing for convenience.

Retain access to Settings through the gear icon; this moves its Home shortcut, not its functionality. Verify the current Home layout before implementation, since the supplied screenshots do not show that location. Preserve Cash In/Out and bottom navigation. Test both the replacement shortcut and Settings gear, including back navigation and larger fonts.

Accounts is the shared balance view, not another manual ledger

Use the same design on both sides:

| Activity | Canonical transaction | Accounts effect |

|---|---|---|

| Completed customer bill | Sale with customer ID and applied payments | Unpaid portion becomes customer due |

| Customer repayment | Payment with allocations to bills/opening debt | Reduces that customer's due once |

| Receive purchased stock | Purchase/receipt with supplier ID and applied payments | Unpaid portion becomes supplier due |

| Supplier payment | Payment with allocations to purchases/opening debt | Reduces that supplier's due once |

| Add customer or supplier | Directory record | No balance effect |

| Edit product details | Catalog change | No purchase, stock receipt or debt effect |

Accounts has two clear sections: Customers owe us and We owe suppliers. Each shows the number of distinct parties with positive dues and total positive dues, plus All / Outstanding views. Keep customer credits and supplier advances separate from positive amounts owed. Do not offset unrelated parties or customer and supplier balances automatically.

Receiving stock versus editing a product

The existing Add Stock & Product flow may remain convenient, but it must clearly distinguish Create product / Receive stock from Edit product details. Purchase details belong to a receipt/purchase, not just a supplier-name field on the product.

- Select a supplier from the shared Accounts directory, or add one inline.

- Record received items, invoice reference, effective date, purchase amount, paid now, payment source and remaining due.

- Show a review summary before committing: stock received, payment made and supplier due created.

- Save purchase, stock movements, initial payment and supplier liability together, with repeat-save protection.

- Support a supplier invoice containing several product lines as one purchase where the existing workflow permits. Do not record the whole invoice liability separately for each product. Where entry is product-by-product, explicitly link lines to the intended purchase and prevent repeated invoice-level charges/payments. Do not guess invoice identity from invoice number alone across different suppliers.

- Later price/name edits never repost the original purchase or opening stock.

- Do not create supplier debt merely from choosing a supplier or importing a catalog supplier name.

Example: one supplier invoice has notebooks ₹600 and pens ₹400. Pay ₹700. Accounts must show one ₹1,000 purchase with ₹300 remaining due, not two ₹1,000 purchases or a second debt entered manually in Accounts. A later ₹300 settlement clears the due without adding stock or operating expense.

One financial event, consistent views

Bills and purchases create the obligations. Payments and linked returns/reversals adjust them. Accounts, Bill History, stock-receipt history and Cash In/Out display the relevant views of those same transactions. Avoid independent editable totals on each screen; any performance caches must remain reconcilable to the canonical records.

For linked customer repayments and supplier settlements, opening an entry from Cash In/Out must use the same transaction/allocations as Accounts. Editing or cancellation must update its source balance and cash/bank effect atomically. Do not let a generic cash edit detach the payment from its bill/purchase. Existing Cash In/Out entry points for these purposes must select the party and show allocation rather than creating unrelated manual cash records.

Keep the accounting purposes distinct: customer collections are not new sales; supplier stock payments are not Shop Expenses; actual Shop Expenses use the shared expense system in Sections 9-12. Preserve explicitly confirmed opening balances, advances and history without inventing balancing records.

Additional acceptance checks: multi-line supplier invoice and partial settlement; duplicate invoice/save warning without guessing identity; receipt edit/reversal after supplier payment; no new liability after product-only edits; correct distinct supplier count; linked cash-payment edits update Accounts once; Home Bill History replaces the specified Settings shortcut while the gear still opens Settings.

18. Latest confirmation and complete integration-test mandate

The user confirmed the supplier explanation: create a supplier once, select that same supplier while receiving stock, record paid-now and remaining due, and settle the remainder through Accounts. Do not require duplicate manual entry of the debt. Both supplier creation entry points share the same directory. Fully paid purchases retain history with zero due.

The user requests thorough testing of ALL features together, especially the combined figures on Sales & Profit. This section expands the required implementation testing; it does not replace the earlier L, S, E and H acceptance cases. Run those cases AND this integration/regression suite against the final candidate build, fixing failures and rerunning affected flows.

Honest status at document update

- Specification updated; no application tests were run by this Markdown update.

- The arithmetic of the synthetic end-to-end fixture below was independently checked with Python. This verifies expected numbers only, not app behavior.

- Every application scenario starts as NOT RUN. Prior screenshots, an earlier APK test or a successful build cannot be substituted for results from the changed candidate.

- Earlier documents may retain obsolete statements such as no app existing, deposits-only Pigmy, bilingual UI or scheduled cloud uploads. Do not reintroduce those requirements. Follow the latest decisions consolidated here and the latest approved feature scope.

19. Combined profit: compulsory financial reconciliation

Never combine unlike totals under Gross Profit

Gross profit must remain a defined calculation, not the sum of cash received, outstanding balances, inventory potential and savings.

| Measure | Meaning | Exclusions / qualifications |

|---|---|---|

| Net product sales | Product bill revenue after discounts and linked returns | Not later repayments or all Cash In |

| Product COGS | Saved costs associated with those sold quantities, adjusted for valid reversals | Not every inventory purchase or supplier payment |

| Product gross profit | Net product sales minus product COGS | Operating expenses are shown separately |

| Net service revenue | Service revenue after discounts/returns | Not exact service profit |

| Net operating expenses | Products + Services + Shared expenses, less linked expense refunds | Count each canonical expense once |

| Combined recorded operating result | Product gross profit + net service revenue - net operating expenses | Qualified result based on recorded costs/expenses; not guaranteed complete accounting net profit |

| Product result after expenses | Product gross profit - net Products expenses | Excludes Shared expenses, clearly disclosed |

| Service result after recorded expenses | Net service revenue - net Services expenses | Excludes Shared expenses; supplies may span periods |

| Cash balance | Confirmed opening cash + actual cash inflows - actual cash outflows | Bank payments excluded; differs from profit |

| Customer dues / supplier dues | Outstanding obligations and settlements | Not amounts to add to profit |

| Expected stock profit | Projection on current unsold inventory | Never added to realized profit |

| Pigmy tracked balance | Confirmed opening savings + deposits - withdrawals | Neither sales nor operating profit |

If a combined overview is displayed, use a clear breakdown with product gross profit, service revenue, each expense classification, refunds and the resulting recorded operating result. Keep product-only and service-only results separately understandable. Show Shared expenses exactly once in the combined result. Do not sum a combined expense total and its category subtotals again.

Inspect the current ₹12,235 expense aggregation in the screenshot: list every contributing record with ID, purpose, classification, effective date and amount. Reconcile the list to the displayed total. Check specifically for purchases, supplier settlements, Pigmy transfers, duplicate service expenses and unrelated date ranges. Report any defect with evidence; do not assume the number is wrong solely because the loss is large.

Mandatory invariants

1. Every displayed amount can be traced to contributing canonical records and exact rounding rules.

2. Summary cards, graphs, drill-downs and exports for the same scope/range reconcile to the paise.

3. Payments settle receivables/payables; they do not create another sale/purchase/expense.

4. Supplier purchases increase inventory; corresponding saved cost enters product COGS when sold. Do not deduct inventory cost twice.

5. Refunds retain their accounting type: sale refund reverses revenue; expense refund reduces expense; Pigmy withdrawal returns savings.

6. Zero-cost products must be distinguishable from unknown costs. Unknown costs produce incomplete labels, not false exact margins.

7. Original sale prices, discounts and cost snapshots survive later catalog edits and receipt regeneration.

8. Closed-period views use consistent effective dates. Creation/update audit times do not replace transaction dates.

9. Current customer/supplier balances and current stock are not silently presented as date-filtered period movement.

10. Unknown pending sync means potentially incomplete reports, not confirmed zero missing transactions.

20. Golden end-to-end scenario: exact expected results

Use a new disposable database, one Owner test device, no pre-existing records, and all following transactions in one selected test day. Confirm opening cash ₹10,000; bank balance, supplier/customer opening debt and savings openings are confirmed zero. These are test values, never production defaults.

Create Supplier A, Customer A, one product Notebook and a reusable photocopy service. Use integer paise internally. All payments below use shop cash. Add representative photos to at least one expense and one linked refund.

Step	Action	Expected immediate effect
G01	Add Supplier A in Accounts	Available in stock selector; no debt, expense or stock change
G02	Receive 100 notebooks for ₹1,000; pay supplier ₹600	100 pieces; supplier due ₹400; cash ₹9,400
G03	Record ₹100 delivery paid to a separate transporter, capitalized into this receipt	Landed stock cost ₹1,100; unit cost ₹11; cash ₹9,300; supplier due still ₹400; no operating expense
G04	Bill Customer A for 10 notebooks at ₹20 each; apply ₹20 whole-bill discount; collect ₹100	Bill net ₹180; COGS ₹110; gross profit ₹70; stock 90; due ₹80; cash ₹9,400
G05	Separate service-only bill: 5 photocopies at newly entered ₹10 each, fully paid	Service revenue ₹50; product stock unchanged; cash ₹9,450; two completed bills total
G06	Receive ₹30 repayment against Customer A's first bill through Accounts	Due ₹50; cash ₹9,480; sale count/revenue/COGS unchanged
G07	Pay Supplier A remaining ₹400 through Accounts	Supplier due ₹0; cash ₹9,080; stock/operating expense unchanged
G08	Cash Out Shop expense: electricity ₹120, Shared	Gross expenses ₹120; cash ₹8,960; same entry visible in Shop Expenses
G09	From Shop Expenses add paper ₹80, Services	Gross expenses ₹200; cash ₹8,880; same movement visible in Cash In/Out
G10	Cash Out Shop expense: shelving repair ₹40, Products	Gross expenses ₹240; cash ₹8,840
G11	Receive ₹20 explicit linked electricity expense refund	Refunds ₹20; net expenses ₹220; Shared net ₹100; cash ₹8,860
G12	Confirm ₹300 Pigmy deposit	Savings ₹300; cash ₹8,560; no expense or profit effect
G13	Withdraw ₹100 Pigmy savings into shop cash	Savings ₹200; cash ₹8,660; no sales/profit effect
G14	Return 2 saleable notebooks from G04, refund ₹36 in cash using explicit cash-refund option	Restore 2 pieces at saved ₹11 cost; sales reverse ₹36 and COGS ₹22; cash ₹8,624; customer remaining due stays ₹50

G14 uses an explicit cash refund of the returned items' net discounted value (₹18 each), not a debt-credit choice. The previously paid ₹130 covers that ₹36 refund. Test the alternative debt-reduction choice separately in a fresh branch of the fixture; do not apply both treatments to the same return.

Final expected dashboard and ledger values

Check	Expected
Sellable notebooks	92 pieces
Remaining notebook inventory cost	₹1,012.00
Net product sales	₹144.00
Product COGS	₹88.00
Product gross profit	₹56.00
Product gross margin	38.89% when displayed to 2 decimals
Net service revenue	₹50.00
Combined net billed revenue	₹194.00
Gross recorded operating expenses	₹240.00
Linked expense refunds	₹20.00
Net operating expenses	₹220.00

| Net Products expenses | ₹40.00 |

| Net Services expenses | ₹80.00 |

| Net Shared expenses | ₹100.00 |

| Product result after Products expenses | ₹16.00 |

| Service revenue minus Services expenses | -₹30.00 |

| Combined recorded operating result: ₹56 + ₹50 - ₹220 | -₹114.00 |

| Physical cash | ₹8,624.00 |

| Bank balance | ₹0.00 |

| Customers with positive due | 1 |

| Total customer due | ₹50.00 |

| Suppliers with positive due | 0 |

| Supplier A due | ₹0.00 |

| Pigmy tracked savings | ₹200.00 |

| Completed original bills | 2 |

| Linked sale return records | 1; not an extra completed sale |

| Net customer collections against those bills after sale refund | ₹144.00 |

Checks: ₹144 customer collections + ₹50 remaining due = ₹194 net billed revenue. Initial stock cost ₹1,100 - net COGS ₹88 = ₹1,012 remaining inventory. Product result ₹16 + service result -₹30 - Shared net expenses ₹100 = combined result -₹114.

Independent balance check for this deliberately simple fixture: cash ₹8,624 + inventory ₹1,012 + customer receivable ₹50 + Pigmy savings ₹200 - supplier payable ₹0 = ₹9,886, equal to opening ₹10,000 plus recorded result -₹114. This check applies to this fixture; it is not permission to infer complete accounting from incomplete real-world data.

Verify every result from UI, persisted records, relevant graph buckets and export. Restart the app and check again. Back up through the real UI, save/retrieve externally, restore into a second disposable installation and check all values/IDs/photos again. Do not replay the actions after restoration to reconstruct missing results.

21. Complete feature regression matrix

The tests below supplement all earlier cases. Use fresh fixtures or reset to a verified disposable snapshot between branches so one scenario does not invalidate another's expected values.

Inventory, receiving and imports

| ID | Test | Required result |

|---|---|---|

| R-I01 | Create product, edit details, receive more, correct count | Each operation has its own effect; editing never re-adds stock |

| R-I02 | Receive 10 pieces at ₹10, then 10 at ₹20, with no intervening sale | 20 pieces, cost ₹300, weighted average ₹15; later sale uses saved applicable cost |

| R-I03 | Proportional transport on multiple lines with paise remainder | Allocations sum exactly to delivery charge; inventory and actual payees reconcile |

| R-I04 | Sell one pack of 10 priced ₹100 | Revenue ₹100, 10 pieces deducted; wholesale per-piece price not mistaken for pack price |

| R-I05 | Different pack sizes and barcode-unit mappings | Correct selected price/unit/quantity; archived mappings handled safely |

| R-I06 | No barcode, leading-zero barcode, long Urdu/mixed name | Search/select/import/export/receipt preserve exact identity and text |

| R-I07 | Low-stock defaults and overrides; archived product | Correct alerts and historical visibility; no erased balances |

| R-I08 | Damage, shop use, free item and donation | Stock decreases once; no sale/debt; known costs separately reported, unknown cost flagged |

| R-I09 | Reverse giveaway/damage; saleable versus damaged return | Correct stock destination once; no double subtraction/restoration |

| R-I10 | XLS/XLSX/CSV file picker, mapping and preview | Advertised formats work; unsupported fields disclosed; leading zeros and currency preserved |

| R-I11 | Cash ledger selected as product import | Explained rejection/routing; no fabricated stock |

| R-I12 | Repeat/overlapping import; legitimate similar rows | No duplicate opening stock; do not discard legitimate transactions based only on matching amounts |

| R-I13 | Corrupt/empty/wrong-type file; provider denial; interruption | Useful error, no partial silent changes; retained input and safe retry |

| R-I14 | Multi-product invoice, paid/part-paid/unpaid | Single correct purchase liability and transport/payment total; supplier history reconciles |

| R-I15 | New purchase after earlier sale and catalog-cost edit | Earlier bill cost snapshots/profit unchanged |

Billing, payments, returns and receipts

| ID | Test | Required result |

|---|---|---|

| R-B01 | Product-only, service-only and mixed bills | Correct line types; stock deducted only for products |

| R-B02 | Products ₹160 + services ₹40, whole-bill 10% discount | ₹180 total; product revenue ₹144, service revenue ₹36; allocations exact |

| R-B03 | Amount/percentage discount, 0%, 100%, decimal rounding | Valid limits; exact line/bill reconciliation; discount not mistaken for unpaid debt |

| R-B04 | ₹48 bill, cash tender ₹50 | Applied ₹48, change ₹2; physical cash increases net ₹48 |

| R-B05 | ₹48 bill, tender ₹47.50 | Explicit remaining ₹0.50 handling; no unassigned debt or hidden discount |

| R-B06 | Cash/digital/credit split payment | Applied amounts sum to paid total; each destination correct; due preserved |

| R-B07 | Bill-only price override versus catalog-price edit | Override affects current bill only; historical bills unchanged |

| R-B08 | Multiple held bills, restart, resume and cancel draft | Durable drafts; no stock/debt/sales posting until completion |

| R-B09 | Double tap/retry at save and payment boundaries | One sale and exactly one set of financial/stock effects |

| R-B10 | Partial return, exchange and cancellation after repayment | Original links retained; explicit refund/debt-credit allocation; no orphan payment |

| R-B11 | History search/filter and cross-navigation to Accounts | Same persisted bill/customer identity, correct distinct customer count |

| R-B12 | Regenerate old receipt after catalog/customer changes | Original transaction number/date/item/price snapshots retained as required |

| R-B13 | Short/long/multi-page receipt, Urdu names and big amounts | No overlaps/crossing rules; correct font shaping, numeric alignment and pagination |

| R-B14 | Receipt reference 1 photocopy ₹2, discount ₹1 | Gross line/subtotal ₹2; discount ₹1; payable/paid ₹1; due ₹0 |

| R-B15 | Preview/PDF/print/share | Same correct selected bill; removed unwanted receipt statements remain absent |

| R-B16 | WhatsApp missing/cancelled; stale cached PDF; permission lifetime | Valid readable attachment when available; no saved-bill mutation; no claim of delivery from launch alone |

| R-B17 | Save PDF/printing unavailable or cancelled | Clear status and retry; no duplicate bill or lost receipt |

Accounts, cash, expenses and services

| ID | Test | Required result |

|---|---|---|

| R-A01 | Same supplier created/selected from both entry points | One directory identity; no debt from creation alone |

| R-A02 | Multiple customers/suppliers, multiple bills/purchases each | Distinct positive-balance counts and totals correct; credits separate |

| R-A03 | Payment allocated across several obligations | Allocations sum to applied payment; no repeated full credit per bill |

| R-A04 | Opening balances with missing historical bills | Explicit reviewed amounts/provenance; no fabricated purchases/sales |

| R-A05 | Cash/bank source changed on linked payment | Old source reversed and new applied once; balances consistent |

| R-A06 | Backdate entry; edit date across day/month boundary | Correct period movement; audit date remains separate |

| R-A07 | Future-dated completed entry | Test only against an explicitly documented existing/confirmed policy; unresolved policy remains an open decision |

| R-A08 | Category create/rename/archive/filter | Stable links and totals; archived historical entries remain visible |

| R-A09 | Photo camera/gallery/replacement/removal/permission denial | Durable readable evidence; no amount change; form survives failure |

| R-A10 | Cash ledger Notes with blank category and malformed dates | Notes preserved; malformed dates not replaced by today; summaries not imported as transactions |

| R-A11 | Ordinary cash income and owner transfer | Cash changes by purpose; no automatic sales or expense reduction |

| R-A12 | Refund earlier expense in later month; reclassify original | Correct date-specific reduction and attribution; original cash not replayed |

| R-A13 | Service definition and new job | No saved/default rate; fresh job requires rate; current held job retains its rate |

| R-A14 | Switch A4 B&W to Colour | Entered rate cleared for fresh option; units explicit and correct |

| R-A15 | Service return and service expense | No product stock restored/consumed automatically; no duplicate service expense |

| R-A16 | Shared shop and service-specific costs | Classification clear; shared cost not arbitrarily allocated or subtracted twice |

| R-A17 | Empty/custom filtered export | Exact selected rows only; attachments honestly represented as indicators |

Pigmy savings

| ID | Test | Required result |

|---|---|---|

| R-P01 | Add fourth/fifth scheme; rename/archive/reactivate | No fixed-three assumption; history/IDs remain intact |

| R-P02 | Deposits A ₹100, B ₹200, C ₹300; edit B ₹250; cancel C | Combined ₹600, then ₹650, then ₹350; linked cash effects once; no profit change |

| R-P03 | ₹10,000 savings; withdraw ₹3,000; edit to ₹2,500; cancel | Tracked balance ₹7,000, ₹7,500, ₹10,000; destination receipt adjusted/reversed once |

| R-P04 | Bank-funded deposit/bank-received withdrawal | Physical cash unchanged; bank/savings effects correct |

| R-P05 | Confirm historical opening savings | No new deposit/outflow; no invented history; correct opening/movement/closing totals |

| R-P06 | Withdrawal above tracked funds or incomplete history | Explained validation/review; no fabricated balancing deposit |

| R-P07 | Manual Pigmy category on generic entry | Does not create a hidden scheme transaction; user directed to linked workflow |

| R-P08 | Retry, restart and restore | No replayed cash transfers; no interest fields/income introduced |

Reports, graphs and financial boundaries

| ID | Test | Required result |

|---|---|---|

| R-F01 | Golden scenario across all financial screens | All exact values in Section 20 match |

| R-F02 | Today/week/month/custom/all-time, midnight boundaries | Same effective dates, timezone and inclusive/exclusive boundary rules everywhere |

| R-F03 | Backdated sale/expense/refund after initial report | Correct affected buckets and summaries, no stale total |

| R-F04 | Zero sales, loss, returns exceeding current-period sales | Honest signed values; margin Not applicable when denominator is unsuitable |

| R-F05 | Missing costs, negative stock, pending sync | Explicit incompleteness; no misleading precise total or stock projection |

| R-F06 | Product/service/shared expense and refund drill-down | Every contributing record once; sum reconciles exactly |

| R-F07 | Daily manual summary plus underlying individual bills | No duplicate revenue; summary purpose documented before counting |

| R-F08 | Chart bar/point, filtered list and export | Same record scope and exact bucket total |

| R-F09 | Large history and range changes | Responsive screen; no stale asynchronous results from previous filter |

| R-F10 | Supplier/customer payment, Pigmy movement and catalog edit | No unintended change to recognized sales/gross profit |

Two-phone operation, access and Android behavior

| ID | Test | Required result |

|---|---|---|

| R-D01 | Owner-approved pairing and revoked invitation/device | Correct role; no self-promotion or unauthorized financial data |

| R-D02 | Billing completes offline sale, then reconnects | Durable local bill; Owner accepts once; stock/debt/payment effects once |

| R-D03 | Drop connection before remote commit and after commit before acknowledgement | Retry safe; acknowledgement only for durable commit |

| R-D04 | Both devices sell last piece offline | Both legitimate bills retained; shortage flagged; no discarded sale |

| R-D05 | Unique receipt numbers through restart/replacement/restore | No collision or unintended renumbering |

| R-D06 | Delayed offline costing | Documented policy; no unsupported exact historical-profit claim |

| R-D07 | Screen off, battery restrictions, Wi-Fi/address change, restart | Recoverable connection; durable queues; honest status |

| R-D08 | Owner expenses/photos/costs through links/exports/sync | Restricted roles cannot retrieve unauthorized records |

| R-D09 | Biometric/device credentials, resume and lock timer | Existing access policy enforced without data loss |

| R-D10 | Navigation, back button, large fonts and keyboard | No dead shortcut, clipped action, lost form or inaccessible Settings |

Two real devices are required to mark the hardware synchronization/lifecycle tests passed. A single device or JVM simulation can support partial evidence only. Unresolved Billing permissions or offline costing rules must be listed and resolved before claiming the corresponding tests passed; do not silently broaden access.

Backup, migration and failure recovery

| ID | Test | Required result |

|---|---|---|

| R-R01 | Backup Now while committed writes exist in WAL | Consistent complete snapshot; no missing recently committed records |

| R-R02 | Full archive manifest and attachment inspection | Bills, drafts, stock, parties, allocations, expenses/refunds, Pigmy, settings and branding covered |

| R-R03 | Save Copy to Drive/Files, retrieve and isolated restore | Verified data/IDs/photos match; local backup retained |

| R-R04 | Provider offline/cancelled/absent/revoked/full | Honest local versus provider-save status; no false cloud confirmation |

| R-R05 | Corrupt/unsafe-path/missing-content/unsupported-schema archive | Rejected before replacing current database; recoverable prior data |

| R-R06 | Interrupt save/restore or exhaust storage | No half-posted finances; old data/evidence recoverable |

| R-R07 | Upgrade supported old schema with real-looking linked history | No destructive fallback; IDs/balances/attachments preserved |

| R-R08 | Restore with pending sync events | Reconciliation by identity; no repeated bills/payments or role overwrite |

| R-R09 | Two dated backups, failed later attempt | Prior backups and successful timestamps retained accurately |

| R-R10 | Release-equivalent/minified build | Fonts/logo/camera/picker/URI/backup/lock tested on Android, not inferred from debug build |

22. Execution procedure, defect handling and evidence

Before testing

1. Identify current source revision, local changes, application ID, version code, signing identity and candidate artifact hash. Preserve existing work and do not copy private signing material into reports or fixtures.

2. Create an isolated build/data environment. Keep test records, receipts, Drive folder and devices separate from production. Do not send synthetic receipts to real customers.

3. Record supported Android/device sizes, actual timezone, font size, locale, permission state and database schema version.

4. Prepare independent expected-value fixtures and record IDs. Do not derive the expected answer by calling the same app calculation under test.

5. Resolve documented policy gaps from existing source/accepted decisions where possible. Otherwise list the precise open question; mark dependent tests BLOCKED rather than inventing behavior.

Test layers

- Calculation tests: exact paise, discount allocation, weighted costs, refund allocations, balance/count calculations and date bucket rules.

- Database/repository integration: transactions, foreign-key relationships, rollback, migrations, unique event IDs and single financial effects.

- UI/device flows: perform saves through real controls; inspect persisted outcomes and cross-screen refresh, accessibility and lifecycle behavior.

- End-to-end: execute the golden scenario, export, restart, externally save backup, restore elsewhere and reconcile the whole app.

- Release regression: rerun affected tests after fixes against the actual candidate. A tested debug APK does not automatically certify a different release artifact.

Per-test result record

For every L/S/E/H/G/R test, record:

```text

Test ID and title:

Candidate version / source revision / APK SHA-256:
```

Device / Android / timezone / schema:

Preconditions and fixture snapshot:

Steps actually performed:

Expected values and records:

Actual values and records:

Status: NOT RUN | PASS | FAIL | BLOCKED | PARTIAL

Evidence: screenshots, exported rows, database checks, logs as applicable

Defect ID / exact blocking dependency:

Fix revision and rerun result:

```

PARTIAL is not PASS. Record the unverified part explicitly. Keep secrets and real customer evidence out of shared logs. Distinguish implementation defects from unavailable hardware or unresolved product decisions.

## Required final reconciliation report

Include a table with expected/actual/difference for cash, bank, each product quantity/cost, product revenue/COGS/gross profit, service revenue, expenses/refunds by classification, combined recorded result, each customer/supplier due, distinct debtor counts, bill count and Pigmy balances. Differences must be zero in exact monetary/quantity units, except documented display rounding of percentages.

Also include:

- UI before/after evidence for Pigmy, stock form, Accounts, Sales & Profit, Shop Expenses, Bill History and Home shortcut placement.

- Cross-links and payment-allocation evidence for customers and suppliers.

- Every contributor to the screenshot's expense total, with any correction explained.

- Graph-to-card-to-export reconciliation for several periods, including a loss and earlier-period refund.

- Golden fixture results before restart, after restart and after isolated restore.

- Complete test matrix with pass/fail/blocked/partial/not-run counts; no blanket “everything works” claim while required checks are unverified.

- Remaining issues, affected features, recovery steps and exact build identity.

## Release gate

Do not recommend production installation with unresolved data loss, duplicate postings, unexplained stock/money differences, unauthorized access, broken migration/restore or incorrect gross-profit aggregation. Fix and rerun the affected integration paths. Cosmetic-only issues should still be listed honestly rather than silently marked passed.

Producing this document is not test execution. Implementers must supply build-specific evidence before saying the features work together. Installing on the live shop phone remains a separate explicitly approved action; never clear data/uninstall to bypass a failure.

## 23. Latest addition: readable numbers with Indian digit grouping

The user reports that numbers lack commas and are difficult to read. Apply consistent Indian-style grouping across the app, receipts and appropriate exported display formats.

| Value | Display |

|---|---|

| 999 rupees | ₹999.00 |
```

| 1000 rupees | ₹1,000.00 |

| 12235 rupees | ₹12,235.00 |

| 100000 rupees | ₹1,00,000.00 |

| 1234567.50 rupees | ₹12,34,567.50 |

| Negative 12197.72 rupees | -₹12,197.72 |

| 123456 pieces | 1,23,456 pcs |

Coverage and consistency

- Use a shared formatting policy in Home summaries, stock costs/prices/quantities, bills, receipts/PDF/print/share, Bill History, customer/supplier Accounts, Cash In/Out, Shop Expenses, Pigmy, graphs and report detail/export previews.

- Monetary display uses two decimal places consistently. Quantity precision follows the supported unit; integer pieces must not acquire meaningless decimal places.

- Use full grouped values for cards, details and chart tooltips. If a chart axis abbreviates lakhs/crores, label the scale and retain the exact amount in the tooltip/table. Do not truncate or abbreviate the only visible balance.

- Reflow long amounts at narrow widths/larger fonts. Commas must not introduce clipping or split a money amount across lines.

- Do not group receipt/invoice numbers, phone numbers, dates, barcodes, customer IDs or other identifiers. Preserve leading zeros.

- Formatting changes presentation only, not stored amounts, arithmetic, rounding or historical values. Continue exact paise calculations; never calculate using formatted text.

- Amount inputs must remain easy to edit. A plain editable number with a grouped amount preview is acceptable; if live grouping is used, preserve caret position and decimal entry. Correctly handle pasted valid grouped amounts and clearly reject malformed input without silently changing its value.

- Verify the actual Android/PDF formatting implementation produces Indian grouping. Do not assume a locale label alone guarantees the grouping pattern on every runtime.

Excel numeric representation

- XLSX amount/quantity cells remain numeric, with a tested display format for Indian grouping. Do not export currency strings merely to obtain commas; SUM, sorting and formulas must work.

- Show the currency in column headings or valid numeric formatting. Dates remain real spreadsheet dates/times with a readable unambiguous display.

- Test representative small, lakh, crore, decimal and negative values in supported spreadsheet viewers. Where a viewer does not honor Indian grouping, disclose the limitation rather than corrupting numeric data.

- CSV cannot carry Excel cell formatting. Export machine-readable numeric values with proper CSV quoting where necessary; do not promise persistent visual commas in CSV. Preserve identifier columns as appropriate to the existing import/export contract.

24. Latest addition: filtered Cash In/Out Excel export and sharing

The user requests selecting a date range and categories when sharing the Cash In/Out spreadsheet. Add a clear Export / Share Excel action that opens a filter-and-preview screen or sheet. Recommend the following controls; reuse existing filters where possible rather than maintaining separate conflicting implementations.

Export controls

| Control | Behavior |

|---|---|

| From date / To date | Calendar pickers; inclusive calendar dates; show actual selected range |

| Quick periods | Today, This Week, This Month, All Time and Custom |

| Direction | All, Cash In, Cash Out; follow existing ledger terminology for bank entries and label sources clearly |

| Categories | All categories or multi-select categories; include an explicit Uncategorized option for legacy entries |

| Payment source/destination | All supported sources, Shop cash, Bank/digital accounts as actually represented by the app |

| Transaction purpose | All or selected actual types: shop expense, linked expense refund, customer repayment, supplier payment, savings transfer and other supported types |

| Expense classification | All, Products, Services, Shared when expense/refund purposes apply; explain scope or hide irrelevant control |

| Status | Active entries by default; optional audit export includes cancelled/reversed entries clearly separated from active totals |

- Seed the export with the current Cash In/Out screen's filters; let the user adjust them before exporting. Show every active filter visibly.

- Category and transaction purpose remain distinct. Selecting a category called Pigmy does not create/reclassify a transfer, and selecting Shop expense purpose does not include supplier settlements.

- Selecting several categories uses OR within that selection; different filter groups combine with AND. Explain an incompatible/no-match combination with an empty state rather than silently relaxing filters.

- Do not require a new filter capability that exposes Owner-only records to Billing roles. Export permissions match underlying data permissions.

Date correctness

- Filter by the transaction's effective date/time, not import time, creation time or latest edit time.

- Include the whole To date in the app's established business timezone. Use a start-inclusive / next-day-start-exclusive interval internally instead of an arbitrary 23:59:59 cutoff.

- Reject From later than To. Preserve date selections after validation errors and cancellation.

- When All Time is selected, show the actual covered dates or an explicit no-records state. Do not silently interpret an empty selection as All Time.

- Backdated records appear in their effective period. Refunds and repayments use their own effective dates, with references to the original transaction retained.

Preview before file creation

Display:

1. Date range and active filters.

2. Number of matching active transactions.

3. Selected inflow total.

4. Selected outflow total.

5. Selected net movement = inflow minus outflow.

6. A preview list and category subtotals where useful.

7. Save Excel and Share Excel actions.

Use “Selected net movement”, not “Profit” or “Shop cash balance”. Filtered electricity spending does not establish the full shop cash balance. All-sources movement is not physical cash. If an opening/closing cash balance is shown, calculate it from the full relevant account ledger and label its scope separately; never derive it from category-filtered rows alone.

For a filtered running column, use “Cumulative selected movement”, starting from zero. Do not label it actual account Balance. Exclude supplier/customer due figures from cash movement unless an actual payment occurred.

No matches: show zero transactions and zero totals, with Change filters. Do not export all transactions instead. An empty workbook may be offered explicitly, but sharing it must not look like a successful export of records that do not exist.

Workbook contents

Use a real XLSX workbook with:

- **Summary**: export generation time, effective date range/timezone, selected categories/purposes/sources/classifications/status, active transaction count, inflow/outflow/net movement and category subtotals. State that this is a filtered ledger export, not a complete backup or profit statement.

- **Transactions**: stable transaction ID, effective date/time, direction, purpose, category, classification where applicable, description/notes, source/destination, party/scheme where applicable, original bill/purchase/expense reference where available, inflow, outflow, status and attachment-present indicator.

- Optional **Audit** sheet if cancelled/reversed entries were requested. Avoid adding cancelled rows as ordinary active cash movements. Follow the canonical reversal model so neither original nor reversal is counted twice.

Keep Notes separate from Category. Preserve Urdu/mixed-script text, leading-zero identifiers and original receipt references. Store user text as text, not spreadsheet formulas; safely handle notes/names beginning with formula-like characters in supported export formats.

Use readable headers, frozen header row, column filters, useful widths, wrapped notes and numeric currency cells. Long notes must remain accessible without making every column excessively wide. Include category subtotals even when the selection is All categories, provided they reconcile to the selected rows.

Photos are not embedded by default. Show attachment presence without private local filesystem paths. State clearly that the spreadsheet does not contain the original evidence images or constitute a restorable backup.

Summary and Transactions must come from the same consistent snapshot. Concurrent saves during generation must not produce a summary for one set of rows and a sheet for another.

Save and share behavior

- Use a meaningful dated filename, for example `Ali_Educational_Cash_2026-11-01_to_2026-11-30.xlsx`. Keep detailed filters inside the workbook instead of putting private party names or an excessively long category list in the filename.

- Generate and validate the workbook before opening the share chooser. Provide a readable content URI with the correct XLSX MIME type and temporary read permission.

- Keep the generated file available long enough for the recipient app to read it. The user chooses/confirms the recipient.

- Cancelled sharing, missing recipient app, provider failure, low storage or generation interruption leaves all underlying transactions untouched and offers a useful retry.

- Opening WhatsApp/chooser does not prove delivery. Saving through Files and sharing are separate statuses.

- Work offline for local generation. Do not require a cloud account to produce the workbook.

25. Additional formatting and filtered-export acceptance tests

| ID | Scenario | Expected |

|---|---|---|

| N01 | 999, 1000, 100000, 1234567.50, zero and negative amounts | Exact Indian-grouped display and two currency decimals across screens/receipts |

| N02 | Large amounts with larger fonts/narrow screen | No clipped digits, lost minus sign or wrapped partial amount |

| N03 | Phone, receipt number, barcode with leading zeros | No numeric grouping or identity change |

| N04 | Type/edit/paste valid grouped amount | Correct exact value; stable editing; invalid text rejected clearly |

| N05 | Export grouped currency to XLSX | Numeric cells remain summable/sortable and show correct display in tested viewers |

| N06 | Grouping after restart/restore and receipt regeneration | Same amount everywhere; no stored-value migration or rounding drift |

| X01 | From/To inclusive date boundary | Includes midnight on From and final instant of To; excludes next day |

| X02 | Same-day range, leap-day/month boundary, backdated entry | Correct effective-date selection and totals |

| X03 | Invalid range and cancelled date picker | Clear validation; prior form/filter values retained |

| X04 | One category, multiple categories, Uncategorized, All | Correct OR-within-category/AND-between-group behavior; archived history retained |

| X05 | Direction/source/purpose/classification combination | Only matching authorized entries; expense refunds and transfers retain their types |

| X06 | No matching rows | Zero preview; no fallback to exporting all data |

| X07 | Preview versus Summary versus Transactions | Same count/inflow/outflow/net movement to the paise |

| X08 | Category-only export | Net movement not labelled profit or full account balance |

| X09 | Active versus audit export with cancellation/reversal | Active totals correct; cancelled audit history clearly separate; no double counting |

| X10 | Notes/Urdu/leading-zero IDs/formula-like text | Preserved safely as intended data; no accidental executable formula |

| X11 | Save/share/retry/cancel/missing app/offline/low storage | Valid XLSX when successful; truthful state; no transaction mutation or duplication |

| X12 | Large filtered history and concurrent new save | Responsive generation; one consistent snapshot and reconciled summary |

| X13 | Attachment indicator and restricted role export | No private paths/photos or unauthorized Owner information leaked |

| X14 | Reopen shared workbook in supported mobile spreadsheet app | Correct sheet contents, numeric types, date display, grouping and usable column layout |

Independent export fixture

On disposable data, record:

| Effective date | Direction | Category | Amount | Purpose |

|---|---|---|---|---:|---|

| 1 Nov 2026 | Out | Electricity | ₹1,000.00 | Shared shop expense |

| 2 Nov 2026 | Out | Paper | ₹200.00 | Services shop expense |

| 3 Nov 2026 | In | Electricity | ₹100.00 | Explicit refund linked to the 1 Nov expense |

| 4 Nov 2026 | In | Other | ₹500.00 | Ordinary non-expense income, with actual supported purpose |

| 5 Nov 2026 | Out | Electricity | ₹300.00 | Shared shop expense |

Select 1-3 November and Electricity: 2 rows, inflow ₹100, outflow ₹1,000, selected net movement -₹900. Select 1-3 November and Electricity + Paper: 3 rows, inflow ₹100, outflow ₹1,200, net -₹1,100. Select 1-3 November, Electricity and Out only: 1 row, outflow ₹1,000, net -₹1,000. The 4/5 November rows must never appear in these exports. The refund remains an inflow linked to its expense; do not rewrite the original expense to ₹900 and also export the ₹100 refund.

Run these cases along with all earlier tests. Add workbook samples, verified numeric cell types and actual preview-to-file reconciliation to the release evidence. This update remains a specification, not a claim that app formatting/export changes have been implemented.

ANTIGRAVITY_SLIM_WIDGET_AND_FINAL_RELEASE_TESTING.md

Ali Educational: slim-bar widget and final release verification

Date: 1 November 2026

1. Scope and status

Implement the user's selected **Slim bar** Android home-screen widget, then rigorously test the whole existing app and fix errors before proposing it as final.

This is a standalone handoff. It does not repeat or add the earlier layout, graphs, supplier, Bill History or spreadsheet feature-change requests. Existing implemented and separately approved features are regression-test scope, not authorization for unrelated redesigns.

This document specifies work and acceptance criteria. It does not claim the widget exists, tests have passed or the app is ready for production.

2. Selected widget: Slim bar

Use one compact, wide Android launcher widget with a small Ali Educational heading and four equally spaced actions in one horizontal row:

```text
Ali Educational
[incoming] [outgoing] [receipt +] [box +]
Cash In Cash Out New Bill Add Stock
```

The illustrated icons above describe their meaning; use crisp, locally bundled assets consistent with the app.

Appearance

- Match the existing dark charcoal/deep green background, rounded corners, gold accents and readable light labels.

- Cash In: green incoming arrow; Cash Out: orange outgoing arrow; New Bill: gold receipt icon; Add Stock: gold/light box icon.

- Keep the exact labels **Cash In**, **Cash Out**, **New Bill**, **Add Stock**. Do not relabel them Income/Expense: customer repayments, supplier settlements and savings transfers have different accounting meanings.

- Use four equal action areas, icon above label, generous touch targets and consistent alignment.

- Keep the widget compact; no balances, charts, customer information or extra shortcuts.

- Support an appropriate wide launcher span, typically four columns, while measuring the actual available space. Launcher cell dimensions vary; do not promise one universal physical size.

- Set a supported minimum width/height so labels remain readable. Prevent unsupported shrinking rather than clipping text or silently removing an action. Reflow is not a substitute for the user's selected four-action slim layout.

- Test supported resizing, display scaling and accessibility font sizes. Never split Cash Out or Add Stock into broken word fragments.

- Provide clear accessible action descriptions and sufficient contrast. Do not rely solely on icon color.

3. Shortcut behavior

Action	Destination	Required behavior
Cash In	Existing new Cash In form	Incoming direction preselected; no amount or transaction automatically saved
Cash Out	Existing new Cash Out form	Outgoing direction preselected; existing purpose/category workflow retained
New Bill	Existing billing flow	Start/resume through the established draft-safe workflow; never discard an unfinished bill
Add Stock	Existing stock receiving/add stock flow	Existing product/supplier/purchase validation retained; no stock automatically added

- These are shortcuts into the same app screens, commands, database and permission checks. Do not create a second widget-specific accounting or billing implementation.

- Tapping any action opens a form; it does not post money, save a bill, change stock or create debt.

- Keep ordinary Save/Confirm/Cancel behavior. Cancelling the form leaves records unchanged.

- Cash In/Out must retain the distinction between sales collections, repayments, supplier settlements, shop expenses, refunds and savings transfers supported by the app.

- If a draft or partially entered form already exists, preserve it and use the existing hold/resume/new-work flow. Repeated widget taps must not stack duplicate screens or overwrite input.

- Back navigation should behave predictably and must not reopen the same action endlessly.

- Tapping the heading may open the app Home screen, if implemented; it must not trigger a financial action.

4. Lock, permissions and lifecycle

- If the app requires authentication, unlock first and then continue to the intended action. Authentication cancellation must not open the protected form or commit anything.

- Validate role permissions inside the destination screen/command. Widget navigation is not a permission bypass.

- Owner has the four shortcuts. On a restricted device, retain clear role-appropriate behavior: unavailable actions may show a locked state and explanation, but must not reveal Owner data or grant rights. Do not broaden Billing-role permissions for this widget.

- Handle cold start, warm start, background app, process death, device restart and app update.

- Preserve the intended action through authentication/recreation without replaying a Save command.

- Use distinct, correctly routed action identities so Cash In cannot accidentally open Cash Out or another shortcut.

- Validate externally supplied navigation arguments. Do not accept arbitrary amounts, customer IDs or privileged operations as trusted merely because an action resembles a widget request.

- Support multiple widget instances without shared navigation collisions.

- The widget is a launcher shortcut surface: local actions should not require an internet connection. It does not need polling, scheduled financial refresh or background uploads.

- After an app update, existing widget instances must remain functional or recover gracefully with a clear refresh/setup path.

5. Widget acceptance tests

| ID | Scenario | Expected |

|---|---|---|

| W01 | Add widget from launcher picker | Named Ali Educational; slim layout with four correct labels/icons |

| W02 | Tap each action from terminated app | Correct destination and direction; no automatic record creation |

| W03 | Tap each action from warm/background app | Correct screen without duplicated navigation or lost input |

| W04 | Locked app, successful authentication | Selected destination opens after unlock |

| W05 | Cancel/fail authentication | Protected destination/data unavailable; no financial change |

| W06 | Existing bill/form, then widget action | Draft/input preserved; no silent replacement |

| W07 | Rapid repeated taps | No duplicate bill/form save or financial posting |

| W08 | Save then return/back/reopen | One committed transaction; no repeated submission |

| W09 | Cancel each opened form | No stock, money, account or bill-count changes |

| W10 | Restricted role uses each shortcut | Existing permissions enforced at navigation and command layers |

| W11 | Airplane mode | Local permitted flows work and persist normally |

| W12 | Restart device/process, rotate where supported | Routing and pending form behavior remain correct |

| W13 | Resize and large text/display scaling | All four actions readable and reachable at supported size |

| W14 | Multiple widget instances | Each action routes correctly without stale/crossed intents |

| W15 | Update existing installed test app | Widget remains functional; app records preserved |

| W16 | Supported launcher/device variations | Touch areas, clipping, background and labels verified on actual devices |

| W17 | Accessibility navigation/readout | Four meaningful action names and predictable focus order |

| W18 | Actual saved action initiated through widget | Same financial result as initiating it inside the app |

6. Final testing mandate: test the entire app together

Before calling the app final, inventory every current screen, action, setting, transaction type, importer/exporter, integration and permission. Map each to tests. Do not limit testing to the widget or happy-path screenshots.

Inspect implementation and persisted effects as well as UI output. Correct reproducible bugs and rerun affected scenarios. Do not hide wrong values, erase records or weaken validation to make tests pass.

Use existing detailed approved specifications as expected behavior where applicable. If implementation differs from an ambiguous or unresolved business rule, document the difference and ask a focused question rather than inventing financial policy. Ordinary coding defects do not require repeated permission to repair within the authorized scope.

Before execution

1. Identify current source revision, uncommitted changes, app ID, schema, version, signing compatibility and candidate APK hash.

2. Preserve current work. Test with disposable databases/devices and synthetic records; never modify live shop records to test a theory.

3. Record device/Android version, timezone, locale, font scale, permissions and connectivity state.

4. Prepare independent expected records and exact totals. Do not calculate expected results using the same application function being tested.

5. Inventory prior defects and unverified features. Include them explicitly; earlier successful tests do not certify later changed builds.

7. Whole-app regression coverage

For every area below test: valid use, invalid input, zero/decimal/large amounts where applicable, edit, cancel/reverse, duplicate tap, restart, interruption and cross-screen reconciliation.

| Area | Required checks |

|---|---|

| Navigation and settings | Every button/shortcut opens the correct screen; back navigation, theme and lock settings persist; keyboard and larger fonts do not hide controls |

| Products and stock | Add/edit/receive/count correction distinct; variants, units/packs, barcodes, search, categories, low stock, archives, photos and multilingual names work |

| Purchase costs | Weighted-average calculations, exact transport allocation, actual payees and historical sale cost snapshots remain consistent |

| Suppliers and purchases | Supplier identity reused; multi-line invoice counted once; paid/part-paid/unpaid purchases, later settlements, opening debt and reversals reconcile |

| Billing | Products/services/mixed bills, manual prices, quantity/unit changes, discounts, drafts, split cash/digital/credit payments and change reconcile |

| Customers and Accounts | Distinct debtor count, due amount, repayment allocation, credits/openings and bill/account cross-links agree; no unassigned walk-in debt |

| Bill history | Completed/cancelled/held statuses and counts correct; search/date filters, original bill identity and later payment history preserved |

| Returns and cancellations | Stock, original revenue/cost, payments and debt adjusted once; later repayments handled; history retained |

| Services | Required job rates, correct units, option changes, held rates and service expenses; service-only lines do not change product stock |

| Cash In/Out | Correct direction, source/destination, effective dates, purposes, categories and attachments; bank does not affect physical cash |

| Shop expenses | One canonical expense across screens; classification, linked refunds, edits and cancelled records produce correct totals without duplication |

| Pigmy | Expandable schemes, openings, deposits, withdrawals, linked cash/bank effects, archives and tracked balances; no sales/expense/interest confusion |

| Giveaways and damage | Stock changes once, known cost separately visible, no fabricated revenue/debt, valid reversals |

| Reports and graphs | Identical scope/dates across cards, buckets, detail and export; negative/empty/incomplete data shown honestly |

| Formatting | Indian commas, currency decimals, negative amounts and Urdu/mixed names; identifiers and leading zeros unchanged |

| Spreadsheet import | Supported formats, mapping, validation, wrong-type files, repeat imports, malformed dates/currency, leading zeros and atomic recovery |

| Spreadsheet export | Effective-date/category/source/purpose filters, empty results, exact preview/file reconciliation, numeric cell types and safe text |

| Receipts and sharing | Correct original bill, discount/payment totals, readable long/multi-page layout, fonts, PDF content permissions, save/print/share cancellation |

| Backup and restore | Complete consistent data/photos/settings/branding/links; actual external save/retrieval and isolated restore; no replayed transactions |

| Migration and update | Supported older schemas and signing compatibility; no destructive fallback; records and widgets preserved |

| Security and roles | Lock lifecycle, permitted actions, deep links, exports and sync enforce the same restrictions |

| Two-phone operation | Pairing, offline durability, retries/acknowledgements, identity/receipt collisions, shortages and incomplete-sync reporting |

| Reliability and scale | Representative large catalog/history, storage exhaustion, permission loss, process death, rapid taps and concurrent operations |

For optional/unimplemented features, state NOT IMPLEMENTED or NOT APPLICABLE with evidence and scope justification. Do not mark them PASS. Missing required approved features remain incomplete work.

8. Gross profit and combined reporting: mandatory audit

Trace every displayed financial total back to canonical contributing records. In particular, audit the reported expense aggregation rather than assuming all cash outflows are expenses.

- Net product sales = product revenue after discounts and linked returns.

- Product COGS = the saved costs of those sold items, adjusted by valid reversals.

- Product gross profit = net product sales minus product COGS.

- Net operating expenses = active recorded operating expenses minus explicit linked expense refunds, each counted once.

- Combined recorded operating result, if shown = product gross profit + net service revenue - net operating expenses.

- Shared expenses count once in a combined result, not once in each subtotal and again in the total.

- Customer repayments do not add sales. Supplier stock payments do not create a second expense. Pigmy transfers do not affect profit.

- Unsold inventory cost/potential is not realized sales/profit. Current cash balance is not profit.

- Missing costs and incomplete sync produce clear warnings. Unknown is not zero.

- Period filtering uses effective dates consistently. Date-filtered movements are not automatically opening/closing balances.

- Changes to catalog prices/costs must not rewrite saved historical sale amounts/costs.

- Verify signed losses and margin calculation, including zero or negative net-sales denominators.

For a combined screen, reconcile product-only, service-only and Shared components and list precisely what the combined result includes. Do not claim complete accounting net profit when some costs/history are unrecorded.

9. End-to-end financial fixture

Use an isolated database with confirmed opening cash ₹10,000 and zero bank, debt and savings openings. Perform all actions within the same selected day. Record stable IDs and before/after balances at every step.

1. Receive 100 notebooks for ₹1,000 from Supplier A; pay ₹600. Supplier due is ₹400.

2. Pay a separate transporter ₹100 and capitalize it into that stock receipt. Inventory is ₹1,100, cost ₹11 per notebook; supplier due stays ₹400.

3. Sell 10 notebooks at ₹20 each to Customer A with ₹20 total discount. Net bill ₹180; receive ₹100; customer due ₹80; COGS ₹110.

4. Complete a service-only bill for ₹50, paid in cash. No product stock change.

5. Receive ₹30 customer repayment. Customer due becomes ₹50; sales unchanged.

6. Pay supplier the remaining ₹400. Supplier due becomes zero; stock and operating expenses unchanged.

7. Record expenses: Shared electricity ₹120, Services paper ₹80, Products repair ₹40. All paid in cash through the existing shared expense flow.

8. Receive ₹20 cash expense refund explicitly linked to electricity. Net expenses become ₹220.

9. Deposit ₹300 into Pigmy from cash; withdraw ₹100 back to cash. Tracked savings ₹200; profit unchanged.

10. Return two saleable notebooks from step 3 and explicitly refund ₹36 cash, reflecting their discounted ₹18 price. Restore stock at saved ₹11 cost. This is a cash refund, not also a debt reduction; remaining customer due stays ₹50.

Exact expected final state

| Measure | Expected |

|---|---:|

| Notebook quantity | 92 |

| Notebook inventory cost | ₹1,012.00 |

| Net product sales | ₹144.00 |

| Product COGS | ₹88.00 |

| Product gross profit | ₹56.00 |

| Service revenue | ₹50.00 |

| Gross operating expenses | ₹240.00 |

| Explicit expense refunds | ₹20.00 |

| Net operating expenses | ₹220.00 |

| Products / Services / Shared net expenses | ₹40 / ₹80 / ₹100 |

| Combined recorded operating result | -₹114.00 |

| Physical cash | ₹8,624.00 |

| Bank balance | ₹0.00 |

| Customer due / distinct customers owing | ₹50.00 / 1 |

| Supplier due / distinct suppliers owed | ₹0.00 / 0 |

| Pigmy tracked balance | ₹200.00 |

| Completed original bills | 2 |

| Linked sale returns | 1 |

Independent checks: net billed revenue ₹194 = net customer collections ₹144 + customer due ₹50. Inventory ₹1,100 - COGS ₹88 = ₹1,012. Cash ₹8,624 + inventory ₹1,012 + customer due ₹50 + savings ₹200 = ₹9,886, equal to opening ₹10,000 plus the recorded result -₹114 for this deliberately complete fixture.

Run the scenario through actual UI workflows, including widget entry points where available. Compare UI, persistence, reports, graph buckets and exported files. Repeat checks after restart and after restoring a real backup into another disposable installation. Do not repost transactions after restore.

Also test separate branches: bank instead of cash, debt-credit instead of cash refund, later-period repayment/refund, zero-sales period, missing cost, mixed product/service discount and purchase after a previous sale. Do not combine incompatible branches into one expected final balance.

10. Failure and recovery tests

- Interrupt operations before and after database commit and before acknowledgement where applicable. Require either no committed operation or one complete operation; never partial stock/money updates.

- Double tap Save, retry after timeout and reopen after process death. Stable identity must prevent repeated financial effects.

- Fail attachment processing/replacement; preserve input and the previous valid photo. Do not silently discard requested evidence.

- Simulate unavailable provider, denied permissions, offline sharing/save, low storage and corrupt files. Require useful error/retry with no data mutation.

- Reject unsafe or incompatible backup archives before altering live database state. Retain a recoverable pre-restore copy in the supported restore workflow.

- Test real Android release-equivalent/minified behavior for fonts, camera, files, authentication and widget navigation.

- Final two-phone lifecycle/connectivity verification needs two actual devices. Simulated or one-device results must remain qualified.

11. Fix-and-retest procedure

1. Reproduce each failure in isolation; record exact input, expected outcome, actual outcome and affected records.

2. Determine root cause in the relevant UI, calculation, persistence, routing or integration path.

3. Implement the smallest complete fix consistent with approved requirements. Preserve historical data and other features.

4. Add meaningful regression coverage for financial/data-loss/security defects and the discovered failure boundary.

5. Rerun the failed case, related feature tests and affected end-to-end reconciliation. Repeat the complete golden scenario after changes to shared financial logic.

6. Verify migrations/recovery again if schema, serialization or linked record semantics changed.

7. Record fix revision and final evidence. Do not mark a defect resolved solely because the app compiles or a button opens.

Do not automatically repair uncertain production balances by inventing transactions. Any historical discrepancy needs an explainable, reviewed correction path separate from testing.

12. Required evidence and final release gate

For every test, record:

```text

Test ID / feature:

Build version, source revision and APK SHA-256:

Device / Android / schema / timezone:

Initial fixture and record IDs:

Steps actually executed:

Expected result:

Actual result:

Status: NOT RUN / PASS / FAIL / BLOCKED / PARTIAL / NOT APPLICABLE

Evidence location:

Defect or blocking dependency:

Fix revision and retest result:

```

Provide:

- Feature-to-test coverage inventory covering every actual feature/action.

- Widget screenshots at supported sizes and evidence for all four destinations, lock routing and draft preservation.

- Test counts by status, with every omission explained.

- Expected/actual/difference reconciliation of stock, cash/bank, customer/supplier dues, sales, COGS, expenses/refunds, gross profit, combined result and savings.

- Graph/card/export and receipt reconciliation evidence.

- Actual backup retrieval/isolated restore and nondestructive migration evidence.

- Fixed defects and remaining limitations with severity, impact and recovery guidance.

- Candidate APK filename/version/code/hash and whether it is exactly the artifact tested.

Do not call the app final while critical crashes, data loss, unauthorized access, duplicate postings, incorrect profit, unexplained balance differences or failed migration/recovery remain. Required but blocked/unrun tests must remain visible; lack of testing is not a pass.

No finite test suite guarantees absence of every possible bug. The release claim must be bounded by actual coverage, tested devices and remaining limitations, not a blanket promise that everything is error-free.

Never install the production update, uninstall the live app, clear its storage or overwrite live shop data without explicit user authorization. Prepare and verify the candidate first; production installation is a separate approval step.

ANTIGRAVITY_BACKUP_PACKAGE_HISTORY_LIGHT_MODE_AND_BUILD.md

Ali Educational: complete backup package, Bill History repair, readable light mode and Accounts layout

Date: 1 November 2026

1. Latest user request and execution order

Implement these changes in the current app, rigorously verify them, and **build the candidate APK now**. After the user connects the intended phone for this update, install the compatible tested update while preserving its existing app data.

This document is an implementation handoff, not a claim that fixes/build/tests have already occurred. Do not merely document proposed fixes: inspect, implement, test, correct failures and produce the build.

Required work:

1. Back Up Everything produces both a complete restorable backup and readable Excel records from one consistent snapshot.

2. Make saving/sharing the two outputs and restoring on another installation straightforward.

3. Repair the nonworking Bill History button on Home.

4. Fix light-mode readability throughout the app.

5. Fix placement and spacing on the supplied Accounts screen and its supplier counterpart.

6. Create synthetic test files and transactions; prove backup completeness and restoration with record-level reconciliation.

7. Build now; update the intended phone when connected and ready, using the conditions in Section 12.

Preserve all existing features, relationships, shop records, approved permissions and prior completed work. Do not replace the current project with an old review copy or introduce unrelated redesigns.

2. Evidence and limits

The supplied screenshot shows the Customers Owe tab in light mode. The summary title competes with the Add Customer button, the amount is pale against a pale card, the button wraps awkwardly, and the search field shares a crowded row with filters. The empty-state text sits far below the controls.

The supplied `ali_backup_20261101_202423.zip` was inspected read-only and contains:

- `ali_educational.db`: 327,680 bytes.

- `manifest.json`: 221 bytes.

- One JPEG under `attachments/`: 11,673 bytes.

The ZIP CRC check passed. The manifest reports format version 1, schema version 4 and package `com.alieducational.app.debug`. These facts do not prove complete settings/attachment coverage, database integrity, restore success or compatibility with the phone's current installed package. An attempted in-memory database inspection did not complete; no database integrity result was established.

Inspect the current implementation and actual installed package rather than assuming this old archive identifies the next build or signing key. Preserve this archive as a reference; do not overwrite it or restore it over live shop records for testing.

3. Back Up Everything: two useful outputs

One consistent snapshot

On Back Up Everything:

1. Acquire a consistent snapshot of committed data, including any committed WAL data, using a supported database backup/snapshot method.

2. Capture required settings, branding and attachment contents consistently with that snapshot. Prevent concurrent evidence replacement from creating dangling references.

3. Validate the snapshot and build a complete recovery archive.

4. Generate the readable Excel workbook from that exact snapshot, not a second later query of changing live tables.

5. Validate both outputs and their shared snapshot identity.

6. Present accurate per-output status and save/share actions.

Do not casually copy a live database file or separately collect related tables at inconsistent times. Prevent simultaneous backup jobs, show progress, preserve prior successful outputs and allow useful retry after failure.

Output A: complete recovery archive

Suggested stable safe filename:

`Ali_Educational_Complete_Backup_2026-11-01_20-24-23.zip`

Its purpose is to restore the app. It contains the database and every required portable file/settings resource, plus a truthful manifest and integrity checks. The user should never need to extract it or open its database manually.

Output B: readable records

Suggested filename using the same timestamp:

`Ali_Educational_Shop_Records_2026-11-01_20-24-23.xlsx`

Its purpose is reading and checking records outside the app. It is not a replacement for the recovery archive and must not be described as a complete restore source.

Save Both package

Offer a convenient outer package:

```text
Ali_Educational_Backup_Package_2026-11-01_20-24-23.zip
Complete_Backup.zip
Shop_Records.xlsx
Read_Me.txt
```

The recovery archive retains its own manifest and attachment structure. Read_Me explains which file is for recovery and which is for reading, with timestamp and snapshot ID. Do not include passwords, signing keys, private machine paths or unnecessary diagnostics.

Use one outer file for Save Both through Android's native document picker; do not assume every provider supports atomically creating a folder and two files. Also provide Save/Share complete backup only and Save/Share Excel only. If multi-file sharing is supported, handle receiver support honestly; the outer package is a reliable single-file alternative.

Restore Backup must accept either the complete recovery ZIP or the outer package and safely locate the contained recovery archive, so the user does not have to unzip manually. Reject ambiguous packages with multiple candidate backups rather than guessing. Bound nested archive handling; do not recursively unpack arbitrary archives.

4. Backup screen and honest status

Recommended completion view:

```text
Backup ready
1 November 2026, 8:24 PM
Complete recovery file Ready
Readable Excel records Ready
Photos and files [verified count]
[ Save Both to Drive / Files ]
[ Share package ]
[ View details ]
Restore Backup
```

- Counts must come from the validated snapshot. Never hardcode “Everything backed up” based on a nonzero ZIP size.

- View details shows snapshot identity, date, app/schema/format versions, file sizes, verified record/file counts and any warnings.

- Distinguish Local backup created, Excel created, Package ready and Copy saved to selected provider.

- If Excel fails but the complete archive succeeds, retain the recovery file and explain the partial result. Offer retry; do not discard a good backup or claim both succeeded.

- If external saving fails, retain local files and previous successful timestamps. Cancellation is not success.

- Use dated filenames and keep previous backups; do not silently overwrite or delete them.

- Keep manual native Drive/Files saving. No unattended uploads or new OAuth setup in this scope.

- Provider-write completion is not proof of cloud upload completion. Report only what can be verified and show available provider/destination information without fabricated cloud paths.

- Display Owner backup limitations if a Billing phone has pending or unknown unsent records. An absent device's unknown state is not zero.

5. Readable workbook specification

Include meaningful sheets for the actual data model. At minimum, where supported by the app:

| Sheet/group | Readable contents |

|---|---|

| Summary | Snapshot/date, reporting scope, record counts, stock/account/cash/savings totals and incompleteness warnings |

| Products and Stock | Product/variant/unit identities, prices/costs, quantities, categories, archive state and unknown-cost indicators |

| Stock Movements / Purchases | Receipts, invoices, transport allocations, adjustments, damage/giveaways and supplier references |

| Bills / Bill Items | Original numbers/dates/status, product/service lines, discounts, totals, payments and remaining due |

| Payments / Allocations | Customer collections, supplier settlements, linked bill/purchase/opening-debt allocations and reversals |

| Customers / Suppliers | Contact details where authorized, opening balances, positive dues and separate credits/advances |

| Cash In Out | Effective dates, direction, purpose, category, source/destination, amounts, notes and linked IDs |

| Expenses / Refunds | Classification, category, original expense link, source/destination, status and attachment presence |

| Services | Definitions, job details/rates in their historical records and expense attribution |

| Pigmy | Schemes, confirmed openings, deposits, withdrawals, history and tracked balances |

| Categories | Relevant categories and archive state |

- Where sheet names must be shortened to meet XLSX limits, use clear names and a Summary index.

- Include stable references so records can be traced across sheets. Preserve historical status and distinguish active totals from cancelled/reversed audit rows.

- State each summary's scope: current balance, all-time movement or other explicitly defined measure. Do not label cash balance or unsold stock potential as profit.

- Use real numeric cells, Indian currency grouping, two monetary decimals and unambiguous real spreadsheet dates. Verify formulas/sorting remain usable.

- Preserve Urdu/mixed text, leading-zero identifiers and notes. User text must not become executable formulas.

- Use readable headers, frozen rows, filters, appropriate widths and wrapped notes.

- Show attachment presence/count, not inaccessible local paths. Original photos remain in the complete backup; Excel is not an evidence-image backup.

- For very large datasets, respect workbook limits and split clearly indexed sheets/files or explain a recoverable export limit. Never truncate silently while reporting a complete export.

6. Complete backup coverage audit

Do not rely on a fixed list alone: inspect every persistent table, preference store, managed file directory and relevant schema relationship in the current app. Maintain a coverage inventory with storage location, backup method, restore mapping and verification.

Explicitly verify:

- Products, variants, piece/pack mappings, barcodes, categories, photos, costs/prices and archived states.

- All stock receipts/movements/count corrections, purchases, invoices, transport allocation, damage/shop use/giveaways and reversals.

- Completed/cancelled bills, held drafts, lines, discount allocations, original receipt IDs/dates, service snapshots and saved cost snapshots.

- Customer/supplier identities, opening balances, invoices, payments, allocations, credits/advances and audit history.

- Cash/bank accounts or supported sources, confirmed openings, ledger purposes, effective/audit timestamps and linked movements.

- Expenses, linked refunds, Products/Services/Shared classification, categories and actual photos/evidence.

- Service definitions and relevant service reporting data.

- Every Pigmy scheme, opening provenance, deposit, withdrawal, linked transfer, archive status and photo.

- Shop profile, branding/logo assets, relevant preferences, themes, thresholds and supported numbering configuration.

- Applicable sync event identities and acknowledgements needed for safe reconciliation. Do not restore a device role or pairing secret blindly onto another phone.

What “complete” excludes or cannot promise

- Disposable caches may be regenerated if all source records/assets needed to regenerate them are restored. Document that exclusion.

- Android biometric enrollment, device credentials and keystore-bound secrets are not portable ordinary backup files; re-establish protection/pairing through the supported secure flow.

- Signing keys and developer secrets must never be packaged.

- Launcher widget placement may need to be added again on a new phone; preserve app-owned widget preferences if relevant and portable.

- Unsent records on another device cannot be included until received. Clearly disclose this.

- Do not silently omit a real data store by calling it a cache. Every exclusion must have a reason and restoration impact.

Manifest and validation

Record true format/schema/app version, creation time with timezone information, snapshot ID, expected entries, file sizes/hashes and relevant record counts. Hashes detect corruption, not authenticity against a maliciously replaced archive; do not overclaim security.

Validate database readability/integrity, referential relationships, referenced attachments and required settings/assets. File/database checks supplement but do not replace restoring and comparing actual records.

7. Restore workflow and safety

1. User selects complete backup or outer package through Files.

2. Safely inspect in staging: prevent path traversal, absolute paths, oversized decompression, ambiguous nested backups and unsupported formats.

3. Check integrity, schema compatibility, required content and attachment references before changing current records.

4. Show backup date, source app/schema, available counts and warnings; explain whether restore replaces current data. Do not silently merge unrelated snapshots.

5. Require explicit confirmation for the destructive replacement step and create a consistent pre-restore recovery copy.

6. Restore atomically or with a tested recoverable staged swap; reopen/validate the restored data.

7. Reconnect attachments to valid managed locations, not the original phone's absolute paths.

8. Re-establish device-specific access/sync safely; never replay historic transactions as new postings.

9. Show verified completion and any required setup, such as adding the launcher widget again.

Failures must leave original data recoverable and provide a useful error/retry path. Do not uninstall or clear storage to make restoration appear successful.

8. Repair Home Bill History button

The user reports the main-page Bill History button does not work. Treat this as a defect, not a request for a second history implementation.

- Reproduce on the current build and trace tap handler, route, navigation host, role guard and destination initialization.

- Wire the existing Home shortcut to the actual Bill History destination. Preserve its approved placement below Cash In/Out in the former Settings shortcut position.

- Keep Settings accessible through the gear icon.

- Test cold/warm starts, repeated taps, Back, lock/unlock and recreation. No dead tap area, exception swallowed as silence or incorrect route.

- Handle zero bills with a proper empty state; an empty database must not make navigation appear broken.

- Existing bill rows open the selected persisted bill and customer-account links correctly. Viewing/sharing does not duplicate the bill or financial effects.

- If permission denies access, show a clear allowed response; do not bypass role restrictions or fail silently.

- Include a regression test at the actual Home entry point, not only a direct destination test.

9. Light-mode readability throughout the app

Audit all screens and dialogs in light mode, including selected/unselected tabs, chips, cards, amount labels, placeholders, disabled states, error text, charts, dropdowns and bottom navigation.

- Use explicit light-theme semantic colors rather than reusing pale dark-theme accents as body text.

- Use dark primary text and sufficiently dark secondary text on light surfaces. Reserve pale green/gold fills for backgrounds; use darker foregrounds.

- Use a darker readable green for positive amounts on pale green cards. For buttons, choose a foreground/background pair with measured contrast; white on a pale green button is not acceptable merely because it matches the brand.

- Target at least 4.5:1 contrast for normal text and 3:1 for large text; meaningful control boundaries/icons should target 3:1 against adjacent colors. Verify actual rendered pairs, including alpha overlays. These are explicit acceptance targets for this task.

- Disabled controls must remain understandable without resembling active controls. Do not rely on color alone for selected/error/negative state.

- Preserve light mode's appearance; do not fix it by turning all surfaces dark or shrinking labels.

- Verify dark mode has not regressed. Theme changes, process restart and system-theme following must behave consistently with existing settings.

- Test on a real phone at normal and enlarged font/display sizes; record screenshots and measured contrast results for problematic combinations.

10. Accounts placement fixes from the screenshot

Apply the same responsive structure to Customers and Suppliers, preserving the financial meaning of each side.

Recommended arrangement

```text

[ We Owe ] [ Customers Owe ]

Amount customers owe us

₹0.00

No outstanding customer balances

[ + Add Customer ]

[ Search customers... ]

[ All ] [ Outstanding ]

Customer list, or explanatory empty state

```

- Separate summary text/amount from the action button; do not place a long title underneath or against the button.

- A full-width Add Customer row below the summary is preferred on narrow screens. On wider layouts a separate aligned action may fit, but no overlap or awkward word wrapping is allowed.

- Keep the amount prominent and readable with Indian commas; allow large values without truncation.

- Give search its own full-width row and All/Outstanding their own row beneath it. Keep labels visible at larger fonts.

- Use consistent horizontal padding and vertical spacing. Keep content clear of system bars and bottom navigation.

- Position the empty state naturally beneath the search/filter area with readable line length; remove arbitrary large spacers. Distinguish No customers yet, No outstanding balances and No search matches.

- Supplier view follows the same pattern with Add Supplier and correct payable wording.

- Verify populated lists, long party names, large dues, zero balances, credits, archived records and keyboard-open search.

- Preserve correct counts: distinct parties with positive dues, not number of bills. Layout changes must not change account calculations.

11. Synthetic files and end-to-end verification

Create fixtures in a disposable test workspace, never the live shop database. Keep input files, expected-value manifest, generated outputs and test results available for review. Do not use private production records as public sample data.

Required fixture set

1. Stock XLSX/CSV with leading-zero barcode, Urdu/mixed product name, categories, pack/unit examples, supplier references and exact cost/transport totals; also an XLS fixture if XLS import is advertised.

2. Cash-ledger files with effective dates across month boundaries, category/notes separation, expenses, linked refunds where the import contract supports them, and reviewed opening balance treatment.

3. Deliberately malformed/empty/corrupt/duplicate/overlapping files to verify rejection and retry without partial changes.

4. At least two test photos attached to different records and one branding/product image where supported, so a single surviving image cannot falsely prove all file classes are covered.

5. Expected-value manifest with stable IDs, normalized business fields, quantities, money in paise, relationships, attachment hashes and settings.

Do not silently map unsupported import fields to unrelated columns. Use actual UI transactions to create relationships that the importer does not support.

Required populated dataset

Include at least one record in every supported persistent entity class: active/archived products and parties, multi-line purchase, partial supplier payment, product/service/mixed bills, held draft, customer debt and later repayment, return, expense and linked refund, each expense classification, cash/bank movement, Pigmy opening/deposit/withdrawal, category, giveaway/damage, photos, branding and nondefault settings. Include cancelled records and links with later activity.

Import fixtures, perform representative transactions and independently reconcile stock, cash/bank, dues, sales, COGS, gross profit, expenses/refunds and savings. Reuse the earlier approved golden financial scenario where applicable; its final checks include cash ₹8,624, product gross profit ₹56, combined recorded operating result -₹114 and customer due ₹50. Those values apply only when its exact preconditions/actions are followed.

Backup and restore test matrix

| ID | Test | Required result |

|---|---|---|

| B01 | Audit every persistent store against coverage inventory | All required stores covered; exclusions justified |

| B02 | Back Up Everything with populated fixture | Recovery ZIP and workbook share snapshot identity and totals |

| B03 | Committed WAL data and concurrent save during backup | Consistent snapshot; summary/workbook/archive do not disagree |

| B04 | Replace/remove photo during snapshot capture | Referenced contents consistent; no dangling file reference |

| B05 | Open workbook independently | Numeric amounts, dates, IDs, text, sheet counts and summaries correct |

| B06 | Save outer package through actual Files/Drive provider | Valid package retained; actual provider write verified; no false cloud claim |

| B07 | Retrieve saved package and restore elsewhere | App accepts package without manual extraction; full comparison passes |

| B08 | Restore standalone complete ZIP | Same correct outcome as outer package |

| B09 | Compare restored records | IDs/fields/links/amounts/statuses match, not just row counts |

| B10 | Open restored photos/logo on separate installation | Contents valid without access to original phone paths |

| B11 | Restore/restart and inspect linked finances | No replayed sales/payments/stock/savings effects |

| B12 | Change setting/theme before backup and restore | Relevant portable settings retained; device-specific secrets handled safely |

| B13 | Excel generation failure after recovery ZIP success | Recovery file retained; partial result honestly reported and retry available |

| B14 | Provider cancelled/offline/missing/full/revoked | Local outputs intact; failed save does not update successful-save timestamp |

| B15 | Truncated ZIP, missing photo/DB, wrong hash, newer schema | Clear staged rejection; existing database untouched/recoverable |

| B16 | Unsafe paths, excessive nested/compressed archive, multiple backup candidates | Safe rejection without unexpected writes or guessing |

| B17 | Interruption/storage exhaustion during create/restore | No false success; old data and valid prior files recoverable |

| B18 | Two dated backups and later failed attempt | Previous good files/timestamps retained |

| B19 | Owner backup with offline Billing phone | Pending/unknown coverage disclosed; no claim of complete unseen records |

| B20 | Export/share permissions and sensitive content | Authorized user only; no signing secrets/private device paths leaked |

Navigation and visual tests

| ID | Test | Required result |

|---|---|---|

| U01 | Tap Home Bill History with zero bills and populated data | Destination opens and shows appropriate content |

| U02 | History detail -> customer account -> Back | Correct identity/stack; no new financial record |

| U03 | Locked/warm/cold app and repeat tap | Correct permission-aware route; no crash or dead button |

| U04 | Settings gear after shortcut fix | Settings still accessible |

| U05 | Customer/supplier summary with long text and large amounts | No overlap/clipping; distinct action row |

| U06 | Search with keyboard and enlarged fonts | Search/filters/list remain usable; form not obscured |

| U07 | Light-mode screen/dialog/state audit | Recorded contrast targets met; readable values and controls |

| U08 | Switch light/dark, restart and revisit | Theme persists as expected; dark mode remains readable |

| U09 | Empty/search-empty/zero-due/populated Accounts | Correct different explanations and amounts/counts |

| U10 | Widget, billing, reports, sharing and backup smoke regression | Existing entry points and behavior preserved |

For each test report build/device, exact steps, expected/actual values, evidence and PASS/FAIL/BLOCKED/PARTIAL/NOT RUN. Fix failures and rerun related scenarios, including financial reconciliation after changes to shared data paths. A successful ZIP creation/build or a screenshot alone is not acceptance.

12. Build now; update the intended phone when connected

Build immediately after implementation and available verification

- Inspect the current project/build configuration and installed-version evidence if available. The supplied archive's `.debug` package label is a compatibility clue, not permission to switch package IDs or signing identities.

- Produce a candidate APK with correct app ID, compatible signing identity and an appropriate incremented version code. Do not include synthetic records, developer secrets or test-only destructive actions.

- Run available automated/emulator/device-independent checks now. Clearly list any hardware/provider checks that require the later connection; do not mark them passed.

- Report APK absolute path, filename, version name/code, size, SHA-256, source revision/changes and test status. Keep the exact tested artifact identifiable.

- Do not postpone all work until the phone connects. Finish implementation, build and all possible isolated verification first.

Update when the user connects the phone for this task

The user has requested this follow-on update. Proceed when the intended phone is connected for it and all prerequisites below pass; do not repeatedly ask for generic permission already given. A random connected device is not enough evidence of intended target identity. If multiple devices or an unexpected app/package appear, ask a targeted clarification.

1. Identify the intended device and currently installed app ID/version/signing compatibility. The phone may need the user to enable/accept normal USB debugging authorization.

2. Verify a fresh complete pre-update recovery backup can be preserved off-device. Do not count an untested empty archive as a safety copy. Preserve existing recovery files.

3. Confirm supported nondestructive migration and review required test results. If critical financial/data-loss/security/recovery failures remain, stop installation and explain the blocker.

4. Install the compatible APK **as an update**, preserving app data. Never uninstall, clear app storage, force destructive migration or install a different package and pretend the original app was updated.

5. If signature/package/version incompatibility blocks update, report it and resolve the legitimate compatible build path. Do not bypass it with data deletion.

6. Run nondestructive post-update checks: launch/unlock, Home Bill History, existing bill/customer/supplier records, light/dark readability, Settings and backup list. Compare existing counts/balances to the pre-update baseline where available.

7. Test transactional writes using disposable data/app isolation, not real transactions on the live shop database. Do not restore test data over the live phone to demonstrate success.

8. Report installed version, checks passed and any remaining unverified device/provider features.

If connection is not available in the current session, deliver the built candidate and exact pending checklist. Do not claim to monitor future connections or auto-update later unless an actual supported monitoring mechanism has been explicitly established. Resume these steps when the user returns with the connected phone.

An older APK is not automatically a safe rollback after schema migration. Preserve a recovery plan that accounts for data created since the backup; never silently replace current data with an older snapshot.

13. Required completion handoff

Deliver implementation summary, changed files, before/after Accounts and light-mode screenshots, working Home Bill History evidence, fixture files/expected manifest, sample package/workbook, complete backup coverage inventory, actual restore comparison and test matrix.

State separately: implemented, built, automatically tested, device tested, provider save tested, restore tested and installed. Never collapse these into “done” if some stages remain pending.

Do not call the app final with unexplained record omissions, missing attachments, incorrect financial totals, unreadable critical controls, broken navigation or failed recovery. Fix and retest before release. This handoff authorizes preparing the implementation/build and the conditional in-place phone update described above, not destructive operations on shop data.

14. Latest addition: correct the widget to match the approved slim-bar reference

Reference assets and selected design

- Approved concept board: `output/widget-reference/Ali_Educational_Widget_Reference.png`, supplied alongside this document.

- Use ONLY the **middle concept, labelled 2. Slim bar**. The left and right concepts are not selected.

- Actual current-phone screenshot: `C:/Users/Musid/Downloads/WhatsApp Image 2026-11-01 at 8.46.27 PM.jpeg`.

- The concept is a visual target, not a literal phone wallpaper, launcher layout or source-code asset. Do not implement the surrounding phone frame, wallpaper, app icons, presentation heading or labels outside the widget.

The user reports that the implemented widget does not look like the selected design. The actual screenshot has the four correct actions, but very small thin icons, smaller labels, a small gold title, no reference-style header separator and different icon shapes. Preserve the working actions while correcting these visible differences.

Required visual changes

1. Retain one compact wide rounded charcoal/deep-green widget and a single horizontal row of four equal action areas.

2. Use a more prominent, readable light/white **Ali Educational** heading aligned to the left with consistent internal padding, matching the middle reference. Do not keep the tiny gold heading as the final design.

3. Add the subtle horizontal divider between the heading and actions, with comfortable spacing. It must not intersect labels or icons.

4. Replace the small diagonal Cash In/Out arrows with clearly visible upright incoming/down and outgoing/up arrows over a short baseline, like the reference. Cash In is green; Cash Out is orange.

5. Use a receipt-with-plus icon for New Bill and a box-with-plus icon for Add Stock. A plain small square plus does not match the Add Stock reference.

6. Enlarge and visually balance all four icons. Approximately 28-32 dp is an initial implementation target; verify at the actual launcher size rather than treating reference-image pixels as Android dp.

7. Increase label readability using medium/bold light text, with consistent icon-to-label spacing. Approximately 13-14 sp is a starting point, subject to actual fit and accessibility testing.

8. Preserve exact labels: Cash In, Cash Out, New Bill, Add Stock. Keep labels on one line at supported dimensions and prevent clipped or split words.

9. Use equal-width columns, aligned icon centers and label baselines. Every whole action area is tappable, not just its small icon. Maintain at least 48 dp touch targets.

10. Use subtle separators between actions if needed to match the reference, without making the bar visually busy. Preserve adequate spacing from the widget border.

11. Keep the outer shape rounded and compact, but do not force an excessively short height that shrinks the heading/actions. Configure sensible minimum/resize dimensions for supported launchers.

12. Use bundled vector/drawable assets, not Unicode/emoji approximations whose appearance varies by phone. Do not ship a flattened screenshot as a noninteractive widget.

The goal is a faithful practical Android rendering, not a promise of identical pixels across different launchers. Compare proportions, icon meaning/weight, text hierarchy, spacing and colors at matched displayed widths. If a platform constraint prevents an element, document the exact constraint and show the closest working result rather than claiming an exact match.

Preserve behavior and security

- Cash In opens the incoming form; Cash Out opens the outgoing form; New Bill uses the existing draft-safe billing flow; Add Stock opens existing stock receiving.

- No tap automatically saves data. Retain lock/unlock routing and existing role checks.

- Preserve existing unfinished bills/forms; repeated taps must not stack duplicate financial actions.

- Keep multiple widget instances and pending action identities distinct. A visual change must not swap shortcut destinations.

- Do not add balances, extra features or a second selected design.

Verification and evidence

| ID | Test | Expected |

|---|---|---|

| V01 | Compare actual widget with middle reference at matched widths | Readable light heading, separator, four balanced icons and aligned labels |

| V02 | Check icon semantics | Down/in green, up/out orange, receipt-plus and box-plus; no tiny diagonal/generic-square substitutes |

| V03 | Supported launcher resize/font/display sizes | Compact single row remains readable; minimum sizing prevents clipping |

| V04 | Tap full area of each action from cold/warm/locked state | Correct existing form after required unlock; no automatic posting |

| V05 | Existing draft and rapid taps | Input preserved; no duplicated navigation/save |

| V06 | Update an existing widget instance | New layout appears after supported update/refresh; no need to clear app data |

| V07 | Fresh widget instance and multiple instances | Same correct layout/actions without stale routing |

| V08 | Actual phone screenshot after compatible update | Visual comparison and remaining launcher differences documented |

Provide a before/after widget screenshot and a side-by-side comparison with the **middle reference only**. Verify real tappable controls, not merely a static preview. Build this correction into the same candidate as Sections 1-13; follow the same build-now and later connected-phone update instructions.

Part V - Complete text-source appendix

Paths are relative to the source project. Line numbers refer to the original file; continuation lines are indented. Text is reproduced for inspection, including source comments whose claims are not independently certified.

app\src\main\AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <uses-permission android:name="android.permission.USE_BIOMETRIC" />
    <uses-permission android:name="android.permission.CAMERA" />
    <uses-permission android:name="android.permission.INTERNET" />
    <uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
    <uses-permission android:name="android.permission.ACCESS_WIFI_STATE" />
    <uses-permission android:name="android.permission.CHANGE_WIFI_MULTICAST_STATE" />

    <uses-feature
        android:name="android.hardware.camera"
        android:required="false" />
    <uses-feature
        android:name="android.hardware.camera.autofocus"
        android:required="false" />

    <application
        android:usesCleartextTraffic="true"
        android:name=".AliEducationalApp"
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="false"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.AliEducational"
        tools:targetApi="36">

        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:launchMode="singleTask"
            android:theme="@style/Theme.AliEducational">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>

        <!-- Slim-Bar Home Screen Widget -->
        <receiver
            android:name=".widget.AliSlimBarWidgetProvider"
            android:exported="true"
            android:label="@string/widget_slim_bar_name">
            <intent-filter>
                <action android:name="android.appwidget.action.APPWIDGET_UPDATE" />
            </intent-filter>
            <meta-data
                android:name="android.appwidget.provider"
                android:resource="@xml/ali_slim_widget_info" />
        </receiver>

        <provider
            android:name="androidx.core.content.FileProvider"
            android:authorities="${applicationId}.fileprovider"
            android:exported="false"
            android:grantUriPermissions="true">
            <meta-data
                android:name="android.support.FILE_PROVIDER_PATHS"
                android:resource="@xml/file_paths" />
        </provider>
    </application>

</manifest>
```

app\src\main\java\com\alieducational\app\AliEducationalApp.kt

```kotlin
package com.alieducational.app

import android.app.Application
import dagger.hilt.android.HiltAndroidApp

@HiltAndroidApp
class AliEducationalApp : Application() {
    override fun onCreate() {
        super.onCreate()
        System.setProperty("org.apache.poi.util.POILogger", "org.apache.poi.util.NullLogger")
    }
}
```

app\src\main\java\com\alieducational\app\data\local\AppDatabase.kt

```
package com.alieducational.app.data.local

import androidx.room.Database
import androidx.room.RoomDatabase
import androidx.room.migration.Migration
import androidx.sqlite.db.SupportSQLiteDatabase
import com.alieducational.app.data.local.dao.AccountPaymentDao
import com.alieducational.app.data.local.dao.CustomerDao
import com.alieducational.app.data.local.dao.HeldBillDao
import com.alieducational.app.data.local.dao.ImportBatchDao
import com.alieducational.app.data.local.dao.PigmyDepositDao
import com.alieducational.app.data.local.dao.CashCategoryDao
import com.alieducational.app.data.local.dao.PigmySchemeDao
import com.alieducational.app.data.local.dao.PigmyWithdrawalDao
import com.alieducational.app.data.local.entity.CashCategoryEntity
import com.alieducational.app.data.local.entity.PigmyWithdrawalEntity
import com.alieducational.app.data.local.dao.ProductBarcodeDao
import com.alieducational.app.data.local.dao.ProductDao
import com.alieducational.app.data.local.dao.ProductGiveawayDao
import com.alieducational.app.data.local.dao.PurchaseDao
import com.alieducational.app.data.local.dao.SaleDao
import com.alieducational.app.data.local.dao.ServiceDefinitionDao
import com.alieducational.app.data.local.dao.ShopExpenseDao
import com.alieducational.app.data.local.dao.StockMovementDao
import com.alieducational.app.data.local.dao.SupplierDao
import com.alieducational.app.data.local.dao.SyncEventDao
import com.alieducational.app.data.local.entity.AccountPaymentEntity
import com.alieducational.app.data.local.entity.CustomerEntity
import com.alieducational.app.data.local.entity.HeldBillEntity
import com.alieducational.app.data.local.entity.ImportBatchEntity
import com.alieducational.app.data.local.entity.PigmyDepositEntity
import com.alieducational.app.data.local.entity.PigmySchemeEntity
import com.alieducational.app.data.local.entity.ProductBarcodeEntity
import com.alieducational.app.data.local.entity.ProductEntity
import com.alieducational.app.data.local.entity.ProductGiveawayEntity
import com.alieducational.app.data.local.entity.PurchaseEntity
import com.alieducational.app.data.local.entity.PurchaseLineEntity
import com.alieducational.app.data.local.entity.SaleEntity
import com.alieducational.app.data.local.entity.SaleLineEntity
import com.alieducational.app.data.local.entity.ServiceDefinitionEntity
import com.alieducational.app.data.local.entity.ShopExpenseEntity
import com.alieducational.app.data.local.entity.StockMovementEntity
import com.alieducational.app.data.local.entity.SupplierEntity
import com.alieducational.app.data.local.entity.SyncEventEntity

@Database(
    entities = [
        ProductEntity::class,
        ProductBarcodeEntity::class,
        StockMovementEntity::class,
        SupplierEntity::class,
        CustomerEntity::class,
        PurchaseEntity::class,
        PurchaseLineEntity::class,
        SaleEntity::class,
        SaleLineEntity::class,
        AccountPaymentEntity::class,
        ShopExpenseEntity::class,
        CashCategoryEntity::class,
        ImportBatchEntity::class,
        HeldBillEntity::class,
        PigmyDepositEntity::class,
        PigmyWithdrawalEntity::class,
        PigmySchemeEntity::class,
        ProductGiveawayEntity::class,
        ServiceDefinitionEntity::class,
        SyncEventEntity::class
    ],
    version = 8,
    exportSchema = true
)
abstract class AppDatabase : RoomDatabase() {
    abstract fun productDao(): ProductDao
    abstract fun productBarcodeDao(): ProductBarcodeDao
    abstract fun stockMovementDao(): StockMovementDao
    abstract fun supplierDao(): SupplierDao
    abstract fun customerDao(): CustomerDao
    abstract fun purchaseDao(): PurchaseDao
    abstract fun saleDao(): SaleDao
    abstract fun accountPaymentDao(): AccountPaymentDao
    abstract fun shopExpenseDao(): ShopExpenseDao
    abstract fun importBatchDao(): ImportBatchDao
```

```
    abstract fun heldBillDao(): HeldBillDao
    abstract fun pigmyDepositDao(): PigmyDepositDao
    abstract fun pigmyWithdrawalDao(): PigmyWithdrawalDao
    abstract fun pigmySchemeDao(): PigmySchemeDao
    abstract fun cashCategoryDao(): CashCategoryDao
    abstract fun productGiveawayDao(): ProductGiveawayDao
    abstract fun serviceDefinitionDao(): ServiceDefinitionDao
    abstract fun syncEventDao(): SyncEventDao

    companion object {
        val MIGRATION_1_2 = object : Migration(1, 2) {
            override fun migrate(db: SupportSQLiteDatabase) {
                db.execSQL("ALTER TABLE products ADD COLUMN isFavorite INTEGER NOT NULL DEFAULT 0")
                db.execSQL("ALTER TABLE products ADD COLUMN barcode TEXT DEFAULT NULL")
                db.execSQL("ALTER TABLE sales ADD COLUMN changeGivenPaise INTEGER NOT NULL DEFAULT 0")
                db.execSQL("""
                    CREATE TABLE IF NOT EXISTS product_barcodes (
                        id TEXT NOT NULL PRIMARY KEY,
                        barcode TEXT NOT NULL,
                        productId TEXT NOT NULL,
                        unitType TEXT NOT NULL DEFAULT 'PIECE',
                        packQuantity INTEGER NOT NULL DEFAULT 1,
                        createdAt INTEGER NOT NULL
                    )
                """.trimIndent())
                db.execSQL("CREATE UNIQUE INDEX IF NOT EXISTS index_product_barcodes_barcode ON product_barcodes(barcode)")
                db.execSQL("""
                    CREATE TABLE IF NOT EXISTS held_bills (
                        id TEXT NOT NULL PRIMARY KEY,
                        note TEXT,
                        customerId TEXT,
                        customerName TEXT,
                        discountType TEXT NOT NULL DEFAULT 'NONE',
                        discountValuePaise INTEGER NOT NULL DEFAULT 0,
                        discountPercent REAL NOT NULL DEFAULT 0.0,
                        draftJson TEXT NOT NULL,
                        itemCount INTEGER NOT NULL DEFAULT 0,
                        totalPaise INTEGER NOT NULL DEFAULT 0,
                        createdAt INTEGER NOT NULL
                    )
                """.trimIndent())
            }
        }

        val MIGRATION_2_3 = object : Migration(2, 3) {
            override fun migrate(db: SupportSQLiteDatabase) {
                db.execSQL("ALTER TABLE sales ADD COLUMN isVoided INTEGER NOT NULL DEFAULT 0")
                db.execSQL("ALTER TABLE sales ADD COLUMN voidReason TEXT DEFAULT NULL")
                db.execSQL("ALTER TABLE sales ADD COLUMN voidTimestamp INTEGER DEFAULT NULL")
                db.execSQL("ALTER TABLE sales ADD COLUMN refundPayoutPaise INTEGER NOT NULL DEFAULT 0")
                db.execSQL("ALTER TABLE sales ADD COLUMN stockRestoredCondition TEXT DEFAULT NULL")
                db.execSQL("ALTER TABLE shop_expenses ADD COLUMN isVoided INTEGER NOT NULL DEFAULT 0")
                db.execSQL("ALTER TABLE shop_expenses ADD COLUMN voidReason TEXT DEFAULT NULL")
            }
        }

        val MIGRATION_3_4 = object : Migration(3, 4) {
            override fun migrate(db: SupportSQLiteDatabase) {
                db.execSQL("ALTER TABLE products ADD COLUMN wholesalePricePiecePaise INTEGER NOT NULL DEFAULT 0")
            }
        }

        val MIGRATION_4_5 = object : Migration(4, 5) {
            override fun migrate(db: SupportSQLiteDatabase) {
                // 1. Alter sales
                db.execSQL("ALTER TABLE sales ADD COLUMN originDeviceId TEXT DEFAULT NULL")
                db.execSQL("ALTER TABLE sales ADD COLUMN syncStatus TEXT NOT NULL DEFAULT 'LOCAL'")

                // 2. Alter sale_lines
                db.execSQL("ALTER TABLE sale_lines ADD COLUMN lineType TEXT NOT NULL DEFAULT 'PRODUCT'")
                db.execSQL("ALTER TABLE sale_lines ADD COLUMN serviceId TEXT DEFAULT NULL")
                db.execSQL("ALTER TABLE sale_lines ADD COLUMN serviceName TEXT DEFAULT NULL")
                db.execSQL("ALTER TABLE sale_lines ADD COLUMN serviceOption TEXT DEFAULT NULL")
                db.execSQL("ALTER TABLE sale_lines ADD COLUMN unitLabel TEXT DEFAULT NULL")

                // 3. Alter shop_expenses
                db.execSQL("ALTER TABLE shop_expenses ADD COLUMN expenseCategory TEXT DEFAULT NULL")
                db.execSQL("ALTER TABLE shop_expenses ADD COLUMN serviceTarget TEXT DEFAULT NULL")
                db.execSQL("ALTER TABLE shop_expenses ADD COLUMN attachmentUri TEXT DEFAULT NULL")
                db.execSQL("ALTER TABLE shop_expenses ADD COLUMN isSavingsTransfer INTEGER NOT NULL DEFAULT 0")

                // 4. Create pigmy_deposits table
                db.execSQL("""
                    CREATE TABLE IF NOT EXISTS `pigmy_deposits` (
                        `id` TEXT NOT NULL,
```

```
                        `accountSchemeId` TEXT NOT NULL,
                        `paidTimestamp` INTEGER NOT NULL,
                        `amountPaise` INTEGER NOT NULL,
                        `paymentSource` TEXT NOT NULL,
                        `notes` TEXT,
                        `linkedExpenseId` TEXT,
                        `isVoided` INTEGER NOT NULL,
                        `voidReason` TEXT,
                        `createdAt` INTEGER NOT NULL,
                        `updatedAt` INTEGER NOT NULL,
                        PRIMARY KEY(`id`)
                    )
                """.trimIndent())

                // 5. Create product_giveaways table
                db.execSQL("""
                    CREATE TABLE IF NOT EXISTS `product_giveaways` (
                        `id` TEXT NOT NULL,
                        `productId` TEXT NOT NULL,
                        `productName` TEXT NOT NULL,
                        `quantityBase` INTEGER NOT NULL,
                        `isPack` INTEGER NOT NULL,
                        `packSizeSnapshot` INTEGER NOT NULL,
                        `unitCostSnapshotPaise` INTEGER,
                        `reason` TEXT NOT NULL,
                        `recipientNote` TEXT,
                        `timestamp` INTEGER NOT NULL,
                        `isCancelled` INTEGER NOT NULL,
                        `cancelledTimestamp` INTEGER,
                        PRIMARY KEY(`id`)
                    )
                """.trimIndent())

                // 6. Create service_definitions table
                db.execSQL("""
                    CREATE TABLE IF NOT EXISTS `service_definitions` (
                        `id` TEXT NOT NULL,
                        `name` TEXT NOT NULL,
                        `unitLabel` TEXT NOT NULL,
                        `isPredefined` INTEGER NOT NULL,
                        `isArchived` INTEGER NOT NULL,
                        `createdAt` INTEGER NOT NULL,
                        PRIMARY KEY(`id`)
                    )
                """.trimIndent())

                // 7. Create sync_events table
                db.execSQL("""
                    CREATE TABLE IF NOT EXISTS `sync_events` (
                        `id` TEXT NOT NULL,
                        `eventType` TEXT NOT NULL,
                        `entityId` TEXT NOT NULL,
                        `payloadJson` TEXT NOT NULL,
                        `originDeviceId` TEXT NOT NULL,
                        `status` TEXT NOT NULL,
                        `retryCount` INTEGER NOT NULL,
                        `timestamp` INTEGER NOT NULL,
                        PRIMARY KEY(`id`)
                    )
                """.trimIndent())
            }
        }

        val MIGRATION_5_6 = object : Migration(5, 6) {
            override fun migrate(db: SupportSQLiteDatabase) {
                db.execSQL("""
                    CREATE TABLE IF NOT EXISTS `pigmy_schemes` (
                        `id` TEXT NOT NULL,
                        `name` TEXT NOT NULL,
                        `usualDailyAmountPaise` INTEGER NOT NULL,
                        `isArchived` INTEGER NOT NULL DEFAULT 0,
                        `createdAt` INTEGER NOT NULL,
                        `updatedAt` INTEGER NOT NULL,
                        PRIMARY KEY(`id`)
                    )
                """.trimIndent())

                val now = System.currentTimeMillis()
                db.execSQL("""
                    INSERT OR IGNORE INTO `pigmy_schemes` (`id`, `name`, `usualDailyAmountPaise`, `isArchived`, `createdAt`, `updatedAt`)
                    VALUES
                        ('DEFAULT_PIGMY', 'Pigmy Scheme 1', 10000, 0, $now, $now),
                        ('scheme_2', 'Pigmy Scheme 2', 20000, 0, $now + 1, $now + 1),
                        ('scheme_3', 'Pigmy Scheme 3', 30000, 0, $now + 2, $now + 2)
                """.trimIndent())
            }
        }
```

```
255
256         val MIGRATION_6_7 = object : Migration(6, 7) {
257             override fun migrate(db: SupportSQLiteDatabase) {
258                 // 1. Create cash_categories table
259                 db.execSQL("""
260                     CREATE TABLE IF NOT EXISTS `cash_categories` (
261                         `id` TEXT NOT NULL,
262                         `name` TEXT NOT NULL,
263                         `isSystem` INTEGER NOT NULL DEFAULT 0,
264                         `isArchived` INTEGER NOT NULL DEFAULT 0,
265                         `createdAt` INTEGER NOT NULL,
266                         `updatedAt` INTEGER NOT NULL,
267                         PRIMARY KEY(`id`)
268                     )
269                 """.trimIndent())
270
271                 // 2. Populate default categories
272                 val now = System.currentTimeMillis()
273                 db.execSQL("""
274                     INSERT OR IGNORE INTO `cash_categories` (`id`, `name`, `isSystem`, `isArchived`,
    `createdAt`, `updatedAt`)
275                     VALUES
276                         ('cat_general', 'General', 1, 0, $now, $now),
277                         ('cat_pigmy', 'Pigmy', 1, 0, $now + 1, $now + 1),
278                         ('cat_rent', 'Rent', 0, 0, $now + 2, $now + 2),
279                         ('cat_electricity', 'Electricity', 0, 0, $now + 3, $now + 3),
280                         ('cat_transport', 'Transport', 0, 0, $now + 4, $now + 4),
281                         ('cat_salary', 'Salary', 0, 0, $now + 5, $now + 5),
282                         ('cat_tea_snacks', 'Tea & Snacks', 0, 0, $now + 6, $now + 6)
283                 """.trimIndent())
284
285                 // 3. Alter shop_expenses to add categoryId
286                 db.execSQL("ALTER TABLE shop_expenses ADD COLUMN categoryId TEXT DEFAULT NULL")
287
288                 // 4. Create pigmy_withdrawals table
289                 db.execSQL("""
290                     CREATE TABLE IF NOT EXISTS `pigmy_withdrawals` (
291                         `id` TEXT NOT NULL,
292                         `accountSchemeId` TEXT NOT NULL,
293                         `withdrawalTimestamp` INTEGER NOT NULL,
294                         `amountPaise` INTEGER NOT NULL,
295                         `destination` TEXT NOT NULL,
296                         `notes` TEXT,
297                         `linkedExpenseId` TEXT,
298                         `isVoided` INTEGER NOT NULL DEFAULT 0,
299                         `voidReason` TEXT,
300                         `createdAt` INTEGER NOT NULL,
301                         `updatedAt` INTEGER NOT NULL,
302                         PRIMARY KEY(`id`)
303                     )
304                 """.trimIndent())
305
306                 // 5. Alter pigmy_schemes to add openingBalancePaise and openingBalanceDate
307                 db.execSQL("ALTER TABLE pigmy_schemes ADD COLUMN openingBalancePaise INTEGER NOT NULL
    DEFAULT 0")
308                 db.execSQL("ALTER TABLE pigmy_schemes ADD COLUMN openingBalanceDate INTEGER DEFAULT NULL")
309             }
310         }
311
312         val MIGRATION_7_8 = object : Migration(7, 8) {
313             override fun migrate(db: SupportSQLiteDatabase) {
314                 db.execSQL("ALTER TABLE shop_expenses ADD COLUMN isExpenseRefund INTEGER NOT NULL DEFAULT
    0")
315                 db.execSQL("ALTER TABLE shop_expenses ADD COLUMN linkedExpenseId TEXT DEFAULT NULL")
316                 db.execSQL("ALTER TABLE shop_expenses ADD COLUMN classification TEXT NOT NULL DEFAULT
    'SHARED'")
317                 db.execSQL("ALTER TABLE shop_expenses ADD COLUMN paymentMethod TEXT NOT NULL DEFAULT
    'CASH'")
318                 db.execSQL("UPDATE shop_expenses SET classification = 'SERVICES' WHERE category =
    'SERVICE_EXPENSE' OR serviceTarget IS NOT NULL")
319             }
320         }
321
322     }
323 }
```

app\src\main\java\com\alieducational\app\data\local\dao\Daos.kt

```
package com.alieducational.app.data.local.dao

import androidx.room.Dao
import androidx.room.Insert
import androidx.room.OnConflictStrategy
import androidx.room.Query
import androidx.room.Transaction
import androidx.room.Update
import com.alieducational.app.data.local.entity.AccountPaymentEntity
import com.alieducational.app.data.local.entity.CashCategoryEntity
import com.alieducational.app.data.local.entity.PigmyWithdrawalEntity
import com.alieducational.app.data.local.entity.CustomerEntity
import com.alieducational.app.data.local.entity.HeldBillEntity
import com.alieducational.app.data.local.entity.ImportBatchEntity
import com.alieducational.app.data.local.entity.PigmyDepositEntity
import com.alieducational.app.data.local.entity.PigmySchemeEntity
import com.alieducational.app.data.local.entity.ProductBarcodeEntity
import com.alieducational.app.data.local.entity.ProductEntity
import com.alieducational.app.data.local.entity.ProductGiveawayEntity
import com.alieducational.app.data.local.entity.PurchaseEntity
import com.alieducational.app.data.local.entity.PurchaseLineEntity
import com.alieducational.app.data.local.entity.SaleEntity
import com.alieducational.app.data.local.entity.SaleLineEntity
import com.alieducational.app.data.local.entity.ServiceDefinitionEntity
import com.alieducational.app.data.local.entity.ShopExpenseEntity
import com.alieducational.app.data.local.entity.StockMovementEntity
import com.alieducational.app.data.local.entity.SupplierEntity
import com.alieducational.app.data.local.entity.SyncEventEntity
import kotlinx.coroutines.flow.Flow

@Dao
interface ProductDao {
    @Query("SELECT * FROM products ORDER BY name ASC")
    fun getAllProducts(): Flow<List<ProductEntity>>

    @Query("SELECT * FROM products WHERE isArchived = 0 ORDER BY name ASC")
    fun getAllActiveProducts(): Flow<List<ProductEntity>>

    @Query("SELECT * FROM products WHERE isArchived = 0 AND (name LIKE '%' || :query || '%' OR sku LIKE '%' || :query || '%' OR barcode LIKE '%' || :query || '%' OR brand LIKE '%' || :query || '%' OR category LIKE '%' || :query || '%') ORDER BY name ASC")
    fun searchProducts(query: String): Flow<List<ProductEntity>>

    @Query("SELECT * FROM products WHERE isArchived = 0 AND currentSellableStock <= lowStockThreshold ORDER BY currentSellableStock ASC")
    fun getLowStockProducts(): Flow<List<ProductEntity>>

    @Query("SELECT * FROM products WHERE id = :id")
    suspend fun getProductById(id: String): ProductEntity?

    @Query("SELECT * FROM products WHERE sku = :sku AND isArchived = 0 LIMIT 1")
    suspend fun getProductBySku(sku: String): ProductEntity?

    @Query("SELECT * FROM products WHERE barcode = :barcode AND isArchived = 0 LIMIT 1")
    suspend fun getProductByBarcode(barcode: String): ProductEntity?

    @Query("SELECT * FROM products WHERE LOWER(name) = LOWER(:name) AND isArchived = 0 LIMIT 1")
    suspend fun getProductByName(name: String): ProductEntity?

    @Query("SELECT * FROM products WHERE isArchived = 0 AND isFavorite = 1 ORDER BY name ASC")
    fun getFavoriteProducts(): Flow<List<ProductEntity>>

    @Query("UPDATE products SET isFavorite = :isFav WHERE id = :productId")
    suspend fun setFavorite(productId: String, isFav: Boolean)

    @Query("SELECT DISTINCT category FROM products WHERE isArchived = 0 ORDER BY category ASC")
    fun getAllCategories(): Flow<List<String>>

    @Query("SELECT * FROM products WHERE isArchived = 0 AND category = :category ORDER BY name ASC")
    fun getProductsByCategory(category: String): Flow<List<ProductEntity>>

    @Query("SELECT SUM(currentSellableStock * movingAvgCostPaise) FROM products WHERE isArchived = 0")
    fun getTotalStockValuePaise(): Flow<Long?>

    @Query("SELECT COUNT(*) FROM products WHERE isArchived = 0 AND currentSellableStock <= lowStockThreshold")
    fun getLowStockCount(): Flow<Int>

    @Query("SELECT COUNT(*) FROM products WHERE isArchived = 0")
    fun getTotalProductCount(): Flow<Int>

    @Insert(onConflict = OnConflictStrategy.REPLACE)
```

```
    suspend fun insertProduct(product: ProductEntity)

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertProducts(products: List<ProductEntity>)

    @Update
    suspend fun updateProduct(product: ProductEntity)

    @Query("UPDATE products SET currentSellableStock = currentSellableStock + :changeBase, updatedAt = :timestamp WHERE id = :productId")
    suspend fun adjustSellableStock(productId: String, changeBase: Int, timestamp: Long = System.currentTimeMillis())

    @Query("UPDATE products SET currentSellableStock = currentSellableStock - :qty, currentDamagedStock = currentDamagedStock + :qty, updatedAt = :timestamp WHERE id = :productId")
    suspend fun moveToDamagedStock(productId: String, qty: Int, timestamp: Long = System.currentTimeMillis())

    @Query("UPDATE products SET movingAvgCostPaise = :newCostPaise, updatedAt = :timestamp WHERE id = :productId")
    suspend fun updateWeightedAverageCost(productId: String, newCostPaise: Long, timestamp: Long = System.currentTimeMillis())

    @Query("UPDATE products SET isArchived = 1, updatedAt = :timestamp WHERE id = :productId")
    suspend fun archiveProduct(productId: String, timestamp: Long = System.currentTimeMillis())
}

@Dao
interface StockMovementDao {
    @Query("SELECT * FROM stock_movements ORDER BY timestamp DESC")
    fun getAllMovements(): Flow<List<StockMovementEntity>>

    @Query("SELECT * FROM stock_movements WHERE productId = :productId ORDER BY timestamp DESC")
    fun getMovementsForProduct(productId: String): Flow<List<StockMovementEntity>>

    @Query("SELECT * FROM stock_movements ORDER BY timestamp DESC LIMIT 50")
    fun getRecentMovements(): Flow<List<StockMovementEntity>>

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertMovement(movement: StockMovementEntity)

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertMovements(movements: List<StockMovementEntity>)
}

@Dao
interface SupplierDao {
    @Query("SELECT * FROM suppliers WHERE isArchived = 0 ORDER BY name ASC")
    fun getAllSuppliers(): Flow<List<SupplierEntity>>

    @Query("SELECT * FROM suppliers WHERE id = :id")
    suspend fun getSupplierById(id: String): SupplierEntity?

    @Query("SELECT * FROM suppliers WHERE LOWER(name) = LOWER(:name) AND isArchived = 0 LIMIT 1")
    suspend fun getSupplierByName(name: String): SupplierEntity?

    @Query("SELECT SUM(currentBalancePaise) FROM suppliers WHERE isArchived = 0 AND currentBalancePaise > 0")
    fun getTotalWeOweSuppliersPaise(): Flow<Long?>

    @Query("SELECT COUNT(*) FROM suppliers WHERE isArchived = 0 AND currentBalancePaise > 0")
    fun getDistinctCreditorsCount(): Flow<Int>

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertSupplier(supplier: SupplierEntity)

    @Update
    suspend fun updateSupplier(supplier: SupplierEntity)

    @Query("UPDATE suppliers SET currentBalancePaise = currentBalancePaise + :amountChangePaise WHERE id = :supplierId")
    suspend fun adjustBalance(supplierId: String, amountChangePaise: Long)

    @Query("UPDATE suppliers SET isArchived = 1, currentBalancePaise = 0 WHERE id = :supplierId")
    suspend fun archiveSupplier(supplierId: String)

    @Query("DELETE FROM suppliers WHERE id = :supplierId")
    suspend fun deleteSupplier(supplierId: String)
}

@Dao
interface CustomerDao {
    @Query("SELECT * FROM customers WHERE isArchived = 0 ORDER BY name ASC")
    fun getAllCustomers(): Flow<List<CustomerEntity>>

    @Query("SELECT * FROM customers WHERE id = :id")
    suspend fun getCustomerById(id: String): CustomerEntity?

```

```
    @Query("SELECT SUM(currentBalancePaise) FROM customers WHERE isArchived = 0 AND currentBalancePaise > 0")
    fun getTotalCustomersOweUsPaise(): Flow<Long?>

    @Query("SELECT COUNT(*) FROM customers WHERE isArchived = 0 AND currentBalancePaise > 0")
    fun getDistinctDebtorsCount(): Flow<Int>

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertCustomer(customer: CustomerEntity)

    @Update
    suspend fun updateCustomer(customer: CustomerEntity)

    @Query("UPDATE customers SET currentBalancePaise = currentBalancePaise + :amountChangePaise WHERE id = :customerId")
    suspend fun adjustBalance(customerId: String, amountChangePaise: Long)

    @Query("UPDATE customers SET isArchived = 1, currentBalancePaise = 0 WHERE id = :customerId")
    suspend fun archiveCustomer(customerId: String)

    @Query("DELETE FROM customers WHERE id = :customerId")
    suspend fun deleteCustomer(customerId: String)
}

@Dao
interface PurchaseDao {
    @Query("SELECT * FROM purchase_lines")
    fun getAllPurchaseLines(): Flow<List<PurchaseLineEntity>>

    @Query("SELECT * FROM purchases ORDER BY timestamp DESC")
    fun getAllPurchases(): Flow<List<PurchaseEntity>>

    @Query("SELECT * FROM purchases WHERE id = :id")
    suspend fun getPurchaseById(id: String): PurchaseEntity?

    @Query("SELECT * FROM purchases WHERE supplierId = :supplierId ORDER BY timestamp DESC")
    fun getPurchasesForSupplier(supplierId: String): Flow<List<PurchaseEntity>>

    @Query("SELECT * FROM purchase_lines WHERE purchaseId = :purchaseId")
    suspend fun getLinesForPurchase(purchaseId: String): List<PurchaseLineEntity>

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertPurchase(purchase: PurchaseEntity)

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertPurchaseLines(lines: List<PurchaseLineEntity>)
}

@Dao
interface SaleDao {
    @Query("SELECT * FROM sales WHERE isVoided = 0 ORDER BY timestamp DESC")
    fun getAllSales(): Flow<List<SaleEntity>>

    @Query("SELECT * FROM sales ORDER BY timestamp DESC")
    fun getAllSalesWithVoided(): Flow<List<SaleEntity>>

    @Query("SELECT * FROM sales WHERE id = :id")
    suspend fun getSaleById(id: String): SaleEntity?

    @Query("SELECT * FROM sales WHERE customerId = :customerId ORDER BY timestamp DESC")
    fun getSalesForCustomer(customerId: String): Flow<List<SaleEntity>>

    @Query("SELECT * FROM sale_lines WHERE saleId = :saleId")
    suspend fun getLinesForSale(saleId: String): List<SaleLineEntity>

    @Query("SELECT * FROM sale_lines")
    fun getAllSaleLines(): Flow<List<SaleLineEntity>>

    @Query("SELECT * FROM sales WHERE isVoided = 0 AND timestamp >= :startTime AND timestamp <= :endTime ORDER BY timestamp DESC")
    fun getSalesByDateRange(startTime: Long, endTime: Long): Flow<List<SaleEntity>>

    @Query("SELECT SUM(netTotalPaise) FROM sales WHERE isVoided = 0 AND timestamp >= :startTime AND timestamp <= :endTime")
    fun getTotalSalesInRange(startTime: Long, endTime: Long): Flow<Long?>

    @Query("SELECT * FROM sales WHERE unresolvedStockDetails = 1 AND isVoided = 0 ORDER BY timestamp DESC")
    fun getUnresolvedStockSales(): Flow<List<SaleEntity>>

    @Query("SELECT * FROM sale_lines WHERE lineType = 'SERVICE'")
    fun getAllServiceLines(): Flow<List<SaleLineEntity>>

    @Query("SELECT sl.* FROM sale_lines sl INNER JOIN sales s ON sl.saleId = s.id WHERE s.isVoided = 0 AND sl.lineType = 'SERVICE' AND s.timestamp >= :startTime AND s.timestamp <= :endTime")
    fun getServiceLinesInRange(startTime: Long, endTime: Long): Flow<List<SaleLineEntity>>

    @Query("SELECT * FROM sales WHERE syncStatus = 'PENDING_SYNC' ORDER BY timestamp ASC")
    suspend fun getPendingSyncSales(): List<SaleEntity>
```

```
242
243     @Query("SELECT COUNT(*) FROM sales WHERE syncStatus = 'PENDING_SYNC'")
244     fun getPendingSyncSalesCount(): Flow<Int>
245
246     @Query("UPDATE sales SET syncStatus = :status WHERE id = :saleId")
247     suspend fun updateSaleSyncStatus(saleId: String, status: String)
248
249     @Insert(onConflict = OnConflictStrategy.REPLACE)
250     suspend fun insertSale(sale: SaleEntity)
251
252     @Insert(onConflict = OnConflictStrategy.REPLACE)
253     suspend fun insertSaleLines(lines: List<SaleLineEntity>)
254
255     @Query("UPDATE sales SET isSummaryEntry = 0, unresolvedStockDetails = 0 WHERE id = :saleId")
256     suspend fun markStockResolved(saleId: String)
257
258     @Query("UPDATE sales SET isVoided = 1, voidReason = :reason, voidTimestamp = :timestamp,
    refundPayoutPaise = :refundPayout, stockRestoredCondition = :condition WHERE id = :saleId")
259     suspend fun voidSale(saleId: String, reason: String, timestamp: Long, refundPayout: Long, condition:
    String)
260 }
261
262 @Dao
263 interface AccountPaymentDao {
264     @Query("SELECT * FROM account_payments ORDER BY timestamp DESC")
265     fun getAllAccountPayments(): Flow<List<AccountPaymentEntity>>
266
267     @Query("SELECT * FROM account_payments WHERE accountType = :accountType AND accountId = :accountId ORDER
     BY timestamp DESC")
268     fun getPaymentsForAccount(accountType: String, accountId: String): Flow<List<AccountPaymentEntity>>
269
270     @Query("SELECT * FROM account_payments ORDER BY timestamp DESC LIMIT 50")
271     fun getRecentPayments(): Flow<List<AccountPaymentEntity>>
272
273     @Insert(onConflict = OnConflictStrategy.REPLACE)
274     suspend fun insertPayment(payment: AccountPaymentEntity)
275 }
276
277 @Dao
278 interface ShopExpenseDao {
279     @Query("SELECT * FROM shop_expenses WHERE isVoided = 0 ORDER BY timestamp DESC")
280     fun getAllExpenses(): Flow<List<ShopExpenseEntity>>
281
282     @Query("SELECT * FROM shop_expenses WHERE linkedExpenseId = :expenseId AND isVoided = 0")
283     suspend fun getActiveRefundsForExpense(expenseId: String): List<ShopExpenseEntity>
284
285     @Query("SELECT * FROM shop_expenses WHERE isVoided = 0 AND isIncome = 0 AND isSavingsTransfer = 0 ORDER
    BY timestamp DESC")
286     fun getRefundableExpenses(): Flow<List<ShopExpenseEntity>>
287
288     @Query("SELECT * FROM shop_expenses WHERE isVoided = 0 AND timestamp >= :startTime AND timestamp <=
    :endTime ORDER BY timestamp DESC")
289     fun getExpensesInRange(startTime: Long, endTime: Long): Flow<List<ShopExpenseEntity>>
290
291     @Query("SELECT * FROM shop_expenses WHERE isVoided = 0 AND serviceTarget IS NOT NULL AND timestamp >=
    :startTime AND timestamp <= :endTime ORDER BY timestamp DESC")
292     fun getServiceExpensesInRange(startTime: Long, endTime: Long): Flow<List<ShopExpenseEntity>>
293
294     @Query("SELECT * FROM shop_expenses WHERE isVoided = 0 AND isSavingsTransfer = 0 AND timestamp >=
    :startTime AND timestamp <= :endTime ORDER BY timestamp DESC")
295     fun getOperatingExpensesInRange(startTime: Long, endTime: Long): Flow<List<ShopExpenseEntity>>
296
297     @Query("SELECT * FROM shop_expenses WHERE id = :id")
298     suspend fun getExpenseById(id: String): ShopExpenseEntity?
299
300     @Insert(onConflict = OnConflictStrategy.REPLACE)
301     suspend fun insertExpense(expense: ShopExpenseEntity)
302
303     @Insert(onConflict = OnConflictStrategy.REPLACE)
304     suspend fun insertExpenses(expenses: List<ShopExpenseEntity>)
305
306     @Update
307     suspend fun updateExpense(expense: ShopExpenseEntity)
308
309     @Query("UPDATE shop_expenses SET isVoided = 1, voidReason = :reason WHERE id = :expenseId")
310     suspend fun voidExpense(expenseId: String, reason: String)
311 }
312
313 @Dao
314 interface ImportBatchDao {
315     @Query("SELECT * FROM import_batches ORDER BY timestamp DESC")
316     fun getAllBatches(): Flow<List<ImportBatchEntity>>
317
318     @Insert(onConflict = OnConflictStrategy.REPLACE)
319     suspend fun insertBatch(batch: ImportBatchEntity)
320 }
321
322 @Dao
```

```
interface ProductBarcodeDao {
    @Query("SELECT * FROM product_barcodes WHERE barcode = :barcode LIMIT 1")
    suspend fun findByBarcode(barcode: String): ProductBarcodeEntity?

    @Query("SELECT * FROM product_barcodes WHERE productId = :productId")
    suspend fun getBarcodesForProduct(productId: String): List<ProductBarcodeEntity>

    @Insert(onConflict = OnConflictStrategy.ABORT)
    suspend fun insertBarcode(barcode: ProductBarcodeEntity)

    @Query("DELETE FROM product_barcodes WHERE id = :id")
    suspend fun deleteBarcode(id: String)
}

@Dao
interface HeldBillDao {
    @Query("SELECT * FROM held_bills ORDER BY createdAt DESC")
    fun getAllHeldBills(): Flow<List<HeldBillEntity>>

    @Query("SELECT * FROM held_bills WHERE id = :id")
    suspend fun getHeldBillById(id: String): HeldBillEntity?

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertHeldBill(bill: HeldBillEntity)

    @Query("DELETE FROM held_bills WHERE id = :id")
    suspend fun deleteHeldBill(id: String)
}

@Dao
interface PigmyDepositDao {
    @Query("SELECT * FROM pigmy_deposits WHERE accountSchemeId = :schemeId ORDER BY paidTimestamp DESC")
    fun getDepositsForScheme(schemeId: String): Flow<List<PigmyDepositEntity>>

    @Query("SELECT SUM(amountPaise) FROM pigmy_deposits WHERE accountSchemeId = :schemeId AND isVoided = 0")
    fun getTotalForScheme(schemeId: String): Flow<Long?>

    @Query("SELECT * FROM pigmy_deposits ORDER BY paidTimestamp DESC")
    fun getAllDeposits(): Flow<List<PigmyDepositEntity>>

    @Query("SELECT * FROM pigmy_deposits WHERE isVoided = 0 ORDER BY paidTimestamp DESC")
    fun getActiveDeposits(): Flow<List<PigmyDepositEntity>>

    @Query("SELECT * FROM pigmy_deposits WHERE paidTimestamp >= :startTime AND paidTimestamp <= :endTime
ORDER BY paidTimestamp DESC")
    fun getDepositsInRange(startTime: Long, endTime: Long): Flow<List<PigmyDepositEntity>>

    @Query("SELECT * FROM pigmy_deposits WHERE id = :id")
    suspend fun getDepositById(id: String): PigmyDepositEntity?

    @Query("SELECT * FROM pigmy_deposits WHERE linkedExpenseId = :expenseId LIMIT 1")
    suspend fun getDepositByLinkedExpenseId(expenseId: String): PigmyDepositEntity?

    @Query("UPDATE pigmy_deposits SET isVoided = 1, voidReason = :reason, updatedAt = :updatedAt WHERE
linkedExpenseId = :expenseId")
    suspend fun voidByLinkedExpenseId(expenseId: String, reason: String, updatedAt: Long =
System.currentTimeMillis())

    @Query("SELECT SUM(amountPaise) FROM pigmy_deposits WHERE isVoided = 0")
    fun getTotalRecordedDepositsPaise(): Flow<Long?>

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertDeposit(deposit: PigmyDepositEntity)

    @Update
    suspend fun updateDeposit(deposit: PigmyDepositEntity)

    @Query("UPDATE pigmy_deposits SET isVoided = 1, voidReason = :reason, updatedAt = :timestamp WHERE id =
:id")
    suspend fun voidDeposit(id: String, reason: String, timestamp: Long = System.currentTimeMillis())
}

@Dao
interface ProductGiveawayDao {
    @Query("SELECT * FROM product_giveaways ORDER BY timestamp DESC")
    fun getAllGiveaways(): Flow<List<ProductGiveawayEntity>>

    @Query("SELECT * FROM product_giveaways WHERE isCancelled = 0 ORDER BY timestamp DESC")
    fun getActiveGiveaways(): Flow<List<ProductGiveawayEntity>>

    @Query("SELECT * FROM product_giveaways WHERE timestamp >= :startTime AND timestamp <= :endTime ORDER BY
 timestamp DESC")
    fun getGiveawaysInRange(startTime: Long, endTime: Long): Flow<List<ProductGiveawayEntity>>

    @Query("SELECT * FROM product_giveaways WHERE id = :id")
    suspend fun getGiveawayById(id: String): ProductGiveawayEntity?

    @Query("SELECT SUM(quantityBase * COALESCE(unitCostSnapshotPaise, 0)) FROM product_giveaways WHERE
```

```
    isCancelled = 0 AND timestamp >= :startTime AND timestamp <= :endTime")
    fun getTotalGiveawayCostInRange(startTime: Long, endTime: Long): Flow<Long?>

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertGiveaway(giveaway: ProductGiveawayEntity)

    @Query("UPDATE product_giveaways SET isCancelled = 1, cancelledTimestamp = :timestamp WHERE id = :id")
    suspend fun cancelGiveaway(id: String, timestamp: Long = System.currentTimeMillis())
}

@Dao
interface ServiceDefinitionDao {
    @Query("SELECT * FROM service_definitions WHERE isArchived = 0 ORDER BY name ASC")
    fun getAllActiveServices(): Flow<List<ServiceDefinitionEntity>>

    @Query("SELECT * FROM service_definitions ORDER BY name ASC")
    fun getAllServices(): Flow<List<ServiceDefinitionEntity>>

    @Query("SELECT * FROM service_definitions WHERE id = :id")
    suspend fun getServiceById(id: String): ServiceDefinitionEntity?

    @Query("SELECT * FROM service_definitions WHERE LOWER(name) = LOWER(:name) AND isArchived = 0 LIMIT 1")
    suspend fun getServiceByName(name: String): ServiceDefinitionEntity?

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertService(service: ServiceDefinitionEntity)

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertServices(services: List<ServiceDefinitionEntity>)

    @Update
    suspend fun updateService(service: ServiceDefinitionEntity)

    @Query("UPDATE service_definitions SET isArchived = 1 WHERE id = :id")
    suspend fun archiveService(id: String)
}

@Dao
interface SyncEventDao {
    @Query("SELECT * FROM sync_events WHERE status = 'PENDING' ORDER BY timestamp ASC")
    suspend fun getPendingEvents(): List<SyncEventEntity>

    @Query("SELECT COUNT(*) FROM sync_events WHERE status = 'PENDING'")
    fun getPendingCount(): Flow<Int>

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertEvent(event: SyncEventEntity)

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertEvents(events: List<SyncEventEntity>)

    @Query("UPDATE sync_events SET status = :status WHERE id = :id")
    suspend fun updateEventStatus(id: String, status: String)

    @Query("DELETE FROM sync_events WHERE id = :id")
    suspend fun deleteEvent(id: String)

    @Query("SELECT * FROM sync_events WHERE entityId = :entityId LIMIT 1")
    suspend fun getEventByEntityId(entityId: String): SyncEventEntity?
}

@Dao
interface PigmySchemeDao {
    @Query("SELECT * FROM pigmy_schemes ORDER BY createdAt ASC")
    fun getAllSchemes(): Flow<List<PigmySchemeEntity>>

    @Query("SELECT * FROM pigmy_schemes WHERE isArchived = 0 ORDER BY createdAt ASC")
    fun getActiveSchemes(): Flow<List<PigmySchemeEntity>>

    @Query("SELECT * FROM pigmy_schemes WHERE id = :id LIMIT 1")
    suspend fun getSchemeById(id: String): PigmySchemeEntity?

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertScheme(scheme: PigmySchemeEntity)

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertSchemes(schemes: List<PigmySchemeEntity>)

    @Update
    suspend fun updateScheme(scheme: PigmySchemeEntity)

    @Query("UPDATE pigmy_schemes SET isArchived = :isArchived, updatedAt = :updatedAt WHERE id = :schemeId")
    suspend fun setArchived(schemeId: String, isArchived: Boolean, updatedAt: Long =
System.currentTimeMillis())

    @Query("UPDATE pigmy_schemes SET name = :name, usualDailyAmountPaise = :usualPaise, updatedAt =
:updatedAt WHERE id = :schemeId")
    suspend fun updateSchemeDetails(schemeId: String, name: String, usualPaise: Long, updatedAt: Long =
System.currentTimeMillis())
```

```
491
492     @Query("SELECT COUNT(*) FROM pigmy_schemes")
493     suspend fun getSchemeCount(): Int
494
495     @Query("UPDATE pigmy_schemes SET openingBalancePaise = :openingBalancePaise, openingBalanceDate = :date,
     updatedAt = :updatedAt WHERE id = :schemeId")
496     suspend fun updateOpeningBalance(schemeId: String, openingBalancePaise: Long, date: Long?, updatedAt:
    Long = System.currentTimeMillis())
497 }
498
499 @Dao
500 interface CashCategoryDao {
501     @Query("SELECT * FROM cash_categories ORDER BY name ASC")
502     fun getAllCategories(): Flow<List<CashCategoryEntity>>
503
504     @Query("SELECT * FROM cash_categories WHERE isArchived = 0 ORDER BY name ASC")
505     fun getActiveCategories(): Flow<List<CashCategoryEntity>>
506
507     @Query("DELETE FROM cash_categories WHERE id = :id")
508     suspend fun deleteCategory(id: String)
509
510     @Query("SELECT * FROM cash_categories WHERE id = :id LIMIT 1")
511     suspend fun getCategoryById(id: String): CashCategoryEntity?
512
513     @Query("SELECT * FROM cash_categories WHERE LOWER(TRIM(name)) = LOWER(TRIM(:name)) LIMIT 1")
514     suspend fun getCategoryByName(name: String): CashCategoryEntity?
515
516     @Insert(onConflict = OnConflictStrategy.REPLACE)
517     suspend fun insertCategory(category: CashCategoryEntity)
518
519     @Insert(onConflict = OnConflictStrategy.REPLACE)
520     suspend fun insertCategories(categories: List<CashCategoryEntity>)
521
522     @Update
523     suspend fun updateCategory(category: CashCategoryEntity)
524
525     @Query("UPDATE cash_categories SET isArchived = :isArchived, updatedAt = :timestamp WHERE id = :id")
526     suspend fun setArchived(id: String, isArchived: Boolean, timestamp: Long = System.currentTimeMillis())
527
528     @Query("UPDATE cash_categories SET name = :newName, updatedAt = :timestamp WHERE id = :id")
529     suspend fun renameCategory(id: String, newName: String, timestamp: Long = System.currentTimeMillis())
530
531     @Query("SELECT COUNT(*) FROM cash_categories")
532     suspend fun getCategoryCount(): Int
533 }
534
535 @Dao
536 interface PigmyWithdrawalDao {
537     @Query("SELECT * FROM pigmy_withdrawals WHERE accountSchemeId = :schemeId ORDER BY withdrawalTimestamp
    DESC")
538     fun getWithdrawalsForScheme(schemeId: String): Flow<List<PigmyWithdrawalEntity>>
539
540     @Query("SELECT * FROM pigmy_withdrawals ORDER BY withdrawalTimestamp DESC")
541     fun getAllWithdrawals(): Flow<List<PigmyWithdrawalEntity>>
542
543     @Query("SELECT * FROM pigmy_withdrawals WHERE isVoided = 0 ORDER BY withdrawalTimestamp DESC")
544     fun getActiveWithdrawals(): Flow<List<PigmyWithdrawalEntity>>
545
546     @Query("SELECT * FROM pigmy_withdrawals WHERE withdrawalTimestamp BETWEEN :startTime AND :endTime ORDER
    BY withdrawalTimestamp DESC")
547     fun getWithdrawalsInRange(startTime: Long, endTime: Long): Flow<List<PigmyWithdrawalEntity>>
548
549     @Query("SELECT * FROM pigmy_withdrawals WHERE id = :id LIMIT 1")
550     suspend fun getWithdrawalById(id: String): PigmyWithdrawalEntity?
551
552     @Query("SELECT * FROM pigmy_withdrawals WHERE linkedExpenseId = :expenseId LIMIT 1")
553     suspend fun getWithdrawalByLinkedExpenseId(expenseId: String): PigmyWithdrawalEntity?
554
555     @Query("UPDATE pigmy_withdrawals SET isVoided = 1, voidReason = :reason, updatedAt = :updatedAt WHERE
    linkedExpenseId = :expenseId")
556     suspend fun voidByLinkedExpenseId(expenseId: String, reason: String, updatedAt: Long =
    System.currentTimeMillis())
557
558     @Query("SELECT SUM(amountPaise) FROM pigmy_withdrawals WHERE isVoided = 0")
559     fun getTotalRecordedWithdrawalsPaise(): Flow<Long?>
560
561     @Query("SELECT SUM(amountPaise) FROM pigmy_withdrawals WHERE accountSchemeId = :schemeId AND isVoided =
    0")
562     fun getTotalForScheme(schemeId: String): Flow<Long?>
563
564     @Insert(onConflict = OnConflictStrategy.REPLACE)
565     suspend fun insertWithdrawal(withdrawal: PigmyWithdrawalEntity)
566
567     @Update
568     suspend fun updateWithdrawal(withdrawal: PigmyWithdrawalEntity)
569
570     @Query("UPDATE pigmy_withdrawals SET isVoided = 1, voidReason = :reason, updatedAt = :timestamp WHERE id
     = :id")
```

```
571      suspend fun voidWithdrawal(id: String, reason: String, timestamp: Long = System.currentTimeMillis())
572  }
573
```

app\src\main\java\com\alieducational\app\data\local\entity\Entities.kt

```
package com.alieducational.app.data.local.entity

import androidx.room.Entity
import androidx.room.Index
import androidx.room.PrimaryKey
import java.util.UUID

@Entity(tableName = "products")
data class ProductEntity(
    @PrimaryKey
    val id: String = UUID.randomUUID().toString(),
    val name: String,
    val urduName: String? = null,
    val category: String,
    val subcategory: String? = null,
    val brand: String? = null,
    val sku: String? = null,
    val barcode: String? = null, // primary piece barcode if assigned
    val isFavorite: Boolean = false, // quick tap on counter billing
    val photoUri: String? = null,
    val notes: String? = null,
    val baseUnit: String = "piece", // e.g. "piece", "packet"
    val packSize: Int = 1,          // e.g. 10 pieces per box
    val movingAvgCostPaise: Long = 0L,     // weighted-average cost in paise
    val sellingPricePiecePaise: Long = 0L, // selling price per piece
    val sellingPricePackPaise: Long = 0L,  // selling price per box/pack
    val wholesalePricePiecePaise: Long = 0L, // wholesale price per piece (optional)
    val currentSellableStock: Int = 0,     // current sellable quantity in base units
    val currentDamagedStock: Int = 0,      // current damaged quantity in base units
    val lowStockThreshold: Int = 5,        // alert threshold in base units
    val isArchived: Boolean = false,
    val createdAt: Long = System.currentTimeMillis(),
    val updatedAt: Long = System.currentTimeMillis()
)

@Entity(
    tableName = "product_barcodes",
    indices = [Index(value = ["barcode"], unique = true)]
)
data class ProductBarcodeEntity(
    @PrimaryKey
    val id: String = UUID.randomUUID().toString(),
    val barcode: String, // String preserves leading zeros, never numeric
    val productId: String,
    val unitType: String = "PIECE", // PIECE, PACK
    val packQuantity: Int = 1,
    val createdAt: Long = System.currentTimeMillis()
)

@Entity(tableName = "stock_movements")
data class StockMovementEntity(
    @PrimaryKey
    val id: String = UUID.randomUUID().toString(),
    val productId: String,
    val type: String, // RECEIVE, SALE, SHOP_USE, GIVEAWAY, DAMAGE, RETURN, CORRECTION
    val quantityChangeBase: Int, // positive for addition, negative for deduction
    val countedTotalBase: Int? = null, // for CORRECTION
    val unitCostPaise: Long = 0L, // unit cost at time of movement
    val reason: String? = null,
    val referenceId: String? = null, // linked purchaseId or saleId
    val timestamp: Long = System.currentTimeMillis()
)

@Entity(tableName = "suppliers")
data class SupplierEntity(
    @PrimaryKey
    val id: String = UUID.randomUUID().toString(),
    val name: String,
    val phone: String? = null,
    val address: String? = null,
    val taxDetails: String? = null,
    val bankDetails: String? = null,
    val notes: String? = null,
    val currentBalancePaise: Long = 0L, // positive means shop owes supplier
    val isArchived: Boolean = false,
    val createdAt: Long = System.currentTimeMillis()
)

@Entity(tableName = "customers")
data class CustomerEntity(
    @PrimaryKey
    val id: String = UUID.randomUUID().toString(),
```

```
    val name: String,
    val phone: String? = null,
    val notes: String? = null,
    val currentBalancePaise: Long = 0L, // positive means customer owes shop
    val isArchived: Boolean = false,
    val createdAt: Long = System.currentTimeMillis()
)

@Entity(tableName = "purchases")
data class PurchaseEntity(
    @PrimaryKey
    val id: String = UUID.randomUUID().toString(),
    val supplierId: String,
    val invoiceNumber: String? = null,
    val invoiceDate: Long = System.currentTimeMillis(),
    val totalGoodsValuePaise: Long,
    val deliveryChargePaise: Long = 0L,
    val deliveryPaidTo: String = "SUPPLIER", // SUPPLIER, CARRIER, SETTLED
    val amountPaidPaise: Long = 0L,
    val remainingPayablePaise: Long = 0L,
    val paymentMethod: String = "CASH", // CASH, UPI, BANK_TRANSFER, CHEQUE
    val notes: String? = null,
    val photoUri: String? = null,
    val timestamp: Long = System.currentTimeMillis()
)

@Entity(tableName = "purchase_lines")
data class PurchaseLineEntity(
    @PrimaryKey
    val id: String = UUID.randomUUID().toString(),
    val purchaseId: String,
    val productId: String,
    val quantityBase: Int,
    val unitPurchasePricePaise: Long,
    val allocatedDeliveryPaise: Long = 0L,
    val effectiveUnitCostPaise: Long
)

@Entity(tableName = "sales")
data class SaleEntity(
    @PrimaryKey
    val id: String = UUID.randomUUID().toString(),
    val receiptNumber: String,
    val customerId: String? = null, // null for anonymous cash sale
    val customerName: String? = null,
    val grossTotalPaise: Long,
    val discountType: String = "NONE", // NONE, PERCENT, FIXED
    val discountValuePaise: Long = 0L,
    val discountPercent: Double = 0.0,
    val netTotalPaise: Long,
    val amountReceivedPaise: Long,
    val changeGivenPaise: Long = 0L, // change returned to customer
    val remainingCreditPaise: Long = 0L,
    val paymentMethod: String = "CASH", // CASH, UPI, CARD, CREDIT
    val isSummaryEntry: Boolean = false, // true for money-only daily summaries
    val unresolvedStockDetails: Boolean = false,
    val notes: String? = null,
    val isVoided: Boolean = false,
    val voidReason: String? = null,
    val voidTimestamp: Long? = null,
    val refundPayoutPaise: Long = 0L,
    val stockRestoredCondition: String? = null, // SELLABLE, DAMAGED, NONE
    val originDeviceId: String? = null,
    val syncStatus: String = "LOCAL", // LOCAL, PENDING_SYNC, SYNCED
    val timestamp: Long = System.currentTimeMillis()
)

@Entity(tableName = "sale_lines")
data class SaleLineEntity(
    @PrimaryKey
    val id: String = UUID.randomUUID().toString(),
    val saleId: String,
    val productId: String?, // null for money-only daily summaries or services
    val productName: String,
    val quantityBase: Int,
    val isPack: Boolean = false,
    val unitSellingPricePaise: Long,
    val unitCostSnapshotPaise: Long?, // cost at time of sale; null if unknown or service
    val lineGrossPaise: Long,
    val lineDiscountAllocatedPaise: Long = 0L,
    val lineNetPaise: Long,
    val lineProfitPaise: Long?, // null if cost unknown or service
    val lineType: String = "PRODUCT", // PRODUCT, SERVICE
    val serviceId: String? = null,
    val serviceName: String? = null,
    val serviceOption: String? = null, // e.g. "A4 B&W", "A4 Colour"
    val unitLabel: String? = null // e.g. "sides", "copies", "sheets"
)
```

```
171
172 @Entity(tableName = "account_payments")
173 data class AccountPaymentEntity(
174     @PrimaryKey
175     val id: String = UUID.randomUUID().toString(),
176     val accountType: String, // SUPPLIER or CUSTOMER
177     val accountId: String,
178     val amountPaise: Long,
179     val paymentMethod: String = "CASH",
180     val notes: String? = null,
181     val receiptReference: String? = null,
182     val timestamp: Long = System.currentTimeMillis()
183 )
184
185 @Entity(tableName = "shop_expenses")
186 data class ShopExpenseEntity(
187     @PrimaryKey
188     val id: String = UUID.randomUUID().toString(),
189     val category: String, // RENT, ELECTRICITY, SALARY, OTHER_EXPENSE, OTHER_INCOME, SERVICE_EXPENSE,
    SAVINGS_TRANSFER
190     val amountPaise: Long,
191     val isIncome: Boolean = false,
192     val notes: String? = null,
193     val isVoided: Boolean = false,
194     val voidReason: String? = null,
195     val expenseCategory: String? = null, // PAPER, INK_TONER, STICKER_SHEETS, MACHINE_REPAIR, OTHER
196     val serviceTarget: String? = null, // PRINTING, PHOTOCOPYING, STICKER_PRINTING, SHARED, or custom
    serviceId
197     val attachmentUri: String? = null,
198     val isSavingsTransfer: Boolean = false,
199     val categoryId: String? = null,
200     val isExpenseRefund: Boolean = false,
201     val linkedExpenseId: String? = null,
202     val classification: String = "SHARED", // PRODUCTS, SERVICES, SHARED
203     val paymentMethod: String = "CASH", // CASH, BANK, UPI
204     val timestamp: Long = System.currentTimeMillis()
205 )
206
207 @Entity(tableName = "cash_categories")
208 data class CashCategoryEntity(
209     @PrimaryKey
210     val id: String = UUID.randomUUID().toString(),
211     val name: String,
212     val isSystem: Boolean = false,
213     val isArchived: Boolean = false,
214     val createdAt: Long = System.currentTimeMillis(),
215     val updatedAt: Long = System.currentTimeMillis()
216 )
217
218 @Entity(tableName = "import_batches")
219 data class ImportBatchEntity(
220     @PrimaryKey
221     val id: String = UUID.randomUUID().toString(),
222     val fileName: String,
223     val totalRows: Int,
224     val importedCount: Int,
225     val skippedCount: Int = 0,
226     val timestamp: Long = System.currentTimeMillis()
227 )
228
229 @Entity(tableName = "held_bills")
230 data class HeldBillEntity(
231     @PrimaryKey
232     val id: String = UUID.randomUUID().toString(),
233     val note: String? = null,
234     val customerId: String? = null,
235     val customerName: String? = null,
236     val discountType: String = "NONE",
237     val discountValuePaise: Long = 0L,
238     val discountPercent: Double = 0.0,
239     val draftJson: String, // serialized lines for recovery across app restart/lock
240     val itemCount: Int = 0,
241     val totalPaise: Long = 0L,
242     val createdAt: Long = System.currentTimeMillis()
243 )
244
245 @Entity(tableName = "pigmy_schemes")
246 data class PigmySchemeEntity(
247     @PrimaryKey
248     val id: String = UUID.randomUUID().toString(),
249     val name: String,
250     val usualDailyAmountPaise: Long = 10000L, // in paise (e.g. ₹100.00)
251     val isArchived: Boolean = false,
252     val openingBalancePaise: Long = 0L,
253     val openingBalanceDate: Long? = null,
254     val createdAt: Long = System.currentTimeMillis(),
255     val updatedAt: Long = System.currentTimeMillis()
256 )
```

```
257
258 @Entity(tableName = "pigmy_deposits")
259 data class PigmyDepositEntity(
260     @PrimaryKey
261     val id: String = UUID.randomUUID().toString(),
262     val accountSchemeId: String = "DEFAULT_PIGMY",
263     val paidTimestamp: Long = System.currentTimeMillis(),
264     val amountPaise: Long,
265     val paymentSource: String = "CASH", // CASH, BANK
266     val notes: String? = null,
267     val linkedExpenseId: String? = null, // links to ShopExpenseEntity
268     val isVoided: Boolean = false,
269     val voidReason: String? = null,
270     val createdAt: Long = System.currentTimeMillis(),
271     val updatedAt: Long = System.currentTimeMillis()
272 )
273
274 @Entity(tableName = "pigmy_withdrawals")
275 data class PigmyWithdrawalEntity(
276     @PrimaryKey
277     val id: String = UUID.randomUUID().toString(),
278     val accountSchemeId: String,
279     val withdrawalTimestamp: Long = System.currentTimeMillis(),
280     val amountPaise: Long,
281     val destination: String = "CASH", // CASH, BANK
282     val notes: String? = null,
283     val linkedExpenseId: String? = null, // links to ShopExpenseEntity (Cash In)
284     val isVoided: Boolean = false,
285     val voidReason: String? = null,
286     val createdAt: Long = System.currentTimeMillis(),
287     val updatedAt: Long = System.currentTimeMillis()
288 )
289
290 @Entity(tableName = "product_giveaways")
291 data class ProductGiveawayEntity(
292     @PrimaryKey
293     val id: String = UUID.randomUUID().toString(),
294     val productId: String,
295     val productName: String,
296     val quantityBase: Int,
297     val isPack: Boolean = false,
298     val packSizeSnapshot: Int = 1,
299     val unitCostSnapshotPaise: Long? = null,
300     val reason: String = "FREE_ITEM", // FREE_ITEM, DONATION
301     val recipientNote: String? = null,
302     val timestamp: Long = System.currentTimeMillis(),
303     val isCancelled: Boolean = false,
304     val cancelledTimestamp: Long? = null
305 )
306
307 @Entity(tableName = "service_definitions")
308 data class ServiceDefinitionEntity(
309     @PrimaryKey
310     val id: String = UUID.randomUUID().toString(),
311     val name: String,
312     val unitLabel: String = "job", // sides, copies, sheets, job
313     val isPredefined: Boolean = false,
314     val isArchived: Boolean = false,
315     val createdAt: Long = System.currentTimeMillis()
316 )
317
318 @Entity(tableName = "sync_events")
319 data class SyncEventEntity(
320     @PrimaryKey
321     val id: String = UUID.randomUUID().toString(),
322     val eventType: String,
323     val entityId: String,
324     val payloadJson: String,
325     val originDeviceId: String,
326     val status: String = "PENDING", // PENDING, SENT, ACKNOWLEDGED, FAILED
327     val retryCount: Int = 0,
328     val timestamp: Long = System.currentTimeMillis()
329 )
```

app\src\main\java\com\alieducational\app\data\preferences\AppPreferences.kt

```kotlin
package com.alieducational.app.data.preferences

import android.content.Context
import androidx.datastore.core.DataStore
import androidx.datastore.preferences.core.Preferences
import androidx.datastore.preferences.core.booleanPreferencesKey
import androidx.datastore.preferences.core.edit
import androidx.datastore.preferences.core.intPreferencesKey
import androidx.datastore.preferences.core.longPreferencesKey
import androidx.datastore.preferences.core.stringPreferencesKey
import androidx.datastore.preferences.preferencesDataStore
import dagger.hilt.android.qualifiers.ApplicationContext
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.map
import javax.inject.Inject
import javax.inject.Singleton

private val Context.dataStore: DataStore<Preferences> by preferencesDataStore(name = "ali_educational_preferences")

@Singleton
class AppPreferences @Inject constructor(
    @ApplicationContext private val context: Context
) {
    private object Keys {
        val IS_FIRST_LAUNCH = booleanPreferencesKey("is_first_launch")
        val IS_LOCK_ENABLED = booleanPreferencesKey("is_lock_enabled")
        val LOCK_TIMEOUT_SECONDS = intPreferencesKey("lock_timeout_seconds")
        val THEME_MODE = stringPreferencesKey("theme_mode")
        val APP_LANGUAGE = stringPreferencesKey("app_language")
        val SHOP_NAME = stringPreferencesKey("shop_name")
        val LAST_BACKUP_TIME = longPreferencesKey("last_backup_time")
        val LAST_EXTERNAL_SAVE_TIME = longPreferencesKey("last_external_save_time")
        val LAST_EXTERNAL_SAVE_DESTINATION = stringPreferencesKey("last_external_save_destination")
        val RECEIPT_ACTION = stringPreferencesKey("receipt_action")
        val RECEIPT_LANGUAGE = stringPreferencesKey("receipt_language")
        val PIGMY_USUAL_AMOUNT_PAISE = longPreferencesKey("pigmy_usual_amount_paise")
        val DEVICE_ROLE = stringPreferencesKey("device_role") // "OWNER" or "BILLING"
        val DEVICE_ID = stringPreferencesKey("device_id")
        val PAIRED_OWNER_HOST = stringPreferencesKey("paired_owner_host")
        val SYNC_TOKEN = stringPreferencesKey("sync_token")
        val SYNC_PORT = intPreferencesKey("sync_port")
    }

    val isFirstLaunch: Flow<Boolean> = context.dataStore.data.map { preferences ->
        preferences[Keys.IS_FIRST_LAUNCH] ?: true
    }

    val isLockEnabled: Flow<Boolean> = context.dataStore.data.map { preferences ->
        preferences[Keys.IS_LOCK_ENABLED] ?: false
    }

    val lockTimeoutSeconds: Flow<Int> = context.dataStore.data.map { preferences ->
        preferences[Keys.LOCK_TIMEOUT_SECONDS] ?: 120
    }

    val themeMode: Flow<String> = context.dataStore.data.map { preferences ->
        preferences[Keys.THEME_MODE] ?: "dark"
    }

    val appLanguage: Flow<String> = context.dataStore.data.map { preferences ->
        // Enforce English UI as per latest decision
        "en"
    }

    val shopName: Flow<String> = context.dataStore.data.map { preferences ->
        preferences[Keys.SHOP_NAME] ?: "Ali Educational"
    }

    val lastBackupTime: Flow<Long> = context.dataStore.data.map { preferences ->
        preferences[Keys.LAST_BACKUP_TIME] ?: 0L
    }

    val receiptAction: Flow<String> = context.dataStore.data.map { preferences ->
        preferences[Keys.RECEIPT_ACTION] ?: "preview"
    }

    val receiptLanguage: Flow<String> = context.dataStore.data.map { preferences ->
        preferences[Keys.RECEIPT_LANGUAGE] ?: "en"
    }

    val pigmyUsualAmountPaise: Flow<Long> = context.dataStore.data.map { preferences ->
```

```
        preferences[Keys.PIGMY_USUAL_AMOUNT_PAISE] ?: 100000L // default ₹1,000.00
    }

    val deviceRole: Flow<String> = context.dataStore.data.map { preferences ->
        preferences[Keys.DEVICE_ROLE] ?: "OWNER"
    }

    val deviceId: Flow<String> = context.dataStore.data.map { preferences ->
        preferences[Keys.DEVICE_ID] ?: "device-main"
    }

    val pairedOwnerHost: Flow<String> = context.dataStore.data.map { preferences ->
        preferences[Keys.PAIRED_OWNER_HOST] ?: ""
    }

    val syncToken: Flow<String> = context.dataStore.data.map { preferences ->
        preferences[Keys.SYNC_TOKEN] ?: ""
    }

    val syncPort: Flow<Int> = context.dataStore.data.map { preferences ->
        preferences[Keys.SYNC_PORT] ?: 8080
    }

    suspend fun setPigmyUsualAmountPaise(amountPaise: Long) {
        context.dataStore.edit { preferences ->
            preferences[Keys.PIGMY_USUAL_AMOUNT_PAISE] = amountPaise
        }
    }

    suspend fun setDeviceRole(role: String) {
        context.dataStore.edit { preferences ->
            preferences[Keys.DEVICE_ROLE] = role
        }
    }

    suspend fun setDeviceId(id: String) {
        context.dataStore.edit { preferences ->
            preferences[Keys.DEVICE_ID] = id
        }
    }

    suspend fun setPairedOwnerHost(host: String) {
        context.dataStore.edit { preferences ->
            preferences[Keys.PAIRED_OWNER_HOST] = host
        }
    }

    suspend fun setSyncToken(token: String) {
        context.dataStore.edit { preferences ->
            preferences[Keys.SYNC_TOKEN] = token
        }
    }

    suspend fun setSyncPort(port: Int) {
        context.dataStore.edit { preferences ->
            preferences[Keys.SYNC_PORT] = port
        }
    }

    suspend fun setFirstLaunch(isFirst: Boolean) {
        context.dataStore.edit { preferences ->
            preferences[Keys.IS_FIRST_LAUNCH] = isFirst
        }
    }

    suspend fun setLockEnabled(enabled: Boolean) {
        context.dataStore.edit { preferences ->
            preferences[Keys.IS_LOCK_ENABLED] = enabled
        }
    }

    suspend fun setLockTimeoutSeconds(seconds: Int) {
        context.dataStore.edit { preferences ->
            preferences[Keys.LOCK_TIMEOUT_SECONDS] = seconds
        }
    }

    suspend fun setThemeMode(mode: String) {
        context.dataStore.edit { preferences ->
            preferences[Keys.THEME_MODE] = mode
        }
    }

    suspend fun setAppLanguage(language: String) {
        context.dataStore.edit { preferences ->
            preferences[Keys.APP_LANGUAGE] = language
        }
    }
```

```
170
171      suspend fun setShopName(name: String) {
172          context.dataStore.edit { preferences ->
173              preferences[Keys.SHOP_NAME] = name
174          }
175      }
176
177          val lastExternalSaveTime: Flow<Long> = context.dataStore.data.map { preferences ->
178          preferences[Keys.LAST_EXTERNAL_SAVE_TIME] ?: 0L
179      }
180
181      val lastExternalSaveDestination: Flow<String> = context.dataStore.data.map { preferences ->
182          preferences[Keys.LAST_EXTERNAL_SAVE_DESTINATION] ?: ""
183      }
184
185      suspend fun setLastExternalSave(timestamp: Long, destination: String) {
186          context.dataStore.edit { preferences ->
187              preferences[Keys.LAST_EXTERNAL_SAVE_TIME] = timestamp
188              preferences[Keys.LAST_EXTERNAL_SAVE_DESTINATION] = destination
189          }
190      }
191
192  suspend fun setLastBackupTime(timestamp: Long) {
193          context.dataStore.edit { preferences ->
194              preferences[Keys.LAST_BACKUP_TIME] = timestamp
195          }
196      }
197
198      suspend fun setReceiptAction(action: String) {
199          context.dataStore.edit { preferences ->
200              preferences[Keys.RECEIPT_ACTION] = action
201          }
202      }
203
204      suspend fun setReceiptLanguage(language: String) {
205          context.dataStore.edit { preferences ->
206              preferences[Keys.RECEIPT_LANGUAGE] = language
207          }
208      }
209  }
```

app\src\main\java\com\alieducational\app\data\repository\AppRepository.kt

```
package com.alieducational.app.data.repository

import androidx.room.withTransaction
import com.alieducational.app.data.local.AppDatabase
import com.alieducational.app.data.local.dao.AccountPaymentDao
import com.alieducational.app.data.local.dao.CustomerDao
import com.alieducational.app.data.local.dao.HeldBillDao
import com.alieducational.app.data.local.dao.ImportBatchDao
import com.alieducational.app.data.local.dao.PigmyDepositDao
import com.alieducational.app.data.local.dao.CashCategoryDao
import com.alieducational.app.data.local.dao.PigmySchemeDao
import com.alieducational.app.data.local.dao.PigmyWithdrawalDao
import com.alieducational.app.data.local.entity.CashCategoryEntity
import com.alieducational.app.data.local.entity.PigmyWithdrawalEntity
import com.alieducational.app.data.local.dao.ProductBarcodeDao
import com.alieducational.app.data.local.dao.ProductDao
import com.alieducational.app.data.local.dao.ProductGiveawayDao
import com.alieducational.app.data.local.dao.PurchaseDao
import com.alieducational.app.data.local.dao.SaleDao
import com.alieducational.app.data.local.dao.ServiceDefinitionDao
import com.alieducational.app.data.local.dao.ShopExpenseDao
import com.alieducational.app.data.local.dao.StockMovementDao
import com.alieducational.app.data.local.dao.SupplierDao
import com.alieducational.app.data.local.dao.SyncEventDao
import com.alieducational.app.data.local.entity.AccountPaymentEntity
import com.alieducational.app.data.local.entity.CustomerEntity
import com.alieducational.app.data.local.entity.HeldBillEntity
import com.alieducational.app.data.local.entity.PigmyDepositEntity
import com.alieducational.app.data.local.entity.PigmySchemeEntity
import com.alieducational.app.data.local.entity.ProductBarcodeEntity
import com.alieducational.app.data.local.entity.ProductEntity
import com.alieducational.app.data.local.entity.ProductGiveawayEntity
import com.alieducational.app.data.local.entity.PurchaseEntity
import com.alieducational.app.data.local.entity.PurchaseLineEntity
import com.alieducational.app.data.local.entity.SaleEntity
import com.alieducational.app.data.local.entity.SaleLineEntity
import com.alieducational.app.data.local.entity.ServiceDefinitionEntity
import com.alieducational.app.data.local.entity.ShopExpenseEntity
import com.alieducational.app.data.local.entity.StockMovementEntity
import com.alieducational.app.data.local.entity.SupplierEntity
import com.alieducational.app.data.local.entity.SyncEventEntity
import com.alieducational.app.domain.calculation.CostCalculator
import com.alieducational.app.domain.calculation.PurchaseLineInput
import com.alieducational.app.domain.calculation.SaleLineInput
import kotlinx.coroutines.flow.Flow
import java.text.SimpleDateFormat
import java.util.Date
import java.util.Locale
import java.util.UUID
import javax.inject.Inject
import javax.inject.Singleton

data class PurchaseItemParam(
    val productId: String,
    val quantityBase: Int,
    val unitPurchasePricePaise: Long
)

data class SaleItemParam(
    val productId: String?,
    val productName: String,
    val quantityBase: Int,
    val isPack: Boolean = false,
    val unitSellingPricePaise: Long,
    val lineId: String = UUID.randomUUID().toString(),
    val lineType: String = "PRODUCT", // "PRODUCT", "SERVICE"
    val serviceId: String? = null,
    val serviceName: String? = null,
    val serviceOption: String? = null,
    val unitLabel: String? = null
)

data class BarcodeLookupResult(
    val product: ProductEntity,
    val unitType: String, // "PIECE" or "PACK"
    val packQuantity: Int,
    val barcode: String
)

@Singleton
class AppRepository @Inject constructor(
    private val db: AppDatabase,
```

```
    private val productDao: ProductDao,
    private val productBarcodeDao: ProductBarcodeDao,
    private val stockMovementDao: StockMovementDao,
    private val supplierDao: SupplierDao,
    private val customerDao: CustomerDao,
    private val purchaseDao: PurchaseDao,
    private val saleDao: SaleDao,
    private val paymentDao: AccountPaymentDao,
    private val expenseDao: ShopExpenseDao,
    private val heldBillDao: HeldBillDao,
    private val pigmyDepositDao: PigmyDepositDao,
    private val pigmyWithdrawalDao: PigmyWithdrawalDao,
    private val pigmySchemeDao: PigmySchemeDao,
    private val cashCategoryDao: CashCategoryDao,
    private val productGiveawayDao: ProductGiveawayDao,
    private val serviceDefinitionDao: ServiceDefinitionDao,
    private val syncEventDao: SyncEventDao
) {

    // ── Stock & Inventory ──────────────────────────────────────

    val allProducts: Flow<List<ProductEntity>> = productDao.getAllActiveProducts()
    val favoriteProducts: Flow<List<ProductEntity>> = productDao.getFavoriteProducts()
    val allCategories: Flow<List<String>> = productDao.getAllCategories()
    val lowStockProducts: Flow<List<ProductEntity>> = productDao.getLowStockProducts()
    val totalStockValuePaise: Flow<Long?> = productDao.getTotalStockValuePaise()
    val lowStockCount: Flow<Int> = productDao.getLowStockCount()
    val totalProductCount: Flow<Int> = productDao.getTotalProductCount()
    val recentStockMovements: Flow<List<StockMovementEntity>> = stockMovementDao.getRecentMovements()
    val allHeldBills: Flow<List<HeldBillEntity>> = heldBillDao.getAllHeldBills()

    fun searchProducts(query: String): Flow<List<ProductEntity>> = productDao.searchProducts(query)
    fun getProductsByCategory(category: String): Flow<List<ProductEntity>> =
productDao.getProductsByCategory(category)
    suspend fun setFavorite(productId: String, isFav: Boolean) = productDao.setFavorite(productId, isFav)

    // ── Barcode Resolution & Linking ──────────────────────────

    suspend fun lookupBarcode(rawBarcode: String): BarcodeLookupResult? {
        val code = rawBarcode.trim()
        if (code.isEmpty()) return null

        // 1. Check mapped barcodes table (supports distinct piece and pack barcodes)
        val mapping = productBarcodeDao.findByBarcode(code)
        if (mapping != null) {
            val prod = productDao.getProductById(mapping.productId)
            if (prod != null) {
                return BarcodeLookupResult(
                    product = prod,
                    unitType = mapping.unitType,
                    packQuantity = mapping.packQuantity,
                    barcode = code
                )
            }
        }

        // 2. Direct fallback to product's primary barcode or SKU
        val directProd = productDao.getProductByBarcode(code) ?: productDao.getProductBySku(code)
        if (directProd != null) {
            return BarcodeLookupResult(
                product = directProd,
                unitType = "PIECE",
                packQuantity = 1,
                barcode = code
            )
        }

        return null
    }

    suspend fun linkBarcode(barcode: String, productId: String, unitType: String = "PIECE", packQty: Int =
1) {
        val code = barcode.trim()
        productBarcodeDao.insertBarcode(
            ProductBarcodeEntity(
                barcode = code,
                productId = productId,
                unitType = unitType,
                packQuantity = packQty
            )
        )
        if (unitType == "PIECE") {
            val p = productDao.getProductById(productId)
            if (p != null && p.barcode.isNullOrBlank()) {
                productDao.updateProduct(p.copy(barcode = code))
            }
        }
    }
```

```
169
170     // ── Held Bills (Draft parking) ──────────────────────────────
171
172     suspend fun holdBill(bill: HeldBillEntity) = heldBillDao.insertHeldBill(bill)
173     suspend fun deleteHeldBill(id: String) = heldBillDao.deleteHeldBill(id)
174     suspend fun getHeldBillById(id: String): HeldBillEntity? = heldBillDao.getHeldBillById(id)
175
176     val allSaleLines: Flow<List<SaleLineEntity>> = saleDao.getAllSaleLines()
177
178     suspend fun addOrUpdateProductWithStock(
179         existingId: String?,
180         name: String,
181         barcode: String?,
182         category: String,
183         brand: String?,
184         packSize: Int,
185         boxesBrought: Int,
186         loosePiecesBrought: Int,
187         costPerPiecePaise: Long,
188         sellingPricePiecePaise: Long,
189         wholesalePricePiecePaise: Long,
190         lowStockThreshold: Int,
191         supplierName: String?,
192         billNo: String? = null,
193         transportChargesPaise: Long = 0L,
194         isPaidNow: Boolean = false,
195         paymentMethod: String = "CASH"
196     ): ProductEntity = db.withTransaction {
197         val qtyBrought = (boxesBrought * packSize) + loosePiecesBrought
198
199         // Calculate landed cost including allocated transportation
200         val transportPerPiecePaise = if (qtyBrought > 0 && transportChargesPaise > 0L) {
201             transportChargesPaise / qtyBrought
202         } else 0L
203         val effectiveLandedCostPaise = costPerPiecePaise + transportPerPiecePaise
204
205         // 1. Resolve or create supplier if provided
206         var supplierId: String? = null
207         val cleanSupplierName = supplierName?.trim()
208         if (!cleanSupplierName.isNullOrBlank()) {
209             val existingSupplier = supplierDao.getSupplierByName(cleanSupplierName)
210             if (existingSupplier != null) {
211                 supplierId = existingSupplier.id
212             } else {
213                 val newSupplier = SupplierEntity(name = cleanSupplierName)
214                 supplierDao.insertSupplier(newSupplier)
215                 supplierId = newSupplier.id
216             }
217         }
218
219         // 2. Resolve existing product or create new
220         val existingProduct = if (!existingId.isNullOrBlank()) {
221             productDao.getProductById(existingId)
222         } else if (!barcode.isNullOrBlank()) {
223             productDao.getProductByBarcode(barcode)
224         } else {
225             productDao.getProductByName(name.trim())
226         }
227
228         val targetProduct: ProductEntity
229         if (existingProduct != null) {
230             val incomingTotalLandedCostPaise = qtyBrought.toLong() * effectiveLandedCostPaise
231             val newAvgCost = if (qtyBrought > 0) {
232                 CostCalculator.calculateMovingWeightedAverage(
233                     existingStockQuantity = existingProduct.currentSellableStock,
234                     existingAverageCostPaise = existingProduct.movingAvgCostPaise,
235                     incomingQuantity = qtyBrought,
236                     incomingTotalLandedCostPaise = incomingTotalLandedCostPaise
237                 )
238             } else {
239                 if (effectiveLandedCostPaise > 0L) effectiveLandedCostPaise else
    existingProduct.movingAvgCostPaise
240             }
241
242             targetProduct = existingProduct.copy(
243                 name = name.trim(),
244                 barcode = barcode?.trim()?.ifBlank { null } ?: existingProduct.barcode,
245                 category = category.trim().ifBlank { "Stationery" },
246                 brand = brand?.trim()?.ifBlank { null },
247                 packSize = packSize,
248                 movingAvgCostPaise = newAvgCost,
249                 sellingPricePiecePaise = sellingPricePiecePaise,
250                 sellingPricePackPaise = sellingPricePiecePaise * packSize,
251                 wholesalePricePiecePaise = wholesalePricePiecePaise,
252                 lowStockThreshold = lowStockThreshold,
253                 currentSellableStock = existingProduct.currentSellableStock + qtyBrought,
254                 updatedAt = System.currentTimeMillis()
255             )
```

```
            productDao.updateProduct(targetProduct)
        } else {
            targetProduct = ProductEntity(
                name = name.trim(),
                barcode = barcode?.trim()?.ifBlank { null },
                category = category.trim().ifBlank { "Stationery" },
                brand = brand?.trim()?.ifBlank { null },
                packSize = packSize,
                movingAvgCostPaise = effectiveLandedCostPaise,
                sellingPricePiecePaise = sellingPricePiecePaise,
                sellingPricePackPaise = sellingPricePiecePaise * packSize,
                wholesalePricePiecePaise = wholesalePricePiecePaise,
                lowStockThreshold = lowStockThreshold,
                currentSellableStock = qtyBrought
            )
            productDao.insertProduct(targetProduct)
        }

        // 3. Register barcode in product_barcodes table if provided
        val cleanBarcode = barcode?.trim()
        if (!cleanBarcode.isNullOrBlank()) {
            val existingBarcodeMapping = productBarcodeDao.findByBarcode(cleanBarcode)
            if (existingBarcodeMapping == null) {
                productBarcodeDao.insertBarcode(
                    ProductBarcodeEntity(
                        barcode = cleanBarcode,
                        productId = targetProduct.id,
                        unitType = "PIECE",
                        packQuantity = 1
                    )
                )
            }
        }

        // 4. Record stock movement if qtyBrought > 0
        if (qtyBrought > 0) {
            val movementType = if (supplierId != null) "RECEIVE" else (if (existingProduct != null)
"RECEIVE" else "OPENING_STOCK")
            val cleanBillNo = billNo?.trim()
            val reason = when {
                !cleanBillNo.isNullOrBlank() && transportChargesPaise > 0L ->
                    "Bill #$cleanBillNo (incl. ${CostCalculator.formatPaiseToRupees(transportChargesPaise)}
transport)"
                !cleanBillNo.isNullOrBlank() -> "Bill #$cleanBillNo"
                !cleanSupplierName.isNullOrBlank() -> "Received from $cleanSupplierName"
                existingProduct != null -> "Stock Restocked"
                else -> "Opening Stock"
            }

            stockMovementDao.insertMovement(
                StockMovementEntity(
                    productId = targetProduct.id,
                    type = movementType,
                    quantityChangeBase = qtyBrought,
                    unitCostPaise = effectiveLandedCostPaise,
                    reason = reason,
                    referenceId = supplierId
                )
            )
        }

        // 5. Record supplier bill / purchase & handle payment
        if (supplierId != null && qtyBrought > 0) {
            val totalGoodsCostPaise = qtyBrought.toLong() * costPerPiecePaise
            val totalBillAmountPaise = totalGoodsCostPaise + transportChargesPaise

            if (totalBillAmountPaise > 0L) {
                if (isPaidNow) {
                    purchaseDao.insertPurchase(
                        PurchaseEntity(
                            supplierId = supplierId,
                            invoiceNumber = billNo?.trim()?.ifBlank { null },
                            totalGoodsValuePaise = totalGoodsCostPaise,
                            deliveryChargePaise = transportChargesPaise,
                            amountPaidPaise = totalBillAmountPaise,
                            remainingPayablePaise = 0L,
                            paymentMethod = paymentMethod,
                            notes = "Bill #${billNo?.trim() ?: "N/A"}: ${targetProduct.name} ($qtyBrought
pcs) - Paid ($paymentMethod)"
                        )
                    )
                    paymentDao.insertPayment(
                        AccountPaymentEntity(
                            accountType = "SUPPLIER",
                            accountId = supplierId,
                            amountPaise = totalBillAmountPaise,
                            paymentMethod = paymentMethod,
                            notes = "Paid Bill #${billNo?.trim() ?: "N/A"} for ${targetProduct.name}
```

```
    ($qtyBrought pcs)",
                            receiptReference = billNo?.trim()?.ifBlank { null }
                        )
                    )
                } else {
                    supplierDao.adjustBalance(supplierId, totalBillAmountPaise)
                    purchaseDao.insertPurchase(
                        PurchaseEntity(
                            supplierId = supplierId,
                            invoiceNumber = billNo?.trim()?.ifBlank { null },
                            totalGoodsValuePaise = totalGoodsCostPaise,
                            deliveryChargePaise = transportChargesPaise,
                            amountPaidPaise = 0L,
                            remainingPayablePaise = totalBillAmountPaise,
                            paymentMethod = paymentMethod,
                            notes = "Bill #${billNo?.trim() ?: "N/A"}: ${targetProduct.name} ($qtyBrought
    pcs) - Unpaid Credit"
                        )
                    )
                }
            }
        }

        targetProduct
    }

    suspend fun getProductById(id: String): ProductEntity? = productDao.getProductById(id)

    suspend fun addProduct(product: ProductEntity) {
        db.withTransaction {
            productDao.insertProduct(product)
            if (product.currentSellableStock > 0) {
                stockMovementDao.insertMovement(
                    StockMovementEntity(
                        productId = product.id,
                        type = "OPENING_STOCK",
                        quantityChangeBase = product.currentSellableStock,
                        unitCostPaise = product.movingAvgCostPaise,
                        reason = "Opening Stock"
                    )
                )
            }
        }
    }

    suspend fun updateProduct(product: ProductEntity) {
        productDao.updateProduct(product)
    }

    suspend fun recordStockAdjustment(
        productId: String,
        type: String, // SHOP_USE, GIVEAWAY, DAMAGE, RETURN, CORRECTION
        quantityChange: Int,
        countedTotal: Int? = null,
        reason: String?
    ) {
        db.withTransaction {
            val product = productDao.getProductById(productId) ?: return@withTransaction
            val actualChange = if (type == "CORRECTION" && countedTotal != null) {
                countedTotal - product.currentSellableStock
            } else {
                quantityChange
            }

            if (type == "DAMAGE") {
                productDao.moveToDamagedStock(productId, kotlin.math.abs(actualChange))
            } else {
                productDao.adjustSellableStock(productId, actualChange)
            }

            stockMovementDao.insertMovement(
                StockMovementEntity(
                    productId = productId,
                    type = type,
                    quantityChangeBase = actualChange,
                    countedTotalBase = countedTotal,
                    unitCostPaise = product.movingAvgCostPaise,
                    reason = reason
                )
            )
        }
    }

    // ── Purchases & Goods Received ──────────────────────────────

    val allSuppliers: Flow<List<SupplierEntity>> = supplierDao.getAllSuppliers()

    val totalWeOweSuppliersPaise: Flow<Long?> = supplierDao.getTotalWeOweSuppliersPaise()
    val distinctCreditorsCount: Flow<Int> = supplierDao.getDistinctCreditorsCount()
```

```
    val allPurchases: Flow<List<PurchaseEntity>> = purchaseDao.getAllPurchases()

    suspend fun addSupplier(supplier: SupplierEntity) {
        supplierDao.insertSupplier(supplier)
    }

    suspend fun getSupplierById(id: String): SupplierEntity? = supplierDao.getSupplierById(id)

    suspend fun receiveGoods(
        supplierId: String,
        invoiceNumber: String?,
        items: List<PurchaseItemParam>,
        totalDeliveryPaise: Long,
        deliveryPaidTo: String, // SUPPLIER, CARRIER, SETTLED
        amountPaidPaise: Long,
        paymentMethod: String,
        notes: String?
    ): String = db.withTransaction {
        val purchaseId = UUID.randomUUID().toString()
        val lineInputs = items.map {
            PurchaseLineInput(
                id = it.productId,
                quantity = it.quantityBase,
                unitPricePaise = it.unitPurchasePricePaise
            )
        }

        val allocatedDelivery = CostCalculator.allocateDelivery(lineInputs, totalDeliveryPaise)
        val totalGoodsValuePaise = items.sumOf { it.quantityBase.toLong() * it.unitPurchasePricePaise }

        val purchaseLines = items.map { item ->
            val alloc = allocatedDelivery.first { it.lineId == item.productId }
            PurchaseLineEntity(
                purchaseId = purchaseId,
                productId = item.productId,
                quantityBase = item.quantityBase,
                unitPurchasePricePaise = item.unitPurchasePricePaise,
                allocatedDeliveryPaise = alloc.allocatedDeliveryPaise,
                effectiveUnitCostPaise = alloc.effectiveUnitCostPaise
            )
        }

        // Update product stock and moving weighted-average cost
        items.forEach { item ->
            val product = productDao.getProductById(item.productId) ?: return@forEach
            val alloc = allocatedDelivery.first { it.lineId == item.productId }

            val newAvgCost = CostCalculator.calculateMovingWeightedAverage(
                existingStockQuantity = product.currentSellableStock,
                existingAverageCostPaise = product.movingAvgCostPaise,
                incomingQuantity = item.quantityBase,
                incomingTotalLandedCostPaise = alloc.totalLandedCostPaise
            )

            productDao.updateWeightedAverageCost(item.productId, newAvgCost)
            productDao.adjustSellableStock(item.productId, item.quantityBase)

            stockMovementDao.insertMovement(
                StockMovementEntity(
                    productId = item.productId,
                    type = "RECEIVE",
                    quantityChangeBase = item.quantityBase,
                    unitCostPaise = alloc.effectiveUnitCostPaise,
                    reason = if (!invoiceNumber.isNullOrBlank()) "Invoice $invoiceNumber" else "Purchase
received",
                    referenceId = purchaseId
                )
            )
        }

        purchaseDao.insertPurchaseLines(purchaseLines)

        // Calculate liability to supplier
        val deliveryOwedToSupplier = if (deliveryPaidTo == "SUPPLIER") totalDeliveryPaise else 0L
        val totalBilledBySupplier = totalGoodsValuePaise + deliveryOwedToSupplier
        val remainingPayablePaise = totalBilledBySupplier - amountPaidPaise

        val purchase = PurchaseEntity(
            id = purchaseId,
            supplierId = supplierId,
            invoiceNumber = invoiceNumber,
            totalGoodsValuePaise = totalGoodsValuePaise,
            deliveryChargePaise = totalDeliveryPaise,
            deliveryPaidTo = deliveryPaidTo,
            amountPaidPaise = amountPaidPaise,
            remainingPayablePaise = remainingPayablePaise,
            paymentMethod = paymentMethod,
            notes = notes
        )
```

```
        purchaseDao.insertPurchase(purchase)

        // Adjust supplier running balance: + billed - paid = + remainingPayable
        supplierDao.adjustBalance(supplierId, remainingPayablePaise)

        if (amountPaidPaise > 0) {
            paymentDao.insertPayment(
                AccountPaymentEntity(
                    accountType = "SUPPLIER",
                    accountId = supplierId,
                    amountPaise = amountPaidPaise,
                    paymentMethod = paymentMethod,
                    notes = "Payment for purchase invoice ${invoiceNumber ?: ""}",
                    receiptReference = invoiceNumber
                )
            )
        }

        return@withTransaction purchaseId
    }

    // ── Customers & Credit ──────────────────────────────────────

    val allCustomers: Flow<List<CustomerEntity>> = customerDao.getAllCustomers()
    val totalCustomersOweUsPaise: Flow<Long?> = customerDao.getTotalCustomersOweUsPaise()
    val distinctDebtorsCount: Flow<Int> = customerDao.getDistinctDebtorsCount()

    suspend fun addCustomer(customer: CustomerEntity) {
        customerDao.insertCustomer(customer)
    }

    suspend fun getCustomerById(id: String): CustomerEntity? = customerDao.getCustomerById(id)

    suspend fun recordAccountPayment(
        accountType: String, // SUPPLIER or CUSTOMER
        accountId: String,
        amountPaise: Long,
        paymentMethod: String,
        notes: String?,
        receiptRef: String?
    ) = db.withTransaction {
        paymentDao.insertPayment(
            AccountPaymentEntity(
                accountType = accountType,
                accountId = accountId,
                amountPaise = amountPaise,
                paymentMethod = paymentMethod,
                notes = notes,
                receiptReference = receiptRef
            )
        )

        if (accountType == "SUPPLIER") {
            // We paid supplier: our debt decreases
            supplierDao.adjustBalance(accountId, -amountPaise)
        } else {
            // Customer paid us: their debt decreases
            customerDao.adjustBalance(accountId, -amountPaise)
        }
    }

    fun getPaymentsForAccount(accountType: String, accountId: String): Flow<List<AccountPaymentEntity>> =
        paymentDao.getPaymentsForAccount(accountType, accountId)

    // ── Sales, Discounts & Profit ──────────────────────────────

    val allSales: Flow<List<SaleEntity>> = saleDao.getAllSales()
    val unresolvedStockSales: Flow<List<SaleEntity>> = saleDao.getUnresolvedStockSales()

    suspend fun recordSale(
        customerId: String?,
        customerName: String?,
        items: List<SaleItemParam>,
        discountType: String, // NONE, PERCENT, FIXED
        discountInput: Double, // percent value e.g. 10.0 or fixed rupee value e.g. 50.0
        amountReceivedPaise: Long,
        paymentMethod: String,
        notes: String?,
        originDeviceId: String? = null
    ): SaleEntity = db.withTransaction {
        val saleId = UUID.randomUUID().toString()
        val dateStr = SimpleDateFormat("yyyyMMdd", Locale.getDefault()).format(Date())
        val randomSuffix = (1000..9999).random()
        val devicePrefix = if (originDeviceId != null && originDeviceId != "device-main") "AE-B1" else
"AE-M"
        val receiptNumber = "$devicePrefix-$dateStr-$randomSuffix"

        val grossTotalPaise = items.sumOf { it.quantityBase.toLong() * it.unitSellingPricePaise }
```

```
602
603         val discountValuePaise: Long = when (discountType) {
604             "PERCENT" -> ((grossTotalPaise * discountInput) / 100.0).toLong()
605             "FIXED" -> (discountInput * 100.0).toLong().coerceAtMost(grossTotalPaise)
606             else -> 0L
607         }
608
609         val discountPercent: Double = if (grossTotalPaise > 0) {
610             (discountValuePaise.toDouble() / grossTotalPaise.toDouble()) * 100.0
611         } else 0.0
612
613         val netTotalPaise = grossTotalPaise - discountValuePaise
614         val remainingCreditPaise = (netTotalPaise - amountReceivedPaise).coerceAtLeast(0L)
615         val changeGivenPaise = (amountReceivedPaise - netTotalPaise).coerceAtLeast(0L)
616
617         // Allocate whole-sale discount proportionally across items using lineId (not productId)
618         val saleLineInputs = items.map { item ->
619             val costSnapshot = if (item.lineType == "PRODUCT" && item.productId != null) {
620                 val product = productDao.getProductById(item.productId)
621                 product?.movingAvgCostPaise
622             } else null
623
624             SaleLineInput(
625                 id = item.lineId,
626                 quantity = item.quantityBase,
627                 unitSellingPricePaise = item.unitSellingPricePaise,
628                 unitCostPaise = costSnapshot
629             )
630         }
631
632         val allocatedDiscountLines = CostCalculator.allocateWholeSaleDiscount(saleLineInputs,
    discountValuePaise)
633
634         val saleLines = items.map { item ->
635             val alloc = allocatedDiscountLines.first { it.lineId == item.lineId }
636             val isProduct = item.lineType == "PRODUCT" && item.productId != null
637             val costSnapshot = if (isProduct) {
638                 val product = productDao.getProductById(item.productId!!)
639                 product?.movingAvgCostPaise
640             } else null
641
642             SaleLineEntity(
643                 id = item.lineId,
644                 saleId = saleId,
645                 productId = item.productId,
646                 productName = item.productName,
647                 quantityBase = item.quantityBase,
648                 isPack = item.isPack,
649                 unitSellingPricePaise = item.unitSellingPricePaise,
650                 unitCostSnapshotPaise = costSnapshot,
651                 lineGrossPaise = alloc.grossValuePaise,
652                 lineDiscountAllocatedPaise = alloc.allocatedDiscountPaise,
653                 lineNetPaise = alloc.netValuePaise,
654                 lineProfitPaise = if (isProduct) alloc.actualProfitPaise else null,
655                 lineType = item.lineType,
656                 serviceId = item.serviceId,
657                 serviceName = item.serviceName,
658                 serviceOption = item.serviceOption,
659                 unitLabel = item.unitLabel
660             )
661         }
662
663         saleDao.insertSaleLines(saleLines)
664
665         // Deduct stock ONLY for products. Services DO NOT deduct product inventory.
666         items.filter { it.lineType == "PRODUCT" && it.productId != null }.forEach { item ->
667             val pId = item.productId!!
668             productDao.adjustSellableStock(pId, -item.quantityBase)
669             val product = productDao.getProductById(pId)
670             stockMovementDao.insertMovement(
671                 StockMovementEntity(
672                     productId = pId,
673                     type = "SALE",
674                     quantityChangeBase = -item.quantityBase,
675                     unitCostPaise = product?.movingAvgCostPaise ?: 0L,
676                     reason = "Sale $receiptNumber",
677                     referenceId = saleId
678                 )
679             )
680         }
681
682         val sale = SaleEntity(
683             id = saleId,
684             receiptNumber = receiptNumber,
685             customerId = customerId,
686             customerName = customerName,
687             grossTotalPaise = grossTotalPaise,
688             discountType = discountType,
```

```
            discountValuePaise = discountValuePaise,
            discountPercent = discountPercent,
            netTotalPaise = netTotalPaise,
            amountReceivedPaise = amountReceivedPaise,
            changeGivenPaise = changeGivenPaise,
            remainingCreditPaise = remainingCreditPaise,
            paymentMethod = paymentMethod,
            isSummaryEntry = false,
            unresolvedStockDetails = false,
            notes = notes,
            originDeviceId = originDeviceId,
            syncStatus = "LOCAL"
        )
        saleDao.insertSale(sale)

        // If customer owed credit, update customer debt
        if (customerId != null && remainingCreditPaise > 0) {
            customerDao.adjustBalance(customerId, remainingCreditPaise)
        }

        sale
    }

    // ── Cash In/Out Categories ──────────────────────────────

    val allCashCategories: Flow<List<CashCategoryEntity>> = cashCategoryDao.getAllCategories()
    val activeCashCategories: Flow<List<CashCategoryEntity>> = cashCategoryDao.getActiveCategories()

    suspend fun ensureDefaultCashCategories() {
        if (cashCategoryDao.getCategoryCount() == 0) {
            val now = System.currentTimeMillis()
            val defaults = listOf(
                CashCategoryEntity(id = "cat_general", name = "General", isSystem = true, isArchived =
false, createdAt = now, updatedAt = now),
                CashCategoryEntity(id = "cat_pigmy", name = "Pigmy", isSystem = true, isArchived = false,
createdAt = now + 1, updatedAt = now + 1),
                CashCategoryEntity(id = "cat_rent", name = "Rent", isSystem = false, isArchived = false,
createdAt = now + 2, updatedAt = now + 2),
                CashCategoryEntity(id = "cat_electricity", name = "Electricity", isSystem = false,
isArchived = false, createdAt = now + 3, updatedAt = now + 3),
                CashCategoryEntity(id = "cat_transport", name = "Transport", isSystem = false, isArchived =
false, createdAt = now + 4, updatedAt = now + 4),
                CashCategoryEntity(id = "cat_salary", name = "Salary", isSystem = false, isArchived = false,
 createdAt = now + 5, updatedAt = now + 5),
                CashCategoryEntity(id = "cat_tea_snacks", name = "Tea & Snacks", isSystem = false,
isArchived = false, createdAt = now + 6, updatedAt = now + 6)
            )
            cashCategoryDao.insertCategories(defaults)
        }
    }

    suspend fun createCashCategory(name: String): String {
        val trimmed = name.trim()
        require(trimmed.isNotBlank()) { "Category name cannot be blank" }
        val existing = cashCategoryDao.getCategoryByName(trimmed)
        if (existing != null) {
            if (existing.isArchived) {
                cashCategoryDao.setArchived(existing.id, false)
            }
            return existing.id
        }
        val id = UUID.randomUUID().toString()
        val now = System.currentTimeMillis()
        val cat = CashCategoryEntity(id = id, name = trimmed, isSystem = false, isArchived = false,
createdAt = now, updatedAt = now)
        cashCategoryDao.insertCategory(cat)
        return id
    }

    suspend fun renameCashCategory(id: String, newName: String) {
        val trimmed = newName.trim()
        require(trimmed.isNotBlank()) { "Category name cannot be blank" }
        cashCategoryDao.renameCategory(id, trimmed)
    }

    suspend fun setCashCategoryArchived(id: String, isArchived: Boolean) {
        cashCategoryDao.setArchived(id, isArchived)
    }

    // ── Pigmy Daily Savings Tracker ─────────────────────────

    val allPigmySchemes: Flow<List<PigmySchemeEntity>> = pigmySchemeDao.getAllSchemes()
    val activePigmySchemes: Flow<List<PigmySchemeEntity>> = pigmySchemeDao.getActiveSchemes()

    val allPigmyDeposits: Flow<List<PigmyDepositEntity>> = pigmyDepositDao.getAllDeposits()
    val activePigmyDeposits: Flow<List<PigmyDepositEntity>> = pigmyDepositDao.getActiveDeposits()
    val totalPigmyDepositsPaise: Flow<Long?> = pigmyDepositDao.getTotalRecordedDepositsPaise()

```

```
    fun getPigmyDepositsInRange(startTime: Long, endTime: Long): Flow<List<PigmyDepositEntity>> =
        pigmyDepositDao.getDepositsInRange(startTime, endTime)

    fun getPigmyDepositsForScheme(schemeId: String): Flow<List<PigmyDepositEntity>> =
        pigmyDepositDao.getDepositsForScheme(schemeId)

    fun getTotalPigmyDepositsForScheme(schemeId: String): Flow<Long?> =
        pigmyDepositDao.getTotalForScheme(schemeId)

    suspend fun ensureDefaultPigmySchemes() {
        if (pigmySchemeDao.getSchemeCount() == 0) {
            val now = System.currentTimeMillis()
            val defaults = listOf(
                PigmySchemeEntity(
                    id = "DEFAULT_PIGMY",
                    name = "Pigmy Scheme 1",
                    usualDailyAmountPaise = 10000L,
                    isArchived = false,
                    createdAt = now,
                    updatedAt = now
                ),
                PigmySchemeEntity(
                    id = "scheme_2",
                    name = "Pigmy Scheme 2",
                    usualDailyAmountPaise = 20000L,
                    isArchived = false,
                    createdAt = now + 1,
                    updatedAt = now + 1
                ),
                PigmySchemeEntity(
                    id = "scheme_3",
                    name = "Pigmy Scheme 3",
                    usualDailyAmountPaise = 30000L,
                    isArchived = false,
                    createdAt = now + 2,
                    updatedAt = now + 2
                )
            )
            pigmySchemeDao.insertSchemes(defaults)
        }
    }

    suspend fun createPigmyScheme(name: String, usualDailyAmountPaise: Long): String {
        val id = UUID.randomUUID().toString()
        val now = System.currentTimeMillis()
        val scheme = PigmySchemeEntity(
            id = id,
            name = name.trim(),
            usualDailyAmountPaise = maxOf(100L, usualDailyAmountPaise),
            isArchived = false,
            createdAt = now,
            updatedAt = now
        )
        pigmySchemeDao.insertScheme(scheme)
        return id
    }

    suspend fun updatePigmyScheme(id: String, name: String, usualDailyAmountPaise: Long) {
        pigmySchemeDao.updateSchemeDetails(id, name.trim(), maxOf(100L, usualDailyAmountPaise))
    }

    suspend fun setPigmySchemeArchived(id: String, isArchived: Boolean) {
        pigmySchemeDao.setArchived(id, isArchived)
    }

    suspend fun recordPigmyDeposit(
        schemeId: String = "DEFAULT_PIGMY",
        amountPaise: Long,
        paymentSource: String = "CASH",
        notes: String? = null,
        paidTimestamp: Long = System.currentTimeMillis(),
        attachmentUri: String? = null
    ): String = db.withTransaction {
        val scheme = pigmySchemeDao.getSchemeById(schemeId)
        val schemeName = scheme?.name ?: "Pigmy Savings"
        val depositId = UUID.randomUUID().toString()
        var linkedExpenseId: String? = null

        // If paid from shop cash, create a linked savings-transfer expense entry atomically
        if (paymentSource == "CASH") {
            linkedExpenseId = UUID.randomUUID().toString()
            val noteText = if (!notes.isNullOrBlank()) "Pigmy Deposit ($schemeName): $notes" else "Pigmy
Deposit ($schemeName)"
            val expense = ShopExpenseEntity(
                id = linkedExpenseId,
                category = schemeName,
                amountPaise = amountPaise,
                isIncome = false,
```

```
                notes = noteText,
                isSavingsTransfer = true,
                categoryId = schemeId,
                timestamp = paidTimestamp,
                attachmentUri = attachmentUri
            )
            expenseDao.insertExpense(expense)
        }

        val deposit = PigmyDepositEntity(
            id = depositId,
            accountSchemeId = schemeId,
            paidTimestamp = paidTimestamp,
            amountPaise = amountPaise,
            paymentSource = paymentSource,
            notes = notes,
            linkedExpenseId = linkedExpenseId,
            isVoided = false,
            createdAt = System.currentTimeMillis(),
            updatedAt = System.currentTimeMillis()
        )
        pigmyDepositDao.insertDeposit(deposit)
        depositId
    }

    suspend fun updatePigmyDeposit(
        depositId: String,
        schemeId: String,
        amountPaise: Long,
        paymentSource: String,
        notes: String?,
        paidTimestamp: Long
    ) = db.withTransaction {
        val oldDeposit = pigmyDepositDao.getDepositById(depositId) ?: return@withTransaction
        val scheme = pigmySchemeDao.getSchemeById(schemeId)
        val schemeName = scheme?.name ?: "Pigmy Savings"
        val noteText = if (!notes.isNullOrBlank()) "Pigmy Deposit ($schemeName): $notes" else "Pigmy Deposit
($schemeName)"

        var linkedExpenseId = oldDeposit.linkedExpenseId

        if (paymentSource == "CASH") {
            if (linkedExpenseId != null) {
                // Update existing linked cash expense atomically
                val existingExpense = expenseDao.getExpenseById(linkedExpenseId)
                if (existingExpense != null) {
                    expenseDao.insertExpense(
                        existingExpense.copy(
                            amountPaise = amountPaise,
                            notes = noteText,
                            timestamp = paidTimestamp,
                            isVoided = false
                        )
                    )
                }
            } else {
                // Old was bank or missing; create new linked cash expense
                linkedExpenseId = UUID.randomUUID().toString()
                val expense = ShopExpenseEntity(
                    id = linkedExpenseId,
                    category = schemeName,
                    amountPaise = amountPaise,
                    isIncome = false,
                    notes = noteText,
                    isSavingsTransfer = true,
                    categoryId = schemeId,
                    timestamp = paidTimestamp
                )
                expenseDao.insertExpense(expense)
            }
        } else {
            // Source is BANK: reverse/void the linked cash expense if one existed
            if (linkedExpenseId != null) {
                expenseDao.voidExpense(linkedExpenseId, "Payment source changed to BANK")
                linkedExpenseId = null
            }
        }

        val updated = oldDeposit.copy(
            accountSchemeId = schemeId,
            amountPaise = amountPaise,
            paymentSource = paymentSource,
            notes = notes,
            paidTimestamp = paidTimestamp,
            linkedExpenseId = linkedExpenseId,
            updatedAt = System.currentTimeMillis()
        )
        pigmyDepositDao.updateDeposit(updated)
```

```
    }

    suspend fun voidPigmyDeposit(depositId: String, reason: String) = db.withTransaction {
        val deposit = pigmyDepositDao.getDepositById(depositId) ?: return@withTransaction
        pigmyDepositDao.voidDeposit(depositId, reason)
        if (deposit.linkedExpenseId != null) {
            expenseDao.voidExpense(deposit.linkedExpenseId, "Voided with Pigmy deposit: $reason")
        }
    }

    // ── Pigmy Withdrawals ──────────────────────────────────────

    val allPigmyWithdrawals: Flow<List<PigmyWithdrawalEntity>> = pigmyWithdrawalDao.getAllWithdrawals()
    val activePigmyWithdrawals: Flow<List<PigmyWithdrawalEntity>> =
pigmyWithdrawalDao.getActiveWithdrawals()
    val totalPigmyWithdrawalsPaise: Flow<Long?> = pigmyWithdrawalDao.getTotalRecordedWithdrawalsPaise()

    fun getPigmyWithdrawalsForScheme(schemeId: String): Flow<List<PigmyWithdrawalEntity>> =
        pigmyWithdrawalDao.getWithdrawalsForScheme(schemeId)

    fun getTotalPigmyWithdrawalsForScheme(schemeId: String): Flow<Long?> =
        pigmyWithdrawalDao.getTotalForScheme(schemeId)

    suspend fun updatePigmySchemeOpeningBalance(schemeId: String, openingBalancePaise: Long, date: Long? =
null) {
        pigmySchemeDao.updateOpeningBalance(schemeId, maxOf(0L, openingBalancePaise), date)
    }

    suspend fun recordPigmyWithdrawal(
        schemeId: String,
        amountPaise: Long,
        destination: String = "CASH", // CASH, BANK
        notes: String? = null,
        withdrawalTimestamp: Long = System.currentTimeMillis(),
        attachmentUri: String? = null
    ): String = db.withTransaction {
        require(amountPaise > 0) { "Withdrawal amount must be greater than zero" }
        val scheme = pigmySchemeDao.getSchemeById(schemeId)
        val schemeName = scheme?.name ?: "Pigmy Savings"

        val withdrawalId = UUID.randomUUID().toString()
        var linkedExpenseId: String? = null

        // If received into shop cash, create a linked savings-transfer income entry atomically
        if (destination == "CASH") {
            linkedExpenseId = UUID.randomUUID().toString()
            val noteText = if (!notes.isNullOrBlank()) "Pigmy withdrawal — $schemeName: $notes" else "Pigmy
withdrawal — $schemeName"
            val expense = ShopExpenseEntity(
                id = linkedExpenseId,
                category = schemeName,
                amountPaise = amountPaise,
                isIncome = true, // Cash In!
                notes = noteText,
                isSavingsTransfer = true, // Returned savings, NOT sales revenue or operating profit!
                categoryId = schemeId,
                timestamp = withdrawalTimestamp,
                attachmentUri = attachmentUri
            )
            expenseDao.insertExpense(expense)
        }

        val withdrawal = PigmyWithdrawalEntity(
            id = withdrawalId,
            accountSchemeId = schemeId,
            withdrawalTimestamp = withdrawalTimestamp,
            amountPaise = amountPaise,
            destination = destination,
            notes = notes,
            linkedExpenseId = linkedExpenseId,
            isVoided = false,
            createdAt = System.currentTimeMillis(),
            updatedAt = System.currentTimeMillis()
        )
        pigmyWithdrawalDao.insertWithdrawal(withdrawal)
        withdrawalId
    }

    suspend fun updatePigmyWithdrawal(
        withdrawalId: String,
        schemeId: String,
        amountPaise: Long,
        destination: String,
        notes: String?,
        withdrawalTimestamp: Long
    ) = db.withTransaction {
        val oldWithdrawal = pigmyWithdrawalDao.getWithdrawalById(withdrawalId) ?: return@withTransaction
        val scheme = pigmySchemeDao.getSchemeById(schemeId)
        val schemeName = scheme?.name ?: "Pigmy Savings"
```

```
        val noteText = if (!notes.isNullOrBlank()) "Pigmy withdrawal — $schemeName: $notes" else "Pigmy withdrawal — $schemeName"

        var linkedExpenseId = oldWithdrawal.linkedExpenseId

        if (destination == "CASH") {
            if (linkedExpenseId != null) {
                // Update existing linked cash expense atomically
                val existingExpense = expenseDao.getExpenseById(linkedExpenseId)
                if (existingExpense != null) {
                    expenseDao.insertExpense(
                        existingExpense.copy(
                            amountPaise = amountPaise,
                            category = schemeName,
                            categoryId = schemeId,
                            notes = noteText,
                            timestamp = withdrawalTimestamp,
                            isVoided = false
                        )
                    )
                }
            } else {
                // Old was bank or missing; create new linked cash expense
                linkedExpenseId = UUID.randomUUID().toString()
                val expense = ShopExpenseEntity(
                    id = linkedExpenseId,
                    category = schemeName,
                    amountPaise = amountPaise,
                    isIncome = true,
                    notes = noteText,
                    isSavingsTransfer = true,
                    categoryId = schemeId,
                    timestamp = withdrawalTimestamp
                )
                expenseDao.insertExpense(expense)
            }
        } else {
            // Destination is BANK: reverse/void the linked cash expense if one existed
            if (linkedExpenseId != null) {
                expenseDao.voidExpense(linkedExpenseId, "Withdrawal destination changed to BANK")
                linkedExpenseId = null
            }
        }

        val updated = oldWithdrawal.copy(
            accountSchemeId = schemeId,
            amountPaise = amountPaise,
            destination = destination,
            notes = notes,
            withdrawalTimestamp = withdrawalTimestamp,
            linkedExpenseId = linkedExpenseId,
            updatedAt = System.currentTimeMillis()
        )
        pigmyWithdrawalDao.updateWithdrawal(updated)
    }

    suspend fun voidPigmyWithdrawal(withdrawalId: String, reason: String) = db.withTransaction {
        val withdrawal = pigmyWithdrawalDao.getWithdrawalById(withdrawalId) ?: return@withTransaction
        pigmyWithdrawalDao.voidWithdrawal(withdrawalId, reason)
        if (withdrawal.linkedExpenseId != null) {
            expenseDao.voidExpense(withdrawal.linkedExpenseId, "Voided with Pigmy withdrawal: $reason")
        }
    }

    // ── Product Giveaways & Donations ──────────────────────────────

    val allGiveaways: Flow<List<ProductGiveawayEntity>> = productGiveawayDao.getAllGiveaways()
    val activeGiveaways: Flow<List<ProductGiveawayEntity>> = productGiveawayDao.getActiveGiveaways()

    fun getGiveawaysInRange(startTime: Long, endTime: Long): Flow<List<ProductGiveawayEntity>> =
        productGiveawayDao.getGiveawaysInRange(startTime, endTime)

    suspend fun recordProductGiveaway(
        productId: String,
        quantityBase: Int,
        isPack: Boolean = false,
        packSizeSnapshot: Int = 1,
        reason: String = "FREE_ITEM", // FREE_ITEM or DONATION
        recipientNote: String? = null,
        timestamp: Long = System.currentTimeMillis()
    ): String = db.withTransaction {
        val product = productDao.getProductById(productId)
            ?: throw IllegalArgumentException("Product not found")

        val giveawayId = UUID.randomUUID().toString()
        val costSnapshot = if (product.movingAvgCostPaise > 0) product.movingAvgCostPaise else null

        // 1. Deduct sellable stock
```

```
        productDao.adjustSellableStock(productId, -quantityBase, timestamp)

        // 2. Insert audit stock movement
        stockMovementDao.insertMovement(
            StockMovementEntity(
                productId = productId,
                type = "GIVEAWAY",
                quantityChangeBase = -quantityBase,
                unitCostPaise = product.movingAvgCostPaise,
                reason = "${if (reason == "DONATION") "Donation" else "Free item"}: ${recipientNote ?:
    ""}".trim(),
                referenceId = giveawayId,
                timestamp = timestamp
            )
        )

        // 3. Record giveaway record
        val giveaway = ProductGiveawayEntity(
            id = giveawayId,
            productId = productId,
            productName = product.name,
            quantityBase = quantityBase,
            isPack = isPack,
            packSizeSnapshot = packSizeSnapshot,
            unitCostSnapshotPaise = costSnapshot,
            reason = reason,
            recipientNote = recipientNote,
            timestamp = timestamp,
            isCancelled = false
        )
        productGiveawayDao.insertGiveaway(giveaway)
        giveawayId
    }

    suspend fun cancelProductGiveaway(giveawayId: String, reason: String? = null) = db.withTransaction {
        val giveaway = productGiveawayDao.getGiveawayById(giveawayId) ?: return@withTransaction
        if (giveaway.isCancelled) return@withTransaction

        // 1. Mark cancelled
        productGiveawayDao.cancelGiveaway(giveawayId)

        // 2. Restore sellable stock once
        productDao.adjustSellableStock(giveaway.productId, giveaway.quantityBase)

        // 3. Insert audit reversal movement
        stockMovementDao.insertMovement(
            StockMovementEntity(
                productId = giveaway.productId,
                type = "CORRECTION",
                quantityChangeBase = giveaway.quantityBase,
                unitCostPaise = giveaway.unitCostSnapshotPaise ?: 0L,
                reason = "Cancelled giveaway: ${reason ?: ""}".trim(),
                referenceId = giveawayId
            )
        )
    }

    // ── Services & Owner Expenses ──────────────────────────────

    val activeServices: Flow<List<ServiceDefinitionEntity>> = serviceDefinitionDao.getAllActiveServices()
    val allServices: Flow<List<ServiceDefinitionEntity>> = serviceDefinitionDao.getAllServices()

    suspend fun ensurePredefinedServices() {
        val existing = serviceDefinitionDao.getServiceByName("Printing")
        if (existing == null) {
            val predefined = listOf(
                ServiceDefinitionEntity(
                    name = "Printing",
                    unitLabel = "sides",
                    isPredefined = true
                ),
                ServiceDefinitionEntity(
                    name = "Photocopying",
                    unitLabel = "copies",
                    isPredefined = true
                ),
                ServiceDefinitionEntity(
                    name = "Sticker printing",
                    unitLabel = "sheets",
                    isPredefined = true
                )
            )
            serviceDefinitionDao.insertServices(predefined)
        }
    }

    suspend fun addCustomService(name: String, unitLabel: String): ServiceDefinitionEntity {
        val service = ServiceDefinitionEntity(
```

```
            name = name.trim(),
            unitLabel = unitLabel.trim().ifBlank { "job" },
            isPredefined = false
        )
        serviceDefinitionDao.insertService(service)
        return service
    }

    suspend fun archiveService(serviceId: String) {
        serviceDefinitionDao.archiveService(serviceId)
    }

    suspend fun recordServiceExpense(
        category: String, // PAPER, INK_TONER, STICKER_SHEETS, MACHINE_REPAIR, OTHER
        amountPaise: Long,
        serviceTarget: String, // PRINTING, PHOTOCOPYING, STICKER_PRINTING, SHARED
        notes: String? = null,
        attachmentUri: String? = null,
        timestamp: Long = System.currentTimeMillis()
    ): String {
        val expenseId = UUID.randomUUID().toString()
        val expense = ShopExpenseEntity(
            id = expenseId,
            category = "SERVICE_EXPENSE",
            amountPaise = amountPaise,
            isIncome = false,
            notes = notes,
            expenseCategory = category,
            serviceTarget = serviceTarget,
            attachmentUri = attachmentUri,
            isSavingsTransfer = false,
            timestamp = timestamp
        )
        expenseDao.insertExpense(expense)
        return expenseId
    }

    fun getServiceExpensesInRange(startTime: Long, endTime: Long): Flow<List<ShopExpenseEntity>> =
        expenseDao.getServiceExpensesInRange(startTime, endTime)

    fun getServiceLinesInRange(startTime: Long, endTime: Long): Flow<List<SaleLineEntity>> =
        saleDao.getServiceLinesInRange(startTime, endTime)

    // ── Two-Phone Sync Foundations ──────────────────────────────

    val pendingSyncCount: Flow<Int> = syncEventDao.getPendingCount()

    suspend fun getPendingSyncEvents(): List<SyncEventEntity> =
        syncEventDao.getPendingEvents()

    suspend fun markEventSynced(eventId: String) {
        syncEventDao.updateEventStatus(eventId, "ACKNOWLEDGED")
    }

    suspend fun insertSyncEvent(eventType: String, entityId: String, payloadJson: String, originDeviceId:
String) {
        syncEventDao.insertEvent(
            SyncEventEntity(
                eventType = eventType,
                entityId = entityId,
                payloadJson = payloadJson,
                originDeviceId = originDeviceId,
                status = "PENDING"
            )
        )
    }

    suspend fun recordDailyMonetaryTotal(
        dateTimestamp: Long,
        amountPaise: Long,
        isFullDayTotal: Boolean,
        notes: String?
    ): SaleEntity = db.withTransaction {
        val saleId = UUID.randomUUID().toString()
        val dateStr = SimpleDateFormat("yyyyMMdd", Locale.getDefault()).format(Date(dateTimestamp))
        val receiptNumber = "AE-DAILY-$dateStr"

        val sale = SaleEntity(
            id = saleId,
            receiptNumber = receiptNumber,
            customerId = null,
            customerName = "Daily Cash Summary",
            grossTotalPaise = amountPaise,
            discountType = "NONE",
            discountValuePaise = 0L,
            discountPercent = 0.0,
            netTotalPaise = amountPaise,
            amountReceivedPaise = amountPaise,
```

```
            remainingCreditPaise = 0L,
            paymentMethod = "CASH",
            isSummaryEntry = true,
            unresolvedStockDetails = true,
            notes = if (isFullDayTotal) "Full Day Net Sales Summary. $notes" else "Additional Unrecorded Sales Summary. $notes",
            timestamp = dateTimestamp
        )

        saleDao.insertSale(sale)
        return@withTransaction sale
    }

    // ── Shop Expenses & Other Income ──────────────────────────────

    val allExpenses: Flow<List<ShopExpenseEntity>> = expenseDao.getAllExpenses()

    val refundableExpenses: Flow<List<ShopExpenseEntity>> = expenseDao.getRefundableExpenses()

    fun getSalesForCustomer(customerId: String): Flow<List<SaleEntity>> =
        saleDao.getSalesForCustomer(customerId)

    suspend fun recordShopExpense(
        category: String,
        amountPaise: Long,
        isIncome: Boolean,
        notes: String?,
        timestamp: Long = System.currentTimeMillis(),
        categoryId: String? = null,
        attachmentUri: String? = null,
        classification: String = "SHARED",
        paymentMethod: String = "CASH",
        isExpenseRefund: Boolean = false,
        linkedExpenseId: String? = null
    ) {
        expenseDao.insertExpense(
            ShopExpenseEntity(
                category = category,
                amountPaise = amountPaise,
                isIncome = isIncome,
                notes = notes,
                categoryId = categoryId,
                attachmentUri = attachmentUri,
                classification = classification,
                paymentMethod = paymentMethod,
                isExpenseRefund = isExpenseRefund,
                linkedExpenseId = linkedExpenseId,
                timestamp = timestamp
            )
        )
    }

    suspend fun recordExpenseRefund(
        originalExpenseId: String,
        amountPaise: Long,
        paymentMethod: String = "CASH",
        notes: String? = null,
        timestamp: Long = System.currentTimeMillis(),
        attachmentUri: String? = null
    ) = db.withTransaction {
        val original = expenseDao.getExpenseById(originalExpenseId)
            ?: throw IllegalArgumentException("Original expense $originalExpenseId not found")
        if (amountPaise <= 0L) {
            throw IllegalArgumentException("Refund amount must be positive")
        }
        val activeRefunds = expenseDao.getActiveRefundsForExpense(originalExpenseId)
        val priorTotal = activeRefunds.sumOf { it.amountPaise }
        val remainingRefundable = original.amountPaise - priorTotal
        if (amountPaise > remainingRefundable) {
            throw IllegalArgumentException("Refund amount of ₹${amountPaise/100.0} exceeds remaining refundable amount of ₹${remainingRefundable/100.0}")
        }

        val refundExpense = ShopExpenseEntity(
            category = original.category,
            amountPaise = amountPaise,
            isIncome = true,
            isExpenseRefund = true,
            linkedExpenseId = original.id,
            classification = original.classification,
            expenseCategory = original.expenseCategory,
            serviceTarget = original.serviceTarget,
            notes = notes ?: "Refund of ${original.category}",
            paymentMethod = paymentMethod,
            attachmentUri = attachmentUri,
            timestamp = timestamp
        )
        expenseDao.insertExpense(refundExpense)
```

```
    }

    suspend fun updateExpenseAttachment(expenseId: String, attachmentUri: String?) {
        val existing = expenseDao.getExpenseById(expenseId) ?: return
        expenseDao.updateExpense(existing.copy(attachmentUri = attachmentUri))
    }

    suspend fun recordShopExpensesBatch(expenses: List<ShopExpenseEntity>) {
        expenseDao.insertExpenses(expenses)
    }

    fun getSalesByDateRange(startTime: Long, endTime: Long): Flow<List<SaleEntity>> =
        saleDao.getSalesByDateRange(startTime, endTime)

    suspend fun getLinesForSale(saleId: String): List<SaleLineEntity> =
        saleDao.getLinesForSale(saleId)

    suspend fun voidSale(
        saleId: String,
        stockCondition: String, // SELLABLE, DAMAGED, NONE
        voidReason: String
    ) = db.withTransaction {
        val sale = saleDao.getSaleById(saleId) ?: throw IllegalArgumentException("Sale not found: $saleId")
        if (sale.isVoided) throw IllegalStateException("Sale $saleId is already voided")

        val lines = saleDao.getLinesForSale(saleId)
        val timestamp = System.currentTimeMillis()

        // 1. Inventory Return (policy: ask owner whether sellable or damaged)
        lines.forEach { line ->
            val pId = line.productId
            if (pId != null) {
                when (stockCondition) {
                    "SELLABLE" -> {
                        productDao.adjustSellableStock(pId, line.quantityBase)
                        stockMovementDao.insertMovement(
                            StockMovementEntity(
                                productId = pId,
                                type = "SALE_RETURN",
                                quantityChangeBase = line.quantityBase,
                                unitCostPaise = line.unitCostSnapshotPaise ?: 0L,
                                reason = "Voided sale #${sale.receiptNumber}: Restored to sellable stock.
Reason: $voidReason",
                                referenceId = saleId
                            )
                        )
                    }
                    "DAMAGED" -> {
                        productDao.moveToDamagedStock(pId, line.quantityBase)
                        stockMovementDao.insertMovement(
                            StockMovementEntity(
                                productId = pId,
                                type = "DAMAGED",
                                quantityChangeBase = line.quantityBase,
                                unitCostPaise = line.unitCostSnapshotPaise ?: 0L,
                                reason = "Voided sale #${sale.receiptNumber}: Returned damaged/unusable.
Reason: $voidReason",
                                referenceId = saleId
                            )
                        )
                    }
                    else -> {
                        // NONE / No stock restoration
                    }
                }
            }
        }

        // 2. Refund Payout (policy: refund the payout)
        val refundPayoutPaise = minOf(sale.amountReceivedPaise, sale.netTotalPaise)

        if (refundPayoutPaise > 0) {
            paymentDao.insertPayment(
                AccountPaymentEntity(
                    accountType = if (sale.customerId != null) "CUSTOMER" else "CASH_REFUND",
                    accountId = sale.customerId ?: "ANONYMOUS_COUNTER",
                    amountPaise = -refundPayoutPaise,
                    paymentMethod = sale.paymentMethod,
                    notes = "Refund payout for voided sale #${sale.receiptNumber}",
                    receiptReference = sale.receiptNumber
                )
            )
        }

        // 3. Customer balance adjustment: if customer owed remaining credit, reverse that credit debt
        if (sale.customerId != null && sale.remainingCreditPaise > 0) {
            customerDao.adjustBalance(sale.customerId, -sale.remainingCreditPaise)
        }
```

```
1462
1463          // 4. Mark sale as voided with full audit trail
1464          saleDao.voidSale(
1465              saleId = saleId,
1466              reason = voidReason,
1467              timestamp = timestamp,
1468              refundPayout = refundPayoutPaise,
1469              condition = stockCondition
1470          )
1471      }
1472
1473      suspend fun voidExpense(expenseId: String, reason: String) = db.withTransaction {
1474          val refunds = expenseDao.getActiveRefundsForExpense(expenseId)
1475          if (refunds.isNotEmpty()) {
1476              throw IllegalStateException("Cannot cancel an expense with active linked refunds. Please cancel
      linked refunds first.")
1477          }
1478          expenseDao.voidExpense(expenseId, reason)
1479          pigmyDepositDao.voidByLinkedExpenseId(expenseId, reason)
1480          pigmyWithdrawalDao.voidByLinkedExpenseId(expenseId, reason)
1481      }
1482
1483      suspend fun deleteCashCategory(id: String) {
1484          cashCategoryDao.deleteCategory(id)
1485      }
1486
1487      suspend fun archiveCustomer(customerId: String) {
1488          customerDao.archiveCustomer(customerId)
1489      }
1490
1491      suspend fun deleteCustomer(customerId: String) {
1492          customerDao.deleteCustomer(customerId)
1493      }
1494
1495      suspend fun archiveSupplier(supplierId: String) {
1496          supplierDao.archiveSupplier(supplierId)
1497      }
1498
1499      suspend fun deleteSupplier(supplierId: String) {
1500          supplierDao.deleteSupplier(supplierId)
1501      }
1502
1503      fun getPurchasesForSupplier(supplierId: String):
      kotlinx.coroutines.flow.Flow<List<com.alieducational.app.data.local.entity.PurchaseEntity>> =
1504          purchaseDao.getPurchasesForSupplier(supplierId)
1505  }
```

app\src\main\java\com\alieducational\app\data\service\BackupRestoreService.kt

```kotlin
package com.alieducational.app.data.service

import android.content.Context
import android.content.Intent
import android.net.Uri
import android.os.Environment
import androidx.core.content.FileProvider
import com.alieducational.app.data.local.AppDatabase
import com.alieducational.app.data.preferences.AppPreferences
import dagger.hilt.android.qualifiers.ApplicationContext
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.flow.first
import kotlinx.coroutines.withContext
import org.json.JSONObject
import java.io.File
import java.io.FileInputStream
import java.io.FileOutputStream
import java.text.SimpleDateFormat
import java.util.Date
import java.util.Locale
import java.util.zip.ZipEntry
import java.util.zip.ZipInputStream
import java.util.zip.ZipOutputStream
import javax.inject.Inject
import javax.inject.Singleton

data class BackupMetadata(
    val fileName: String,
    val filePath: String,
    val sizeBytes: Long,
    val timestamp: Long,
    val isValid: Boolean = true,
    val integrityNote: String = "Verified SQLite Archive",
    val snapshotId: String = "",
    val packageFilePath: String? = null,
    val completeBackupFilePath: String? = null,
    val excelFilePath: String? = null,
    val photoCount: Int = 0,
    val productCount: Int = 0,
    val billCount: Int = 0,
    val customerCount: Int = 0,
    val supplierCount: Int = 0,
    val cashInOutCount: Int = 0,
    val pigmyCount: Int = 0
)

@Singleton
class BackupRestoreService @Inject constructor(
    @ApplicationContext private val context: Context,
    private val db: AppDatabase,
    private val preferences: AppPreferences,
    private val excelGenerator: ShopRecordsExcelGenerator
) {

    val primaryBackupFolder: File
        get() {
            val appExternal = context.getExternalFilesDir("backups")
            val dir = if (appExternal != null) {
                File(appExternal, "snapshots")
            } else {
                File(context.filesDir, "backups")
            }
            if (!dir.exists()) dir.mkdirs()
            return dir
        }

    val fallbackBackupFolder: File
        get() = primaryBackupFolder

    val backupFolder: File
        get() = primaryBackupFolder

    val backupFolderPath: String
        get() = backupFolder.absolutePath

    suspend fun createBackup(): BackupMetadata = withContext(Dispatchers.IO) {
        // 1. Checkpoint the SQLite WAL file so all committed records are in the main db file
        try {
            val dbHelper = db.openHelper.writableDatabase
            val cursor = dbHelper.query("PRAGMA wal_checkpoint(FULL)")
            cursor.moveToFirst()
            cursor.close()
```

```
        } catch (_: Exception) {}

        val dbFile = context.getDatabasePath("ali_educational.db")
        if (!dbFile.exists()) {
            throw IllegalStateException("Database file 'ali_educational.db' not found.")
        }

        val timestamp = System.currentTimeMillis()
        val dateStr = SimpleDateFormat("yyyy-MM-dd_HH-mm-ss", Locale.US).format(Date(timestamp))
        val snapshotId = "SNAP_$dateStr"
        val shopName = preferences.shopName.first()

        // 2. Fetch all entity collections from Room
        val products = db.productDao().getAllProducts().first()
        val movements = db.stockMovementDao().getAllMovements().first()
        val purchases = db.purchaseDao().getAllPurchases().first()
        val purchaseLines = db.purchaseDao().getAllPurchaseLines().first()
        val sales = db.saleDao().getAllSalesWithVoided().first()
        val saleLines = db.saleDao().getAllSaleLines().first()
        val accountPayments = db.accountPaymentDao().getAllAccountPayments().first()
        val customers = db.customerDao().getAllCustomers().first()
        val suppliers = db.supplierDao().getAllSuppliers().first()
        val cashExpenses = db.shopExpenseDao().getAllExpenses().first()
        val pigmySchemes = db.pigmySchemeDao().getAllSchemes().first()
        val pigmyDeposits = db.pigmyDepositDao().getAllDeposits().first()
        val pigmyWithdrawals = db.pigmyWithdrawalDao().getAllWithdrawals().first()
        val services = db.serviceDefinitionDao().getAllServices().first()
        val cashCategories = db.cashCategoryDao().getAllCategories().first()

        val attachmentsDir = File(context.filesDir, "attachments")
        val photoFiles = if (attachmentsDir.exists() && attachmentsDir.isDirectory) {
            attachmentsDir.listFiles()?.filter { it.isFile && it.length() > 0 } ?: emptyList()
        } else emptyList()

        val snapshotData = ShopSnapshotData(
            snapshotId = snapshotId,
            timestamp = timestamp,
            shopName = shopName,
            products = products,
            movements = movements,
            purchases = purchases,
            purchaseLines = purchaseLines,
            sales = sales,
            saleLines = saleLines,
            accountPayments = accountPayments,
            customers = customers,
            suppliers = suppliers,
            cashExpenses = cashExpenses,
            pigmySchemes = pigmySchemes,
            pigmyDeposits = pigmyDeposits,
            pigmyWithdrawals = pigmyWithdrawals,
            services = services,
            cashCategories = cashCategories,
            photoCount = photoFiles.size
        )

        // 3. Build Output A: Complete recovery archive
        val completeBackupFileName = "Ali_Educational_Complete_Backup_$dateStr.zip"
        val completeBackupFile = File(backupFolder, completeBackupFileName)
        val tempCompleteZip = File(context.cacheDir, "temp_$completeBackupFileName")

        ZipOutputStream(FileOutputStream(tempCompleteZip)).use { zipOut ->
            // A. Main SQLite DB file
            FileInputStream(dbFile).use { fileIn ->
                zipOut.putNextEntry(ZipEntry("ali_educational.db"))
                fileIn.copyTo(zipOut)
                zipOut.closeEntry()
            }

            // B. Manifest JSON
            val manifestJson = JSONObject().apply {
                put("formatVersion", 1)
                put("app", "Ali Educational")
                put("packageName", context.packageName)
                put("schemaVersion", 8)
                put("snapshotId", snapshotId)
                put("timestamp", timestamp)
                put("date", dateStr)
                put("dbSizeBytes", dbFile.length())
                put("productCount", products.size)
                put("billCount", sales.size)
                put("customerCount", customers.size)
                put("supplierCount", suppliers.size)
                put("cashInOutCount", cashExpenses.size)
                put("pigmyCount", pigmyDeposits.size + pigmyWithdrawals.size)
                put("photoCount", photoFiles.size)
            }.toString(2)

```

```
            zipOut.putNextEntry(ZipEntry("manifest.json"))
            zipOut.write(manifestJson.toByteArray(Charsets.UTF_8))
            zipOut.closeEntry()

            // C. Attachment photos
            photoFiles.forEach { file ->
                zipOut.putNextEntry(ZipEntry("attachments/${file.name}"))
                FileInputStream(file).use { fIn -> fIn.copyTo(zipOut) }
                zipOut.closeEntry()
            }
        }

        tempCompleteZip.copyTo(completeBackupFile, overwrite = true)
        tempCompleteZip.delete()

        // 4. Build Output B: Readable Excel records
        val excelFileName = "Ali_Educational_Shop_Records_$dateStr.xlsx"
        val excelFile = File(backupFolder, excelFileName)
        excelGenerator.generateWorkbook(snapshotData, excelFile)

        // 5. Build Outer Save Both package
        val packageFileName = "Ali_Educational_Backup_Package_$dateStr.zip"
        val packageFile = File(backupFolder, packageFileName)
        val tempPackageZip = File(context.cacheDir, "temp_$packageFileName")

        ZipOutputStream(FileOutputStream(tempPackageZip)).use { zipOut ->
            // Add Complete_Backup.zip
            FileInputStream(completeBackupFile).use { fIn ->
                zipOut.putNextEntry(ZipEntry("Complete_Backup.zip"))
                fIn.copyTo(zipOut)
                zipOut.closeEntry()
            }

            // Add Shop_Records.xlsx
            FileInputStream(excelFile).use { fIn ->
                zipOut.putNextEntry(ZipEntry("Shop_Records.xlsx"))
                fIn.copyTo(zipOut)
                zipOut.closeEntry()
            }

            // Add Read_Me.txt
            val readMeText = """
                Ali Educational - Complete Backup Package
                Snapshot ID: $snapshotId
                Created: $dateStr

                Package Contents:
                1. Complete_Backup.zip:
                   The full offline database, settings, and photo attachments archive.
                   Use this file in the Ali Educational app under "Restore Backup" to completely restore your shop data.
                   You do NOT need to extract this zip file manually.

                2. Shop_Records.xlsx:
                   Human-readable Excel workbook containing all business records across 11 sheets:
                   - Summary
                   - Products and Stock
                   - Stock Movements
                   - Bills and Items
                   - Payments and Dues
                   - Customers and Suppliers
                   - Cash In Out
                   - Expenses and Refunds
                   - Services
                   - Pigmy Savings
                   - Categories

                Notice: 100% offline and securely packaged for Ali Educational.
            """.trimIndent()

            zipOut.putNextEntry(ZipEntry("Read_Me.txt"))
            zipOut.write(readMeText.toByteArray(Charsets.UTF_8))
            zipOut.closeEntry()
        }

        tempPackageZip.copyTo(packageFile, overwrite = true)
        tempPackageZip.delete()

        // 6. Try copying package to public Documents directory if accessible
        try {
            val publicDocs = Environment.getExternalStoragePublicDirectory(Environment.DIRECTORY_DOCUMENTS)
            val publicAliDir = File(publicDocs, "AliEducational/Backups")
            if (publicAliDir.exists() || publicAliDir.mkdirs()) {
                packageFile.copyTo(File(publicAliDir, packageFileName), overwrite = true)
                excelFile.copyTo(File(publicAliDir, excelFileName), overwrite = true)
            }
        } catch (_: Exception) {}

```

```
        preferences.setLastBackupTime(timestamp)

        return@withContext BackupMetadata(
            fileName = packageFileName,
            filePath = packageFile.absolutePath,
            sizeBytes = packageFile.length(),
            timestamp = timestamp,
            isValid = true,
            integrityNote = "Verified Complete Archive + Excel Workbook",
            snapshotId = snapshotId,
            packageFilePath = packageFile.absolutePath,
            completeBackupFilePath = completeBackupFile.absolutePath,
            excelFilePath = excelFile.absolutePath,
            photoCount = photoFiles.size,
            productCount = products.size,
            billCount = sales.size,
            customerCount = customers.size,
            supplierCount = suppliers.size,
            cashInOutCount = cashExpenses.size,
            pigmyCount = pigmyDeposits.size + pigmyWithdrawals.size
        )
    }

    suspend fun getAvailableBackups(): List<BackupMetadata> = withContext(Dispatchers.IO) {
        val allFiles = mutableListOf<File>()
        primaryBackupFolder.listFiles { file ->
            file.extension.equals("zip", ignoreCase = true)
        }?.let { allFiles.addAll(it) }

        // Find package files and complete backup files
        val packages = allFiles.filter { it.name.startsWith("Ali_Educational_Backup_Package_") }
        val completeBackups = allFiles.filter { it.name.startsWith("Ali_Educational_Complete_Backup_") ||
it.name.startsWith("ali_backup_") }

        val resultList = mutableListOf<BackupMetadata>()

        // 1. Process package files
        packages.sortedByDescending { it.lastModified() }.forEach { pkgFile ->
            val (isValid, note, manifest) = inspectPackageOrBackup(pkgFile)
            val baseName = pkgFile.nameWithoutExtension.removePrefix("Ali_Educational_Backup_Package_")
            val excelFile = File(primaryBackupFolder, "Ali_Educational_Shop_Records_$baseName.xlsx")
            val completeFile = File(primaryBackupFolder, "Ali_Educational_Complete_Backup_$baseName.zip")

            resultList.add(
                BackupMetadata(
                    fileName = pkgFile.name,
                    filePath = pkgFile.absolutePath,
                    sizeBytes = pkgFile.length(),
                    timestamp = pkgFile.lastModified(),
                    isValid = isValid,
                    integrityNote = note,
                    snapshotId = manifest?.optString("snapshotId") ?: ("SNAP_$baseName"),
                    packageFilePath = pkgFile.absolutePath,
                    completeBackupFilePath = if (completeFile.exists()) completeFile.absolutePath else null,
                    excelFilePath = if (excelFile.exists()) excelFile.absolutePath else null,
                    photoCount = manifest?.optInt("photoCount") ?: 0,
                    productCount = manifest?.optInt("productCount") ?: 0,
                    billCount = manifest?.optInt("billCount") ?: 0,
                    customerCount = manifest?.optInt("customerCount") ?: 0,
                    supplierCount = manifest?.optInt("supplierCount") ?: 0,
                    cashInOutCount = manifest?.optInt("cashInOutCount") ?: 0,
                    pigmyCount = manifest?.optInt("pigmyCount") ?: 0
                )
            )
        }

        // 2. Add standalone complete backups if no corresponding package was created
        completeBackups.sortedByDescending { it.lastModified() }.forEach { cbFile ->
            val isAlreadyRepresented = resultList.any { it.completeBackupFilePath == cbFile.absolutePath }
            if (!isAlreadyRepresented) {
                val (isValid, note, manifest) = inspectPackageOrBackup(cbFile)
                resultList.add(
                    BackupMetadata(
                        fileName = cbFile.name,
                        filePath = cbFile.absolutePath,
                        sizeBytes = cbFile.length(),
                        timestamp = cbFile.lastModified(),
                        isValid = isValid,
                        integrityNote = note,
                        snapshotId = manifest?.optString("snapshotId") ?: "",
                        completeBackupFilePath = cbFile.absolutePath,
                        photoCount = manifest?.optInt("photoCount") ?: 0,
                        productCount = manifest?.optInt("productCount") ?: 0,
                        billCount = manifest?.optInt("billCount") ?: 0,
                        customerCount = manifest?.optInt("customerCount") ?: 0,
                        supplierCount = manifest?.optInt("supplierCount") ?: 0
                    )
                )
```

```
            }
        }

        resultList.sortByDescending { it.timestamp }
        return@withContext resultList
    }

    private fun inspectPackageOrBackup(file: File): Triple<Boolean, String, JSONObject?> {
        try {
            ZipInputStream(FileInputStream(file)).use { zipIn ->
                var foundDb = false
                var foundInnerCompleteZip = false
                var manifestJson: JSONObject? = null
                var entry: ZipEntry?

                while (zipIn.nextEntry.also { entry = it } != null) {
                    val name = entry!!.name
                    if (name == "ali_educational.db") {
                        foundDb = true
                    } else if (name == "Complete_Backup.zip" ||
    name.startsWith("Ali_Educational_Complete_Backup_")) {
                        foundInnerCompleteZip = true
                    } else if (name == "manifest.json") {
                        try {
                            val bytes = zipIn.readBytes()
                            manifestJson = JSONObject(String(bytes, Charsets.UTF_8))
                        } catch (_: Exception) {}
                    }
                    zipIn.closeEntry()
                }

                if (foundInnerCompleteZip) {
                    return Triple(true, "Verified Backup Package (Archive + Excel)", manifestJson)
                }
                if (foundDb) {
                    return Triple(true, "Verified SQLite Recovery Archive", manifestJson)
                }
                return Triple(false, "Invalid Archive: missing database or backup package", null)
            }
        } catch (e: Exception) {
            return Triple(false, "Corrupt file: ${e.localizedMessage}", null)
        }
    }

    suspend fun restoreBackup(backupFile: File): Boolean = withContext(Dispatchers.IO) {
        val destDbFile = context.getDatabasePath("ali_educational.db")

        // 1. Safety backup of current database before restoring
        if (destDbFile.exists()) {
            val safetyDir = File(context.cacheDir, "pre_restore_safety")
            if (!safetyDir.exists()) safetyDir.mkdirs()
            val safetyBackup = File(safetyDir, "safety_${System.currentTimeMillis()}.db")
            destDbFile.copyTo(safetyBackup, overwrite = true)
        }

        // 2. Check if selected file is the outer package containing Complete_Backup.zip
        var sourceArchive = backupFile
        var isTempStaged = false

        ZipInputStream(FileInputStream(backupFile)).use { zipIn ->
            var entry: ZipEntry?
            while (zipIn.nextEntry.also { entry = it } != null) {
                val name = entry!!.name
                if (name == "Complete_Backup.zip" || name.startsWith("Ali_Educational_Complete_Backup_")) {
                    val staged = File(context.cacheDir, "staged_complete_backup.zip")
                    FileOutputStream(staged).use { out -> zipIn.copyTo(out) }
                    sourceArchive = staged
                    isTempStaged = true
                    break
                }
                zipIn.closeEntry()
            }
        }

        // 3. Restore from sourceArchive (with strict zip-slip protection)
        var restoredDb = false
        try {
            ZipInputStream(FileInputStream(sourceArchive)).use { zipIn ->
                var entry: ZipEntry?
                while (zipIn.nextEntry.also { entry = it } != null) {
                    val entryName = entry!!.name

                    // Strict zip slip check
                    if (entryName.contains("..") || entryName.startsWith("/") || entryName.startsWith("\\"))
    {
                        throw SecurityException("Malicious path detected in archive entry: $entryName")
                    }

```

```
                    if (entryName == "ali_educational.db") {
                        val tempDb = File(context.cacheDir, "restored_temp.db")
                        FileOutputStream(tempDb).use { out -> zipIn.copyTo(out) }

                        // Close DB and replace main DB file
                        db.close()
                        tempDb.copyTo(destDbFile, overwrite = true)
                        tempDb.delete()

                        // Remove existing WAL / SHM files if any
                        File(destDbFile.parent, "ali_educational.db-wal").delete()
                        File(destDbFile.parent, "ali_educational.db-shm").delete()

                        restoredDb = true
                    } else if (entryName.startsWith("attachments/") && !entry!!.isDirectory) {
                        val destDir = File(context.filesDir, "attachments")
                        if (!destDir.exists()) destDir.mkdirs()
                        val filename = File(entryName).name
                        val destFile = File(destDir, filename)
                        FileOutputStream(destFile).use { out -> zipIn.copyTo(out) }
                    }
                    zipIn.closeEntry()
                }
            }
        } finally {
            if (isTempStaged) {
                sourceArchive.delete()
            }
        }

        return@withContext restoredDb
    }

    suspend fun restoreFromUri(uri: Uri): Boolean = withContext(Dispatchers.IO) {
        val tempFile = File(context.cacheDir, "imported_restore_${System.currentTimeMillis()}.zip")
        try {
            context.contentResolver.openInputStream(uri)?.use { input ->
                FileOutputStream(tempFile).use { output ->
                    input.copyTo(output)
                }
            }
            val result = restoreBackup(tempFile)
            return@withContext result
        } finally {
            tempFile.delete()
        }
    }

    fun getShareIntent(filePath: String): Intent {
        val file = File(filePath)
        val uri = FileProvider.getUriForFile(
            context,
            "${context.packageName}.fileprovider",
            file
        )
        val mimeType = when {
            file.extension.equals("xlsx", ignoreCase = true) -> "application/vnd.openxmlformats-
officedocument.spreadsheetml.sheet"
            else -> "application/zip"
        }
        return Intent(Intent.ACTION_SEND).apply {
            type = mimeType
            putExtra(Intent.EXTRA_STREAM, uri)
            putExtra(Intent.EXTRA_SUBJECT, "Ali Educational Backup - " + file.name)
            addFlags(Intent.FLAG_GRANT_READ_URI_PERMISSION)
        }
    }

    suspend fun exportBackupToUri(backupFile: File, targetUri: Uri): Boolean = withContext(Dispatchers.IO) {
        try {
            context.contentResolver.openOutputStream(targetUri)?.use { out ->
                FileInputStream(backupFile).use { input ->
                    input.copyTo(out)
                }
            }
            true
        } catch (_: Exception) {
            false
        }
    }
}
```

app\src\main\java\com\alieducational\app\data\service\CashTrackerExcelService.kt

```kotlin
package com.alieducational.app.data.service

import android.content.Context
import android.content.Intent
import android.net.Uri
import android.os.Environment
import androidx.core.content.FileProvider
import com.alieducational.app.data.local.entity.ShopExpenseEntity
import com.alieducational.app.domain.calculation.CostCalculator
import dagger.hilt.android.qualifiers.ApplicationContext
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.withContext
import org.apache.poi.ss.usermodel.BorderStyle
import org.apache.poi.ss.usermodel.Cell
import org.apache.poi.ss.usermodel.CellStyle
import org.apache.poi.ss.usermodel.CellType
import org.apache.poi.ss.usermodel.DateUtil
import org.apache.poi.ss.usermodel.FillPatternType
import org.apache.poi.ss.usermodel.HorizontalAlignment
import org.apache.poi.ss.usermodel.IndexedColors
import org.apache.poi.ss.usermodel.Row
import org.apache.poi.ss.usermodel.VerticalAlignment
import org.apache.poi.ss.usermodel.WorkbookFactory
import org.apache.poi.xssf.usermodel.XSSFWorkbook
import java.io.File
import java.io.FileOutputStream
import java.io.InputStream
import java.text.SimpleDateFormat
import java.util.Date
import java.util.Locale
import javax.inject.Inject
import javax.inject.Singleton
import kotlin.math.roundToLong

data class CashTrackerImportPreview(
    val fileName: String,
    val items: List<ShopExpenseEntity> = emptyList(),
    val totalIncomePaise: Long = 0L,
    val totalExpensePaise: Long = 0L,
    val incomeCount: Int = 0,
    val expenseCount: Int = 0,
    val skippedRows: Int = 0,
    val errorMessage: String? = null
)

@Singleton
class CashTrackerExcelService @Inject constructor(
    @ApplicationContext private val context: Context?
) {
    constructor() : this(null)

    private val exportFolder: File
        get() {
            val docs = Environment.getExternalStoragePublicDirectory(Environment.DIRECTORY_DOCUMENTS)
            val dir = File(docs, "AliEducational/Exports")
            if (!dir.exists()) {
                dir.mkdirs()
            }
            return if (dir.exists()) dir else File(context?.getExternalFilesDir(null), "exports").apply { mkdirs() }
        }

    /**
     * Parse an Excel file (.xlsx or .xls) from content Uri into a preview of ShopExpenseEntity objects.
     */
    suspend fun parseExcelFile(uri: Uri, fileName: String): CashTrackerImportPreview = withContext(Dispatchers.IO) {
        try {
            val inputStream: InputStream = context?.contentResolver?.openInputStream(uri)
                ?: return@withContext CashTrackerImportPreview(
                    fileName = fileName,
                    errorMessage = "Cannot access the selected file. Please pick it again."
                )

            inputStream.use { stream ->
                val workbook = WorkbookFactory.create(stream)
                workbook.use { wb ->
                    parseWorkbook(wb, fileName)
                }
            }
        } catch (e: Exception) {
            CashTrackerImportPreview(
```

```
                fileName = fileName,
                errorMessage = "Failed to parse Excel file: ${e.localizedMessage ?: "Unknown format error"}"
            )
        }
    }

    /**
     * Parse an in-memory or loaded Workbook into a preview.
     */
    fun parseWorkbook(workbook: org.apache.poi.ss.usermodel.Workbook, fileName: String): CashTrackerImportPreview {
        if (workbook.numberOfSheets == 0) {
            return CashTrackerImportPreview(
                fileName = fileName,
                errorMessage = "Excel workbook contains no sheets."
            )
        }

        val sheet = workbook.getSheetAt(0)
        if (sheet.physicalNumberOfRows < 2) {
            return CashTrackerImportPreview(
                fileName = fileName,
                errorMessage = "The sheet does not contain enough data rows (headers + entries)."
            )
        }

        // Find header row (usually row 0 or 1)
        var headerRowIndex = -1
        var dateCol = -1
        var descCol = -1
        var categoryCol = -1
        var incomeCol = -1
        var expenseCol = -1
        var modeCol = -1
        var singleAmountCol = -1
        var typeCol = -1

        for (r in 0..minOf(5, sheet.lastRowNum)) {
            val row = sheet.getRow(r) ?: continue
            var catCol = -1
            var dCol = -1
            var desCol = -1
            var inCol = -1
            var outCol = -1
            var mCol = -1
            var amtCol = -1
            var tCol = -1

            for (c in 0 until row.lastCellNum) {
                val cell = row.getCell(c) ?: continue
                val text = getCellString(cell).trim().lowercase(Locale.ROOT)
                if (text.isBlank()) continue

                when {
                    text == "category" || text.contains("category") || text.contains("      ") -> catCol = c
                    text.contains("date") || text.contains("dated") || text.contains("      ") || text.contains("     ") -> dCol = c
                    text.contains("desc") || text.contains("particular") ||
                            text.contains("item") || text.contains("name") || text.contains("details") || text.contains("     ") -> desCol = c
                    text.contains("cash in") || text.contains("income") || text.contains("credit") ||
                            text == "cr" || text.contains("received") || text.contains("   ") || text == "in" -> inCol = c
                    text.contains("cash out") || text.contains("expense") || text.contains("debit") ||
                            text == "dr" || text.contains("paid") || text.contains("    ") || text == "out" -> outCol = c
                    text.contains("mode") || text.contains("method") || text.contains("type") || text.contains("note") || text.contains("remark") -> mCol = c
                    text.contains("amount") || text.contains("total") || text.contains("₹") || text.contains("rs") -> amtCol = c
                    text == "flow" || text.contains("in/out") || text.contains("income/expense") -> tCol = c
                }
            }

            // Header matches if we found at least description or date, and an income/expense or amount column
            if ((inCol != -1 || outCol != -1 || amtCol != -1) && (desCol != -1 || dCol != -1)) {
                headerRowIndex = r
                categoryCol = catCol
                dateCol = dCol
                descCol = desCol
                incomeCol = inCol
                expenseCol = outCol
                modeCol = mCol
                singleAmountCol = amtCol
                typeCol = tCol
                break
            }
```

```
        }

        // Fallback: If no explicit headers recognized, assume standard columns:
        // Col 0: Date, Col 1: Description, Col 2: Cash In, Col 3: Cash Out, Col 4: Payment Mode
        if (headerRowIndex == -1) {
            headerRowIndex = 0
            dateCol = 0
            descCol = 1
            incomeCol = 2
            expenseCol = 3
            modeCol = 4
        }

        val parsedItems = mutableListOf<ShopExpenseEntity>()
        var skipped = 0

        for (r in (headerRowIndex + 1)..sheet.lastRowNum) {
            val row = sheet.getRow(r) ?: continue
            if (isRowEmpty(row)) continue

            val dateTimestamp = if (dateCol != -1) parseCellDate(row.getCell(dateCol)) else System.currentTimeMillis()
            val catText = if (categoryCol != -1) getCellString(row.getCell(categoryCol)).trim() else ""
            val desc = if (descCol != -1) getCellString(row.getCell(descCol)).trim() else ""
            val mode = if (modeCol != -1) getCellString(row.getCell(modeCol)).trim() else ""

            // Exclude summary rows, opening/previous balance, and totals from transactions
            val combinedText = (desc + " " + mode).lowercase(Locale.ROOT)
            if (combinedText.contains("previous balance") || combinedText.contains("opening balance") ||
                combinedText.contains("closing balance") || combinedText.contains("total") ||
                combinedText.contains("subtotal") || combinedText.contains("grand total")
            ) {
                skipped++
                continue
            }

            var incomePaise = 0L
            var expensePaise = 0L

            if (incomeCol != -1) {
                val inAmt = parseCellAmount(row.getCell(incomeCol))
                if (inAmt > 0) incomePaise = (inAmt * 100).roundToLong()
            }
            if (expenseCol != -1) {
                val outAmt = parseCellAmount(row.getCell(expenseCol))
                if (outAmt > 0) expensePaise = (outAmt * 100).roundToLong()
            }

            // If separate income/expense columns weren't both present, check single amount column
            if (incomePaise == 0L && expensePaise == 0L && singleAmountCol != -1) {
                val amt = parseCellAmount(row.getCell(singleAmountCol))
                if (amt > 0) {
                    val typeStr = if (typeCol != -1) getCellString(row.getCell(typeCol)).lowercase(Locale.ROOT) else ""
                    if (typeStr.contains("in") || typeStr.contains("cr") || typeStr.contains("credit") || typeStr.contains("income") || typeStr.contains("jama")) {
                        incomePaise = (amt * 100).roundToLong()
                    } else {
                        expensePaise = (amt * 100).roundToLong()
                    }
                }
            }

            if (incomePaise == 0L && expensePaise == 0L) {
                skipped++
                continue
            }

            val finalCat = when {
                catText.isNotBlank() -> catText
                desc.isNotBlank() -> desc
                else -> "General"
            }
            val finalNotes = when {
                mode.isNotBlank() -> mode
                desc.isNotBlank() && desc != finalCat -> desc
                else -> "Excel Import"
            }

            if (incomePaise > 0L) {
                parsedItems.add(
                    ShopExpenseEntity(
                        category = finalCat,
                        amountPaise = incomePaise,
                        isIncome = true,
                        notes = finalNotes,
                        timestamp = dateTimestamp
                    )
```

```
                )
            }

            if (expensePaise > 0L) {
                parsedItems.add(
                    ShopExpenseEntity(
                        category = finalCat,
                        amountPaise = expensePaise,
                        isIncome = false,
                        notes = finalNotes,
                        timestamp = dateTimestamp
                    )
                )
            }

            if (incomePaise == 0L && expensePaise == 0L) {
                skipped++
            }
        }

        if (parsedItems.isEmpty()) {
            return CashTrackerImportPreview(
                fileName = fileName,
                errorMessage = "No valid income or expense entries found in the file. Please ensure amounts are greater than 0."
            )
        }

        val incomeItems = parsedItems.filter { it.isIncome }
        val expenseItems = parsedItems.filter { !it.isIncome }

        return CashTrackerImportPreview(
            fileName = fileName,
            items = parsedItems,
            totalIncomePaise = incomeItems.sumOf { it.amountPaise },
            totalExpensePaise = expenseItems.sumOf { it.amountPaise },
            incomeCount = incomeItems.size,
            expenseCount = expenseItems.size,
            skippedRows = skipped
        )
    }

    /**
     * Export Cash Tracker items to a real multi-sheet Excel workbook (.xlsx):
     * - Summary sheet: metadata, filters, transaction count, inflow, outflow, net movement, category subtotals.
     * - Transactions sheet: frozen header, stable ID, date, time, direction, purpose, category, classification, sanitized notes, source, inflow/outflow numeric cells.
     */
    suspend fun exportCashTrackerExcel(
        items: List<ShopExpenseEntity>,
        periodTitle: String = "All Transactions",
        fromDate: Long? = null,
        toDate: Long? = null,
        activeFiltersSummary: String = "All Categories, Active Transactions"
    ): Intent = withContext(Dispatchers.IO) {
        val workbook = XSSFWorkbook()
        val dataFormat = workbook.createDataFormat()
        val currencyFormatIndex = dataFormat.getFormat("#,##,##0.00")

        // ── STYLES ──────────────────────────────────────
        val headerFont = workbook.createFont().apply {
            bold = true
            fontHeightInPoints = 11.toShort()
            color = IndexedColors.WHITE.index
        }

        val headerStyle = workbook.createCellStyle().apply {
            setFont(headerFont)
            fillForegroundColor = IndexedColors.DARK_BLUE.index
            fillPattern = FillPatternType.SOLID_FOREGROUND
            alignment = HorizontalAlignment.CENTER
            verticalAlignment = VerticalAlignment.CENTER
            borderBottom = BorderStyle.THIN
            borderTop = BorderStyle.THIN
            borderLeft = BorderStyle.THIN
            borderRight = BorderStyle.THIN
        }

        val normalStyle = workbook.createCellStyle().apply {
            borderBottom = BorderStyle.THIN
            borderTop = BorderStyle.THIN
            borderLeft = BorderStyle.THIN
            borderRight = BorderStyle.THIN
        }

        val normalNumberStyle = workbook.createCellStyle().apply {
            setDataFormat(currencyFormatIndex)
```

```
            alignment = HorizontalAlignment.RIGHT
            borderBottom = BorderStyle.THIN
            borderTop = BorderStyle.THIN
            borderLeft = BorderStyle.THIN
            borderRight = BorderStyle.THIN
        }

        val incomeNumberStyle = workbook.createCellStyle().apply {
            setDataFormat(currencyFormatIndex)
            alignment = HorizontalAlignment.RIGHT
            val f = workbook.createFont().apply {
                color = IndexedColors.DARK_GREEN.index
                bold = true
            }
            setFont(f)
            borderBottom = BorderStyle.THIN
            borderTop = BorderStyle.THIN
            borderLeft = BorderStyle.THIN
            borderRight = BorderStyle.THIN
        }

        val expenseNumberStyle = workbook.createCellStyle().apply {
            setDataFormat(currencyFormatIndex)
            alignment = HorizontalAlignment.RIGHT
            val f = workbook.createFont().apply {
                color = IndexedColors.DARK_RED.index
                bold = true
            }
            setFont(f)
            borderBottom = BorderStyle.THIN
            borderTop = BorderStyle.THIN
            borderLeft = BorderStyle.THIN
            borderRight = BorderStyle.THIN
        }

        val totalStyle = workbook.createCellStyle().apply {
            val f = workbook.createFont().apply {
                bold = true
                fontHeightInPoints = 11.toShort()
            }
            setFont(f)
            setDataFormat(currencyFormatIndex)
            fillForegroundColor = IndexedColors.GREY_25_PERCENT.index
            fillPattern = FillPatternType.SOLID_FOREGROUND
            borderBottom = BorderStyle.MEDIUM
            borderTop = BorderStyle.MEDIUM
            borderLeft = BorderStyle.THIN
            borderRight = BorderStyle.THIN
        }

        val sortedItems = items.sortedBy { it.timestamp }
        val activeItems = sortedItems.filter { !it.isVoided }
        val totalInflowPaise = activeItems.filter { it.isIncome }.sumOf { it.amountPaise }
        val totalOutflowPaise = activeItems.filter { !it.isIncome }.sumOf { it.amountPaise }
        val netMovementPaise = totalInflowPaise - totalOutflowPaise

        val dateFormat = SimpleDateFormat("dd-MMM-yyyy", Locale.US)
        val timeFormat = SimpleDateFormat("hh:mm a", Locale.US)

        // ── SHEET 1: SUMMARY ──────────────────────────────
        val summarySheet = workbook.createSheet("Summary")

        // Title
        val sRow0 = summarySheet.createRow(0)
        sRow0.createCell(0).setCellValue("Ali Educational - Cash In & Out Ledger Report")
        sRow0.getCell(0).cellStyle = workbook.createCellStyle().apply {
            val f = workbook.createFont().apply { bold = true; fontHeightInPoints = 14.toShort() }
            setFont(f)
        }

        val sRow1 = summarySheet.createRow(1)
        sRow1.createCell(0).setCellValue("Notice: Filtered ledger export (not a complete backup or profit
    statement)")

        var sRowIdx = 3
        fun addSummaryPair(label: String, value: String) {
            val r = summarySheet.createRow(sRowIdx++)
            r.createCell(0).setCellValue(label)
            r.createCell(1).setCellValue(value)
        }
        fun addSummaryNumber(label: String, rupees: Double) {
            val r = summarySheet.createRow(sRowIdx++)
            r.createCell(0).setCellValue(label)
            val c = r.createCell(1)
            c.setCellValue(rupees)
            c.cellStyle = normalNumberStyle
        }

```

```
        val nowStr = SimpleDateFormat("dd MMM yyyy, hh:mm a", Locale.US).format(Date())
        addSummaryPair("Export Generation Time:", nowStr)
        addSummaryPair("Selected Period:", periodTitle)
        addSummaryPair("Active Filters:", activeFiltersSummary)
        addSummaryPair("Matching Active Transactions:", "${activeItems.size}")
        addSummaryNumber("Selected Inflow Total (₹):", totalInflowPaise / 100.0)
        addSummaryNumber("Selected Outflow Total (₹):", totalOutflowPaise / 100.0)
        addSummaryNumber("Selected Net Movement (₹):", netMovementPaise / 100.0)

        // Category Subtotals
        sRowIdx++
        val catHeaderRow = summarySheet.createRow(sRowIdx++)
        catHeaderRow.createCell(0).apply { setCellValue("Category"); cellStyle = headerStyle }
        catHeaderRow.createCell(1).apply { setCellValue("Inflow (₹)"); cellStyle = headerStyle }
        catHeaderRow.createCell(2).apply { setCellValue("Outflow (₹)"); cellStyle = headerStyle }
        catHeaderRow.createCell(3).apply { setCellValue("Net Movement (₹)"); cellStyle = headerStyle }

        val catGroups = activeItems.groupBy { it.category }
        catGroups.forEach { (cat, catList) ->
            val r = summarySheet.createRow(sRowIdx++)
            val catIn = catList.filter { it.isIncome }.sumOf { it.amountPaise } / 100.0
            val catOut = catList.filter { !it.isIncome }.sumOf { it.amountPaise } / 100.0
            val catNet = catIn - catOut
            r.createCell(0).apply { setCellValue(cat); cellStyle = normalStyle }
            r.createCell(1).apply { setCellValue(catIn); cellStyle = normalNumberStyle }
            r.createCell(2).apply { setCellValue(catOut); cellStyle = normalNumberStyle }
            r.createCell(3).apply { setCellValue(catNet); cellStyle = normalNumberStyle }
        }

        summarySheet.setColumnWidth(0, 26 * 256)
        summarySheet.setColumnWidth(1, 18 * 256)
        summarySheet.setColumnWidth(2, 18 * 256)
        summarySheet.setColumnWidth(3, 20 * 256)

        // ── SHEET 2: TRANSACTIONS ──────────────────────────
        val txSheet = workbook.createSheet("Transactions")
        txSheet.createFreezePane(0, 1)

        val txHeaders = listOf(
            "Tx ID",
            "Date",
            "Time",
            "Direction",
            "Purpose",
            "Category",
            "Classification",
            "Description / Notes",
            "Source / Destination",
            "Inflow (₹)",
            "Outflow (₹)",
            "Attachment",
            "Status"
        )

        val thRow = txSheet.createRow(0)
        thRow.heightInPoints = 24f
        txHeaders.forEachIndexed { i, h ->
            thRow.createCell(i).apply {
                setCellValue(h)
                cellStyle = headerStyle
            }
        }

        var tRowIdx = 1
        for (item in sortedItems) {
            val r = txSheet.createRow(tRowIdx++)
            val d = Date(item.timestamp)

            // 0. Tx ID
            r.createCell(0).apply { setCellValue(item.id.take(8)); cellStyle = normalStyle }
            // 1. Date
            r.createCell(1).apply { setCellValue(dateFormat.format(d)); cellStyle = normalStyle }
            // 2. Time
            r.createCell(2).apply { setCellValue(timeFormat.format(d)); cellStyle = normalStyle }
            // 3. Direction
            r.createCell(3).apply { setCellValue(if (item.isIncome) "Cash In" else "Cash Out"); cellStyle =
    normalStyle }
            // 4. Purpose
            val purpose = when {
                item.isSavingsTransfer -> "Savings Transfer (Pigmy)"
                item.isExpenseRefund -> "Expense Refund"
                item.isIncome -> "Income"
                else -> "Shop Expense"
            }
            r.createCell(4).apply { setCellValue(purpose); cellStyle = normalStyle }
            // 5. Category
            r.createCell(5).apply { setCellValue(item.category); cellStyle = normalStyle }
            // 6. Classification
            r.createCell(6).apply { setCellValue(item.classification); cellStyle = normalStyle }
```

```
506
507             // 7. Notes (sanitized against formula injection)
508             var rawNotes = item.notes ?: "-"
509             if (rawNotes.startsWith("=") || rawNotes.startsWith("+") || rawNotes.startsWith("-") ||
    rawNotes.startsWith("@")) {
510                 rawNotes = "'$rawNotes"
511             }
512             r.createCell(7).apply { setCellValue(rawNotes); cellStyle = normalStyle }
513
514             // 8. Payment Source
515             r.createCell(8).apply { setCellValue(item.paymentMethod); cellStyle = normalStyle }
516
517             // 9 & 10. Inflow & Outflow (Numeric cells!)
518             val amtRupees = item.amountPaise / 100.0
519             if (item.isIncome) {
520                 r.createCell(9).apply { setCellValue(amtRupees); cellStyle = incomeNumberStyle }
521                 r.createCell(10).apply { setCellValue(0.0); cellStyle = normalNumberStyle }
522             } else {
523                 r.createCell(9).apply { setCellValue(0.0); cellStyle = normalNumberStyle }
524                 r.createCell(10).apply { setCellValue(amtRupees); cellStyle = expenseNumberStyle }
525             }
526
527             // 11. Attachment
528             r.createCell(11).apply { setCellValue(if (!item.attachmentUri.isNullOrBlank()) "Yes" else "No");
     cellStyle = normalStyle }
529             // 12. Status
530             r.createCell(12).apply { setCellValue(if (item.isVoided) "VOIDED" else "Active"); cellStyle =
    normalStyle }
531         }
532
533         // Transactions Totals Row
534         val totalRow = txSheet.createRow(tRowIdx)
535         totalRow.heightInPoints = 22f
536         totalRow.createCell(0).apply { setCellValue("TOTAL ACTIVE"); cellStyle = totalStyle }
537         for (col in 1..8) {
538             totalRow.createCell(col).apply { setCellValue(""); cellStyle = totalStyle }
539         }
540         totalRow.createCell(9).apply { setCellValue(totalInflowPaise / 100.0); cellStyle = totalStyle }
541         totalRow.createCell(10).apply { setCellValue(totalOutflowPaise / 100.0); cellStyle = totalStyle }
542         totalRow.createCell(11).apply { setCellValue(""); cellStyle = totalStyle }
543         totalRow.createCell(12).apply { setCellValue("${activeItems.size} active"); cellStyle = totalStyle }
544
545         val txWidths = intArrayOf(
546             12 * 256, // ID
547             14 * 256, // Date
548             11 * 256, // Time
549             12 * 256, // Direction
550             20 * 256, // Purpose
551             18 * 256, // Category
552             16 * 256, // Classification
553             28 * 256, // Notes
554             16 * 256, // Source
555             16 * 256, // Inflow
556             16 * 256, // Outflow
557             12 * 256, // Attachment
558             12 * 256  // Status
559         )
560         txWidths.forEachIndexed { i, w -> txSheet.setColumnWidth(i, w) }
561
562         // Meaningful dated filename: Ali_Educational_Cash_YYYY-MM-DD_to_YYYY-MM-DD.xlsx
563         val dfFile = SimpleDateFormat("yyyy-MM-dd", Locale.US)
564         val fromStr = if (fromDate != null && fromDate > 0L) dfFile.format(Date(fromDate)) else "start"
565         val toStr = if (toDate != null && toDate < Long.MAX_VALUE) dfFile.format(Date(toDate)) else
    dfFile.format(Date())
566         val fileName = "Ali_Educational_Cash_${fromStr}_to_${toStr}.xlsx"
567
568         // 1. Save to public Documents/AliEducational/Exports (visible in My Files)
569         val exportFile = File(exportFolder, fileName)
570         FileOutputStream(exportFile).use { fos ->
571             workbook.write(fos)
572         }
573
574         val ctx = context ?: throw IllegalStateException("Context required for export")
575         val cacheFile = File(ctx.cacheDir, fileName)
576         exportFile.copyTo(cacheFile, overwrite = true)
577         workbook.close()
578
579         val shareUri = FileProvider.getUriForFile(
580             ctx,
581             "${ctx.packageName}.fileprovider",
582             cacheFile
583         )
584
585         val shareMessage = buildString {
586             appendLine("  *Ali Educational - Cash In & Out Ledger*")
587             appendLine("  Period: $periodTitle")
588             appendLine("  Total Inflow: ${CostCalculator.formatPaiseToRupees(totalInflowPaise)}")
589             appendLine("  Total Outflow: ${CostCalculator.formatPaiseToRupees(totalOutflowPaise)}")
```

```
            appendLine("   Selected Net Movement: ${CostCalculator.formatPaiseToRupees(netMovementPaise)}")
            appendLine("  Matching Active Transactions: ${activeItems.size}")
            appendLine("Complete spreadsheet (.xlsx) attached.")
        }

        Intent(Intent.ACTION_SEND).apply {
            type = "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet"
            putExtra(Intent.EXTRA_STREAM, shareUri)
            putExtra(Intent.EXTRA_SUBJECT, "Ali Educational Cash Ledger - $periodTitle")
            putExtra(Intent.EXTRA_TEXT, shareMessage)
            addFlags(Intent.FLAG_GRANT_READ_URI_PERMISSION)
        }
    }

    private fun getCellString(cell: Cell?): String {
        if (cell == null) return ""
        return try {
            when (cell.cellType) {
                CellType.STRING -> cell.stringCellValue
                CellType.NUMERIC -> {
                    if (DateUtil.isCellDateFormatted(cell)) {
                        SimpleDateFormat("dd-MM-yyyy", Locale.US).format(cell.dateCellValue)
                    } else {
                        val num = cell.numericCellValue
                        if (num == num.toLong().toDouble()) num.toLong().toString() else num.toString()
                    }
                }
                CellType.BOOLEAN -> cell.booleanCellValue.toString()
                CellType.FORMULA -> {
                    try {
                        cell.stringCellValue
                    } catch (e: Exception) {
                        try {
                            cell.numericCellValue.toString()
                        } catch (e2: Exception) {
                            ""
                        }
                    }
                }
                else -> ""
            }
        } catch (e: Exception) {
            ""
        }
    }

    private fun parseCellAmount(cell: Cell?): Double {
        if (cell == null) return 0.0
        return try {
            when (cell.cellType) {
                CellType.NUMERIC -> cell.numericCellValue
                CellType.STRING -> {
                    val raw = cell.stringCellValue
                        .replace("₹", "")
                        .replace("Rs.", "", ignoreCase = true)
                        .replace("Rs", "", ignoreCase = true)
                        .replace(",", "")
                        .trim()
                    raw.toDoubleOrNull() ?: 0.0
                }
                CellType.FORMULA -> {
                    try {
                        cell.numericCellValue
                    } catch (e: Exception) {
                        cell.stringCellValue.replace("₹", "").replace(",", "").trim().toDoubleOrNull() ?:
0.0
                    }
                }
                else -> 0.0
            }
        } catch (e: Exception) {
            0.0
        }
    }

    private fun parseCellDate(cell: Cell?): Long {
        if (cell == null) return System.currentTimeMillis()
        try {
            if (cell.cellType == CellType.NUMERIC && DateUtil.isCellDateFormatted(cell)) {
                return cell.dateCellValue.time
            }
            val str = getCellString(cell).trim()
            if (str.isBlank()) return System.currentTimeMillis()

            val formats = listOf(
                "dd-MM-yyyy HH:mm:ss",
                "dd/MM/yyyy HH:mm:ss",
                "yyyy-MM-dd HH:mm:ss",
```

```
                "dd-MM-yyyy HH:mm",
                "dd/MM/yyyy HH:mm",
                "dd-MMM-yyyy",
                "dd-MM-yyyy",
                "dd/MM/yyyy",
                "yyyy-MM-dd",
                "d/M/yyyy",
                "d-M-yyyy",
                "d/M/yy",
                "d-M-yy"
            )

            for (pattern in formats) {
                try {
                    val sdf = SimpleDateFormat(pattern, Locale.US).apply { isLenient = false }
                    val d = sdf.parse(str)
                    if (d != null) return d.time
                } catch (ignored: Exception) {}
            }
        } catch (ignored: Exception) {}
        return System.currentTimeMillis()
    }

    private fun isRowEmpty(row: Row): Boolean {
        for (c in 0 until row.lastCellNum) {
            val cell = row.getCell(c)
            if (cell != null && cell.cellType != CellType.BLANK && getCellString(cell).isNotBlank()) {
                return false
            }
        }
        return true
    }
}
```

app\src\main\java\com\alieducational\app\data\service\ImportExportService.kt

```kotlin
package com.alieducational.app.data.service

import com.alieducational.app.domain.calculation.CostCalculator

import android.content.Context
import android.net.Uri
import com.alieducational.app.data.local.dao.ImportBatchDao
import com.alieducational.app.data.local.dao.ProductDao
import com.alieducational.app.data.local.dao.StockMovementDao
import com.alieducational.app.data.local.entity.ImportBatchEntity
import com.alieducational.app.data.local.entity.ProductEntity
import com.alieducational.app.data.local.entity.StockMovementEntity
import dagger.hilt.android.qualifiers.ApplicationContext
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.withContext
import org.apache.poi.ss.usermodel.CellType
import org.apache.poi.ss.usermodel.Row
import org.apache.poi.ss.usermodel.WorkbookFactory
import java.io.BufferedReader
import java.io.InputStream
import java.io.InputStreamReader
import java.util.Locale
import javax.inject.Inject
import javax.inject.Singleton

data class SpreadsheetPreview(
    val fileName: String,
    val sheetNames: List<String> = emptyList(),
    val selectedSheetIndex: Int = 0,
    val headers: List<String> = emptyList(),
    val previewRows: List<List<String>> = emptyList(),
    val totalEstimatedRows: Int = 0,
    val errorMessage: String? = null,
    val isCashLedger: Boolean = false
)

data class ColumnMapping(
    val nameIndex: Int = -1,
    val categoryIndex: Int = -1,
    val brandIndex: Int = -1,
    val skuIndex: Int = -1,
    val barcodeIndex: Int = -1,
    val baseUnitIndex: Int = -1,
    val packSizeIndex: Int = -1,
    val openingQtyIndex: Int = -1,
    val purchaseCostIndex: Int = -1,
    val sellingPricePieceIndex: Int = -1,
    val sellingPricePackIndex: Int = -1,
    val lowStockIndex: Int = -1
)

data class ImportResult(
    val totalRead: Int,
    val importedCount: Int,
    val skippedDuplicatesCount: Int,
    val updatedCount: Int = 0,
    val updatedDetails: List<String> = emptyList(),
    val skippedDetails: List<String> = emptyList(),
    val errors: List<String> = emptyList(),
    val isSuccess: Boolean = true,
    val failureReason: String? = null
)

@Singleton
class ImportExportService @Inject constructor(
    @ApplicationContext private val context: Context,
    private val productDao: ProductDao,
    private val movementDao: StockMovementDao,
    private val importBatchDao: ImportBatchDao
) {

    suspend fun previewFile(
        uri: Uri,
        fileName: String,
        selectedSheetIndex: Int = 0
    ): SpreadsheetPreview = withContext(Dispatchers.IO) {
        try {
            val stream = context.contentResolver.openInputStream(uri)
                ?: return@withContext SpreadsheetPreview(
                    fileName = fileName,
                    errorMessage = "Cannot access the selected file. Please select the file again."
                )
```

```
83
84              if (fileName.endsWith(".csv", ignoreCase = true)) {
85                  previewCsv(stream, fileName)
86              } else {
87                  previewExcel(stream, fileName, selectedSheetIndex)
88              }
89          } catch (e: Exception) {
90              val friendlyMsg = mapExceptionToMessage(e)
91              SpreadsheetPreview(
92                  fileName = fileName,
93                  errorMessage = friendlyMsg
94              )
95          }
96      }
97
98      private fun mapExceptionToMessage(e: Exception): String {
99          val msg = e.localizedMessage ?: e.javaClass.simpleName
100         return when {
101             msg.contains("password", ignoreCase = true) || msg.contains("Encrypted", ignoreCase = true) ->
102                 "This file is password-protected. Please open it on your computer or phone, remove the
    password, and try again."
103             msg.contains("NotOfficeXmlFileException", ignoreCase = true) ||
    msg.contains("NotOLE2FileException", ignoreCase = true) ->
104                 "The file format is not a recognized Excel spreadsheet. Please ensure it is a valid .xlsx,
    .xls, or .csv file."
105             msg.contains("EmptyFileException", ignoreCase = true) || msg.contains("empty", ignoreCase =
    true) ->
106                 "The selected spreadsheet is completely empty. Please select a file that contains column
    headings and product rows."
107             else ->
108                 "Could not read spreadsheet: $msg. Please verify the file is not corrupted."
109         }
110     }
111
112     private fun isCashLedgerHeaders(headers: List<String>, sheetOrFileName: String = ""): Boolean {
113         val hLower = headers.map { it.trim().lowercase(Locale.ROOT) }
114         val hasIncomeOrExpense = hLower.any {
115             it.contains("income") || it.contains("expense") || it.contains("cash in") ||
116                     it.contains("cash out") || it.contains("credit") || it.contains("debit")
117         }
118         val hasBalanceOrDate = hLower.any {
119             it.contains("balance") || it.contains("notes") || it.contains("particular") ||
    it.contains("date")
120         }
121         val isTransactionsContext = sheetOrFileName.contains("transaction", ignoreCase = true) ||
122                 sheetOrFileName.contains("cash", ignoreCase = true)
123         return (hasIncomeOrExpense && (hasBalanceOrDate || isTransactionsContext)) || (isTransactionsContext
     && hasIncomeOrExpense)
124     }
125
126     private fun previewCsv(stream: InputStream, fileName: String): SpreadsheetPreview {
127         val reader = BufferedReader(InputStreamReader(stream))
128         val lines = mutableListOf<List<String>>()
129         var line: String?
130
131         var lineCount = 0
132         while (reader.readLine().also { line = it } != null) {
133             lineCount++
134             val parts = parseCsvLine(line!!)
135             if (parts.any { it.isNotBlank() }) {
136                 lines.add(parts)
137             }
138             if (lines.size >= 15) break
139         }
140         reader.close()
141
142         if (lines.isEmpty()) {
143             return SpreadsheetPreview(
144                 fileName = fileName,
145                 errorMessage = "CSV file is empty. Please provide a CSV with header columns (e.g. Name,
    Category, Price, Stock)."
146             )
147         }
148
149         // Detect header row (first row with multiple non-blank items)
150         var headerIndex = 0
151         for (i in 0 until minOf(5, lines.size)) {
152             if (lines[i].count { it.isNotBlank() } >= 2) {
153                 headerIndex = i
154                 break
155             }
156         }
157
158         val headers = lines[headerIndex]
159         val preview = if (lines.size > headerIndex + 1) lines.subList(headerIndex + 1, lines.size) else
    emptyList()
160
161         if (isCashLedgerHeaders(headers, fileName)) {
```

```
            return SpreadsheetPreview(
                fileName = fileName,
                sheetNames = listOf("CSV Data"),
                selectedSheetIndex = 0,
                headers = headers,
                previewRows = preview,
                totalEstimatedRows = lineCount,
                isCashLedger = true,
                errorMessage = "This file contains Cash In/Out transactions, not product stock. Import it from Cash In/Out Tracker."
            )
        }

        return SpreadsheetPreview(
            fileName = fileName,
            sheetNames = listOf("CSV Data"),
            selectedSheetIndex = 0,
            headers = headers,
            previewRows = preview,
            totalEstimatedRows = lineCount
        )
    }

    private fun previewExcel(stream: InputStream, fileName: String, sheetIndex: Int): SpreadsheetPreview {
        val workbook = WorkbookFactory.create(stream)
        val numSheets = workbook.numberOfSheets
        if (numSheets == 0) {
            workbook.close()
            stream.close()
            return SpreadsheetPreview(fileName = fileName, errorMessage = "Excel workbook contains no worksheets.")
        }

        val sheetNames = (0 until numSheets).map { workbook.getSheetName(it) }
        val activeSheetIndex = if (sheetIndex in 0 until numSheets) sheetIndex else 0
        val sheet = workbook.getSheetAt(activeSheetIndex)

        val allRows = mutableListOf<Pair<Int, List<String>>>()
        val lastRow = sheet.lastRowNum
        for (i in 0..minOf(lastRow, 25)) {
            val row = sheet.getRow(i) ?: continue
            val rowValues = (0 until row.lastCellNum).map { getCellString(row, it) }
            if (rowValues.any { it.isNotBlank() }) {
                allRows.add(Pair(i, rowValues))
            }
        }
        workbook.close()
        stream.close()

        if (allRows.isEmpty()) {
            return SpreadsheetPreview(
                fileName = fileName,
                sheetNames = sheetNames,
                selectedSheetIndex = activeSheetIndex,
                errorMessage = "Sheet '${sheetNames[activeSheetIndex]}' contains no data rows."
            )
        }

        // Find the best header row (e.g. Row with maximum non-blank columns, or explicit keywords)
        var bestRowIdx = 0
        var maxColCount = 0
        for (idx in 0 until minOf(allRows.size, 10)) {
            val cols = allRows[idx].second
            val nonBlank = cols.count { it.isNotBlank() }
            val hasKeywords = cols.any { c ->
                val l = c.lowercase(Locale.ROOT)
                l.contains("name") || l.contains("product") || l.contains("item") ||
                        l.contains("price") || l.contains("cost") || l.contains("stock") ||
                        l.contains("sku") || l.contains("barcode") || l.contains("income") ||
                        l.contains("expense") || l.contains("date")
            }
            if (hasKeywords && nonBlank >= 2) {
                bestRowIdx = idx
                break
            }
            if (nonBlank > maxColCount) {
                maxColCount = nonBlank
                bestRowIdx = idx
            }
        }

        val headers = allRows[bestRowIdx].second
        val preview = if (allRows.size > bestRowIdx + 1) {
            allRows.subList(bestRowIdx + 1, allRows.size).map { it.second }
        } else emptyList()

        if (isCashLedgerHeaders(headers, sheetNames[activeSheetIndex]) || isCashLedgerHeaders(headers, fileName)) {
```

```
247             return SpreadsheetPreview(
248                 fileName = fileName,
249                 sheetNames = sheetNames,
250                 selectedSheetIndex = activeSheetIndex,
251                 headers = headers,
252                 previewRows = preview,
253                 totalEstimatedRows = lastRow + 1,
254                 isCashLedger = true,
255                 errorMessage = "This file contains Cash In/Out transactions, not product stock. Import it
    from Cash In/Out Tracker."
256             )
257         }
258 
259         return SpreadsheetPreview(
260             fileName = fileName,
261             sheetNames = sheetNames,
262             selectedSheetIndex = activeSheetIndex,
263             headers = headers,
264             previewRows = preview,
265             totalEstimatedRows = lastRow + 1
266         )
267     }
268 
269     suspend fun importProducts(
270         uri: Uri,
271         fileName: String,
272         sheetIndex: Int,
273         mapping: ColumnMapping,
274         updateExisting: Boolean = false
275     ): ImportResult = withContext(Dispatchers.IO) {
276         val stream = try {
277             context.contentResolver.openInputStream(uri)
278                 ?: return@withContext ImportResult(0, 0, 0, isSuccess = false, failureReason = "Cannot open
    file stream")
279         } catch (e: Exception) {
280             return@withContext ImportResult(0, 0, 0, isSuccess = false, failureReason =
    mapExceptionToMessage(e))
281         }
282 
283         val rows = try {
284             if (fileName.endsWith(".csv", ignoreCase = true)) {
285                 readAllCsv(stream)
286             } else {
287                 readAllExcel(stream, sheetIndex)
288             }
289         } catch (e: Exception) {
290             return@withContext ImportResult(0, 0, 0, isSuccess = false, failureReason =
    mapExceptionToMessage(e))
291         }
292 
293         if (rows.size <= 1) {
294             return@withContext ImportResult(0, 0, 0, isSuccess = false, failureReason = "No product rows
    found to import.")
295         }
296 
297         // Safety check: block writing products if file is detected as cash-ledger
298         val firstFewRows = rows.take(minOf(10, rows.size))
299         for (r in firstFewRows) {
300             if (isCashLedgerHeaders(r, fileName)) {
301                 return@withContext ImportResult(
302                     totalRead = 0,
303                     importedCount = 0,
304                     skippedDuplicatesCount = 0,
305                     isSuccess = false,
306                     failureReason = "This file contains Cash In/Out transactions, not product stock. Import
    it from Cash In/Out Tracker."
307                 )
308             }
309         }
310 
311         var imported = 0
312         var updatedCount = 0
313         var skippedDuplicates = 0
314         val updatedDetails = mutableListOf<String>()
315         val skippedDetails = mutableListOf<String>()
316         val errors = mutableListOf<String>()
317 
318         // Process rows atomically one by one
319         for (i in 1 until rows.size) {
320             val row = rows[i]
321             val rowNum = i + 1
322             val name = if (mapping.nameIndex in row.indices) row[mapping.nameIndex].trim() else ""
323             if (name.isBlank()) {
324                 // If entire row is blank, skip silently; otherwise note missing name
325                 if (row.any { it.isNotBlank() }) {
326                     errors.add("Row $rowNum: Skipped because product name was empty.")
327                 }
328                 continue
```

```
            }

            val sku = if (mapping.skuIndex in row.indices) row[mapping.skuIndex].trim().ifBlank { null } else null
            val barcode = if (mapping.barcodeIndex in row.indices) row[mapping.barcodeIndex].trim().ifBlank { null } else null

            // Detect existing product by SKU, Barcode, or exact Name
            val existing = (if (sku != null) productDao.getProductBySku(sku) else null)
                ?: (if (barcode != null) productDao.getProductByBarcode(barcode) else null)
                ?: productDao.getProductByName(name)

            val category = if (mapping.categoryIndex in row.indices) row[mapping.categoryIndex].trim().ifBlank { "General" } else "General"
            val brand = if (mapping.brandIndex in row.indices) row[mapping.brandIndex].trim().ifBlank { null } else null
            val baseUnit = if (mapping.baseUnitIndex in row.indices) row[mapping.baseUnitIndex].trim().ifBlank { "piece" } else "piece"
            val packSize = if (mapping.packSizeIndex in row.indices) row[mapping.packSizeIndex].trim().toIntOrNull()?.coerceAtLeast(1) ?: 1 else 1

            val openingQty = if (mapping.openingQtyIndex in row.indices) row[mapping.openingQtyIndex].trim().toIntOrNull() ?: 0 else 0
            val purchaseCostPaise = if (mapping.purchaseCostIndex in row.indices) parseCurrencyToPaise(row[mapping.purchaseCostIndex]) else 0L
            val sellingPiecePaise = if (mapping.sellingPricePieceIndex in row.indices) parseCurrencyToPaise(row[mapping.sellingPricePieceIndex]) else 0L
            val sellingPackPaise = if (mapping.sellingPricePackIndex in row.indices) parseCurrencyToPaise(row[mapping.sellingPricePackIndex]) else sellingPiecePaise * packSize
            val lowStock = if (mapping.lowStockIndex in row.indices) row[mapping.lowStockIndex].trim().toIntOrNull() ?: 5 else 5

            if (existing != null) {
                if (!updateExisting) {
                    skippedDuplicates++
                    if (skippedDetails.size < 10) {
                        val matchField = if (sku != null && existing.sku == sku) "SKU '$sku'"
                        else if (barcode != null && existing.barcode == barcode) "Barcode '$barcode'"
                        else "Name '${existing.name}'"
                        skippedDetails.add("Row $rowNum: '$name' (matches existing $matchField)")
                    }
                    continue
                } else {
                    // Update existing item
                    try {
                        val newQty = existing.currentSellableStock + openingQty
                        val newCost = if (purchaseCostPaise > 0) {
                            if (existing.currentSellableStock > 0 && openingQty > 0) {
                                val totalVal = (existing.currentSellableStock.toLong() * existing.movingAvgCostPaise) + (openingQty.toLong() * purchaseCostPaise)
                                totalVal / newQty.coerceAtLeast(1)
                            } else {
                                purchaseCostPaise
                            }
                        } else {
                            existing.movingAvgCostPaise
                        }

                        val updatedProduct = existing.copy(
                            currentSellableStock = newQty,
                            movingAvgCostPaise = newCost,
                            sellingPricePiecePaise = if (sellingPiecePaise > 0) sellingPiecePaise else existing.sellingPricePiecePaise,
                            sellingPricePackPaise = if (sellingPackPaise > 0) sellingPackPaise else existing.sellingPricePackPaise,
                            sku = existing.sku ?: sku,
                            barcode = existing.barcode ?: barcode,
                            category = if (existing.category == "General" && category != "General") category else existing.category,
                            brand = existing.brand ?: brand,
                            updatedAt = System.currentTimeMillis()
                        )
                        productDao.updateProduct(updatedProduct)

                        if (openingQty > 0) {
                            movementDao.insertMovement(
                                StockMovementEntity(
                                    productId = existing.id,
                                    type = "STOCK_ADJUSTMENT",
                                    quantityChangeBase = openingQty,
                                    unitCostPaise = purchaseCostPaise,
                                    reason = "Catalog Import: Added +$openingQty from $fileName"
                                )
                            )
                        }

                        updatedCount++
                        if (updatedDetails.size < 10) {
```

```
                            updatedDetails.add("Row $rowNum: '$name' (Stock: ${existing.currentSellableStock} -> $newQty)")
                        }
                    } catch (e: Exception) {
                        errors.add("Row $rowNum ('$name'): Failed to update: ${e.localizedMessage}")
                    }
                    continue
                }
            }

            val product = ProductEntity(
                name = name,
                category = category,
                brand = brand,
                sku = sku,
                barcode = barcode,
                baseUnit = baseUnit,
                packSize = packSize,
                currentSellableStock = openingQty,
                movingAvgCostPaise = purchaseCostPaise,
                sellingPricePiecePaise = sellingPiecePaise,
                sellingPricePackPaise = sellingPackPaise,
                lowStockThreshold = lowStock
            )

            try {
                productDao.insertProduct(product)
                if (openingQty > 0) {
                    movementDao.insertMovement(
                        StockMovementEntity(
                            productId = product.id,
                            type = "OPENING_STOCK",
                            quantityChangeBase = openingQty,
                            unitCostPaise = purchaseCostPaise,
                            reason = "Catalog Import: $fileName"
                        )
                    )
                }
                imported++
            } catch (e: Exception) {
                errors.add("Row $rowNum ('$name'): ${e.localizedMessage}")
            }
        }

        // Record import batch history
        try {
            importBatchDao.insertBatch(
                ImportBatchEntity(
                    fileName = fileName,
                    totalRows = rows.size - 1,
                    importedCount = imported + updatedCount,
                    skippedCount = skippedDuplicates
                )
            )
        } catch (_: Exception) {}

        return@withContext ImportResult(
            totalRead = rows.size - 1,
            importedCount = imported,
            skippedDuplicatesCount = skippedDuplicates,
            updatedCount = updatedCount,
            updatedDetails = updatedDetails,
            skippedDetails = skippedDetails,
            errors = errors,
            isSuccess = true
        )
    }

    private fun readAllCsv(stream: InputStream): List<List<String>> {
        val reader = BufferedReader(InputStreamReader(stream))
        val rows = mutableListOf<List<String>>()
        var line: String?
        while (reader.readLine().also { line = it } != null) {
            val parts = parseCsvLine(line!!)
            if (parts.any { it.isNotBlank() }) {
                rows.add(parts)
            }
        }
        reader.close()
        return rows
    }

    private fun readAllExcel(stream: InputStream, sheetIndex: Int): List<List<String>> {
        val workbook = WorkbookFactory.create(stream)
        val activeSheetIndex = if (sheetIndex in 0 until workbook.numberOfSheets) sheetIndex else 0
        val sheet = workbook.getSheetAt(activeSheetIndex)
        val rows = mutableListOf<List<String>>()

        for (i in 0..sheet.lastRowNum) {
```

```
            val row = sheet.getRow(i) ?: continue
            val rowValues = (0 until row.lastCellNum).map { getCellString(row, it) }
            if (rowValues.any { it.isNotBlank() }) {
                rows.add(rowValues)
            }
        }
        workbook.close()
        stream.close()
        return rows
    }

    private fun getCellString(row: Row, colIndex: Int): String {
        val cell = row.getCell(colIndex) ?: return ""
        return try {
            when (cell.cellType) {
                CellType.STRING -> cell.stringCellValue?.trim() ?: ""
                CellType.NUMERIC -> {
                    val num = cell.numericCellValue
                    if (num == num.toLong().toDouble()) num.toLong().toString() else num.toString()
                }
                CellType.BOOLEAN -> cell.booleanCellValue.toString()
                CellType.FORMULA -> {
                    try {
                        cell.stringCellValue?.trim() ?: cell.numericCellValue.toString()
                    } catch (_: Exception) {
                        cell.numericCellValue.toString()
                    }
                }
                else -> ""
            }
        } catch (_: Exception) {
            ""
        }
    }

    private fun parseCsvLine(line: String): List<String> {
        val result = mutableListOf<String>()
        var curVal = StringBuilder()
        var inQuotes = false

        for (c in line) {
            if (c == '\"') {
                inQuotes = !inQuotes
            } else if (c == ',' && !inQuotes) {
                result.add(curVal.toString().trim())
                curVal = StringBuilder()
            } else {
                curVal.append(c)
            }
        }
        result.add(curVal.toString().trim())
        return result
    }

    private fun parseCurrencyToPaise(str: String): Long {
        val clean = str.replace("₹", "").replace("Rs.", "", ignoreCase = true).replace("Rs", "", ignoreCase
= true).replace(",", "").trim()
        return CostCalculator.parseRupeesToPaise(clean) ?: 0L
    }
}
```

app\src\main\java\com\alieducational\app\data\service\ReceiptService.kt

```
package com.alieducational.app.data.service

import android.app.Activity
import android.content.Context
import android.content.Intent
import android.content.pm.PackageManager
import android.graphics.Bitmap
import android.graphics.BitmapFactory
import android.graphics.Canvas
import android.graphics.Color
import android.graphics.Paint
import android.graphics.Rect
import android.graphics.RectF
import android.graphics.Typeface
import android.graphics.pdf.PdfDocument
import android.net.Uri
import android.os.Bundle
import android.os.CancellationSignal
import android.os.ParcelFileDescriptor
import android.print.PageRange
import android.print.PrintAttributes
import android.print.PrintDocumentAdapter
import android.print.PrintDocumentInfo
import android.print.PrintManager
import androidx.core.content.FileProvider
import com.alieducational.app.R
import com.alieducational.app.data.local.entity.SaleEntity
import com.alieducational.app.data.local.entity.SaleLineEntity
import com.alieducational.app.domain.calculation.CostCalculator
import dagger.hilt.android.qualifiers.ApplicationContext
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.withContext
import java.io.File
import java.io.FileInputStream
import java.io.FileOutputStream
import java.text.SimpleDateFormat
import java.util.Date
import java.util.Locale
import javax.inject.Inject
import javax.inject.Singleton

@Singleton
class ReceiptService @Inject constructor(
    @ApplicationContext private val context: Context?
) {
    constructor() : this(null)

    /**
     * Generate short accompanying message text for WhatsApp or SMS.
     */
    fun generateReceiptSummaryText(sale: SaleEntity, lines: List<SaleLineEntity>): String {
        val dateFormat = SimpleDateFormat("dd MMM yyyy, hh:mm a", Locale.getDefault())
        val dateStr = dateFormat.format(Date(sale.timestamp))
        val sb = StringBuilder()

        sb.append("Ali Educational\n")
        sb.append("Stationery & Educational Supplies\n")
        sb.append("RECEIPT\n")
        sb.append("------------------------------\n")
        sb.append("Receipt No: ${sale.receiptNumber}\n")
        sb.append("Date: $dateStr\n")
        val customer = if (!sale.customerName.isNullOrBlank()) sale.customerName else "Walk-in Customer"
        sb.append("Customer: $customer\n")
        val paymentMode = when (sale.paymentMethod) {
            "CASH" -> "Cash"
            "UPI" -> "UPI"
            "CREDIT" -> "Credit"
            else -> sale.paymentMethod
        }
        sb.append("Payment: $paymentMode\n")
        sb.append("------------------------------\n")
        lines.forEach { line ->
            val unit = if (!line.unitLabel.isNullOrBlank()) {
                if (line.quantityBase == 1 && line.unitLabel.equals("copies", ignoreCase = true)) "copy"
                else if (line.quantityBase == 1 && line.unitLabel.equals("sheets", ignoreCase = true)) "sheet"
                else line.unitLabel
            } else if (line.isPack) {
                if (line.quantityBase == 1) "pack" else "packs"
            } else {
                if (line.quantityBase == 1) "pc" else "pcs"
            }
```

```
            val lineGross = if (line.lineGrossPaise > 0L) line.lineGrossPaise else line.quantityBase *
line.unitSellingPricePaise
            val lineTotal = CostCalculator.formatPaiseToRupees(lineGross)
            sb.append("• ${line.productName}: ${line.quantityBase} $unit @
${CostCalculator.formatPaiseToRupees(line.unitSellingPricePaise)} = $lineTotal\n")
        }
        sb.append("------------------------------\n")
        sb.append("Subtotal: ${CostCalculator.formatPaiseToRupees(sale.grossTotalPaise)}\n")
        if (sale.discountValuePaise > 0) {
            val pctStr = if (sale.discountPercent > 0.0) {
                if (sale.discountPercent % 1.0 == 0.0) String.format(Locale.ROOT, "%.0f%%",
sale.discountPercent) else String.format(Locale.ROOT, "%.1f%%", sale.discountPercent)
            } else ""
            val pctLabel = if (pctStr.isNotEmpty()) "Discount ($pctStr):" else "Discount:"
            sb.append("$pctLabel -${CostCalculator.formatPaiseToRupees(sale.discountValuePaise)}\n")
        }
        sb.append("TOTAL PAYABLE: ${CostCalculator.formatPaiseToRupees(sale.netTotalPaise)}\n")
        sb.append("Amount paid: ${CostCalculator.formatPaiseToRupees(sale.amountReceivedPaise)}\n")
        if (sale.changeGivenPaise > 0) {
            sb.append("Change: ${CostCalculator.formatPaiseToRupees(sale.changeGivenPaise)}\n")
        }
        if (sale.remainingCreditPaise > 0) {
            sb.append("Balance due: ${CostCalculator.formatPaiseToRupees(sale.remainingCreditPaise)}\n")
        } else {
            sb.append("Balance due: ₹0.00\n")
        }
        val badge = when {
            sale.isVoided -> "VOIDED"
            sale.remainingCreditPaise > 0L && sale.amountReceivedPaise > 0L -> "PARTIAL"
            sale.remainingCreditPaise > 0L -> "DUE"
            else -> "PAID"
        }
        sb.append("Status: $badge\n")
        sb.append("------------------------------\n")
        sb.append("Thank you for visiting Ali Educational!\n")

        return sb.toString()
    }

    /**
     * Backward-compatible helper for legacy text sharing.
     */
    fun generateReceiptText(sale: SaleEntity, lines: List<SaleLineEntity>, isUrdu: Boolean = false): String
{
        return generateReceiptSummaryText(sale, lines)
    }

    fun shareReceipt(receiptText: String) {
        val sendIntent = Intent().apply {
            action = Intent.ACTION_SEND
            putExtra(Intent.EXTRA_TEXT, receiptText)
            type = "text/plain"
            addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)
        }
        val chooser = Intent.createChooser(sendIntent, "Share Receipt")
        chooser.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)
        context?.startActivity(chooser)
    }

    private data class ProcessedItem(
        val entity: SaleLineEntity,
        val displayNameLines: List<String>,
        val qtyStr: String,
        val rateStr: String,
        val amountStr: String,
        val height: Float
    )

    /**
     * Creates a high-quality branded PDF receipt document on white background.
     * Features:
     * - Approved monochrome Ali Educational logo with proper vertical spacing
     * - "ALI EDUCATIONAL" and "Stationery & Educational Supplies" centered
     * - Clean "RECEIPT" document title without unwanted GST statements
     * - Consistent 2-column metadata with right-aligned values
     * - Clean tabular items layout with right-aligned amounts and wrapped descriptions
     * - Single discount deduction shown in the totals section
     * - Light grey shaded "TOTAL PAYABLE" banner
     * - Centered status badge: [ PAID ], [ PARTIAL ], [ DUE ], [ VOIDED ]
     * - Responsive text measurement and multi-page pagination for long bills
     * - Snug fit dynamically sized page for short receipts with zero trailing void
     */
    suspend fun createReceiptPdf(sale: SaleEntity, lines: List<SaleLineEntity>): File =
withContext(Dispatchers.IO) {
        val document = PdfDocument()

        val titlePaint = Paint(Paint.ANTI_ALIAS_FLAG).apply {
            color = Color.BLACK
```

```
            textSize = 17f
            isFakeBoldText = true
            typeface = Typeface.create(Typeface.SANS_SERIF, Typeface.BOLD)
        }

        val subtitlePaint = Paint(Paint.ANTI_ALIAS_FLAG).apply {
            color = Color.rgb(35, 45, 60)
            textSize = 10.5f
            typeface = Typeface.create(Typeface.SANS_SERIF, Typeface.NORMAL)
        }

        val docTypePaint = Paint(Paint.ANTI_ALIAS_FLAG).apply {
            color = Color.rgb(75, 85, 99)
            textSize = 9.5f
            letterSpacing = 0.18f
            typeface = Typeface.create(Typeface.SANS_SERIF, Typeface.NORMAL)
        }

        val metaKeyPaint = Paint(Paint.ANTI_ALIAS_FLAG).apply {
            color = Color.rgb(55, 65, 81)
            textSize = 10f
            typeface = Typeface.create(Typeface.SANS_SERIF, Typeface.NORMAL)
        }

        val metaValPaint = Paint(Paint.ANTI_ALIAS_FLAG).apply {
            color = Color.BLACK
            textSize = 10f
            typeface = Typeface.create(Typeface.SANS_SERIF, Typeface.NORMAL)
        }

        val linePaint = Paint(Paint.ANTI_ALIAS_FLAG).apply {
            color = Color.rgb(209, 213, 219)
            strokeWidth = 0.8f
            style = Paint.Style.STROKE
        }

        val tableHeaderPaint = Paint(Paint.ANTI_ALIAS_FLAG).apply {
            color = Color.BLACK
            textSize = 10.5f
            isFakeBoldText = true
            typeface = Typeface.create(Typeface.SANS_SERIF, Typeface.BOLD)
        }

        val itemTextPaint = Paint(Paint.ANTI_ALIAS_FLAG).apply {
            color = Color.BLACK
            textSize = 10f
            typeface = Typeface.create(Typeface.SANS_SERIF, Typeface.NORMAL)
        }

        val itemSubPaint = Paint(Paint.ANTI_ALIAS_FLAG).apply {
            color = Color.rgb(55, 65, 81)
            textSize = 10f
            typeface = Typeface.create(Typeface.SANS_SERIF, Typeface.NORMAL)
        }

        val summaryLabelPaint = Paint(Paint.ANTI_ALIAS_FLAG).apply {
            color = Color.rgb(55, 65, 81)
            textSize = 10f
            typeface = Typeface.create(Typeface.SANS_SERIF, Typeface.NORMAL)
        }

        val summaryValPaint = Paint(Paint.ANTI_ALIAS_FLAG).apply {
            color = Color.BLACK
            textSize = 10f
            typeface = Typeface.create(Typeface.SANS_SERIF, Typeface.NORMAL)
        }

        val totalBgPaint = Paint(Paint.ANTI_ALIAS_FLAG).apply {
            color = Color.rgb(243, 244, 246)
            style = Paint.Style.FILL
        }

        val totalLabelPaint = Paint(Paint.ANTI_ALIAS_FLAG).apply {
            color = Color.BLACK
            textSize = 11.5f
            isFakeBoldText = true
            typeface = Typeface.create(Typeface.SANS_SERIF, Typeface.BOLD)
        }

        val totalValPaint = Paint(Paint.ANTI_ALIAS_FLAG).apply {
            color = Color.BLACK
            textSize = 13.5f
            isFakeBoldText = true
            typeface = Typeface.create(Typeface.SANS_SERIF, Typeface.BOLD)
        }

        val badgeBorderPaint = Paint(Paint.ANTI_ALIAS_FLAG).apply {
            color = if (sale.isVoided) Color.rgb(220, 38, 38) else Color.BLACK
```

```
            strokeWidth = 1f
            style = Paint.Style.STROKE
        }

        val badgeTextPaint = Paint(Paint.ANTI_ALIAS_FLAG).apply {
            color = if (sale.isVoided) Color.rgb(220, 38, 38) else Color.BLACK
            textSize = 9.5f
            isFakeBoldText = true
            letterSpacing = 0.2f
        }

        val footerPaint = Paint(Paint.ANTI_ALIAS_FLAG).apply {
            color = Color.BLACK
            textSize = 11.5f
            isFakeBoldText = true
            typeface = Typeface.create(Typeface.SANS_SERIF, Typeface.BOLD)
        }

        fun wrapText(text: String, paint: Paint, maxWidth: Float): List<String> {
            if (paint.measureText(text) <= maxWidth) return listOf(text)
            val words = text.split(" ")
            val result = mutableListOf<String>()
            var current = ""
            for (word in words) {
                val test = if (current.isEmpty()) word else "$current $word"
                if (paint.measureText(test) <= maxWidth) {
                    current = test
                } else {
                    if (current.isNotEmpty()) result.add(current)
                    current = word
                }
            }
            if (current.isNotEmpty()) result.add(current)
            return if (result.isEmpty()) listOf(text) else result
        }

        // Layout constants
        val pageWidth = 380
        val margin = 24f
        val qtyColX = 230f
        val rateColX = 288f
        val rightColX = pageWidth - margin
        val summaryX = 170f

        // Process item lines and measure heights
        val processedItems = lines.map { line ->
            val desc = line.productName.ifBlank { "Item" }
            val wrapped = wrapText(desc, itemTextPaint, 160f)
            val unitStr = if (!line.unitLabel.isNullOrBlank()) {
                if (line.quantityBase == 1 && line.unitLabel.equals("copies", ignoreCase = true)) "copy"
                else if (line.quantityBase == 1 && line.unitLabel.equals("sheets", ignoreCase = true))
"sheet"
                else line.unitLabel
            } else if (line.isPack) {
                if (line.quantityBase == 1) "pack" else "packs"
            } else {
                if (line.quantityBase == 1) "pc" else "pcs"
            }
            val qtyStr = "${line.quantityBase} $unitStr"
            val rateStr = CostCalculator.formatPaiseToRupees(line.unitSellingPricePaise)
            val lineGross = if (line.lineGrossPaise > 0L) line.lineGrossPaise else line.quantityBase *
line.unitSellingPricePaise
            val amountStr = CostCalculator.formatPaiseToRupees(lineGross)
            val rowHeight = 18f + (wrapped.size - 1) * 13f
            ProcessedItem(line, wrapped, qtyStr, rateStr, amountStr, rowHeight)
        }

        // Decode logo bitmap
        val logoBitmap = try {
            context?.let { BitmapFactory.decodeResource(it.resources, R.drawable.ali_logo_receipt_mono) }
        } catch (_: Exception) {
            null
        }

        val logoW = 74f
        val logoH = if (logoBitmap != null) (logoW * logoBitmap.height) / logoBitmap.width else 39f

        // Header height
        val fullHeaderHeight = 24f + logoH + 18f + 18f + 16f + 24f + (4 * 16f) + 8f + 16f + 18f + 8f + 16f
        val summaryBlockHeight = 16f + (if (sale.discountValuePaise > 0L) 16f else 0f) + 12f + 28f + 14f +
16f + 16f + 22f + 20f + 16f + 28f
        val totalItemsHeight = processedItems.sumOf { it.height.toDouble() }.toFloat()
        val totalEstimatedHeight = fullHeaderHeight + totalItemsHeight + summaryBlockHeight

        val MAX_SINGLE_PAGE_HEIGHT = 650f

        val dateFormat = SimpleDateFormat("dd MMM yyyy, hh:mm a", Locale.getDefault())
        val dateStr = dateFormat.format(Date(sale.timestamp))
```

```
        val customerName = if (!sale.customerName.isNullOrBlank()) sale.customerName else "Walk-in Customer"
        val paymentLabel = when (sale.paymentMethod) {
            "CASH" -> "Cash"
            "UPI" -> "UPI"
            "CREDIT" -> "Credit"
            else -> sale.paymentMethod
        }

        val badgeText = when {
            sale.isVoided -> "V O I D E D"
            sale.remainingCreditPaise > 0L && sale.amountReceivedPaise > 0L -> "P A R T I A L"
            sale.remainingCreditPaise > 0L -> "D U E"
            else -> "P A I D"
        }

        // Draw Helper Functions
        fun drawReceiptHeader(canvas: Canvas): Float {
            var y = 24f
            if (logoBitmap != null) {
                val logoX = (pageWidth - logoW) / 2f
                val srcRect = Rect(0, 0, logoBitmap.width, logoBitmap.height)
                val dstRect = RectF(logoX, y, logoX + logoW, y + logoH)
                canvas.drawBitmap(logoBitmap, srcRect, dstRect, null)
                y += logoH + 18f
            } else {
                y += 12f
            }

            val titleText = "ALI EDUCATIONAL"
            val titleW = titlePaint.measureText(titleText)
            canvas.drawText(titleText, (pageWidth - titleW) / 2f, y, titlePaint)
            y += 18f

            val subtitleText = "Stationery & Educational Supplies"
            val subtitleW = subtitlePaint.measureText(subtitleText)
            canvas.drawText(subtitleText, (pageWidth - subtitleW) / 2f, y, subtitlePaint)
            y += 16f

            val docTypeText = "R E C E I P T"
            val docTypeW = docTypePaint.measureText(docTypeText)
            canvas.drawText(docTypeText, (pageWidth - docTypeW) / 2f, y, docTypePaint)
            y += 24f

            // Metadata: 4 rows
            val metaRows = listOf(
                "Receipt No." to sale.receiptNumber,
                "Date" to dateStr,
                "Customer" to customerName,
                "Payment" to paymentLabel
            )

            for ((k, v) in metaRows) {
                canvas.drawText(k, margin, y, metaKeyPaint)
                val vW = metaValPaint.measureText(v)
                canvas.drawText(v, rightColX - vW, y, metaValPaint)
                y += 16f
            }
            y += 6f

            // Voided Banner if cancelled
            if (sale.isVoided) {
                val voidBg = RectF(margin, y, rightColX, y + 26f)
                val voidBgPaint = Paint(Paint.ANTI_ALIAS_FLAG).apply { color = Color.rgb(254, 226, 226);
    style = Paint.Style.FILL }
                canvas.drawRoundRect(voidBg, 4f, 4f, voidBgPaint)
                val voidTextPaint = Paint(Paint.ANTI_ALIAS_FLAG).apply { color = Color.rgb(220, 38, 38);
    textSize = 9.5f; isFakeBoldText = true }
                val vText = "CANCELLED & REFUNDED: Refund
    ₹${CostCalculator.formatPaiseToRupees(sale.refundPayoutPaise)}"
                canvas.drawText(vText, margin + 8f, y + 17f, voidTextPaint)
                y += 34f
            }

            return y
        }

        fun drawTableHeader(canvas: Canvas, startY: Float): Float {
            var y = startY
            canvas.drawLine(margin, y, rightColX, y, linePaint)
            y += 16f

            canvas.drawText("Item", margin, y, tableHeaderPaint)
            val qtyHdrW = tableHeaderPaint.measureText("Qty")
            canvas.drawText("Qty", qtyColX - qtyHdrW, y, tableHeaderPaint)
            val rateHdrW = tableHeaderPaint.measureText("Rate")
            canvas.drawText("Rate", rateColX - rateHdrW, y, tableHeaderPaint)

            val amtHdrW = tableHeaderPaint.measureText("Amount")
            canvas.drawText("Amount", rightColX - amtHdrW, y, tableHeaderPaint)
```

```
            y += 7f

            canvas.drawLine(margin, y, rightColX, y, linePaint)
            y += 17f
            return y
        }

        fun drawItemRow(canvas: Canvas, item: ProcessedItem, startY: Float): Float {
            var y = startY
            val baseBaseline = y
            val firstLine = item.displayNameLines.firstOrNull() ?: ""
            canvas.drawText(firstLine, margin, baseBaseline, itemTextPaint)

            val qtyW = itemSubPaint.measureText(item.qtyStr)
            canvas.drawText(item.qtyStr, qtyColX - qtyW, baseBaseline, itemSubPaint)

            val rateW = itemSubPaint.measureText(item.rateStr)
            canvas.drawText(item.rateStr, rateColX - rateW, baseBaseline, itemSubPaint)

            val amtW = itemTextPaint.measureText(item.amountStr)
            canvas.drawText(item.amountStr, rightColX - amtW, baseBaseline, itemTextPaint)

            if (item.displayNameLines.size > 1) {
                for (i in 1 until item.displayNameLines.size) {
                    y += 13f
                    canvas.drawText(item.displayNameLines[i], margin, y, itemTextPaint)
                }
            }
            y += 18f
            return y
        }

        fun drawTotalsAndFooter(canvas: Canvas, startY: Float) {
            var y = startY
            canvas.drawLine(margin, y, rightColX, y, linePaint)
            y += 18f

            // Subtotal
            canvas.drawText("Subtotal", summaryX, y, summaryLabelPaint)
            val subVal = CostCalculator.formatPaiseToRupees(sale.grossTotalPaise)
            canvas.drawText(subVal, rightColX - summaryValPaint.measureText(subVal), y, summaryValPaint)
            y += 16f

            // Discount
            if (sale.discountValuePaise > 0L) {
                val pctStr = if (sale.discountPercent > 0.0) {
                    val formatted = if (sale.discountPercent % 1.0 == 0.0) {
                        String.format(Locale.ROOT, "%.0f", sale.discountPercent)
                    } else {
                        String.format(Locale.ROOT, "%.1f", sale.discountPercent)
                    }
                    "Discount ($formatted%)"
                } else {
                    "Discount"
                }
                canvas.drawText(pctStr, summaryX, y, summaryLabelPaint)
                val discVal = "-${CostCalculator.formatPaiseToRupees(sale.discountValuePaise)}"
                canvas.drawText(discVal, rightColX - summaryValPaint.measureText(discVal), y,
summaryValPaint)
                y += 16f
            }

            // TOTAL PAYABLE Shaded Banner
            val bannerH = 28f
            val bannerRect = RectF(margin - 8f, y, rightColX + 8f, y + bannerH)
            canvas.drawRoundRect(bannerRect, 3f, 3f, totalBgPaint)
            val bannerTextY = y + 19f
            canvas.drawText("TOTAL PAYABLE", summaryX, bannerTextY, totalLabelPaint)
            val totalVal = CostCalculator.formatPaiseToRupees(sale.netTotalPaise)
            canvas.drawText(totalVal, rightColX - totalValPaint.measureText(totalVal), bannerTextY,
totalValPaint)
            y += bannerH + 16f

            // Amount paid
            canvas.drawText("Amount paid", summaryX, y, summaryLabelPaint)
            val paidVal = CostCalculator.formatPaiseToRupees(sale.amountReceivedPaise)
            canvas.drawText(paidVal, rightColX - summaryValPaint.measureText(paidVal), y, summaryValPaint)
            y += 16f

            // Balance due or Change
            if (sale.remainingCreditPaise > 0L) {
                canvas.drawText("Balance due", summaryX, y, summaryLabelPaint)
                val dueVal = CostCalculator.formatPaiseToRupees(sale.remainingCreditPaise)
                canvas.drawText(dueVal, rightColX - summaryValPaint.measureText(dueVal), y, summaryValPaint)
                y += 22f
            } else if (sale.changeGivenPaise > 0L) {
                canvas.drawText("Change", summaryX, y, summaryLabelPaint)
                val changeVal = CostCalculator.formatPaiseToRupees(sale.changeGivenPaise)
                canvas.drawText(changeVal, rightColX - summaryValPaint.measureText(changeVal), y,
```

```
    summaryValPaint)
                y += 16f
                canvas.drawText("Balance due", summaryX, y, summaryLabelPaint)
                canvas.drawText("₹0.00", rightColX - summaryValPaint.measureText("₹0.00"), y,
    summaryValPaint)
                y += 22f
            } else {
                canvas.drawText("Balance due", summaryX, y, summaryLabelPaint)
                canvas.drawText("₹0.00", rightColX - summaryValPaint.measureText("₹0.00"), y,
    summaryValPaint)
                y += 22f
            }

            // Status Badge [ PAID ] / [ PARTIAL ] / [ DUE ] / [ VOIDED ]
            val bTextW = badgeTextPaint.measureText(badgeText)
            val bBoxW = maxOf(64f, bTextW + 20f)
            val bBoxH = 20f
            val bBoxX = (pageWidth - bBoxW) / 2f
            val bRect = RectF(bBoxX, y, bBoxX + bBoxW, y + bBoxH)
            canvas.drawRoundRect(bRect, 2f, 2f, badgeBorderPaint)
            canvas.drawText(badgeText, bBoxX + (bBoxW - bTextW) / 2f, y + 14f, badgeTextPaint)
            y += bBoxH + 18f

            canvas.drawLine(margin, y, rightColX, y, linePaint)
            y += 22f

            // Footer
            val thanksText = "Thank you for visiting Ali Educational!"
            val thanksW = footerPaint.measureText(thanksText)
            canvas.drawText(thanksText, (pageWidth - thanksW) / 2f, y, footerPaint)
        }

        if (totalEstimatedHeight <= MAX_SINGLE_PAGE_HEIGHT) {
            // SINGLE PAGE RECEIPT (Snug fit, zero blank void)
            val singlePageHeight = maxOf(340, totalEstimatedHeight.toInt())
            val pageInfo = PdfDocument.PageInfo.Builder(pageWidth, singlePageHeight, 1).create()
            val page = document.startPage(pageInfo)
            val canvas = page.canvas
            canvas.drawColor(Color.WHITE)

            var curY = drawReceiptHeader(canvas)
            curY = drawTableHeader(canvas, curY)
            for (item in processedItems) {
                curY = drawItemRow(canvas, item, curY)
            }
            drawTotalsAndFooter(canvas, curY)

            document.finishPage(page)
        } else {
            // MULTI-PAGE RECEIPT (Responsive pagination for long bills)
            val standardPageHeight = 600
            val usableBottom = 540f

            // Partition items across pages
            val pagesItems = mutableListOf<MutableList<ProcessedItem>>()
            var curList = mutableListOf<ProcessedItem>()
            var curPageAvailableY = fullHeaderHeight

            for (item in processedItems) {
                if (curPageAvailableY + item.height > usableBottom && curList.isNotEmpty()) {
                    pagesItems.add(curList)
                    curList = mutableListOf()
                    curPageAvailableY = 90f // Continuation header + table header height
                }
                curList.add(item)
                curPageAvailableY += item.height
            }
            if (curList.isNotEmpty()) {
                pagesItems.add(curList)
            }

            // Determine if summary fits on last page or needs a final page
            val summaryNeedsNewPage = (curPageAvailableY + summaryBlockHeight > usableBottom)
            val totalPages = pagesItems.size + (if (summaryNeedsNewPage) 1 else 0)

            for (pIdx in pagesItems.indices) {
                val pageNum = pIdx + 1
                val pItems = pagesItems[pIdx]
                val pageInfo = PdfDocument.PageInfo.Builder(pageWidth, standardPageHeight, pageNum).create()
                val page = document.startPage(pageInfo)
                val canvas = page.canvas
                canvas.drawColor(Color.WHITE)

                var curY: Float
                if (pageNum == 1) {
                    curY = drawReceiptHeader(canvas)
                    curY = drawTableHeader(canvas, curY)
                } else {
```

```
                    // Compact continuation header
                    curY = 22f
                    canvas.drawText("ALI EDUCATIONAL", margin, curY, titlePaint)
                    val contMeta = "Receipt #${sale.receiptNumber}  •  Page $pageNum of $totalPages"
                    canvas.drawText(contMeta, rightColX - subtitlePaint.measureText(contMeta), curY,
    subtitlePaint)
                    curY += 10f
                    curY = drawTableHeader(canvas, curY)
                }

                for (item in pItems) {
                    curY = drawItemRow(canvas, item, curY)
                }

                val isLastItemPage = (pIdx == pagesItems.size - 1)
                if (isLastItemPage && !summaryNeedsNewPage) {
                    drawTotalsAndFooter(canvas, curY)
                } else {
                    // Page footer indicator
                    canvas.drawLine(margin, usableBottom, rightColX, usableBottom, linePaint)
                    val contText = "Page $pageNum of $totalPages (Continued...)"
                    canvas.drawText(contText, (pageWidth - subtitlePaint.measureText(contText)) / 2f,
    usableBottom + 16f, subtitlePaint)
                }

                document.finishPage(page)
            }

            if (summaryNeedsNewPage) {
                val finalPageNum = totalPages
                val pageInfo = PdfDocument.PageInfo.Builder(pageWidth, standardPageHeight,
    finalPageNum).create()
                val page = document.startPage(pageInfo)
                val canvas = page.canvas
                canvas.drawColor(Color.WHITE)

                var curY = 22f
                canvas.drawText("ALI EDUCATIONAL", margin, curY, titlePaint)
                val contMeta = "Receipt #${sale.receiptNumber}  •  Page $finalPageNum of $totalPages"
                canvas.drawText(contMeta, rightColX - subtitlePaint.measureText(contMeta), curY,
    subtitlePaint)
                curY += 14f

                drawTotalsAndFooter(canvas, curY)
                document.finishPage(page)
            }
        }

        // Save generated PDF to cacheDir/receipts
        val ctx = context ?: throw IllegalStateException("Context required to generate PDF file on device")
        val receiptsDir = File(ctx.cacheDir, "receipts")
        if (!receiptsDir.exists()) receiptsDir.mkdirs()
        val pdfFile = File(receiptsDir, "Receipt_${sale.receiptNumber}.pdf")
        FileOutputStream(pdfFile).use { out ->
            document.writeTo(out)
        }
        document.close()

        return@withContext pdfFile
    }

    /**
     * Create Android Share Intent for sending the receipt PDF directly to WhatsApp or Android Chooser.
     */
    suspend fun createWhatsAppReceiptIntent(sale: SaleEntity, lines: List<SaleLineEntity>): Intent {
        val ctx = context ?: throw IllegalStateException("Context required to share receipt")
        val pdfFile = createReceiptPdf(sale, lines)
        val uri: Uri = FileProvider.getUriForFile(
            ctx,
            "${ctx.packageName}.fileprovider",
            pdfFile
        )
        val summaryText = generateReceiptSummaryText(sale, lines)

        val intent = Intent(Intent.ACTION_SEND).apply {
            type = "application/pdf"
            putExtra(Intent.EXTRA_STREAM, uri)
            putExtra(Intent.EXTRA_TEXT, summaryText)
            putExtra(Intent.EXTRA_SUBJECT, "Receipt #${sale.receiptNumber} - Ali Educational")
            addFlags(Intent.FLAG_GRANT_READ_URI_PERMISSION)
        }

        // Try direct WhatsApp package if installed
        val pm = ctx.packageManager
        val isWhatsAppInstalled = isPackageInstalled("com.whatsapp", pm) ||
    isPackageInstalled("com.whatsapp.w4b", pm)
        if (isWhatsAppInstalled) {
            val targetPackage = if (isPackageInstalled("com.whatsapp", pm)) "com.whatsapp" else
```

```
    "com.whatsapp.w4b"
            intent.setPackage(targetPackage)
            intent.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)
            return intent
        }

        // Fallback: standard Android share sheet
        val chooser = Intent.createChooser(intent, "Share Receipt (WhatsApp)")
        chooser.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)
        return chooser
    }

    /**
     * Print receipt directly via Android PrintManager.
     */
    fun printReceipt(activity: Activity, sale: SaleEntity, lines: List<SaleLineEntity>) {
        val printManager = activity.getSystemService(Context.PRINT_SERVICE) as? PrintManager ?: return
        val jobName = "AliEducational_Receipt_${sale.receiptNumber}"

        printManager.print(
            jobName,
            object : PrintDocumentAdapter() {
                var pdfFile: File? = null

                override fun onLayout(
                    oldAttributes: PrintAttributes?,
                    newAttributes: PrintAttributes?,
                    cancellationSignal: CancellationSignal?,
                    callback: LayoutResultCallback?,
                    extras: Bundle?
                ) {
                    if (cancellationSignal?.isCanceled == true) {
                        callback?.onLayoutCancelled()
                        return
                    }

                    val info = PrintDocumentInfo.Builder("receipt_${sale.receiptNumber}.pdf")
                        .setContentType(PrintDocumentInfo.CONTENT_TYPE_DOCUMENT)
                        .setPageCount(1)
                        .build()

                    callback?.onLayoutFinished(info, true)
                }

                override fun onWrite(
                    pages: Array<out PageRange>?,
                    destination: ParcelFileDescriptor?,
                    cancellationSignal: CancellationSignal?,
                    callback: WriteResultCallback?
                ) {
                    try {
                        kotlinx.coroutines.runBlocking {
                            pdfFile = createReceiptPdf(sale, lines)
                        }
                        val input = FileInputStream(pdfFile)
                        val output = FileOutputStream(destination?.fileDescriptor)
                        input.copyTo(output)
                        input.close()
                        output.close()
                        callback?.onWriteFinished(arrayOf(PageRange.ALL_PAGES))
                    } catch (e: Exception) {
                        callback?.onWriteFailed(e.message)
                    }
                }
            },
            PrintAttributes.Builder()
                .setColorMode(PrintAttributes.COLOR_MODE_MONOCHROME)
                .build()
        )
    }

    private fun isPackageInstalled(packageName: String, packageManager: PackageManager): Boolean {
        return try {
            packageManager.getPackageInfo(packageName, 0)
            true
        } catch (_: PackageManager.NameNotFoundException) {
            false
        }
    }
}
```

app\src\main\java\com\alieducational\app\data\service\ShopRecordsExcelGenerator.kt

```kotlin
package com.alieducational.app.data.service

import com.alieducational.app.data.local.entity.AccountPaymentEntity
import com.alieducational.app.data.local.entity.CashCategoryEntity
import com.alieducational.app.data.local.entity.CustomerEntity
import com.alieducational.app.data.local.entity.PigmyDepositEntity
import com.alieducational.app.data.local.entity.PigmySchemeEntity
import com.alieducational.app.data.local.entity.PigmyWithdrawalEntity
import com.alieducational.app.data.local.entity.ProductEntity
import com.alieducational.app.data.local.entity.PurchaseEntity
import com.alieducational.app.data.local.entity.PurchaseLineEntity
import com.alieducational.app.data.local.entity.SaleEntity
import com.alieducational.app.data.local.entity.SaleLineEntity
import com.alieducational.app.data.local.entity.ServiceDefinitionEntity
import com.alieducational.app.data.local.entity.ShopExpenseEntity
import com.alieducational.app.data.local.entity.StockMovementEntity
import com.alieducational.app.data.local.entity.SupplierEntity
import org.apache.poi.ss.usermodel.BorderStyle
import org.apache.poi.ss.usermodel.FillPatternType
import org.apache.poi.ss.usermodel.HorizontalAlignment
import org.apache.poi.ss.usermodel.IndexedColors
import org.apache.poi.ss.usermodel.Row
import org.apache.poi.xssf.usermodel.XSSFWorkbook
import java.io.File
import java.io.FileOutputStream
import java.text.SimpleDateFormat
import java.util.Date
import java.util.Locale
import javax.inject.Inject
import javax.inject.Singleton

data class ShopSnapshotData(
    val snapshotId: String,
    val timestamp: Long,
    val shopName: String,
    val products: List<ProductEntity>,
    val movements: List<StockMovementEntity>,
    val purchases: List<PurchaseEntity>,
    val purchaseLines: List<PurchaseLineEntity>,
    val sales: List<SaleEntity>,
    val saleLines: List<SaleLineEntity>,
    val accountPayments: List<AccountPaymentEntity>,
    val customers: List<CustomerEntity>,
    val suppliers: List<SupplierEntity>,
    val cashExpenses: List<ShopExpenseEntity>,
    val pigmySchemes: List<PigmySchemeEntity>,
    val pigmyDeposits: List<PigmyDepositEntity>,
    val pigmyWithdrawals: List<PigmyWithdrawalEntity>,
    val services: List<ServiceDefinitionEntity>,
    val cashCategories: List<CashCategoryEntity>,
    val photoCount: Int
)

@Singleton
class ShopRecordsExcelGenerator @Inject constructor() {

    fun generateWorkbook(data: ShopSnapshotData, outputFile: File) {
        val workbook = XSSFWorkbook()
        val dateFormat = SimpleDateFormat("dd-MMM-yyyy hh:mm a", Locale.US)

        // Styles
        val headerStyle = workbook.createCellStyle().apply {
            val font = workbook.createFont().apply {
                bold = true
                color = IndexedColors.WHITE.index
            }
            setFont(font)
            fillForegroundColor = IndexedColors.GREY_80_PERCENT.index
            fillPattern = FillPatternType.SOLID_FOREGROUND
            alignment = HorizontalAlignment.CENTER
            borderBottom = BorderStyle.THIN
            borderTop = BorderStyle.THIN
            borderLeft = BorderStyle.THIN
            borderRight = BorderStyle.THIN
        }

        val normalStyle = workbook.createCellStyle().apply {
            borderBottom = BorderStyle.THIN
            borderTop = BorderStyle.THIN
            borderLeft = BorderStyle.THIN
            borderRight = BorderStyle.THIN
        }
```

```
83
84          val currencyFormat = workbook.createDataFormat().getFormat("#,##0.00")
85          val numberStyle = workbook.createCellStyle().apply {
86              setDataFormat(currencyFormat)
87              alignment = HorizontalAlignment.RIGHT
88              borderBottom = BorderStyle.THIN
89              borderTop = BorderStyle.THIN
90              borderLeft = BorderStyle.THIN
91              borderRight = BorderStyle.THIN
92          }
93
94          val integerStyle = workbook.createCellStyle().apply {
95              alignment = HorizontalAlignment.RIGHT
96              borderBottom = BorderStyle.THIN
97              borderTop = BorderStyle.THIN
98              borderLeft = BorderStyle.THIN
99              borderRight = BorderStyle.THIN
100         }
101
102         fun createHeaderRow(sheet: org.apache.poi.ss.usermodel.Sheet, rowIdx: Int, headers: List<String>) {
103             val row = sheet.createRow(rowIdx)
104             headers.forEachIndexed { i, title ->
105                 val cell = row.createCell(i)
106                 cell.setCellValue(title)
107                 cell.cellStyle = headerStyle
108             }
109         }
110
111         fun writeString(row: Row, col: Int, value: String?) {
112             val cell = row.createCell(col)
113             cell.setCellValue(value ?: "")
114             cell.cellStyle = normalStyle
115         }
116
117         fun writeNumber(row: Row, col: Int, rupees: Double) {
118             val cell = row.createCell(col)
119             cell.setCellValue(rupees)
120             cell.cellStyle = numberStyle
121         }
122
123         fun writeInt(row: Row, col: Int, value: Int) {
124             val cell = row.createCell(col)
125             cell.setCellValue(value.toDouble())
126             cell.cellStyle = integerStyle
127         }
128
129         // ═══════════════════════════════════════════════════
130         // SHEET 1: Summary
131         // ═══════════════════════════════════════════════════
132         val summarySheet = workbook.createSheet("Summary")
133         var sRow = 0
134
135         val titleRow = summarySheet.createRow(sRow++)
136         titleRow.createCell(0).apply {
137             setCellValue("${data.shopName} - Complete Shop Records")
138             cellStyle = workbook.createCellStyle().apply {
139                 val f = workbook.createFont().apply { bold = true; fontHeightInPoints = 14.toShort() }
140                 setFont(f)
141             }
142         }
143
144         val noticeRow = summarySheet.createRow(sRow++)
145         noticeRow.createCell(0).setCellValue("Readable Auditing Workbook (Generated from Snapshot:
    ${data.snapshotId})")
146
147         sRow++ // Blank line
148
149         fun addSummaryLine(label: String, value: Any) {
150             val r = summarySheet.createRow(sRow++)
151             r.createCell(0).apply { setCellValue(label); cellStyle = normalStyle }
152             val c = r.createCell(1)
153             when (value) {
154                 is Double -> {
155                     c.setCellValue(value)
156                     c.cellStyle = numberStyle
157                 }
158                 is Number -> {
159                     c.setCellValue(value.toDouble())
160                     c.cellStyle = integerStyle
161                 }
162                 else -> {
163                     c.setCellValue(value.toString())
164                     c.cellStyle = normalStyle
165                 }
166             }
167         }
168
169         addSummaryLine("Snapshot Identifier:", data.snapshotId)
```

```
        addSummaryLine("Snapshot Timestamp:", dateFormat.format(Date(data.timestamp)))
        addSummaryLine("Total Products Count:", data.products.size)
        val totalSellableStock = data.products.sumOf { it.currentSellableStock }
        addSummaryLine("Total Sellable Stock Units:", totalSellableStock)
        val stockValuationRupees = data.products.sumOf { it.currentSellableStock * it.movingAvgCostPaise } / 100.0
        addSummaryLine("Total Stock Inventory Valuation (₹):", stockValuationRupees)

        val totalCustomersDueRupees = data.customers.sumOf { it.currentBalancePaise } / 100.0
        addSummaryLine("Total Outstanding Customers Owe Us (₹):", totalCustomersDueRupees)
        val totalSuppliersDueRupees = data.suppliers.sumOf { it.currentBalancePaise } / 100.0
        addSummaryLine("Total Outstanding We Owe Suppliers (₹):", totalSuppliersDueRupees)

        val completedBills = data.sales.filter { !it.isVoided }
        val totalSalesRupees = completedBills.sumOf { it.netTotalPaise } / 100.0
        addSummaryLine("Total Completed Bills Count:", completedBills.size)
        addSummaryLine("Total Recorded Sales Revenue (₹):", totalSalesRupees)

        val activeExpenses = data.cashExpenses.filter { !it.isVoided }
        val totalInflowRupees = activeExpenses.filter { it.isIncome }.sumOf { it.amountPaise } / 100.0
        val totalOutflowRupees = activeExpenses.filter { !it.isIncome }.sumOf { it.amountPaise } / 100.0
        addSummaryLine("Total Recorded Cash Inflow (₹):", totalInflowRupees)
        addSummaryLine("Total Recorded Cash Outflow (₹):", totalOutflowRupees)

        val activePigmyDeposits = data.pigmyDeposits.filter { !it.isVoided }.sumOf { it.amountPaise }
        val activePigmyWithdrawals = data.pigmyWithdrawals.filter { !it.isVoided }.sumOf { it.amountPaise }
        val trackedPigmyBalanceRupees = (activePigmyDeposits - activePigmyWithdrawals) / 100.0
        addSummaryLine("Total Pigmy Schemes Count:", data.pigmySchemes.size)
        addSummaryLine("Net Tracked Pigmy Savings Balance (₹):", trackedPigmyBalanceRupees)
        addSummaryLine("Photo & File Attachments Count:", data.photoCount)

        summarySheet.setColumnWidth(0, 36 * 256)
        summarySheet.setColumnWidth(1, 24 * 256)

        // ═══════════════════════════════════════════════════════
        // SHEET 2: Products and Stock
        // ═══════════════════════════════════════════════════════
        val prodSheet = workbook.createSheet("Products and Stock")
        val prodHeaders = listOf(
            "Product ID", "Barcode", "Product Name", "Category", "Brand",
            "Base Unit", "Pack Size", "Sellable Stock", "Avg Cost (₹)",
            "Sell Price Piece (₹)", "Sell Price Pack (₹)", "Stock Value (₹)",
            "Low Stock Threshold", "Status"
        )
        createHeaderRow(prodSheet, 0, prodHeaders)
        data.products.forEachIndexed { idx, p ->
            val row = prodSheet.createRow(idx + 1)
            writeString(row, 0, p.id)
            writeString(row, 1, p.barcode ?: "")
            writeString(row, 2, p.name)
            writeString(row, 3, p.category)
            writeString(row, 4, p.brand ?: "")
            writeString(row, 5, p.baseUnit)
            writeInt(row, 6, p.packSize)
            writeInt(row, 7, p.currentSellableStock)
            writeNumber(row, 8, p.movingAvgCostPaise / 100.0)
            writeNumber(row, 9, p.sellingPricePiecePaise / 100.0)
            writeNumber(row, 10, p.sellingPricePackPaise / 100.0)
            writeNumber(row, 11, (p.currentSellableStock * p.movingAvgCostPaise) / 100.0)
            writeInt(row, 12, p.lowStockThreshold)
            writeString(row, 13, if (p.isArchived) "Archived" else "Active")
        }

        // ═══════════════════════════════════════════════════════
        // SHEET 3: Stock Movements / Purchases
        // ═══════════════════════════════════════════════════════
        val movSheet = workbook.createSheet("Stock Movements")
        val movHeaders = listOf(
            "Movement ID", "Date & Time", "Product Name", "Type",
            "Quantity Change", "Unit Cost (₹)", "Total Cost (₹)", "Reason / Notes", "Reference ID"
        )
        createHeaderRow(movSheet, 0, movHeaders)
        val productMap = data.products.associateBy { it.id }
        data.movements.forEachIndexed { idx, m ->
            val row = movSheet.createRow(idx + 1)
            writeString(row, 0, m.id)
            writeString(row, 1, dateFormat.format(Date(m.timestamp)))
            writeString(row, 2, productMap[m.productId]?.name ?: m.productId)
            writeString(row, 3, m.type)
            writeInt(row, 4, m.quantityChangeBase)
            writeNumber(row, 5, m.unitCostPaise / 100.0)
            writeNumber(row, 6, (m.quantityChangeBase * m.unitCostPaise) / 100.0)
            writeString(row, 7, m.reason ?: "")
            writeString(row, 8, m.referenceId ?: "")
        }

        // ═══════════════════════════════════════════════════════
        // SHEET 4: Bills / Bill Items
```

```
        // ═══════════════════════════════════════════════════════
        val billsSheet = workbook.createSheet("Bills and Items")
        val billHeaders = listOf(
            "Receipt No", "Date & Time", "Customer Name", "Line Type",
            "Item Name", "Quantity", "Rate (₹)", "Line Total (₹)",
            "Bill Subtotal (₹)", "Discount (₹)", "Payable (₹)", "Paid (₹)",
            "Balance Due (₹)", "Payment Mode", "Status"
        )
        createHeaderRow(billsSheet, 0, billHeaders)
        val customerMap = data.customers.associateBy { it.id }
        val saleLinesGrouped = data.saleLines.groupBy { it.saleId }

        var billRowIdx = 1
        data.sales.forEach { sale ->
            val lines = saleLinesGrouped[sale.id] ?: emptyList()
            val custName = sale.customerName ?: sale.customerId?.let { customerMap[it]?.name } ?: "Walk-in Customer"
            val status = if (sale.isVoided) "CANCELLED" else "COMPLETED"
            val balanceDue = sale.remainingCreditPaise

            if (lines.isEmpty()) {
                val row = billsSheet.createRow(billRowIdx++)
                writeString(row, 0, sale.receiptNumber)
                writeString(row, 1, dateFormat.format(Date(sale.timestamp)))
                writeString(row, 2, custName)
                writeString(row, 3, "GENERAL")
                writeString(row, 4, "-")
                writeInt(row, 5, 0)
                writeNumber(row, 6, 0.0)
                writeNumber(row, 7, 0.0)
                writeNumber(row, 8, sale.grossTotalPaise / 100.0)
                writeNumber(row, 9, sale.discountValuePaise / 100.0)
                writeNumber(row, 10, sale.netTotalPaise / 100.0)
                writeNumber(row, 11, sale.amountReceivedPaise / 100.0)
                writeNumber(row, 12, balanceDue / 100.0)
                writeString(row, 13, sale.paymentMethod)
                writeString(row, 14, status)
            } else {
                lines.forEach { line ->
                    val row = billsSheet.createRow(billRowIdx++)
                    writeString(row, 0, sale.receiptNumber)
                    writeString(row, 1, dateFormat.format(Date(sale.timestamp)))
                    writeString(row, 2, custName)
                    writeString(row, 3, line.lineType)
                    writeString(row, 4, line.productName)
                    writeInt(row, 5, line.quantityBase)
                    writeNumber(row, 6, line.unitSellingPricePaise / 100.0)
                    writeNumber(row, 7, line.lineNetPaise / 100.0)
                    writeNumber(row, 8, sale.grossTotalPaise / 100.0)
                    writeNumber(row, 9, sale.discountValuePaise / 100.0)
                    writeNumber(row, 10, sale.netTotalPaise / 100.0)
                    writeNumber(row, 11, sale.amountReceivedPaise / 100.0)
                    writeNumber(row, 12, balanceDue / 100.0)
                    writeString(row, 13, sale.paymentMethod)
                    writeString(row, 14, status)
                }
            }
        }

        // ═══════════════════════════════════════════════════════
        // SHEET 5: Payments / Allocations
        // ═══════════════════════════════════════════════════════
        val paymentsSheet = workbook.createSheet("Payments and Dues")
        val paymentHeaders = listOf(
            "Payment ID", "Date & Time", "Party Type", "Party Name",
            "Amount (₹)", "Payment Mode", "Notes / Reference"
        )
        createHeaderRow(paymentsSheet, 0, paymentHeaders)
        val supplierMap = data.suppliers.associateBy { it.id }
        data.accountPayments.forEachIndexed { idx, p ->
            val row = paymentsSheet.createRow(idx + 1)
            val partyName = if (p.accountType == "CUSTOMER") {
                customerMap[p.accountId]?.name ?: p.accountId
            } else {
                supplierMap[p.accountId]?.name ?: p.accountId
            }
            writeString(row, 0, p.id)
            writeString(row, 1, dateFormat.format(Date(p.timestamp)))
            writeString(row, 2, p.accountType)
            writeString(row, 3, partyName)
            writeNumber(row, 4, p.amountPaise / 100.0)
            writeString(row, 5, p.paymentMethod)
            writeString(row, 6, p.notes ?: "")
        }

        // ═══════════════════════════════════════════════════════
        // SHEET 6: Customers & Suppliers
        // ═══════════════════════════════════════════════════════
```

```
        val partiesSheet = workbook.createSheet("Customers and Suppliers")
        val partyHeaders = listOf(
            "Account Type", "Party ID", "Name", "Phone", "Outstanding Balance (₹)", "Status"
        )
        createHeaderRow(partiesSheet, 0, partyHeaders)
        var pRowIdx = 1
        data.customers.forEach { c ->
            val row = partiesSheet.createRow(pRowIdx++)
            writeString(row, 0, "CUSTOMER")
            writeString(row, 1, c.id)
            writeString(row, 2, c.name)
            writeString(row, 3, c.phone ?: "")
            writeNumber(row, 4, c.currentBalancePaise / 100.0)
            writeString(row, 5, if (c.isArchived) "Archived" else "Active")
        }
        data.suppliers.forEach { s ->
            val row = partiesSheet.createRow(pRowIdx++)
            writeString(row, 0, "SUPPLIER")
            writeString(row, 1, s.id)
            writeString(row, 2, s.name)
            writeString(row, 3, s.phone ?: "")
            writeNumber(row, 4, s.currentBalancePaise / 100.0)
            writeString(row, 5, if (s.isArchived) "Archived" else "Active")
        }

        // ═══════════════════════════════════════════════════════
        // SHEET 7: Cash In Out
        // ═══════════════════════════════════════════════════════
        val cashSheet = workbook.createSheet("Cash In Out")
        val cashHeaders = listOf(
            "Entry ID", "Date & Time", "Direction", "Category",
            "Notes", "Amount (₹)", "Payment Method", "Linked ID", "Status"
        )
        createHeaderRow(cashSheet, 0, cashHeaders)
        data.cashExpenses.forEachIndexed { idx, ce ->
            val row = cashSheet.createRow(idx + 1)
            writeString(row, 0, ce.id)
            writeString(row, 1, dateFormat.format(Date(ce.timestamp)))
            writeString(row, 2, if (ce.isIncome) "CASH IN" else "CASH OUT")
            writeString(row, 3, ce.category)
            writeString(row, 4, ce.notes ?: "")
            writeNumber(row, 5, ce.amountPaise / 100.0)
            writeString(row, 6, ce.paymentMethod)
            writeString(row, 7, ce.linkedExpenseId ?: "")
            writeString(row, 8, if (ce.isVoided) "CANCELLED" else "ACTIVE")
        }

        // ═══════════════════════════════════════════════════════
        // SHEET 8: Expenses / Refunds
        // ═══════════════════════════════════════════════════════
        val expSheet = workbook.createSheet("Expenses and Refunds")
        val expHeaders = listOf(
            "ID", "Date & Time", "Classification", "Category",
            "Amount (₹)", "Notes", "Linked Refund ID", "Has Attachment"
        )
        createHeaderRow(expSheet, 0, expHeaders)
        val actualExpenses = data.cashExpenses.filter { !it.isIncome }
        actualExpenses.forEachIndexed { idx, ex ->
            val row = expSheet.createRow(idx + 1)
            val classification = if (ex.isSavingsTransfer) "SAVINGS_TRANSFER" else ex.classification
            writeString(row, 0, ex.id)
            writeString(row, 1, dateFormat.format(Date(ex.timestamp)))
            writeString(row, 2, classification)
            writeString(row, 3, ex.category)
            writeNumber(row, 4, ex.amountPaise / 100.0)
            writeString(row, 5, ex.notes ?: "")
            writeString(row, 6, ex.linkedExpenseId ?: "")
            writeString(row, 7, if (!ex.attachmentUri.isNullOrBlank()) "YES" else "NO")
        }

        // ═══════════════════════════════════════════════════════
        // SHEET 9: Services
        // ═══════════════════════════════════════════════════════
        val svcSheet = workbook.createSheet("Services")
        val svcHeaders = listOf("Service ID", "Service Name", "Unit Label", "Predefined", "Status")
        createHeaderRow(svcSheet, 0, svcHeaders)
        data.services.forEachIndexed { idx, svc ->
            val row = svcSheet.createRow(idx + 1)
            writeString(row, 0, svc.id)
            writeString(row, 1, svc.name)
            writeString(row, 2, svc.unitLabel)
            writeString(row, 3, if (svc.isPredefined) "YES" else "NO")
            writeString(row, 4, if (svc.isArchived) "Archived" else "Active")
        }

        // ═══════════════════════════════════════════════════════
        // SHEET 10: Pigmy Daily Savings
        // ═══════════════════════════════════════════════════════
```

```
        val pigmySheet = workbook.createSheet("Pigmy Savings")
        val pigmyHeaders = listOf(
            "Scheme Name", "Transaction Type", "Date & Time", "Amount (₹)",
            "Payment Source / Destination", "Notes", "Linked ID"
        )
        createHeaderRow(pigmySheet, 0, pigmyHeaders)
        val schemeMap = data.pigmySchemes.associateBy { it.id }

        var pigmyRowIdx = 1
        data.pigmyDeposits.forEach { dep ->
            val row = pigmySheet.createRow(pigmyRowIdx++)
            val schemeName = schemeMap[dep.accountSchemeId]?.name ?: dep.accountSchemeId
            writeString(row, 0, schemeName)
            writeString(row, 1, "DEPOSIT")
            writeString(row, 2, dateFormat.format(Date(dep.paidTimestamp)))
            writeNumber(row, 3, dep.amountPaise / 100.0)
            writeString(row, 4, dep.paymentSource)
            writeString(row, 5, dep.notes ?: "")
            writeString(row, 6, dep.linkedExpenseId ?: "")
        }
        data.pigmyWithdrawals.forEach { w ->
            val row = pigmySheet.createRow(pigmyRowIdx++)
            val schemeName = schemeMap[w.accountSchemeId]?.name ?: w.accountSchemeId
            writeString(row, 0, schemeName)
            writeString(row, 1, "WITHDRAWAL")
            writeString(row, 2, dateFormat.format(Date(w.withdrawalTimestamp)))
            writeNumber(row, 3, w.amountPaise / 100.0)
            writeString(row, 4, w.destination)
            writeString(row, 5, w.notes ?: "")
            writeString(row, 6, w.linkedExpenseId ?: "")
        }

        // ═══════════════════════════════════════════════════
        // SHEET 11: Categories
        // ═══════════════════════════════════════════════════
        val catSheet = workbook.createSheet("Categories")
        val catHeaders = listOf("Category Type", "Name", "System Category", "Status")
        createHeaderRow(catSheet, 0, catHeaders)
        var catRowIdx = 1
        data.cashCategories.forEach { cc ->
            val row = catSheet.createRow(catRowIdx++)
            writeString(row, 0, "CASH_CATEGORY")
            writeString(row, 1, cc.name)
            writeString(row, 2, if (cc.isSystem) "YES" else "NO")
            writeString(row, 3, if (cc.isArchived) "Archived" else "Active")
        }
        val distinctProductCategories = data.products.map { it.category }.distinct()
        distinctProductCategories.forEach { pc ->
            val row = catSheet.createRow(catRowIdx++)
            writeString(row, 0, "PRODUCT_CATALOG")
            writeString(row, 1, pc)
            writeString(row, 2, "NO")
            writeString(row, 3, "Active")
        }

        // Set standard readable column widths across all sheets (Android-safe without java.awt)
        for (i in 0 until workbook.numberOfSheets) {
            val sheet = workbook.getSheetAt(i)
            val lastCol = sheet.getRow(0)?.lastCellNum?.toInt() ?: 0
            for (col in 0 until lastCol) {
                try {
                    sheet.setColumnWidth(col, 20 * 256)
                } catch (_: Throwable) {
                    // Ignore width adjustment failures on constrained runtimes
                }
            }
        }

        // Write to output file
        FileOutputStream(outputFile).use { out ->
            workbook.write(out)
        }
        workbook.close()
    }
}
```

app\src\main\java\com\alieducational\app\data\service\TwoPhoneSyncService.kt

```
package com.alieducational.app.data.service

import android.content.Context
import android.net.wifi.WifiManager
import com.alieducational.app.data.local.dao.CustomerDao
import com.alieducational.app.data.local.dao.ProductDao
import com.alieducational.app.data.local.dao.SaleDao
import com.alieducational.app.data.local.dao.ServiceDefinitionDao
import com.alieducational.app.data.local.dao.StockMovementDao
import com.alieducational.app.data.local.entity.SaleEntity
import com.alieducational.app.data.local.entity.SaleLineEntity
import com.alieducational.app.data.local.entity.StockMovementEntity
import com.alieducational.app.data.preferences.AppPreferences
import dagger.hilt.android.qualifiers.ApplicationContext
import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.Job
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.flow.first
import kotlinx.coroutines.launch
import kotlinx.coroutines.withContext
import org.json.JSONArray
import org.json.JSONObject
import java.io.BufferedReader
import java.io.InputStreamReader
import java.io.OutputStreamWriter
import java.net.HttpURLConnection
import java.net.Inet4Address
import java.net.NetworkInterface
import java.net.ServerSocket
import java.net.Socket
import java.net.URL
import java.util.Locale
import java.util.UUID
import javax.inject.Inject
import javax.inject.Singleton

sealed class SyncConnectionState {
    data object Idle : SyncConnectionState()
    data class Hosting(val ipAddress: String, val port: Int) : SyncConnectionState()
    data class Connected(val host: String) : SyncConnectionState()
    data class Syncing(val message: String) : SyncConnectionState()
    data class Error(val message: String) : SyncConnectionState()
}

@Singleton
class TwoPhoneSyncService @Inject constructor(
    @ApplicationContext private val context: Context,
    private val preferences: AppPreferences,
    private val productDao: ProductDao,
    private val serviceDao: ServiceDefinitionDao,
    private val saleDao: SaleDao,
    private val customerDao: CustomerDao,
    private val stockMovementDao: StockMovementDao
) {
    private val scope = CoroutineScope(Dispatchers.IO + Job())
    private var serverSocket: ServerSocket? = null
    private var serverJob: Job? = null

    private val _connectionState = MutableStateFlow<SyncConnectionState>(SyncConnectionState.Idle)
    val connectionState: StateFlow<SyncConnectionState> = _connectionState.asStateFlow()

    fun getLocalIpAddress(): String {
        try {
            val wifiManager = context.applicationContext.getSystemService(Context.WIFI_SERVICE) as?
WifiManager
            val ipInt = wifiManager?.connectionInfo?.ipAddress ?: 0
            if (ipInt != 0) {
                return String.format(
                    Locale.US,
                    "%d.%d.%d.%d",
                    ipInt and 0xff,
                    ipInt shr 8 and 0xff,
                    ipInt shr 16 and 0xff,
                    ipInt shr 24 and 0xff
                )
            }
        } catch (_: Exception) {}

        try {
```

```
            val interfaces = NetworkInterface.getNetworkInterfaces()
            while (interfaces.hasMoreElements()) {
                val iface = interfaces.nextElement()
                val addresses = iface.inetAddresses
                while (addresses.hasMoreElements()) {
                    val addr = addresses.nextElement()
                    if (!addr.isLoopbackAddress && addr is Inet4Address) {
                        return addr.hostAddress ?: "127.0.0.1"
                    }
                }
            }
        } catch (_: Exception) {}

        return "127.0.0.1"
    }

    // ── Owner Server Functions ──────────────────────────────

    fun startOwnerServer(port: Int = 8080) {
        if (serverJob != null && serverJob?.isActive == true) return

        serverJob = scope.launch {
            try {
                serverSocket = ServerSocket(port)
                val ip = getLocalIpAddress()
                _connectionState.value = SyncConnectionState.Hosting(ip, port)

                while (serverSocket?.isClosed == false) {
                    val clientSocket = serverSocket?.accept() ?: break
                    launch {
                        handleClientRequest(clientSocket)
                    }
                }
            } catch (e: Exception) {
                _connectionState.value = SyncConnectionState.Error("Server error: ${e.message}")
            }
        }
    }

    fun stopOwnerServer() {
        try {
            serverSocket?.close()
            serverSocket = null
            serverJob?.cancel()
            serverJob = null
            _connectionState.value = SyncConnectionState.Idle
        } catch (_: Exception) {}
    }

    private suspend fun handleClientRequest(socket: Socket) = withContext(Dispatchers.IO) {
        try {
            val reader = BufferedReader(InputStreamReader(socket.getInputStream()))
            val writer = OutputStreamWriter(socket.getOutputStream())

            val requestLine = reader.readLine() ?: return@withContext
            val parts = requestLine.split(" ")
            if (parts.size < 2) return@withContext

            val method = parts[0]
            val path = parts[1]

            // Read headers to get content length if POST
            var contentLength = 0
            var line: String?
            while (reader.readLine().also { line = it } != null) {
                if (line.isNullOrBlank()) break
                if (line!!.lowercase().startsWith("content-length:")) {
                    contentLength = line!!.substringAfter(":").trim().toIntOrNull() ?: 0
                }
            }

            if (method == "GET" && path.startsWith("/api/catalog")) {
                // Send role-filtered catalog: selling prices and stock only (NO purchase costs or supplier dues!)
                val products = productDao.getAllActiveProducts().first()
                val services = serviceDao.getAllActiveServices().first()

                val root = JSONObject()
                val productsArray = JSONArray()
                products.forEach { p ->
                    val obj = JSONObject()
                    obj.put("id", p.id)
                    obj.put("name", p.name)
                    obj.put("urduName", p.urduName ?: "")
                    obj.put("category", p.category)
                    obj.put("brand", p.brand ?: "")
                    obj.put("sku", p.sku ?: "")
                    obj.put("barcode", p.barcode ?: "")
```

```
                    obj.put("packSize", p.packSize)
                    obj.put("sellingPricePiecePaise", p.sellingPricePiecePaise)
                    obj.put("sellingPricePackPaise", p.sellingPricePackPaise)
                    obj.put("currentSellableStock", p.currentSellableStock)
                    productsArray.put(obj)
                }
                root.put("products", productsArray)

                val servicesArray = JSONArray()
                services.forEach { s ->
                    val obj = JSONObject()
                    obj.put("id", s.id)
                    obj.put("name", s.name)
                    obj.put("unitLabel", s.unitLabel)
                    obj.put("isPredefined", s.isPredefined)
                    servicesArray.put(obj)
                }
                root.put("services", servicesArray)

                val responseBytes = root.toString().toByteArray(Charsets.UTF_8)
                writer.write("HTTP/1.1 200 OK\r\n")
                writer.write("Content-Type: application/json; charset=utf-8\r\n")
                writer.write("Content-Length: ${responseBytes.size}\r\n")
                writer.write("\r\n")
                writer.flush()
                socket.getOutputStream().write(responseBytes)
                socket.getOutputStream().flush()

            } else if (method == "POST" && path.startsWith("/api/sync-bills")) {
                // Read body
                val bodyChars = CharArray(contentLength)
                var readTotal = 0
                while (readTotal < contentLength) {
                    val r = reader.read(bodyChars, readTotal, contentLength - readTotal)
                    if (r < 0) break
                    readTotal += r
                }
                val body = String(bodyChars, 0, readTotal)
                val billsArray = JSONArray(body)

                val acknowledgedIds = JSONArray()

                for (i in 0 until billsArray.length()) {
                    val billObj = billsArray.getJSONObject(i)
                    val saleId = billObj.getString("id")

                    // Deduplicate: check if already committed
                    val existingSale = saleDao.getSaleById(saleId)
                    if (existingSale == null) {
                        val receiptNumber = billObj.getString("receiptNumber")
                        val customerId = billObj.optString("customerId").ifBlank { null }
                        val customerName = billObj.optString("customerName").ifBlank { null }
                        val grossTotal = billObj.getLong("grossTotalPaise")
                        val discountType = billObj.getString("discountType")
                        val discountValue = billObj.getLong("discountValuePaise")
                        val discountPercent = billObj.optDouble("discountPercent", 0.0)
                        val netTotal = billObj.getLong("netTotalPaise")
                        val amountReceived = billObj.getLong("amountReceivedPaise")
                        val changeGiven = billObj.optLong("changeGivenPaise", 0L)
                        val remainingCredit = billObj.optLong("remainingCreditPaise", 0L)
                        val paymentMethod = billObj.getString("paymentMethod")
                        val timestamp = billObj.getLong("timestamp")
                        val originDevice = billObj.optString("originDeviceId", "device-billing")

                        val sale = SaleEntity(
                            id = saleId,
                            receiptNumber = receiptNumber,
                            customerId = customerId,
                            customerName = customerName,
                            grossTotalPaise = grossTotal,
                            discountType = discountType,
                            discountValuePaise = discountValue,
                            discountPercent = discountPercent,
                            netTotalPaise = netTotal,
                            amountReceivedPaise = amountReceived,
                            changeGivenPaise = changeGiven,
                            remainingCreditPaise = remainingCredit,
                            paymentMethod = paymentMethod,
                            isSummaryEntry = false,
                            unresolvedStockDetails = false,
                            originDeviceId = originDevice,
                            syncStatus = "SYNCED",
                            timestamp = timestamp
                        )
                        saleDao.insertSale(sale)

                        // Insert lines and adjust stock
                        val linesArray = billObj.getJSONArray("lines")
```

```
                        val saleLines = mutableListOf<SaleLineEntity>()
                        for (j in 0 until linesArray.length()) {
                            val lineObj = linesArray.getJSONObject(j)
                            val lineType = lineObj.optString("lineType", "PRODUCT")
                            val productId = lineObj.optString("productId").ifBlank { null }
                            val productName = lineObj.getString("productName")
                            val qtyBase = lineObj.getInt("quantityBase")
                            val isPack = lineObj.optBoolean("isPack", false)
                            val unitSellingPrice = lineObj.getLong("unitSellingPricePaise")
                            val lineGross = lineObj.getLong("lineGrossPaise")
                            val lineDiscount = lineObj.optLong("lineDiscountAllocatedPaise", 0L)
                            val lineNet = lineObj.getLong("lineNetPaise")

                            // Owner phone looks up product moving average cost snapshot
                            val costSnapshot = if (lineType == "PRODUCT" && productId != null) {
                                val p = productDao.getProductById(productId)
                                p?.movingAvgCostPaise
                            } else null

                            val lineProfit = if (costSnapshot != null) {
                                lineNet - (qtyBase * costSnapshot)
                            } else null

                            val line = SaleLineEntity(
                                id = lineObj.optString("id").ifBlank { UUID.randomUUID().toString() },
                                saleId = saleId,
                                productId = productId,
                                productName = productName,
                                quantityBase = qtyBase,
                                isPack = isPack,
                                unitSellingPricePaise = unitSellingPrice,
                                unitCostSnapshotPaise = costSnapshot,
                                lineGrossPaise = lineGross,
                                lineDiscountAllocatedPaise = lineDiscount,
                                lineNetPaise = lineNet,
                                lineProfitPaise = lineProfit,
                                lineType = lineType,
                                serviceId = lineObj.optString("serviceId").ifBlank { null },
                                serviceName = lineObj.optString("serviceName").ifBlank { null },
                                serviceOption = lineObj.optString("serviceOption").ifBlank { null },
                                unitLabel = lineObj.optString("unitLabel").ifBlank { null }
                            )
                            saleLines.add(line)

                            // Deduct stock for product lines
                            if (lineType == "PRODUCT" && productId != null) {
                                productDao.adjustSellableStock(productId, -qtyBase)
                                val product = productDao.getProductById(productId)
                                stockMovementDao.insertMovement(
                                    StockMovementEntity(
                                        productId = productId,
                                        type = "SALE",
                                        quantityChangeBase = -qtyBase,
                                        unitCostPaise = product?.movingAvgCostPaise ?: 0L,
                                        reason = "Synced Bill $receiptNumber",
                                        referenceId = saleId
                                    )
                                )
                            }
                        }
                        saleDao.insertSaleLines(saleLines)

                        // Customer debt adjustment if credit
                        if (customerId != null && remainingCredit > 0) {
                            customerDao.adjustBalance(customerId, remainingCredit)
                        }
                    }

                    acknowledgedIds.put(saleId)
                }

                val responseRoot = JSONObject()
                responseRoot.put("status", "SUCCESS")
                responseRoot.put("acknowledgedIds", acknowledgedIds)

                val responseBytes = responseRoot.toString().toByteArray(Charsets.UTF_8)
                writer.write("HTTP/1.1 200 OK\r\n")
                writer.write("Content-Type: application/json; charset=utf-8\r\n")
                writer.write("Content-Length: ${responseBytes.size}\r\n")
                writer.write("\r\n")
                writer.flush()
                socket.getOutputStream().write(responseBytes)
                socket.getOutputStream().flush()
            } else {
                writer.write("HTTP/1.1 404 Not Found\r\n\r\n")
                writer.flush()
            }
        } catch (_: Exception) {
```

```
        } finally {
            try { socket.close() } catch (_: Exception) {}
        }
    }

    // ── Billing Phone Client Functions ──────────────────────────────

    suspend fun syncWithMainPhone(ownerHost: String, port: Int = 8080): Boolean =
withContext(Dispatchers.IO) {
        try {
            _connectionState.value = SyncConnectionState.Syncing("Connecting to $ownerHost:$port...")

            // 1. Fetch latest catalog
            val catalogUrl = URL("http://$ownerHost:$port/api/catalog")
            val catalogConn = (catalogUrl.openConnection() as HttpURLConnection).apply {
                connectTimeout = 4000
                readTimeout = 6000
                requestMethod = "GET"
            }

            if (catalogConn.responseCode != 200) {
                _connectionState.value = SyncConnectionState.Error("Failed to reach Main Phone (Code
${catalogConn.responseCode})")
                return@withContext false
            }

            val catalogJsonStr = catalogConn.inputStream.bufferedReader().use { it.readText() }
            val catalogRoot = JSONObject(catalogJsonStr)
            val productsArray = catalogRoot.getJSONArray("products")

            // Upsert catalog items on billing phone
            for (i in 0 until productsArray.length()) {
                val pObj = productsArray.getJSONObject(i)
                val id = pObj.getString("id")
                val existing = productDao.getProductById(id)
                val updated = (existing ?: com.alieducational.app.data.local.entity.ProductEntity(
                    id = id,
                    name = pObj.getString("name"),
                    category = pObj.getString("category")
                )).copy(
                    name = pObj.getString("name"),
                    urduName = pObj.optString("urduName").ifBlank { null },
                    category = pObj.getString("category"),
                    brand = pObj.optString("brand").ifBlank { null },
                    sku = pObj.optString("sku").ifBlank { null },
                    barcode = pObj.optString("barcode").ifBlank { null },
                    packSize = pObj.optInt("packSize", 1),
                    sellingPricePiecePaise = pObj.getLong("sellingPricePiecePaise"),
                    sellingPricePackPaise = pObj.getLong("sellingPricePackPaise"),
                    currentSellableStock = pObj.getInt("currentSellableStock")
                )
                productDao.insertProduct(updated)
            }

            // 2. Deliver pending bills to Main Phone
            val pendingSales = saleDao.getPendingSyncSales()
            if (pendingSales.isNotEmpty()) {
                _connectionState.value = SyncConnectionState.Syncing("Delivering ${pendingSales.size}
pending bills...")

                val billsArray = JSONArray()
                for (sale in pendingSales) {
                    val lines = saleDao.getLinesForSale(sale.id)
                    val sObj = JSONObject().apply {
                        put("id", sale.id)
                        put("receiptNumber", sale.receiptNumber)
                        put("customerId", sale.customerId ?: "")
                        put("customerName", sale.customerName ?: "")
                        put("grossTotalPaise", sale.grossTotalPaise)
                        put("discountType", sale.discountType)
                        put("discountValuePaise", sale.discountValuePaise)
                        put("discountPercent", sale.discountPercent)
                        put("netTotalPaise", sale.netTotalPaise)
                        put("amountReceivedPaise", sale.amountReceivedPaise)
                        put("changeGivenPaise", sale.changeGivenPaise)
                        put("remainingCreditPaise", sale.remainingCreditPaise)
                        put("paymentMethod", sale.paymentMethod)
                        put("timestamp", sale.timestamp)
                        put("originDeviceId", sale.originDeviceId ?: "device-billing")

                        val lArray = JSONArray()
                        for (line in lines) {
                            val lObj = JSONObject().apply {
                                put("id", line.id)
                                put("lineType", line.lineType)
                                put("productId", line.productId ?: "")
                                put("productName", line.productName)
                                put("quantityBase", line.quantityBase)
```

```
                                put("isPack", line.isPack)
                                put("unitSellingPricePaise", line.unitSellingPricePaise)
                                put("lineGrossPaise", line.lineGrossPaise)
                                put("lineDiscountAllocatedPaise", line.lineDiscountAllocatedPaise)
                                put("lineNetPaise", line.lineNetPaise)
                                put("serviceId", line.serviceId ?: "")
                                put("serviceName", line.serviceName ?: "")
                                put("serviceOption", line.serviceOption ?: "")
                                put("unitLabel", line.unitLabel ?: "")
                            }
                            lArray.put(lObj)
                        }
                        put("lines", lArray)
                    }
                    billsArray.put(sObj)
                }

                val billsUrl = URL("http://$ownerHost:$port/api/sync-bills")
                val billsConn = (billsUrl.openConnection() as HttpURLConnection).apply {
                    connectTimeout = 6000
                    readTimeout = 10000
                    requestMethod = "POST"
                    doOutput = true
                    setRequestProperty("Content-Type", "application/json; charset=utf-8")
                }

                val bodyBytes = billsArray.toString().toByteArray(Charsets.UTF_8)
                billsConn.outputStream.use { it.write(bodyBytes) }

                if (billsConn.responseCode == 200) {
                    val respStr = billsConn.inputStream.bufferedReader().use { it.readText() }
                    val respObj = JSONObject(respStr)
                    val ackIds = respObj.getJSONArray("acknowledgedIds")
                    for (i in 0 until ackIds.length()) {
                        val ackId = ackIds.getString(i)
                        saleDao.updateSaleSyncStatus(ackId, "SYNCED")
                    }
                } else {
                    _connectionState.value = SyncConnectionState.Error("Sync delivery failed (Code
${billsConn.responseCode})")
                    return@withContext false
                }
            }

            _connectionState.value = SyncConnectionState.Connected(ownerHost)
            true
        } catch (e: Exception) {
            _connectionState.value = SyncConnectionState.Error("Cannot reach Main Phone: ${e.message ?:
"Connection timed out"}")
            false
        }
    }
}
```

app\src\main\java\com\alieducational\app\di\DatabaseModule.kt

```kotlin
package com.alieducational.app.di

import android.content.Context
import androidx.room.Room
import com.alieducational.app.data.local.AppDatabase
import com.alieducational.app.data.local.dao.AccountPaymentDao
import com.alieducational.app.data.local.dao.CustomerDao
import com.alieducational.app.data.local.dao.HeldBillDao
import com.alieducational.app.data.local.dao.ImportBatchDao
import com.alieducational.app.data.local.dao.PigmyDepositDao
import com.alieducational.app.data.local.dao.CashCategoryDao
import com.alieducational.app.data.local.dao.PigmySchemeDao
import com.alieducational.app.data.local.dao.PigmyWithdrawalDao
import com.alieducational.app.data.local.dao.ProductBarcodeDao
import com.alieducational.app.data.local.dao.ProductDao
import com.alieducational.app.data.local.dao.ProductGiveawayDao
import com.alieducational.app.data.local.dao.PurchaseDao
import com.alieducational.app.data.local.dao.SaleDao
import com.alieducational.app.data.local.dao.ServiceDefinitionDao
import com.alieducational.app.data.local.dao.ShopExpenseDao
import com.alieducational.app.data.local.dao.StockMovementDao
import com.alieducational.app.data.local.dao.SupplierDao
import com.alieducational.app.data.local.dao.SyncEventDao
import dagger.Module
import dagger.Provides
import dagger.hilt.InstallIn
import dagger.hilt.android.qualifiers.ApplicationContext
import dagger.hilt.components.SingletonComponent
import javax.inject.Singleton

@Module
@InstallIn(SingletonComponent::class)
object DatabaseModule {

    @Provides
    @Singleton
    fun provideAppDatabase(@ApplicationContext context: Context): AppDatabase {
        return Room.databaseBuilder(
            context,
            AppDatabase::class.java,
            "ali_educational.db"
        )
        .addMigrations(
            AppDatabase.MIGRATION_1_2,
            AppDatabase.MIGRATION_2_3,
            AppDatabase.MIGRATION_3_4,
            AppDatabase.MIGRATION_4_5,
            AppDatabase.MIGRATION_5_6,
            AppDatabase.MIGRATION_6_7,
            AppDatabase.MIGRATION_7_8
        )
        .build()
    }

    @Provides
    fun provideProductDao(db: AppDatabase): ProductDao = db.productDao()

    @Provides
    fun provideProductBarcodeDao(db: AppDatabase): ProductBarcodeDao = db.productBarcodeDao()

    @Provides
    fun provideStockMovementDao(db: AppDatabase): StockMovementDao = db.stockMovementDao()

    @Provides
    fun provideSupplierDao(db: AppDatabase): SupplierDao = db.supplierDao()

    @Provides
    fun provideCustomerDao(db: AppDatabase): CustomerDao = db.customerDao()

    @Provides
    fun providePurchaseDao(db: AppDatabase): PurchaseDao = db.purchaseDao()

    @Provides
    fun provideSaleDao(db: AppDatabase): SaleDao = db.saleDao()

    @Provides
    fun provideAccountPaymentDao(db: AppDatabase): AccountPaymentDao = db.accountPaymentDao()

    @Provides
    fun provideShopExpenseDao(db: AppDatabase): ShopExpenseDao = db.shopExpenseDao()

    @Provides
```

```
    fun provideImportBatchDao(db: AppDatabase): ImportBatchDao = db.importBatchDao()

    @Provides
    fun provideHeldBillDao(db: AppDatabase): HeldBillDao = db.heldBillDao()

    @Provides
    fun providePigmyDepositDao(db: AppDatabase): PigmyDepositDao = db.pigmyDepositDao()

    @Provides
    fun providePigmySchemeDao(db: AppDatabase): PigmySchemeDao = db.pigmySchemeDao()

    @Provides
    fun provideProductGiveawayDao(db: AppDatabase): ProductGiveawayDao = db.productGiveawayDao()

    @Provides
    fun provideServiceDefinitionDao(db: AppDatabase): ServiceDefinitionDao = db.serviceDefinitionDao()

    @Provides
    fun provideSyncEventDao(db: AppDatabase): SyncEventDao = db.syncEventDao()

    @Provides
    fun provideCashCategoryDao(db: AppDatabase): CashCategoryDao = db.cashCategoryDao()

    @Provides
    fun providePigmyWithdrawalDao(db: AppDatabase): PigmyWithdrawalDao = db.pigmyWithdrawalDao()
}
```

app\src\main\java\com\alieducational\app\domain\calculation\CostCalculator.kt

```kotlin
package com.alieducational.app.domain.calculation

data class PurchaseLineInput(
    val id: String,
    val quantity: Int,
    val unitPricePaise: Long, // in paise
) {
    val totalValuePaise: Long = quantity * unitPricePaise
}

data class AllocatedDeliveryResult(
    val lineId: String,
    val goodsValuePaise: Long,
    val allocatedDeliveryPaise: Long,
    val totalLandedCostPaise: Long,
    val effectiveUnitCostPaise: Long, // rounded to nearest paise for unit display/storage
)

data class SaleLineInput(
    val id: String,
    val quantity: Int,
    val unitSellingPricePaise: Long,
    val unitCostPaise: Long?, // null if cost unknown
) {
    val grossTotalPaise: Long = quantity * unitSellingPricePaise
}

data class AllocatedDiscountLineResult(
    val lineId: String,
    val quantity: Int,
    val grossValuePaise: Long,
    val allocatedDiscountPaise: Long,
    val netValuePaise: Long,
    val unitCostPaise: Long?,
    val actualProfitPaise: Long?, // null if unitCost is null
)

object CostCalculator {

    /**
     * Proportional delivery allocation according to each line's purchase value.
     * Line delivery share = total delivery * line purchase value / total purchase value.
     * Rounding differences are reconciled so sum of allocated delivery == total delivery exactly.
     */
    fun allocateDelivery(
        lines: List<PurchaseLineInput>,
        totalDeliveryPaise: Long
    ): List<AllocatedDeliveryResult> {
        if (lines.isEmpty()) return emptyList()
        if (totalDeliveryPaise <= 0L) {
            return lines.map {
                val goods = it.totalValuePaise
                AllocatedDeliveryResult(
                    lineId = it.id,
                    goodsValuePaise = goods,
                    allocatedDeliveryPaise = 0L,
                    totalLandedCostPaise = goods,
                    effectiveUnitCostPaise = if (it.quantity > 0) goods / it.quantity else 0L
                )
            }
        }

        val totalGoodsValue = lines.sumOf { it.totalValuePaise }

        // If total goods value is 0 (e.g. all free samples), split delivery equally
        if (totalGoodsValue == 0L) {
            val count = lines.size
            val baseShare = totalDeliveryPaise / count
            var remainder = totalDeliveryPaise % count

            return lines.map { line ->
                val extra = if (remainder > 0) { remainder--; 1L } else 0L
                val alloc = baseShare + extra
                AllocatedDeliveryResult(
                    lineId = line.id,
                    goodsValuePaise = 0L,
                    allocatedDeliveryPaise = alloc,
                    totalLandedCostPaise = alloc,
                    effectiveUnitCostPaise = if (line.quantity > 0) alloc / line.quantity else 0L
                )
            }
        }
```

```
83
84          // Proportional allocation
85          val initialAllocations = lines.map { line ->
86              val share = (totalDeliveryPaise * line.totalValuePaise) / totalGoodsValue
87              line to share
88          }
89
90          val allocatedSum = initialAllocations.sumOf { it.second }
91          val discrepancy = totalDeliveryPaise - allocatedSum
92
93          // Add discrepancy to line with largest purchase value
94          val largestIndex = lines.indices.maxByOrNull { lines[it].totalValuePaise } ?: 0
95
96          return initialAllocations.mapIndexed { index, (line, initialShare) ->
97              val finalAlloc = if (index == largestIndex) initialShare + discrepancy else initialShare
98              val totalLanded = line.totalValuePaise + finalAlloc
99              val unitCost = if (line.quantity > 0) (totalLanded + (line.quantity / 2)) / line.quantity else
    0L
100
101             AllocatedDeliveryResult(
102                 lineId = line.id,
103                 goodsValuePaise = line.totalValuePaise,
104                 allocatedDeliveryPaise = finalAlloc,
105                 totalLandedCostPaise = totalLanded,
106                 effectiveUnitCostPaise = unitCost
107             )
108         }
109     }
110
111     /**
112      * Moving weighted-average cost implementation per product/variant:
113      * new average = (existing stock cost value + incoming goods cost including allocated delivery)
114      *               / (existing quantity + incoming quantity)
115      * If existing stock is zero or negative, the incoming stock establishes the new average cost.
116      */
117     fun calculateMovingWeightedAverage(
118         existingStockQuantity: Int,
119         existingAverageCostPaise: Long,
120         incomingQuantity: Int,
121         incomingTotalLandedCostPaise: Long
122     ): Long {
123         if (incomingQuantity <= 0) return existingAverageCostPaise
124         if (existingStockQuantity <= 0) {
125             return (incomingTotalLandedCostPaise + (incomingQuantity / 2)) / incomingQuantity
126         }
127
128         val existingTotalValue = existingStockQuantity.toLong() * existingAverageCostPaise
129         val combinedValue = existingTotalValue + incomingTotalLandedCostPaise
130         val combinedQuantity = (existingStockQuantity + incomingQuantity).toLong()
131
132         return (combinedValue + (combinedQuantity / 2)) / combinedQuantity
133     }
134
135     /**
136      * Allocates whole-sale discount proportionally across sale lines by line gross value.
137      * Reconciles rounding remainder onto the largest line so sum of discounts == totalDiscount.
138      */
139     fun allocateWholeSaleDiscount(
140         lines: List<SaleLineInput>,
141         totalDiscountPaise: Long
142     ): List<AllocatedDiscountLineResult> {
143         if (lines.isEmpty()) return emptyList()
144         val totalGross = lines.sumOf { it.grossTotalPaise }
145
146         if (totalDiscountPaise <= 0L || totalGross <= 0L) {
147             return lines.map { line ->
148                 val cost = line.unitCostPaise
149                 val profit = if (cost != null) line.grossTotalPaise - (line.quantity * cost) else null
150                 AllocatedDiscountLineResult(
151                     lineId = line.id,
152                     quantity = line.quantity,
153                     grossValuePaise = line.grossTotalPaise,
154                     allocatedDiscountPaise = 0L,
155                     netValuePaise = line.grossTotalPaise,
156                     unitCostPaise = cost,
157                     actualProfitPaise = profit
158                 )
159             }
160         }
161
162         val initialAllocations = lines.map { line ->
163             val share = (totalDiscountPaise * line.grossTotalPaise) / totalGross
164             line to share
165         }
166
167         val allocatedSum = initialAllocations.sumOf { it.second }
168         val discrepancy = totalDiscountPaise - allocatedSum
169         val largestIndex = lines.indices.maxByOrNull { lines[it].grossTotalPaise } ?: 0
```

```

        return initialAllocations.mapIndexed { index, (line, initialShare) ->
            val finalDiscount = if (index == largestIndex) initialShare + discrepancy else initialShare
            val net = line.grossTotalPaise - finalDiscount
            val cost = line.unitCostPaise
            val profit = if (cost != null) net - (line.quantity * cost) else null

            AllocatedDiscountLineResult(
                lineId = line.id,
                quantity = line.quantity,
                grossValuePaise = line.grossTotalPaise,
                allocatedDiscountPaise = finalDiscount,
                netValuePaise = net,
                unitCostPaise = cost,
                actualProfitPaise = profit
            )
        }
    }

    /**
     * Converts a base quantity into familiar "X boxes + Y pieces (total Z pieces)"
     */
    fun formatBoxesAndPieces(totalBaseUnits: Int, packSize: Int): Pair<Int, Int> {
        val safePackSize = if (packSize > 0) packSize else 1
        val boxes = totalBaseUnits / safePackSize
        val pieces = totalBaseUnits % safePackSize
        return Pair(boxes, pieces)
    }

    /**
     * Formats paise to Indian Rupees string e.g. 10050 -> "₹100.50"
     */
    /**
     * Parses a string representation of Rupees (e.g. "123.45", "48", "47.50", "0") into exact paise.
     * Rejects malformed strings, negative amounts, and excess decimal precision (> 2 decimals).
     */
    fun parseRupeesToPaise(input: String?): Long? {
        if (input == null) return null
        val trimmed = input.replace("₹", "").replace(",", "").trim()
        if (trimmed.isEmpty()) return null
        val parts = trimmed.split(".")
        if (parts.size > 2) return null
        val rupees = parts[0].toLongOrNull() ?: return null
        if (rupees < 0) return null
        val paise = if (parts.size == 2) {
            val decimal = parts[1]
            when (decimal.length) {
                0 -> 0L
                1 -> (decimal + "0").toLongOrNull() ?: return null
                2 -> decimal.toLongOrNull() ?: return null
                else -> return null // Reject excess precision (> 2 decimal places)
            }
        } else {
            0L
        }
        return rupees * 100L + paise
    }

    /**
     * Formats an integer value according to Indian number grouping system:
     * - Last 3 digits grouped together
     * - Preceding digits grouped in sets of 2
     * e.g. 999 -> "999", 1000 -> "1,000", 12235 -> "12,235", 100000 -> "1,00,000", 1234567 -> "12,34,567"
     */
    fun formatIndianGrouping(value: Long): String {
        val s = kotlin.math.abs(value).toString()
        if (s.length <= 3) return s
        val lastThree = s.takeLast(3)
        val remaining = s.dropLast(3)
        val sb = StringBuilder()

        var i = remaining.length
        while (i > 0) {
            val start = kotlin.math.max(0, i - 2)
            val part = remaining.substring(start, i)
            if (sb.isNotEmpty()) {
                sb.insert(0, ",")
            }
            sb.insert(0, part)
            i -= 2
        }
        return "$sb,$lastThree"
    }

    /**
     * Formats integer quantity with Indian grouping and optional unit (e.g. 1,23,456 pcs).
     */
    fun formatQuantityWithGrouping(qty: Int, unit: String? = null): String {
        val isNegative = qty < 0
```

```
        val absQty = if (isNegative) -qty else qty
        val grouped = formatIndianGrouping(absQty.toLong())
        val sign = if (isNegative) "-" else ""
        return if (unit.isNullOrBlank()) "$sign$grouped" else "$sign$grouped $unit"
    }

    fun formatPaiseToRupees(paise: Long): String {
        val isNegative = paise < 0
        val absPaise = if (isNegative) -paise else paise
        val rupees = absPaise / 100
        val remainder = absPaise % 100
        val prefix = if (isNegative) "-₹" else "₹"
        val groupedRupees = formatIndianGrouping(rupees)
        return String.format(java.util.Locale.US, "%s%s.%02d", prefix, groupedRupees, remainder)
    }
}
```

app\src\main\java\com\alieducational\app\MainActivity.kt

```
package com.alieducational.app

import android.os.Bundle
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Scaffold
import androidx.compose.runtime.CompositionLocalProvider
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Modifier
import androidx.compose.ui.platform.LocalLayoutDirection
import androidx.compose.ui.unit.LayoutDirection
import androidx.fragment.app.FragmentActivity
import androidx.hilt.navigation.compose.hiltViewModel
import androidx.navigation.compose.currentBackStackEntryAsState
import androidx.navigation.compose.rememberNavController
import com.alieducational.app.ui.AliViewModel
import com.alieducational.app.ui.navigation.AppNavigation
import com.alieducational.app.ui.navigation.BottomNavBar
import com.alieducational.app.ui.navigation.Screen
import com.alieducational.app.ui.theme.AliEducationalTheme
import com.alieducational.app.util.BiometricAuthManager
import dagger.hilt.android.AndroidEntryPoint
import javax.inject.Inject

@AndroidEntryPoint
class MainActivity : FragmentActivity() {

    @Inject
    lateinit var biometricAuthManager: BiometricAuthManager

    private var lastPausedTime: Long = 0L
    private var isAppLocked by mutableStateOf(false)
    private var pendingWidgetAction by mutableStateOf<String?>(null)

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        extractWidgetAction(intent)
        com.alieducational.app.widget.AliSlimBarWidgetProvider.updateAllWidgets(this)

        setContent {
            val viewModel: AliViewModel = hiltViewModel()
            val isDarkTheme by viewModel.isDarkTheme.collectAsState()
            val appLanguage by viewModel.appLanguage.collectAsState()
            val isFirstLaunch by viewModel.isFirstLaunch.collectAsState()
            val isLockEnabled by viewModel.isLockEnabled.collectAsState()

            val layoutDirection = LayoutDirection.Ltr

            CompositionLocalProvider(LocalLayoutDirection provides layoutDirection) {
                AliEducationalTheme(darkTheme = isDarkTheme) {
                    val navController = rememberNavController()
                    val navBackStackEntry by navController.currentBackStackEntryAsState()
                    val currentRoute = navBackStackEntry?.destination?.route

                    val startDestination = when {
                        isFirstLaunch -> Screen.Setup.route
                        isLockEnabled && isAppLocked -> Screen.Unlock.route
                        else -> Screen.Home.route
                    }

                    val mainTabRoutes = listOf(
                        Screen.Home.route,
                        Screen.Stock.route,
                        Screen.Billing.route,
                        Screen.Accounts.route
                    )

                    val showBottomBar = currentRoute in mainTabRoutes &&
                            !isFirstLaunch &&
                            (!isLockEnabled || !isAppLocked)

                    androidx.compose.runtime.LaunchedEffect(pendingWidgetAction, isLockEnabled, isAppLocked,
    isFirstLaunch) {
                        val action = pendingWidgetAction
                        if (action != null && !isFirstLaunch && (!isLockEnabled || !isAppLocked)) {
                            when (action) {
                                com.alieducational.app.widget.AliSlimBarWidgetProvider.ACTION_CASH_IN -> {

```

```
    viewModel.triggerWidgetAction(com.alieducational.app.widget.AliSlimBarWidgetProvider.ACTION_CASH_IN)
 85                                 navController.navigate(Screen.CashTracker.route) {
 86                                     launchSingleTop = true
 87                                 }
 88                             }
 89                             com.alieducational.app.widget.AliSlimBarWidgetProvider.ACTION_CASH_OUT -> {
 90
    viewModel.triggerWidgetAction(com.alieducational.app.widget.AliSlimBarWidgetProvider.ACTION_CASH_OUT)
 91                                 navController.navigate(Screen.CashTracker.route) {
 92                                     launchSingleTop = true
 93                                 }
 94                             }
 95                             com.alieducational.app.widget.AliSlimBarWidgetProvider.ACTION_NEW_BILL -> {
 96                                 navController.navigate(Screen.Billing.route) {
 97                                     launchSingleTop = true
 98                                 }
 99                             }
100                             com.alieducational.app.widget.AliSlimBarWidgetProvider.ACTION_ADD_STOCK -> {
101
    viewModel.triggerWidgetAction(com.alieducational.app.widget.AliSlimBarWidgetProvider.ACTION_ADD_STOCK)
102                                 navController.navigate(Screen.Stock.route) {
103                                     launchSingleTop = true
104                                 }
105                             }
106                             com.alieducational.app.widget.AliSlimBarWidgetProvider.ACTION_HOME -> {
107                                 navController.navigate(Screen.Home.route) {
108                                     launchSingleTop = true
109                                 }
110                             }
111                         }
112                         pendingWidgetAction = null
113                     }
114                 }
115
116                 Scaffold(
117                     modifier = Modifier.fillMaxSize(),
118                     bottomBar = {
119                         if (showBottomBar) {
120                             BottomNavBar(navController = navController)
121                         }
122                     }
123                 ) { innerPadding ->
124                     AppNavigation(
125                         navController = navController,
126                         startDestination = startDestination,
127                         viewModel = viewModel,
128                         biometricAuthManager = biometricAuthManager,
129                         onUnlocked = { isAppLocked = false },
130                         modifier = Modifier.padding(innerPadding)
131                     )
132                 }
133             }
134         }
135     }
136
137     override fun onPause() {
138         super.onPause()
139         lastPausedTime = System.currentTimeMillis()
140     }
141
142     override fun onNewIntent(intent: android.content.Intent) {
143         super.onNewIntent(intent)
144         setIntent(intent)
145         extractWidgetAction(intent)
146     }
147
148     private fun extractWidgetAction(intent: android.content.Intent?) {
149         if (intent?.action == "com.alieducational.app.action.PIN_WIDGET" || intent?.getStringExtra("action")
     == "PIN_WIDGET") {
150             requestPinWidget()
151             return
152         }
153         val action =
    intent?.getStringExtra(com.alieducational.app.widget.AliSlimBarWidgetProvider.EXTRA_WIDGET_ACTION)
154         if (action != null) {
155             pendingWidgetAction = action
156         }
157     }
158
159     fun requestPinWidget() {
160         if (android.os.Build.VERSION.SDK_INT >= android.os.Build.VERSION_CODES.O) {
161             val appWidgetManager = getSystemService(android.appwidget.AppWidgetManager::class.java)
162             val myProvider = android.content.ComponentName(this,
    com.alieducational.app.widget.AliSlimBarWidgetProvider::class.java)
163             if (appWidgetManager != null && appWidgetManager.isRequestPinAppWidgetSupported) {
164                 appWidgetManager.requestPinAppWidget(myProvider, null, null)
165             }
166
```

```
        }
        com.alieducational.app.widget.AliSlimBarWidgetProvider.updateAllWidgets(this)
    }

    override fun onResume() {
        super.onResume()
        com.alieducational.app.widget.AliSlimBarWidgetProvider.updateAllWidgets(this)
        if (lastPausedTime > 0L) {
            val elapsedSeconds = (System.currentTimeMillis() - lastPausedTime) / 1000
            // If background timeout exceeded (default 120s), relock
            if (elapsedSeconds >= 120) {
                isAppLocked = true
            }
        }
    }
}
```

app\src\main\java\com\alieducational\app\MainViewModel.kt

```kotlin
package com.alieducational.app

import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.alieducational.app.data.preferences.AppPreferences
import dagger.hilt.android.lifecycle.HiltViewModel
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.flow.first
import kotlinx.coroutines.launch
import javax.inject.Inject

data class MainUiState(
    val isLoading: Boolean = true,
    val isFirstLaunch: Boolean = true,
    val isLockEnabled: Boolean = false,
    val isDarkTheme: Boolean = true,
)

@HiltViewModel
class MainViewModel @Inject constructor(
    private val appPreferences: AppPreferences,
) : ViewModel() {

    private val _uiState = MutableStateFlow(MainUiState())
    val uiState: StateFlow<MainUiState> = _uiState.asStateFlow()

    init {
        viewModelScope.launch {
            val isFirstLaunch = appPreferences.isFirstLaunch.first()
            val isLockEnabled = appPreferences.isLockEnabled.first()
            val themeMode = appPreferences.themeMode.first()

            _uiState.value = MainUiState(
                isLoading = false,
                isFirstLaunch = isFirstLaunch,
                isLockEnabled = isLockEnabled,
                isDarkTheme = themeMode == "dark",
            )
        }
    }
}
```

app\src\main\java\com\alieducational\app\ui\accounts\AccountsScreen.kt

```
package com.alieducational.app.ui.accounts

import android.content.Intent
import androidx.compose.foundation.BorderStroke
import androidx.compose.foundation.background
import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.foundation.shape.CircleShape
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Add
import androidx.compose.material.icons.filled.ArrowDownward
import androidx.compose.material.icons.filled.ArrowUpward
import androidx.compose.material.icons.filled.Check
import androidx.compose.material.icons.filled.Payment
import androidx.compose.material.icons.filled.Phone
import androidx.compose.material.icons.filled.Share
import androidx.compose.material.icons.filled.Search
import androidx.compose.material.icons.filled.Close
import androidx.compose.material.icons.filled.DeleteOutline
import androidx.compose.material.icons.filled.ReceiptLong
import androidx.compose.ui.text.style.TextAlign

import androidx.compose.material3.AlertDialog
import androidx.compose.material3.Button
import androidx.compose.material3.ButtonDefaults
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.FilterChip
import androidx.compose.material3.FloatingActionButton
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedButton
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Surface
import androidx.compose.material3.Tab
import androidx.compose.material3.TabRow
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.res.stringResource
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.alieducational.app.R
import com.alieducational.app.data.local.entity.CustomerEntity
import com.alieducational.app.data.local.entity.SupplierEntity
import com.alieducational.app.domain.calculation.CostCalculator
import com.alieducational.app.ui.AliViewModel
import com.alieducational.app.ui.theme.*
import java.text.SimpleDateFormat
import java.util.Date
import java.util.Locale

@OptIn(ExperimentalMaterial3Api::class)
@Composable
```

```
fun AccountsScreen(
    viewModel: AliViewModel,
    initialTab: Int = 0,
    onNavigateBack: () -> Unit = {},
    onNavigateToBillHistory: () -> Unit = {},
    modifier: Modifier = Modifier
) {
    androidx.activity.compose.BackHandler {
        onNavigateBack()
    }
    var selectedTab by remember { mutableStateOf(initialTab) } // 0 = We Owe (Suppliers), 1 = Customers Owe Us
    val suppliers by viewModel.allSuppliers.collectAsState()
    val customers by viewModel.allCustomers.collectAsState()

    val totalWeOwe by viewModel.totalWeOweSuppliersPaise.collectAsState()
    val totalCustomersOwe by viewModel.totalCustomersOweUsPaise.collectAsState()
    val distinctCreditors by viewModel.distinctCreditorsCount.collectAsState(initial = 0)
    val distinctDebtors by viewModel.distinctDebtorsCount.collectAsState(initial = 0)

    var showOnlyOutstanding by remember { mutableStateOf(false) }
    var searchQuery by remember { mutableStateOf("") }

    val filteredSuppliers = remember(suppliers, showOnlyOutstanding, searchQuery) {
        suppliers.filter { sup ->
            val matchesOutstanding = if (showOnlyOutstanding) sup.currentBalancePaise > 0L else true
            val matchesSearch = if (searchQuery.isBlank()) true else {
                sup.name.contains(searchQuery, ignoreCase = true) || (sup.phone?.contains(searchQuery) == true)
            }
            matchesOutstanding && matchesSearch
        }
    }

    val filteredCustomers = remember(customers, showOnlyOutstanding, searchQuery) {
        customers.filter { cust ->
            val matchesOutstanding = if (showOnlyOutstanding) cust.currentBalancePaise > 0L else true
            val matchesSearch = if (searchQuery.isBlank()) true else {
                cust.name.contains(searchQuery, ignoreCase = true) || (cust.phone?.contains(searchQuery) == true)
            }
            matchesOutstanding && matchesSearch
        }
    }

    var showAddSupplierDialog by remember { mutableStateOf(false) }
    var showAddCustomerDialog by remember { mutableStateOf(false) }

    var paymentTargetSupplier by remember { mutableStateOf<SupplierEntity?>(null) }
    var paymentTargetCustomer by remember { mutableStateOf<CustomerEntity?>(null) }
    var removeTargetSupplier by remember { mutableStateOf<SupplierEntity?>(null) }
    var removeTargetCustomer by remember { mutableStateOf<CustomerEntity?>(null) }
    var completedVoucher by remember { mutableStateOf<PaymentVoucher?>(null) }
    val shopName by viewModel.shopName.collectAsState()
    val isDarkTheme by viewModel.isDarkTheme.collectAsState()
    val positiveColor = if (isDarkTheme) PositiveGreenDark else PositiveGreenLight
    val negativeColor = if (isDarkTheme) NegativeRedDark else NegativeRedLight

    Surface(
        modifier = modifier.fillMaxSize(),
        color = MaterialTheme.colorScheme.background
    ) {
            Column(modifier = Modifier.fillMaxSize()) {

                // Tab Row: We Owe vs Customers Owe Us
                TabRow(
                    selectedTabIndex = selectedTab,
                    containerColor = MaterialTheme.colorScheme.surface,
                    contentColor = MaterialTheme.colorScheme.primary
                ) {
                    Tab(
                        selected = selectedTab == 0,
                        onClick = { selectedTab = 0 },
                        text = {
                            Row(verticalAlignment = Alignment.CenterVertically) {
                                Icon(Icons.Filled.ArrowUpward, contentDescription = null, tint = negativeColor, modifier = Modifier.size(16.dp))
                                Spacer(modifier = Modifier.width(6.dp))
                                Text("We Owe (${suppliers.size})", fontWeight = FontWeight.Bold)
                            }
                        }
                    )
                    Tab(
                        selected = selectedTab == 1,
                        onClick = { selectedTab = 1 },
                        text = {
                            Row(verticalAlignment = Alignment.CenterVertically) {
                                Icon(Icons.Filled.ArrowDownward, contentDescription = null, tint = positiveColor, modifier = Modifier.size(16.dp))
                                Spacer(modifier = Modifier.width(6.dp))
```

```
                                Text("Customers Owe (${customers.size})", fontWeight = FontWeight.Bold)
                            }
                        }
                    )
                }

                // Balance Banner Card
                Card(
                    modifier = Modifier
                        .fillMaxWidth()
                        .padding(horizontal = 16.dp, vertical = 8.dp),
                    colors = CardDefaults.cardColors(
                        containerColor = if (isDarkTheme) {
                            if (selectedTab == 0) NegativeRedDark.copy(alpha = 0.15f)
                            else PositiveGreenDark.copy(alpha = 0.15f)
                        } else {
                            if (selectedTab == 0) Color(0xFFFFEBEE)
                            else Color(0xFFE8F5E9)
                        }
                    ),
                    border = BorderStroke(
                        1.dp,
                        if (selectedTab == 0) negativeColor.copy(alpha = 0.35f)
                        else positiveColor.copy(alpha = 0.35f)
                    ),
                    shape = RoundedCornerShape(14.dp)
                ) {
                    Column(
                        modifier = Modifier
                            .fillMaxWidth()
                            .padding(16.dp)
                    ) {
                        Text(
                            text = if (selectedTab == 0) "Amount We Owe Suppliers" else "Amount Customers Owe Us",
                            style = MaterialTheme.typography.bodyMedium.copy(fontWeight = FontWeight.Medium),
                            color = MaterialTheme.colorScheme.onSurface
                        )
                        Spacer(modifier = Modifier.height(4.dp))
                        Text(
                            text = CostCalculator.formatPaiseToRupees(
                                if (selectedTab == 0) totalWeOwe ?: 0L else totalCustomersOwe ?: 0L
                            ),
                            style = MaterialTheme.typography.headlineMedium.copy(fontWeight = FontWeight.Bold),
                            color = if (selectedTab == 0) negativeColor else positiveColor
                        )
                        Spacer(modifier = Modifier.height(4.dp))
                        Text(
                            text = if (selectedTab == 0) {
                                if (distinctCreditors > 0) "$distinctCreditors suppliers with outstanding dues" else "No outstanding supplier balances"
                            } else {
                                if (distinctDebtors > 0) "$distinctDebtors customers with outstanding dues" else "No outstanding customer balances"
                            },
                            style = MaterialTheme.typography.bodySmall,
                            color = MaterialTheme.colorScheme.onSurfaceVariant
                        )
                    }
                }

                // Dedicated Full-Width Add Action Button Row
                Button(
                    onClick = {
                        if (selectedTab == 0) showAddSupplierDialog = true
                        else showAddCustomerDialog = true
                    },
                    shape = RoundedCornerShape(10.dp),
                    colors = ButtonDefaults.buttonColors(
                        containerColor = if (selectedTab == 0) negativeColor else positiveColor,
                        contentColor = Color.White
                    ),
                    modifier = Modifier
                        .fillMaxWidth()
                        .padding(horizontal = 16.dp, vertical = 4.dp)
                        .height(46.dp)
                ) {
                    Icon(Icons.Filled.Add, contentDescription = null, modifier = Modifier.size(18.dp))
                    Spacer(modifier = Modifier.width(6.dp))
                    Text(
                        if (selectedTab == 0) "Add Supplier" else "Add Customer",
                        fontWeight = FontWeight.Bold,
                        fontSize = 14.sp
                    )
                }

```

```
                // Dedicated Full-Width Search Field Row
                OutlinedTextField(
                    value = searchQuery,
                    onValueChange = { searchQuery = it },
                    placeholder = { Text(if (selectedTab == 0) "Search suppliers..." else "Search customers...", fontSize = 13.sp) },
                    leadingIcon = { Icon(Icons.Filled.Search, contentDescription = null, modifier = Modifier.size(18.dp)) },
                    trailingIcon = if (searchQuery.isNotEmpty()) {
                        {
                            IconButton(onClick = { searchQuery = "" }) {
                                Icon(Icons.Filled.Close, contentDescription = "Clear", modifier = Modifier.size(18.dp))
                            }
                        }
                    } else null,
                    singleLine = true,
                    modifier = Modifier
                        .fillMaxWidth()
                        .padding(horizontal = 16.dp, vertical = 4.dp),
                    shape = RoundedCornerShape(10.dp)
                )

                // Dedicated Filter Chips Row (All vs Outstanding)
                Row(
                    modifier = Modifier
                        .fillMaxWidth()
                        .padding(horizontal = 16.dp, vertical = 4.dp),
                    horizontalArrangement = Arrangement.spacedBy(8.dp),
                    verticalAlignment = Alignment.CenterVertically
                ) {
                    FilterChip(
                        selected = !showOnlyOutstanding,
                        onClick = { showOnlyOutstanding = false },
                        label = { Text("All", fontWeight = if (!showOnlyOutstanding) FontWeight.Bold else FontWeight.Normal) }
                    )
                    FilterChip(
                        selected = showOnlyOutstanding,
                        onClick = { showOnlyOutstanding = true },
                        label = { Text("Outstanding", fontWeight = if (showOnlyOutstanding) FontWeight.Bold else FontWeight.Normal) }
                    )
                }

                Spacer(modifier = Modifier.height(4.dp))

                // Accounts List or Explanatory Empty State
                if (selectedTab == 0) {
                    // Suppliers List
                    if (filteredSuppliers.isEmpty()) {
                        Box(
                            modifier = Modifier
                                .fillMaxWidth()
                                .padding(top = 36.dp, start = 24.dp, end = 24.dp),
                            contentAlignment = Alignment.Center
                        ) {
                            val emptyMsg = when {
                                suppliers.isEmpty() -> "No suppliers added yet.\nTap 'Add Supplier' above to create one."
                                showOnlyOutstanding && suppliers.none { it.currentBalancePaise > 0L } -> "No outstanding supplier balances.\nAll supplier dues are cleared."
                                else -> "No matching suppliers found."
                            }
                            Text(
                                emptyMsg,
                                color = MaterialTheme.colorScheme.onSurfaceVariant,
                                textAlign = TextAlign.Center,
                                style = MaterialTheme.typography.bodyMedium
                            )
                        }
                    } else {
                        LazyColumn(
                            modifier = Modifier
                                .fillMaxWidth()
                                .weight(1f)
                                .padding(horizontal = 16.dp),
                            verticalArrangement = Arrangement.spacedBy(10.dp)
                        ) {
                            items(filteredSuppliers) { supplier ->
                                SupplierAccountCard(
                                    supplier = supplier,
                                    isDarkTheme = isDarkTheme,
                                    onRecordPayment = { paymentTargetSupplier = supplier },
                                    onRemoveSupplier = { removeTargetSupplier = supplier }
                                )
                            }
                            item { Spacer(modifier = Modifier.height(72.dp)) }
```

```
                        }
                    }
                } else {
                    // Customers List
                    if (filteredCustomers.isEmpty()) {
                        Box(
                            modifier = Modifier
                                .fillMaxWidth()
                                .padding(top = 36.dp, start = 24.dp, end = 24.dp),
                            contentAlignment = Alignment.Center
                        ) {
                            val emptyMsg = when {
                                customers.isEmpty() -> "No customers added yet.\nTap 'Add Customer' above to create one."
                                showOnlyOutstanding && customers.none { it.currentBalancePaise > 0L } -> "No outstanding customer balances.\nAll customer credit dues are cleared."
                                else -> "No matching customers found."
                            }
                            Text(
                                emptyMsg,
                                color = MaterialTheme.colorScheme.onSurfaceVariant,
                                textAlign = TextAlign.Center,
                                style = MaterialTheme.typography.bodyMedium
                            )
                        }
                    } else {
                        LazyColumn(
                            modifier = Modifier
                                .fillMaxWidth()
                                .weight(1f)
                                .padding(horizontal = 16.dp),
                            verticalArrangement = Arrangement.spacedBy(10.dp)
                        ) {
                            items(filteredCustomers) { customer ->
                                CustomerAccountCard(
                                    customer = customer,
                                    isDarkTheme = isDarkTheme,
                                    onRecordRepayment = { paymentTargetCustomer = customer },
                                    onNavigateToBillHistory = onNavigateToBillHistory,
                                    onRemoveCustomer = { removeTargetCustomer = customer }
                                )
                            }
                            item { Spacer(modifier = Modifier.height(72.dp)) }
                        }
                    }
                }
            }
        }

    // Add Supplier Dialog
    if (showAddSupplierDialog) {
        var name by remember { mutableStateOf("") }
        var phone by remember { mutableStateOf("") }
        var openingBalanceStr by remember { mutableStateOf("0") }

        AlertDialog(
            onDismissRequest = { showAddSupplierDialog = false },
            title = { Text("Add Supplier Account") },
            text = {
                Column(verticalArrangement = Arrangement.spacedBy(8.dp)) {
                    OutlinedTextField(
                        value = name,
                        onValueChange = { name = it },
                        label = { Text("Supplier / Company Name *") },
                        singleLine = true,
                        modifier = Modifier.fillMaxWidth()
                    )
                    OutlinedTextField(
                        value = phone,
                        onValueChange = { phone = it },
                        label = { Text("Phone Number") },
                        keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Phone),
                        singleLine = true,
                        modifier = Modifier.fillMaxWidth()
                    )
                    OutlinedTextField(
                        value = openingBalanceStr,
                        onValueChange = { openingBalanceStr = it },
                        label = { Text("Old Outstanding Balance (₹)") },
                        keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Decimal),
                        singleLine = true,
                        modifier = Modifier.fillMaxWidth()
                    )
                }
            },
            confirmButton = {
                Button(
                    onClick = {
```

```
                        if (name.isBlank()) return@Button
                        val balancePaise = ((openingBalanceStr.toDoubleOrNull() ?: 0.0) * 100).toLong()
                        viewModel.addSupplier(
                            SupplierEntity(
                                name = name.trim(),
                                phone = phone.trim().ifBlank { null },
                                currentBalancePaise = balancePaise
                            )
                        ) {
                            showAddSupplierDialog = false
                        }
                    }
                ) { Text("Save") }
            },
            dismissButton = {
                OutlinedButton(onClick = { showAddSupplierDialog = false }) { Text("Cancel") }
            }
        )
    }

    // Add Customer Dialog
    if (showAddCustomerDialog) {
        var name by remember { mutableStateOf("") }
        var phone by remember { mutableStateOf("") }
        var openingBalanceStr by remember { mutableStateOf("0") }

        AlertDialog(
            onDismissRequest = { showAddCustomerDialog = false },
            title = { Text("Add Customer Account") },
            text = {
                Column(verticalArrangement = Arrangement.spacedBy(8.dp)) {
                    OutlinedTextField(
                        value = name,
                        onValueChange = { name = it },
                        label = { Text("Customer Name *") },
                        singleLine = true,
                        modifier = Modifier.fillMaxWidth()
                    )
                    OutlinedTextField(
                        value = phone,
                        onValueChange = { phone = it },
                        label = { Text("Phone Number") },
                        keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Phone),
                        singleLine = true,
                        modifier = Modifier.fillMaxWidth()
                    )
                    OutlinedTextField(
                        value = openingBalanceStr,
                        onValueChange = { openingBalanceStr = it },
                        label = { Text("Old Credit Due (₹)") },
                        keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Decimal),
                        singleLine = true,
                        modifier = Modifier.fillMaxWidth()
                    )
                }
            },
            confirmButton = {
                Button(
                    onClick = {
                        if (name.isBlank()) return@Button
                        val balancePaise = ((openingBalanceStr.toDoubleOrNull() ?: 0.0) * 100).toLong()
                        viewModel.addCustomer(
                            CustomerEntity(
                                name = name.trim(),
                                phone = phone.trim().ifBlank { null },
                                currentBalancePaise = balancePaise
                            )
                        ) {
                            showAddCustomerDialog = false
                        }
                    }
                ) { Text("Save") }
            },
            dismissButton = {
                OutlinedButton(onClick = { showAddCustomerDialog = false }) { Text("Cancel") }
            }
        )
    }

    // Record Payment Dialog
    if (paymentTargetSupplier != null || paymentTargetCustomer != null) {
        val isSupplier = paymentTargetSupplier != null
        val accountName = paymentTargetSupplier?.name ?: paymentTargetCustomer?.name ?: ""
        val currentBalance = paymentTargetSupplier?.currentBalancePaise ?:
paymentTargetCustomer?.currentBalancePaise ?: 0L

        var amountStr by remember { mutableStateOf("") }
        var paymentMethod by remember { mutableStateOf("CASH") }
        var notes by remember { mutableStateOf("") }
```

```
504
505         AlertDialog(
506             onDismissRequest = {
507                 paymentTargetSupplier = null
508                 paymentTargetCustomer = null
509             },
510             title = {
511                 Text(if (isSupplier) "Pay Supplier: $accountName" else "Receive Repayment: $accountName")
512             },
513             text = {
514                 Column(verticalArrangement = Arrangement.spacedBy(10.dp)) {
515                     Text(
516                         text = "Current Balance: ${CostCalculator.formatPaiseToRupees(currentBalance)}",
517                         fontWeight = FontWeight.Bold,
518                         color = if (isSupplier) Color(0xFFE57373) else Color(0xFF75B890)
519                     )
520
521                     // Payment Method FilterChips
522                     Row(
523                         modifier = Modifier.fillMaxWidth(),
524                         horizontalArrangement = Arrangement.spacedBy(8.dp)
525                     ) {
526                         FilterChip(
527                             selected = paymentMethod == "CASH",
528                             onClick = { paymentMethod = "CASH" },
529                             label = { Text("Cash") },
530                             modifier = Modifier.weight(1f)
531                         )
532                         FilterChip(
533                             selected = paymentMethod == "UPI",
534                             onClick = { paymentMethod = "UPI" },
535                             label = { Text("UPI") },
536                             modifier = Modifier.weight(1f)
537                         )
538                         FilterChip(
539                             selected = paymentMethod == "BANK",
540                             onClick = { paymentMethod = "BANK" },
541                             label = { Text("Bank") },
542                             modifier = Modifier.weight(1f)
543                         )
544                     }
545
546                     OutlinedTextField(
547                         value = amountStr,
548                         onValueChange = { amountStr = it },
549                         label = { Text("Payment Amount (₹) *") },
550                         keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Decimal),
551                         singleLine = true,
552                         modifier = Modifier.fillMaxWidth()
553                     )
554                     OutlinedTextField(
555                         value = notes,
556                         onValueChange = { notes = it },
557                         label = { Text("Notes / Reference (Optional)") },
558                         singleLine = true,
559                         modifier = Modifier.fillMaxWidth()
560                     )
561                 }
562             },
563             confirmButton = {
564                 Button(
565                     onClick = {
566                         val amountPaise = ((amountStr.toDoubleOrNull() ?: 0.0) * 100).toLong()
567                         if (amountPaise <= 0) return@Button
568
569                         val prevBal = currentBalance
570                         val remBal = prevBal - amountPaise
571
572                         if (isSupplier) {
573                             val sup = paymentTargetSupplier!!
574                             viewModel.recordAccountPayment(
575                                 accountType = "SUPPLIER",
576                                 accountId = sup.id,
577                                 amountPaise = amountPaise,
578                                 paymentMethod = paymentMethod,
579                                 notes = notes.trim().ifBlank { null },
580                                 receiptRef = null,
581                                 onDone = {
582                                     paymentTargetSupplier = null
583                                     completedVoucher = PaymentVoucher(
584                                         accountName = accountName,
585                                         accountType = "SUPPLIER",
586                                         amountPaise = amountPaise,
587                                         paymentMethod = paymentMethod,
588                                         previousBalancePaise = prevBal,
589                                         remainingBalancePaise = remBal,
590                                         notes = notes.trim().ifBlank { null }
591                                     )
```

```
                                }
                            )
                        } else {
                            val cust = paymentTargetCustomer!!
                            viewModel.recordAccountPayment(
                                accountType = "CUSTOMER",
                                accountId = cust.id,
                                amountPaise = amountPaise,
                                paymentMethod = paymentMethod,
                                notes = notes.trim().ifBlank { null },
                                receiptRef = null,
                                onDone = {
                                    paymentTargetCustomer = null
                                    completedVoucher = PaymentVoucher(
                                        accountName = accountName,
                                        accountType = "CUSTOMER",
                                        amountPaise = amountPaise,
                                        paymentMethod = paymentMethod,
                                        previousBalancePaise = prevBal,
                                        remainingBalancePaise = remBal,
                                        notes = notes.trim().ifBlank { null }
                                    )
                                }
                            )
                        }
                    }
                ) { Text("Confirm Payment") }
            },
            dismissButton = {
                OutlinedButton(
                    onClick = {
                        paymentTargetSupplier = null
                        paymentTargetCustomer = null
                    }
                ) { Text("Cancel") }
            }
        )
    }

    // Payment Voucher Dialog (Option 2)
    completedVoucher?.let { voucher ->
        PaymentVoucherDialog(
            voucher = voucher,
            shopName = shopName,
            onDismiss = { completedVoucher = null }
        )
    }

    // Remove Supplier Confirmation Dialog
    if (removeTargetSupplier != null) {
        val sup = removeTargetSupplier!!
        AlertDialog(
            onDismissRequest = { removeTargetSupplier = null },
            title = { Text("Remove Supplier Account") },
            text = {
                Column(verticalArrangement = Arrangement.spacedBy(8.dp)) {
                    Text("Are you sure you want to remove supplier '${sup.name}'?")
                    if (sup.currentBalancePaise > 0) {
                        Text(
                            "Warning: You currently owe this supplier ${CostCalculator.formatPaiseToRupees(sup.currentBalancePaise)}. Removing this account will clear the balance from active records.",
                            color = MaterialTheme.colorScheme.error,
                            fontWeight = FontWeight.SemiBold
                        )
                    }
                }
            },
            confirmButton = {
                Button(
                    onClick = {
                        viewModel.deleteSupplier(sup.id) {
                            removeTargetSupplier = null
                        }
                    },
                    colors = ButtonDefaults.buttonColors(containerColor = MaterialTheme.colorScheme.error)
                ) {
                    Text("Remove", color = Color.White)
                }
            },
            dismissButton = {
                OutlinedButton(onClick = { removeTargetSupplier = null }) {
                    Text("Cancel")
                }
            }
        )
    }

```

```
    // Remove Customer Confirmation Dialog
    if (removeTargetCustomer != null) {
        val cust = removeTargetCustomer!!
        AlertDialog(
            onDismissRequest = { removeTargetCustomer = null },
            title = { Text("Remove Customer Account") },
            text = {
                Column(verticalArrangement = Arrangement.spacedBy(8.dp)) {
                    Text("Are you sure you want to remove customer '${cust.name}'?")
                    if (cust.currentBalancePaise > 0) {
                        Text(
                            "Warning: This customer currently owes you
${CostCalculator.formatPaiseToRupees(cust.currentBalancePaise)}. Removing this account will clear the
balance from active records.",
                            color = MaterialTheme.colorScheme.error,
                            fontWeight = FontWeight.SemiBold
                        )
                    }
                }
            },
            confirmButton = {
                Button(
                    onClick = {
                        viewModel.deleteCustomer(cust.id) {
                            removeTargetCustomer = null
                        }
                    },
                    colors = ButtonDefaults.buttonColors(containerColor = MaterialTheme.colorScheme.error)
                ) {
                    Text("Remove", color = Color.White)
                }
            },
            dismissButton = {
                OutlinedButton(onClick = { removeTargetCustomer = null }) {
                    Text("Cancel")
                }
            }
        )
    }
}

data class PaymentVoucher(
    val accountName: String,
    val accountType: String, // "CUSTOMER" or "SUPPLIER"
    val amountPaise: Long,
    val paymentMethod: String,
    val previousBalancePaise: Long,
    val remainingBalancePaise: Long,
    val timestamp: Long = System.currentTimeMillis(),
    val notes: String? = null
)

@Composable
fun PaymentVoucherDialog(
    voucher: PaymentVoucher,
    shopName: String,
    onDismiss: () -> Unit
) {
    val context = LocalContext.current
    val isSupplier = voucher.accountType == "SUPPLIER"
    val formattedDate = remember(voucher.timestamp) {
        SimpleDateFormat("dd MMMM yyyy, hh:mm a", Locale.getDefault()).format(Date(voucher.timestamp))
    }

    AlertDialog(
        onDismissRequest = onDismiss,
        title = {
            Row(verticalAlignment = Alignment.CenterVertically) {
                Box(
                    modifier = Modifier
                        .size(32.dp)
                        .clip(CircleShape)
                        .background(MaterialTheme.colorScheme.primary),
                    contentAlignment = Alignment.Center
                ) {
                    Icon(
                        imageVector = Icons.Filled.Check,
                        contentDescription = null,
                        tint = MaterialTheme.colorScheme.onPrimary,
                        modifier = Modifier.size(20.dp)
                    )
                }
                Spacer(modifier = Modifier.width(10.dp))
                Text(
                    text = stringResource(R.string.voucher_title),
                    style = MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.Bold)
                )
            }
```

```
764         },
765         text = {
766             Card(
767                 colors = CardDefaults.cardColors(containerColor =
    MaterialTheme.colorScheme.surfaceVariant.copy(alpha = 0.5f)),
768                 shape = RoundedCornerShape(12.dp),
769                 modifier = Modifier.fillMaxWidth()
770             ) {
771                 Column(
772                     modifier = Modifier
773                         .fillMaxWidth()
774                         .padding(14.dp),
775                     verticalArrangement = Arrangement.spacedBy(8.dp)
776                 ) {
777                     Text(
778                         text = shopName,
779                         style = MaterialTheme.typography.titleSmall.copy(fontWeight = FontWeight.Bold),
780                         color = MaterialTheme.colorScheme.onSurface
781                     )
782                     Text(
783                         text = if (isSupplier) "Supplier Payment Paid" else "Debt Repayment Received",
784                         style = MaterialTheme.typography.labelSmall.copy(fontWeight = FontWeight.Bold),
785                         color = MaterialTheme.colorScheme.primary
786                     )
787                     Text(
788                         text = "${if (isSupplier) "Supplier" else "Customer"}: ${voucher.accountName}",
789                         style = MaterialTheme.typography.bodyMedium.copy(fontWeight = FontWeight.SemiBold)
790                     )
791                     Text(
792                         text = "Date: $formattedDate",
793                         style = MaterialTheme.typography.bodySmall,
794                         color = MaterialTheme.colorScheme.onSurfaceVariant
795                     )
796                     Text(
797                         text = "${stringResource(R.string.voucher_payment_mode)}: ${voucher.paymentMethod}",
798                         style = MaterialTheme.typography.bodySmall,
799                         color = MaterialTheme.colorScheme.onSurfaceVariant
800                     )
801 
802                     Spacer(modifier = Modifier.height(4.dp))
803                     Box(modifier =
    Modifier.fillMaxWidth().height(1.dp).background(MaterialTheme.colorScheme.outline.copy(alpha = 0.2f)))
804                     Spacer(modifier = Modifier.height(4.dp))
805 
806                     Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement =
    Arrangement.SpaceBetween) {
807                         Text(stringResource(R.string.voucher_amount_paid), style =
    MaterialTheme.typography.bodyMedium)
808                         Text(
809                             CostCalculator.formatPaiseToRupees(voucher.amountPaise),
810                             style = MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.Bold),
811                             color = MaterialTheme.colorScheme.primary
812                         )
813                     }
814 
815                     Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement =
    Arrangement.SpaceBetween) {
816                         Text("Previous Balance", style = MaterialTheme.typography.bodySmall, color =
    MaterialTheme.colorScheme.onSurfaceVariant)
817                         Text(CostCalculator.formatPaiseToRupees(voucher.previousBalancePaise), style =
    MaterialTheme.typography.bodySmall)
818                     }
819 
820                     Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement =
    Arrangement.SpaceBetween) {
821                         Text(stringResource(R.string.voucher_remaining_balance), style =
    MaterialTheme.typography.bodyMedium.copy(fontWeight = FontWeight.Bold))
822                         Text(
823                             CostCalculator.formatPaiseToRupees(voucher.remainingBalancePaise),
824                             style = MaterialTheme.typography.bodyMedium.copy(fontWeight = FontWeight.Bold),
825                             color = if (voucher.remainingBalancePaise > 0) Color(0xFFD32F2F) else
    Color(0xFF388E3C)
826                         )
827                     }
828 
829                     if (!voucher.notes.isNullOrBlank()) {
830                         Text(
831                             text = "Note: ${voucher.notes}",
832                             style = MaterialTheme.typography.bodySmall,
833                             color = MaterialTheme.colorScheme.onSurfaceVariant
834                         )
835                     }
836                 }
837             }
838         },
839         confirmButton = {
840             Button(
841                 onClick = {
```

```
                    val shareMsg = """
                        *$shopName - Payment Voucher*
                        ${if (isSupplier) "Supplier" else "Customer"}: ${voucher.accountName}
                        Date: $formattedDate
                        Payment Mode: ${voucher.paymentMethod}
                        Amount Paid: ${CostCalculator.formatPaiseToRupees(voucher.amountPaise)}
                        Previous Balance:
${CostCalculator.formatPaiseToRupees(voucher.previousBalancePaise)}
                        Remaining Balance:
${CostCalculator.formatPaiseToRupees(voucher.remainingBalancePaise)}
                        Thank you! Stored safely offline.
                    """.trimIndent()
                    val intent = Intent().apply {
                        action = Intent.ACTION_SEND
                        putExtra(Intent.EXTRA_TEXT, shareMsg)
                        type = "text/plain"
                    }
                    context.startActivity(Intent.createChooser(intent, "Share Payment Voucher"))
                }
            ) {
                Icon(Icons.Filled.Share, contentDescription = null, modifier = Modifier.size(16.dp))
                Spacer(modifier = Modifier.width(6.dp))
                Text(stringResource(R.string.voucher_share_button))
            }
        },
        dismissButton = {
            OutlinedButton(onClick = onDismiss) {
                Text(stringResource(R.string.voucher_done_button))
            }
        }
    )
}

@Composable
fun SupplierAccountCard(
    supplier: SupplierEntity,
    isDarkTheme: Boolean = true,
    onRecordPayment: () -> Unit,
    onRemoveSupplier: () -> Unit = {}
) {
    val negColor = if (isDarkTheme) NegativeRedDark else NegativeRedLight
    Card(
        modifier = Modifier.fillMaxWidth(),
        colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surface),
        border = BorderStroke(1.dp, MaterialTheme.colorScheme.outline.copy(alpha = if (isDarkTheme) 0.3f
else 0.5f)),
        shape = RoundedCornerShape(12.dp)
    ) {
        Row(
            modifier = Modifier
                .fillMaxWidth()
                .padding(14.dp),
            horizontalArrangement = Arrangement.SpaceBetween,
            verticalAlignment = Alignment.CenterVertically
        ) {
            Column(modifier = Modifier.weight(1f)) {
                Text(
                    text = supplier.name,
                    style = MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.Bold),
                    color = MaterialTheme.colorScheme.onSurface
                )
                if (!supplier.phone.isNullOrBlank()) {
                    Row(verticalAlignment = Alignment.CenterVertically) {
                        Icon(Icons.Filled.Phone, contentDescription = null, modifier = Modifier.size(12.dp),
 tint = MaterialTheme.colorScheme.onSurfaceVariant)
                        Spacer(modifier = Modifier.width(4.dp))
                        Text(
                            text = supplier.phone,
                            style = MaterialTheme.typography.bodySmall,
                            color = MaterialTheme.colorScheme.onSurfaceVariant
                        )
                    }
                }
            }

            Column(horizontalAlignment = Alignment.End) {
                Text(
                    text = CostCalculator.formatPaiseToRupees(supplier.currentBalancePaise),
                    style = MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.Bold),
                    color = if (supplier.currentBalancePaise > 0) negColor else
MaterialTheme.colorScheme.onSurface
                )
                Spacer(modifier = Modifier.height(6.dp))
                Row(horizontalArrangement = Arrangement.spacedBy(6.dp), verticalAlignment =
Alignment.CenterVertically) {
                    IconButton(
                        onClick = onRemoveSupplier,
                        modifier = Modifier.size(34.dp)
```

```
                    ) {
                        Icon(
                            Icons.Filled.DeleteOutline,
                            contentDescription = "Remove Supplier",
                            tint = MaterialTheme.colorScheme.error.copy(alpha = 0.85f),
                            modifier = Modifier.size(18.dp)
                        )
                    }
                    Button(
                        onClick = onRecordPayment,
                        shape = RoundedCornerShape(8.dp),
                        contentPadding = ButtonDefaults.TextButtonContentPadding,
                        colors = ButtonDefaults.buttonColors(
                            containerColor = MaterialTheme.colorScheme.primaryContainer,
                            contentColor = MaterialTheme.colorScheme.onPrimaryContainer
                        )
                    ) {
                        Icon(Icons.Filled.Payment, contentDescription = null, modifier =
Modifier.size(14.dp))
                        Spacer(modifier = Modifier.width(4.dp))
                        Text("Pay Bill", fontSize = 12.sp, fontWeight = FontWeight.Bold)
                    }
                }
            }
        }
    }
}

@Composable
fun CustomerAccountCard(
    customer: CustomerEntity,
    isDarkTheme: Boolean = true,
    onRecordRepayment: () -> Unit,
    onNavigateToBillHistory: () -> Unit = {},
    onRemoveCustomer: () -> Unit = {}
) {
    val posColor = if (isDarkTheme) PositiveGreenDark else PositiveGreenLight
    Card(
        modifier = Modifier.fillMaxWidth(),
        colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surface),
        border = BorderStroke(1.dp, MaterialTheme.colorScheme.outline.copy(alpha = if (isDarkTheme) 0.3f
else 0.5f)),
        shape = RoundedCornerShape(12.dp)
    ) {
        Row(
            modifier = Modifier
                .fillMaxWidth()
                .padding(14.dp),
            horizontalArrangement = Arrangement.SpaceBetween,
            verticalAlignment = Alignment.CenterVertically
        ) {
            Column(modifier = Modifier.weight(1f)) {
                Text(
                    text = customer.name,
                    style = MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.Bold),
                    color = MaterialTheme.colorScheme.onSurface
                )
                if (!customer.phone.isNullOrBlank()) {
                    Row(verticalAlignment = Alignment.CenterVertically) {
                        Icon(Icons.Filled.Phone, contentDescription = null, modifier = Modifier.size(12.dp),
 tint = MaterialTheme.colorScheme.onSurfaceVariant)
                        Spacer(modifier = Modifier.width(4.dp))
                        Text(
                            text = customer.phone,
                            style = MaterialTheme.typography.bodySmall,
                            color = MaterialTheme.colorScheme.onSurfaceVariant
                        )
                    }
                }
            }

            Column(horizontalAlignment = Alignment.End) {
                Text(
                    text = CostCalculator.formatPaiseToRupees(customer.currentBalancePaise),
                    style = MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.Bold),
                    color = if (customer.currentBalancePaise > 0) posColor else
MaterialTheme.colorScheme.onSurface
                )
                Spacer(modifier = Modifier.height(6.dp))
                Row(horizontalArrangement = Arrangement.spacedBy(4.dp), verticalAlignment =
Alignment.CenterVertically) {
                    IconButton(
                        onClick = onRemoveCustomer,
                        modifier = Modifier.size(34.dp)
                    ) {
                        Icon(
                            Icons.Filled.DeleteOutline,
                            contentDescription = "Remove Customer",
                            tint = MaterialTheme.colorScheme.error.copy(alpha = 0.85f),
```

```
                            modifier = Modifier.size(18.dp)
                        )
                    }
                    OutlinedButton(
                        onClick = onNavigateToBillHistory,
                        shape = RoundedCornerShape(8.dp),
                        contentPadding = ButtonDefaults.TextButtonContentPadding
                    ) {
                        Icon(Icons.Filled.ReceiptLong, contentDescription = null, modifier =
    Modifier.size(14.dp))
                        Spacer(modifier = Modifier.width(4.dp))
                        Text("Bills", fontSize = 11.5.sp, fontWeight = FontWeight.SemiBold)
                    }
                    Button(
                        onClick = onRecordRepayment,
                        shape = RoundedCornerShape(8.dp),
                        contentPadding = ButtonDefaults.TextButtonContentPadding,
                        colors = ButtonDefaults.buttonColors(
                            containerColor = MaterialTheme.colorScheme.primaryContainer,
                            contentColor = MaterialTheme.colorScheme.onPrimaryContainer
                        )
                    ) {
                        Icon(Icons.Filled.Payment, contentDescription = null, modifier =
    Modifier.size(14.dp))
                        Spacer(modifier = Modifier.width(4.dp))
                        Text("Receive", fontSize = 12.sp, fontWeight = FontWeight.Bold)
                    }
                }
            }
        }
    }
}
```

app\src\main\java\com\alieducational\app\ui\AliViewModel.kt

```
package com.alieducational.app.ui

import android.content.Intent
import android.net.Uri
import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.alieducational.app.data.local.entity.CustomerEntity
import com.alieducational.app.data.local.entity.HeldBillEntity
import com.alieducational.app.data.local.entity.ProductEntity
import com.alieducational.app.data.local.entity.SaleEntity
import com.alieducational.app.data.local.entity.SaleLineEntity
import com.alieducational.app.data.local.entity.ServiceDefinitionEntity
import com.alieducational.app.data.local.entity.StockMovementEntity
import com.alieducational.app.data.local.entity.CashCategoryEntity
import com.alieducational.app.data.local.entity.PigmySchemeEntity
import com.alieducational.app.data.local.entity.PigmyWithdrawalEntity
import com.alieducational.app.data.local.entity.ShopExpenseEntity
import com.alieducational.app.data.local.entity.SupplierEntity
import com.alieducational.app.data.preferences.AppPreferences
import com.alieducational.app.domain.calculation.CostCalculator
import com.alieducational.app.data.repository.AppRepository
import com.alieducational.app.data.repository.BarcodeLookupResult
import com.alieducational.app.data.repository.PurchaseItemParam
import com.alieducational.app.data.repository.SaleItemParam
import com.alieducational.app.data.service.BackupMetadata
import com.alieducational.app.data.service.BackupRestoreService
import com.alieducational.app.data.service.CashTrackerExcelService
import com.alieducational.app.data.service.CashTrackerImportPreview
import com.alieducational.app.data.service.ColumnMapping
import com.alieducational.app.data.service.ImportExportService
import com.alieducational.app.data.service.ImportResult
import com.alieducational.app.data.service.ReceiptService
import com.alieducational.app.data.service.SpreadsheetPreview
import com.alieducational.app.data.service.SyncConnectionState
import com.alieducational.app.data.service.TwoPhoneSyncService
import dagger.hilt.android.lifecycle.HiltViewModel
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.SharingStarted
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.flow.combine
import kotlinx.coroutines.flow.stateIn
import kotlinx.coroutines.launch
import java.util.UUID
import org.json.JSONArray
import org.json.JSONObject
import java.io.File
import javax.inject.Inject

data class BillingDraftItem(
    val lineId: String = UUID.randomUUID().toString(),
    val productId: String,
    val productName: String,
    val urduName: String? = null,
    val unitType: String = "PIECE", // "PIECE", "PACK", "SERVICE"
    val packSize: Int = 1,
    val quantity: Int = 1, // number of chosen units (pieces, packs, sides, copies, sheets)
    val unitPricePaise: Long = 0L, // selling price per chosen unit
    val movingAvgCostPaise: Long = 0L,
    val lineType: String = "PRODUCT", // "PRODUCT" or "SERVICE"
    val serviceId: String? = null,
    val serviceName: String? = null,
    val serviceOption: String? = null, // e.g. "A4 B&W", "A4 Colour"
    val unitLabel: String? = null // e.g. "sides", "copies", "sheets"
) {
    val totalPieces: Int
        get() = if (lineType == "SERVICE") 0 else if (unitType == "PACK") quantity * packSize else quantity

    val lineGrossPaise: Long
        get() = quantity.toLong() * unitPricePaise
}

data class BillingState(
    val items: List<BillingDraftItem> = emptyList(),
    val customer: CustomerEntity? = null,
    val discountType: String = "NONE", // NONE, PERCENT, FIXED
    val discountValuePaise: Long = 0L,
    val discountPercent: Double = 0.0,
    val paymentMethod: String = "CASH", // CASH, UPI, CREDIT, OTHER
    val cashReceivedStr: String = "",
    val receiptAction: String = "NONE", // NONE, PREVIEW, PRINT, SHARE
    val lastAddedItem: BillingDraftItem? = null,
```

```
    val lastScannedBarcode: String? = null,
    val unknownScannedBarcode: String? = null,
    val isTorchOn: Boolean = false,
    val lastSavedSale: SaleEntity? = null
) {
    val grossTotalPaise: Long
        get() = items.sumOf { it.lineGrossPaise }

    val actualDiscountPaise: Long
        get() = when (discountType) {
            "PERCENT" -> ((grossTotalPaise * discountPercent) / 100.0).toLong().coerceAtMost(grossTotalPaise)
            "FIXED" -> discountValuePaise.coerceAtMost(grossTotalPaise)
            else -> 0L
        }

    val netTotalPaise: Long
        get() = (grossTotalPaise - actualDiscountPaise).coerceAtLeast(0L)

    val totalItemCount: Int
        get() = items.sumOf { it.quantity }

    val cashReceivedPaise: Long
        get() = CostCalculator.parseRupeesToPaise(cashReceivedStr) ?: 0L

    val changeToGivePaise: Long
        get() = if (paymentMethod == "CASH" && cashReceivedPaise > netTotalPaise) cashReceivedPaise - netTotalPaise else 0L

    val remainingCreditPaise: Long
        get() = if (paymentMethod == "CREDIT" || paymentMethod == "OTHER") {
            (netTotalPaise - cashReceivedPaise).coerceAtLeast(0L)
        } else 0L
}

@HiltViewModel
class AliViewModel @Inject constructor(
    private val repository: AppRepository,
    private val preferences: AppPreferences,
    private val importExportService: ImportExportService,
    private val backupService: BackupRestoreService,
    val receiptService: ReceiptService,
    val cashTrackerExcelService: CashTrackerExcelService,
    val twoPhoneSyncService: TwoPhoneSyncService
) : ViewModel() {

    val syncConnectionState: StateFlow<SyncConnectionState> = twoPhoneSyncService.connectionState

    fun startSyncServer(port: Int = 8080) {
        twoPhoneSyncService.startOwnerServer(port)
    }

    fun stopSyncServer() {
        twoPhoneSyncService.stopOwnerServer()
    }

    fun syncWithMainPhone(host: String, port: Int = 8080, onResult: (Boolean) -> Unit = {}) {
        viewModelScope.launch {
            val success = twoPhoneSyncService.syncWithMainPhone(host, port)
            onResult(success)
        }
    }

    // ── Preferences ──────────────────────────────────────────
    val isDarkTheme: StateFlow<Boolean> = preferences.themeMode
        .combine(preferences.isFirstLaunch) { mode, _ -> mode == "dark" }
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), true)

    val appLanguage: StateFlow<String> = preferences.appLanguage
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), "en")

    val isLockEnabled: StateFlow<Boolean> = preferences.isLockEnabled
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), false)

    val lockTimeoutSeconds: StateFlow<Int> = preferences.lockTimeoutSeconds
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), 120)

    val lastBackupTime: StateFlow<Long> = preferences.lastBackupTime
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), 0L)

    val isFirstLaunch: StateFlow<Boolean> = preferences.isFirstLaunch
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), true)

    val shopName: StateFlow<String> = preferences.shopName
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), "Ali Educational")

    val pigmyUsualAmountPaise: StateFlow<Long> = preferences.pigmyUsualAmountPaise
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), 100000L)
```

```
169
170     val deviceRole: StateFlow<String> = preferences.deviceRole
171         .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), "OWNER")
172
173     val deviceId: StateFlow<String> = preferences.deviceId
174         .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), "device-main")
175
176     val pairedOwnerHost: StateFlow<String> = preferences.pairedOwnerHost
177         .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), "")
178
179         val allPigmySchemes = repository.allPigmySchemes
180         .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), emptyList())
181
182     val activePigmySchemes = repository.activePigmySchemes
183         .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), emptyList())
184
185     val lastExternalSaveTime = preferences.lastExternalSaveTime
186         .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), 0L)
187
188     val lastExternalSaveDestination = preferences.lastExternalSaveDestination
189         .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), "")
190

191     val allPigmyDeposits = repository.allPigmyDeposits
192         .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), emptyList())
193
194     val activePigmyDeposits = repository.activePigmyDeposits
195         .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), emptyList())
196
197     // ── Home Screen Widget Action Trigger ──────────────────────────
198     private val _widgetActionTrigger = kotlinx.coroutines.flow.MutableStateFlow<String?>(null)
199     val widgetActionTrigger: StateFlow<String?> = _widgetActionTrigger.asStateFlow()
200
201     fun triggerWidgetAction(action: String) {
202         _widgetActionTrigger.value = action
203     }
204
205     fun consumeWidgetAction() {
206         _widgetActionTrigger.value = null
207     }
208
209     val totalPigmyDepositsPaise = repository.totalPigmyDepositsPaise
210         .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), 0L)
211
212     val allPigmyWithdrawals = repository.allPigmyWithdrawals
213         .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), emptyList())
214
215     val activePigmyWithdrawals = repository.activePigmyWithdrawals
216         .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), emptyList())
217
218     val totalPigmyWithdrawalsPaise = repository.totalPigmyWithdrawalsPaise
219         .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), 0L)
220
221     val allCashCategories = repository.allCashCategories
222         .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), emptyList())
223
224     val activeCashCategories = repository.activeCashCategories
225         .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), emptyList())
226
227     val allGiveaways = repository.allGiveaways
228         .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), emptyList())
229
230     val activeGiveaways = repository.activeGiveaways
231         .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), emptyList())
232
233     val activeServices = repository.activeServices
234         .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), emptyList())
235
236     val pendingSyncCount = repository.pendingSyncCount
237         .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), 0)
238

239     init {
240         viewModelScope.launch {
241             repository.ensureDefaultPigmySchemes()
242             repository.ensureDefaultCashCategories()
243         }
244         viewModelScope.launch {
245             preferences.setAppLanguage("en")
246             repository.ensurePredefinedServices()
247         }
248     }
249
250     // ── Dashboard Metrics ──────────────────────────────────────────
251     val totalStockValuePaise = repository.totalStockValuePaise
252         .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), 0L)
253
254     val lowStockCount = repository.lowStockCount
255         .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), 0)
256
```

```
    val totalProductCount = repository.totalProductCount
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), 0)

    val totalWeOweSuppliersPaise = repository.totalWeOweSuppliersPaise
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), 0L)
    val distinctCreditorsCount = repository.distinctCreditorsCount
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), 0)

    val totalCustomersOweUsPaise = repository.totalCustomersOweUsPaise
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), 0L)
    val distinctDebtorsCount = repository.distinctDebtorsCount
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), 0)

    val refundableExpenses = repository.refundableExpenses
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), emptyList())

    val recentMovements = repository.recentStockMovements
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), emptyList())

    // ── Inventory ──────────────────────────────────────────────
    val allProducts = repository.allProducts
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), emptyList())

    val lowStockProducts = repository.lowStockProducts
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), emptyList())

    private val _searchQuery = MutableStateFlow("")
    val searchQuery: StateFlow<String> = _searchQuery.asStateFlow()

    fun updateSearchQuery(query: String) {
        _searchQuery.value = query
    }

    val allSaleLines = repository.allSaleLines
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), emptyList())

    fun addOrUpdateProductWithStock(
        existingId: String?,
        name: String,
        barcode: String?,
        category: String,
        brand: String?,
        packSize: Int,
        boxesBrought: Int,
        loosePiecesBrought: Int,
        costPerPiecePaise: Long,
        sellingPricePiecePaise: Long,
        wholesalePricePiecePaise: Long,
        lowStockThreshold: Int,
        supplierName: String?,
        billNo: String? = null,
        transportChargesPaise: Long = 0L,
        isPaidNow: Boolean = false,
        paymentMethod: String = "CASH",
        onDone: (ProductEntity) -> Unit = {}
    ) {
        viewModelScope.launch {
            val result = repository.addOrUpdateProductWithStock(
                existingId = existingId,
                name = name,
                barcode = barcode,
                category = category,
                brand = brand,
                packSize = packSize,
                boxesBrought = boxesBrought,
                loosePiecesBrought = loosePiecesBrought,
                costPerPiecePaise = costPerPiecePaise,
                sellingPricePiecePaise = sellingPricePiecePaise,
                wholesalePricePiecePaise = wholesalePricePiecePaise,
                lowStockThreshold = lowStockThreshold,
                supplierName = supplierName,
                billNo = billNo,
                transportChargesPaise = transportChargesPaise,
                isPaidNow = isPaidNow,
                paymentMethod = paymentMethod
            )
            onDone(result)
        }
    }

    fun getPurchasesForSupplier(supplierId: String) = repository.getPurchasesForSupplier(supplierId)

    fun addProduct(product: ProductEntity, onDone: () -> Unit = {}) {
        viewModelScope.launch {
            repository.addProduct(product)
            onDone()
        }
    }
```

```
345
346     fun updateProduct(product: ProductEntity, onDone: () -> Unit = {}) {
347         viewModelScope.launch {
348             repository.updateProduct(product)
349             onDone()
350         }
351     }
352
353     fun recordStockAdjustment(
354         productId: String,
355         type: String,
356         quantityChange: Int,
357         countedTotal: Int? = null,
358         reason: String?,
359         onDone: () -> Unit = {}
360     ) {
361         viewModelScope.launch {
362             repository.recordStockAdjustment(productId, type, quantityChange, countedTotal, reason)
363             onDone()
364         }
365     }
366
367     // ── Suppliers & Goods Received ──────────────────────────────
368     val allSuppliers = repository.allSuppliers
369         .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), emptyList())
370
371     fun addSupplier(supplier: SupplierEntity, onDone: () -> Unit = {}) {
372         viewModelScope.launch {
373             repository.addSupplier(supplier)
374             onDone()
375         }
376     }
377
378     fun receiveGoods(
379         supplierId: String,
380         invoiceNumber: String?,
381         items: List<PurchaseItemParam>,
382         totalDeliveryPaise: Long,
383         deliveryPaidTo: String,
384         amountPaidPaise: Long,
385         paymentMethod: String,
386         notes: String?,
387         onDone: () -> Unit = {}
388     ) {
389         viewModelScope.launch {
390             repository.receiveGoods(
391                 supplierId = supplierId,
392                 invoiceNumber = invoiceNumber,
393                 items = items,
394                 totalDeliveryPaise = totalDeliveryPaise,
395                 deliveryPaidTo = deliveryPaidTo,
396                 amountPaidPaise = amountPaidPaise,
397                 paymentMethod = paymentMethod,
398                 notes = notes
399             )
400             onDone()
401         }
402     }
403
404     // ── Customers & Credit ──────────────────────────────────────
405     val allCustomers = repository.allCustomers
406         .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), emptyList())
407
408     fun addCustomer(customer: CustomerEntity, onDone: () -> Unit = {}) {
409         viewModelScope.launch {
410             repository.addCustomer(customer)
411             onDone()
412         }
413     }
414
415     fun recordAccountPayment(
416         accountType: String,
417         accountId: String,
418         amountPaise: Long,
419         paymentMethod: String,
420         notes: String?,
421         receiptRef: String?,
422         onDone: () -> Unit = {}
423     ) {
424         viewModelScope.launch {
425             repository.recordAccountPayment(
426                 accountType = accountType,
427                 accountId = accountId,
428                 amountPaise = amountPaise,
429                 paymentMethod = paymentMethod,
430                 notes = notes,
431                 receiptRef = receiptRef
432             )
```

```
            onDone()
        }
    }

    // ── Sales ──────────────────────────────────────────────────
    val allSales = repository.allSales
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), emptyList())

    val unresolvedStockSales = repository.unresolvedStockSales
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), emptyList())

    val allExpenses = repository.allExpenses
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), emptyList())

    fun recordSale(
        customerId: String?,
        customerName: String?,
        items: List<SaleItemParam>,
        discountType: String,
        discountInput: Double,
        amountReceivedPaise: Long,
        paymentMethod: String,
        notes: String?,
        onSuccess: (SaleEntity) -> Unit = {}
    ) {
        viewModelScope.launch {
            val sale = repository.recordSale(
                customerId = customerId,
                customerName = customerName,
                items = items,
                discountType = discountType,
                discountInput = discountInput,
                amountReceivedPaise = amountReceivedPaise,
                paymentMethod = paymentMethod,
                notes = notes
            )
            onSuccess(sale)
        }
    }

    // ── Counter Fast Billing & Barcode Scanning ──────────────────

    val favoriteProducts = repository.favoriteProducts
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), emptyList())

    val allCategories = repository.allCategories
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), emptyList())

    val allHeldBills = repository.allHeldBills
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), emptyList())

    private val _billingState = MutableStateFlow(BillingState())
    val billingState: StateFlow<BillingState> = _billingState.asStateFlow()

    private val _selectedBillingCategory = MutableStateFlow<String?>(null)
    val selectedBillingCategory: StateFlow<String?> = _selectedBillingCategory.asStateFlow()

    fun selectBillingCategory(category: String?) {
        _selectedBillingCategory.value = category
    }

    fun toggleFavorite(productId: String, isFav: Boolean) {
        viewModelScope.launch {
            repository.setFavorite(productId, isFav)
        }
    }

    fun addItemToBill(product: ProductEntity, unitType: String = "PIECE", quantity: Int = 1) {
        val currentItems = _billingState.value.items.toMutableList()
        val price = if (unitType == "PACK") {
            if (product.sellingPricePackPaise > 0) product.sellingPricePackPaise else
product.sellingPricePiecePaise * product.packSize
        } else {
            product.sellingPricePiecePaise
        }

        val existingIndex = currentItems.indexOfFirst { it.productId == product.id && it.unitType ==
unitType }
        val addedItem: BillingDraftItem
        if (existingIndex >= 0) {
            val existing = currentItems[existingIndex]
            addedItem = existing.copy(quantity = existing.quantity + quantity)
            currentItems[existingIndex] = addedItem
        } else {
            addedItem = BillingDraftItem(
                productId = product.id,
                productName = product.name,
                urduName = product.urduName,
                unitType = unitType,
```

```
                packSize = product.packSize,
                quantity = quantity,
                unitPricePaise = price,
                movingAvgCostPaise = product.movingAvgCostPaise
            )
            currentItems.add(addedItem)
        }

        _billingState.value = _billingState.value.copy(
            items = currentItems,
            lastAddedItem = addedItem,
            lastScannedBarcode = null
        )
    }

    fun handleScannedBarcode(barcode: String, onUnknown: (String) -> Unit = {}) {
        val code = barcode.trim()
        if (code.isEmpty()) return

        viewModelScope.launch {
            val result = repository.lookupBarcode(code)
            if (result != null) {
                addItemToBill(result.product, result.unitType, 1)
                _billingState.value = _billingState.value.copy(lastScannedBarcode = code)
            } else {
                _billingState.value = _billingState.value.copy(unknownScannedBarcode = code)
                onUnknown(code)
            }
        }
    }

    fun clearUnknownBarcode() {
        _billingState.value = _billingState.value.copy(unknownScannedBarcode = null)
    }

    fun linkAndAddUnknownBarcode(
        barcode: String,
        productId: String,
        unitType: String = "PIECE",
        packQty: Int = 1,
        onDone: () -> Unit = {}
    ) {
        viewModelScope.launch {
            repository.linkBarcode(barcode, productId, unitType, packQty)
            val product = repository.getProductById(productId)
            if (product != null) {
                addItemToBill(product, unitType, 1)
            }
            clearUnknownBarcode()
            onDone()
        }
    }

    fun undoLastAddition() {
        val lastItem = _billingState.value.lastAddedItem ?: return
        val currentItems = _billingState.value.items.toMutableList()
        val index = currentItems.indexOfFirst { it.productId == lastItem.productId && it.unitType ==
lastItem.unitType }
        if (index >= 0) {
            val existing = currentItems[index]
            if (existing.quantity > 1) {
                currentItems[index] = existing.copy(quantity = existing.quantity - 1)
            } else {
                currentItems.removeAt(index)
            }
            _billingState.value = _billingState.value.copy(
                items = currentItems,
                lastAddedItem = null
            )
        }
    }

    fun addAgainLastItem() {
        val lastItem = _billingState.value.lastAddedItem ?: return
        val currentItems = _billingState.value.items.toMutableList()
        val index = currentItems.indexOfFirst { it.productId == lastItem.productId && it.unitType ==
lastItem.unitType }
        if (index >= 0) {
            val existing = currentItems[index]
            val updated = existing.copy(quantity = existing.quantity + 1)
            currentItems[index] = updated
            _billingState.value = _billingState.value.copy(
                items = currentItems,
                lastAddedItem = updated
            )
        }
    }

```

```
606     fun updateDraftItemQuantity(productId: String, unitType: String, delta: Int) {
607         val currentItems = _billingState.value.items.toMutableList()
608         val index = currentItems.indexOfFirst { it.productId == productId && it.unitType == unitType }
609         if (index >= 0) {
610             val existing = currentItems[index]
611             val newQty = existing.quantity + delta
612             if (newQty > 0) {
613                 currentItems[index] = existing.copy(quantity = newQty)
614             } else {
615                 currentItems.removeAt(index)
616             }
617             _billingState.value = _billingState.value.copy(items = currentItems)
618         }
619     }
620
621     fun updateDraftItemUnit(productId: String, currentUnit: String, newUnit: String) {
622         val currentItems = _billingState.value.items.toMutableList()
623         val index = currentItems.indexOfFirst { it.productId == productId && it.unitType == currentUnit }
624         if (index >= 0) {
625             viewModelScope.launch {
626                 val product = repository.getProductById(productId)
627                 if (product != null) {
628                     val newPrice = if (newUnit == "PACK") {
629                         if (product.sellingPricePackPaise > 0) product.sellingPricePackPaise else
    product.sellingPricePiecePaise * product.packSize
630                     } else {
631                         product.sellingPricePiecePaise
632                     }
633                     val existing = currentItems[index]
634                     currentItems[index] = existing.copy(
635                         unitType = newUnit,
636                         unitPricePaise = newPrice
637                     )
638                     _billingState.value = _billingState.value.copy(items = currentItems)
639                 }
640             }
641         }
642     }
643
644     fun removeDraftItem(productId: String, unitType: String) {
645         val currentItems = _billingState.value.items.filterNot { it.productId == productId && it.unitType ==
    unitType }
646         _billingState.value = _billingState.value.copy(items = currentItems)
647     }
648
649     fun setWholeBillDiscountPercent(percent: Double) {
650         _billingState.value = _billingState.value.copy(
651             discountType = "PERCENT",
652             discountPercent = percent,
653             discountValuePaise = 0L
654         )
655     }
656
657     fun setWholeBillDiscountFixed(amountPaise: Long) {
658         _billingState.value = _billingState.value.copy(
659             discountType = "FIXED",
660             discountValuePaise = amountPaise,
661             discountPercent = 0.0
662         )
663     }
664
665     fun clearBillDiscount() {
666         _billingState.value = _billingState.value.copy(
667             discountType = "NONE",
668             discountValuePaise = 0L,
669             discountPercent = 0.0
670         )
671     }
672
673     fun setBillingCustomer(customer: CustomerEntity?) {
674         _billingState.value = _billingState.value.copy(customer = customer)
675     }
676
677     fun setBillingPaymentMethod(method: String) {
678         _billingState.value = _billingState.value.copy(paymentMethod = method)
679     }
680
681     fun setCashReceived(text: String) {
682         _billingState.value = _billingState.value.copy(cashReceivedStr = text)
683     }
684
685     fun setReceiptAction(action: String) {
686         _billingState.value = _billingState.value.copy(receiptAction = action)
687     }
688
689     fun toggleTorch() {
690         _billingState.value = _billingState.value.copy(isTorchOn = !_billingState.value.isTorchOn)
691     }
```

```
692
693     fun resetBilling() {
694         _billingState.value = BillingState()
695     }
696
697     fun holdCurrentBill(note: String? = null, onDone: () -> Unit = {}) {
698         val state = _billingState.value
699         if (state.items.isEmpty()) return
700
701         val json = itemsToJson(state.items)
702         val heldBill = HeldBillEntity(
703             note = note,
704             customerId = state.customer?.id,
705             customerName = state.customer?.name,
706             discountType = state.discountType,
707             discountValuePaise = state.discountValuePaise,
708             discountPercent = state.discountPercent,
709             draftJson = json,
710             itemCount = state.totalItemCount,
711             totalPaise = state.netTotalPaise
712         )
713
714         viewModelScope.launch {
715             repository.holdBill(heldBill)
716             resetBilling()
717             onDone()
718         }
719     }
720
721     fun resumeHeldBill(bill: HeldBillEntity, onDone: () -> Unit = {}) {
722         viewModelScope.launch {
723             val restoredItems = jsonToItems(bill.draftJson)
724             val customer = if (bill.customerId != null) repository.getCustomerById(bill.customerId) else
    null
725
726             _billingState.value = BillingState(
727                 items = restoredItems,
728                 customer = customer,
729                 discountType = bill.discountType,
730                 discountValuePaise = bill.discountValuePaise,
731                 discountPercent = bill.discountPercent
732             )
733             repository.deleteHeldBill(bill.id)
734             onDone()
735         }
736     }
737
738     fun discardHeldBill(billId: String) {
739         viewModelScope.launch {
740             repository.deleteHeldBill(billId)
741         }
742     }
743
744     suspend fun getSaleLines(saleId: String): List<SaleLineEntity> =
745         repository.getLinesForSale(saleId)
746
747     private var isPostingBill = false
748
749     fun postCurrentBill(onSuccess: (SaleEntity) -> Unit, onError: (String) -> Unit) {
750         if (isPostingBill) return
751         val state = _billingState.value
752         if (state.items.isEmpty()) {
753             onError("Cannot save an empty bill")
754             return
755         }
756
757         if ((state.paymentMethod == "CREDIT" || state.paymentMethod == "OTHER") &&
    state.remainingCreditPaise > 0 && state.customer == null) {
758             onError("Customer account is required when credit or unpaid balance remains")
759             return
760         }
761
762         isPostingBill = true
763         viewModelScope.launch {
764             try {
765                 val saleItems = state.items.map { item ->
766                     SaleItemParam(
767                         productId = if (item.lineType == "SERVICE") null else item.productId,
768                         productName = item.productName,
769                         quantityBase = if (item.lineType == "SERVICE") item.quantity else item.totalPieces,
770                         isPack = item.unitType == "PACK",
771                         unitSellingPricePaise = item.unitPricePaise,
772                         lineId = item.lineId,
773                         lineType = item.lineType,
774                         serviceId = item.serviceId,
775                         serviceName = item.serviceName,
776                         serviceOption = item.serviceOption,
777                         unitLabel = item.unitLabel
```

```
                    )
                }

                val amountPaid = if (state.paymentMethod == "CASH") {
                    if (state.cashReceivedPaise > 0L) {
                        state.cashReceivedPaise.coerceAtMost(state.netTotalPaise)
                    } else {
                        state.netTotalPaise
                    }
                } else if (state.paymentMethod == "UPI" || state.paymentMethod == "CARD") {
                    state.netTotalPaise
                } else {
                    state.cashReceivedPaise.coerceAtMost(state.netTotalPaise)
                }

                val discountInput = if (state.discountType == "PERCENT") {
                    state.discountPercent
                } else {
                    state.discountValuePaise / 100.0
                }

                val devId = deviceId.value
                val sale = repository.recordSale(
                    customerId = state.customer?.id,
                    customerName = state.customer?.name,
                    items = saleItems,
                    discountType = state.discountType,
                    discountInput = discountInput,
                    amountReceivedPaise = amountPaid,
                    paymentMethod = state.paymentMethod,
                    notes = null,
                    originDeviceId = devId
                )

                _billingState.value = _billingState.value.copy(lastSavedSale = sale)
                onSuccess(sale)
            } catch (e: Exception) {
                onError(e.message ?: "Failed to post bill")
            } finally {
                isPostingBill = false
            }
        }
    }

    private fun itemsToJson(items: List<BillingDraftItem>): String {
        val array = JSONArray()
        for (item in items) {
            val obj = JSONObject()
            obj.put("lineId", item.lineId)
            obj.put("productId", item.productId)
            obj.put("productName", item.productName)
            obj.put("urduName", item.urduName ?: "")
            obj.put("unitType", item.unitType)
            obj.put("packSize", item.packSize)
            obj.put("quantity", item.quantity)
            obj.put("unitPricePaise", item.unitPricePaise)
            obj.put("movingAvgCostPaise", item.movingAvgCostPaise)
            obj.put("lineType", item.lineType)
            obj.put("serviceId", item.serviceId ?: "")
            obj.put("serviceName", item.serviceName ?: "")
            obj.put("serviceOption", item.serviceOption ?: "")
            obj.put("unitLabel", item.unitLabel ?: "")
            array.put(obj)
        }
        return array.toString()
    }

    private fun jsonToItems(json: String): List<BillingDraftItem> {
        val list = mutableListOf<BillingDraftItem>()
        try {
            val array = JSONArray(json)
            for (i in 0 until array.length()) {
                val obj = array.getJSONObject(i)
                list.add(
                    BillingDraftItem(
                        lineId = obj.optString("lineId").ifBlank { UUID.randomUUID().toString() },
                        productId = obj.getString("productId"),
                        productName = obj.getString("productName"),
                        urduName = obj.optString("urduName").ifBlank { null },
                        unitType = obj.optString("unitType", "PIECE"),
                        packSize = obj.optInt("packSize", 1),
                        quantity = obj.optInt("quantity", 1),
                        unitPricePaise = obj.optLong("unitPricePaise", 0L),
                        movingAvgCostPaise = obj.optLong("movingAvgCostPaise", 0L),
                        lineType = obj.optString("lineType", "PRODUCT"),
                        serviceId = obj.optString("serviceId").ifBlank { null },
                        serviceName = obj.optString("serviceName").ifBlank { null },
                        serviceOption = obj.optString("serviceOption").ifBlank { null },
```

```
                        unitLabel = obj.optString("unitLabel").ifBlank { null }
                    )
                )
            }
        } catch (_: Exception) {}
        return list
    }

    fun recordDailyMonetaryTotal(
        dateTimestamp: Long,
        amountPaise: Long,
        isFullDayTotal: Boolean,
        notes: String?,
        onDone: () -> Unit = {}
    ) {
        viewModelScope.launch {
            repository.recordDailyMonetaryTotal(dateTimestamp, amountPaise, isFullDayTotal, notes)
            onDone()
        }
    }

    fun recordShopExpense(
        category: String,
        amountPaise: Long,
        isIncome: Boolean,
        notes: String?,
        timestamp: Long = System.currentTimeMillis(),
        categoryId: String? = null,
        attachmentUri: String? = null,
        classification: String = "SHARED",
        paymentMethod: String = "CASH",
        isExpenseRefund: Boolean = false,
        linkedExpenseId: String? = null,
        onDone: () -> Unit = {}
    ) {
        viewModelScope.launch {
            repository.recordShopExpense(
                category = category,
                amountPaise = amountPaise,
                isIncome = isIncome,
                notes = notes,
                timestamp = timestamp,
                categoryId = categoryId,
                attachmentUri = attachmentUri,
                classification = classification,
                paymentMethod = paymentMethod,
                isExpenseRefund = isExpenseRefund,
                linkedExpenseId = linkedExpenseId
            )
            onDone()
        }
    }

    fun recordExpenseRefund(
        originalExpenseId: String,
        amountPaise: Long,
        paymentMethod: String = "CASH",
        notes: String? = null,
        timestamp: Long = System.currentTimeMillis(),
        attachmentUri: String? = null,
        onDone: () -> Unit = {},
        onError: (String) -> Unit = {}
    ) {
        viewModelScope.launch {
            try {
                repository.recordExpenseRefund(
                    originalExpenseId = originalExpenseId,
                    amountPaise = amountPaise,
                    paymentMethod = paymentMethod,
                    notes = notes,
                    timestamp = timestamp,
                    attachmentUri = attachmentUri
                )
                onDone()
            } catch (e: Exception) {
                onError(e.localizedMessage ?: "Failed to record expense refund")
            }
        }
    }

    fun updateExpenseAttachment(expenseId: String, attachmentUri: String?, onDone: () -> Unit = {}) {
        viewModelScope.launch {
            repository.updateExpenseAttachment(expenseId, attachmentUri)
            onDone()
        }
    }

    fun voidSale(
```

```
        saleId: String,
        stockCondition: String, // SELLABLE, DAMAGED, NONE
        voidReason: String,
        onDone: () -> Unit = {},
        onError: (String) -> Unit = {}
    ) {
        viewModelScope.launch {
            try {
                repository.voidSale(saleId, stockCondition, voidReason)
                onDone()
            } catch (e: Exception) {
                onError(e.message ?: "Failed to void sale")
            }
        }
    }

    fun voidExpense(expenseId: String, reason: String, onDone: () -> Unit = {}) {
        viewModelScope.launch {
            repository.voidExpense(expenseId, reason)
            onDone()
        }
    }

    fun parseCashTrackerExcel(
        uri: Uri,
        fileName: String,
        onResult: (CashTrackerImportPreview) -> Unit
    ) {
        viewModelScope.launch {
            val preview = cashTrackerExcelService.parseExcelFile(uri, fileName)
            onResult(preview)
        }
    }

    fun confirmCashTrackerImport(
        items: List<ShopExpenseEntity>,
        onDone: () -> Unit
    ) {
        viewModelScope.launch {
            repository.recordShopExpensesBatch(items)
            onDone()
        }
    }

    fun exportCashTrackerExcel(
        items: List<ShopExpenseEntity>,
        periodTitle: String,
        fromDate: Long? = null,
        toDate: Long? = null,
        activeFiltersSummary: String = "All Categories, Active Transactions",
        onIntentReady: (Intent) -> Unit
    ) {
        viewModelScope.launch {
            val intent = cashTrackerExcelService.exportCashTrackerExcel(
                items = items,
                periodTitle = periodTitle,
                fromDate = fromDate,
                toDate = toDate,
                activeFiltersSummary = activeFiltersSummary
            )
            onIntentReady(intent)
        }
    }

    fun archiveCustomer(customerId: String, onDone: () -> Unit = {}) {
        viewModelScope.launch {
            repository.archiveCustomer(customerId)
            onDone()
        }
    }

    fun deleteCustomer(customerId: String, onDone: () -> Unit = {}) {
        viewModelScope.launch {
            repository.deleteCustomer(customerId)
            onDone()
        }
    }

    fun archiveSupplier(supplierId: String, onDone: () -> Unit = {}) {
        viewModelScope.launch {
            repository.archiveSupplier(supplierId)
            onDone()
        }
    }

    fun deleteSupplier(supplierId: String, onDone: () -> Unit = {}) {
        viewModelScope.launch {
            repository.deleteSupplier(supplierId)
            onDone()
```

```
        }
    }

    // ── Backup & Restore ──────────────────────────────────────────

    val backupFolderPath: String
        get() = backupService.backupFolderPath

    fun getBackupShareIntent(filePath: String): Intent {
        return backupService.getShareIntent(filePath)
    }

    private val _backupStatus = MutableStateFlow<String?>(null)
    val backupStatus: StateFlow<String?> = _backupStatus.asStateFlow()

    private val _availableBackups = MutableStateFlow<List<BackupMetadata>>(emptyList())
    val availableBackups: StateFlow<List<BackupMetadata>> = _availableBackups.asStateFlow()

    private val _isCreatingBackup = MutableStateFlow(false)
    val isCreatingBackup: StateFlow<Boolean> = _isCreatingBackup.asStateFlow()

    fun loadBackups() {
        viewModelScope.launch {
            _availableBackups.value = backupService.getAvailableBackups()
        }
    }

    fun createBackup(onDone: () -> Unit = {}) {
        if (_isCreatingBackup.value) return // Prevent overlapping jobs
        _isCreatingBackup.value = true
        _backupStatus.value = "Creating backup..."
        viewModelScope.launch {
            try {
                val meta = backupService.createBackup()
                _backupStatus.value = "Backup created: ${meta.fileName}"
                loadBackups()
                onDone()
            } catch (e: Exception) {
                _backupStatus.value = "Backup failed: ${e.localizedMessage ?: "Unknown error"}"
            } finally {
                _isCreatingBackup.value = false
            }
        }
    }

    fun restoreBackup(file: File, onDone: () -> Unit = {}) {
        viewModelScope.launch {
            val success = backupService.restoreBackup(file)
            _backupStatus.value = if (success) "Restored successfully" else "Restore failed"
            onDone()
        }
    }

    fun restoreFromUri(uri: Uri, onDone: () -> Unit = {}) {
        viewModelScope.launch {
            val success = backupService.restoreFromUri(uri)
            _backupStatus.value = if (success) "Restored successfully" else "Restore failed"
            onDone()
        }
    }

    fun exportBackupToUri(backupFile: File, targetUri: Uri, onDone: (Boolean) -> Unit = {}) {
        viewModelScope.launch {
            val success = backupService.exportBackupToUri(backupFile, targetUri)
            _backupStatus.value = if (success) "Saved to Google Drive / Files successfully" else "Failed to
save backup to destination"
            onDone(success)
        }
    }

    // ── Import / Export ──────────────────────────────────────────
    private val _previewData = MutableStateFlow<SpreadsheetPreview?>(null)
    val previewData: StateFlow<SpreadsheetPreview?> = _previewData.asStateFlow()

    private val _importResult = MutableStateFlow<ImportResult?>(null)
    val importResult: StateFlow<ImportResult?> = _importResult.asStateFlow()

    private val _isImporting = MutableStateFlow(false)
    val isImporting: StateFlow<Boolean> = _isImporting.asStateFlow()

    fun previewSpreadsheet(uri: Uri, fileName: String, sheetIndex: Int = 0) {
        viewModelScope.launch {
            _previewData.value = importExportService.previewFile(uri, fileName, sheetIndex)
        }
    }

    fun executeImport(
        uri: Uri,
        fileName: String,
```

```
        sheetIndex: Int = 0,
        mapping: ColumnMapping,
        updateExisting: Boolean = false,
        onDone: () -> Unit = {}
    ) {
        if (_isImporting.value) return
        viewModelScope.launch {
            _isImporting.value = true
            try {
                val result = importExportService.importProducts(uri, fileName, sheetIndex, mapping,
    updateExisting)
                _importResult.value = result

                onDone()
            } finally {
                _isImporting.value = false
            }
        }
    }

    // ── Settings ──────────────────────────────────────────────
    fun setTheme(mode: String) {
        viewModelScope.launch { preferences.setThemeMode(mode) }
    }

    fun setLanguage(lang: String) {
        viewModelScope.launch { preferences.setAppLanguage(lang) }
    }

    fun setLockEnabled(enabled: Boolean) {
        viewModelScope.launch { preferences.setLockEnabled(enabled) }
    }

    fun setLockTimeout(seconds: Int) {
        viewModelScope.launch { preferences.setLockTimeoutSeconds(seconds) }
    }

    fun completeSetup(shopName: String, language: String, lockEnabled: Boolean) {
        viewModelScope.launch {
            preferences.setShopName(shopName)
            preferences.setAppLanguage(language)
            preferences.setLockEnabled(lockEnabled)
            preferences.setFirstLaunch(false)
        }
    }

    // ── Pigmy Savings Actions ─────────────────────────────────
    fun setPigmyUsualAmount(amountPaise: Long) {
        viewModelScope.launch {
            preferences.setPigmyUsualAmountPaise(amountPaise)
        }
    }

    fun createPigmyScheme(name: String, usualDailyAmountPaise: Long, onDone: (String) -> Unit = {}) {
        viewModelScope.launch {
            val id = repository.createPigmyScheme(name, usualDailyAmountPaise)
            onDone(id)
        }
    }

    fun updatePigmyScheme(id: String, name: String, usualDailyAmountPaise: Long, onDone: () -> Unit = {}) {
        viewModelScope.launch {
            repository.updatePigmyScheme(id, name, usualDailyAmountPaise)
            onDone()
        }
    }

    fun setPigmySchemeArchived(id: String, isArchived: Boolean, onDone: () -> Unit = {}) {
        viewModelScope.launch {
            repository.setPigmySchemeArchived(id, isArchived)
            onDone()
        }
    }

    fun recordPigmyDeposit(
        schemeId: String = "DEFAULT_PIGMY",
        amountPaise: Long,
        paymentSource: String = "CASH",
        notes: String? = null,
        paidTimestamp: Long = System.currentTimeMillis(),
        attachmentUri: String? = null,
        onDone: (String) -> Unit = {}
    ) {
        viewModelScope.launch {
            val id = repository.recordPigmyDeposit(schemeId, amountPaise, paymentSource, notes,
    paidTimestamp, attachmentUri)
            onDone(id)
        }
    }
```

```
1216
1217      fun updatePigmyDeposit(
1218          depositId: String,
1219          schemeId: String,
1220          amountPaise: Long,
1221          paymentSource: String,
1222          notes: String?,
1223          paidTimestamp: Long,
1224          onDone: () -> Unit = {}
1225      ) {
1226          viewModelScope.launch {
1227              repository.updatePigmyDeposit(depositId, schemeId, amountPaise, paymentSource, notes,
      paidTimestamp)
1228              onDone()
1229          }
1230      }
1231
1232      fun voidPigmyDeposit(depositId: String, reason: String, onDone: () -> Unit = {}) {
1233          viewModelScope.launch {
1234              repository.voidPigmyDeposit(depositId, reason)
1235              onDone()
1236          }
1237      }
1238
1239      fun recordPigmyWithdrawal(
1240          schemeId: String,
1241          amountPaise: Long,
1242          destination: String = "CASH",
1243          notes: String? = null,
1244          withdrawalTimestamp: Long = System.currentTimeMillis(),
1245          attachmentUri: String? = null,
1246          onDone: (String) -> Unit = {},
1247          onError: (String) -> Unit = {}
1248      ) {
1249          viewModelScope.launch {
1250              try {
1251                  val id = repository.recordPigmyWithdrawal(schemeId, amountPaise, destination, notes,
      withdrawalTimestamp, attachmentUri)
1252                  onDone(id)
1253              } catch (e: Exception) {
1254                  onError(e.message ?: "Failed to record withdrawal")
1255              }
1256          }
1257      }
1258
1259      fun updatePigmyWithdrawal(
1260          withdrawalId: String,
1261          schemeId: String,
1262          amountPaise: Long,
1263          destination: String,
1264          notes: String?,
1265          withdrawalTimestamp: Long,
1266          onDone: () -> Unit = {}
1267      ) {
1268          viewModelScope.launch {
1269              repository.updatePigmyWithdrawal(withdrawalId, schemeId, amountPaise, destination, notes,
      withdrawalTimestamp)
1270              onDone()
1271          }
1272      }
1273
1274      fun voidPigmyWithdrawal(withdrawalId: String, reason: String, onDone: () -> Unit = {}) {
1275          viewModelScope.launch {
1276              repository.voidPigmyWithdrawal(withdrawalId, reason)
1277              onDone()
1278          }
1279      }
1280
1281      fun updatePigmySchemeOpeningBalance(schemeId: String, openingBalancePaise: Long, date: Long? = null,
      onDone: () -> Unit = {}) {
1282          viewModelScope.launch {
1283              repository.updatePigmySchemeOpeningBalance(schemeId, openingBalancePaise, date)
1284              onDone()
1285          }
1286      }
1287
1288      fun createCashCategory(name: String, onDone: (String) -> Unit = {}) {
1289          viewModelScope.launch {
1290              try {
1291                  val id = repository.createCashCategory(name)
1292                  onDone(id)
1293              } catch (ignored: Exception) {}
1294          }
1295      }
1296
1297      fun renameCashCategory(id: String, newName: String, onDone: () -> Unit = {}) {
1298          viewModelScope.launch {
1299              repository.renameCashCategory(id, newName)
```

```
            onDone()
        }
    }

    fun setCashCategoryArchived(id: String, isArchived: Boolean, onDone: () -> Unit = {}) {
        viewModelScope.launch {
            repository.setCashCategoryArchived(id, isArchived)
            onDone()
        }
    }

    fun deleteCashCategory(id: String, onDone: () -> Unit = {}) {
        viewModelScope.launch {
            repository.deleteCashCategory(id)
            onDone()
        }
    }

    // ── Product Giveaways & Donations Actions ──────────────────────
    fun recordProductGiveaway(
        productId: String,
        quantityBase: Int,
        isPack: Boolean = false,
        packSizeSnapshot: Int = 1,
        reason: String = "FREE_ITEM",
        recipientNote: String? = null,
        onDone: () -> Unit = {}
    ) {
        viewModelScope.launch {
            repository.recordProductGiveaway(
                productId = productId,
                quantityBase = quantityBase,
                isPack = isPack,
                packSizeSnapshot = packSizeSnapshot,
                reason = reason,
                recipientNote = recipientNote
            )
            onDone()
        }
    }

    fun cancelProductGiveaway(giveawayId: String, reason: String? = null, onDone: () -> Unit = {}) {
        viewModelScope.launch {
            repository.cancelProductGiveaway(giveawayId, reason)
            onDone()
        }
    }

    // ── Services Actions ──────────────────────────────────────────
    fun addCustomService(name: String, unitLabel: String, onDone: (ServiceDefinitionEntity) -> Unit = {}) {
        viewModelScope.launch {
            val svc = repository.addCustomService(name, unitLabel)
            onDone(svc)
        }
    }

    fun archiveService(serviceId: String, onDone: () -> Unit = {}) {
        viewModelScope.launch {
            repository.archiveService(serviceId)
            onDone()
        }
    }

    fun addServiceToBill(
        serviceName: String,
        option: String?,
        unitLabel: String,
        quantity: Int,
        unitRatePaise: Long,
        serviceId: String? = null
    ) {
        val currentItems = _billingState.value.items.toMutableList()
        val displayName = if (!option.isNullOrBlank()) "$serviceName — $option" else serviceName
        val serviceLine = BillingDraftItem(
            lineId = java.util.UUID.randomUUID().toString(),
            productId = serviceId ?: "SERVICE_${serviceName.uppercase().replace(" ", "_")}",
            productName = displayName,
            urduName = null,
            unitType = "SERVICE",
            packSize = 1,
            quantity = quantity,
            unitPricePaise = unitRatePaise,
            movingAvgCostPaise = 0L,
            lineType = "SERVICE",
            serviceId = serviceId,
            serviceName = serviceName,
            serviceOption = option,
            unitLabel = unitLabel
```

```
        )
        currentItems.add(serviceLine)
        _billingState.value = _billingState.value.copy(
            items = currentItems,
            lastAddedItem = serviceLine
        )
    }

    fun updateItemPrice(lineId: String, newPricePaise: Long) {
        val currentItems = _billingState.value.items.toMutableList()
        val index = currentItems.indexOfFirst { it.lineId == lineId }
        if (index >= 0) {
            val item = currentItems[index]
            currentItems[index] = item.copy(unitPricePaise = newPricePaise)
            _billingState.value = _billingState.value.copy(items = currentItems)
        }
    }

    fun updateItemQuantity(lineId: String, newQuantity: Int) {
        val currentItems = _billingState.value.items.toMutableList()
        val index = currentItems.indexOfFirst { it.lineId == lineId }
        if (index >= 0) {
            if (newQuantity <= 0) {
                currentItems.removeAt(index)
            } else {
                currentItems[index] = currentItems[index].copy(quantity = newQuantity)
            }
            _billingState.value = _billingState.value.copy(items = currentItems)
        }
    }

    fun removeItemFromBill(lineId: String) {
        val currentItems = _billingState.value.items.toMutableList()
        currentItems.removeAll { it.lineId == lineId }
        _billingState.value = _billingState.value.copy(items = currentItems)
    }

    fun recordServiceExpense(
        category: String,
        amountPaise: Long,
        serviceTarget: String,
        notes: String? = null,
        attachmentUri: String? = null,
        timestamp: Long = System.currentTimeMillis(),
        onDone: (String) -> Unit = {}
    ) {
        viewModelScope.launch {
            val id = repository.recordServiceExpense(
                category = category,
                amountPaise = amountPaise,
                serviceTarget = serviceTarget,
                notes = notes,
                attachmentUri = attachmentUri,
                timestamp = timestamp
            )
            onDone(id)
        }
    }

    fun getServiceExpensesInRange(startTime: Long, endTime: Long) =
        repository.getServiceExpensesInRange(startTime, endTime)

    fun getServiceLinesInRange(startTime: Long, endTime: Long) =
        repository.getServiceLinesInRange(startTime, endTime)

    fun setDeviceRole(role: String) {
        viewModelScope.launch {
            preferences.setDeviceRole(role)
        }
    }

    fun setPairedOwnerHost(host: String) {
        viewModelScope.launch {
            preferences.setPairedOwnerHost(host)
        }
    }
}
```

app\src\main\java\com\alieducational\app\ui\auth\UnlockScreen.kt

```
package com.alieducational.app.ui.auth

import androidx.compose.foundation.Image
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Fingerprint
import androidx.compose.material3.Button
import androidx.compose.material3.ButtonDefaults
import androidx.compose.material3.Icon
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedButton
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.LaunchedEffect
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.layout.ContentScale
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.res.painterResource
import androidx.compose.ui.res.stringResource
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.style.TextAlign
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.fragment.app.FragmentActivity
import com.alieducational.app.R
import com.alieducational.app.util.BiometricAuthManager

@Composable
fun UnlockScreen(
    biometricAuthManager: BiometricAuthManager,
    onUnlocked: () -> Unit,
    modifier: Modifier = Modifier
) {
    val context = LocalContext.current
    val activity = context as? FragmentActivity

    fun triggerUnlock() {
        if (activity != null && biometricAuthManager.canAuthenticate()) {
            biometricAuthManager.promptBiometric(
                activity = activity,
                title = context.getString(R.string.unlock_title),
                subtitle = context.getString(R.string.unlock_prompt),
                onSuccess = onUnlocked,
                onError = { /* Keep on unlock screen */ }
            )
        } else {
            // If device has no screen lock, allow proceed
            onUnlocked()
        }
    }

    LaunchedEffect(Unit) {
        triggerUnlock()
    }

    Surface(
        modifier = modifier.fillMaxSize(),
        color = MaterialTheme.colorScheme.background
    ) {
        Column(
            modifier = Modifier
                .fillMaxSize()
                .padding(32.dp),
            horizontalAlignment = Alignment.CenterHorizontally,
            verticalArrangement = Arrangement.Center
        ) {
            Image(
                painter = painterResource(id = R.drawable.ali_logo_dark),
                contentDescription = "Ali Educational Logo",
                contentScale = ContentScale.Fit,
                modifier = Modifier
```

```
                    .width(190.dp)
                    .height(100.dp)
            )

            Spacer(modifier = Modifier.height(24.dp))

            Text(
                text = stringResource(R.string.unlock_title),
                style = MaterialTheme.typography.headlineLarge.copy(
                    fontWeight = FontWeight.Bold,
                    fontSize = 28.sp
                ),
                color = MaterialTheme.colorScheme.onBackground
            )

            Spacer(modifier = Modifier.height(8.dp))

            Text(
                text = stringResource(R.string.unlock_subtitle),
                style = MaterialTheme.typography.bodyLarge,
                color = MaterialTheme.colorScheme.onSurfaceVariant,
                textAlign = TextAlign.Center
            )

            Spacer(modifier = Modifier.height(48.dp))

            Button(
                onClick = { triggerUnlock() },
                modifier = Modifier
                    .fillMaxWidth(0.85f)
                    .height(56.dp),
                shape = RoundedCornerShape(12.dp),
                colors = ButtonDefaults.buttonColors(
                    containerColor = MaterialTheme.colorScheme.primary,
                    contentColor = MaterialTheme.colorScheme.onPrimary
                )
            ) {
                Icon(
                    imageVector = Icons.Filled.Fingerprint,
                    contentDescription = null,
                    modifier = Modifier.size(28.dp)
                )
                Spacer(modifier = Modifier.size(12.dp))
                Text(
                    text = stringResource(R.string.unlock_button),
                    style = MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.SemiBold)
                )
            }

            Spacer(modifier = Modifier.height(16.dp))

            OutlinedButton(
                onClick = { triggerUnlock() },
                modifier = Modifier.height(48.dp),
                shape = RoundedCornerShape(12.dp)
            ) {
                Text(text = "Use Device PIN / Password")
            }

            Spacer(modifier = Modifier.height(8.dp))

            androidx.compose.material3.TextButton(
                onClick = onUnlocked
            ) {
                Text(
                    text = "Enter Shop Directly",
                    style = MaterialTheme.typography.bodySmall,
                    color = MaterialTheme.colorScheme.primary
                )
            }
        }
    }
}
```

app\src\main\java\com\alieducational\app\ui\billing\BarcodeAnalyzer.kt

```
package com.alieducational.app.ui.billing

import androidx.annotation.OptIn
import androidx.camera.core.ExperimentalGetImage
import androidx.camera.core.ImageAnalysis
import androidx.camera.core.ImageProxy
import com.google.zxing.BarcodeFormat
import com.google.zxing.BinaryBitmap
import com.google.zxing.DecodeHintType
import com.google.zxing.MultiFormatReader
import com.google.zxing.PlanarYUVLuminanceSource
import com.google.zxing.common.HybridBinarizer
import java.nio.ByteBuffer
import java.util.EnumMap

class BarcodeAnalyzer(
    private val onBarcodeDetected: (barcode: String, format: String) -> Unit,
    private val onFrameEmpty: () -> Unit = {}
) : ImageAnalysis.Analyzer {

    private val reader = MultiFormatReader().apply {
        val hints = EnumMap<DecodeHintType, Any>(DecodeHintType::class.java).apply {
            put(
                DecodeHintType.POSSIBLE_FORMATS,
                listOf(
                    BarcodeFormat.EAN_13,
                    BarcodeFormat.EAN_8,
                    BarcodeFormat.UPC_A,
                    BarcodeFormat.UPC_E,
                    BarcodeFormat.CODE_128,
                    BarcodeFormat.CODE_39,
                    BarcodeFormat.ITF,
                    BarcodeFormat.QR_CODE
                )
            )
            put(DecodeHintType.TRY_HARDER, java.lang.Boolean.TRUE)
        }
        setHints(hints)
    }

    private var lastScannedCode: String? = null
    private var lastScannedTimestamp: Long = 0L
    private var consecutiveEmptyFrames = 0

    @OptIn(ExperimentalGetImage::class)
    override fun analyze(imageProxy: ImageProxy) {
        val mediaImage = imageProxy.image
        if (mediaImage != null) {
            val plane = mediaImage.planes[0]
            val buffer: ByteBuffer = plane.buffer
            val rowStride = plane.rowStride
            val pixelStride = plane.pixelStride
            val width = imageProxy.width
            val height = imageProxy.height

            val yData = ByteArray(width * height)
            if (rowStride == width && pixelStride == 1) {
                buffer.get(yData)
            } else {
                for (row in 0 until height) {
                    buffer.position(row * rowStride)
                    if (pixelStride == 1) {
                        buffer.get(yData, row * width, width)
                    } else {
                        for (col in 0 until width) {
                            yData[row * width + col] = buffer.get(row * rowStride + col * pixelStride)
                        }
                    }
                }
            }

            val rotation = imageProxy.imageInfo.rotationDegrees

            val source = when (rotation) {
                90 -> {
                    val rotatedData = ByteArray(width * height)
                    for (y in 0 until height) {
                        for (x in 0 until width) {
                            rotatedData[x * height + (height - y - 1)] = yData[x + y * width]
                        }
                    }
                    PlanarYUVLuminanceSource(
```

```
                        rotatedData, height, width, 0, 0, height, width, false
                    )
                }
                270 -> {
                    val rotatedData = ByteArray(width * height)
                    for (y in 0 until height) {
                        for (x in 0 until width) {
                            rotatedData[(width - x - 1) * height + y] = yData[x + y * width]
                        }
                    }
                    PlanarYUVLuminanceSource(
                        rotatedData, height, width, 0, 0, height, width, false
                    )
                }
                180 -> {
                    val rotatedData = ByteArray(width * height)
                    for (i in 0 until width * height) {
                        rotatedData[i] = yData[width * height - 1 - i]
                    }
                    PlanarYUVLuminanceSource(
                        rotatedData, width, height, 0, 0, width, height, false
                    )
                }
                else -> {
                    PlanarYUVLuminanceSource(
                        yData, width, height, 0, 0, width, height, false
                    )
                }
            }

            var detected = false
            try {
                val bitmap = BinaryBitmap(HybridBinarizer(source))
                val result = reader.decodeWithState(bitmap)
                val rawCode = result.text?.trim()
                if (!rawCode.isNullOrEmpty()) {
                    detected = true
                    handleDetectedBarcode(rawCode, result.barcodeFormat.name)
                }
            } catch (e: Exception) {
                // Fallback attempt with unrotated buffer if rotation failed
                if (rotation != 0) {
                    try {
                        val fallbackSource = PlanarYUVLuminanceSource(
                            yData, width, height, 0, 0, width, height, false
                        )
                        val fallbackBitmap = BinaryBitmap(HybridBinarizer(fallbackSource))
                        val fallbackResult = reader.decodeWithState(fallbackBitmap)
                        val rawCode = fallbackResult.text?.trim()
                        if (!rawCode.isNullOrEmpty()) {
                            detected = true
                            handleDetectedBarcode(rawCode, fallbackResult.barcodeFormat.name)
                        }
                    } catch (e2: Exception) {
                        // ignore
                    }
                }
            } finally {
                reader.reset()
            }

            if (!detected) {
                consecutiveEmptyFrames++
                if (consecutiveEmptyFrames > 15) {
                    lastScannedCode = null
                    onFrameEmpty()
                }
            }
        }
        imageProxy.close()
    }

    private fun handleDetectedBarcode(rawCode: String, formatName: String) {
        consecutiveEmptyFrames = 0
        val now = System.currentTimeMillis()
        if (rawCode != lastScannedCode || now - lastScannedTimestamp > 2500) {
            lastScannedCode = rawCode
            lastScannedTimestamp = now
            onBarcodeDetected(rawCode, formatName)
        }
    }

    fun resetLastScanned() {
        lastScannedCode = null
        lastScannedTimestamp = 0L
    }
}
```

app\src\main\java\com\alieducational\app\ui\billing\BarcodeScannerScreen.kt

```
package com.alieducational.app.ui.billing

import android.Manifest
import android.content.pm.PackageManager
import androidx.activity.compose.rememberLauncherForActivityResult
import androidx.activity.result.contract.ActivityResultContracts
import androidx.camera.core.Camera
import androidx.camera.core.CameraSelector
import androidx.camera.core.ImageAnalysis
import androidx.camera.core.Preview
import androidx.camera.lifecycle.ProcessCameraProvider
import androidx.camera.view.PreviewView
import androidx.compose.foundation.Image
import androidx.compose.foundation.background
import androidx.compose.foundation.border
import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.ArrowBack
import androidx.compose.material.icons.filled.FiberManualRecord
import androidx.compose.material.icons.filled.FlashOff
import androidx.compose.material.icons.filled.FlashOn
import androidx.compose.material3.AlertDialog
import androidx.compose.material3.Button
import androidx.compose.material3.ButtonDefaults
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.DropdownMenuItem
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.ExposedDropdownMenuBox
import androidx.compose.material3.ExposedDropdownMenuDefaults
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedButton
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.material3.TextButton
import androidx.compose.runtime.Composable
import androidx.compose.runtime.DisposableEffect
import androidx.compose.runtime.LaunchedEffect
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.layout.ContentScale
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.res.painterResource
import androidx.compose.ui.res.stringResource
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.compose.ui.viewinterop.AndroidView
import androidx.core.content.ContextCompat
import androidx.lifecycle.compose.LocalLifecycleOwner
import com.alieducational.app.R
import com.alieducational.app.domain.calculation.CostCalculator
import com.alieducational.app.ui.AliViewModel
import java.util.concurrent.Executors

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun BarcodeScannerScreen(
    viewModel: AliViewModel,
    onNavigateBack: () -> Unit,
```

```
    onNavigateToReview: () -> Unit,
    modifier: Modifier = Modifier
) {
    val context = LocalContext.current
    val lifecycleOwner = LocalLifecycleOwner.current
    val isDarkTheme by viewModel.isDarkTheme.collectAsState()
    val billingState by viewModel.billingState.collectAsState()
    val allProducts by viewModel.allProducts.collectAsState()

    var hasCameraPermission by remember {
        mutableStateOf(
            ContextCompat.checkSelfPermission(context, Manifest.permission.CAMERA) ==
PackageManager.PERMISSION_GRANTED
        )
    }

    val permissionLauncher = rememberLauncherForActivityResult(
        contract = ActivityResultContracts.RequestPermission()
    ) { granted ->
        hasCameraPermission = granted
    }

    LaunchedEffect(Unit) {
        if (!hasCameraPermission) {
            permissionLauncher.launch(Manifest.permission.CAMERA)
        }
    }

    var camera by remember { mutableStateOf<Camera?>(null) }
    var isTorchOn by remember { mutableStateOf(false) }

    // Unknown Barcode Link Dialog State
    var unknownBarcodeToLink by remember { mutableStateOf<String?>(null) }
    var selectedProductForLink by remember { mutableStateOf("") }
    var selectedUnitForLink by remember { mutableStateOf("PIECE") }
    var linkDropdownExpanded by remember { mutableStateOf(false) }

    Surface(
        modifier = modifier.fillMaxSize(),
        color = MaterialTheme.colorScheme.background
    ) {
        Column(
            modifier = Modifier
                .fillMaxSize()
                .padding(horizontal = 16.dp),
            verticalArrangement = Arrangement.spacedBy(10.dp)
        ) {
            // Header
            Spacer(modifier = Modifier.height(4.dp))
            Row(
                modifier = Modifier.fillMaxWidth(),
                verticalAlignment = Alignment.CenterVertically
            ) {
                IconButton(onClick = onNavigateBack) {
                    Icon(Icons.Filled.ArrowBack, contentDescription = "Back", tint =
MaterialTheme.colorScheme.onBackground)
                }
                val logoRes = if (isDarkTheme) R.drawable.ali_logo_dark else R.drawable.ali_logo_light
                Image(
                    painter = painterResource(id = logoRes),
                    contentDescription = null,
                    modifier = Modifier
                        .height(38.dp)
                        .width(72.dp),
                    contentScale = ContentScale.Fit
                )
                Spacer(modifier = Modifier.width(10.dp))
                Column {
                    Text(
                        text = "Ali Educational",
                        style = MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.Bold),
                        color = MaterialTheme.colorScheme.onBackground
                    )
                    Text(
                        text = stringResource(R.string.billing_header_subtitle),
                        style = MaterialTheme.typography.bodySmall,
                        color = MaterialTheme.colorScheme.onSurfaceVariant
                    )
                }
            }

            // Title & Frame Guidance
            Text(
                text = stringResource(R.string.scanner_title),
                style = MaterialTheme.typography.headlineMedium.copy(fontWeight = FontWeight.Bold),
                color = MaterialTheme.colorScheme.onBackground
            )
            Text(
```

```
                text = stringResource(R.string.scanner_guidance),
                style = MaterialTheme.typography.bodySmall,
                color = MaterialTheme.colorScheme.onSurfaceVariant
            )

            // Camera Viewfinder Box with Barcode Reticle
            Box(
                modifier = Modifier
                    .fillMaxWidth()
                    .height(260.dp)
                    .clip(RoundedCornerShape(16.dp))
                    .background(Color.Black)
            ) {
                if (hasCameraPermission) {
                    AndroidView(
                        factory = { ctx ->
                            val previewView = PreviewView(ctx)
                            val cameraProviderFuture = ProcessCameraProvider.getInstance(ctx)
                            val cameraExecutor = Executors.newSingleThreadExecutor()

                            cameraProviderFuture.addListener({
                                val cameraProvider = cameraProviderFuture.get()

                                val preview = Preview.Builder().build().also {
                                    it.surfaceProvider = previewView.surfaceProvider
                                }

                                val imageAnalyzer = ImageAnalysis.Builder()
                                    .setBackpressureStrategy(ImageAnalysis.STRATEGY_KEEP_ONLY_LATEST)
                                    .build()
                                    .also { analysis ->
                                        analysis.setAnalyzer(
                                            cameraExecutor,
                                            BarcodeAnalyzer(
                                                onBarcodeDetected = { barcode, _ ->
                                                    viewModel.handleScannedBarcode(
                                                        barcode = barcode,
                                                        onUnknown = { unknownCode ->
                                                            unknownBarcodeToLink = unknownCode
                                                        }
                                                    )
                                                }
                                            )
                                        )
                                    }

                                val cameraSelector = CameraSelector.DEFAULT_BACK_CAMERA

                                try {
                                    cameraProvider.unbindAll()
                                    camera = cameraProvider.bindToLifecycle(
                                        lifecycleOwner,
                                        cameraSelector,
                                        preview,
                                        imageAnalyzer
                                    )
                                } catch (e: Exception) {
                                    // Camera bind failure fallback
                                }
                            }, ContextCompat.getMainExecutor(ctx))

                            previewView
                        },
                        modifier = Modifier.fillMaxSize()
                    )

                    // Reticle overlay
                    Box(
                        modifier = Modifier
                            .size(width = 240.dp, height = 130.dp)
                            .align(Alignment.Center)
                            .border(2.dp, MaterialTheme.colorScheme.primary, RoundedCornerShape(12.dp))
                    )
                } else {
                    // Permission not granted fallback
                    Column(
                        modifier = Modifier
                            .fillMaxSize()
                            .padding(20.dp),
                        horizontalAlignment = Alignment.CenterHorizontally,
                        verticalArrangement = Arrangement.Center
                    ) {
                        Text(
                            text = stringResource(R.string.scanner_permission_needed),
                            color = Color.White,
                            style = MaterialTheme.typography.bodyMedium
                        )
                        Spacer(modifier = Modifier.height(10.dp))
                        Button(
```

```
                            onClick = { permissionLauncher.launch(Manifest.permission.CAMERA) },
                            colors = ButtonDefaults.buttonColors(containerColor =
MaterialTheme.colorScheme.primary)
                        ) {
                            Text(stringResource(R.string.scanner_grant_permission))
                        }
                    }
                }

                // Torch Toggle Control inside Viewfinder
                Row(
                    modifier = Modifier
                        .align(Alignment.BottomCenter)
                        .padding(bottom = 12.dp)
                        .clip(RoundedCornerShape(20.dp))
                        .background(Color(0x99000000))
                        .clickable {
                            isTorchOn = !isTorchOn
                            camera?.cameraControl?.enableTorch(isTorchOn)
                        }
                        .padding(horizontal = 14.dp, vertical = 6.dp),
                    verticalAlignment = Alignment.CenterVertically
                ) {
                    Icon(
                        if (isTorchOn) Icons.Filled.FlashOn else Icons.Filled.FlashOff,
                        contentDescription = null,
                        tint = if (isTorchOn) MaterialTheme.colorScheme.primary else Color.White,
                        modifier = Modifier.size(16.dp)
                    )
                    Spacer(modifier = Modifier.width(6.dp))
                    Text(
                        text = stringResource(R.string.scanner_torch),
                        color = Color.White,
                        fontSize = 12.sp,
                        fontWeight = FontWeight.Medium
                    )
                }
            }

            // Last Scanned Item Banner with Undo
            val lastItem = billingState.lastAddedItem
            if (lastItem != null) {
                Card(
                    modifier = Modifier.fillMaxWidth(),
                    shape = RoundedCornerShape(12.dp),
                    colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surface)
                ) {
                    Row(
                        modifier = Modifier
                            .fillMaxWidth()
                            .padding(14.dp),
                        horizontalArrangement = Arrangement.SpaceBetween,
                        verticalAlignment = Alignment.CenterVertically
                    ) {
                        Column {
                            Text(
                                text = "${stringResource(R.string.scanner_added_label)}:
${lastItem.productName}",
                                style = MaterialTheme.typography.bodyMedium.copy(fontWeight =
FontWeight.Bold),
                                color = MaterialTheme.colorScheme.onSurface
                            )
                            val unitStr = if (lastItem.unitType == "PACK") "pack" else "piece"
                            val priceStr = CostCalculator.formatPaiseToRupees(lastItem.unitPricePaise)
                            Text(
                                text = "${lastItem.quantity} $unitStr · $priceStr",
                                style = MaterialTheme.typography.bodySmall,
                                color = MaterialTheme.colorScheme.onSurfaceVariant
                            )
                        }

                        TextButton(onClick = { viewModel.undoLastAddition() }) {
                            Text(
                                text = stringResource(R.string.scanner_undo),
                                color = Color(0xFFE57373),
                                fontWeight = FontWeight.Bold
                            )
                        }
                    }
                }
            }

            // Anti-duplicate Guidance Note
            Text(
                text = stringResource(R.string.scanner_debounce_hint),
                style = MaterialTheme.typography.bodySmall,
                color = MaterialTheme.colorScheme.onSurfaceVariant
            )
```

```
342
343             // Bottom Action Row: [Add again] and [View bill · ₹165]
344             Row(
345                 modifier = Modifier.fillMaxWidth(),
346                 horizontalArrangement = Arrangement.spacedBy(10.dp)
347             ) {
348                 OutlinedButton(
349                     onClick = { viewModel.addAgainLastItem() },
350                     enabled = lastItem != null,
351                     modifier = Modifier
352                         .weight(1f)
353                         .height(52.dp),
354                     shape = RoundedCornerShape(12.dp)
355                 ) {
356                     Text(
357                         text = stringResource(R.string.scanner_add_again),
358                         style = MaterialTheme.typography.titleSmall.copy(fontWeight = FontWeight.Bold)
359                     )
360                 }
361
362                 val totalStr = CostCalculator.formatPaiseToRupees(billingState.grossTotalPaise)
363                 Button(
364                     onClick = onNavigateToReview,
365                     modifier = Modifier
366                         .weight(1.3f)
367                         .height(52.dp),
368                     shape = RoundedCornerShape(12.dp),
369                     colors = ButtonDefaults.buttonColors(
370                         containerColor = MaterialTheme.colorScheme.primary,
371                         contentColor = MaterialTheme.colorScheme.onPrimary
372                     )
373                 ) {
374                     Text(
375                         text = "${stringResource(R.string.scanner_view_bill)} · $totalStr",
376                         style = MaterialTheme.typography.titleSmall.copy(fontWeight = FontWeight.Bold)
377                     )
378                 }
379             }
380
381             // Fallback: No barcode? Search or browse items
382             Box(
383                 modifier = Modifier
384                     .fillMaxWidth()
385                     .padding(vertical = 8.dp),
386                 contentAlignment = Alignment.Center
387             ) {
388                 Text(
389                     text = stringResource(R.string.scanner_no_barcode_fallback),
390                     style = MaterialTheme.typography.bodyMedium.copy(fontWeight = FontWeight.SemiBold),
391                     color = MaterialTheme.colorScheme.primary,
392                     modifier = Modifier.clickable { onNavigateBack() }
393                 )
394             }
395         }
396     }
397
398     // Dialog for Unknown Barcode: Link to existing product or Cancel
399     if (unknownBarcodeToLink != null) {
400         val code = unknownBarcodeToLink!!
401         AlertDialog(
402             onDismissRequest = {
403                 unknownBarcodeToLink = null
404                 viewModel.clearUnknownBarcode()
405             },
406             title = { Text(stringResource(R.string.scanner_unknown_title)) },
407             text = {
408                 Column(verticalArrangement = Arrangement.spacedBy(10.dp)) {
409                     Text(
410                         text = "${stringResource(R.string.scanner_scanned_code)}: $code",
411                         fontWeight = FontWeight.Bold,
412                         color = MaterialTheme.colorScheme.primary
413                     )
414                     Text(stringResource(R.string.scanner_unknown_prompt))
415
416                     ExposedDropdownMenuBox(
417                         expanded = linkDropdownExpanded,
418                         onExpandedChange = { linkDropdownExpanded = !linkDropdownExpanded }
419                     ) {
420                         OutlinedTextField(
421                             value = allProducts.find { it.id == selectedProductForLink }?.name ?:
    stringResource(R.string.scanner_select_product_to_link),
422                             onValueChange = {},
423                             readOnly = true,
424                             trailingIcon = { ExposedDropdownMenuDefaults.TrailingIcon(expanded =
    linkDropdownExpanded) },
425                             modifier = Modifier
426                                 .fillMaxWidth()
427                                 .menuAnchor()
```

```
                        )
                        ExposedDropdownMenu(
                            expanded = linkDropdownExpanded,
                            onDismissRequest = { linkDropdownExpanded = false }
                        ) {
                            allProducts.forEach { p ->
                                DropdownMenuItem(
                                    text = { Text(p.name) },
                                    onClick = {
                                        selectedProductForLink = p.id
                                        linkDropdownExpanded = false
                                    }
                                )
                            }
                        }
                    }
                }
            },
            confirmButton = {
                Button(
                    onClick = {
                        if (selectedProductForLink.isNotBlank()) {
                            viewModel.linkAndAddUnknownBarcode(
                                barcode = code,
                                productId = selectedProductForLink,
                                unitType = selectedUnitForLink,
                                packQty = 1,
                                onDone = { unknownBarcodeToLink = null }
                            )
                        }
                    },
                    enabled = selectedProductForLink.isNotBlank(),
                    colors = ButtonDefaults.buttonColors(containerColor = MaterialTheme.colorScheme.primary)
                ) {
                    Text(stringResource(R.string.scanner_link_btn))
                }
            },
            dismissButton = {
                TextButton(onClick = {
                    unknownBarcodeToLink = null
                    viewModel.clearUnknownBarcode()
                }) {
                    Text(stringResource(R.string.common_cancel))
                }
            }
        )
    }
}
```

app\src\main\java\com\alieducational\app\ui\billing\BillHistoryScreen.kt

```
package com.alieducational.app.ui.billing

import android.app.DatePickerDialog
import androidx.activity.compose.BackHandler
import androidx.compose.foundation.background
import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.ArrowBack
import androidx.compose.material.icons.filled.CalendarMonth
import androidx.compose.material.icons.filled.CheckCircle
import androidx.compose.material.icons.filled.Close
import androidx.compose.material.icons.filled.CreditCard
import androidx.compose.material.icons.filled.MoneyOff
import androidx.compose.material.icons.filled.Payment
import androidx.compose.material.icons.filled.Person
import androidx.compose.material.icons.filled.Print
import androidx.compose.material.icons.filled.Receipt
import androidx.compose.material.icons.filled.Search
import androidx.compose.material3.AlertDialog
import androidx.compose.material3.Button
import androidx.compose.material3.ButtonDefaults
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.DividerDefaults
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.FilterChip
import androidx.compose.material3.HorizontalDivider
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedButton
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.material3.TextButton
import androidx.compose.material3.TopAppBar
import androidx.compose.material3.TopAppBarDefaults
import androidx.compose.runtime.Composable
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.alieducational.app.data.local.entity.SaleEntity
import com.alieducational.app.data.local.entity.SaleLineEntity
import com.alieducational.app.domain.calculation.CostCalculator
import com.alieducational.app.ui.AliViewModel
import java.text.SimpleDateFormat
import java.util.Calendar
import java.util.Date
import java.util.Locale

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun BillHistoryScreen(
    viewModel: AliViewModel,
    onNavigateBack: () -> Unit,
    onNavigateToReceipt: (String) -> Unit = {},
    onNavigateToCustomerAccount: (String) -> Unit = {},
    modifier: Modifier = Modifier
```

```
) {
    BackHandler {
        onNavigateBack()
    }

    val context = LocalContext.current
    val allSales by viewModel.allSales.collectAsState()
    val allSaleLines by viewModel.allSaleLines.collectAsState()

    var searchQuery by remember { mutableStateOf("") }
    var selectedPeriod by remember { mutableStateOf("Today") } // Today, This Week, This Month, All Time,
Custom
    var selectedStatus by remember { mutableStateOf("All") } // All, Paid, Due, Voided

    // Custom date range bounds
    var customStartDate by remember { mutableStateOf<Long?>(null) }
    var customEndDate by remember { mutableStateOf<Long?>(null) }

    var selectedSaleForDetail by remember { mutableStateOf<SaleEntity?>(null) }
    var repaymentSaleTarget by remember { mutableStateOf<SaleEntity?>(null) }

    // Calculate time interval
    val now = System.currentTimeMillis()
    val calendar = Calendar.getInstance()
    val (startTime, endTime) = when (selectedPeriod) {
        "Today" -> {
            calendar.timeInMillis = now
            calendar.set(Calendar.HOUR_OF_DAY, 0)
            calendar.set(Calendar.MINUTE, 0)
            calendar.set(Calendar.SECOND, 0)
            calendar.set(Calendar.MILLISECOND, 0)
            val start = calendar.timeInMillis
            calendar.set(Calendar.HOUR_OF_DAY, 23)
            calendar.set(Calendar.MINUTE, 59)
            calendar.set(Calendar.SECOND, 59)
            calendar.set(Calendar.MILLISECOND, 999)
            Pair(start, calendar.timeInMillis)
        }
        "This Week" -> {
            calendar.timeInMillis = now
            calendar.set(Calendar.DAY_OF_WEEK, calendar.firstDayOfWeek)
            calendar.set(Calendar.HOUR_OF_DAY, 0)
            calendar.set(Calendar.MINUTE, 0)
            calendar.set(Calendar.SECOND, 0)
            calendar.set(Calendar.MILLISECOND, 0)
            val start = calendar.timeInMillis
            Pair(start, now)
        }
        "This Month" -> {
            calendar.timeInMillis = now
            calendar.set(Calendar.DAY_OF_MONTH, 1)
            calendar.set(Calendar.HOUR_OF_DAY, 0)
            calendar.set(Calendar.MINUTE, 0)
            calendar.set(Calendar.SECOND, 0)
            calendar.set(Calendar.MILLISECOND, 0)
            val start = calendar.timeInMillis
            Pair(start, now)
        }
        "Custom" -> {
            val s = customStartDate ?: 0L
            val e = customEndDate ?: Long.MAX_VALUE
            Pair(s, e)
        }
        else -> Pair(0L, Long.MAX_VALUE) // All Time
    }

    val displayDateRangeText = remember(selectedPeriod, startTime, endTime) {
        val df = SimpleDateFormat("dd MMM yyyy", Locale.US)
        when (selectedPeriod) {
            "Today" -> df.format(Date())
            "This Week", "This Month", "Custom" -> {
                if (startTime == 0L && endTime == Long.MAX_VALUE) "All Time"
                else "${df.format(Date(startTime))} - ${df.format(Date(minOf(endTime, now)))}"
            }
            else -> "All recorded bills"
        }
    }

    // Filtered sales
    val filteredSales = remember(allSales, startTime, endTime, searchQuery, selectedStatus) {
        allSales.filter { sale ->
            val matchesTime = sale.timestamp in startTime..endTime
            val matchesSearch = if (searchQuery.isBlank()) true else {
                sale.receiptNumber.contains(searchQuery, ignoreCase = true) ||
                        (sale.customerName?.contains(searchQuery, ignoreCase = true) == true)
            }
            val matchesStatus = when (selectedStatus) {
                "Paid" -> !sale.isVoided && sale.remainingCreditPaise == 0L
```

```
                "Due" -> !sale.isVoided && sale.remainingCreditPaise > 0L
                "Voided" -> sale.isVoided
                else -> true
            }
            matchesTime && matchesSearch && matchesStatus
        }
    }

    val completedBills = filteredSales.filter { !it.isVoided }
    val totalBilledPaise = completedBills.sumOf { it.netTotalPaise }
    val totalCollectedPaise = completedBills.sumOf { it.amountReceivedPaise }
    val totalDuesPaise = completedBills.sumOf { it.remainingCreditPaise }

    Scaffold(
        topBar = {
            TopAppBar(
                title = {
                    Column {
                        Text("Bill History", fontWeight = FontWeight.Bold, fontSize = 18.sp)
                        Text(
                            displayDateRangeText,
                            style = MaterialTheme.typography.bodySmall,
                            color = MaterialTheme.colorScheme.onSurfaceVariant
                        )
                    }
                },
                navigationIcon = {
                    IconButton(onClick = onNavigateBack) {
                        Icon(Icons.Filled.ArrowBack, contentDescription = "Back")
                    }
                },
                colors = TopAppBarDefaults.topAppBarColors(
                    containerColor = MaterialTheme.colorScheme.surface,
                    titleContentColor = MaterialTheme.colorScheme.onSurface
                )
            )
        }
    ) { padding ->
        Surface(
            modifier = modifier
                .fillMaxSize()
                .padding(padding),
            color = MaterialTheme.colorScheme.background
        ) {
            LazyColumn(
                modifier = Modifier
                    .fillMaxSize()
                    .padding(horizontal = 16.dp),
                verticalArrangement = Arrangement.spacedBy(12.dp)
            ) {
                item {
                    Spacer(modifier = Modifier.height(6.dp))

                    // Period Selector Chips
                    Row(
                        modifier = Modifier.fillMaxWidth(),
                        horizontalArrangement = Arrangement.spacedBy(6.dp)
                    ) {
                        listOf("Today", "This Week", "This Month", "All Time", "Custom").forEach { period ->
                            FilterChip(
                                selected = selectedPeriod == period,
                                onClick = {
                                    selectedPeriod = period
                                    if (period == "Custom" && customStartDate == null) {
                                        val c = Calendar.getInstance()
                                        DatePickerDialog(
                                            context,
                                            { _, y, m, d ->
                                                val startCal = Calendar.getInstance().apply {
                                                    set(y, m, d, 0, 0, 0)
                                                    set(Calendar.MILLISECOND, 0)
                                                }
                                                customStartDate = startCal.timeInMillis

                                                DatePickerDialog(
                                                    context,
                                                    { _, ey, em, ed ->
                                                        val endCal = Calendar.getInstance().apply {
                                                            set(ey, em, ed, 23, 59, 59)
                                                            set(Calendar.MILLISECOND, 999)
                                                        }
                                                        customEndDate = endCal.timeInMillis
                                                    },
                                                    y, m, d
                                                ).show()
                                            },
                                            c.get(Calendar.YEAR),
                                            c.get(Calendar.MONTH),
```

```
                                        c.get(Calendar.DAY_OF_MONTH)
                                    ).show()
                                }
                            },
                            label = { Text(period, fontSize = 12.sp) }
                        )
                    }
                }

                Spacer(modifier = Modifier.height(6.dp))

                // Search & Status Row
                OutlinedTextField(
                    value = searchQuery,
                    onValueChange = { searchQuery = it },
                    modifier = Modifier.fillMaxWidth(),
                    placeholder = { Text("Search by receipt # or customer name") },
                    leadingIcon = { Icon(Icons.Filled.Search, contentDescription = null) },
                    trailingIcon = {
                        if (searchQuery.isNotEmpty()) {
                            IconButton(onClick = { searchQuery = "" }) {
                                Icon(Icons.Filled.Close, contentDescription = "Clear")
                            }
                        }
                    },
                    singleLine = true,
                    shape = RoundedCornerShape(10.dp)
                )

                Spacer(modifier = Modifier.height(8.dp))

                // Status Chips
                Row(
                    modifier = Modifier.fillMaxWidth(),
                    horizontalArrangement = Arrangement.spacedBy(8.dp)
                ) {
                    listOf("All", "Paid", "Due", "Voided").forEach { st ->
                        FilterChip(
                            selected = selectedStatus == st,
                            onClick = { selectedStatus = st },
                            label = { Text(st, fontSize = 12.sp) }
                        )
                    }
                }

                Spacer(modifier = Modifier.height(8.dp))

                // Financial Metrics Banner
                Card(
                    modifier = Modifier.fillMaxWidth(),
                    colors = CardDefaults.cardColors(containerColor =
MaterialTheme.colorScheme.surface),
                    shape = RoundedCornerShape(12.dp)
                ) {
                    Column(modifier = Modifier.padding(14.dp)) {
                        Row(
                            modifier = Modifier.fillMaxWidth(),
                            horizontalArrangement = Arrangement.SpaceBetween
                        ) {
                            Column {
                                Text("Bills", style = MaterialTheme.typography.labelSmall, color =
MaterialTheme.colorScheme.onSurfaceVariant)
                                Text(
                                    "${completedBills.size}",
                                    style = MaterialTheme.typography.titleMedium.copy(fontWeight =
FontWeight.Bold)
                                )
                            }
                            Column(horizontalAlignment = Alignment.End) {
                                Text("Net Billed", style = MaterialTheme.typography.labelSmall, color =
MaterialTheme.colorScheme.onSurfaceVariant)
                                Text(
                                    CostCalculator.formatPaiseToRupees(totalBilledPaise),
                                    style = MaterialTheme.typography.titleMedium.copy(fontWeight =
FontWeight.Bold),
                                    color = MaterialTheme.colorScheme.primary
                                )
                            }
                        }

                        Spacer(modifier = Modifier.height(8.dp))
                        HorizontalDivider(color = MaterialTheme.colorScheme.outlineVariant.copy(alpha =
0.5f))
                        Spacer(modifier = Modifier.height(8.dp))

                        Row(
                            modifier = Modifier.fillMaxWidth(),
                            horizontalArrangement = Arrangement.SpaceBetween
```

```
                            ) {
                                Column {
                                    Text("Total Collected", style = MaterialTheme.typography.labelSmall,
    color = Color(0xFF2E7D32))
                                    Text(
                                        CostCalculator.formatPaiseToRupees(totalCollectedPaise),
                                        fontWeight = FontWeight.Bold,
                                        color = Color(0xFF2E7D32)
                                    )
                                }
                                Column(horizontalAlignment = Alignment.End) {
                                    Text("Remaining Dues", style = MaterialTheme.typography.labelSmall,
    color = if (totalDuesPaise > 0) Color(0xFFD32F2F) else MaterialTheme.colorScheme.onSurfaceVariant)
                                    Text(
                                        CostCalculator.formatPaiseToRupees(totalDuesPaise),
                                        fontWeight = FontWeight.Bold,
                                        color = if (totalDuesPaise > 0) Color(0xFFD32F2F) else
    MaterialTheme.colorScheme.onSurface
                                    )
                                }
                            }
                        }
                    }
                }

                // List of Bills
                if (filteredSales.isEmpty()) {
                    item {
                        Card(
                            modifier = Modifier
                                .fillMaxWidth()
                                .padding(vertical = 24.dp),
                            shape = RoundedCornerShape(12.dp),
                            colors = CardDefaults.cardColors(containerColor =
    MaterialTheme.colorScheme.surface)
                        ) {
                            Column(
                                modifier = Modifier
                                    .fillMaxWidth()
                                    .padding(24.dp),
                                horizontalAlignment = Alignment.CenterHorizontally
                            ) {
                                Text(
                                    text = "No bills found in this period or filter.",
                                    style = MaterialTheme.typography.bodyMedium,
                                    color = MaterialTheme.colorScheme.onSurfaceVariant
                                )
                            }
                        }
                    }
                } else {
                    items(filteredSales, key = { it.id }) { sale ->
                        BillHistoryRow(
                            sale = sale,
                            onClick = { selectedSaleForDetail = sale },
                            onRepay = { repaymentSaleTarget = sale },
                            onOpenCustomer = {
                                sale.customerId?.let { cId -> onNavigateToCustomerAccount(cId) }
                            }
                        )
                    }
                }

                item {
                    Spacer(modifier = Modifier.height(32.dp))
                }
            }
        }
    }

    // Bill Detail Dialog
    selectedSaleForDetail?.let { sale ->
        val saleLines = remember(sale.id, allSaleLines) {
            allSaleLines.filter { it.saleId == sale.id }
        }

        AlertDialog(
            onDismissRequest = { selectedSaleForDetail = null },
            title = {
                Column {
                    Row(
                        modifier = Modifier.fillMaxWidth(),
                        horizontalArrangement = Arrangement.SpaceBetween,
                        verticalAlignment = Alignment.CenterVertically
                    ) {
                        Text(sale.receiptNumber, fontWeight = FontWeight.Bold, fontSize = 16.sp)
                        if (sale.isVoided) {
                            Text("VOIDED", color = MaterialTheme.colorScheme.error, fontWeight =
```

```
    FontWeight.Bold, fontSize = 12.sp)
424                         } else if (sale.remainingCreditPaise > 0) {
425                             Text("DUE", color = Color(0xFFD32F2F), fontWeight = FontWeight.Bold, fontSize =
    12.sp)
426                         } else {
427                             Text("PAID", color = Color(0xFF2E7D32), fontWeight = FontWeight.Bold, fontSize =
     12.sp)
428                         }
429                     }
430                     val dateFormatted = SimpleDateFormat("dd MMM yyyy, hh:mm a",
    Locale.US).format(Date(sale.timestamp))
431                     Text(dateFormatted, style = MaterialTheme.typography.bodySmall, color =
    MaterialTheme.colorScheme.onSurfaceVariant)
432                 }
433             },
434             text = {
435                 Column(
436                     modifier = Modifier.fillMaxWidth(),
437                     verticalArrangement = Arrangement.spacedBy(8.dp)
438                 ) {
439                     // Customer Row
440                     Row(
441                         modifier = Modifier.fillMaxWidth(),
442                         horizontalArrangement = Arrangement.SpaceBetween,
443                         verticalAlignment = Alignment.CenterVertically
444                     ) {
445                         Text(
446                             text = "Customer: ${sale.customerName ?: "Walk-in Customer"}",
447                             style = MaterialTheme.typography.bodyMedium.copy(fontWeight =
    FontWeight.SemiBold)
448                         )
449                         if (sale.customerId != null) {
450                             TextButton(
451                                 onClick = {
452                                     val cId = sale.customerId
453                                     selectedSaleForDetail = null
454                                     onNavigateToCustomerAccount(cId)
455                                 }
456                             ) {
457                                 Text("View in Accounts", fontSize = 12.sp)
458                             }
459                         }
460                     }
461
462                     HorizontalDivider()
463
464                     // Items Snapshot
465                     Text("Items / Services:", style = MaterialTheme.typography.labelMedium.copy(fontWeight =
     FontWeight.Bold))
466                     Column(verticalArrangement = Arrangement.spacedBy(4.dp)) {
467                         saleLines.forEach { line ->
468                             Row(
469                                 modifier = Modifier.fillMaxWidth(),
470                                 horizontalArrangement = Arrangement.SpaceBetween
471                             ) {
472                                 val name = line.productName ?: line.serviceName ?: "Item"
473                                 val qtyUnit = if (line.lineType == "SERVICE") "${line.quantityBase}
    ${line.unitLabel ?: "job"}"
474                                 else CostCalculator.formatQuantityWithGrouping(line.quantityBase, "pcs")
475                                 Column(modifier = Modifier.weight(1f)) {
476                                     Text(name, style = MaterialTheme.typography.bodySmall, fontWeight =
    FontWeight.Medium)
477                                     Text(
478                                         "$qtyUnit @
    ${CostCalculator.formatPaiseToRupees(line.unitSellingPricePaise)}",
479                                         style = MaterialTheme.typography.labelSmall,
480                                         color = MaterialTheme.colorScheme.onSurfaceVariant
481                                     )
482                                 }
483                                 Text(
484                                     CostCalculator.formatPaiseToRupees(line.lineNetPaise),
485                                     style = MaterialTheme.typography.bodySmall,
486                                     fontWeight = FontWeight.Bold
487                                 )
488                             }
489                         }
490                     }
491
492                     HorizontalDivider()
493
494                     // Financial Summary
495                     Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement =
    Arrangement.SpaceBetween) {
496                         Text("Gross Total:", style = MaterialTheme.typography.bodySmall)
497                         Text(CostCalculator.formatPaiseToRupees(sale.grossTotalPaise), style =
    MaterialTheme.typography.bodySmall)
498                     }
499                     if (sale.discountValuePaise > 0) {
500                         Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement =
```

```
    Arrangement.SpaceBetween) {
501                             Text("Discount:", style = MaterialTheme.typography.bodySmall, color =
    Color(0xFF2E7D32))
502                             Text("-${CostCalculator.formatPaiseToRupees(sale.discountValuePaise)}", style =
    MaterialTheme.typography.bodySmall, color = Color(0xFF2E7D32))
503                         }
504                     }
505                     Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement =
    Arrangement.SpaceBetween) {
506                         Text("Net Payable:", style = MaterialTheme.typography.bodyMedium.copy(fontWeight =
    FontWeight.Bold))
507                         Text(CostCalculator.formatPaiseToRupees(sale.netTotalPaise), style =
    MaterialTheme.typography.bodyMedium.copy(fontWeight = FontWeight.Bold), color =
    MaterialTheme.colorScheme.primary)
508                     }
509                     Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement =
    Arrangement.SpaceBetween) {
510                         Text("Amount Paid (${sale.paymentMethod}):", style =
    MaterialTheme.typography.bodySmall)
511                         Text(CostCalculator.formatPaiseToRupees(sale.amountReceivedPaise), style =
    MaterialTheme.typography.bodySmall)
512                     }
513                     if (sale.remainingCreditPaise > 0) {
514                         Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement =
    Arrangement.SpaceBetween) {
515                             Text("Remaining Due:", style =
    MaterialTheme.typography.bodyMedium.copy(fontWeight = FontWeight.Bold), color = Color(0xFFD32F2F))
516                             Text(CostCalculator.formatPaiseToRupees(sale.remainingCreditPaise), style =
    MaterialTheme.typography.bodyMedium.copy(fontWeight = FontWeight.Bold), color = Color(0xFFD32F2F))
517                         }
518                     }
519                 }
520             },
521             confirmButton = {
522                 Button(
523                     onClick = {
524                         val num = sale.receiptNumber
525                         selectedSaleForDetail = null
526                         onNavigateToReceipt(num)
527                     }
528                 ) {
529                     Icon(Icons.Filled.Print, contentDescription = null, modifier = Modifier.size(16.dp))
530                     Spacer(modifier = Modifier.width(4.dp))
531                     Text("View / Share Receipt")
532                 }
533             },
534             dismissButton = {
535                 OutlinedButton(onClick = { selectedSaleForDetail = null }) {
536                     Text("Close")
537                 }
538             }
539         )
540     }
541
542     // Direct Repayment Dialog for this bill
543     repaymentSaleTarget?.let { sale ->
544         var amountStr by remember { mutableStateOf((sale.remainingCreditPaise / 100.0).toString()) }
545         var paymentMethod by remember { mutableStateOf("CASH") }
546         var notes by remember { mutableStateOf("Repayment for Bill ${sale.receiptNumber}") }
547
548         AlertDialog(
549             onDismissRequest = { repaymentSaleTarget = null },
550             title = { Text("Receive Repayment for ${sale.receiptNumber}") },
551             text = {
552                 Column(verticalArrangement = Arrangement.spacedBy(10.dp)) {
553                     Text(
554                         "Customer: ${sale.customerName ?: "Customer"}",
555                         style = MaterialTheme.typography.bodyMedium.copy(fontWeight = FontWeight.SemiBold)
556                     )
557                     Text(
558                         "Remaining Due on Bill:
    ${CostCalculator.formatPaiseToRupees(sale.remainingCreditPaise)}",
559                         color = Color(0xFFD32F2F),
560                         fontWeight = FontWeight.Bold
561                     )
562
563                     Row(
564                         modifier = Modifier.fillMaxWidth(),
565                         horizontalArrangement = Arrangement.spacedBy(8.dp)
566                     ) {
567                         FilterChip(
568                             selected = paymentMethod == "CASH",
569                             onClick = { paymentMethod = "CASH" },
570                             label = { Text("Cash") },
571                             modifier = Modifier.weight(1f)
572                         )
573                         FilterChip(
574                             selected = paymentMethod == "UPI",
```

```
                            onClick = { paymentMethod = "UPI" },
                            label = { Text("UPI") },
                            modifier = Modifier.weight(1f)
                        )
                        FilterChip(
                            selected = paymentMethod == "BANK",
                            onClick = { paymentMethod = "BANK" },
                            label = { Text("Bank") },
                            modifier = Modifier.weight(1f)
                        )
                    }

                    OutlinedTextField(
                        value = amountStr,
                        onValueChange = { amountStr = it },
                        label = { Text("Repayment Amount (₹) *") },
                        singleLine = true,
                        modifier = Modifier.fillMaxWidth()
                    )

                    OutlinedTextField(
                        value = notes,
                        onValueChange = { notes = it },
                        label = { Text("Notes / Reference") },
                        singleLine = true,
                        modifier = Modifier.fillMaxWidth()
                    )
                }
            },
            confirmButton = {
                Button(
                    onClick = {
                        val amtPaise = CostCalculator.parseRupeesToPaise(amountStr) ?: 0L
                        if (amtPaise > 0L && sale.customerId != null) {
                            viewModel.recordAccountPayment(
                                accountType = "CUSTOMER",
                                accountId = sale.customerId,
                                amountPaise = amtPaise,
                                paymentMethod = paymentMethod,
                                notes = notes.trim().ifBlank { null },
                                receiptRef = sale.receiptNumber,
                                onDone = {
                                    repaymentSaleTarget = null
                                }
                            )
                        }
                    }
                ) {
                    Text("Confirm Repayment")
                }
            },
            dismissButton = {
                OutlinedButton(onClick = { repaymentSaleTarget = null }) {
                    Text("Cancel")
                }
            }
        )
    }
}

@Composable
fun BillHistoryRow(
    sale: SaleEntity,
    onClick: () -> Unit,
    onRepay: () -> Unit,
    onOpenCustomer: () -> Unit,
    modifier: Modifier = Modifier
) {
    Card(
        modifier = modifier
            .fillMaxWidth()
            .clickable(onClick = onClick),
        colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surface),
        shape = RoundedCornerShape(12.dp)
    ) {
        Column(modifier = Modifier.padding(14.dp)) {
            Row(
                modifier = Modifier.fillMaxWidth(),
                horizontalArrangement = Arrangement.SpaceBetween,
                verticalAlignment = Alignment.CenterVertically
            ) {
                Row(verticalAlignment = Alignment.CenterVertically) {
                    Icon(
                        Icons.Filled.Receipt,
                        contentDescription = null,
                        modifier = Modifier.size(18.dp),
                        tint = MaterialTheme.colorScheme.primary
                    )
```

```
                    Spacer(modifier = Modifier.width(6.dp))
                    Text(
                        sale.receiptNumber,
                        style = MaterialTheme.typography.titleSmall.copy(fontWeight = FontWeight.Bold),
                        color = MaterialTheme.colorScheme.onSurface
                    )
                }

                if (sale.isVoided) {
                    Text(
                        "VOIDED",
                        style = MaterialTheme.typography.labelSmall.copy(fontWeight = FontWeight.Bold),
                        color = MaterialTheme.colorScheme.error
                    )
                } else if (sale.remainingCreditPaise > 0) {
                    Card(
                        colors = CardDefaults.cardColors(containerColor = Color(0xFFFFEBEE)),
                        shape = RoundedCornerShape(4.dp)
                    ) {
                        Text(
                            "DUE ${CostCalculator.formatPaiseToRupees(sale.remainingCreditPaise)}",
                            modifier = Modifier.padding(horizontal = 6.dp, vertical = 2.dp),
                            style = MaterialTheme.typography.labelSmall.copy(fontWeight = FontWeight.Bold),
                            color = Color(0xFFC62828)
                        )
                    }
                } else {
                    Card(
                        colors = CardDefaults.cardColors(containerColor = Color(0xFFE8F5E9)),
                        shape = RoundedCornerShape(4.dp)
                    ) {
                        Text(
                            "PAID",
                            modifier = Modifier.padding(horizontal = 6.dp, vertical = 2.dp),
                            style = MaterialTheme.typography.labelSmall.copy(fontWeight = FontWeight.Bold),
                            color = Color(0xFF2E7D32)
                        )
                    }
                }
            }

            Spacer(modifier = Modifier.height(6.dp))

            Row(
                modifier = Modifier.fillMaxWidth(),
                horizontalArrangement = Arrangement.SpaceBetween,
                verticalAlignment = Alignment.CenterVertically
            ) {
                Column {
                    Text(
                        sale.customerName ?: "Walk-in Customer",
                        style = MaterialTheme.typography.bodyMedium.copy(fontWeight = FontWeight.Medium),
                        color = MaterialTheme.colorScheme.onSurface
                    )
                    val dateFormatted = SimpleDateFormat("dd MMM yyyy, hh:mm a",
    Locale.US).format(Date(sale.timestamp))
                    Text(
                        dateFormatted,
                        style = MaterialTheme.typography.bodySmall,
                        color = MaterialTheme.colorScheme.onSurfaceVariant
                    )
                }

                Column(horizontalAlignment = Alignment.End) {
                    Text(
                        CostCalculator.formatPaiseToRupees(sale.netTotalPaise),
                        style = MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.Bold),
                        color = MaterialTheme.colorScheme.primary
                    )
                    Text(
                        "Paid: ${CostCalculator.formatPaiseToRupees(sale.amountReceivedPaise)}",
                        style = MaterialTheme.typography.labelSmall,
                        color = MaterialTheme.colorScheme.onSurfaceVariant
                    )
                }
            }

            if (sale.remainingCreditPaise > 0 && !sale.isVoided && sale.customerId != null) {
                Spacer(modifier = Modifier.height(8.dp))
                Row(
                    modifier = Modifier.fillMaxWidth(),
                    horizontalArrangement = Arrangement.End
                ) {
                    OutlinedButton(
                        onClick = onRepay,
                        shape = RoundedCornerShape(8.dp),
                        modifier = Modifier.height(36.dp)
                    ) {
```

```
750                         Icon(Icons.Filled.Payment, contentDescription = null, modifier =
    Modifier.size(14.dp))
751                         Spacer(modifier = Modifier.width(4.dp))
752                         Text("Record Repayment", fontSize = 12.sp)
753                     }
754                 }
755             }
756         }
757     }
758 }
```

app\src\main\java\com\alieducational\app\ui\billing\BillingScreen.kt

```
package com.alieducational.app.ui.billing

import androidx.compose.foundation.Image
import androidx.compose.foundation.background
import androidx.compose.foundation.border
import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.LazyRow
import androidx.compose.foundation.lazy.items
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Add
import androidx.compose.material.icons.filled.CameraAlt
import androidx.compose.material.icons.filled.DeleteOutline
import androidx.compose.material.icons.filled.FiberManualRecord
import androidx.compose.material.icons.filled.ContentCopy
import androidx.compose.material.icons.filled.Label
import androidx.compose.material.icons.filled.PauseCircleOutline
import androidx.compose.material.icons.filled.PlayCircleOutline
import androidx.compose.material.icons.filled.Print
import androidx.compose.material.icons.filled.ReceiptLong
import androidx.compose.material.icons.filled.Remove
import androidx.compose.material.icons.filled.Search
import androidx.compose.material.icons.filled.Star
import androidx.compose.material.icons.filled.Sync
import androidx.compose.material.icons.outlined.StarOutline
import androidx.compose.material3.AlertDialog
import androidx.compose.material3.Button
import androidx.compose.material3.ButtonDefaults
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.FilterChip
import androidx.compose.material3.FilterChipDefaults
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedButton
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.OutlinedTextFieldDefaults
import androidx.compose.material3.Surface
import androidx.compose.material3.Tab
import androidx.compose.material3.TabRow
import androidx.compose.material3.Text
import androidx.compose.material3.TextButton
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.ui.text.input.KeyboardType
import com.alieducational.app.data.local.entity.ServiceDefinitionEntity
import androidx.compose.runtime.Composable
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.layout.ContentScale
import androidx.compose.ui.res.painterResource
import androidx.compose.ui.res.stringResource
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.alieducational.app.R
import com.alieducational.app.domain.calculation.CostCalculator
import com.alieducational.app.ui.AliViewModel

@Composable
fun BillingScreen(
    viewModel: AliViewModel,
    onNavigateToScanner: () -> Unit,
```

```
    onNavigateToReview: () -> Unit,
    onNavigateBack: () -> Unit = {},
    onNavigateToBillHistory: () -> Unit = {},
    modifier: Modifier = Modifier
) {
    androidx.activity.compose.BackHandler {
        onNavigateBack()
    }
    val isDarkTheme by viewModel.isDarkTheme.collectAsState()
    val billingState by viewModel.billingState.collectAsState()
    val allProducts by viewModel.allProducts.collectAsState()
    val favoriteProducts by viewModel.favoriteProducts.collectAsState()
    val allCategories by viewModel.allCategories.collectAsState()
    val allHeldBills by viewModel.allHeldBills.collectAsState()
    val activeServices by viewModel.activeServices.collectAsState()
    val deviceRole by viewModel.deviceRole.collectAsState()
    val pendingSyncCount by viewModel.pendingSyncCount.collectAsState()
    val pairedOwnerHost by viewModel.pairedOwnerHost.collectAsState()

    var activeTab by remember { mutableStateOf("PRODUCTS") } // PRODUCTS or SERVICES
    var showEnterServiceDialog by remember { mutableStateOf(false) }
    var selectedServiceForJob by remember { mutableStateOf<ServiceDefinitionEntity?>(null) }
    var showAddServiceDialog by remember { mutableStateOf(false) }

    var searchQuery by remember { mutableStateOf("") }
    var selectedFilter by remember { mutableStateOf("ALL") } // ALL, FAVOURITES, CATEGORY, RECENT
    var selectedCategory by remember { mutableStateOf<String?>(null) }
    var showHeldBillsDialog by remember { mutableStateOf(false) }

    // Filter products
    val displayedProducts = remember(allProducts, favoriteProducts, searchQuery, selectedFilter,
selectedCategory) {
        var list = when (selectedFilter) {
            "FAVOURITES" -> favoriteProducts
            "CATEGORY" -> if (selectedCategory != null) allProducts.filter { it.category == selectedCategory
 } else allProducts
            else -> allProducts
        }
        if (searchQuery.isNotBlank()) {
            val q = searchQuery.trim().lowercase()
            list = list.filter {
                it.name.lowercase().contains(q) ||
                        (it.sku?.lowercase()?.contains(q) == true) ||
                        (it.brand?.lowercase()?.contains(q) == true) ||
                        (it.urduName?.contains(q) == true)
            }
        }
        list
    }

    Surface(
        modifier = modifier.fillMaxSize(),
        color = MaterialTheme.colorScheme.background
    ) {
        Box(modifier = Modifier.fillMaxSize()) {
            LazyColumn(
                modifier = Modifier
                    .fillMaxSize()
                    .padding(horizontal = 16.dp),
                verticalArrangement = Arrangement.spacedBy(14.dp)
            ) {
                // Top Shop Branding Bar
                item {
                    Spacer(modifier = Modifier.height(6.dp))
                    Row(
                        modifier = Modifier.fillMaxWidth(),
                        horizontalArrangement = Arrangement.SpaceBetween,
                        verticalAlignment = Alignment.CenterVertically
                    ) {
                        Row(verticalAlignment = Alignment.CenterVertically) {
                            val logoRes = if (isDarkTheme) R.drawable.ali_logo_dark else
R.drawable.ali_logo_light
                            Image(
                                painter = painterResource(id = logoRes),
                                contentDescription = "Ali Educational Logo",
                                modifier = Modifier
                                    .height(42.dp)
                                    .width(80.dp),
                                contentScale = ContentScale.Fit
                            )
                            Spacer(modifier = Modifier.width(10.dp))
                            Column {
                                Text(
                                    text = "Ali Educational",
                                    style = MaterialTheme.typography.titleLarge.copy(
                                        fontWeight = FontWeight.Bold,
                                        fontSize = 19.sp
                                    ),
```

```
                                    color = MaterialTheme.colorScheme.onBackground
                                )
                                Text(
                                    text = stringResource(R.string.billing_header_subtitle),
                                    style = MaterialTheme.typography.bodySmall,
                                    color = MaterialTheme.colorScheme.onSurfaceVariant
                                )
                            }
                        }

                        // History & Held Bills Action Buttons
                        Row(verticalAlignment = Alignment.CenterVertically) {
                            IconButton(
                                onClick = onNavigateToBillHistory,
                                modifier = Modifier.size(36.dp)
                            ) {
                                Icon(
                                    Icons.Filled.ReceiptLong,
                                    contentDescription = "Bill History",
                                    tint = MaterialTheme.colorScheme.primary,
                                    modifier = Modifier.size(22.dp)
                                )
                            }
                            Spacer(modifier = Modifier.width(4.dp))
                            if (allHeldBills.isNotEmpty()) {
                                Card(
                                    colors = CardDefaults.cardColors(containerColor =
MaterialTheme.colorScheme.surface),
                                    shape = RoundedCornerShape(8.dp),
                                    modifier = Modifier.clickable { showHeldBillsDialog = true }
                                ) {
                                    Row(
                                        modifier = Modifier.padding(horizontal = 8.dp, vertical = 6.dp),
                                        verticalAlignment = Alignment.CenterVertically
                                    ) {
                                        Icon(
                                            Icons.Filled.PlayCircleOutline,
                                            contentDescription = "Held Bills",
                                            tint = MaterialTheme.colorScheme.primary,
                                            modifier = Modifier.size(18.dp)
                                        )
                                        Spacer(modifier = Modifier.width(4.dp))
                                        Text(
                                            text = "${allHeldBills.size}
${stringResource(R.string.billing_held_tag)}",
                                            fontSize = 12.sp,
                                            fontWeight = FontWeight.Bold,
                                            color = MaterialTheme.colorScheme.primary
                                        )
                                    }
                                }
                            }
                            if (billingState.items.isNotEmpty()) {
                                IconButton(onClick = { viewModel.holdCurrentBill() }) {
                                    Icon(
                                        Icons.Filled.PauseCircleOutline,
                                        contentDescription = "Hold bill",
                                        tint = MaterialTheme.colorScheme.onSurfaceVariant
                                    )
                                }
                            }
                        }
                    }
                }

                // Two-Phone Sync Status Card (for Billing Phone)
                if (deviceRole == "BILLING") {
                    item {
                        Card(
                            modifier = Modifier.fillMaxWidth(),
                            colors = CardDefaults.cardColors(
                                containerColor = if (pendingSyncCount > 0) Color(0xFF332A15) else
MaterialTheme.colorScheme.surfaceVariant
                            ),
                            shape = RoundedCornerShape(10.dp)
                        ) {
                            Row(
                                modifier = Modifier
                                    .fillMaxWidth()
                                    .padding(horizontal = 12.dp, vertical = 8.dp),
                                horizontalArrangement = Arrangement.SpaceBetween,
                                verticalAlignment = Alignment.CenterVertically
                            ) {
                                Row(verticalAlignment = Alignment.CenterVertically) {
                                    Icon(
                                        Icons.Filled.Sync,
                                        contentDescription = null,
                                        tint = MaterialTheme.colorScheme.primary,
```

```
                                        modifier = Modifier.size(18.dp)
                                    )
                                    Spacer(modifier = Modifier.width(8.dp))
                                    Column {
                                        Text(
                                            text = "Billing Device Mode",
                                            fontSize = 12.sp,
                                            fontWeight = FontWeight.Bold,
                                            color = MaterialTheme.colorScheme.onSurface
                                        )
                                        Text(
                                            text = if (pendingSyncCount > 0) "$pendingSyncCount pending sync" else "All bills synced",
                                            fontSize = 11.sp,
                                            color = if (pendingSyncCount > 0) Color(0xFFE4B765) else MaterialTheme.colorScheme.onSurfaceVariant
                                        )
                                    }
                                }
                                Button(
                                    onClick = {
                                        if (pairedOwnerHost.isNotBlank()) {
                                            viewModel.syncWithMainPhone(pairedOwnerHost)
                                        }
                                    },
                                    shape = RoundedCornerShape(8.dp),
                                    colors = ButtonDefaults.buttonColors(
                                        containerColor = MaterialTheme.colorScheme.primary,
                                        contentColor = MaterialTheme.colorScheme.onPrimary
                                    ),
                                    contentPadding = ButtonDefaults.TextButtonContentPadding
                                ) {
                                    Text("Sync Now", fontSize = 11.sp, fontWeight = FontWeight.Bold)
                                }
                            }
                        }
                    }
                }

                // Status Badge: Saved on this phone - Offline ready
                item {
                    Row(verticalAlignment = Alignment.CenterVertically) {
                        Icon(
                            imageVector = Icons.Filled.FiberManualRecord,
                            contentDescription = null,
                            tint = Color(0xFF75B890),
                            modifier = Modifier.size(10.dp)
                        )
                        Spacer(modifier = Modifier.width(6.dp))
                        Text(
                            text = stringResource(R.string.billing_offline_ready),
                            style = MaterialTheme.typography.bodySmall,
                            color = MaterialTheme.colorScheme.onSurfaceVariant
                        )
                    }
                }

                // Section Title: New bill
                item {
                    Column {
                        Text(
                            text = stringResource(R.string.billing_new_bill_title),
                            style = MaterialTheme.typography.headlineMedium.copy(fontWeight = FontWeight.Bold),
                            color = MaterialTheme.colorScheme.onBackground
                        )
                        Text(
                            text = stringResource(R.string.billing_new_bill_desc),
                            style = MaterialTheme.typography.bodyMedium,
                            color = MaterialTheme.colorScheme.onSurfaceVariant
                        )
                    }
                }

                // Tab Switcher: Products vs Services
                item {
                    TabRow(
                        selectedTabIndex = if (activeTab == "PRODUCTS") 0 else 1,
                        containerColor = MaterialTheme.colorScheme.surface,
                        contentColor = MaterialTheme.colorScheme.primary
                    ) {
                        Tab(
                            selected = activeTab == "PRODUCTS",
                            onClick = { activeTab = "PRODUCTS" },
                            text = { Text("Products", fontWeight = FontWeight.Bold) }
                        )
                        Tab(
                            selected = activeTab == "SERVICES",
```

```
                            onClick = { activeTab = "SERVICES" },
                            text = { Text("Services", fontWeight = FontWeight.Bold) }
                        )
                    }
                }

                if (activeTab == "PRODUCTS") {
                    // Scan with camera button (Primary Gold Banner Button)
                    item {
                        Button(
                            onClick = onNavigateToScanner,
                            modifier = Modifier
                                .fillMaxWidth()
                                .height(54.dp),
                            shape = RoundedCornerShape(12.dp),
                            colors = ButtonDefaults.buttonColors(
                                containerColor = MaterialTheme.colorScheme.primary,
                                contentColor = MaterialTheme.colorScheme.onPrimary
                            )
                        ) {
                            Icon(
                                imageVector = Icons.Filled.CameraAlt,
                                contentDescription = null,
                                modifier = Modifier.size(22.dp)
                            )
                            Spacer(modifier = Modifier.width(10.dp))
                            Text(
                                text = stringResource(R.string.billing_scan_with_camera),
                                style = MaterialTheme.typography.titleMedium.copy(fontWeight =
FontWeight.Bold)
                            )
                        }
                    }

                    // Product Search Bar
                    item {
                        OutlinedTextField(
                            value = searchQuery,
                            onValueChange = { searchQuery = it },
                            modifier = Modifier.fillMaxWidth(),
                            placeholder = { Text(stringResource(R.string.billing_search_placeholder)) },
                            leadingIcon = {
                                Icon(Icons.Filled.Search, contentDescription = null, tint =
MaterialTheme.colorScheme.onSurfaceVariant)
                            },
                            singleLine = true,
                            shape = RoundedCornerShape(12.dp),
                            colors = OutlinedTextFieldDefaults.colors(
                                focusedContainerColor = MaterialTheme.colorScheme.surface,
                                unfocusedContainerColor = MaterialTheme.colorScheme.surface,
                                focusedBorderColor = MaterialTheme.colorScheme.primary,
                                unfocusedBorderColor = MaterialTheme.colorScheme.outline
                            )
                        )
                    }

                    // Filter Chips: Favourites, Categories, Recent
                    item {
                        LazyRow(
                            horizontalArrangement = Arrangement.spacedBy(8.dp),
                            modifier = Modifier.fillMaxWidth()
                        ) {
                            item {
                                FilterChip(
                                    selected = selectedFilter == "FAVOURITES",
                                    onClick = {
                                        selectedFilter = if (selectedFilter == "FAVOURITES") "ALL" else
"FAVOURITES"
                                    },
                                    label = { Text(stringResource(R.string.billing_filter_favourites)) },
                                    leadingIcon = {
                                        Icon(
                                            if (selectedFilter == "FAVOURITES") Icons.Filled.Star else
Icons.Outlined.StarOutline,
                                            contentDescription = null,
                                            modifier = Modifier.size(16.dp)
                                        )
                                    },
                                    colors = FilterChipDefaults.filterChipColors(
                                        selectedContainerColor = MaterialTheme.colorScheme.primaryContainer,
                                        selectedLabelColor = MaterialTheme.colorScheme.onPrimaryContainer
                                    )
                                )
                            }

                            items(allCategories) { category ->
                                FilterChip(
                                    selected = selectedFilter == "CATEGORY" && selectedCategory == category,
```

```
                                    onClick = {
                                        if (selectedFilter == "CATEGORY" && selectedCategory == category) {
                                            selectedFilter = "ALL"
                                            selectedCategory = null
                                        } else {
                                            selectedFilter = "CATEGORY"
                                            selectedCategory = category
                                        }
                                    },
                                    label = { Text(category) },
                                    colors = FilterChipDefaults.filterChipColors(
                                        selectedContainerColor = MaterialTheme.colorScheme.primaryContainer,
                                        selectedLabelColor = MaterialTheme.colorScheme.onPrimaryContainer
                                    )
                                )
                            }
                        }
                    }

                    // Quick Add Product Cards (2-column layout or list)
                    item {
                        Row(
                            modifier = Modifier.fillMaxWidth(),
                            horizontalArrangement = Arrangement.spacedBy(10.dp)
                        ) {
                            val firstTwo = displayedProducts.take(2)
                            firstTwo.forEach { prod ->
                                Card(
                                    modifier = Modifier
                                        .weight(1f)
                                        .clickable { viewModel.addItemToBill(prod, "PIECE", 1) },
                                    shape = RoundedCornerShape(12.dp),
                                    colors = CardDefaults.cardColors(containerColor =
MaterialTheme.colorScheme.surface)
                                ) {
                                    Column(modifier = Modifier.padding(12.dp)) {
                                        Text(
                                            text = prod.name,
                                            style = MaterialTheme.typography.bodyMedium.copy(fontWeight =
FontWeight.Bold),
                                            color = MaterialTheme.colorScheme.onSurface,
                                            maxLines = 1
                                        )
                                        val priceStr =
CostCalculator.formatPaiseToRupees(prod.sellingPricePiecePaise)
                                        Text(
                                            text = "One piece · $priceStr",
                                            style = MaterialTheme.typography.bodySmall,
                                            color = MaterialTheme.colorScheme.onSurfaceVariant
                                        )
                                        Spacer(modifier = Modifier.height(8.dp))
                                        Row(
                                            verticalAlignment = Alignment.CenterVertically,
                                            modifier = Modifier
                                                .clip(RoundedCornerShape(6.dp))
                                                .background(MaterialTheme.colorScheme.surfaceVariant)
                                                .padding(horizontal = 8.dp, vertical = 4.dp)
                                        ) {
                                            Icon(
                                                Icons.Filled.Add,
                                                contentDescription = null,
                                                tint = MaterialTheme.colorScheme.primary,
                                                modifier = Modifier.size(14.dp)
                                            )
                                            Spacer(modifier = Modifier.width(4.dp))
                                            Text(
                                                text = stringResource(R.string.billing_add_btn),
                                                fontSize = 12.sp,
                                                fontWeight = FontWeight.SemiBold,
                                                color = MaterialTheme.colorScheme.primary
                                            )
                                        }
                                    }
                                }
                            }
                        }
                    }
                } else {
                    // SERVICES TAB
                    item {
                        Row(
                            modifier = Modifier.fillMaxWidth(),
                            horizontalArrangement = Arrangement.SpaceBetween,
                            verticalAlignment = Alignment.CenterVertically
                        ) {
                            Text(
                                text = "Services",
                                style = MaterialTheme.typography.titleMedium.copy(fontWeight =
FontWeight.Bold),
```

```
                                color = MaterialTheme.colorScheme.onBackground
                            )
                            TextButton(onClick = { showAddServiceDialog = true }) {
                                Icon(Icons.Filled.Add, contentDescription = null, modifier =
Modifier.size(16.dp))
                                Spacer(modifier = Modifier.width(4.dp))
                                Text("Add Service", fontWeight = FontWeight.Bold)
                            }
                        }
                    }

                    items(activeServices) { service ->
                        Card(
                            modifier = Modifier
                                .fillMaxWidth()
                                .clickable {
                                    selectedServiceForJob = service
                                    showEnterServiceDialog = true
                                },
                            shape = RoundedCornerShape(12.dp),
                            colors = CardDefaults.cardColors(containerColor =
MaterialTheme.colorScheme.surface)
                        ) {
                            Row(
                                modifier = Modifier
                                    .fillMaxWidth()
                                    .padding(14.dp),
                                horizontalArrangement = Arrangement.SpaceBetween,
                                verticalAlignment = Alignment.CenterVertically
                            ) {
                                Row(verticalAlignment = Alignment.CenterVertically) {
                                    val icon = when {
                                        service.name.contains("Print", ignoreCase = true) ->
Icons.Filled.Print
                                        service.name.contains("Copy", ignoreCase = true) ->
Icons.Filled.ContentCopy
                                        service.name.contains("Sticker", ignoreCase = true) ->
Icons.Filled.Label
                                        else -> Icons.Filled.Print
                                    }
                                    Box(
                                        modifier = Modifier
                                            .size(40.dp)
                                            .clip(RoundedCornerShape(8.dp))
                                            .background(MaterialTheme.colorScheme.primaryContainer),
                                        contentAlignment = Alignment.Center
                                    ) {
                                        Icon(icon, contentDescription = null, tint =
MaterialTheme.colorScheme.onPrimaryContainer)
                                    }
                                    Spacer(modifier = Modifier.width(12.dp))
                                    Column {
                                        Text(
                                            text = service.name,
                                            style = MaterialTheme.typography.titleMedium.copy(fontWeight =
FontWeight.Bold),
                                            color = MaterialTheme.colorScheme.onSurface
                                        )
                                        Text(
                                            text = "Unit: ${service.unitLabel} · Variable job rate",
                                            style = MaterialTheme.typography.bodySmall,
                                            color = MaterialTheme.colorScheme.onSurfaceVariant
                                        )
                                    }
                                }
                                Icon(Icons.Filled.Add, contentDescription = "Add Job", tint =
MaterialTheme.colorScheme.primary)
                            }
                        }
                    }
                }

                // Section: In this bill
                if (billingState.items.isNotEmpty()) {
                    item {
                        Spacer(modifier = Modifier.height(4.dp))
                        Text(
                            text = stringResource(R.string.billing_in_this_bill_title),
                            style = MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.Bold),
                            color = MaterialTheme.colorScheme.onBackground
                        )
                    }

                    items(billingState.items, key = { it.lineId }) { item ->
                        Card(
                            modifier = Modifier.fillMaxWidth(),
                            shape = RoundedCornerShape(12.dp),
                            colors = CardDefaults.cardColors(containerColor =
```

```
    MaterialTheme.colorScheme.surface)
                        ) {
                            Column(
                                modifier = Modifier
                                    .fillMaxWidth()
                                    .padding(12.dp)
                            ) {
                                Row(
                                    modifier = Modifier.fillMaxWidth(),
                                    horizontalArrangement = Arrangement.SpaceBetween,
                                    verticalAlignment = Alignment.CenterVertically
                                ) {
                                    Column(modifier = Modifier.weight(1f)) {
                                        Text(
                                            text = item.productName,
                                            style = MaterialTheme.typography.bodyMedium.copy(fontWeight =
    FontWeight.SemiBold),
                                            color = MaterialTheme.colorScheme.onSurface
                                        )
                                        val lineSubtitle = if (item.lineType == "SERVICE") {
                                            val opt = if (!item.serviceOption.isNullOrBlank()) "
    (${item.serviceOption})" else ""
                                            val unit = item.unitLabel ?: "unit"
                                            val unitPrice =
    CostCalculator.formatPaiseToRupees(item.unitPricePaise)
                                            "${item.quantity} $unit$opt · $unitPrice each"
                                        } else {
                                            val unitLabel = if (item.unitType == "PACK") "pack" else "piece"
                                            val unitPrice =
    CostCalculator.formatPaiseToRupees(item.unitPricePaise)
                                            "${item.quantity} $unitLabel(s) · $unitPrice each"
                                        }
                                        Text(
                                            text = lineSubtitle,
                                            style = MaterialTheme.typography.bodySmall,
                                            color = MaterialTheme.colorScheme.onSurfaceVariant
                                        )
                                    }

                                    Row(verticalAlignment = Alignment.CenterVertically) {
                                        Text(
                                            text = CostCalculator.formatPaiseToRupees(item.lineGrossPaise),
                                            style = MaterialTheme.typography.titleMedium.copy(fontWeight =
    FontWeight.Bold),
                                            color = MaterialTheme.colorScheme.primary
                                        )
                                        Spacer(modifier = Modifier.width(4.dp))
                                        IconButton(
                                            onClick = { viewModel.removeItemFromBill(item.lineId) },
                                            modifier = Modifier.size(32.dp)
                                        ) {
                                            Icon(
                                                imageVector = Icons.Filled.DeleteOutline,
                                                contentDescription = "Remove item",
                                                tint = MaterialTheme.colorScheme.error,
                                                modifier = Modifier.size(18.dp)
                                            )
                                        }
                                    }
                                }

                                Spacer(modifier = Modifier.height(4.dp))

                                Row(
                                    modifier = Modifier.fillMaxWidth(),
                                    horizontalArrangement = Arrangement.End,
                                    verticalAlignment = Alignment.CenterVertically
                                ) {
                                    IconButton(
                                        onClick = { viewModel.updateItemQuantity(item.lineId, item.quantity
    - 1) },
                                        modifier = Modifier.size(28.dp)
                                    ) {
                                        Icon(
                                            imageVector = Icons.Filled.Remove,
                                            contentDescription = "Decrease",
                                            tint = MaterialTheme.colorScheme.onSurfaceVariant,
                                            modifier = Modifier.size(16.dp)
                                        )
                                    }
                                    Text(
                                        text = "${item.quantity}",
                                        style = MaterialTheme.typography.bodyMedium.copy(fontWeight =
    FontWeight.Bold),
                                        modifier = Modifier.padding(horizontal = 8.dp)
                                    )
                                    IconButton(
                                        onClick = { viewModel.updateItemQuantity(item.lineId, item.quantity
```

```
    + 1) },
                                        modifier = Modifier.size(28.dp)
                                    ) {
                                        Icon(
                                            imageVector = Icons.Filled.Add,
                                            contentDescription = "Increase",
                                            tint = MaterialTheme.colorScheme.primary,
                                            modifier = Modifier.size(16.dp)
                                        )
                                    }
                                }
                            }
                        }
                    }

                    // Space for bottom sticky bar and tabs
                    item {
                        Spacer(modifier = Modifier.height(90.dp))
                    }
                }

                // Sticky Bottom Action Pill: Review X items · ₹... →
                if (billingState.items.isNotEmpty()) {
                    Box(
                        modifier = Modifier
                            .align(Alignment.BottomCenter)
                            .padding(bottom = 12.dp, start = 16.dp, end = 16.dp)
                    ) {
                        val count = billingState.totalItemCount
                        val totalStr = CostCalculator.formatPaiseToRupees(billingState.grossTotalPaise)
                        Button(
                            onClick = onNavigateToReview,
                            modifier = Modifier
                                .fillMaxWidth()
                                .height(52.dp),
                            shape = RoundedCornerShape(26.dp),
                            colors = ButtonDefaults.buttonColors(
                                containerColor = MaterialTheme.colorScheme.primary,
                                contentColor = MaterialTheme.colorScheme.onPrimary
                            )
                        ) {
                            Text(
                                text = "${stringResource(R.string.billing_review_action, count)} · $totalStr →",
                                style = MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.Bold)
                            )
                        }
                    }
                }
            }
        }

        if (showHeldBillsDialog) {
            HeldBillsDialog(
                viewModel = viewModel,
                onDismiss = { showHeldBillsDialog = false }
            )
        }

        if (showEnterServiceDialog && selectedServiceForJob != null) {
            EnterServiceJobDialog(
                service = selectedServiceForJob!!,
                onDismiss = {
                    showEnterServiceDialog = false
                    selectedServiceForJob = null
                },
                onAddJob = { option, quantity, ratePaise ->
                    viewModel.addServiceToBill(
                        serviceName = selectedServiceForJob!!.name,
                        option = option,
                        unitLabel = selectedServiceForJob!!.unitLabel,
                        quantity = quantity,
                        unitRatePaise = ratePaise,
                        serviceId = selectedServiceForJob!!.id
                    )
                    showEnterServiceDialog = false
                    selectedServiceForJob = null
                }
            )
        }

        if (showAddServiceDialog) {
            AddReusableServiceDialog(
                onDismiss = { showAddServiceDialog = false },
                onSave = { name, unitLabel ->
                    viewModel.addCustomService(name, unitLabel)
                    showAddServiceDialog = false
                }
```

```
        )
    }
}

@Composable
fun EnterServiceJobDialog(
    service: ServiceDefinitionEntity,
    onDismiss: () -> Unit,
    onAddJob: (option: String?, quantity: Int, ratePaise: Long) -> Unit
) {
    val isPrinting = service.name.contains("Print", ignoreCase = true)
    var selectedOption by remember { mutableStateOf(if (isPrinting) "A4 B&W" else null) }
    var quantityText by remember { mutableStateOf("1") }
    var rateText by remember { mutableStateOf("") } // Starts BLANK per requirement

    val qty = quantityText.toIntOrNull() ?: 0
    val rateDouble = rateText.toDoubleOrNull() ?: 0.0
    val totalAmount = qty * rateDouble

    AlertDialog(
        onDismissRequest = onDismiss,
        title = { Text("Add ${service.name}", fontWeight = FontWeight.Bold) },
        text = {
            Column(verticalArrangement = Arrangement.spacedBy(12.dp)) {
                if (isPrinting) {
                    Text("Print Mode", style = MaterialTheme.typography.labelMedium, fontWeight =
    FontWeight.Bold)
                    Row(
                        modifier = Modifier.fillMaxWidth(),
                        horizontalArrangement = Arrangement.spacedBy(8.dp)
                    ) {
                        FilterChip(
                            selected = selectedOption == "A4 B&W",
                            onClick = {
                                if (selectedOption != "A4 B&W") {
                                    selectedOption = "A4 B&W"
                                    rateText = "" // Reset rate on toggle per requirement
                                }
                            },
                            label = { Text("A4 B&W") },
                            modifier = Modifier.weight(1f)
                        )
                        FilterChip(
                            selected = selectedOption == "A4 Colour",
                            onClick = {
                                if (selectedOption != "A4 Colour") {
                                    selectedOption = "A4 Colour"
                                    rateText = "" // Reset rate on toggle per requirement
                                }
                            },
                            label = { Text("A4 Colour") },
                            modifier = Modifier.weight(1f)
                        )
                    }
                }

                OutlinedTextField(
                    value = quantityText,
                    onValueChange = { if (it.all { ch -> ch.isDigit() }) quantityText = it },
                    label = { Text("Quantity (${service.unitLabel})") },
                    keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Number),
                    modifier = Modifier.fillMaxWidth(),
                    shape = RoundedCornerShape(10.dp)
                )

                OutlinedTextField(
                    value = rateText,
                    onValueChange = { rateText = it },
                    label = { Text("Rate per ${service.unitLabel} (₹) *") },
                    placeholder = { Text("Enter rate for this job") },
                    keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Decimal),
                    modifier = Modifier.fillMaxWidth(),
                    shape = RoundedCornerShape(10.dp)
                )

                if (qty > 0 && rateDouble > 0) {
                    Card(
                        colors = CardDefaults.cardColors(containerColor =
    MaterialTheme.colorScheme.surfaceVariant),
                        shape = RoundedCornerShape(8.dp),
                        modifier = Modifier.fillMaxWidth()
                    ) {
                        Row(
                            modifier = Modifier
                                .fillMaxWidth()
                                .padding(12.dp),
                            horizontalArrangement = Arrangement.SpaceBetween,
                            verticalAlignment = Alignment.CenterVertically
```

```
                        ) {
                            Text("Job Total:", fontWeight = FontWeight.SemiBold)
                            Text("₹${String.format(java.util.Locale.ENGLISH, "%.2f", totalAmount)}",
    fontWeight = FontWeight.Bold, color = MaterialTheme.colorScheme.primary)
                        }
                    }
                }
            }
        },
        confirmButton = {
            Button(
                onClick = {
                    val ratePaise = (rateDouble * 100).toLong()
                    if (qty > 0 && ratePaise > 0) {
                        onAddJob(selectedOption, qty, ratePaise)
                    }
                },
                enabled = qty > 0 && rateDouble > 0,
                colors = ButtonDefaults.buttonColors(containerColor = MaterialTheme.colorScheme.primary)
            ) {
                Text("Add to Bill")
            }
        },
        dismissButton = {
            TextButton(onClick = onDismiss) {
                Text("Cancel")
            }
        }
    )
}

@Composable
fun AddReusableServiceDialog(
    onDismiss: () -> Unit,
    onSave: (name: String, unitLabel: String) -> Unit
) {
    var name by remember { mutableStateOf("") }
    var unitLabel by remember { mutableStateOf("pages") }

    AlertDialog(
        onDismissRequest = onDismiss,
        title = { Text("Add Reusable Service", fontWeight = FontWeight.Bold) },
        text = {
            Column(verticalArrangement = Arrangement.spacedBy(12.dp)) {
                OutlinedTextField(
                    value = name,
                    onValueChange = { name = it },
                    label = { Text("Service Name * (e.g. Lamination, Spiral Binding)") },
                    modifier = Modifier.fillMaxWidth(),
                    shape = RoundedCornerShape(10.dp)
                )
                OutlinedTextField(
                    value = unitLabel,
                    onValueChange = { unitLabel = it },
                    label = { Text("Unit Label * (e.g. sheets, books, items)") },
                    modifier = Modifier.fillMaxWidth(),
                    shape = RoundedCornerShape(10.dp)
                )
            }
        },
        confirmButton = {
            Button(
                onClick = {
                    if (name.isNotBlank() && unitLabel.isNotBlank()) {
                        onSave(name.trim(), unitLabel.trim())
                    }
                },
                enabled = name.isNotBlank() && unitLabel.isNotBlank(),
                colors = ButtonDefaults.buttonColors(containerColor = MaterialTheme.colorScheme.primary)
            ) {
                Text("Save Service")
            }
        },
        dismissButton = {
            TextButton(onClick = onDismiss) {
                Text("Cancel")
            }
        }
    )
}
```

app\src\main\java\com\alieducational\app\ui\billing\BillReceiptScreen.kt

```
package com.alieducational.app.ui.billing

import android.content.Intent
import android.widget.Toast
import kotlinx.coroutines.launch
import androidx.compose.foundation.background
import androidx.compose.foundation.border
import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.shape.CircleShape
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Cancel
import androidx.compose.material.icons.filled.Check
import androidx.compose.material.icons.filled.Home
import androidx.compose.material.icons.filled.PointOfSale
import androidx.compose.material.icons.filled.Print
import androidx.compose.material.icons.filled.RadioButtonChecked
import androidx.compose.material.icons.filled.RadioButtonUnchecked
import androidx.compose.material.icons.filled.Share
import androidx.compose.material.icons.filled.Warning
import androidx.compose.material3.AlertDialog
import androidx.compose.material3.Button
import androidx.compose.material3.ButtonDefaults
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.Icon
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedButton
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.material3.TextButton
import androidx.compose.runtime.Composable
import androidx.compose.runtime.LaunchedEffect
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.res.stringResource
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.alieducational.app.R
import com.alieducational.app.data.local.entity.SaleEntity
import com.alieducational.app.data.local.entity.SaleLineEntity
import com.alieducational.app.domain.calculation.CostCalculator
import com.alieducational.app.ui.AliViewModel
import java.text.SimpleDateFormat
import java.util.Date
import java.util.Locale

@Composable
fun BillReceiptScreen(
    receiptNumber: String,
    viewModel: AliViewModel,
    onNewBill: () -> Unit,
    onDone: () -> Unit,
    modifier: Modifier = Modifier
) {
    val context = LocalContext.current
    val shopName by viewModel.shopName.collectAsState()
    val billingState by viewModel.billingState.collectAsState()
    var currentSale by remember { mutableStateOf(billingState.lastSavedSale) }
    var saleLines by remember { mutableStateOf<List<SaleLineEntity>>(emptyList()) }
```

```
83
84      LaunchedEffect(currentSale?.id, receiptNumber) {
85          val saleId = currentSale?.id
86          if (saleId != null) {
87              saleLines = viewModel.getSaleLines(saleId)
88          }
89      }
90
91      var showVoidDialog by remember { mutableStateOf(false) }
92      var stockCondition by remember { mutableStateOf("SELLABLE") }
93      var voidReason by remember { mutableStateOf("") }
94      var isVoiding by remember { mutableStateOf(false) }
95
96      val dateFormat = remember { SimpleDateFormat("dd MMMM yyyy, hh:mm a", Locale.getDefault()) }
97      val formattedDate = remember { dateFormat.format(Date()) }
98
99      val isVoided = currentSale?.isVoided == true
100     val paymentMethod = currentSale?.paymentMethod ?: billingState.paymentMethod
101     val paymentMethodLabel = when (paymentMethod) {
102         "CASH" -> stringResource(R.string.payment_cash)
103         "UPI" -> stringResource(R.string.payment_upi)
104         "CREDIT" -> stringResource(R.string.payment_credit)
105         else -> paymentMethod
106     }
107     val memoTag = when {
108         isVoided -> "VOIDED MEMO"
109         paymentMethod == "CREDIT" -> "CREDIT MEMO"
110         paymentMethod == "UPI" -> "UPI MEMO"
111         else -> "CASH MEMO"
112     }
113
114     val totalPaise = currentSale?.netTotalPaise ?: billingState.netTotalPaise
115     val discountPaise = if (currentSale != null) currentSale!!.grossTotalPaise - currentSale!!.netTotalPaise
    else billingState.actualDiscountPaise
116     val grossPaise = totalPaise + discountPaise
117     val changeGivenPaise = currentSale?.changeGivenPaise ?: 0L
118
119     Surface(
120         modifier = modifier.fillMaxSize(),
121         color = MaterialTheme.colorScheme.background
122     ) {
123         LazyColumn(
124             modifier = Modifier
125                 .fillMaxSize()
126                 .padding(16.dp),
127             horizontalAlignment = Alignment.CenterHorizontally,
128             verticalArrangement = Arrangement.spacedBy(16.dp)
129         ) {
130             item {
131                 Spacer(modifier = Modifier.height(16.dp))
132
133                 val isVoided = currentSale?.isVoided == true
134
135                 // Success or Voided Badge
136                 Box(
137                     modifier = Modifier
138                         .size(64.dp)
139                         .clip(CircleShape)
140                         .background(if (isVoided) Color(0xFFE57373) else MaterialTheme.colorScheme.primary),
141                     contentAlignment = Alignment.Center
142                 ) {
143                     Icon(
144                         imageVector = if (isVoided) Icons.Filled.Cancel else Icons.Filled.Check,
145                         contentDescription = null,
146                         tint = MaterialTheme.colorScheme.onPrimary,
147                         modifier = Modifier.size(36.dp)
148                     )
149                 }
150
151                 Spacer(modifier = Modifier.height(12.dp))
152
153                 Text(
154                     text = if (isVoided) stringResource(R.string.receipt_voided_badge) else
    stringResource(R.string.receipt_saved_title),
155                     style = MaterialTheme.typography.headlineSmall.copy(fontWeight = FontWeight.Bold),
156                     color = if (isVoided) Color(0xFFE57373) else MaterialTheme.colorScheme.onBackground
157                 )
158
159                 Text(
160                     text = "${stringResource(R.string.receipt_reference)}: $receiptNumber",
161                     style = MaterialTheme.typography.bodyMedium.copy(fontWeight = FontWeight.SemiBold),
162                     color = MaterialTheme.colorScheme.primary
163                 )
164             }
165
166             // Printable / Sharable Receipt Card
167             item {
168                 val isVoided = currentSale?.isVoided == true
```

```
169
170                 Card(
171                     colors = CardDefaults.cardColors(
172                         containerColor = MaterialTheme.colorScheme.surface
173                     ),
174                     shape = RoundedCornerShape(14.dp),
175                     modifier = Modifier
176                         .fillMaxWidth()
177                         .border(
178                             width = 1.dp,
179                             color = if (isVoided) Color(0xFFE57373).copy(alpha = 0.5f) else
    MaterialTheme.colorScheme.outline.copy(alpha = 0.2f),
180                             shape = RoundedCornerShape(14.dp)
181                         )
182                 ) {
183                     Column(
184                         modifier = Modifier
185                             .fillMaxWidth()
186                             .padding(16.dp),
187                         verticalArrangement = Arrangement.spacedBy(10.dp)
188                     ) {
189                         // Voided Banner
190                         if (isVoided) {
191                             Box(
192                                 modifier = Modifier
193                                     .fillMaxWidth()
194                                     .background(Color(0xFFE57373).copy(alpha = 0.15f),
    RoundedCornerShape(8.dp))
195                                     .padding(10.dp)
196                             ) {
197                                 Column {
198                                     Text(
199                                         text = "CANCELLED & REFUNDED",
200                                         fontWeight = FontWeight.Bold,
201                                         color = Color(0xFFE57373),
202                                         fontSize = 14.sp
203                                     )
204                                     val refundAmt = currentSale?.refundPayoutPaise ?: 0L
205                                     Text(
206                                         text = "Cash Refund Paid:
    ${CostCalculator.formatPaiseToRupees(refundAmt)}",
207                                         style = MaterialTheme.typography.bodySmall,
208                                         color = MaterialTheme.colorScheme.onSurface
209                                     )
210                                     val cond = currentSale?.stockRestoredCondition ?: "SELLABLE"
211                                     Text(
212                                         text = "Stock Condition: $cond",
213                                         style = MaterialTheme.typography.bodySmall,
214                                         color = MaterialTheme.colorScheme.onSurfaceVariant
215                                     )
216                                 }
217                             }
218                         }
219
220                         // Header
221                         Row(
222                             modifier = Modifier.fillMaxWidth(),
223                             horizontalArrangement = Arrangement.SpaceBetween,
224                             verticalAlignment = Alignment.CenterVertically
225                         ) {
226                             Text(
227                                 text = shopName,
228                                 style = MaterialTheme.typography.titleLarge.copy(fontWeight =
    FontWeight.Bold),
229                                 color = MaterialTheme.colorScheme.onSurface
230                             )
231                             Text(
232                                 text = memoTag,
233                                 style = MaterialTheme.typography.labelSmall.copy(fontWeight =
    FontWeight.Bold),
234                                 color = if (isVoided) Color(0xFFE57373) else
    MaterialTheme.colorScheme.primary
235                             )
236                         }
237
238                         Text(
239                             text = formattedDate,
240                             style = MaterialTheme.typography.bodySmall,
241                             color = MaterialTheme.colorScheme.onSurfaceVariant
242                         )
243
244                         if (currentSale?.customerName != null) {
245                             Text(
246                                 text = "${stringResource(R.string.review_customer)}:
    ${currentSale?.customerName}",
247                                 style = MaterialTheme.typography.bodyMedium.copy(fontWeight =
    FontWeight.SemiBold),
248                                 color = MaterialTheme.colorScheme.onSurface
```

```
                            )
                        }

                        Box(
                            modifier = Modifier
                                .fillMaxWidth()
                                .height(1.dp)
                                .background(MaterialTheme.colorScheme.outline.copy(alpha = 0.2f))
                        )

                        // Payment Method Row
                        Row(
                            modifier = Modifier.fillMaxWidth(),
                            horizontalArrangement = Arrangement.SpaceBetween,
                            verticalAlignment = Alignment.CenterVertically
                        ) {
                            Text(
                                text = stringResource(R.string.receipt_payment_mode),
                                style = MaterialTheme.typography.bodyMedium,
                                color = MaterialTheme.colorScheme.onSurfaceVariant
                            )
                            Text(
                                text = paymentMethodLabel,
                                style = MaterialTheme.typography.bodyMedium.copy(fontWeight =
    FontWeight.Bold),
                                color = if (paymentMethod == "CREDIT") Color(0xFFFFA000) else
    MaterialTheme.colorScheme.primary
                            )
                        }

                        Box(
                            modifier = Modifier
                                .fillMaxWidth()
                                .height(1.dp)
                                .background(MaterialTheme.colorScheme.outline.copy(alpha = 0.2f))
                        )

                        // Items Purchased List
                        Text(
                            text = stringResource(R.string.receipt_items_purchased),
                            style = MaterialTheme.typography.labelMedium.copy(fontWeight = FontWeight.Bold),
                            color = MaterialTheme.colorScheme.onSurfaceVariant
                        )

                        if (saleLines.isNotEmpty()) {
                            Column(verticalArrangement = Arrangement.spacedBy(8.dp)) {
                                saleLines.forEach { line ->
                                    Row(
                                        modifier = Modifier.fillMaxWidth(),
                                        horizontalArrangement = Arrangement.SpaceBetween,
                                        verticalAlignment = Alignment.CenterVertically
                                    ) {
                                        Column(modifier = Modifier.weight(1f)) {
                                            Text(
                                                text = line.productName,
                                                style = MaterialTheme.typography.bodyMedium.copy(fontWeight
    = FontWeight.SemiBold),
                                                color = MaterialTheme.colorScheme.onSurface
                                            )
                                            val unitStr = if (line.isPack)
    stringResource(R.string.billing_pack) else stringResource(R.string.billing_piece)
                                            val unitPrice =
    CostCalculator.formatPaiseToRupees(line.unitSellingPricePaise)
                                            Text(
                                                text = "${line.quantityBase} $unitStr @ $unitPrice",
                                                style = MaterialTheme.typography.bodySmall,
                                                color = MaterialTheme.colorScheme.onSurfaceVariant
                                            )
                                        }
                                        val lineGross = if (line.lineGrossPaise > 0L) line.lineGrossPaise
    else line.quantityBase * line.unitSellingPricePaise
                                        Text(
                                            text = CostCalculator.formatPaiseToRupees(lineGross),
                                            style = MaterialTheme.typography.bodyMedium.copy(fontWeight =
    FontWeight.Bold),
                                            color = MaterialTheme.colorScheme.onSurface
                                        )
                                    }

                                }
                            }
                        } else if (billingState.items.isNotEmpty()) {
                            Column(verticalArrangement = Arrangement.spacedBy(8.dp)) {
                                billingState.items.forEach { draftItem ->
                                    Row(
                                        modifier = Modifier.fillMaxWidth(),
                                        horizontalArrangement = Arrangement.SpaceBetween,
                                        verticalAlignment = Alignment.CenterVertically
                                    ) {
```

```
                                    Column(modifier = Modifier.weight(1f)) {
                                        Text(
                                            text = draftItem.productName,
                                            style = MaterialTheme.typography.bodyMedium.copy(fontWeight
= FontWeight.SemiBold),
                                            color = MaterialTheme.colorScheme.onSurface
                                        )
                                        val unitStr = if (draftItem.unitType == "PACK")
stringResource(R.string.billing_pack) else stringResource(R.string.billing_piece)
                                        val unitPrice =
CostCalculator.formatPaiseToRupees(draftItem.unitPricePaise)
                                        Text(
                                            text = "${draftItem.quantity} $unitStr @ $unitPrice",
                                            style = MaterialTheme.typography.bodySmall,
                                            color = MaterialTheme.colorScheme.onSurfaceVariant
                                        )
                                    }
                                    Text(
                                        text =
CostCalculator.formatPaiseToRupees(draftItem.lineGrossPaise),
                                        style = MaterialTheme.typography.bodyMedium.copy(fontWeight =
FontWeight.Bold),
                                        color = MaterialTheme.colorScheme.onSurface
                                    )
                                }
                            }
                        }
                    }

                    Box(
                        modifier = Modifier
                            .fillMaxWidth()
                            .height(1.dp)
                            .background(MaterialTheme.colorScheme.outline.copy(alpha = 0.2f))
                    )

                    // Financial Breakdown
                    Row(
                        modifier = Modifier.fillMaxWidth(),
                        horizontalArrangement = Arrangement.SpaceBetween
                    ) {
                        Text(
                            text = "Subtotal",
                            style = MaterialTheme.typography.bodyMedium,
                            color = MaterialTheme.colorScheme.onSurfaceVariant
                        )
                        Text(
                            text = CostCalculator.formatPaiseToRupees(grossPaise),
                            style = MaterialTheme.typography.bodyMedium,
                            color = MaterialTheme.colorScheme.onSurface
                        )
                    }

                    if (discountPaise > 0L) {
                        val discountPct = currentSale?.discountPercent ?: billingState.discountPercent
                        val discountLabel = if (discountPct > 0.0) {
                            val formatted = if (discountPct % 1.0 == 0.0) String.format(Locale.ROOT,
"%.0f", discountPct) else String.format(Locale.ROOT, "%.1f", discountPct)
                            "Discount ($formatted%)"
                        } else "Discount"
                        Row(
                            modifier = Modifier.fillMaxWidth(),
                            horizontalArrangement = Arrangement.SpaceBetween
                        ) {
                            Text(
                                text = discountLabel,
                                style = MaterialTheme.typography.bodyMedium,
                                color = MaterialTheme.colorScheme.onSurfaceVariant
                            )
                            Text(
                                text = "-${CostCalculator.formatPaiseToRupees(discountPaise)}",
                                style = MaterialTheme.typography.bodyMedium.copy(fontWeight =
FontWeight.Bold),
                                color = MaterialTheme.colorScheme.primary
                            )
                        }
                    }

                    // Shaded TOTAL PAYABLE banner matching receipt PDF
                    Box(
                        modifier = Modifier
                            .fillMaxWidth()
                            .background(MaterialTheme.colorScheme.surfaceVariant.copy(alpha = 0.6f),
RoundedCornerShape(6.dp))
                            .padding(horizontal = 12.dp, vertical = 8.dp)
                    ) {
                        Row(
                            modifier = Modifier.fillMaxWidth(),
```

```
                                horizontalArrangement = Arrangement.SpaceBetween,
                                verticalAlignment = Alignment.CenterVertically
                            ) {
                                Text(
                                    text = "TOTAL PAYABLE",
                                    style = MaterialTheme.typography.titleMedium.copy(fontWeight =
    FontWeight.Bold),
                                    color = MaterialTheme.colorScheme.onSurface
                                )
                                Text(
                                    text = CostCalculator.formatPaiseToRupees(totalPaise),
                                    style = MaterialTheme.typography.titleLarge.copy(
                                        fontWeight = FontWeight.Bold,
                                        fontSize = 20.sp
                                    ),
                                    color = if (isVoided) Color(0xFFE57373) else
    MaterialTheme.colorScheme.onSurface
                                )
                            }
                        }

                        // Amount Paid
                        val amountPaidPaise = currentSale?.amountReceivedPaise ?: totalPaise
                        Row(
                            modifier = Modifier.fillMaxWidth(),
                            horizontalArrangement = Arrangement.SpaceBetween
                        ) {
                            Text(
                                text = "Amount paid",
                                style = MaterialTheme.typography.bodyMedium,
                                color = MaterialTheme.colorScheme.onSurfaceVariant
                            )
                            Text(
                                text = CostCalculator.formatPaiseToRupees(amountPaidPaise),
                                style = MaterialTheme.typography.bodyMedium,
                                color = MaterialTheme.colorScheme.onSurface
                            )
                        }

                        // Balance due or Change
                        val remainingCredit = currentSale?.remainingCreditPaise ?: 0L
                        if (remainingCredit > 0L) {
                            Row(
                                modifier = Modifier.fillMaxWidth(),
                                horizontalArrangement = Arrangement.SpaceBetween
                            ) {
                                Text(
                                    text = "Balance due",
                                    style = MaterialTheme.typography.bodyMedium.copy(fontWeight =
    FontWeight.SemiBold),
                                    color = Color(0xFFFFA000)
                                )
                                Text(
                                    text = CostCalculator.formatPaiseToRupees(remainingCredit),
                                    style = MaterialTheme.typography.bodyMedium.copy(fontWeight =
    FontWeight.Bold),
                                    color = Color(0xFFFFA000)
                                )
                            }
                        } else if (changeGivenPaise > 0L) {
                            Row(
                                modifier = Modifier.fillMaxWidth(),
                                horizontalArrangement = Arrangement.SpaceBetween
                            ) {
                                Text(
                                    text = "Change",
                                    style = MaterialTheme.typography.bodyMedium,
                                    color = MaterialTheme.colorScheme.primary
                                )
                                Text(
                                    text = CostCalculator.formatPaiseToRupees(changeGivenPaise),
                                    style = MaterialTheme.typography.bodyMedium.copy(fontWeight =
    FontWeight.Bold),
                                    color = MaterialTheme.colorScheme.primary
                                )
                            }
                            Row(
                                modifier = Modifier.fillMaxWidth(),
                                horizontalArrangement = Arrangement.SpaceBetween
                            ) {
                                Text(
                                    text = "Balance due",
                                    style = MaterialTheme.typography.bodyMedium,
                                    color = MaterialTheme.colorScheme.onSurfaceVariant
                                )
                                Text(
                                    text = "₹0.00",
                                    style = MaterialTheme.typography.bodyMedium,
                                    color = MaterialTheme.colorScheme.onSurface
```

```
                                )
                            }
                        } else {
                            Row(
                                modifier = Modifier.fillMaxWidth(),
                                horizontalArrangement = Arrangement.SpaceBetween
                            ) {
                                Text(
                                    text = "Balance due",
                                    style = MaterialTheme.typography.bodyMedium,
                                    color = MaterialTheme.colorScheme.onSurfaceVariant
                                )
                                Text(
                                    text = "₹0.00",
                                    style = MaterialTheme.typography.bodyMedium,
                                    color = MaterialTheme.colorScheme.onSurface
                                )
                            }
                        }

                        // Centered Status Badge
                        val badgeStr = when {
                            isVoided -> "V O I D E D"
                            remainingCredit > 0L && amountPaidPaise > 0L -> "P A R T I A L"
                            remainingCredit > 0L -> "D U E"
                            else -> "P A I D"
                        }
                        Box(
                            modifier = Modifier
                                .fillMaxWidth()
                                .padding(top = 6.dp),
                            contentAlignment = Alignment.Center
                        ) {
                            Box(
                                modifier = Modifier
                                    .border(
                                        width = 1.dp,
                                        color = if (isVoided) Color(0xFFE57373) else MaterialTheme.colorScheme.onSurface,
                                        shape = RoundedCornerShape(4.dp)
                                    )
                                    .padding(horizontal = 14.dp, vertical = 4.dp)
                            ) {
                                Text(
                                    text = badgeStr,
                                    style = MaterialTheme.typography.labelMedium.copy(
                                        fontWeight = FontWeight.Bold,
                                        letterSpacing = 2.sp
                                    ),
                                    color = if (isVoided) Color(0xFFE57373) else MaterialTheme.colorScheme.onSurface
                                )
                            }
                        }
                    }
                }
            }
        }

        // Share & Print Actions
        item {
            val coroutineScope = androidx.compose.runtime.rememberCoroutineScope()
            var isSharingPdf by remember { mutableStateOf(false) }

            Column(
                modifier = Modifier.fillMaxWidth(),
                verticalArrangement = Arrangement.spacedBy(10.dp)
            ) {
                // 1. Primary Action: Branded WhatsApp PDF Receipt
                Button(
                    onClick = {
                        val sale = currentSale ?: return@Button
                        coroutineScope.launch {
                            isSharingPdf = true
                            try {
                                val intent = viewModel.receiptService.createWhatsAppReceiptIntent(sale, saleLines)
                                context.startActivity(intent)
                                Toast.makeText(context, "Opened in WhatsApp", Toast.LENGTH_SHORT).show()
                            } catch (e: Exception) {
                                Toast.makeText(context, "Failed to share: ${e.localizedMessage}", Toast.LENGTH_LONG).show()
                            } finally {
                                isSharingPdf = false
                            }
                        }
                    },
                    enabled = !isSharingPdf && currentSale != null,
                    modifier = Modifier
```

```
                            .fillMaxWidth()
                            .height(48.dp),
                        shape = RoundedCornerShape(12.dp),
                        colors = ButtonDefaults.buttonColors(
                            containerColor = MaterialTheme.colorScheme.primary,
                            contentColor = MaterialTheme.colorScheme.onPrimary
                        )
                    ) {
                        if (isSharingPdf) {
                            androidx.compose.material3.CircularProgressIndicator(
                                modifier = Modifier.size(18.dp),
                                color = MaterialTheme.colorScheme.onPrimary,
                                strokeWidth = 2.dp
                            )
                            Spacer(modifier = Modifier.width(8.dp))
                            Text("Generating Branded PDF...")
                        } else {
                            Icon(
                                imageVector = Icons.Filled.Share,
                                contentDescription = null,
                                modifier = Modifier.size(18.dp)
                            )
                            Spacer(modifier = Modifier.width(8.dp))
                            Text("Share WhatsApp PDF Receipt", fontWeight = FontWeight.Bold)
                        }
                    }

                    // 2. Secondary Row: Print Receipt (Android PrintManager) & Save/Share PDF
                    Row(
                        modifier = Modifier.fillMaxWidth(),
                        horizontalArrangement = Arrangement.spacedBy(10.dp)
                    ) {
                        OutlinedButton(
                            onClick = {
                                val activity = context as? android.app.Activity
                                val sale = currentSale
                                if (activity != null && sale != null) {
                                    viewModel.receiptService.printReceipt(activity, sale, saleLines)
                                } else {
                                    Toast.makeText(context, "Printing not supported on this device",
Toast.LENGTH_SHORT).show()
                                }
                            },
                            enabled = currentSale != null,
                            modifier = Modifier.weight(1f),
                            shape = RoundedCornerShape(10.dp)
                        ) {
                            Icon(
                                imageVector = Icons.Filled.Print,
                                contentDescription = null,
                                modifier = Modifier.size(18.dp)
                            )
                            Spacer(modifier = Modifier.width(6.dp))
                            Text("Print Receipt")
                        }

                        OutlinedButton(
                            onClick = {
                                val sale = currentSale ?: return@OutlinedButton
                                coroutineScope.launch {
                                    try {
                                        val pdfFile = viewModel.receiptService.createReceiptPdf(sale,
saleLines)
                                        val uri = androidx.core.content.FileProvider.getUriForFile(
                                            context,
                                            "${context.packageName}.fileprovider",
                                            pdfFile
                                        )
                                        val sendIntent = Intent(Intent.ACTION_SEND).apply {
                                            type = "application/pdf"
                                            putExtra(Intent.EXTRA_STREAM, uri)
                                            addFlags(Intent.FLAG_GRANT_READ_URI_PERMISSION)
                                        }
                                        context.startActivity(Intent.createChooser(sendIntent, "Share
Receipt PDF"))
                                    } catch (e: Exception) {
                                        Toast.makeText(context, "Failed: ${e.message}",
Toast.LENGTH_SHORT).show()
                                    }
                                }
                            },
                            enabled = currentSale != null,
                            modifier = Modifier.weight(1f),
                            shape = RoundedCornerShape(10.dp)
                        ) {
                            Icon(
                                imageVector = Icons.Filled.Share,
                                contentDescription = null,
```

```
                                modifier = Modifier.size(18.dp)
                            )
                            Spacer(modifier = Modifier.width(6.dp))
                            Text("Save / Other")
                        }
                    }
                }
            }

            // Void / Cancellation Action (F13 policy: Ask condition & refund payout)
            if (currentSale != null && currentSale?.isVoided != true) {
                item {
                    OutlinedButton(
                        onClick = { showVoidDialog = true },
                        modifier = Modifier
                            .fillMaxWidth()
                            .height(48.dp),
                        shape = RoundedCornerShape(12.dp),
                        colors = ButtonDefaults.outlinedButtonColors(
                            contentColor = Color(0xFFE57373)
                        )
                    ) {
                        Icon(Icons.Filled.Cancel, contentDescription = null, modifier =
Modifier.size(18.dp))
                        Spacer(modifier = Modifier.width(8.dp))
                        Text(
                            text = stringResource(R.string.receipt_void_button),
                            fontWeight = FontWeight.Bold
                        )
                    }
                }
            }

            // Navigation Actions: New Bill & Home
            item {
                Spacer(modifier = Modifier.height(6.dp))
                Button(
                    onClick = {
                        viewModel.resetBilling()
                        onNewBill()
                    },
                    modifier = Modifier
                        .fillMaxWidth()
                        .height(52.dp),
                    shape = RoundedCornerShape(12.dp),
                    colors = ButtonDefaults.buttonColors(
                        containerColor = MaterialTheme.colorScheme.primary,
                        contentColor = MaterialTheme.colorScheme.onPrimary
                    )
                ) {
                    Icon(
                        imageVector = Icons.Filled.PointOfSale,
                        contentDescription = null
                    )
                    Spacer(modifier = Modifier.width(8.dp))
                    Text(
                        text = stringResource(R.string.receipt_new_bill),
                        style = MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.Bold)
                    )
                }

                Spacer(modifier = Modifier.height(8.dp))

                OutlinedButton(
                    onClick = {
                        viewModel.resetBilling()
                        onDone()
                    },
                    modifier = Modifier
                        .fillMaxWidth()
                        .height(48.dp),
                    shape = RoundedCornerShape(12.dp)
                ) {
                    Icon(
                        imageVector = Icons.Filled.Home,
                        contentDescription = null
                    )
                    Spacer(modifier = Modifier.width(8.dp))
                    Text(
                        text = stringResource(R.string.receipt_done),
                        style = MaterialTheme.typography.bodyMedium.copy(fontWeight = FontWeight.SemiBold)
                    )
                }
            }
        }
    }

    // Cancellation & Refund Dialog (Asks condition & Refunds payout)
```

```
    if (showVoidDialog && currentSale != null) {
        val sale = currentSale!!
        val refundPaise = minOf(sale.amountReceivedPaise, sale.netTotalPaise)

        AlertDialog(
            onDismissRequest = { if (!isVoiding) showVoidDialog = false },
            icon = {
                Icon(Icons.Filled.Warning, contentDescription = null, tint = Color(0xFFE57373), modifier =
Modifier.size(36.dp))
            },
            title = {
                Text(
                    text = stringResource(R.string.receipt_void_title),
                    fontWeight = FontWeight.Bold
                )
            },
            text = {
                Column(verticalArrangement = Arrangement.spacedBy(12.dp)) {
                    Text(
                        text = stringResource(R.string.receipt_void_warning),
                        style = MaterialTheme.typography.bodySmall,
                        color = MaterialTheme.colorScheme.onSurfaceVariant
                    )

                    // Refund Payout Explanation
                    Card(
                        colors = CardDefaults.cardColors(containerColor = Color(0xFFE57373).copy(alpha =
0.15f)),
                        shape = RoundedCornerShape(8.dp)
                    ) {
                        Column(modifier = Modifier.padding(10.dp)) {
                            Text(
                                text = stringResource(R.string.receipt_void_refund_payout),
                                style = MaterialTheme.typography.labelSmall.copy(fontWeight =
FontWeight.Bold),
                                color = Color(0xFFE57373)
                            )
                            Text(
                                text = CostCalculator.formatPaiseToRupees(refundPaise),
                                style = MaterialTheme.typography.titleMedium.copy(fontWeight =
FontWeight.Bold),
                                color = Color(0xFFE57373)
                            )
                            Text(
                                text = "Cash refund will be deducted from till payout records.",
                                style = MaterialTheme.typography.bodySmall
                            )
                        }
                    }

                    // Stock Return Condition Selection ("It should ask")
                    Text(
                        text = stringResource(R.string.receipt_void_stock_question),
                        style = MaterialTheme.typography.bodyMedium.copy(fontWeight = FontWeight.Bold)
                    )

                    Row(
                        modifier = Modifier
                            .fillMaxWidth()
                            .clickable { stockCondition = "SELLABLE" }
                            .padding(vertical = 4.dp),
                        verticalAlignment = Alignment.CenterVertically
                    ) {
                        Icon(
                            imageVector = if (stockCondition == "SELLABLE") Icons.Filled.RadioButtonChecked
else Icons.Filled.RadioButtonUnchecked,
                            contentDescription = null,
                            tint = if (stockCondition == "SELLABLE") MaterialTheme.colorScheme.primary else
MaterialTheme.colorScheme.onSurfaceVariant
                        )
                        Spacer(modifier = Modifier.width(8.dp))
                        Text(
                            text = stringResource(R.string.receipt_void_condition_sellable),
                            style = MaterialTheme.typography.bodyMedium
                        )
                    }

                    Row(
                        modifier = Modifier
                            .fillMaxWidth()
                            .clickable { stockCondition = "DAMAGED" }
                            .padding(vertical = 4.dp),
                        verticalAlignment = Alignment.CenterVertically
                    ) {
                        Icon(
                            imageVector = if (stockCondition == "DAMAGED") Icons.Filled.RadioButtonChecked
else Icons.Filled.RadioButtonUnchecked,
                            contentDescription = null,
```

```
                            tint = if (stockCondition == "DAMAGED") MaterialTheme.colorScheme.primary else
    MaterialTheme.colorScheme.onSurfaceVariant
                        )
                        Spacer(modifier = Modifier.width(8.dp))
                        Text(
                            text = stringResource(R.string.receipt_void_condition_damaged),
                            style = MaterialTheme.typography.bodyMedium
                        )
                    }

                    OutlinedTextField(
                        value = voidReason,
                        onValueChange = { voidReason = it },
                        label = { Text(stringResource(R.string.receipt_void_reason_label)) },
                        modifier = Modifier.fillMaxWidth(),
                        singleLine = true,
                        shape = RoundedCornerShape(8.dp)
                    )
                }
            },
            confirmButton = {
                Button(
                    onClick = {
                        isVoiding = true
                        viewModel.voidSale(
                            saleId = sale.id,
                            stockCondition = stockCondition,
                            voidReason = voidReason.ifBlank { "Customer return / billing reversal" },
                            onDone = {
                                isVoiding = false
                                showVoidDialog = false
                                currentSale = currentSale?.copy(
                                    isVoided = true,
                                    voidReason = voidReason.ifBlank { "Customer return" },
                                    refundPayoutPaise = refundPaise,
                                    stockRestoredCondition = stockCondition
                                )
                                Toast.makeText(context, "Sale cancelled and refund recorded.",
    Toast.LENGTH_SHORT).show()
                            },
                            onError = { err ->
                                isVoiding = false
                                Toast.makeText(context, err, Toast.LENGTH_LONG).show()
                            }
                        )
                    },
                    colors = ButtonDefaults.buttonColors(containerColor = Color(0xFFE57373)),
                    enabled = !isVoiding
                ) {
                    Text(stringResource(R.string.receipt_void_confirm))
                }
            },
            dismissButton = {
                OutlinedButton(
                    onClick = { showVoidDialog = false },
                    enabled = !isVoiding
                ) {
                    Text("Dismiss")
                }
            }
        )
    }
}
```

app\src\main\java\com\alieducational\app\ui\billing\CustomerSelectionDialog.kt

```
package com.alieducational.app.ui.billing

import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Clear
import androidx.compose.material.icons.filled.Person
import androidx.compose.material.icons.filled.PersonAdd
import androidx.compose.material.icons.filled.Search
import androidx.compose.material3.AlertDialog
import androidx.compose.material3.Button
import androidx.compose.material3.ButtonDefaults
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedButton
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.Tab
import androidx.compose.material3.TabRow
import androidx.compose.material3.Text
import androidx.compose.material3.TextButton
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableIntStateOf
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.res.stringResource
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.input.KeyboardCapitalization
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.alieducational.app.R
import com.alieducational.app.data.local.entity.CustomerEntity
import com.alieducational.app.domain.calculation.CostCalculator

@Composable
fun CustomerSelectionDialog(
    currentCustomer: CustomerEntity?,
    allCustomers: List<CustomerEntity>,
    onSelectCustomer: (CustomerEntity?) -> Unit,
    onCreateCustomer: (name: String, phone: String, notes: String) -> Unit,
    onDismiss: () -> Unit,
    modifier: Modifier = Modifier
) {
    var selectedTab by remember { mutableIntStateOf(0) }
    var searchQuery by remember { mutableStateOf("") }

    // New customer form state
    var newName by remember { mutableStateOf("") }
    var newPhone by remember { mutableStateOf("") }
    var newNotes by remember { mutableStateOf("") }

    val filteredCustomers = remember(searchQuery, allCustomers) {
        if (searchQuery.isBlank()) {
            allCustomers
        } else {
            val q = searchQuery.trim().lowercase()
            allCustomers.filter {
                it.name.lowercase().contains(q) || (it.phone?.contains(q) == true)
            }
        }
    }

```

```
    AlertDialog(
        onDismissRequest = onDismiss,
        modifier = modifier.fillMaxWidth(),
        title = {
            Text(
                text = stringResource(R.string.review_select_customer),
                style = MaterialTheme.typography.titleLarge.copy(fontWeight = FontWeight.Bold)
            )
        },
        text = {
            Column(
                modifier = Modifier.fillMaxWidth()
            ) {
                // Two Tabs: Existing vs New Customer
                TabRow(
                    selectedTabIndex = selectedTab,
                    containerColor = MaterialTheme.colorScheme.surface
                ) {
                    Tab(
                        selected = selectedTab == 0,
                        onClick = { selectedTab = 0 },
                        text = {
                            Row(verticalAlignment = Alignment.CenterVertically) {
                                Icon(Icons.Filled.Person, contentDescription = null, modifier =
Modifier.size(16.dp))
                                Spacer(modifier = Modifier.width(6.dp))
                                Text(stringResource(R.string.billing_existing_customer), fontWeight =
FontWeight.Bold)
                            }
                        }
                    )
                    Tab(
                        selected = selectedTab == 1,
                        onClick = { selectedTab = 1 },
                        text = {
                            Row(verticalAlignment = Alignment.CenterVertically) {
                                Icon(Icons.Filled.PersonAdd, contentDescription = null, modifier =
Modifier.size(16.dp))
                                Spacer(modifier = Modifier.width(6.dp))
                                Text(stringResource(R.string.billing_new_customer), fontWeight =
FontWeight.Bold)
                            }
                        }
                    )
                }

                Spacer(modifier = Modifier.height(12.dp))

                if (selectedTab == 0) {
                    // TAB 0: EXISTING CUSTOMERS
                    OutlinedTextField(
                        value = searchQuery,
                        onValueChange = { searchQuery = it },
                        placeholder = { Text(stringResource(R.string.billing_search_customer), fontSize =
13.sp) },
                        leadingIcon = {
                            Icon(Icons.Filled.Search, contentDescription = null, modifier =
Modifier.size(18.dp))
                        },
                        trailingIcon = {
                            if (searchQuery.isNotEmpty()) {
                                IconButton(onClick = { searchQuery = "" }) {
                                    Icon(Icons.Filled.Clear, contentDescription = "Clear", modifier =
Modifier.size(16.dp))
                                }
                            }
                        },
                        singleLine = true,
                        modifier = Modifier.fillMaxWidth(),
                        shape = RoundedCornerShape(10.dp)
                    )

                    Spacer(modifier = Modifier.height(10.dp))

                    LazyColumn(
                        modifier = Modifier
                            .fillMaxWidth()
                            .height(260.dp),
                        verticalArrangement = Arrangement.spacedBy(8.dp)
                    ) {
                        // Walk-in Option
                        item {
                            Card(
                                colors = CardDefaults.cardColors(
                                    containerColor = if (currentCustomer == null)
                                        MaterialTheme.colorScheme.primaryContainer
                                    else
                                        MaterialTheme.colorScheme.surfaceVariant
```

```
                                ),
                                shape = RoundedCornerShape(10.dp),
                                modifier = Modifier
                                    .fillMaxWidth()
                                    .clickable { onSelectCustomer(null) }
                            ) {
                                Row(
                                    modifier = Modifier
                                        .fillMaxWidth()
                                        .padding(12.dp),
                                    verticalAlignment = Alignment.CenterVertically,
                                    horizontalArrangement = Arrangement.SpaceBetween
                                ) {
                                    Column {
                                        Text(
                                            text = stringResource(R.string.review_walk_in),
                                            style = MaterialTheme.typography.bodyMedium.copy(fontWeight =
    FontWeight.Bold)
                                        )
                                        Text(
                                            text = "No Khata / Immediate Cash",
                                            style = MaterialTheme.typography.bodySmall,
                                            color = MaterialTheme.colorScheme.onSurfaceVariant
                                        )
                                    }
                                }
                            }
                        }

                        // Filtered existing customers
                        items(filteredCustomers, key = { it.id }) { customer ->
                            val isSelected = currentCustomer?.id == customer.id
                            Card(
                                colors = CardDefaults.cardColors(
                                    containerColor = if (isSelected)
                                        MaterialTheme.colorScheme.primaryContainer
                                    else
                                        MaterialTheme.colorScheme.surfaceVariant
                                ),
                                shape = RoundedCornerShape(10.dp),
                                modifier = Modifier
                                    .fillMaxWidth()
                                    .clickable { onSelectCustomer(customer) }
                            ) {
                                Row(
                                    modifier = Modifier
                                        .fillMaxWidth()
                                        .padding(12.dp),
                                    horizontalArrangement = Arrangement.SpaceBetween,
                                    verticalAlignment = Alignment.CenterVertically
                                ) {
                                    Column(modifier = Modifier.weight(1f)) {
                                        Text(
                                            text = customer.name,
                                            style = MaterialTheme.typography.bodyMedium.copy(fontWeight =
    FontWeight.Bold)
                                        )
                                        if (!customer.phone.isNullOrBlank()) {
                                            Text(
                                                text = customer.phone ?: "",
                                                style = MaterialTheme.typography.bodySmall,
                                                color = MaterialTheme.colorScheme.onSurfaceVariant
                                            )
                                        }
                                    }

                                    // Khata Balance Standing
                                    val duePaise = customer.currentBalancePaise
                                    if (duePaise > 0L) {
                                        Text(
                                            text = stringResource(R.string.billing_customer_khata_due,
    CostCalculator.formatPaiseToRupees(duePaise)),
                                            style = MaterialTheme.typography.labelSmall.copy(fontWeight =
    FontWeight.Bold),
                                            color = Color(0xFFD32F2F)
                                        )
                                    } else {
                                        Text(
                                            text = stringResource(R.string.billing_customer_khata_clear),
                                            style = MaterialTheme.typography.labelSmall,
                                            color = Color(0xFF388E3C)
                                        )
                                    }
                                }
                            }
                        }

                        if (filteredCustomers.isEmpty() && searchQuery.isNotBlank()) {
```

```
                            item {
                                Column(
                                    modifier = Modifier
                                        .fillMaxWidth()
                                        .padding(vertical = 16.dp),
                                    horizontalAlignment = Alignment.CenterHorizontally
                                ) {
                                    Text(
                                        text = "No customer found for '$searchQuery'",
                                        style = MaterialTheme.typography.bodyMedium,
                                        color = MaterialTheme.colorScheme.onSurfaceVariant
                                    )
                                    Spacer(modifier = Modifier.height(8.dp))
                                    OutlinedButton(
                                        onClick = {
                                            newName = searchQuery.trim()
                                            selectedTab = 1
                                        },
                                        shape = RoundedCornerShape(8.dp)
                                    ) {
                                        Text("+ Create '$searchQuery'")
                                    }
                                }
                            }
                        }
                    }
                } else {
                    // TAB 1: CREATE NEW CUSTOMER
                    Column(
                        modifier = Modifier
                            .fillMaxWidth()
                            .padding(top = 4.dp),
                        verticalArrangement = Arrangement.spacedBy(10.dp)
                    ) {
                        OutlinedTextField(
                            value = newName,
                            onValueChange = { newName = it },
                            label = { Text(stringResource(R.string.billing_customer_name)) },
                            singleLine = true,
                            keyboardOptions = KeyboardOptions(capitalization =
KeyboardCapitalization.Words),
                            modifier = Modifier.fillMaxWidth(),
                            shape = RoundedCornerShape(10.dp)
                        )

                        OutlinedTextField(
                            value = newPhone,
                            onValueChange = { newPhone = it },
                            label = { Text(stringResource(R.string.billing_customer_phone)) },
                            singleLine = true,
                            keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Phone),
                            modifier = Modifier.fillMaxWidth(),
                            shape = RoundedCornerShape(10.dp)
                        )

                        OutlinedTextField(
                            value = newNotes,
                            onValueChange = { newNotes = it },
                            label = { Text(stringResource(R.string.billing_customer_notes)) },
                            singleLine = true,
                            modifier = Modifier.fillMaxWidth(),
                            shape = RoundedCornerShape(10.dp)
                        )

                        Spacer(modifier = Modifier.height(4.dp))

                        Button(
                            onClick = {
                                if (newName.isNotBlank()) {
                                    onCreateCustomer(newName.trim(), newPhone.trim(), newNotes.trim())
                                }
                            },
                            enabled = newName.isNotBlank(),
                            modifier = Modifier
                                .fillMaxWidth()
                                .height(48.dp),
                            shape = RoundedCornerShape(10.dp),
                            colors = ButtonDefaults.buttonColors(
                                containerColor = MaterialTheme.colorScheme.primary,
                                contentColor = MaterialTheme.colorScheme.onPrimary
                            )
                        ) {
                            Icon(Icons.Filled.PersonAdd, contentDescription = null, modifier =
Modifier.size(18.dp))
                            Spacer(modifier = Modifier.width(8.dp))
                            Text(
                                text = stringResource(R.string.billing_save_select_customer),
                                fontWeight = FontWeight.Bold
```

```
                            )
                        }
                    }
                }
            }
        },
        confirmButton = {
            TextButton(onClick = onDismiss) {
                Text(stringResource(R.string.review_cancel))
            }
        }
    )
}
```

app\src\main\java\com\alieducational\app\ui\billing\HeldBillsDialog.kt

```
package com.alieducational.app.ui.billing

import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Delete
import androidx.compose.material.icons.filled.PauseCircleOutline
import androidx.compose.material.icons.filled.PlayArrow
import androidx.compose.material3.AlertDialog
import androidx.compose.material3.Button
import androidx.compose.material3.ButtonDefaults
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedButton
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.Text
import androidx.compose.material3.TextButton
import androidx.compose.runtime.Composable
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.res.stringResource
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.alieducational.app.R
import com.alieducational.app.domain.calculation.CostCalculator
import com.alieducational.app.ui.AliViewModel
import java.text.SimpleDateFormat
import java.util.Date
import java.util.Locale

@Composable
fun HeldBillsDialog(
    viewModel: AliViewModel,
    onDismiss: () -> Unit
) {
    val heldBills by viewModel.allHeldBills.collectAsState()
    val billingState by viewModel.billingState.collectAsState()
    var holdNote by remember { mutableStateOf("") }
    var isHolding by remember { mutableStateOf(false) }

    val dateFormat = remember { SimpleDateFormat("dd MMM, hh:mm a", Locale.getDefault()) }

    AlertDialog(
        onDismissRequest = onDismiss,
        title = {
            Row(
                verticalAlignment = Alignment.CenterVertically,
                horizontalArrangement = Arrangement.spacedBy(8.dp)
            ) {
                Icon(
                    imageVector = Icons.Filled.PauseCircleOutline,
                    contentDescription = null,
                    tint = MaterialTheme.colorScheme.primary
                )
                Text(
                    text = stringResource(R.string.held_bills_title),
                    style = MaterialTheme.typography.titleLarge.copy(
                        fontWeight = FontWeight.Bold,
                        fontSize = 19.sp
                    )
                )
            }
        },
        text = {
```

```
            Column(
                modifier = Modifier
                    .fillMaxWidth()
                    .padding(vertical = 4.dp),
                verticalArrangement = Arrangement.spacedBy(12.dp)
            ) {
                // Option to hold current bill if it has items
                if (billingState.items.isNotEmpty()) {
                    Card(
                        colors = CardDefaults.cardColors(
                            containerColor = MaterialTheme.colorScheme.primaryContainer.copy(alpha = 0.4f)
                        ),
                        shape = RoundedCornerShape(12.dp),
                        modifier = Modifier.fillMaxWidth()
                    ) {
                        Column(modifier = Modifier.padding(12.dp)) {
                            Text(
                                text = stringResource(
                                    R.string.billing_items_summary,
                                    billingState.totalItemCount,
                                    CostCalculator.formatPaiseToRupees(billingState.netTotalPaise)
                                ),
                                style = MaterialTheme.typography.bodyMedium.copy(fontWeight =
    FontWeight.Bold),
                                color = MaterialTheme.colorScheme.onSurface
                            )
                            Spacer(modifier = Modifier.height(8.dp))
                            OutlinedTextField(
                                value = holdNote,
                                onValueChange = { holdNote = it },
                                label = { Text(stringResource(R.string.held_bill_prompt_note)) },
                                singleLine = true,
                                modifier = Modifier.fillMaxWidth()
                            )
                            Spacer(modifier = Modifier.height(8.dp))
                            Button(
                                onClick = {
                                    if (!isHolding) {
                                        isHolding = true
                                        viewModel.holdCurrentBill(note = holdNote.ifBlank { null }) {
                                            isHolding = false
                                            onDismiss()
                                        }
                                    }
                                },
                                modifier = Modifier.fillMaxWidth(),
                                colors = ButtonDefaults.buttonColors(
                                    containerColor = MaterialTheme.colorScheme.primary,
                                    contentColor = MaterialTheme.colorScheme.onPrimary
                                )
                            ) {
                                Text(stringResource(R.string.held_bill_confirm_hold))
                            }
                        }
                    }
                }

                // List of existing held bills
                if (heldBills.isEmpty()) {
                    Text(
                        text = stringResource(R.string.held_bills_empty),
                        style = MaterialTheme.typography.bodyMedium,
                        color = MaterialTheme.colorScheme.onSurfaceVariant,
                        modifier = Modifier.padding(vertical = 12.dp)
                    )
                } else {
                    LazyColumn(
                        modifier = Modifier
                            .fillMaxWidth()
                            .height(260.dp),
                        verticalArrangement = Arrangement.spacedBy(8.dp)
                    ) {
                        items(heldBills, key = { it.id }) { bill ->
                            Card(
                                colors = CardDefaults.cardColors(
                                    containerColor = MaterialTheme.colorScheme.surfaceVariant
                                ),
                                shape = RoundedCornerShape(10.dp),
                                modifier = Modifier.fillMaxWidth()
                            ) {
                                Row(
                                    modifier = Modifier
                                        .fillMaxWidth()
                                        .padding(12.dp),
                                    verticalAlignment = Alignment.CenterVertically,
                                    horizontalArrangement = Arrangement.SpaceBetween
                                ) {
                                    Column(modifier = Modifier.weight(1f)) {
```

```
                                        Text(
                                            text = stringResource(
                                                R.string.billing_items_summary,
                                                bill.itemCount,
                                                CostCalculator.formatPaiseToRupees(bill.totalPaise)
                                            ),
                                            style = MaterialTheme.typography.titleMedium.copy(fontWeight =
    FontWeight.Bold),
                                            color = MaterialTheme.colorScheme.onSurface
                                        )
                                        if (!bill.customerName.isNullOrBlank()) {
                                            Text(
                                                text = bill.customerName,
                                                style = MaterialTheme.typography.bodySmall,
                                                color = MaterialTheme.colorScheme.primary
                                            )
                                        }
                                        if (!bill.note.isNullOrBlank()) {
                                            Text(
                                                text = bill.note,
                                                style = MaterialTheme.typography.bodySmall,
                                                color = MaterialTheme.colorScheme.onSurfaceVariant

                                            )
                                        }
                                        Text(
                                            text = dateFormat.format(Date(bill.createdAt)),
                                            style = MaterialTheme.typography.labelSmall,
                                            color = MaterialTheme.colorScheme.onSurfaceVariant.copy(alpha =
    0.7f)
                                        )
                                    }

                                    Row(
                                        verticalAlignment = Alignment.CenterVertically,
                                        horizontalArrangement = Arrangement.spacedBy(4.dp)
                                    ) {
                                        IconButton(
                                            onClick = { viewModel.discardHeldBill(bill.id) }
                                        ) {
                                            Icon(
                                                imageVector = Icons.Filled.Delete,
                                                contentDescription =
    stringResource(R.string.billing_discard),
                                                tint = MaterialTheme.colorScheme.error
                                            )
                                        }

                                        Button(
                                            onClick = {
                                                viewModel.resumeHeldBill(bill) {
                                                    onDismiss()
                                                }
                                            },
                                            shape = RoundedCornerShape(8.dp),
                                            colors = ButtonDefaults.buttonColors(
                                                containerColor = MaterialTheme.colorScheme.primary,
                                                contentColor = MaterialTheme.colorScheme.onPrimary
                                            )
                                        ) {
                                            Icon(
                                                imageVector = Icons.Filled.PlayArrow,
                                                contentDescription = null
                                            )
                                            Spacer(modifier = Modifier.width(4.dp))
                                            Text(stringResource(R.string.billing_resume))
                                        }
                                    }
                                }
                            }
                        }
                    }
                }
            }
        },
        confirmButton = {
            TextButton(onClick = onDismiss) {
                Text(stringResource(R.string.review_cancel))
            }
        }
    )
}
```

app\src\main\java\com\alieducational\app\ui\billing\PaymentScreen.kt

```
1  package com.alieducational.app.ui.billing
2
3  import androidx.compose.foundation.background
4  import androidx.compose.foundation.border
5  import androidx.compose.foundation.clickable
6  import androidx.compose.foundation.layout.Arrangement
7  import androidx.compose.foundation.layout.Box
8  import androidx.compose.foundation.layout.Column
9  import androidx.compose.foundation.layout.Row
10 import androidx.compose.foundation.layout.Spacer
11 import androidx.compose.foundation.layout.fillMaxSize
12 import androidx.compose.foundation.layout.fillMaxWidth
13 import androidx.compose.foundation.layout.height
14 import androidx.compose.foundation.layout.padding
15 import androidx.compose.foundation.layout.size
16 import androidx.compose.foundation.layout.width
17 import androidx.compose.foundation.lazy.LazyColumn
18 import androidx.compose.foundation.lazy.items
19 import androidx.compose.foundation.shape.RoundedCornerShape
20 import androidx.compose.foundation.text.KeyboardOptions
21 import androidx.compose.material.icons.Icons
22 import androidx.compose.material.icons.automirrored.filled.ArrowBack
23 import androidx.compose.material.icons.filled.AccountBalanceWallet
24 import androidx.compose.material.icons.filled.ArrowDropDown
25 import androidx.compose.material.icons.filled.Check
26 import androidx.compose.material.icons.filled.Clear
27 import androidx.compose.material3.TextButton
28 import androidx.compose.material.icons.filled.CurrencyRupee
29 import androidx.compose.material.icons.filled.Info
30 import androidx.compose.material.icons.filled.Payment
31 import androidx.compose.material.icons.filled.Person
32 import androidx.compose.material.icons.filled.Print
33 import androidx.compose.material.icons.filled.QrCode
34 import androidx.compose.material.icons.filled.Share
35 import androidx.compose.material3.AlertDialog
36 import androidx.compose.material3.Button
37 import androidx.compose.material3.ButtonDefaults
38 import androidx.compose.material3.Card
39 import androidx.compose.material3.CardDefaults
40 import androidx.compose.material3.DropdownMenu
41 import androidx.compose.material3.DropdownMenuItem
42 import androidx.compose.material3.ExperimentalMaterial3Api
43 import androidx.compose.material3.FilterChip
44 import androidx.compose.material3.FilterChipDefaults
45 import androidx.compose.material3.Icon
46 import androidx.compose.material3.IconButton
47 import androidx.compose.material3.MaterialTheme
48 import androidx.compose.material3.OutlinedButton
49 import androidx.compose.material3.OutlinedTextField
50 import androidx.compose.material3.OutlinedTextFieldDefaults
51 import androidx.compose.material3.Scaffold
52 import androidx.compose.material3.SnackbarHost
53 import androidx.compose.material3.SnackbarHostState
54 import androidx.compose.material3.Surface
55 import androidx.compose.material3.Text
56 import androidx.compose.material3.TextButton
57 import androidx.compose.material3.TopAppBar
58 import androidx.compose.material3.TopAppBarDefaults
59 import androidx.compose.runtime.Composable
60 import androidx.compose.runtime.collectAsState
61 import androidx.compose.runtime.getValue
62 import androidx.compose.runtime.mutableStateOf
63 import androidx.compose.runtime.remember
64 import androidx.compose.runtime.rememberCoroutineScope
65 import androidx.compose.runtime.setValue
66 import androidx.compose.ui.Alignment
67 import androidx.compose.ui.Modifier
68 import androidx.compose.ui.draw.clip
69 import androidx.compose.ui.graphics.Color
70 import androidx.compose.ui.res.stringResource
71 import androidx.compose.ui.text.font.FontWeight
72 import androidx.compose.ui.text.input.KeyboardType
73 import androidx.compose.ui.unit.dp
74 import androidx.compose.ui.unit.sp
75 import com.alieducational.app.R
76 import com.alieducational.app.data.local.entity.CustomerEntity
77 import com.alieducational.app.domain.calculation.CostCalculator
78 import com.alieducational.app.ui.AliViewModel
79 import kotlinx.coroutines.launch
80
81 @OptIn(ExperimentalMaterial3Api::class)
82 @Composable
```

```
fun PaymentScreen(
    viewModel: AliViewModel,
    onNavigateBack: () -> Unit,
    onBillSaved: (String) -> Unit,
    modifier: Modifier = Modifier
) {
    val billingState by viewModel.billingState.collectAsState()
    val allCustomers by viewModel.allCustomers.collectAsState()
    val scope = rememberCoroutineScope()
    val snackbarHostState = remember { SnackbarHostState() }

    var isSubmitting by remember { mutableStateOf(false) }
    var showCustomerDialog by remember { mutableStateOf(false) }
    var showReceiptDropdown by remember { mutableStateOf(false) }

    val netTotalPaise = billingState.netTotalPaise
    val netTotalRupeesStr = CostCalculator.formatPaiseToRupees(netTotalPaise)

    // Calculate change or short amount
    val cashReceivedPaise = billingState.cashReceivedPaise
    val isCash = billingState.paymentMethod == "CASH"
    val changePaise = if (isCash && cashReceivedPaise > netTotalPaise) cashReceivedPaise - netTotalPaise else 0L
    val shortPaise = if (isCash && cashReceivedPaise in 1 until netTotalPaise) netTotalPaise - cashReceivedPaise else 0L

    Scaffold(
        modifier = modifier.fillMaxSize(),
        snackbarHost = { SnackbarHost(snackbarHostState) },
        topBar = {
            TopAppBar(
                title = {
                    Text(
                        text = stringResource(R.string.payment_title),
                        style = MaterialTheme.typography.titleLarge.copy(fontWeight = FontWeight.Bold)
                    )
                },
                navigationIcon = {
                    IconButton(onClick = onNavigateBack) {
                        Icon(
                            imageVector = Icons.AutoMirrored.Filled.ArrowBack,
                            contentDescription = "Back"
                        )
                    }
                },
                colors = TopAppBarDefaults.topAppBarColors(
                    containerColor = MaterialTheme.colorScheme.background
                )
            )
        },
        bottomBar = {
            Surface(
                color = MaterialTheme.colorScheme.surface,
                tonalElevation = 8.dp,
                shadowElevation = 8.dp
            ) {
                Column(
                    modifier = Modifier
                        .fillMaxWidth()
                        .padding(horizontal = 16.dp, vertical = 12.dp)
                ) {
                    val canSave = when {
                        isSubmitting -> false
                        billingState.items.isEmpty() -> false
                        billingState.paymentMethod == "CREDIT" && billingState.customer == null -> false
                        billingState.paymentMethod == "OTHER" && shortPaise > 0 && billingState.customer == null -> false
                        else -> true
                    }

                    Button(
                        onClick = {
                            if (!canSave) return@Button
                            isSubmitting = true
                            viewModel.postCurrentBill(
                                onSuccess = { sale ->
                                    isSubmitting = false
                                    onBillSaved(sale.receiptNumber)
                                },
                                onError = { errorMsg ->
                                    isSubmitting = false
                                    scope.launch {
                                        snackbarHostState.showSnackbar(errorMsg)
                                    }
                                }
                            )
                        },
                        enabled = canSave,
```

```
                        modifier = Modifier
                            .fillMaxWidth()
                            .height(54.dp),
                        shape = RoundedCornerShape(12.dp),
                        colors = ButtonDefaults.buttonColors(
                            containerColor = MaterialTheme.colorScheme.primary,
                            contentColor = MaterialTheme.colorScheme.onPrimary
                        )
                    ) {
                        Text(
                            text = stringResource(R.string.payment_save_bill, netTotalRupeesStr),
                            style = MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.Bold)
                        )
                    }
                }
            }
        }
    ) { innerPadding ->
        LazyColumn(
            modifier = Modifier
                .fillMaxSize()
                .padding(innerPadding)
                .padding(horizontal = 16.dp),
            verticalArrangement = Arrangement.spacedBy(14.dp)
        ) {
            // Bill Summary Banner
            item {
                Card(
                    colors = CardDefaults.cardColors(
                        containerColor = MaterialTheme.colorScheme.surface
                    ),
                    shape = RoundedCornerShape(14.dp),
                    modifier = Modifier.fillMaxWidth()
                ) {
                    Column(
                        modifier = Modifier
                            .fillMaxWidth()
                            .padding(16.dp)
                    ) {
                        Row(
                            modifier = Modifier.fillMaxWidth(),
                            horizontalArrangement = Arrangement.SpaceBetween,
                            verticalAlignment = Alignment.CenterVertically
                        ) {
                            Column {
                                Text(
                                    text = stringResource(R.string.payment_summary),
                                    style = MaterialTheme.typography.labelMedium,
                                    color = MaterialTheme.colorScheme.onSurfaceVariant
                                )
                                Text(
                                    text = stringResource(R.string.home_items_count,
    billingState.totalItemCount),
                                    style = MaterialTheme.typography.bodySmall,
                                    color = MaterialTheme.colorScheme.onSurfaceVariant
                                )
                            }

                            Text(
                                text = netTotalRupeesStr,
                                style = MaterialTheme.typography.headlineMedium.copy(
                                    fontWeight = FontWeight.Bold,
                                    fontSize = 28.sp
                                ),
                                color = MaterialTheme.colorScheme.primary
                            )
                        }

                        Spacer(modifier = Modifier.height(12.dp))
                        Box(
                            modifier = Modifier
                                .fillMaxWidth()
                                .height(1.dp)
                                .background(MaterialTheme.colorScheme.outline.copy(alpha = 0.2f))
                        )
                        Spacer(modifier = Modifier.height(8.dp))

                        // Customer info
                        Row(
                            modifier = Modifier
                                .fillMaxWidth()
                                .clickable { showCustomerDialog = true },
                            horizontalArrangement = Arrangement.SpaceBetween,
                            verticalAlignment = Alignment.CenterVertically
                        ) {
                            Row(verticalAlignment = Alignment.CenterVertically) {
                                Icon(
                                    imageVector = Icons.Filled.Person,
```

```
                                    contentDescription = null,
                                    tint = MaterialTheme.colorScheme.primary,
                                    modifier = Modifier.size(18.dp)
                                )
                                Spacer(modifier = Modifier.width(6.dp))
                                Text(
                                    text = billingState.customer?.name
                                        ?: stringResource(R.string.review_walk_in),
                                    style = MaterialTheme.typography.bodyMedium.copy(fontWeight =
FontWeight.SemiBold),
                                    color = MaterialTheme.colorScheme.onSurface
                                )
                            }
                            Text(
                                text = stringResource(R.string.review_select_customer),
                                style = MaterialTheme.typography.labelSmall.copy(fontWeight =
FontWeight.Bold),
                                color = MaterialTheme.colorScheme.primary
                            )
                        }
                    }
                }
            }

            // Payment Method Selector Chips (Responsive 2-row layout avoiding crowding)
            item {
                Text(
                    text = stringResource(R.string.payment_method),
                    style = MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.Bold),
                    color = MaterialTheme.colorScheme.onBackground
                )

                Spacer(modifier = Modifier.height(8.dp))

                // Row 1: Cash & UPI
                Row(
                    modifier = Modifier.fillMaxWidth(),
                    horizontalArrangement = Arrangement.spacedBy(8.dp)
                ) {
                    FilterChip(
                        selected = billingState.paymentMethod == "CASH",
                        onClick = { viewModel.setBillingPaymentMethod("CASH") },
                        label = { Text(stringResource(R.string.payment_cash), fontWeight =
FontWeight.SemiBold) },
                        leadingIcon = {
                            Icon(
                                imageVector = Icons.Filled.CurrencyRupee,
                                contentDescription = null,
                                modifier = Modifier.size(16.dp)
                            )
                        },
                        modifier = Modifier.weight(1f)
                    )

                    FilterChip(
                        selected = billingState.paymentMethod == "UPI",
                        onClick = { viewModel.setBillingPaymentMethod("UPI") },
                        label = { Text(stringResource(R.string.payment_upi), fontWeight =
FontWeight.SemiBold) },
                        leadingIcon = {
                            Icon(
                                imageVector = Icons.Filled.QrCode,
                                contentDescription = null,
                                modifier = Modifier.size(16.dp)
                            )
                        },
                        modifier = Modifier.weight(1f)
                    )
                }

                Spacer(modifier = Modifier.height(6.dp))

                // Row 2: Customer Credit (Full width to avoid text crowding)
                val customerName = billingState.customer?.name
                val creditLabel = if (customerName.isNullOrBlank()) {
                    stringResource(R.string.payment_credit)
                } else {
                    "Customer Credit ($customerName)"
                }
                FilterChip(
                    selected = billingState.paymentMethod == "CREDIT",
                    onClick = {
                        viewModel.setBillingPaymentMethod("CREDIT")
                        if (billingState.customer == null) {
                            showCustomerDialog = true
                        }
                    },
                    label = { Text(creditLabel, fontWeight = FontWeight.SemiBold, maxLines = 1) },
```

```
                    leadingIcon = {
                        Icon(
                            imageVector = Icons.Filled.AccountBalanceWallet,
                            contentDescription = null,
                            modifier = Modifier.size(16.dp)
                        )
                    },
                    modifier = Modifier.fillMaxWidth()
                )
            }

            // Cash Tender Details (when Cash selected)
            if (isCash) {
                item {
                    val isAmountValid = remember(billingState.cashReceivedStr) {
                        billingState.cashReceivedStr.isBlank() ||
CostCalculator.parseRupeesToPaise(billingState.cashReceivedStr) != null
                    }

                    Card(
                        colors = CardDefaults.cardColors(
                            containerColor = MaterialTheme.colorScheme.surface
                        ),
                        shape = RoundedCornerShape(14.dp),
                        modifier = Modifier.fillMaxWidth()
                    ) {
                        Column(
                            modifier = Modifier
                                .fillMaxWidth()
                                .padding(16.dp),
                            verticalArrangement = Arrangement.spacedBy(12.dp)
                        ) {
                            Row(
                                modifier = Modifier.fillMaxWidth(),
                                horizontalArrangement = Arrangement.SpaceBetween,
                                verticalAlignment = Alignment.CenterVertically
                            ) {
                                Text(
                                    text = stringResource(R.string.payment_cash_received),
                                    style = MaterialTheme.typography.titleMedium.copy(fontWeight =
FontWeight.Bold),
                                    color = MaterialTheme.colorScheme.onSurface
                                )
                                if (billingState.cashReceivedStr.isNotBlank()) {
                                    TextButton(onClick = { viewModel.setCashReceived("") }) {
                                        Text("Clear", color = MaterialTheme.colorScheme.primary, style =
MaterialTheme.typography.labelSmall)
                                    }
                                }
                            }

                            OutlinedTextField(
                                value = billingState.cashReceivedStr,
                                onValueChange = { input ->
                                    viewModel.setCashReceived(input)
                                },
                                placeholder = { Text(netTotalRupeesStr) },
                                leadingIcon = {
                                    Icon(
                                        imageVector = Icons.Filled.CurrencyRupee,
                                        contentDescription = null,
                                        tint = MaterialTheme.colorScheme.primary
                                    )
                                },
                                trailingIcon = {
                                    if (billingState.cashReceivedStr.isNotEmpty()) {
                                        IconButton(onClick = { viewModel.setCashReceived("") }) {
                                            Icon(Icons.Filled.Clear, contentDescription = "Clear")
                                        }
                                    }
                                },
                                isError = !isAmountValid,
                                supportingText = if (!isAmountValid) {
                                    { Text("Enter valid amount (e.g. 123.45, max 2 decimals)", color =
MaterialTheme.colorScheme.error) }
                                } else null,
                                keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Decimal),
                                singleLine = true,
                                modifier = Modifier.fillMaxWidth(),
                                colors = OutlinedTextFieldDefaults.colors(
                                    focusedBorderColor = MaterialTheme.colorScheme.primary,
                                    unfocusedBorderColor = MaterialTheme.colorScheme.outline.copy(alpha =
0.5f)
                                )
                            )

                            // Quick Tender Buttons: 2 rows with adequate minimum touch target and no
vertical character wrapping
```

```
                            // Row 1: Exact (payable) and Set ₹500
                            Row(
                                modifier = Modifier.fillMaxWidth(),
                                horizontalArrangement = Arrangement.spacedBy(8.dp)
                            ) {
                                OutlinedButton(
                                    onClick = {
                                        viewModel.setCashReceived(netTotalRupeesStr.replace("₹", "").trim())
                                    },
                                    shape = RoundedCornerShape(8.dp),
                                    modifier = Modifier.weight(1f).height(44.dp)
                                ) {
                                    Text(
                                        text = "Exact ($netTotalRupeesStr)",
                                        style = MaterialTheme.typography.labelMedium.copy(fontWeight =
FontWeight.Bold),
                                        maxLines = 1
                                    )
                                }

                                OutlinedButton(
                                    onClick = {
                                        viewModel.setCashReceived("500.00")
                                    },
                                    shape = RoundedCornerShape(8.dp),
                                    modifier = Modifier.weight(1f).height(44.dp)
                                ) {
                                    Text(
                                        text = "₹500",
                                        style = MaterialTheme.typography.labelMedium.copy(fontWeight =
FontWeight.Bold),
                                        maxLines = 1
                                    )
                                }
                            }

                            // Row 2: Additive Increments (+₹10, +₹50, +₹100)
                            Row(
                                modifier = Modifier.fillMaxWidth(),
                                horizontalArrangement = Arrangement.spacedBy(8.dp)
                            ) {
                                listOf(10, 50, 100).forEach { inc ->
                                    OutlinedButton(
                                        onClick = {
                                            val currentPaise =
CostCalculator.parseRupeesToPaise(billingState.cashReceivedStr) ?: netTotalPaise
                                            val newPaise = currentPaise + (inc * 100L)
                                            val rupeesStr =
CostCalculator.formatPaiseToRupees(newPaise).replace("₹", "")
                                            viewModel.setCashReceived(rupeesStr)
                                        },
                                        shape = RoundedCornerShape(8.dp),
                                        modifier = Modifier.weight(1f).height(44.dp)
                                    ) {
                                        Text(
                                            text = "+₹$inc",
                                            style = MaterialTheme.typography.labelMedium.copy(fontWeight =
FontWeight.Bold),
                                            maxLines = 1
                                        )
                                    }
                                }
                            }

                            // Change Calculation Display
                            if (changePaise > 0L) {
                                Card(
                                    colors = CardDefaults.cardColors(
                                        containerColor =
MaterialTheme.colorScheme.primaryContainer.copy(alpha = 0.5f)
                                    ),
                                    shape = RoundedCornerShape(10.dp),
                                    modifier = Modifier.fillMaxWidth()
                                ) {
                                    Row(
                                        modifier = Modifier
                                            .fillMaxWidth()
                                            .padding(14.dp),
                                        horizontalArrangement = Arrangement.SpaceBetween,
                                        verticalAlignment = Alignment.CenterVertically
                                    ) {
                                        Text(
                                            text = stringResource(
                                                R.string.payment_change_to_give,
                                                CostCalculator.formatPaiseToRupees(changePaise)
                                            ),
                                            style = MaterialTheme.typography.titleMedium.copy(
                                                fontWeight = FontWeight.Bold,
                                                fontSize = 17.sp
```

```
                                        ),
                                        color = MaterialTheme.colorScheme.primary
                                    )
                                    Icon(
                                        imageVector = Icons.Filled.Check,
                                        contentDescription = null,
                                        tint = MaterialTheme.colorScheme.primary
                                    )
                                }
                            }
                        } else if (shortPaise > 0L) {
                            Card(
                                colors = CardDefaults.cardColors(
                                    containerColor = MaterialTheme.colorScheme.errorContainer.copy(alpha = 0.4f)
                                ),
                                shape = RoundedCornerShape(10.dp),
                                modifier = Modifier.fillMaxWidth()
                            ) {
                                Column(
                                    modifier = Modifier
                                        .fillMaxWidth()
                                        .padding(12.dp),
                                    verticalArrangement = Arrangement.spacedBy(8.dp)
                                ) {
                                    Text(
                                        text = stringResource(
                                            R.string.payment_short_amount,
                                            CostCalculator.formatPaiseToRupees(shortPaise)
                                        ),
                                        style = MaterialTheme.typography.titleSmall.copy(fontWeight = FontWeight.Bold),
                                        color = MaterialTheme.colorScheme.error
                                    )

                                    Row(
                                        modifier = Modifier.fillMaxWidth(),
                                        horizontalArrangement = Arrangement.spacedBy(8.dp)
                                    ) {
                                        OutlinedButton(
                                            onClick = {
                                                // Treat remaining difference as discount
                                                val currentDiscount = billingState.discountValuePaise
                                                viewModel.setWholeBillDiscountFixed(currentDiscount + shortPaise)
                                            },
                                            modifier = Modifier.weight(1f),
                                            shape = RoundedCornerShape(8.dp)
                                        ) {
                                            Text(
                                                text = stringResource(R.string.payment_treat_as_discount),
                                                style = MaterialTheme.typography.labelSmall
                                            )
                                        }

                                        OutlinedButton(
                                            onClick = {
                                                if (billingState.customer == null) {
                                                    showCustomerDialog = true
                                                } else {
                                                    // It will record as remaining credit owed automatically
                                                }
                                            },
                                            modifier = Modifier.weight(1f),
                                            shape = RoundedCornerShape(8.dp)
                                        ) {
                                            Text(
                                                text = stringResource(R.string.payment_treat_as_credit),
                                                style = MaterialTheme.typography.labelSmall
                                            )
                                        }
                                    }
                                }
                            }
                        }
                    }
                }
            }

            // Warning if Credit payment selected without a customer
            if (billingState.paymentMethod == "CREDIT" && billingState.customer == null) {
                item {
                    Card(
                        colors = CardDefaults.cardColors(
                            containerColor = MaterialTheme.colorScheme.errorContainer.copy(alpha = 0.5f)
```

```
                        ),
                        shape = RoundedCornerShape(10.dp),
                        modifier = Modifier.fillMaxWidth()
                    ) {
                        Row(
                            modifier = Modifier
                                .fillMaxWidth()
                                .padding(12.dp),
                            verticalAlignment = Alignment.CenterVertically,
                            horizontalArrangement = Arrangement.SpaceBetween
                        ) {
                            Text(
                                text = stringResource(R.string.payment_credit_req_customer),
                                style = MaterialTheme.typography.bodySmall.copy(fontWeight =
FontWeight.Bold),
                                color = MaterialTheme.colorScheme.error,
                                modifier = Modifier.weight(1f)
                            )
                            Spacer(modifier = Modifier.width(8.dp))
                            Button(
                                onClick = { showCustomerDialog = true },
                                colors = ButtonDefaults.buttonColors(
                                    containerColor = MaterialTheme.colorScheme.error
                                ),
                                shape = RoundedCornerShape(8.dp)
                            ) {
                                Text(stringResource(R.string.review_customer))
                            }
                        }
                    }
                }
            }

            // Receipt Action Dropdown Option
            item {
                Card(
                    colors = CardDefaults.cardColors(
                        containerColor = MaterialTheme.colorScheme.surface
                    ),
                    shape = RoundedCornerShape(12.dp),
                    modifier = Modifier.fillMaxWidth()
                ) {
                    Column(
                        modifier = Modifier
                            .fillMaxWidth()
                            .padding(14.dp)
                    ) {
                        Text(
                            text = stringResource(R.string.payment_receipt_action),
                            style = MaterialTheme.typography.labelMedium,
                            color = MaterialTheme.colorScheme.onSurfaceVariant
                        )

                        Spacer(modifier = Modifier.height(6.dp))

                        Box(modifier = Modifier.fillMaxWidth()) {
                            Row(
                                modifier = Modifier
                                    .fillMaxWidth()
                                    .clip(RoundedCornerShape(8.dp))
                                    .background(MaterialTheme.colorScheme.surfaceVariant)
                                    .clickable { showReceiptDropdown = true }
                                    .padding(horizontal = 12.dp, vertical = 10.dp),
                                horizontalArrangement = Arrangement.SpaceBetween,
                                verticalAlignment = Alignment.CenterVertically
                            ) {
                                val actionLabel = when (billingState.receiptAction) {
                                    "PREVIEW" -> stringResource(R.string.payment_action_preview)
                                    "PRINT" -> stringResource(R.string.payment_action_print)
                                    "SHARE" -> stringResource(R.string.payment_action_share)
                                    else -> stringResource(R.string.payment_action_none)
                                }
                                Text(
                                    text = actionLabel,
                                    style = MaterialTheme.typography.bodyMedium.copy(fontWeight =
FontWeight.SemiBold),
                                    color = MaterialTheme.colorScheme.onSurface
                                )
                                Icon(
                                    imageVector = Icons.Filled.ArrowDropDown,
                                    contentDescription = null
                                )
                            }

                            DropdownMenu(
                                expanded = showReceiptDropdown,
                                onDismissRequest = { showReceiptDropdown = false }
                            ) {
```

```
                                DropdownMenuItem(
                                    text = { Text(stringResource(R.string.payment_action_none)) },
                                    onClick = {
                                        viewModel.setReceiptAction("NONE")
                                        showReceiptDropdown = false
                                    }
                                )
                                DropdownMenuItem(
                                    text = { Text(stringResource(R.string.payment_action_preview)) },
                                    onClick = {
                                        viewModel.setReceiptAction("PREVIEW")
                                        showReceiptDropdown = false
                                    }
                                )
                                DropdownMenuItem(
                                    text = { Text(stringResource(R.string.payment_action_print)) },
                                    onClick = {
                                        viewModel.setReceiptAction("PRINT")
                                        showReceiptDropdown = false
                                    }
                                )
                                DropdownMenuItem(
                                    text = { Text(stringResource(R.string.payment_action_share)) },
                                    onClick = {
                                        viewModel.setReceiptAction("SHARE")
                                        showReceiptDropdown = false
                                    }
                                )
                            }
                        }
                    }
                }
                Spacer(modifier = Modifier.height(16.dp))
            }
        }
    }

    // Customer Picker Dialog
    if (showCustomerDialog) {
        CustomerSelectionDialog(
            currentCustomer = billingState.customer,
            allCustomers = allCustomers,
            onSelectCustomer = { customer ->
                viewModel.setBillingCustomer(customer)
                showCustomerDialog = false
            },
            onCreateCustomer = { name, phone, notes ->
                val newCust = CustomerEntity(
                    name = name,
                    phone = phone.ifBlank { null },
                    notes = notes.ifBlank { null }
                )
                viewModel.addCustomer(newCust) {
                    viewModel.setBillingCustomer(newCust)
                    showCustomerDialog = false
                }
            },
            onDismiss = { showCustomerDialog = false }
        )
    }
}
```

app\src\main\java\com\alieducational\app\ui\billing\ReviewBillScreen.kt

```
package com.alieducational.app.ui.billing

import androidx.compose.foundation.background
import androidx.compose.foundation.border
import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.foundation.shape.CircleShape
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.automirrored.filled.ArrowBack
import androidx.compose.material.icons.filled.Add
import androidx.compose.material.icons.filled.ArrowDropDown
import androidx.compose.material.icons.filled.Check
import androidx.compose.material.icons.filled.Close
import androidx.compose.material.icons.filled.DeleteOutline
import androidx.compose.material.icons.filled.Edit
import androidx.compose.material.icons.filled.Info
import androidx.compose.material.icons.filled.PauseCircleOutline
import androidx.compose.material.icons.filled.Person
import androidx.compose.material.icons.filled.QrCodeScanner
import androidx.compose.material.icons.filled.Remove
import com.alieducational.app.ui.BillingDraftItem
import androidx.compose.material3.AlertDialog
import androidx.compose.material3.Button
import androidx.compose.material3.ButtonDefaults
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.DropdownMenu
import androidx.compose.material3.DropdownMenuItem
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.FilterChip
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedButton
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.material3.TextButton
import androidx.compose.material3.TopAppBar
import androidx.compose.material3.TopAppBarDefaults
import androidx.compose.runtime.Composable
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.res.stringResource
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.alieducational.app.R
import com.alieducational.app.data.local.entity.CustomerEntity
import com.alieducational.app.domain.calculation.CostCalculator
import com.alieducational.app.ui.AliViewModel

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun ReviewBillScreen(
    viewModel: AliViewModel,
    onNavigateBack: () -> Unit,
    onNavigateToScanner: () -> Unit,
    onNavigateToPayment: () -> Unit,
```

```
    modifier: Modifier = Modifier
) {
    val billingState by viewModel.billingState.collectAsState()
    val allCustomers by viewModel.allCustomers.collectAsState()
    val appLanguage by viewModel.appLanguage.collectAsState()

    var showCustomerDialog by remember { mutableStateOf(false) }
    var showDiscountDialog by remember { mutableStateOf(false) }
    var showHeldBillsDialog by remember { mutableStateOf(false) }
    var itemToOverridePrice by remember { mutableStateOf<BillingDraftItem?>(null) }

    Scaffold(
        modifier = modifier.fillMaxSize(),
        topBar = {
            TopAppBar(
                title = {
                    Text(
                        text = stringResource(R.string.review_title),
                        style = MaterialTheme.typography.titleLarge.copy(fontWeight = FontWeight.Bold)
                    )
                },
                navigationIcon = {
                    IconButton(onClick = onNavigateBack) {
                        Icon(
                            imageVector = Icons.AutoMirrored.Filled.ArrowBack,
                            contentDescription = "Back"
                        )
                    }
                },
                actions = {
                    IconButton(onClick = { showHeldBillsDialog = true }) {
                        Icon(
                            imageVector = Icons.Filled.PauseCircleOutline,
                            contentDescription = stringResource(R.string.billing_hold_bill),
                            tint = MaterialTheme.colorScheme.primary
                        )
                    }
                },
                colors = TopAppBarDefaults.topAppBarColors(
                    containerColor = MaterialTheme.colorScheme.background
                )
            )
        },
        bottomBar = {
            Surface(
                color = MaterialTheme.colorScheme.surface,
                tonalElevation = 8.dp,
                shadowElevation = 8.dp
            ) {
                Column(
                    modifier = Modifier
                        .fillMaxWidth()
                        .padding(horizontal = 16.dp, vertical = 12.dp)
                ) {
                    Button(
                        onClick = onNavigateToPayment,
                        enabled = billingState.items.isNotEmpty(),
                        modifier = Modifier
                            .fillMaxWidth()
                            .height(52.dp),
                        shape = RoundedCornerShape(12.dp),
                        colors = ButtonDefaults.buttonColors(
                            containerColor = MaterialTheme.colorScheme.primary,
                            contentColor = MaterialTheme.colorScheme.onPrimary
                        )
                    ) {
                        Text(
                            text = stringResource(R.string.review_continue_payment),
                            style = MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.Bold)
                        )
                    }
                }
            }
        }
    ) { innerPadding ->
        LazyColumn(
            modifier = Modifier
                .fillMaxSize()
                .padding(innerPadding)
                .padding(horizontal = 16.dp),
            verticalArrangement = Arrangement.spacedBy(14.dp)
        ) {
            // Customer Selector Card
            item {
                Card(
                    modifier = Modifier
                        .fillMaxWidth()
                        .clickable { showCustomerDialog = true },
```

```
                    colors = CardDefaults.cardColors(
                        containerColor = MaterialTheme.colorScheme.surfaceVariant
                    ),
                    shape = RoundedCornerShape(12.dp)
                ) {
                    Row(
                        modifier = Modifier
                            .fillMaxWidth()
                            .padding(14.dp),
                        verticalAlignment = Alignment.CenterVertically,
                        horizontalArrangement = Arrangement.SpaceBetween
                    ) {
                        Row(
                            verticalAlignment = Alignment.CenterVertically,
                            modifier = Modifier.weight(1f)
                        ) {
                            Box(
                                modifier = Modifier
                                    .size(36.dp)
                                    .clip(CircleShape)
                                    .background(MaterialTheme.colorScheme.primary.copy(alpha = 0.2f)),
                                contentAlignment = Alignment.Center
                            ) {
                                Icon(
                                    imageVector = Icons.Filled.Person,
                                    contentDescription = null,
                                    tint = MaterialTheme.colorScheme.primary
                                )
                            }
                            Spacer(modifier = Modifier.width(10.dp))
                            Column {
                                Text(
                                    text = stringResource(R.string.review_customer),
                                    style = MaterialTheme.typography.labelSmall,
                                    color = MaterialTheme.colorScheme.onSurfaceVariant
                                )
                                Text(
                                    text = billingState.customer?.name
                                        ?: stringResource(R.string.review_walk_in),
                                    style = MaterialTheme.typography.titleMedium.copy(fontWeight =
FontWeight.Bold),
                                    color = MaterialTheme.colorScheme.onSurface
                                )
                                if (billingState.customer?.phone?.isNotBlank() == true) {
                                    Text(
                                        text = billingState.customer?.phone.orEmpty(),
                                        style = MaterialTheme.typography.bodySmall,
                                        color = MaterialTheme.colorScheme.onSurfaceVariant
                                    )
                                }
                            }
                        }

                        Text(
                            text = stringResource(R.string.review_select_customer),
                            style = MaterialTheme.typography.bodySmall.copy(fontWeight = FontWeight.Bold),
                            color = MaterialTheme.colorScheme.primary
                        )
                    }
                }
            }

            // Items Header
            item {
                Row(
                    modifier = Modifier.fillMaxWidth(),
                    horizontalArrangement = Arrangement.SpaceBetween,
                    verticalAlignment = Alignment.CenterVertically
                ) {
                    Text(
                        text = stringResource(R.string.review_items),
                        style = MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.Bold),
                        color = MaterialTheme.colorScheme.onBackground
                    )
                    Text(
                        text = stringResource(R.string.home_items_count, billingState.totalItemCount),
                        style = MaterialTheme.typography.bodySmall,
                        color = MaterialTheme.colorScheme.onSurfaceVariant
                    )
                }
            }

            // List of Draft Items
            if (billingState.items.isEmpty()) {
                item {
                    Card(
                        colors = CardDefaults.cardColors(
                            containerColor = MaterialTheme.colorScheme.surfaceVariant.copy(alpha = 0.5f)
```

```
                        ),
                        shape = RoundedCornerShape(12.dp),
                        modifier = Modifier.fillMaxWidth()
                    ) {
                        Column(
                            modifier = Modifier
                                .fillMaxWidth()
                                .padding(24.dp),
                            horizontalAlignment = Alignment.CenterHorizontally
                        ) {
                            Text(
                                text = stringResource(R.string.billing_no_items),
                                style = MaterialTheme.typography.titleMedium.copy(fontWeight =
FontWeight.Bold),
                                color = MaterialTheme.colorScheme.onSurface
                            )
                            Spacer(modifier = Modifier.height(6.dp))
                            Text(
                                text = stringResource(R.string.billing_no_items_desc),
                                style = MaterialTheme.typography.bodySmall,
                                color = MaterialTheme.colorScheme.onSurfaceVariant
                            )
                        }
                    }
                }
            } else {
                items(billingState.items, key = { it.lineId }) { item ->
                    Card(
                        colors = CardDefaults.cardColors(
                            containerColor = MaterialTheme.colorScheme.surface
                        ),
                        shape = RoundedCornerShape(12.dp),
                        modifier = Modifier.fillMaxWidth()
                    ) {
                        Column(
                            modifier = Modifier
                                .fillMaxWidth()
                                .padding(12.dp)
                        ) {
                            // Row 1: Product / Service Name, Line Total, Delete Button
                            Row(
                                modifier = Modifier.fillMaxWidth(),
                                horizontalArrangement = Arrangement.SpaceBetween,
                                verticalAlignment = Alignment.Top
                            ) {
                                Column(modifier = Modifier.weight(1f)) {
                                    Text(
                                        text = if (appLanguage == "ur" && !item.urduName.isNullOrBlank())
item.urduName else item.productName,
                                        style = MaterialTheme.typography.titleMedium.copy(fontWeight =
FontWeight.Bold),
                                        color = MaterialTheme.colorScheme.onSurface
                                    )
                                    Row(verticalAlignment = Alignment.CenterVertically) {
                                        val unitStr = if (item.lineType == "SERVICE") {
                                            item.unitLabel ?: "unit"
                                        } else if (item.unitType == "PACK") {
                                            stringResource(R.string.billing_pack)
                                        } else {
                                            stringResource(R.string.billing_piece)
                                        }
                                        Text(
                                            text = "@
${CostCalculator.formatPaiseToRupees(item.unitPricePaise)} / $unitStr",
                                            style = MaterialTheme.typography.bodySmall,
                                            color = MaterialTheme.colorScheme.onSurfaceVariant
                                        )
                                        Spacer(modifier = Modifier.width(6.dp))
                                        IconButton(
                                            onClick = { itemToOverridePrice = item },
                                            modifier = Modifier.size(24.dp)
                                        ) {
                                            Icon(
                                                imageVector = Icons.Filled.Edit,
                                                contentDescription = "Edit Price",
                                                tint = MaterialTheme.colorScheme.primary,
                                                modifier = Modifier.size(15.dp)
                                            )
                                        }
                                    }
                                }

                                Row(verticalAlignment = Alignment.CenterVertically) {
                                    Text(
                                        text = CostCalculator.formatPaiseToRupees(item.lineGrossPaise),
                                        style = MaterialTheme.typography.titleMedium.copy(fontWeight =
FontWeight.Bold),
                                        color = MaterialTheme.colorScheme.primary
```

```
                                    )
                                    IconButton(
                                        onClick = { viewModel.removeItemFromBill(item.lineId) },
                                        modifier = Modifier.size(36.dp)
                                    ) {
                                        Icon(
                                            imageVector = Icons.Filled.DeleteOutline,
                                            contentDescription = "Remove",
                                            tint = MaterialTheme.colorScheme.error,
                                            modifier = Modifier.size(20.dp)
                                        )
                                    }
                                }
                            }

                            Spacer(modifier = Modifier.height(10.dp))

                            // Row 2: Unit Type Switcher / Service Tag & Quantity Stepper
                            Row(
                                modifier = Modifier.fillMaxWidth(),
                                horizontalArrangement = Arrangement.SpaceBetween,
                                verticalAlignment = Alignment.CenterVertically
                            ) {
                                if (item.lineType == "PRODUCT") {
                                    // Unit Switcher (Piece vs Pack)
                                    Row(
                                        modifier = Modifier
                                            .clip(RoundedCornerShape(8.dp))
                                            .background(MaterialTheme.colorScheme.surfaceVariant)
                                            .padding(2.dp)
                                    ) {
                                        Box(
                                            modifier = Modifier
                                                .clip(RoundedCornerShape(6.dp))
                                                .background(
                                                    if (item.unitType == "PIECE")
MaterialTheme.colorScheme.primary else Color.Transparent
                                                )
                                                .clickable {
                                                    if (item.unitType != "PIECE") {
                                                        viewModel.updateDraftItemUnit(item.productId,
item.unitType, "PIECE")
                                                    }
                                                }
                                                .padding(horizontal = 10.dp, vertical = 6.dp)
                                        ) {
                                            Text(
                                                text = stringResource(R.string.billing_piece),
                                                style = MaterialTheme.typography.labelSmall.copy(fontWeight
= FontWeight.Bold),
                                                color = if (item.unitType == "PIECE")
MaterialTheme.colorScheme.onPrimary else MaterialTheme.colorScheme.onSurfaceVariant
                                            )
                                        }

                                        Box(
                                            modifier = Modifier
                                                .clip(RoundedCornerShape(6.dp))
                                                .background(
                                                    if (item.unitType == "PACK")
MaterialTheme.colorScheme.primary else Color.Transparent
                                                )
                                                .clickable {
                                                    if (item.unitType != "PACK") {
                                                        viewModel.updateDraftItemUnit(item.productId,
item.unitType, "PACK")
                                                    }
                                                }
                                                .padding(horizontal = 10.dp, vertical = 6.dp)
                                        ) {
                                            Text(
                                                text = "${stringResource(R.string.billing_pack)}
(${item.packSize})",
                                                style = MaterialTheme.typography.labelSmall.copy(fontWeight
= FontWeight.Bold),
                                                color = if (item.unitType == "PACK")
MaterialTheme.colorScheme.onPrimary else MaterialTheme.colorScheme.onSurfaceVariant
                                            )
                                        }
                                    }
                                } else {
                                    Surface(
                                        color = MaterialTheme.colorScheme.surfaceVariant,
                                        shape = RoundedCornerShape(8.dp)
                                    ) {
                                        Text(
                                            text = "Service: ${item.unitLabel ?: "unit"}",
                                            style = MaterialTheme.typography.labelSmall.copy(fontWeight =
```

```
    FontWeight.Bold),
                                            color = MaterialTheme.colorScheme.onSurfaceVariant,
                                            modifier = Modifier.padding(horizontal = 10.dp, vertical = 6.dp)
                                        )
                                    }
                                }

                                // Stepper: [-] Qty [+]
                                Row(
                                    verticalAlignment = Alignment.CenterVertically,
                                    modifier = Modifier
                                        .border(
                                            width = 1.dp,
                                            color = MaterialTheme.colorScheme.outline.copy(alpha = 0.3f),
                                            shape = RoundedCornerShape(8.dp)
                                        )
                                        .background(MaterialTheme.colorScheme.surfaceVariant.copy(alpha =
    0.3f))
                                        .padding(horizontal = 4.dp, vertical = 2.dp)
                                ) {
                                    IconButton(
                                        onClick = {
                                            viewModel.updateItemQuantity(item.lineId, item.quantity - 1)
                                        },
                                        modifier = Modifier.size(32.dp)
                                    ) {
                                        Icon(
                                            imageVector = Icons.Filled.Remove,
                                            contentDescription = "Decrease",
                                            modifier = Modifier.size(16.dp)
                                        )
                                    }

                                    Text(
                                        text = "${item.quantity}",
                                        style = MaterialTheme.typography.titleMedium.copy(fontWeight =
    FontWeight.Bold),
                                        modifier = Modifier.padding(horizontal = 12.dp)
                                    )

                                    IconButton(
                                        onClick = {
                                            viewModel.updateItemQuantity(item.lineId, item.quantity + 1)
                                        },
                                        modifier = Modifier.size(32.dp)
                                    ) {
                                        Icon(
                                            imageVector = Icons.Filled.Add,
                                            contentDescription = "Increase",
                                            modifier = Modifier.size(16.dp)
                                        )
                                    }
                                }
                            }
                        }
                    }
                }

                // Button: + Scan or add another item
                item {
                    OutlinedButton(
                        onClick = onNavigateToScanner,
                        modifier = Modifier
                            .fillMaxWidth()
                            .height(48.dp),
                        shape = RoundedCornerShape(10.dp)
                    ) {
                        Icon(
                            imageVector = Icons.Filled.QrCodeScanner,
                            contentDescription = null,
                            modifier = Modifier.size(18.dp)
                        )
                        Spacer(modifier = Modifier.width(8.dp))
                        Text(
                            text = stringResource(R.string.review_add_item),
                            style = MaterialTheme.typography.bodyMedium.copy(fontWeight = FontWeight.Bold)
                        )
                    }
                }

                // Bill Totals & Discount Card
                item {
                    Card(
                        colors = CardDefaults.cardColors(
                            containerColor = MaterialTheme.colorScheme.surface
                        ),
                        shape = RoundedCornerShape(14.dp),
                        modifier = Modifier.fillMaxWidth()
```

```
506                 ) {
507                     Column(
508                         modifier = Modifier
509                             .fillMaxWidth()
510                             .padding(16.dp),
511                         verticalArrangement = Arrangement.spacedBy(10.dp)
512                     ) {
513                         // Subtotal
514                         Row(
515                             modifier = Modifier.fillMaxWidth(),
516                             horizontalArrangement = Arrangement.SpaceBetween
517                         ) {
518                             Text(
519                                 text = stringResource(R.string.review_subtotal),
520                                 style = MaterialTheme.typography.bodyMedium,
521                                 color = MaterialTheme.colorScheme.onSurfaceVariant
522                             )
523                             Text(
524                                 text = CostCalculator.formatPaiseToRupees(billingState.grossTotalPaise),
525                                 style = MaterialTheme.typography.bodyLarge.copy(fontWeight =
    FontWeight.SemiBold),
526                                 color = MaterialTheme.colorScheme.onSurface
527                             )
528                         }
529
530                         // Discount Row
531                         Row(
532                             modifier = Modifier.fillMaxWidth(),
533                             horizontalArrangement = Arrangement.SpaceBetween,
534                             verticalAlignment = Alignment.CenterVertically
535                         ) {
536                             Text(
537                                 text = stringResource(R.string.review_discount),
538                                 style = MaterialTheme.typography.bodyMedium,
539                                 color = MaterialTheme.colorScheme.onSurfaceVariant
540                             )
541
542                             if (billingState.discountType == "NONE" || billingState.actualDiscountPaise ==
    0L) {
543                                 Text(
544                                     text = stringResource(R.string.review_add_discount),
545                                     style = MaterialTheme.typography.bodySmall.copy(fontWeight =
    FontWeight.Bold),
546                                     color = MaterialTheme.colorScheme.primary,
547                                     modifier = Modifier.clickable { showDiscountDialog = true }
548                                 )
549                             } else {
550                                 Row(verticalAlignment = Alignment.CenterVertically) {
551                                     Text(
552                                         text =
    "-${CostCalculator.formatPaiseToRupees(billingState.actualDiscountPaise)}",
553                                         style = MaterialTheme.typography.bodyLarge.copy(fontWeight =
    FontWeight.Bold),
554                                         color = MaterialTheme.colorScheme.primary
555                                     )
556                                     Spacer(modifier = Modifier.width(8.dp))
557                                     Text(
558                                         text = stringResource(R.string.review_remove_discount),
559                                         style = MaterialTheme.typography.labelSmall,
560                                         color = MaterialTheme.colorScheme.error,
561                                         modifier = Modifier.clickable { viewModel.clearBillDiscount() }
562                                     )
563                                 }
564                             }
565                         }
566
567                         Box(
568                             modifier = Modifier
569                                 .fillMaxWidth()
570                                 .height(1.dp)
571                                 .background(MaterialTheme.colorScheme.outline.copy(alpha = 0.2f))
572                         )
573
574                         // Net Total
575                         Row(
576                             modifier = Modifier.fillMaxWidth(),
577                             horizontalArrangement = Arrangement.SpaceBetween,
578                             verticalAlignment = Alignment.CenterVertically
579                         ) {
580                             Text(
581                                 text = stringResource(R.string.review_net_total),
582                                 style = MaterialTheme.typography.titleMedium.copy(fontWeight =
    FontWeight.Bold),
583                                 color = MaterialTheme.colorScheme.onSurface
584                             )
585                             Text(
586                                 text = CostCalculator.formatPaiseToRupees(billingState.netTotalPaise),
587                                 style = MaterialTheme.typography.headlineSmall.copy(
```

```
                                    fontWeight = FontWeight.Bold,
                                    fontSize = 24.sp
                                ),
                                color = MaterialTheme.colorScheme.primary
                            )
                        }
                    }
                }
            }

            // Stock Deduction Warning Notice Box
            item {
                Card(
                    colors = CardDefaults.cardColors(
                        containerColor = MaterialTheme.colorScheme.primaryContainer.copy(alpha = 0.35f)
                    ),
                    shape = RoundedCornerShape(10.dp),
                    modifier = Modifier.fillMaxWidth()
                ) {
                    Row(
                        modifier = Modifier
                            .fillMaxWidth()
                            .padding(12.dp),
                        verticalAlignment = Alignment.CenterVertically
                    ) {
                        Icon(
                            imageVector = Icons.Filled.Info,
                            contentDescription = null,
                            tint = MaterialTheme.colorScheme.primary,
                            modifier = Modifier.size(20.dp)
                        )
                        Spacer(modifier = Modifier.width(10.dp))
                        Text(
                            text = stringResource(R.string.review_notice),
                            style = MaterialTheme.typography.bodySmall,
                            color = MaterialTheme.colorScheme.onSurface
                        )
                    }
                }
                Spacer(modifier = Modifier.height(16.dp))
            }
        }
    }

    // Customer Selection Dialog
    if (showCustomerDialog) {
        CustomerSelectionDialog(
            currentCustomer = billingState.customer,
            allCustomers = allCustomers,
            onSelectCustomer = { customer ->
                viewModel.setBillingCustomer(customer)
                showCustomerDialog = false
            },
            onCreateCustomer = { name, phone, notes ->
                val newCust = CustomerEntity(
                    name = name,
                    phone = phone.ifBlank { null },
                    notes = notes.ifBlank { null }
                )
                viewModel.addCustomer(newCust) {
                    viewModel.setBillingCustomer(newCust)
                    showCustomerDialog = false
                }
            },
            onDismiss = { showCustomerDialog = false }
        )
    }

    // Whole-Bill Discount Dialog
    if (showDiscountDialog) {
        var discountMode by remember { mutableStateOf("PERCENT") } // "PERCENT" or "FIXED"
        var discountInput by remember { mutableStateOf("") }

        AlertDialog(
            onDismissRequest = { showDiscountDialog = false },
            title = {
                Text(
                    text = stringResource(R.string.review_discount_title),
                    style = MaterialTheme.typography.titleLarge.copy(fontWeight = FontWeight.Bold)
                )
            },
            text = {
                Column(
                    modifier = Modifier.fillMaxWidth(),
                    verticalArrangement = Arrangement.spacedBy(12.dp)
                ) {
                    Row(
                        modifier = Modifier.fillMaxWidth(),
```

```
                        horizontalArrangement = Arrangement.spacedBy(8.dp)
                    ) {
                        FilterChip(
                            selected = discountMode == "PERCENT",
                            onClick = { discountMode = "PERCENT" },
                            label = { Text(stringResource(R.string.review_discount_percent)) },
                            modifier = Modifier.weight(1f)
                        )
                        FilterChip(
                            selected = discountMode == "FIXED",
                            onClick = { discountMode = "FIXED" },
                            label = { Text(stringResource(R.string.review_discount_fixed)) },
                            modifier = Modifier.weight(1f)
                        )
                    }

                    OutlinedTextField(
                        value = discountInput,
                        onValueChange = { discountInput = it },
                        label = {
                            Text(
                                if (discountMode == "PERCENT")
                                    stringResource(R.string.review_discount_percent)
                                else
                                    stringResource(R.string.review_discount_fixed)
                            )
                        },
                        keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Number),
                        singleLine = true,
                        modifier = Modifier.fillMaxWidth()
                    )
                }
            },
            confirmButton = {
                Button(
                    onClick = {
                        val num = discountInput.trim().toDoubleOrNull() ?: 0.0
                        if (discountMode == "PERCENT") {
                            viewModel.setWholeBillDiscountPercent(num)
                        } else {
                            viewModel.setWholeBillDiscountFixed((num * 100).toLong())
                        }
                        showDiscountDialog = false
                    }
                ) {
                    Text(stringResource(R.string.review_apply))
                }
            },
            dismissButton = {
                TextButton(onClick = { showDiscountDialog = false }) {
                    Text(stringResource(R.string.review_cancel))
                }
            }
        )
    }

    if (showHeldBillsDialog) {
        HeldBillsDialog(
            viewModel = viewModel,
            onDismiss = { showHeldBillsDialog = false }
        )
    }

    if (itemToOverridePrice != null) {
        OverrideItemPriceDialog(
            item = itemToOverridePrice!!,
            onDismiss = { itemToOverridePrice = null },
            onSave = { newPricePaise ->
                viewModel.updateItemPrice(itemToOverridePrice!!.lineId, newPricePaise)
                itemToOverridePrice = null
            }
        )
    }
}

@Composable
fun OverrideItemPriceDialog(
    item: BillingDraftItem,
    onDismiss: () -> Unit,
    onSave: (newPricePaise: Long) -> Unit
) {
    var priceText by remember {
        mutableStateOf(String.format(java.util.Locale.ENGLISH, "%.2f", item.unitPricePaise / 100.0))
    }

    AlertDialog(
        onDismissRequest = onDismiss,
        title = { Text("Override Item Price", fontWeight = FontWeight.Bold) },
```

```
        text = {
            Column(verticalArrangement = Arrangement.spacedBy(8.dp)) {
                Text(
                    text = item.productName,
                    style = MaterialTheme.typography.bodyMedium,
                    fontWeight = FontWeight.SemiBold
                )
                OutlinedTextField(
                    value = priceText,
                    onValueChange = { priceText = it },
                    label = { Text("New Unit Price (₹)") },
                    keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Decimal),
                    singleLine = true,
                    modifier = Modifier.fillMaxWidth()
                )
            }
        },
        confirmButton = {
            Button(
                onClick = {
                    val p = priceText.toDoubleOrNull() ?: 0.0
                    if (p >= 0.0) {
                        onSave((p * 100).toLong())
                    }
                },
                colors = ButtonDefaults.buttonColors(containerColor = MaterialTheme.colorScheme.primary)
            ) {
                Text("Save Price")
            }
        },
        dismissButton = {
            TextButton(onClick = onDismiss) {
                Text("Cancel")
            }
        }
    )
}
```

app\src\main\java\com\alieducational\app\ui\home\HomeScreen.kt

```
package com.alieducational.app.ui.home

import androidx.compose.foundation.Image
import androidx.compose.foundation.background
import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.foundation.shape.CircleShape
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.AccountBalanceWallet
import androidx.compose.material.icons.filled.Add
import androidx.compose.material.icons.filled.ArrowDownward
import androidx.compose.material.icons.filled.ArrowUpward
import androidx.compose.material.icons.filled.Backup
import androidx.compose.material.icons.filled.ChevronRight
import androidx.compose.material.icons.filled.Inventory2
import androidx.compose.material.icons.filled.Payment
import androidx.compose.material.icons.filled.PointOfSale
import androidx.compose.material.icons.filled.ReceiptLong
import androidx.compose.material.icons.filled.Settings
import androidx.compose.material.icons.filled.TrendingUp
import androidx.compose.material.icons.filled.Warning
import androidx.compose.material3.Button
import androidx.compose.material3.ButtonDefaults
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.vector.ImageVector
import androidx.compose.ui.layout.ContentScale
import androidx.compose.ui.res.painterResource
import androidx.compose.ui.res.stringResource
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.alieducational.app.R
import com.alieducational.app.domain.calculation.CostCalculator
import com.alieducational.app.ui.AliViewModel
import java.text.SimpleDateFormat
import java.util.Date
import java.util.Locale

@Composable
fun HomeScreen(
    viewModel: AliViewModel,
    onNavigateToNewBill: () -> Unit,
    onNavigateToAddStock: () -> Unit,
    onNavigateToRecordPayment: () -> Unit,
    onNavigateToStock: () -> Unit,
    onNavigateToAccounts: (Int) -> Unit, // 0 for suppliers, 1 for customers
    onNavigateToProfit: () -> Unit,
    onNavigateToExpenses: () -> Unit,
    onNavigateToCashTracker: () -> Unit,
    onNavigateToBackup: () -> Unit,
    onNavigateToSettings: () -> Unit,
    onNavigateToPigmy: () -> Unit = {},
    onNavigateToBillHistory: () -> Unit = {},
    modifier: Modifier = Modifier
) {
    val shopName by viewModel.shopName.collectAsState()
```

```
    val appLanguage by viewModel.appLanguage.collectAsState()
    val isDarkTheme by viewModel.isDarkTheme.collectAsState()
    val totalStockValue by viewModel.totalStockValuePaise.collectAsState()
    val lowStockCount by viewModel.lowStockCount.collectAsState()
    val weOweSuppliers by viewModel.totalWeOweSuppliersPaise.collectAsState()
    val customersOweUs by viewModel.totalCustomersOweUsPaise.collectAsState()
    val lastBackupTime by viewModel.lastBackupTime.collectAsState()
    val recentMovements by viewModel.recentMovements.collectAsState()

    Surface(
        modifier = modifier.fillMaxSize(),
        color = MaterialTheme.colorScheme.background
    ) {
        LazyColumn(
            modifier = Modifier
                .fillMaxSize()
                .padding(horizontal = 16.dp),
            verticalArrangement = Arrangement.spacedBy(16.dp)
        ) {
            // Header: Logo & Shop Name
            item {
                Spacer(modifier = Modifier.height(8.dp))
                Row(
                    modifier = Modifier.fillMaxWidth(),
                    verticalAlignment = Alignment.CenterVertically
                ) {
                    Image(
                        painter = painterResource(
                            id = if (isDarkTheme) R.drawable.ali_logo_dark else R.drawable.ali_logo_light
                        ),
                        contentDescription = "Ali Educational Monogram",
                        contentScale = ContentScale.Fit,
                        modifier = Modifier
                            .height(42.dp)
                            .width(80.dp)
                    )
                    Spacer(modifier = Modifier.width(12.dp))
                    Column(
                        modifier = Modifier.weight(1f),
                        verticalArrangement = Arrangement.Center
                    ) {
                        Text(
                            text = if (appLanguage == "ur") stringResource(R.string.app_name) else shopName,
                            style = MaterialTheme.typography.titleLarge.copy(
                                fontWeight = FontWeight.Bold,
                                fontSize = 22.sp
                            ),
                            color = MaterialTheme.colorScheme.onBackground
                        )
                        Text(
                            text = stringResource(R.string.home_stationery_subtitle),
                            style = MaterialTheme.typography.bodySmall,
                            color = MaterialTheme.colorScheme.onSurfaceVariant
                        )
                    }
                    IconButton(
                        onClick = onNavigateToSettings,
                        modifier = Modifier.size(40.dp)
                    ) {
                        Icon(
                            imageVector = Icons.Filled.Settings,
                            contentDescription = stringResource(R.string.home_settings),
                            tint = MaterialTheme.colorScheme.onBackground
                        )
                    }
                }
            }

            // 3 Prominent Everyday Actions: New Bill, Add Stock, Record Payment
            item {
                Row(
                    modifier = Modifier.fillMaxWidth(),
                    horizontalArrangement = Arrangement.spacedBy(10.dp)
                ) {
                    PrimaryActionButton(
                        title = stringResource(R.string.action_new_bill),
                        icon = Icons.Filled.PointOfSale,
                        containerColor = MaterialTheme.colorScheme.primary,
                        contentColor = MaterialTheme.colorScheme.onPrimary,
                        modifier = Modifier.weight(1f),
                        onClick = onNavigateToNewBill
                    )
                    PrimaryActionButton(
                        title = stringResource(R.string.action_add_stock),
                        icon = Icons.Filled.Add,
                        containerColor = MaterialTheme.colorScheme.surfaceVariant,
                        contentColor = MaterialTheme.colorScheme.onSurface,
                        modifier = Modifier.weight(1f),
```

```
                        onClick = onNavigateToAddStock
                    )
                    PrimaryActionButton(
                        title = stringResource(R.string.action_record_payment),
                        icon = Icons.Filled.Payment,
                        containerColor = MaterialTheme.colorScheme.surfaceVariant,
                        contentColor = MaterialTheme.colorScheme.onSurface,
                        modifier = Modifier.weight(1f),
                        onClick = onNavigateToRecordPayment
                    )
                }
            }

            // 4 Metrics Cards (2x2 grid)
            item {
                Column(verticalArrangement = Arrangement.spacedBy(12.dp)) {
                    Row(
                        modifier = Modifier.fillMaxWidth(),
                        horizontalArrangement = Arrangement.spacedBy(12.dp)
                    ) {
                        MetricCard(
                            title = stringResource(R.string.home_stock_value),
                            value = CostCalculator.formatPaiseToRupees(totalStockValue ?: 0L),
                            icon = Icons.Filled.Inventory2,
                            iconTint = MaterialTheme.colorScheme.primary,
                            modifier = Modifier.weight(1f),
                            onClick = onNavigateToStock
                        )
                        MetricCard(
                            title = stringResource(R.string.home_low_stock),
                            value = stringResource(R.string.home_items_count, lowStockCount),
                            icon = Icons.Filled.Warning,
                            iconTint = if (lowStockCount > 0) Color(0xFFE4B765) else MaterialTheme.colorScheme.onSurfaceVariant,
                            modifier = Modifier.weight(1f),
                            onClick = onNavigateToStock
                        )
                    }

                    Row(
                        modifier = Modifier.fillMaxWidth(),
                        horizontalArrangement = Arrangement.spacedBy(12.dp)
                    ) {
                        MetricCard(
                            title = stringResource(R.string.home_we_owe),
                            value = CostCalculator.formatPaiseToRupees(weOweSuppliers ?: 0L),
                            icon = Icons.Filled.ArrowUpward,
                            iconTint = Color(0xFFE57373),
                            modifier = Modifier.weight(1f),
                            onClick = { onNavigateToAccounts(0) }
                        )
                        MetricCard(
                            title = stringResource(R.string.home_customers_owe),
                            value = CostCalculator.formatPaiseToRupees(customersOweUs ?: 0L),
                            icon = Icons.Filled.ArrowDownward,
                            iconTint = Color(0xFF75B890),
                            modifier = Modifier.weight(1f),
                            onClick = { onNavigateToAccounts(1) }
                        )
                    }
                }
            }

            // Backup Status Card with "Back up now"
            item {
                Card(
                    modifier = Modifier.fillMaxWidth(),
                    colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surface),
                    shape = RoundedCornerShape(14.dp)
                ) {
                    Row(
                        modifier = Modifier
                            .fillMaxWidth()
                            .padding(14.dp),
                        verticalAlignment = Alignment.CenterVertically,
                        horizontalArrangement = Arrangement.SpaceBetween
                    ) {
                        Row(verticalAlignment = Alignment.CenterVertically) {
                            Icon(
                                imageVector = Icons.Filled.Backup,
                                contentDescription = null,
                                tint = MaterialTheme.colorScheme.primary,
                                modifier = Modifier.size(24.dp)
                            )
                            Spacer(modifier = Modifier.width(10.dp))
                            Column {
                                val backupText = if (lastBackupTime > 0L) {
                                    val format = SimpleDateFormat("dd MMM, hh:mm a", Locale.getDefault())
```

```
                                    "Last backup: ${format.format(Date(lastBackupTime))}"
                                } else {
                                    stringResource(R.string.home_backup_status)
                                }
                                Text(
                                    text = backupText,
                                    style = MaterialTheme.typography.bodyMedium.copy(fontWeight =
FontWeight.Medium),
                                    color = MaterialTheme.colorScheme.onSurface
                                )
                                Text(
                                    text = stringResource(R.string.home_stored_safely),
                                    style = MaterialTheme.typography.bodySmall,
                                    color = MaterialTheme.colorScheme.onSurfaceVariant
                                )
                            }
                        }

                        Button(
                            onClick = { viewModel.createBackup() },
                            shape = RoundedCornerShape(8.dp),
                            colors = ButtonDefaults.buttonColors(
                                containerColor = MaterialTheme.colorScheme.primary,
                                contentColor = MaterialTheme.colorScheme.onPrimary
                            ),
                            contentPadding = ButtonDefaults.TextButtonContentPadding
                        ) {
                            Text(text = stringResource(R.string.home_backup_now), fontSize = 12.sp)
                        }
                    }
                }
            }

            // Reports & Settings Section (Accessible directly from Home)
            item {
                Text(
                    text = stringResource(R.string.home_reports_and_tools),
                    style = MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.Bold),
                    color = MaterialTheme.colorScheme.onBackground
                )
            }

            item {
                Column(verticalArrangement = Arrangement.spacedBy(8.dp)) {
                    Row(
                        modifier = Modifier.fillMaxWidth(),
                        horizontalArrangement = Arrangement.spacedBy(10.dp)
                    ) {
                        ToolQuickCard(
                            title = stringResource(R.string.home_profit_analysis),
                            icon = Icons.Filled.TrendingUp,
                            modifier = Modifier.weight(1f),
                            onClick = onNavigateToProfit
                        )
                        ToolQuickCard(
                            title = stringResource(R.string.home_expenses),
                            icon = Icons.Filled.ReceiptLong,
                            modifier = Modifier.weight(1f),
                            onClick = onNavigateToExpenses
                        )
                    }

                    Row(
                        modifier = Modifier.fillMaxWidth(),
                        horizontalArrangement = Arrangement.spacedBy(10.dp)
                    ) {
                        ToolQuickCard(
                            title = stringResource(R.string.home_backup_restore),
                            icon = Icons.Filled.Backup,
                            modifier = Modifier.weight(1f),
                            onClick = onNavigateToBackup
                        )
                        ToolQuickCard(
                            title = "Cash In / Out Tracker",
                            icon = Icons.Filled.PointOfSale,
                            modifier = Modifier.weight(1f),
                            onClick = onNavigateToCashTracker
                        )
                    }

                    Row(
                        modifier = Modifier.fillMaxWidth(),
                        horizontalArrangement = Arrangement.spacedBy(10.dp)
                    ) {
                        ToolQuickCard(
                            title = "Pigmy Daily Savings",
                            iconRes = R.drawable.ic_pigmy_coin,
                            modifier = Modifier.weight(1f),
```

```
                            onClick = onNavigateToPigmy
                        )
                        ToolQuickCard(
                            title = "Bill History",
                            icon = Icons.Filled.ReceiptLong,
                            modifier = Modifier.weight(1f),
                            onClick = onNavigateToBillHistory
                        )
                    }
                }
            }

            item {
                Spacer(modifier = Modifier.height(24.dp))
            }
        }
    }
}

@Composable
fun PrimaryActionButton(
    title: String,
    icon: ImageVector,
    containerColor: Color,
    contentColor: Color,
    modifier: Modifier = Modifier,
    onClick: () -> Unit
) {
    Card(
        modifier = modifier
            .height(72.dp)
            .clickable { onClick() },
        colors = CardDefaults.cardColors(containerColor = containerColor),
        shape = RoundedCornerShape(14.dp)
    ) {
        Column(
            modifier = Modifier
                .fillMaxSize()
                .padding(8.dp),
            horizontalAlignment = Alignment.CenterHorizontally,
            verticalArrangement = Arrangement.Center
        ) {
            Icon(
                imageVector = icon,
                contentDescription = null,
                tint = contentColor,
                modifier = Modifier.size(26.dp)
            )
            Spacer(modifier = Modifier.height(4.dp))
            Text(
                text = title,
                style = MaterialTheme.typography.labelSmall.copy(fontWeight = FontWeight.Bold),
                color = contentColor,
                maxLines = 1
            )
        }
    }
}

@Composable
fun MetricCard(
    title: String,
    value: String,
    icon: ImageVector,
    iconTint: Color,
    modifier: Modifier = Modifier,
    onClick: () -> Unit
) {
    Card(
        modifier = modifier
            .clickable { onClick() },
        colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surface),
        shape = RoundedCornerShape(14.dp)
    ) {
        Column(modifier = Modifier.padding(14.dp)) {
            Row(
                modifier = Modifier.fillMaxWidth(),
                horizontalArrangement = Arrangement.SpaceBetween,
                verticalAlignment = Alignment.CenterVertically
            ) {
                Text(
                    text = title,
                    style = MaterialTheme.typography.bodySmall,
                    color = MaterialTheme.colorScheme.onSurfaceVariant
                )
                Icon(
                    imageVector = icon,
                    contentDescription = null,
```

```
                    tint = iconTint,
                    modifier = Modifier.size(18.dp)
                )
            }
            Spacer(modifier = Modifier.height(8.dp))
            Text(
                text = value,
                style = MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.Bold),
                color = MaterialTheme.colorScheme.onSurface
            )
        }
    }
}

@Composable
fun ToolQuickCard(
    title: String,
    icon: ImageVector? = null,
    iconRes: Int? = null,
    modifier: Modifier = Modifier,
    onClick: () -> Unit
) {
    Card(
        modifier = modifier
            .height(58.dp)
            .clickable { onClick() },
        colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surface),
        shape = RoundedCornerShape(12.dp)
    ) {
        Row(
            modifier = Modifier
                .fillMaxSize()
                .padding(horizontal = 12.dp),
            verticalAlignment = Alignment.CenterVertically,
            horizontalArrangement = Arrangement.SpaceBetween
        ) {
            Row(
                verticalAlignment = Alignment.CenterVertically,
                modifier = Modifier.weight(1f)
            ) {
                if (iconRes != null) {
                    Icon(
                        painter = painterResource(id = iconRes),
                        contentDescription = "Pigmy savings",
                        tint = Color(0xFFD97706),
                        modifier = Modifier.size(20.dp)
                    )
                } else if (icon != null) {
                    Icon(
                        imageVector = icon,
                        contentDescription = null,
                        tint = MaterialTheme.colorScheme.primary,
                        modifier = Modifier.size(20.dp)
                    )
                }
                Spacer(modifier = Modifier.width(8.dp))
                Text(
                    text = title,
                    style = MaterialTheme.typography.bodySmall.copy(fontWeight = FontWeight.SemiBold),
                    color = MaterialTheme.colorScheme.onSurface,
                    maxLines = 1
                )
            }
            Icon(
                imageVector = Icons.Filled.ChevronRight,
                contentDescription = null,
                tint = MaterialTheme.colorScheme.onSurfaceVariant,
                modifier = Modifier.size(16.dp)
            )
        }
    }
}
```

app\src\main\java\com\alieducational\app\ui\more\BackupRestoreScreen.kt

```
package com.alieducational.app.ui.more

import android.content.Intent
import androidx.activity.compose.rememberLauncherForActivityResult
import androidx.activity.result.contract.ActivityResultContracts
import androidx.compose.foundation.background
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.ArrowBack
import androidx.compose.material.icons.filled.Backup
import androidx.compose.material.icons.filled.CheckCircle
import androidx.compose.material.icons.filled.Folder
import androidx.compose.material.icons.filled.FolderOpen
import androidx.compose.material.icons.filled.Info
import androidx.compose.material.icons.filled.Restore
import androidx.compose.material.icons.filled.Share
import androidx.compose.material.icons.filled.Warning
import androidx.compose.material3.AlertDialog
import androidx.compose.material3.Button
import androidx.compose.material3.ButtonDefaults
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedButton
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.material3.TopAppBar
import androidx.compose.material3.TopAppBarDefaults
import androidx.compose.runtime.Composable
import androidx.compose.runtime.LaunchedEffect
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.alieducational.app.data.service.BackupMetadata
import com.alieducational.app.ui.AliViewModel
import com.alieducational.app.ui.theme.*
import androidx.compose.foundation.BorderStroke
import java.io.File
import java.text.SimpleDateFormat
import java.util.Date
import java.util.Locale

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun BackupRestoreScreen(
    viewModel: AliViewModel,
    onNavigateBack: () -> Unit,
    modifier: Modifier = Modifier
) {
    val context = LocalContext.current
    val backupStatus by viewModel.backupStatus.collectAsState()
    val availableBackups by viewModel.availableBackups.collectAsState()
    val lastBackupTime by viewModel.lastBackupTime.collectAsState()
    val isCreatingBackup by viewModel.isCreatingBackup.collectAsState()
    val lastExternalSaveTime by viewModel.lastExternalSaveTime.collectAsState()
    val lastExternalSaveDestination by viewModel.lastExternalSaveDestination.collectAsState()
```

```
83
84      var selectedBackupForRestore by remember { mutableStateOf<BackupMetadata?>(null) }
85      var viewingDetailsBackup by remember { mutableStateOf<BackupMetadata?>(null) }
86      val isDarkTheme by viewModel.isDarkTheme.collectAsState()
87      val positiveColor = if (isDarkTheme) PositiveGreenDark else PositiveGreenLight
88      val negativeColor = if (isDarkTheme) NegativeRedDark else NegativeRedLight
89      val warningColor = if (isDarkTheme) WarningOrangeDark else WarningOrangeLight
90
91      LaunchedEffect(Unit) {
92          viewModel.loadBackups()
93      }
94
95      var pendingFileToSave by remember { mutableStateOf<File?>(null) }
96      val saveToDriveLauncher = rememberLauncherForActivityResult(
97          contract = ActivityResultContracts.CreateDocument("application/zip")
98      ) { uri ->
99          if (uri != null && pendingFileToSave != null) {
100             viewModel.exportBackupToUri(pendingFileToSave!!, uri)
101         }
102         pendingFileToSave = null
103     }
104
105     val restoreFilePicker = rememberLauncherForActivityResult(
106         contract = ActivityResultContracts.OpenDocument()
107     ) { uri ->
108         if (uri != null) {
109             viewModel.restoreFromUri(uri)
110         }
111     }
112
113     Scaffold(
114         topBar = {
115             TopAppBar(
116                 title = { Text("Backup & Restore", fontWeight = FontWeight.Bold) },
117                 navigationIcon = {
118                     IconButton(onClick = onNavigateBack) {
119                         Icon(Icons.Filled.ArrowBack, contentDescription = "Back")
120                     }
121                 },
122                 colors = TopAppBarDefaults.topAppBarColors(
123                     containerColor = MaterialTheme.colorScheme.background,
124                     titleContentColor = MaterialTheme.colorScheme.onBackground
125                 )
126             )
127         }
128     ) { padding ->
129         Surface(
130             modifier = modifier
131                 .fillMaxSize()
132                 .padding(padding),
133             color = MaterialTheme.colorScheme.background
134         ) {
135             LazyColumn(
136                 modifier = Modifier
137                     .fillMaxSize()
138                     .padding(horizontal = 16.dp),
139                 verticalArrangement = Arrangement.spacedBy(14.dp)
140             ) {
141                 // Daily Closing Routine Reminder Card
142                 item {
143                     Card(
144                         modifier = Modifier.fillMaxWidth(),
145                         colors = CardDefaults.cardColors(containerColor =
    MaterialTheme.colorScheme.primaryContainer.copy(alpha = 0.5f)),
146                         shape = RoundedCornerShape(12.dp)
147                     ) {
148                         Row(
149                             modifier = Modifier.padding(14.dp),
150                             verticalAlignment = Alignment.CenterVertically
151                         ) {
152                             Icon(
153                                 Icons.Filled.Warning,
154                                 contentDescription = null,
155                                 tint = MaterialTheme.colorScheme.primary,
156                                 modifier = Modifier.size(22.dp)
157                             )
158                             Spacer(modifier = Modifier.width(10.dp))
159                             Column {
160                                 Text(
161                                     text = "Daily Routine Reminder",
162                                     style = MaterialTheme.typography.titleSmall.copy(fontWeight =
    FontWeight.Bold),
163                                     color = MaterialTheme.colorScheme.onPrimaryContainer
164                                 )
165                                 Text(
166                                     text = "Save an external copy to Google Drive / Files after shop closing
     each evening, and always before app updates.",
167                                     style = MaterialTheme.typography.bodySmall,
```

```
                                    color = MaterialTheme.colorScheme.onPrimaryContainer
                                )
                            }
                        }
                    }
                }

                // Backup Status Card matching Section 4 recommendation
                item {
                    val latestBackup = availableBackups.firstOrNull()
                    val hasBackup = latestBackup != null
                    val dateFormatted = if (hasBackup) {
                        SimpleDateFormat("d MMMM yyyy, h:mm a",
Locale.getDefault()).format(Date(latestBackup!!.timestamp))
                    } else if (lastBackupTime > 0) {
                        SimpleDateFormat("d MMMM yyyy, h:mm a",
Locale.getDefault()).format(Date(lastBackupTime))
                    } else "Never"

                    Card(
                        modifier = Modifier.fillMaxWidth(),
                        colors = CardDefaults.cardColors(containerColor =
MaterialTheme.colorScheme.surface),
                        border = BorderStroke(1.dp, MaterialTheme.colorScheme.outline.copy(alpha = if
(isDarkTheme) 0.3f else 0.5f)),
                        shape = RoundedCornerShape(14.dp)
                    ) {
                        Column(modifier = Modifier.padding(16.dp)) {
                            Row(
                                modifier = Modifier.fillMaxWidth(),
                                horizontalArrangement = Arrangement.SpaceBetween,
                                verticalAlignment = Alignment.CenterVertically
                            ) {
                                Column {
                                    Text(
                                        text = if (hasBackup) "Backup ready" else "Local Phone Backup",
                                        style = MaterialTheme.typography.titleMedium.copy(fontWeight =
FontWeight.Bold),
                                        color = MaterialTheme.colorScheme.onSurface
                                    )
                                    Text(
                                        text = dateFormatted,
                                        style = MaterialTheme.typography.bodySmall,
                                        color = MaterialTheme.colorScheme.onSurfaceVariant
                                    )
                                }
                                Box(
                                    modifier = Modifier
                                        .background(
                                            color = if (hasBackup) positiveColor.copy(alpha = 0.15f) else
warningColor.copy(alpha = 0.15f),
                                            shape = RoundedCornerShape(8.dp)
                                        )
                                        .padding(horizontal = 8.dp, vertical = 4.dp)
                                ) {
                                    Text(
                                        text = if (hasBackup) "Verified Ready" else "Pending",
                                        color = if (hasBackup) positiveColor else warningColor,
                                        fontWeight = FontWeight.Bold,
                                        style = MaterialTheme.typography.labelSmall
                                    )
                                }
                            }

                            Spacer(modifier = Modifier.height(14.dp))

                            // Three status rows from Section 4
                            Row(
                                modifier = Modifier.fillMaxWidth(),
                                horizontalArrangement = Arrangement.SpaceBetween
                            ) {
                                Text("Complete recovery file", style = MaterialTheme.typography.bodyMedium,
color = MaterialTheme.colorScheme.onSurface)
                                Text(if (hasBackup) "Ready" else "Not created", fontWeight =
FontWeight.SemiBold, color = if (hasBackup) positiveColor else MaterialTheme.colorScheme.onSurfaceVariant)
                            }
                            Spacer(modifier = Modifier.height(6.dp))
                            Row(
                                modifier = Modifier.fillMaxWidth(),
                                horizontalArrangement = Arrangement.SpaceBetween
                            ) {
                                Text("Readable Excel records", style = MaterialTheme.typography.bodyMedium,
color = MaterialTheme.colorScheme.onSurface)
                                Text(if (hasBackup) "Ready" else "Not created", fontWeight =
FontWeight.SemiBold, color = if (hasBackup) positiveColor else MaterialTheme.colorScheme.onSurfaceVariant)
                            }
                            Spacer(modifier = Modifier.height(6.dp))
                            Row(
```

```
                                modifier = Modifier.fillMaxWidth(),
                                horizontalArrangement = Arrangement.SpaceBetween
                            ) {
                                Text("Photos and files", style = MaterialTheme.typography.bodyMedium, color = MaterialTheme.colorScheme.onSurface)
                                Text(
                                    if (hasBackup) "${latestBackup!!.photoCount} verified" else "0 files",
                                    fontWeight = FontWeight.SemiBold,
                                    color = MaterialTheme.colorScheme.onSurface
                                )
                            }

                            Spacer(modifier = Modifier.height(16.dp))

                            // Primary Save Both Button
                            Button(
                                onClick = {
                                    val target = latestBackup ?: return@Button
                                    val f = File(target.packageFilePath ?: target.filePath)
                                    pendingFileToSave = f
                                    saveToDriveLauncher.launch(f.name)
                                },
                                enabled = hasBackup && !isCreatingBackup,
                                modifier = Modifier.fillMaxWidth().height(48.dp),
                                shape = RoundedCornerShape(10.dp),
                                colors = ButtonDefaults.buttonColors(
                                    containerColor = MaterialTheme.colorScheme.primary,
                                    contentColor = MaterialTheme.colorScheme.onPrimary
                                )
                            ) {
                                Icon(Icons.Filled.Backup, contentDescription = null, modifier = Modifier.size(18.dp))
                                Spacer(modifier = Modifier.width(8.dp))
                                Text("Save Both to Drive / Files", fontWeight = FontWeight.Bold, fontSize = 14.sp)
                            }

                            Spacer(modifier = Modifier.height(8.dp))

                            // Secondary actions: Share Package + View Details
                            Row(
                                modifier = Modifier.fillMaxWidth(),
                                horizontalArrangement = Arrangement.spacedBy(8.dp)
                            ) {
                                OutlinedButton(
                                    onClick = {
                                        val target = latestBackup ?: return@OutlinedButton
                                        val path = target.packageFilePath ?: target.filePath
                                        val shareIntent = viewModel.getBackupShareIntent(path)
                                        context.startActivity(Intent.createChooser(shareIntent, "Share Backup Package"))
                                    },
                                    enabled = hasBackup,
                                    modifier = Modifier.weight(1f),
                                    shape = RoundedCornerShape(8.dp)
                                ) {
                                    Icon(Icons.Filled.Share, contentDescription = null, modifier = Modifier.size(16.dp))
                                    Spacer(modifier = Modifier.width(4.dp))
                                    Text("Share package", fontSize = 12.sp)
                                }

                                OutlinedButton(
                                    onClick = {
                                        viewingDetailsBackup = latestBackup
                                    },
                                    enabled = hasBackup,
                                    modifier = Modifier.weight(1f),
                                    shape = RoundedCornerShape(8.dp)
                                ) {
                                    Icon(Icons.Filled.Info, contentDescription = null, modifier = Modifier.size(16.dp))
                                    Spacer(modifier = Modifier.width(4.dp))
                                    Text("View details", fontSize = 12.sp)
                                }
                            }

                            Spacer(modifier = Modifier.height(10.dp))

                            // Action buttons: Create Backup Now & Restore from File
                            Row(
                                modifier = Modifier.fillMaxWidth(),
                                horizontalArrangement = Arrangement.spacedBy(8.dp)
                            ) {
                                Button(
                                    onClick = { viewModel.createBackup() },
                                    enabled = !isCreatingBackup,
                                    modifier = Modifier.weight(1f),
```

```
                                    shape = RoundedCornerShape(8.dp),
                                    colors = ButtonDefaults.buttonColors(
                                        containerColor = MaterialTheme.colorScheme.surfaceVariant,
                                        contentColor = MaterialTheme.colorScheme.onSurface
                                    )
                                ) {
                                    if (isCreatingBackup) {
                                        androidx.compose.material3.CircularProgressIndicator(
                                            modifier = Modifier.size(16.dp),
                                            strokeWidth = 2.dp
                                        )
                                        Spacer(modifier = Modifier.width(6.dp))
                                        Text("Creating...", fontSize = 12.sp)
                                    } else {
                                        Icon(Icons.Filled.Backup, contentDescription = null, modifier =
    Modifier.size(16.dp))
                                        Spacer(modifier = Modifier.width(4.dp))
                                        Text("Backup Now", fontSize = 12.sp)
                                    }
                                }

                                OutlinedButton(
                                    onClick = {
                                        restoreFilePicker.launch(arrayOf("application/zip",
    "application/octet-stream", "*/*"))
                                    },
                                    modifier = Modifier.weight(1f),
                                    shape = RoundedCornerShape(8.dp)
                                ) {
                                    Icon(Icons.Filled.FolderOpen, contentDescription = null, modifier =
    Modifier.size(16.dp))
                                    Spacer(modifier = Modifier.width(4.dp))
                                    Text("Restore Backup", fontSize = 12.sp)
                                }
                            }
                        }
                    }
                }

                // Status Message
                if (backupStatus != null) {
                    item {
                        val isFailed = backupStatus!!.contains("fail", ignoreCase = true)
                        val statusColor = if (isFailed) Color(0xFFE57373) else Color(0xFF75B890)
                        Card(
                            modifier = Modifier.fillMaxWidth(),
                            colors = CardDefaults.cardColors(containerColor = statusColor.copy(alpha =
    0.15f)),
                            shape = RoundedCornerShape(10.dp)
                        ) {
                            Row(
                                modifier = Modifier
                                    .fillMaxWidth()
                                    .padding(12.dp),
                                horizontalArrangement = Arrangement.SpaceBetween,
                                verticalAlignment = Alignment.CenterVertically
                            ) {
                                Text(
                                    text = backupStatus!!,
                                    style = MaterialTheme.typography.bodyMedium.copy(fontWeight =
    FontWeight.Bold),
                                    color = MaterialTheme.colorScheme.onSurface,
                                    modifier = Modifier.weight(1f)
                                )
                                if (isFailed) {
                                    Spacer(modifier = Modifier.width(8.dp))
                                    Button(
                                        onClick = { viewModel.createBackup() },
                                        shape = RoundedCornerShape(8.dp)
                                    ) {
                                        Text("Retry")
                                    }
                                }
                            }
                        }
                    }
                }

                // Storage Location & File Access Transparency Card (F14 requirement)
                item {
                    Card(
                        modifier = Modifier.fillMaxWidth(),
                        colors = CardDefaults.cardColors(containerColor =
    MaterialTheme.colorScheme.surfaceVariant.copy(alpha = 0.6f)),
                        shape = RoundedCornerShape(12.dp)
                    ) {
                        Column(modifier = Modifier.padding(14.dp)) {

                            Row(verticalAlignment = Alignment.CenterVertically) {
                                Icon(
```

```
                                    imageVector = Icons.Filled.Folder,
                                    contentDescription = null,
                                    tint = MaterialTheme.colorScheme.primary,
                                    modifier = Modifier.size(20.dp)
                                )
                                Spacer(modifier = Modifier.width(8.dp))
                                Text(
                                    text = "Storage Location on Phone",
                                    style = MaterialTheme.typography.titleSmall.copy(fontWeight =
FontWeight.Bold),
                                    color = MaterialTheme.colorScheme.onSurface
                                )
                            }
                            Spacer(modifier = Modifier.height(6.dp))
                            Text(
                                text = viewModel.backupFolderPath,
                                style = MaterialTheme.typography.bodySmall.copy(fontSize = 11.sp),
                                color = MaterialTheme.colorScheme.primary,
                                fontWeight = FontWeight.Medium
                            )
                            Spacer(modifier = Modifier.height(6.dp))
                            Text(
                                text = "Backups are saved directly in your phone's
Documents/AliEducational/Backups folder, easily visible in Samsung My Files. You can also tap 'Export /
Share' to send a backup via WhatsApp, Google Drive, or email.",
                                style = MaterialTheme.typography.bodySmall,
                                color = MaterialTheme.colorScheme.onSurfaceVariant
                            )
                        }
                    }
                }

                // Contents Breakdown Card (F14 requirement)
                item {
                    Card(
                        modifier = Modifier.fillMaxWidth(),
                        colors = CardDefaults.cardColors(containerColor =
MaterialTheme.colorScheme.surface),
                        shape = RoundedCornerShape(12.dp)
                    ) {
                        Column(modifier = Modifier.padding(14.dp)) {
                            Row(verticalAlignment = Alignment.CenterVertically) {
                                Icon(
                                    imageVector = Icons.Filled.Info,
                                    contentDescription = null,
                                    tint = Color(0xFFE4B765),
                                    modifier = Modifier.size(20.dp)
                                )
                                Spacer(modifier = Modifier.width(8.dp))
                                Text(
                                    text = "What is Included in Backups",
                                    style = MaterialTheme.typography.titleSmall.copy(fontWeight =
FontWeight.Bold),
                                    color = MaterialTheme.colorScheme.onSurface
                                )
                            }
                            Spacer(modifier = Modifier.height(8.dp))
                            Text(
                                text = "• Complete inventory, piece/pack prices & moving-average costs\n" +
                                       "• All sales history, cash memos & counter bills\n" +
                                       "• Customer credit balances & khata ledgers\n" +
                                       "• Supplier purchases, dues & payment logs\n" +
                                       "• Shop expenses, daily cash balances & profit reports",
                                style = MaterialTheme.typography.bodySmall,
                                color = MaterialTheme.colorScheme.onSurfaceVariant
                            )
                        }
                    }
                }

                // Available Backups List
                item {
                    Text(
                        text = "Dated Backups Stored on Phone (${availableBackups.size})",
                        style = MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.Bold),
                        color = MaterialTheme.colorScheme.onBackground
                    )
                }

                if (availableBackups.isEmpty()) {
                    item {
                        Text(
                            text = "No previous backups found on this phone.",
                            style = MaterialTheme.typography.bodyMedium,
                            color = MaterialTheme.colorScheme.onSurfaceVariant
                        )
                    }
                } else {
                    items(availableBackups) { backup ->
```

```
                        Card(
                            modifier = Modifier.fillMaxWidth(),
                            colors = CardDefaults.cardColors(containerColor =
MaterialTheme.colorScheme.surface),
                            shape = RoundedCornerShape(12.dp)
                        ) {
                            Column(modifier = Modifier.padding(14.dp)) {
                                Row(
                                    modifier = Modifier.fillMaxWidth(),
                                    horizontalArrangement = Arrangement.SpaceBetween,
                                    verticalAlignment = Alignment.CenterVertically
                                ) {
                                    Column(modifier = Modifier.weight(1f)) {
                                        Text(
                                            text = backup.fileName,
                                            style = MaterialTheme.typography.bodyMedium.copy(fontWeight =
FontWeight.Bold),
                                            color = MaterialTheme.colorScheme.onSurface
                                        )
                                        val dateStr = SimpleDateFormat("dd MMM yyyy, hh:mm a",
Locale.getDefault()).format(Date(backup.timestamp))
                                        val sizeKb = backup.sizeBytes / 1024
                                        Text(
                                            text = "$dateStr • $sizeKb KB",
                                            style = MaterialTheme.typography.bodySmall,
                                            color = MaterialTheme.colorScheme.onSurfaceVariant
                                        )
                                    }

                                    // Verification status badge
                                    Box(
                                        modifier = Modifier
                                            .background(
                                                color = if (backup.isValid) Color(0xFF75B890).copy(alpha =
0.2f) else Color(0xFFE57373).copy(alpha = 0.2f),
                                                shape = RoundedCornerShape(6.dp)
                                            )
                                            .padding(horizontal = 8.dp, vertical = 4.dp)
                                    ) {
                                        Row(verticalAlignment = Alignment.CenterVertically) {
                                            Icon(
                                                imageVector = if (backup.isValid) Icons.Filled.CheckCircle
else Icons.Filled.Warning,
                                                contentDescription = null,
                                                tint = if (backup.isValid) Color(0xFF75B890) else
Color(0xFFE57373),
                                                modifier = Modifier.size(12.dp)
                                            )
                                            Spacer(modifier = Modifier.width(4.dp))
                                            Text(
                                                text = if (backup.isValid) "Verified" else "Invalid",
                                                style = MaterialTheme.typography.labelSmall.copy(fontWeight
= FontWeight.Bold),
                                                color = if (backup.isValid) Color(0xFF75B890) else
Color(0xFFE57373)
                                            )
                                        }
                                    }
                                }

                                Spacer(modifier = Modifier.height(10.dp))

                                // Action Buttons: Share / Export, Save to Drive, and Restore
                                Row(
                                    modifier = Modifier.fillMaxWidth(),
                                    horizontalArrangement = Arrangement.spacedBy(6.dp)
                                ) {
                                    OutlinedButton(
                                        onClick = {
                                            pendingFileToSave = File(backup.filePath)
                                            saveToDriveLauncher.launch(backup.fileName)
                                        },
                                        modifier = Modifier.weight(1f),
                                        shape = RoundedCornerShape(8.dp),
                                        contentPadding = ButtonDefaults.TextButtonContentPadding
                                    ) {
                                        Icon(Icons.Filled.Backup, contentDescription = null, modifier =
Modifier.size(14.dp))
                                        Spacer(modifier = Modifier.width(3.dp))
                                        Text("To Drive", fontSize = 11.sp, maxLines = 1)
                                    }

                                    OutlinedButton(
                                        onClick = {
                                            val shareIntent =
viewModel.getBackupShareIntent(backup.filePath)
                                            context.startActivity(Intent.createChooser(shareIntent, "Export
/ Share Backup"))
```

```
                                        },
                                        modifier = Modifier.weight(1f),
                                        shape = RoundedCornerShape(8.dp),
                                        contentPadding = ButtonDefaults.TextButtonContentPadding
                                    ) {
                                        Icon(Icons.Filled.Share, contentDescription = null, modifier =
    Modifier.size(14.dp))
                                        Spacer(modifier = Modifier.width(3.dp))
                                        Text("Share", fontSize = 11.sp, maxLines = 1)
                                    }

                                    Button(
                                        onClick = { selectedBackupForRestore = backup },
                                        modifier = Modifier.weight(1f),
                                        colors = ButtonDefaults.buttonColors(
                                            containerColor = MaterialTheme.colorScheme.surfaceVariant,
                                            contentColor = MaterialTheme.colorScheme.onSurface
                                        ),
                                        shape = RoundedCornerShape(8.dp),
                                        contentPadding = ButtonDefaults.TextButtonContentPadding
                                    ) {
                                        Icon(Icons.Filled.Restore, contentDescription = null, modifier =
    Modifier.size(14.dp))
                                        Spacer(modifier = Modifier.width(3.dp))
                                        Text("Restore", fontSize = 11.sp, maxLines = 1)
                                    }
                                }
                            }
                        }
                    }
                }

                item { Spacer(modifier = Modifier.height(24.dp)) }
            }
        }
    }

    // Restore Warning Confirmation Dialog
    if (selectedBackupForRestore != null) {
        val backup = selectedBackupForRestore!!
        val dateStr = SimpleDateFormat("dd MMM yyyy, hh:mm a",
    Locale.getDefault()).format(Date(backup.timestamp))
        val sizeKb = backup.sizeBytes / 1024

        AlertDialog(
            onDismissRequest = { selectedBackupForRestore = null },
            icon = {
                Icon(Icons.Filled.Warning, contentDescription = null, tint = Color(0xFFE4B765), modifier =
    Modifier.size(36.dp))
            },
            title = { Text("Restore This Backup?") },
            text = {
                Column {
                    Text("File: ${backup.fileName}", fontWeight = FontWeight.Bold)
                    Text("Created: $dateStr ($sizeKb KB)", style = MaterialTheme.typography.bodySmall)
                    Spacer(modifier = Modifier.height(10.dp))
                    Text(
                        "WARNING: Restoring will overwrite all current local stock, customer khata, and
    sales records with this backup's snapshot.\n\nA safety backup of your existing records will be created
    automatically before restoring.",
                        color = Color(0xFFE4B765),
                        fontWeight = FontWeight.Medium,
                        style = MaterialTheme.typography.bodySmall
                    )
                }
            },
            confirmButton = {
                Button(
                    onClick = {
                        val file = File(backup.filePath)
                        selectedBackupForRestore = null
                        viewModel.restoreBackup(file)
                    },
                    colors = ButtonDefaults.buttonColors(containerColor = Color(0xFFE57373))
                ) {
                    Text("Confirm Restore")
                }
            },
            dismissButton = {
                OutlinedButton(onClick = { selectedBackupForRestore = null }) {
                    Text("Cancel")
                }
            }
        )
    }

// View Details Dialog
    if (viewingDetailsBackup != null) {
```

```
        val b = viewingDetailsBackup!!
        val dateStr = SimpleDateFormat("dd MMMM yyyy, hh:mm a",
    Locale.getDefault()).format(Date(b.timestamp))

        AlertDialog(
            onDismissRequest = { viewingDetailsBackup = null },
            icon = {
                Icon(Icons.Filled.Info, contentDescription = null, tint = MaterialTheme.colorScheme.primary,
     modifier = Modifier.size(32.dp))
            },
            title = { Text("Backup Package Details", fontWeight = FontWeight.Bold) },
            text = {
                Column(verticalArrangement = Arrangement.spacedBy(6.dp)) {
                    Text("Snapshot ID: ${b.snapshotId.ifBlank { "N/A" }}", style =
    MaterialTheme.typography.bodySmall, fontWeight = FontWeight.Bold)
                    Text("Date: $dateStr", style = MaterialTheme.typography.bodySmall)
                    Text("Package Size: ${b.sizeBytes / 1024} KB", style =
    MaterialTheme.typography.bodySmall)
                    Spacer(modifier = Modifier.height(4.dp))
                    Text("Verified Record Counts:", style =
    MaterialTheme.typography.bodySmall.copy(fontWeight = FontWeight.Bold), color =
    MaterialTheme.colorScheme.primary)
                    Text("• Products in Catalog: ${b.productCount}", style =
    MaterialTheme.typography.bodySmall)
                    Text("• Total Bills / Invoices: ${b.billCount}", style =
    MaterialTheme.typography.bodySmall)
                    Text("• Customer Credit Accounts: ${b.customerCount}", style =
    MaterialTheme.typography.bodySmall)
                    Text("• Supplier Accounts: ${b.supplierCount}", style =
    MaterialTheme.typography.bodySmall)
                    Text("• Cash Ledger Transactions: ${b.cashInOutCount}", style =
    MaterialTheme.typography.bodySmall)
                    Text("• Pigmy Savings Records: ${b.pigmyCount}", style =
    MaterialTheme.typography.bodySmall)
                    Text("• Photos & Attachments: ${b.photoCount}", style =
    MaterialTheme.typography.bodySmall)
                    Spacer(modifier = Modifier.height(4.dp))
                    Text("Status: ${b.integrityNote}", style = MaterialTheme.typography.labelSmall, color =
    positiveColor)
                }
            },
            confirmButton = {
                Button(onClick = { viewingDetailsBackup = null }) {
                    Text("Close")
                }
            }
        )
    }
}
```

app\src\main\java\com\alieducational\app\ui\more\ExpensesScreen.kt

```
package com.alieducational.app.ui.more

import androidx.compose.ui.platform.LocalContext
import androidx.compose.foundation.layout.size
import androidx.compose.ui.unit.sp
import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Add
import androidx.compose.material.icons.filled.ArrowBack
import androidx.compose.material3.AlertDialog
import androidx.compose.material3.Button
import androidx.compose.material3.ButtonDefaults
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.DropdownMenuItem
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.ExposedDropdownMenuBox
import androidx.compose.material3.ExposedDropdownMenuDefaults
import androidx.compose.material3.FloatingActionButton
import androidx.compose.material3.FilterChip
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedButton
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.RadioButton
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Surface
import android.app.DatePickerDialog
import android.app.TimePickerDialog
import androidx.compose.material.icons.filled.CalendarToday
import androidx.compose.material.icons.filled.AccessTime
import androidx.compose.material3.Text
import com.alieducational.app.domain.calculation.CostCalculator
import java.text.SimpleDateFormat
import java.util.Calendar
import java.util.Date
import java.util.Locale
import androidx.compose.material3.TopAppBar
import androidx.compose.material3.TopAppBarDefaults
import androidx.compose.runtime.Composable
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.unit.dp
import com.alieducational.app.data.local.entity.ShopExpenseEntity
import com.alieducational.app.ui.AliViewModel

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun ExpensesScreen(
    viewModel: AliViewModel,
    onNavigateBack: () -> Unit,
    modifier: Modifier = Modifier
) {
    val allExpenses by viewModel.allExpenses.collectAsState()
    var showAddDialog by remember { mutableStateOf(false) }

    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text("Other Money & Expenses", fontWeight = FontWeight.Bold) },
```

```
                navigationIcon = {
                    IconButton(onClick = onNavigateBack) {
                        Icon(Icons.Filled.ArrowBack, contentDescription = "Back")
                    }
                },
                colors = TopAppBarDefaults.topAppBarColors(
                    containerColor = MaterialTheme.colorScheme.background,
                    titleContentColor = MaterialTheme.colorScheme.onBackground
                )
            )
        },
        floatingActionButton = {
            FloatingActionButton(
                onClick = { showAddDialog = true },
                containerColor = MaterialTheme.colorScheme.primary,
                contentColor = MaterialTheme.colorScheme.onPrimary
            ) {
                Icon(Icons.Filled.Add, contentDescription = "Add Expense / Income")
            }
        }
    ) { padding ->
        Surface(
            modifier = modifier
                .fillMaxSize()
                .padding(padding),
            color = MaterialTheme.colorScheme.background
        ) {
            LazyColumn(
                modifier = Modifier
                    .fillMaxSize()
                    .padding(horizontal = 16.dp),
                verticalArrangement = Arrangement.spacedBy(10.dp)
            ) {
                item {
                    Spacer(modifier = Modifier.height(4.dp))
                    Text(
                        text = "Separate from sales profit. Tracks operational shop costs like rent, electricity, tea, staff salary, and external deposits.",
                        style = MaterialTheme.typography.bodySmall,
                        color = MaterialTheme.colorScheme.onSurfaceVariant
                    )
                    Spacer(modifier = Modifier.height(8.dp))
                }

                if (allExpenses.isEmpty()) {
                    item {
                        Text(
                            text = "No non-sale expenses or income recorded yet. Tap + to add one.",
                            style = MaterialTheme.typography.bodyMedium,
                            color = MaterialTheme.colorScheme.onSurfaceVariant
                        )
                    }
                } else {
                    items(allExpenses) { expense ->
                        Card(
                            modifier = Modifier.fillMaxWidth(),
                            colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surface),
                            shape = RoundedCornerShape(12.dp)
                        ) {
                            Row(
                                modifier = Modifier
                                    .fillMaxWidth()
                                    .padding(14.dp),
                                horizontalArrangement = Arrangement.SpaceBetween,
                                verticalAlignment = Alignment.CenterVertically
                            ) {
                                Column {
                                    Text(
                                        text = expense.category,
                                        style = MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.Bold),
                                        color = MaterialTheme.colorScheme.onSurface
                                    )
                                    if (!expense.notes.isNullOrBlank()) {
                                        Text(
                                            text = expense.notes,
                                            style = MaterialTheme.typography.bodySmall,
                                            color = MaterialTheme.colorScheme.onSurfaceVariant
                                        )
                                    }
                                    val dateStr = SimpleDateFormat("dd MMM yyyy, hh:mm a", Locale.getDefault()).format(Date(expense.timestamp))
                                    Text(
                                        text = dateStr,
                                        style = MaterialTheme.typography.bodySmall,
                                        color = MaterialTheme.colorScheme.onSurfaceVariant
                                    )
```

```
                                }

                                Text(
                                    text = "${if (expense.isIncome) "+" else "-"}${CostCalculator.formatPaiseToRupees(expense.amountPaise)}",
                                    style = MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.Bold),
                                    color = if (expense.isIncome) Color(0xFF75B890) else Color(0xFFE57373)
                                )
                            }
                        }
                    }

                    item { Spacer(modifier = Modifier.height(72.dp)) }
                }
            }
        }

        if (showAddDialog) {
            var isIncome by remember { mutableStateOf(false) }
            var category by remember { mutableStateOf("Rent") }
            var amountStr by remember { mutableStateOf("") }
            var notes by remember { mutableStateOf("") }
            var categoryExpanded by remember { mutableStateOf(false) }
            var entryTimestamp by remember { mutableStateOf(System.currentTimeMillis()) }
            val dialogContext = LocalContext.current

            val expenseCategories = listOf("Rent", "Electricity", "Staff Salary", "Tea & Refreshments", "Maintenance", "Packaging Supplies", "Other Expense")
            val incomeCategories = listOf("Capital Deposit", "Owner Cash Injection", "Old Loan Recovery", "Scrap Sale", "Other Income")

            AlertDialog(
                onDismissRequest = { showAddDialog = false },
                title = { Text(if (isIncome) "Record Other Money Received" else "Record Shop Expense") },
                text = {
                    Column(verticalArrangement = Arrangement.spacedBy(10.dp)) {
                        Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement = Arrangement.spacedBy(12.dp)) {
                            Row(verticalAlignment = Alignment.CenterVertically, modifier = Modifier.clickable { isIncome = false; category = "Rent" }) {
                                RadioButton(selected = !isIncome, onClick = { isIncome = false; category = "Rent" })
                                Text("Expense")
                            }
                            Row(verticalAlignment = Alignment.CenterVertically, modifier = Modifier.clickable { isIncome = true; category = "Capital Deposit" }) {
                                RadioButton(selected = isIncome, onClick = { isIncome = true; category = "Capital Deposit" })
                                Text("Other Income")
                            }
                        }

                        // Date & Time pickers
                        val cal = Calendar.getInstance().apply { timeInMillis = entryTimestamp }
                        val entryDateStr = SimpleDateFormat("dd MMM yyyy", Locale.US).format(Date(entryTimestamp))
                        val entryTimeStr = SimpleDateFormat("hh:mm a", Locale.US).format(Date(entryTimestamp))

                        Row(
                            modifier = Modifier.fillMaxWidth(),
                            horizontalArrangement = Arrangement.spacedBy(8.dp)
                        ) {
                            OutlinedButton(
                                onClick = {
                                    DatePickerDialog(
                                        dialogContext,
                                        { _, y, m, d ->
                                            val c = Calendar.getInstance().apply {
                                                timeInMillis = entryTimestamp
                                                set(Calendar.YEAR, y)
                                                set(Calendar.MONTH, m)
                                                set(Calendar.DAY_OF_MONTH, d)
                                            }
                                            entryTimestamp = c.timeInMillis
                                        },
                                        cal.get(Calendar.YEAR),
                                        cal.get(Calendar.MONTH),
                                        cal.get(Calendar.DAY_OF_MONTH)
                                    ).show()
                                },
                                shape = RoundedCornerShape(8.dp),
                                modifier = Modifier.weight(1.1f)
                            ) {
                                Icon(Icons.Filled.CalendarToday, contentDescription = null, modifier = Modifier.size(14.dp))
                                Spacer(modifier = Modifier.width(4.dp))
```

```
                            Text(entryDateStr, fontSize = 12.sp, maxLines = 1)
                        }

                        OutlinedButton(
                            onClick = {
                                TimePickerDialog(
                                    dialogContext,
                                    { _, hour, minute ->
                                        val c = Calendar.getInstance().apply {
                                            timeInMillis = entryTimestamp
                                            set(Calendar.HOUR_OF_DAY, hour)
                                            set(Calendar.MINUTE, minute)
                                        }
                                        entryTimestamp = c.timeInMillis
                                    },
                                    cal.get(Calendar.HOUR_OF_DAY),
                                    cal.get(Calendar.MINUTE),
                                    false
                                ).show()
                            },
                            shape = RoundedCornerShape(8.dp),
                            modifier = Modifier.weight(0.9f)
                        ) {
                            Icon(Icons.Filled.AccessTime, contentDescription = null, modifier =
    Modifier.size(14.dp))
                            Spacer(modifier = Modifier.width(4.dp))
                            Text(entryTimeStr, fontSize = 12.sp, maxLines = 1)
                        }
                    }

                    // Category Dropdown
                    ExposedDropdownMenuBox(
                        expanded = categoryExpanded,
                        onExpandedChange = { categoryExpanded = !categoryExpanded }
                    ) {
                        OutlinedTextField(
                            value = category,
                            onValueChange = {},
                            readOnly = true,
                            label = { Text("Category") },
                            trailingIcon = { ExposedDropdownMenuDefaults.TrailingIcon(expanded =
    categoryExpanded) },
                            modifier = Modifier
                                .fillMaxWidth()
                                .menuAnchor(),
                            shape = RoundedCornerShape(8.dp)
                        )
                        ExposedDropdownMenu(
                            expanded = categoryExpanded,
                            onDismissRequest = { categoryExpanded = false }
                        ) {
                            val options = if (isIncome) incomeCategories else expenseCategories
                            options.forEach { opt ->
                                DropdownMenuItem(
                                    text = { Text(opt) },
                                    onClick = {
                                        category = opt
                                        categoryExpanded = false
                                    }
                                )
                            }
                        }
                    }

                    OutlinedTextField(
                        value = amountStr,
                        onValueChange = { amountStr = it },
                        label = { Text("Amount (₹) *") },
                        keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Decimal),
                        singleLine = true,
                        modifier = Modifier.fillMaxWidth()
                    )

                    OutlinedTextField(
                        value = notes,
                        onValueChange = { notes = it },
                        label = { Text("Notes / Description") },
                        singleLine = true,
                        modifier = Modifier.fillMaxWidth()
                    )
                }
            },
            confirmButton = {
                Button(
                    onClick = {
                        val amountPaise = CostCalculator.parseRupeesToPaise(amountStr) ?: 0L
                        if (amountPaise <= 0) return@Button

```

```
                        viewModel.recordShopExpense(
                            category = category,
                            amountPaise = amountPaise,
                            isIncome = isIncome,
                            notes = notes.trim().ifBlank { null },
                            timestamp = entryTimestamp,
                            onDone = { showAddDialog = false }
                        )
                    }
                ) { Text("Save") }
            },
            dismissButton = {
                OutlinedButton(onClick = { showAddDialog = false }) { Text("Cancel") }
            }
        )
    }
}
```

app\src\main\java\com\alieducational\app\ui\more\MoreScreen.kt

```
package com.alieducational.app.ui.more

import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.verticalScroll
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Assessment
import androidx.compose.material.icons.filled.Backup
import androidx.compose.material.icons.filled.ChevronRight
import androidx.compose.material.icons.filled.FileDownload
import androidx.compose.material.icons.filled.MoneyOff
import androidx.compose.material.icons.filled.PointOfSale
import androidx.compose.material.icons.filled.Receipt
import androidx.compose.material.icons.filled.Settings
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.Icon
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.vector.ImageVector
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.alieducational.app.ui.AliViewModel

@Composable
fun MoreScreen(
    viewModel: AliViewModel,
    onNavigateToProfit: () -> Unit,
    onNavigateToDailySales: () -> Unit,
    onNavigateToExpenses: () -> Unit,
    onNavigateToImport: () -> Unit,
    onNavigateToBackup: () -> Unit,
    onNavigateToSettings: () -> Unit,
    modifier: Modifier = Modifier
) {
    Surface(
        modifier = modifier.fillMaxSize(),
        color = MaterialTheme.colorScheme.background
    ) {
        Column(
            modifier = Modifier
                .fillMaxSize()
                .verticalScroll(rememberScrollState())
                .padding(16.dp),
            verticalArrangement = Arrangement.spacedBy(12.dp)
        ) {
            Spacer(modifier = Modifier.height(8.dp))
            Text(
                text = "Reports, Tools & Settings",
                style = MaterialTheme.typography.headlineSmall.copy(fontWeight = FontWeight.Bold),
                color = MaterialTheme.colorScheme.onBackground
            )
            Text(
                text = "Secondary tasks kept off the home screen to preserve everyday simplicity",
                style = MaterialTheme.typography.bodySmall,
                color = MaterialTheme.colorScheme.onSurfaceVariant
            )

            Spacer(modifier = Modifier.height(8.dp))

            // Reports & Analysis Group
            Text(
                text = "REPORTS & FINANCIALS",
                style = MaterialTheme.typography.labelMedium.copy(fontWeight = FontWeight.Bold),
                color = MaterialTheme.colorScheme.primary
            )
```

```
83
84              MoreMenuItem(
85                  title = "Gross Profit Analysis",
86                  subtitle = "Expected vs. actual gross profit after weighted costs",
87                  icon = Icons.Filled.Assessment,
88                  onClick = onNavigateToProfit
89              )
90
91              MoreMenuItem(
92                  title = "Daily Sales & Records",
93                  subtitle = "Daily net sales, wholesale discounts, and money totals",
94                  icon = Icons.Filled.PointOfSale,
95                  onClick = onNavigateToDailySales
96              )
97
98              MoreMenuItem(
99                  title = "Other Money & Expenses",
100                 subtitle = "Shop rent, electricity, salaries, and non-sale deposits",
101                 icon = Icons.Filled.MoneyOff,
102                 onClick = onNavigateToExpenses
103             )
104
105             Spacer(modifier = Modifier.height(8.dp))
106
107             // Management & Tools Group
108             Text(
109                 text = "DATA & BACKUP",
110                 style = MaterialTheme.typography.labelMedium.copy(fontWeight = FontWeight.Bold),
111                 color = MaterialTheme.colorScheme.primary
112             )
113
114             MoreMenuItem(
115                 title = "Import Product Catalog",
116                 subtitle = "Import Excel (.xlsx / .xls) or CSV spreadsheet",
117                 icon = Icons.Filled.FileDownload,
118                 onClick = onNavigateToImport
119             )
120
121             MoreMenuItem(
122                 title = "Backup & Restore",
123                 subtitle = "Save encrypted local backup ZIP and restore data safely",
124                 icon = Icons.Filled.Backup,
125                 onClick = onNavigateToBackup
126             )
127
128             Spacer(modifier = Modifier.height(8.dp))
129
130             // App Settings
131             Text(
132                 text = "PREFERENCES",
133                 style = MaterialTheme.typography.labelMedium.copy(fontWeight = FontWeight.Bold),
134                 color = MaterialTheme.colorScheme.primary
135             )
136
137             MoreMenuItem(
138                 title = "Settings",
139                 subtitle = "Theme, language (English / Urdu), receipts, and biometric lock",
140                 icon = Icons.Filled.Settings,
141                 onClick = onNavigateToSettings
142             )
143
144             Spacer(modifier = Modifier.height(24.dp))
145         }
146     }
147 }
148
149 @Composable
150 fun MoreMenuItem(
151     title: String,
152     subtitle: String,
153     icon: ImageVector,
154     onClick: () -> Unit
155 ) {
156     Card(
157         modifier = Modifier
158             .fillMaxWidth()
159             .clickable { onClick() },
160         colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surface),
161         shape = RoundedCornerShape(12.dp)
162     ) {
163         Row(
164             modifier = Modifier
165                 .fillMaxWidth()
166                 .padding(16.dp),
167             verticalAlignment = Alignment.CenterVertically,
168             horizontalArrangement = Arrangement.SpaceBetween
169         ) {
170             Row(
```

```
                modifier = Modifier.weight(1f),
                verticalAlignment = Alignment.CenterVertically
            ) {
                Icon(
                    imageVector = icon,
                    contentDescription = null,
                    tint = MaterialTheme.colorScheme.primary,
                    modifier = Modifier.size(26.dp)
                )
                Spacer(modifier = Modifier.width(14.dp))
                Column {
                    Text(
                        text = title,
                        style = MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.SemiBold),
                        color = MaterialTheme.colorScheme.onSurface
                    )
                    Text(
                        text = subtitle,
                        style = MaterialTheme.typography.bodySmall,
                        color = MaterialTheme.colorScheme.onSurfaceVariant
                    )
                }
            }
            Icon(
                imageVector = Icons.Filled.ChevronRight,
                contentDescription = null,
                tint = MaterialTheme.colorScheme.onSurfaceVariant,
                modifier = Modifier.size(20.dp)
            )
        }
    }
}
```

app\src\main\java\com\alieducational\app\ui\navigation\AppNavigation.kt

```
package com.alieducational.app.ui.navigation

import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.AccountBalanceWallet
import androidx.compose.material.icons.filled.Home
import androidx.compose.material.icons.filled.Inventory2
import androidx.compose.material.icons.filled.MoreHoriz
import androidx.compose.material.icons.filled.PointOfSale
import androidx.compose.material.icons.outlined.AccountBalanceWallet
import androidx.compose.material.icons.outlined.Home
import androidx.compose.material.icons.outlined.Inventory2
import androidx.compose.material.icons.outlined.MoreHoriz
import androidx.compose.material.icons.outlined.PointOfSale
import androidx.compose.ui.graphics.vector.ImageVector
import com.alieducational.app.R

sealed class Screen(val route: String) {
    // 4 Primary Bottom Bar Tabs
    data object Home : Screen("home")
    data object Stock : Screen("stock")
    data object Billing : Screen("billing")
    data object Accounts : Screen("accounts")

    // Full Counter Billing Workflow
    data object BarcodeScanner : Screen("barcode_scanner")
    data object ReviewBill : Screen("review_bill")
    data object Payment : Screen("payment")
    data object BillReceipt : Screen("bill_receipt/{receiptNumber}") {
        fun createRoute(num: String) = "bill_receipt/$num"
    }
    data object BillHistory : Screen("bill_history")

    // Secondary Management & Reports (Accessible from Home)
    data object More : Screen("more")

    // Auth & Setup
    data object Unlock : Screen("unlock")
    data object Setup : Screen("setup")

    // Workflows
    data object ReceiveStock : Screen("receive_stock")
    data object RemoveStock : Screen("remove_stock")
    data object RecordSale : Screen("record_sale")
    data object RecordDailyTotal : Screen("record_daily_total")
    data object ImportCatalog : Screen("import_catalog")
    data object ProfitAnalysis : Screen("profit_analysis")
    data object Expenses : Screen("expenses")
    data object CashTracker : Screen("cash_tracker")
    data object BackupRestore : Screen("backup_restore")
    data object PigmySavings : Screen("pigmy_savings")
    data object Settings : Screen("settings")
}

data class BottomNavItem(
    val screen: Screen,
    val labelResId: Int,
    val selectedIcon: ImageVector,
    val unselectedIcon: ImageVector,
)

val bottomNavItems = listOf(
    BottomNavItem(
        screen = Screen.Home,
        labelResId = R.string.nav_home,
        selectedIcon = Icons.Filled.Home,
        unselectedIcon = Icons.Outlined.Home,
    ),
    BottomNavItem(
        screen = Screen.Stock,
        labelResId = R.string.nav_stock,
        selectedIcon = Icons.Filled.Inventory2,
        unselectedIcon = Icons.Outlined.Inventory2,
    ),
    BottomNavItem(
        screen = Screen.Billing,
        labelResId = R.string.nav_billing,
        selectedIcon = Icons.Filled.PointOfSale,
        unselectedIcon = Icons.Outlined.PointOfSale,
    ),
    BottomNavItem(
        screen = Screen.Accounts,
        labelResId = R.string.nav_accounts,
```

```
83          selectedIcon = Icons.Filled.AccountBalanceWallet,
84          unselectedIcon = Icons.Outlined.AccountBalanceWallet,
85      ),
86  )
```

app\src\main\java\com\alieducational\app\ui\navigation\BottomNavBar.kt

```
package com.alieducational.app.ui.navigation

import androidx.compose.material3.Icon
import androidx.compose.material3.NavigationBar
import androidx.compose.material3.NavigationBarItem
import androidx.compose.material3.NavigationBarItemDefaults
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.ui.res.stringResource
import androidx.compose.ui.unit.dp
import androidx.navigation.NavGraph.Companion.findStartDestination
import androidx.navigation.NavHostController
import androidx.navigation.compose.currentBackStackEntryAsState

@Composable
fun BottomNavBar(navController: NavHostController) {
    val navBackStackEntry by navController.currentBackStackEntryAsState()
    val currentRoute = navBackStackEntry?.destination?.route

    NavigationBar(
        containerColor = MaterialTheme.colorScheme.surface,
        contentColor = MaterialTheme.colorScheme.onSurface,
        tonalElevation = 8.dp
    ) {
        bottomNavItems.forEach { item ->
            val selected = currentRoute == item.screen.route
            NavigationBarItem(
                selected = selected,
                onClick = {
                    if (currentRoute != item.screen.route) {
                        if (item.screen.route == Screen.Home.route) {
                            if (!navController.popBackStack(Screen.Home.route, inclusive = false)) {
                                navController.navigate(Screen.Home.route) {
                                    launchSingleTop = true
                                }
                            }
                        } else {
                            navController.navigate(item.screen.route) {
                                popUpTo(navController.graph.findStartDestination().id) {
                                    saveState = true
                                }
                                launchSingleTop = true
                                restoreState = true
                            }
                        }
                    }
                },
                icon = {
                    Icon(
                        imageVector = if (selected) item.selectedIcon else item.unselectedIcon,
                        contentDescription = stringResource(item.labelResId),
                    )
                },
                label = {
                    Text(text = stringResource(item.labelResId))
                },
                colors = NavigationBarItemDefaults.colors(
                    selectedIconColor = MaterialTheme.colorScheme.primary,
                    selectedTextColor = MaterialTheme.colorScheme.primary,
                    unselectedIconColor = MaterialTheme.colorScheme.onSurfaceVariant,
                    unselectedTextColor = MaterialTheme.colorScheme.onSurfaceVariant,
                    indicatorColor = MaterialTheme.colorScheme.primaryContainer,
                ),
            )
        }
    }
}
```

app\src\main\java\com\alieducational\app\ui\navigation\NavHost.kt

```
package com.alieducational.app.ui.navigation

import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.navigation.NavHostController
import androidx.navigation.NavType
import androidx.navigation.compose.NavHost
import androidx.navigation.compose.composable
import androidx.navigation.navArgument
import com.alieducational.app.ui.AliViewModel
import com.alieducational.app.ui.accounts.AccountsScreen
import com.alieducational.app.ui.auth.UnlockScreen
import com.alieducational.app.ui.billing.BarcodeScannerScreen
import com.alieducational.app.ui.billing.BillReceiptScreen
import com.alieducational.app.ui.billing.BillHistoryScreen
import com.alieducational.app.ui.billing.BillingScreen
import com.alieducational.app.ui.billing.PaymentScreen
import com.alieducational.app.ui.billing.ReviewBillScreen
import com.alieducational.app.ui.home.HomeScreen
import com.alieducational.app.ui.more.BackupRestoreScreen
import com.alieducational.app.ui.more.ExpensesScreen
import com.alieducational.app.ui.reports.CashTrackerScreen
import com.alieducational.app.ui.more.MoreScreen
import com.alieducational.app.ui.reports.ProfitScreen
import com.alieducational.app.ui.pigmy.PigmySavingsScreen
import com.alieducational.app.ui.sales.RecordDailyTotalScreen
import com.alieducational.app.ui.sales.RecordSaleScreen
import com.alieducational.app.ui.settings.SettingsScreen
import com.alieducational.app.ui.setup.SetupScreen
import com.alieducational.app.ui.stock.ImportCatalogScreen
import com.alieducational.app.ui.stock.ReceiveStockScreen
import com.alieducational.app.ui.stock.RemoveStockScreen
import com.alieducational.app.ui.stock.StockScreen
import com.alieducational.app.util.BiometricAuthManager

@Composable
fun AppNavigation(
    navController: NavHostController,
    startDestination: String,
    viewModel: AliViewModel,
    biometricAuthManager: BiometricAuthManager,
    onUnlocked: () -> Unit = {},
    modifier: Modifier = Modifier,
) {
    NavHost(
        navController = navController,
        startDestination = startDestination,
        modifier = modifier,
    ) {
        // 4 Primary Bottom Bar Screens
        composable(Screen.Home.route) {
            HomeScreen(
                viewModel = viewModel,
                onNavigateToNewBill = {
                    navController.navigate(Screen.Billing.route) {
                        launchSingleTop = true
                    }
                },
                onNavigateToAddStock = {
                    navController.navigate(Screen.Stock.route) {
                        launchSingleTop = true
                    }
                },
                onNavigateToRecordPayment = {
                    navController.navigate(Screen.Accounts.route) {
                        launchSingleTop = true
                    }
                },
                onNavigateToStock = {
                    navController.navigate(Screen.Stock.route) {
                        launchSingleTop = true
                    }
                },
                onNavigateToAccounts = { tab ->
                    navController.navigate(Screen.Accounts.route) {
                        launchSingleTop = true
                    }
                },
                onNavigateToProfit = { navController.navigate(Screen.ProfitAnalysis.route) },
                onNavigateToExpenses = { navController.navigate(Screen.Expenses.route) },
                onNavigateToCashTracker = { navController.navigate(Screen.CashTracker.route) },
                onNavigateToPigmy = { navController.navigate(Screen.PigmySavings.route) },
```

```
                onNavigateToBackup = { navController.navigate(Screen.BackupRestore.route) },
                onNavigateToSettings = { navController.navigate(Screen.Settings.route) },
                onNavigateToBillHistory = { navController.navigate(Screen.BillHistory.route) },
            )
        }

        composable(Screen.Stock.route) {
            StockScreen(
                viewModel = viewModel,
                onNavigateToReceiveStock = { navController.navigate(Screen.ReceiveStock.route) },
                onNavigateToImport = { navController.navigate(Screen.ImportCatalog.route) },
                onNavigateToRemoveStock = { navController.navigate(Screen.RemoveStock.route) },
                onNavigateBack = {
                    if (!navController.popBackStack(Screen.Home.route, inclusive = false)) {
                        navController.navigate(Screen.Home.route) {
                            launchSingleTop = true
                        }
                    }
                }
            )
        }

        composable(Screen.Billing.route) {
            BillingScreen(
                viewModel = viewModel,
                onNavigateToScanner = { navController.navigate(Screen.BarcodeScanner.route) },
                onNavigateToReview = { navController.navigate(Screen.ReviewBill.route) },
                onNavigateToBillHistory = { navController.navigate(Screen.BillHistory.route) },
                onNavigateBack = {
                    if (!navController.popBackStack(Screen.Home.route, inclusive = false)) {
                        navController.navigate(Screen.Home.route) {
                            launchSingleTop = true
                        }
                    }
                }
            )
        }

        composable(Screen.BillHistory.route) {
            BillHistoryScreen(
                viewModel = viewModel,
                onNavigateBack = { navController.popBackStack() },
                onNavigateToReceipt = { receiptNumber ->
                    navController.navigate(Screen.BillReceipt.createRoute(receiptNumber))
                },
                onNavigateToCustomerAccount = { customerId ->
                    navController.navigate(Screen.Accounts.route) {
                        launchSingleTop = true
                    }
                }
            )
        }

        composable(Screen.Accounts.route) {
            AccountsScreen(
                viewModel = viewModel,
                onNavigateBack = {
                    if (!navController.popBackStack(Screen.Home.route, inclusive = false)) {
                        navController.navigate(Screen.Home.route) {
                            launchSingleTop = true
                        }
                    }
                }
            )
        }

        // Full Counter Billing Workflow Screens
        composable(Screen.BarcodeScanner.route) {
            BarcodeScannerScreen(
                viewModel = viewModel,
                onNavigateBack = { navController.popBackStack() },
                onNavigateToReview = { navController.navigate(Screen.ReviewBill.route) }
            )
        }

        composable(Screen.ReviewBill.route) {
            ReviewBillScreen(
                viewModel = viewModel,
                onNavigateBack = { navController.popBackStack() },
                onNavigateToScanner = { navController.navigate(Screen.BarcodeScanner.route) },
                onNavigateToPayment = { navController.navigate(Screen.Payment.route) }
            )
        }

        composable(Screen.Payment.route) {
            PaymentScreen(
                viewModel = viewModel,
                onNavigateBack = { navController.popBackStack() },
```

```
                onBillSaved = { receiptNum ->
                    navController.navigate(Screen.BillReceipt.createRoute(receiptNum)) {
                        popUpTo(Screen.Billing.route) { inclusive = false }
                    }
                }
            )
        }

        composable(
            route = Screen.BillReceipt.route,
            arguments = listOf(navArgument("receiptNumber") { type = NavType.StringType })
        ) { backStackEntry ->
            val receiptNum = backStackEntry.arguments?.getString("receiptNumber") ?: "AE-RECEIPT"
            BillReceiptScreen(
                receiptNumber = receiptNum,
                viewModel = viewModel,
                onNewBill = {
                    navController.navigate(Screen.Billing.route) {
                        popUpTo(Screen.Billing.route) { inclusive = true }
                    }
                },
                onDone = {
                    navController.navigate(Screen.Home.route) {
                        popUpTo(Screen.Home.route) { inclusive = true }
                    }
                }
            )
        }

        // Legacy / Secondary Management & Reports
        composable(Screen.More.route) {
            MoreScreen(
                viewModel = viewModel,
                onNavigateToProfit = { navController.navigate(Screen.ProfitAnalysis.route) },
                onNavigateToDailySales = { navController.navigate(Screen.RecordDailyTotal.route) },
                onNavigateToExpenses = { navController.navigate(Screen.Expenses.route) },
                onNavigateToImport = { navController.navigate(Screen.ImportCatalog.route) },
                onNavigateToBackup = { navController.navigate(Screen.BackupRestore.route) },
                onNavigateToSettings = { navController.navigate(Screen.Settings.route) },
            )
        }

        // Auth & Setup
        composable(Screen.Unlock.route) {
            UnlockScreen(
                biometricAuthManager = biometricAuthManager,
                onUnlocked = {
                    onUnlocked()
                    navController.navigate(Screen.Home.route) {
                        popUpTo(Screen.Unlock.route) { inclusive = true }
                    }
                }
            )
        }

        composable(Screen.Setup.route) {
            SetupScreen(
                viewModel = viewModel,
                onSetupComplete = {
                    navController.navigate(Screen.Home.route) {
                        popUpTo(Screen.Setup.route) { inclusive = true }
                    }
                }
            )
        }

        // Workflows
        composable(Screen.ReceiveStock.route) {
            ReceiveStockScreen(
                viewModel = viewModel,
                onNavigateBack = { navController.popBackStack() }
            )
        }

        composable(Screen.RemoveStock.route) {
            RemoveStockScreen(
                viewModel = viewModel,
                onNavigateBack = { navController.popBackStack() },
                onNavigateToRecordSale = {
                    navController.navigate(Screen.RecordSale.route) {
                        popUpTo(Screen.RemoveStock.route) { inclusive = true }
                    }
                }
            )
        }

        composable(Screen.RecordSale.route) {
            RecordSaleScreen(
```

```
                viewModel = viewModel,
                onNavigateBack = { navController.popBackStack() }
            )
        }

        composable(Screen.RecordDailyTotal.route) {
            RecordDailyTotalScreen(
                viewModel = viewModel,
                onNavigateBack = { navController.popBackStack() }
            )
        }

        composable(Screen.ImportCatalog.route) {
            ImportCatalogScreen(
                viewModel = viewModel,
                onNavigateBack = { navController.popBackStack() }
            )
        }

        composable(Screen.ProfitAnalysis.route) {
            ProfitScreen(
                viewModel = viewModel,
                onNavigateBack = { navController.popBackStack() }
            )
        }

        composable(Screen.Expenses.route) {
            ExpensesScreen(
                viewModel = viewModel,
                onNavigateBack = { navController.popBackStack() }
            )
        }

        composable(Screen.CashTracker.route) {
            CashTrackerScreen(
                viewModel = viewModel,
                onNavigateBack = { navController.popBackStack() }
            )
        }

        composable(Screen.BackupRestore.route) {
            BackupRestoreScreen(
                viewModel = viewModel,
                onNavigateBack = { navController.popBackStack() }
            )
        }

        composable(Screen.PigmySavings.route) {
            PigmySavingsScreen(
                viewModel = viewModel,
                onNavigateBack = { navController.popBackStack() }
            )
        }

        composable(Screen.Settings.route) {
            SettingsScreen(
                viewModel = viewModel,
                onNavigateBack = { navController.popBackStack() }
            )
        }
    }
}
```

app\src\main\java\com\alieducational\app\ui\pigmy\PigmySavingsScreen.kt

```
package com.alieducational.app.ui.pigmy

import android.app.DatePickerDialog
import android.app.TimePickerDialog
import androidx.compose.animation.AnimatedVisibility
import androidx.compose.foundation.background
import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.automirrored.filled.ArrowBack
import androidx.compose.material.icons.filled.AccountBalance
import androidx.compose.material.icons.filled.Add
import androidx.compose.material.icons.filled.Archive
import androidx.compose.material.icons.filled.Block
import androidx.compose.material.icons.filled.CalendarToday
import androidx.compose.material.icons.filled.CheckCircle
import androidx.compose.material.icons.filled.Edit
import androidx.compose.material.icons.filled.MoreVert
import androidx.compose.material.icons.filled.ExpandLess
import androidx.compose.material.icons.filled.ExpandMore
import androidx.compose.material.icons.filled.Remove
import androidx.compose.material.icons.filled.Unarchive
import androidx.compose.material.icons.filled.Warning
import androidx.compose.material3.AlertDialog
import androidx.compose.material3.Button
import androidx.compose.material3.ButtonDefaults
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.DropdownMenu
import androidx.compose.material3.DropdownMenuItem
import androidx.compose.material.icons.filled.Unarchive
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.FilterChip
import androidx.compose.material3.HorizontalDivider
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedButton
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Surface
import androidx.compose.material3.Tab
import androidx.compose.material3.TabRow
import androidx.compose.material3.Text
import androidx.compose.material3.TextButton
import androidx.compose.material3.TopAppBar
import androidx.compose.material3.TopAppBarDefaults
import androidx.compose.runtime.Composable
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableIntStateOf
import androidx.compose.runtime.mutableLongStateOf
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.res.painterResource
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.text.style.TextDecoration
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.alieducational.app.R
import com.alieducational.app.data.local.entity.PigmyDepositEntity
import com.alieducational.app.data.local.entity.PigmySchemeEntity
```

```
import com.alieducational.app.data.local.entity.PigmyWithdrawalEntity
import com.alieducational.app.domain.calculation.CostCalculator
import com.alieducational.app.ui.AliViewModel
import java.text.SimpleDateFormat
import java.util.Calendar
import java.util.Date
import java.util.Locale

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun PigmySavingsScreen(
    viewModel: AliViewModel,
    onNavigateBack: () -> Unit,
    modifier: Modifier = Modifier
) {
    val allSchemes by viewModel.allPigmySchemes.collectAsState()
    val allDeposits by viewModel.allPigmyDeposits.collectAsState()
    val allWithdrawals by viewModel.allPigmyWithdrawals.collectAsState()

    var showAddSchemeDialog by remember { mutableStateOf(false) }
    var schemeToEdit by remember { mutableStateOf<PigmySchemeEntity?>(null) }
    var schemeForDeposit by remember { mutableStateOf<PigmySchemeEntity?>(null) }
    var schemeForWithdrawal by remember { mutableStateOf<PigmySchemeEntity?>(null) }
    var schemeForOpeningBalance by remember { mutableStateOf<PigmySchemeEntity?>(null) }

    var depositToEdit by remember { mutableStateOf<PigmyDepositEntity?>(null) }
    var depositToVoid by remember { mutableStateOf<PigmyDepositEntity?>(null) }
    var withdrawalToEdit by remember { mutableStateOf<PigmyWithdrawalEntity?>(null) }
    var withdrawalToVoid by remember { mutableStateOf<PigmyWithdrawalEntity?>(null) }

    var voidReason by remember { mutableStateOf("") }
    var showArchivedSchemes by remember { mutableStateOf(false) }

    val displayedSchemes = remember(allSchemes, showArchivedSchemes) {
        if (showArchivedSchemes) allSchemes else allSchemes.filter { !it.isArchived }
    }

    // Combined summary calculations
    val combinedOpeningPaise = remember(allSchemes) {
        allSchemes.sumOf { it.openingBalancePaise }
    }
    val combinedRecordedDepositsPaise = remember(allDeposits) {
        allDeposits.filter { !it.isVoided }.sumOf { it.amountPaise }
    }
    val combinedWithdrawalsPaise = remember(allWithdrawals) {
        allWithdrawals.filter { !it.isVoided }.sumOf { it.amountPaise }
    }
    val combinedTrackedSavingsPaise = combinedOpeningPaise + combinedRecordedDepositsPaise -
combinedWithdrawalsPaise

    val todayCombinedDepositsPaise = remember(allDeposits) {
        val startOfDay = Calendar.getInstance().apply {
            set(Calendar.HOUR_OF_DAY, 0)
            set(Calendar.MINUTE, 0)
            set(Calendar.SECOND, 0)
            set(Calendar.MILLISECOND, 0)
        }.timeInMillis
        allDeposits.filter { !it.isVoided && it.paidTimestamp >= startOfDay }.sumOf { it.amountPaise }
    }

    Scaffold(
        topBar = {
            TopAppBar(
                title = {
                    Row(verticalAlignment = Alignment.CenterVertically) {
                        Icon(
                            painter = painterResource(id = R.drawable.ic_pigmy_coin),
                            contentDescription = "Pigmy savings",
                            tint = Color(0xFFD97706),
                            modifier = Modifier.size(24.dp)
                        )
                        Spacer(modifier = Modifier.width(8.dp))
                        Text("Pigmy Savings", fontWeight = FontWeight.Bold)
                    }
                },
                navigationIcon = {
                    IconButton(onClick = onNavigateBack) {
                        Icon(Icons.AutoMirrored.Filled.ArrowBack, contentDescription = "Back")
                    }
                },
                actions = {
                    TextButton(onClick = { showAddSchemeDialog = true }) {
                        Icon(Icons.Filled.Add, contentDescription = null, modifier = Modifier.size(18.dp))
                        Spacer(modifier = Modifier.width(4.dp))
                        Text("Add Scheme", fontWeight = FontWeight.Bold)
                    }
                },
                colors = TopAppBarDefaults.topAppBarColors(
```

```
                    containerColor = MaterialTheme.colorScheme.background,
                    titleContentColor = MaterialTheme.colorScheme.onBackground
                )
            )
        }
    ) { padding ->
        Surface(
            modifier = modifier
                .fillMaxSize()
                .padding(padding),
            color = MaterialTheme.colorScheme.background
        ) {
            LazyColumn(
                modifier = Modifier
                    .fillMaxSize()
                    .padding(horizontal = 16.dp),
                verticalArrangement = Arrangement.spacedBy(14.dp)
            ) {
                // Combined Summary Card at Top
                item {
                    Card(
                        modifier = Modifier.fillMaxWidth(),
                        shape = RoundedCornerShape(16.dp),
                        colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surface)
                    ) {
                        Column(modifier = Modifier.padding(18.dp)) {
                            Row(verticalAlignment = Alignment.CenterVertically) {
                                Icon(
                                    painter = painterResource(id = R.drawable.ic_pigmy_coin),
                                    contentDescription = "Pigmy savings",
                                    tint = Color(0xFFD97706),
                                    modifier = Modifier.size(28.dp)
                                )
                                Spacer(modifier = Modifier.width(10.dp))
                                Column {
                                    Text(
                                        text = "Tracked Savings Balance",
                                        style = MaterialTheme.typography.titleMedium.copy(fontWeight =
FontWeight.Bold),
                                        color = MaterialTheme.colorScheme.onSurface
                                    )
                                    Text(
                                        text = "Combined savings • Returned savings without interest",
                                        style = MaterialTheme.typography.bodySmall,
                                        color = MaterialTheme.colorScheme.onSurfaceVariant
                                    )
                                }
                            }

                            Spacer(modifier = Modifier.height(10.dp))

                            Text(
                                text = CostCalculator.formatPaiseToRupees(combinedTrackedSavingsPaise),
                                fontSize = 28.sp,
                                fontWeight = FontWeight.Bold,
                                color = MaterialTheme.colorScheme.primary
                            )

                            Spacer(modifier = Modifier.height(10.dp))
                            HorizontalDivider(color = MaterialTheme.colorScheme.outlineVariant.copy(alpha =
0.5f))
                            Spacer(modifier = Modifier.height(10.dp))

                            Row(
                                modifier = Modifier.fillMaxWidth(),
                                horizontalArrangement = Arrangement.SpaceBetween
                            ) {
                                Column {
                                    Text("Total Deposits", style = MaterialTheme.typography.labelSmall,
color = Color(0xFF2E7D32))
                                    Text(
                                        CostCalculator.formatPaiseToRupees(combinedRecordedDepositsPaise),
                                        fontWeight = FontWeight.Bold,
                                        color = Color(0xFF2E7D32)
                                    )
                                }
                                Column(horizontalAlignment = Alignment.CenterHorizontally) {
                                    Text("Total Withdrawals", style = MaterialTheme.typography.labelSmall,
color = Color(0xFFD97706))
                                    Text(
                                        CostCalculator.formatPaiseToRupees(combinedWithdrawalsPaise),
                                        fontWeight = FontWeight.Bold,
                                        color = Color(0xFFD97706)
                                    )
                                }
                                if (combinedOpeningPaise > 0) {
                                    Column(horizontalAlignment = Alignment.End) {
                                        Text("Opening", style = MaterialTheme.typography.labelSmall, color =
```

```
     MaterialTheme.colorScheme.onSurfaceVariant)
                                        Text(
                                            CostCalculator.formatPaiseToRupees(combinedOpeningPaise),
                                            fontWeight = FontWeight.SemiBold,
                                            color = MaterialTheme.colorScheme.onSurface
                                        )
                                    }
                                } else {
                                    Column(horizontalAlignment = Alignment.End) {
                                        Text("Today's Deposits", style =
    MaterialTheme.typography.labelSmall, color = MaterialTheme.colorScheme.onSurfaceVariant)
                                        Text(
                                            CostCalculator.formatPaiseToRupees(todayCombinedDepositsPaise),
                                            fontWeight = FontWeight.SemiBold,
                                            color = MaterialTheme.colorScheme.primary
                                        )
                                    }
                                }
                            }
                        }
                    }
                }

                // Scheme Filter Toggle Row
                item {
                    Row(
                        modifier = Modifier.fillMaxWidth(),
                        horizontalArrangement = Arrangement.SpaceBetween,
                        verticalAlignment = Alignment.CenterVertically
                    ) {
                        Text(
                            text = "Pigmy Schemes",
                            style = MaterialTheme.typography.titleSmall.copy(fontWeight = FontWeight.Bold),
                            color = MaterialTheme.colorScheme.onSurface
                        )
                        Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
                            FilterChip(
                                selected = !showArchivedSchemes,
                                onClick = { showArchivedSchemes = false },
                                label = { Text("Active") }
                            )
                            FilterChip(
                                selected = showArchivedSchemes,
                                onClick = { showArchivedSchemes = true },
                                label = { Text("All (${allSchemes.size})") }
                            )
                        }
                    }
                }

                // Scheme Cards
                if (displayedSchemes.isEmpty()) {
                    item {
                        Card(
                            modifier = Modifier.fillMaxWidth(),
                            shape = RoundedCornerShape(12.dp),
                            colors = CardDefaults.cardColors(containerColor =
    MaterialTheme.colorScheme.surface)
                        ) {
                            Column(
                                modifier = Modifier
                                    .fillMaxWidth()
                                    .padding(24.dp),
                                horizontalAlignment = Alignment.CenterHorizontally
                            ) {
                                Text(
                                    text = "No schemes found. Tap 'Add Scheme' above to create one.",
                                    style = MaterialTheme.typography.bodyMedium,
                                    color = MaterialTheme.colorScheme.onSurfaceVariant
                                )
                            }
                        }
                    }
                } else {
                    items(displayedSchemes, key = { it.id }) { scheme ->
                        val schemeDeposits = remember(allDeposits, scheme.id) {
                            allDeposits.filter { it.accountSchemeId == scheme.id }
                        }

                        val schemeWithdrawals = remember(allWithdrawals, scheme.id) {
                            allWithdrawals.filter { it.accountSchemeId == scheme.id }
                        }

                        PigmySchemeCard(
                            scheme = scheme,
                            deposits = schemeDeposits,
                            withdrawals = schemeWithdrawals,
                            onAddDeposit = { schemeForDeposit = scheme },
                            onAddWithdrawal = { schemeForWithdrawal = scheme },
```

```
                            onSetOpeningBalance = { schemeForOpeningBalance = scheme },
                            onEditScheme = { schemeToEdit = scheme },
                            onArchiveToggle = { isArchived ->
                                viewModel.setPigmySchemeArchived(scheme.id, isArchived)
                            },
                            onEditDeposit = { deposit -> depositToEdit = deposit },
                            onVoidDeposit = { deposit -> depositToVoid = deposit },
                            onEditWithdrawal = { withdrawal -> withdrawalToEdit = withdrawal },
                            onVoidWithdrawal = { withdrawal -> withdrawalToVoid = withdrawal }
                        )
                    }
                }

                item {
                    Spacer(modifier = Modifier.height(24.dp))
                }
            }
        }
    }

    // Dialog: Add Scheme
    if (showAddSchemeDialog) {
        AddOrEditSchemeDialog(
            existingSchemes = allSchemes,
            initialScheme = null,
            onDismiss = { showAddSchemeDialog = false },
            onConfirm = { name, usualAmountPaise ->
                viewModel.createPigmyScheme(name, usualAmountPaise) {
                    showAddSchemeDialog = false
                }
            }
        )
    }

    // Dialog: Edit Scheme
    schemeToEdit?.let { scheme ->
        AddOrEditSchemeDialog(
            existingSchemes = allSchemes,
            initialScheme = scheme,
            onDismiss = { schemeToEdit = null },
            onConfirm = { name, usualAmountPaise ->
                viewModel.updatePigmyScheme(scheme.id, name, usualAmountPaise) {
                    schemeToEdit = null
                }
            }
        )
    }

    // Dialog: Set Opening Balance
    schemeForOpeningBalance?.let { scheme ->
        SetOpeningBalanceDialog(
            scheme = scheme,
            onDismiss = { schemeForOpeningBalance = null },
            onConfirm = { openingPaise, date ->
                viewModel.updatePigmySchemeOpeningBalance(scheme.id, openingPaise, date) {
                    schemeForOpeningBalance = null
                }
            }
        )
    }

    // Dialog: Record Deposit for Scheme
    schemeForDeposit?.let { scheme ->
        RecordPigmyDepositDialog(
            scheme = scheme,
            existingDeposits = allDeposits.filter { it.accountSchemeId == scheme.id },
            onDismiss = { schemeForDeposit = null },
            onConfirm = { amountPaise, source, notes, timestamp ->
                viewModel.recordPigmyDeposit(
                    schemeId = scheme.id,
                    amountPaise = amountPaise,
                    paymentSource = source,
                    notes = notes,
                    paidTimestamp = timestamp,
                    onDone = { schemeForDeposit = null }
                )
            }
        )
    }

    // Dialog: Record Withdrawal for Scheme
    schemeForWithdrawal?.let { scheme ->
        val schemeDeposits = allDeposits.filter { it.accountSchemeId == scheme.id && !it.isVoided }
        val schemeWithdrawals = allWithdrawals.filter { it.accountSchemeId == scheme.id && !it.isVoided }
        val currentTrackedBalance = scheme.openingBalancePaise + schemeDeposits.sumOf { it.amountPaise } -
schemeWithdrawals.sumOf { it.amountPaise }

        RecordPigmyWithdrawalDialog(
```

```
            scheme = scheme,
            currentTrackedBalancePaise = currentTrackedBalance,
            onDismiss = { schemeForWithdrawal = null },
            onConfirm = { amountPaise, destination, notes, timestamp ->
                viewModel.recordPigmyWithdrawal(
                    schemeId = scheme.id,
                    amountPaise = amountPaise,
                    destination = destination,
                    notes = notes,
                    withdrawalTimestamp = timestamp,
                    onDone = { schemeForWithdrawal = null },
                    onError = { /* handled in dialog */ }
                )
            }
        )
    }

    // Dialog: Edit Existing Deposit
    depositToEdit?.let { deposit ->
        val currentScheme = allSchemes.find { it.id == deposit.accountSchemeId }
        EditPigmyDepositDialog(
            deposit = deposit,
            allSchemes = allSchemes,
            currentScheme = currentScheme,
            onDismiss = { depositToEdit = null },
            onConfirm = { schemeId, amountPaise, source, notes, timestamp ->
                viewModel.updatePigmyDeposit(
                    depositId = deposit.id,
                    schemeId = schemeId,
                    amountPaise = amountPaise,
                    paymentSource = source,
                    notes = notes,
                    paidTimestamp = timestamp,
                    onDone = { depositToEdit = null }
                )
            }
        )
    }

    // Dialog: Edit Existing Withdrawal
    withdrawalToEdit?.let { withdrawal ->
        val currentScheme = allSchemes.find { it.id == withdrawal.accountSchemeId }
        val schemeDeposits = allDeposits.filter { it.accountSchemeId == withdrawal.accountSchemeId &&
!it.isVoided }
        val otherWithdrawals = allWithdrawals.filter { it.accountSchemeId == withdrawal.accountSchemeId &&
!it.isVoided && it.id != withdrawal.id }
        val maxAvailable = (currentScheme?.openingBalancePaise ?: 0L) + schemeDeposits.sumOf {
it.amountPaise } - otherWithdrawals.sumOf { it.amountPaise }

        EditPigmyWithdrawalDialog(
            withdrawal = withdrawal,
            currentScheme = currentScheme,
            maxAvailablePaise = maxAvailable,
            onDismiss = { withdrawalToEdit = null },
            onConfirm = { amountPaise, dest, notes, timestamp ->
                viewModel.updatePigmyWithdrawal(
                    withdrawalId = withdrawal.id,
                    schemeId = withdrawal.accountSchemeId,
                    amountPaise = amountPaise,
                    destination = dest,
                    notes = notes,
                    withdrawalTimestamp = timestamp,
                    onDone = { withdrawalToEdit = null }
                )
            }
        )
    }

    // Dialog: Void / Cancel Deposit
    depositToVoid?.let { deposit ->
        AlertDialog(
            onDismissRequest = { depositToVoid = null },
            title = { Text("Cancel Pigmy Deposit") },
            text = {
                Column(verticalArrangement = Arrangement.spacedBy(8.dp)) {
                    Text(
                        text = "Are you sure you want to cancel this deposit of
${CostCalculator.formatPaiseToRupees(deposit.amountPaise)}?",
                        style = MaterialTheme.typography.bodyMedium
                    )
                    if (deposit.paymentSource == "CASH" && deposit.linkedExpenseId != null) {
                        Text(
                            text = "• The linked Shop Cash Outflow (Savings Transfer) in Cash In/Out Tracker
 will be automatically reversed.",
                            style = MaterialTheme.typography.bodySmall,
                            color = MaterialTheme.colorScheme.primary
                        )
                    }
                    OutlinedTextField(
```

```
                        value = voidReason,
                        onValueChange = { voidReason = it },
                        label = { Text("Reason for cancellation *") },
                        placeholder = { Text("e.g. Accidental entry, Agent returned money") },
                        singleLine = true,
                        modifier = Modifier.fillMaxWidth()
                    )
                }
            },
            confirmButton = {
                Button(
                    onClick = {
                        if (voidReason.isNotBlank()) {
                            viewModel.voidPigmyDeposit(deposit.id, voidReason.trim()) {
                                depositToVoid = null
                                voidReason = ""
                            }
                        }
                    },
                    colors = ButtonDefaults.buttonColors(containerColor = MaterialTheme.colorScheme.error),
                    enabled = voidReason.isNotBlank()
                ) {
                    Text("Confirm Cancellation")
                }
            },
            dismissButton = {
                TextButton(onClick = { depositToVoid = null; voidReason = "" }) {
                    Text("Go Back")
                }
            }
        )
    }

    // Dialog: Void / Cancel Withdrawal
    withdrawalToVoid?.let { withdrawal ->
        AlertDialog(
            onDismissRequest = { withdrawalToVoid = null },
            title = { Text("Cancel Pigmy Withdrawal") },
            text = {
                Column(verticalArrangement = Arrangement.spacedBy(8.dp)) {
                    Text(
                        text = "Are you sure you want to cancel this withdrawal of
${CostCalculator.formatPaiseToRupees(withdrawal.amountPaise)}?",
                        style = MaterialTheme.typography.bodyMedium
                    )
                    if (withdrawal.destination == "CASH" && withdrawal.linkedExpenseId != null) {
                        Text(
                            text = "• The linked Shop Cash Inflow (Savings Transfer) in Cash In/Out Tracker
will be automatically reversed.",
                            style = MaterialTheme.typography.bodySmall,
                            color = MaterialTheme.colorScheme.primary
                        )
                    }
                    OutlinedTextField(
                        value = voidReason,
                        onValueChange = { voidReason = it },
                        label = { Text("Reason for cancellation *") },
                        placeholder = { Text("e.g. Returned to scheme, Error correction") },
                        singleLine = true,
                        modifier = Modifier.fillMaxWidth()
                    )
                }
            },
            confirmButton = {
                Button(
                    onClick = {
                        if (voidReason.isNotBlank()) {
                            viewModel.voidPigmyWithdrawal(withdrawal.id, voidReason.trim()) {
                                withdrawalToVoid = null
                                voidReason = ""
                            }
                        }
                    },
                    colors = ButtonDefaults.buttonColors(containerColor = MaterialTheme.colorScheme.error),
                    enabled = voidReason.isNotBlank()
                ) {
                    Text("Confirm Cancellation")
                }
            },
            dismissButton = {
                TextButton(onClick = { withdrawalToVoid = null; voidReason = "" }) {
                    Text("Go Back")
                }
            }
        )
    }
}

```

```
sealed class PigmyActivityItem {
    abstract val timestamp: Long
    abstract val isVoided: Boolean

    data class Deposit(val deposit: PigmyDepositEntity) : PigmyActivityItem() {
        override val timestamp: Long get() = deposit.paidTimestamp
        override val isVoided: Boolean get() = deposit.isVoided
    }

    data class Withdrawal(val withdrawal: PigmyWithdrawalEntity) : PigmyActivityItem() {
        override val timestamp: Long get() = withdrawal.withdrawalTimestamp
        override val isVoided: Boolean get() = withdrawal.isVoided
    }
}

@Composable
fun PigmySchemeCard(
    scheme: PigmySchemeEntity,
    deposits: List<PigmyDepositEntity>,
    withdrawals: List<PigmyWithdrawalEntity>,
    onAddDeposit: () -> Unit,
    onAddWithdrawal: () -> Unit,
    onSetOpeningBalance: () -> Unit,
    onEditScheme: () -> Unit,
    onArchiveToggle: (Boolean) -> Unit,
    onEditDeposit: (PigmyDepositEntity) -> Unit,
    onVoidDeposit: (PigmyDepositEntity) -> Unit,
    onEditWithdrawal: (PigmyWithdrawalEntity) -> Unit,
    onVoidWithdrawal: (PigmyWithdrawalEntity) -> Unit,
    modifier: Modifier = Modifier
) {
    var expanded by remember { mutableStateOf(false) }
    var historyFilterTab by remember { mutableIntStateOf(0) } // 0: All, 1: Deposits, 2: Withdrawals
    var showOverflowMenu by remember { mutableStateOf(false) }

    val activeDepositsTotal = remember(deposits) {
        deposits.filter { !it.isVoided }.sumOf { it.amountPaise }
    }
    val activeWithdrawalsTotal = remember(withdrawals) {
        withdrawals.filter { !it.isVoided }.sumOf { it.amountPaise }
    }
    val trackedBalancePaise = scheme.openingBalancePaise + activeDepositsTotal - activeWithdrawalsTotal

    val allActivities = remember(deposits, withdrawals) {
        val list = mutableListOf<PigmyActivityItem>()
        deposits.forEach { list.add(PigmyActivityItem.Deposit(it)) }
        withdrawals.forEach { list.add(PigmyActivityItem.Withdrawal(it)) }
        list.sortedByDescending { it.timestamp }
    }

    val filteredActivities = remember(allActivities, historyFilterTab) {
        when (historyFilterTab) {
            1 -> allActivities.filterIsInstance<PigmyActivityItem.Deposit>()
            2 -> allActivities.filterIsInstance<PigmyActivityItem.Withdrawal>()
            else -> allActivities
        }
    }

    Card(
        modifier = modifier.fillMaxWidth(),
        shape = RoundedCornerShape(14.dp),
        colors = CardDefaults.cardColors(
            containerColor = if (scheme.isArchived) MaterialTheme.colorScheme.surfaceVariant.copy(alpha =
0.5f) else MaterialTheme.colorScheme.surface
        )
    ) {
        Column(modifier = Modifier.padding(16.dp)) {
            // Header Row: Scheme Name + Status + Overflow Menu Button
            Row(
                modifier = Modifier.fillMaxWidth(),
                verticalAlignment = Alignment.CenterVertically,
                horizontalArrangement = Arrangement.SpaceBetween
            ) {
                Row(
                    modifier = Modifier.weight(1f, fill = false),
                    verticalAlignment = Alignment.CenterVertically
                ) {
                    Text(
                        text = scheme.name,
                        style = MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.Bold),
                        color = MaterialTheme.colorScheme.onSurface
                    )
                    if (scheme.isArchived) {
                        Spacer(modifier = Modifier.width(8.dp))
                        Card(
                            shape = RoundedCornerShape(4.dp),
                            colors = CardDefaults.cardColors(containerColor =
MaterialTheme.colorScheme.errorContainer)
```

```
                        ) {
                            Text(
                                text = "Archived",
                                modifier = Modifier.padding(horizontal = 6.dp, vertical = 2.dp),
                                style = MaterialTheme.typography.labelSmall,
                                color = MaterialTheme.colorScheme.onErrorContainer
                            )
                        }
                    }
                }

                Box {
                    IconButton(
                        onClick = { showOverflowMenu = true },
                        modifier = Modifier.size(36.dp)
                    ) {
                        Icon(
                            Icons.Filled.MoreVert,
                            contentDescription = "Scheme Options",
                            tint = MaterialTheme.colorScheme.onSurfaceVariant
                        )
                    }
                    DropdownMenu(
                        expanded = showOverflowMenu,
                        onDismissRequest = { showOverflowMenu = false }
                    ) {
                        DropdownMenuItem(
                            text = { Text("Opening Balance") },
                            leadingIcon = {
                                Icon(Icons.Filled.AccountBalance, contentDescription = null, modifier =
    Modifier.size(18.dp), tint = MaterialTheme.colorScheme.primary)
                            },
                            onClick = {
                                showOverflowMenu = false
                                onSetOpeningBalance()
                            }
                        )
                        DropdownMenuItem(
                            text = { Text("Edit Scheme") },
                            leadingIcon = {
                                Icon(Icons.Filled.Edit, contentDescription = null, modifier =
    Modifier.size(18.dp))
                            },
                            onClick = {
                                showOverflowMenu = false
                                onEditScheme()
                            }
                        )
                        DropdownMenuItem(
                            text = { Text(if (scheme.isArchived) "Reactivate Scheme" else "Archive Scheme")
    },
                            leadingIcon = {
                                Icon(
                                    if (scheme.isArchived) Icons.Filled.Unarchive else Icons.Filled.Archive,
                                    contentDescription = null,
                                    modifier = Modifier.size(18.dp)
                                )
                            },
                            onClick = {
                                showOverflowMenu = false
                                onArchiveToggle(!scheme.isArchived)
                            }
                        )
                    }
                }
            }

            Spacer(modifier = Modifier.height(6.dp))

            // Tracked Balance & Details Row
            Row(
                modifier = Modifier.fillMaxWidth(),
                horizontalArrangement = Arrangement.SpaceBetween,
                verticalAlignment = Alignment.Bottom
            ) {
                Column {
                    Text(
                        text = "Tracked Balance",
                        style = MaterialTheme.typography.bodySmall,
                        color = MaterialTheme.colorScheme.onSurfaceVariant
                    )
                    Text(
                        text = CostCalculator.formatPaiseToRupees(trackedBalancePaise),
                        style = MaterialTheme.typography.headlineSmall.copy(fontWeight = FontWeight.Bold),
                        color = if (trackedBalancePaise >= 0) MaterialTheme.colorScheme.primary else
    MaterialTheme.colorScheme.error
                    )
                    if (scheme.openingBalancePaise > 0) {
```

```
                        Text(
                            text = "Opening: ${CostCalculator.formatPaiseToRupees(scheme.openingBalancePaise)}",
                            style = MaterialTheme.typography.labelSmall,
                            color = MaterialTheme.colorScheme.onSurfaceVariant
                        )
                    }
                }

                Column(horizontalAlignment = Alignment.End) {
                    Text(
                        text = "Deposits: ${CostCalculator.formatPaiseToRupees(activeDepositsTotal)}",
                        style = MaterialTheme.typography.bodySmall.copy(fontWeight = FontWeight.SemiBold),
                        color = Color(0xFF2E7D32)
                    )
                    Text(
                        text = "Withdrawals: ${CostCalculator.formatPaiseToRupees(activeWithdrawalsTotal)}",
                        style = MaterialTheme.typography.bodySmall.copy(fontWeight = FontWeight.SemiBold),
                        color = Color(0xFFD97706)
                    )
                    Text(
                        text = "Usual daily: ${CostCalculator.formatPaiseToRupees(scheme.usualDailyAmountPaise)}",
                        style = MaterialTheme.typography.labelSmall,
                        color = MaterialTheme.colorScheme.onSurfaceVariant
                    )
                }
            }

            Spacer(modifier = Modifier.height(14.dp))

            // Two Action Buttons: Record Deposit & Record Withdrawal (equal weight, min height 48dp, single line)
            Row(
                modifier = Modifier.fillMaxWidth(),
                horizontalArrangement = Arrangement.spacedBy(10.dp),
                verticalAlignment = Alignment.CenterVertically
            ) {
                Button(
                    onClick = onAddDeposit,
                    enabled = !scheme.isArchived,
                    modifier = Modifier
                        .weight(1f)
                        .height(48.dp),
                    shape = RoundedCornerShape(10.dp),
                    colors = ButtonDefaults.buttonColors(containerColor = Color(0xFF2E7D32))
                ) {
                    Icon(Icons.Filled.Add, contentDescription = null, modifier = Modifier.size(18.dp))
                    Spacer(modifier = Modifier.width(6.dp))
                    Text(
                        text = "Deposit",
                        fontWeight = FontWeight.Bold,
                        fontSize = 14.sp,
                        maxLines = 1
                    )
                }

                Button(
                    onClick = onAddWithdrawal,
                    enabled = !scheme.isArchived && trackedBalancePaise > 0,
                    modifier = Modifier
                        .weight(1f)
                        .height(48.dp),
                    shape = RoundedCornerShape(10.dp),
                    colors = ButtonDefaults.buttonColors(containerColor = Color(0xFFD97706))
                ) {
                    Icon(Icons.Filled.Remove, contentDescription = null, modifier = Modifier.size(18.dp))
                    Spacer(modifier = Modifier.width(6.dp))
                    Text(
                        text = "Withdraw",
                        fontWeight = FontWeight.Bold,
                        fontSize = 14.sp,
                        maxLines = 1
                    )
                }
            }

            Spacer(modifier = Modifier.height(10.dp))

            // Dedicated History Row
            OutlinedButton(
                onClick = { expanded = !expanded },
                modifier = Modifier
                    .fillMaxWidth()
                    .height(44.dp),
                shape = RoundedCornerShape(10.dp)
            ) {
                Row(
```

```
                    modifier = Modifier.fillMaxWidth(),
                    horizontalArrangement = Arrangement.SpaceBetween,
                    verticalAlignment = Alignment.CenterVertically
                ) {
                    Text(
                        text = if (expanded) "Hide history (${allActivities.size} entries)" else "View history (${allActivities.size} entries)",
                        style = MaterialTheme.typography.bodyMedium.copy(fontWeight = FontWeight.SemiBold)
                    )
                    Icon(
                        if (expanded) Icons.Filled.ExpandLess else Icons.Filled.ExpandMore,
                        contentDescription = null,
                        modifier = Modifier.size(20.dp)
                    )
                }
            }

            // Expandable Activity History
            AnimatedVisibility(visible = expanded) {
                Column(modifier = Modifier.padding(top = 12.dp)) {
                    // History Filter Tabs
                    TabRow(
                        selectedTabIndex = historyFilterTab,
                        containerColor = MaterialTheme.colorScheme.surfaceVariant.copy(alpha = 0.4f),
                        contentColor = MaterialTheme.colorScheme.primary
                    ) {
                        Tab(
                            selected = historyFilterTab == 0,
                            onClick = { historyFilterTab = 0 },
                            text = { Text("All (${allActivities.size})", fontSize = 11.sp) }
                        )
                        Tab(
                            selected = historyFilterTab == 1,
                            onClick = { historyFilterTab = 1 },
                            text = { Text("Deposits (${deposits.size})", fontSize = 11.sp) }
                        )
                        Tab(
                            selected = historyFilterTab == 2,
                            onClick = { historyFilterTab = 2 },
                            text = { Text("Withdrawals (${withdrawals.size})", fontSize = 11.sp) }
                        )
                    }

                    Spacer(modifier = Modifier.height(10.dp))

                    if (filteredActivities.isEmpty()) {
                        Text(
                            text = "No records for this scheme in the selected filter.",
                            style = MaterialTheme.typography.bodySmall,
                            color = MaterialTheme.colorScheme.onSurfaceVariant,
                            modifier = Modifier.padding(vertical = 8.dp)
                        )
                    } else {
                        filteredActivities.forEach { item ->
                            when (item) {
                                is PigmyActivityItem.Deposit -> {
                                    DepositRow(
                                        deposit = item.deposit,
                                        onEdit = { onEditDeposit(item.deposit) },
                                        onVoid = { onVoidDeposit(item.deposit) }
                                    )
                                }
                                is PigmyActivityItem.Withdrawal -> {
                                    WithdrawalRow(
                                        withdrawal = item.withdrawal,
                                        onEdit = { onEditWithdrawal(item.withdrawal) },
                                        onVoid = { onVoidWithdrawal(item.withdrawal) }
                                    )
                                }
                            }
                            Spacer(modifier = Modifier.height(8.dp))
                        }
                    }
                }
            }
        }
    }
}

@Composable
fun DepositRow(
    deposit: PigmyDepositEntity,
    onEdit: () -> Unit,
    onVoid: () -> Unit,
    modifier: Modifier = Modifier
) {
    val dateFormat = SimpleDateFormat("dd MMM yyyy, hh:mm a", Locale.getDefault())
    val dateStr = dateFormat.format(Date(deposit.paidTimestamp))
```

```
938
939     Card(
940         modifier = modifier.fillMaxWidth(),
941         shape = RoundedCornerShape(8.dp),
942         colors = CardDefaults.cardColors(
943             containerColor = if (deposit.isVoided) MaterialTheme.colorScheme.surfaceVariant.copy(alpha =
    0.3f) else MaterialTheme.colorScheme.surfaceVariant.copy(alpha = 0.6f)
944         )
945     ) {
946         Row(
947             modifier = Modifier
948                 .fillMaxWidth()
949                 .padding(10.dp),
950             horizontalArrangement = Arrangement.SpaceBetween,
951             verticalAlignment = Alignment.CenterVertically
952         ) {
953             Column(modifier = Modifier.weight(1f)) {
954                 Row(verticalAlignment = Alignment.CenterVertically) {
955                     Text(
956                         text = "+ ${CostCalculator.formatPaiseToRupees(deposit.amountPaise)}",
957                         fontWeight = FontWeight.Bold,
958                         fontSize = 15.sp,
959                         color = if (deposit.isVoided) MaterialTheme.colorScheme.onSurfaceVariant else
    Color(0xFF2E7D32),
960                         textDecoration = if (deposit.isVoided) TextDecoration.LineThrough else
    TextDecoration.None
961                     )
962                     Spacer(modifier = Modifier.width(8.dp))
963                     Card(
964                         shape = RoundedCornerShape(4.dp),
965                         colors = CardDefaults.cardColors(
966                             containerColor = if (deposit.paymentSource == "CASH") Color(0xFFE8F5E9) else
    Color(0xFFE3F2FD)
967                         )
968                     ) {
969                         Text(
970                             text = if (deposit.paymentSource == "CASH") "Deposit (Cash Outflow)" else
    "Deposit (Direct / Bank)",
971                             modifier = Modifier.padding(horizontal = 6.dp, vertical = 2.dp),
972                             style = MaterialTheme.typography.labelSmall,
973                             color = if (deposit.paymentSource == "CASH") Color(0xFF2E7D32) else
    Color(0xFF1565C0),
974                             fontWeight = FontWeight.SemiBold
975                         )
976                     }
977                 }
978
979                 Text(
980                     text = dateStr,
981                     style = MaterialTheme.typography.bodySmall,
982                     color = MaterialTheme.colorScheme.onSurfaceVariant
983                 )
984
985                 if (!deposit.notes.isNullOrBlank()) {
986                     Text(
987                         text = "Note: ${deposit.notes}",
988                         style = MaterialTheme.typography.bodySmall,
989                         color = MaterialTheme.colorScheme.onSurfaceVariant
990                     )
991                 }
992
993                 if (deposit.isVoided) {
994                     Text(
995                         text = "CANCELLED: ${deposit.voidReason ?: ""}",
996                         style = MaterialTheme.typography.bodySmall,
997                         color = MaterialTheme.colorScheme.error,
998                         fontWeight = FontWeight.SemiBold
999                     )
1000                 }
1001             }
1002
1003             if (!deposit.isVoided) {
1004                 Row(verticalAlignment = Alignment.CenterVertically) {
1005                     IconButton(onClick = onEdit, modifier = Modifier.size(28.dp)) {
1006                         Icon(Icons.Filled.Edit, contentDescription = "Edit", modifier =
    Modifier.size(16.dp))
1007                     }
1008                     IconButton(onClick = onVoid, modifier = Modifier.size(28.dp)) {
1009                         Icon(Icons.Filled.Block, contentDescription = "Cancel", modifier =
    Modifier.size(16.dp), tint = MaterialTheme.colorScheme.error)
1010                     }
1011                 }
1012             }
1013         }
1014     }
1015 }
1016
1017 @Composable
1018 fun WithdrawalRow(
```

```
    withdrawal: PigmyWithdrawalEntity,
    onEdit: () -> Unit,
    onVoid: () -> Unit,
    modifier: Modifier = Modifier
) {
    val dateFormat = SimpleDateFormat("dd MMM yyyy, hh:mm a", Locale.getDefault())
    val dateStr = dateFormat.format(Date(withdrawal.withdrawalTimestamp))

    Card(
        modifier = modifier.fillMaxWidth(),
        shape = RoundedCornerShape(8.dp),
        colors = CardDefaults.cardColors(
            containerColor = if (withdrawal.isVoided) MaterialTheme.colorScheme.surfaceVariant.copy(alpha = 0.3f) else MaterialTheme.colorScheme.surfaceVariant.copy(alpha = 0.6f)
        )
    ) {
        Row(
            modifier = Modifier
                .fillMaxWidth()
                .padding(10.dp),
            horizontalArrangement = Arrangement.SpaceBetween,
            verticalAlignment = Alignment.CenterVertically
        ) {
            Column(modifier = Modifier.weight(1f)) {
                Row(verticalAlignment = Alignment.CenterVertically) {
                    Text(
                        text = "- ${CostCalculator.formatPaiseToRupees(withdrawal.amountPaise)}",
                        fontWeight = FontWeight.Bold,
                        fontSize = 15.sp,
                        color = if (withdrawal.isVoided) MaterialTheme.colorScheme.onSurfaceVariant else Color(0xFFD97706),
                        textDecoration = if (withdrawal.isVoided) TextDecoration.LineThrough else TextDecoration.None
                    )
                    Spacer(modifier = Modifier.width(8.dp))
                    Card(
                        shape = RoundedCornerShape(4.dp),
                        colors = CardDefaults.cardColors(
                            containerColor = if (withdrawal.destination == "CASH") Color(0xFFFFF3E0) else Color(0xFFE3F2FD)
                        )
                    ) {
                        Text(
                            text = if (withdrawal.destination == "CASH") "Withdrawal (Shop Cash In)" else "Withdrawal (Bank Account)",
                            modifier = Modifier.padding(horizontal = 6.dp, vertical = 2.dp),
                            style = MaterialTheme.typography.labelSmall,
                            color = if (withdrawal.destination == "CASH") Color(0xFFD97706) else Color(0xFF1565C0),
                            fontWeight = FontWeight.SemiBold
                        )
                    }
                }

                Text(
                    text = dateStr,
                    style = MaterialTheme.typography.bodySmall,
                    color = MaterialTheme.colorScheme.onSurfaceVariant
                )

                if (!withdrawal.notes.isNullOrBlank()) {
                    Text(
                        text = "Note: ${withdrawal.notes}",
                        style = MaterialTheme.typography.bodySmall,
                        color = MaterialTheme.colorScheme.onSurfaceVariant
                    )
                }

                if (withdrawal.isVoided) {
                    Text(
                        text = "CANCELLED: ${withdrawal.voidReason ?: ""}",
                        style = MaterialTheme.typography.bodySmall,
                        color = MaterialTheme.colorScheme.error,
                        fontWeight = FontWeight.SemiBold
                    )
                }
            }

            if (!withdrawal.isVoided) {
                Row(verticalAlignment = Alignment.CenterVertically) {
                    IconButton(onClick = onEdit, modifier = Modifier.size(28.dp)) {
                        Icon(Icons.Filled.Edit, contentDescription = "Edit", modifier = Modifier.size(16.dp))
                    }
                    IconButton(onClick = onVoid, modifier = Modifier.size(28.dp)) {
                        Icon(Icons.Filled.Block, contentDescription = "Cancel", modifier = Modifier.size(16.dp), tint = MaterialTheme.colorScheme.error)
                    }
```

```
                }
            }
        }
    }
}

@Composable
fun AddOrEditSchemeDialog(
    existingSchemes: List<PigmySchemeEntity>,
    initialScheme: PigmySchemeEntity?,
    onDismiss: () -> Unit,
    onConfirm: (name: String, usualAmountPaise: Long) -> Unit
) {
    var name by remember { mutableStateOf(initialScheme?.name ?: "") }
    var amountStr by remember { mutableStateOf(if (initialScheme != null)
(initialScheme.usualDailyAmountPaise / 100).toString() else "100") }

    val amountPaise = CostCalculator.parseRupeesToPaise(amountStr) ?: 0L
    val isDuplicateName = remember(name, initialScheme) {
        existingSchemes.any { it.name.equals(name.trim(), ignoreCase = true) && it.id != initialScheme?.id }
    }

    val isValid = name.isNotBlank() && amountPaise > 0L

    AlertDialog(
        onDismissRequest = onDismiss,
        title = { Text(if (initialScheme == null) "Add Pigmy Scheme" else "Edit Pigmy Scheme") },
        text = {
            Column(verticalArrangement = Arrangement.spacedBy(10.dp)) {
                OutlinedTextField(
                    value = name,
                    onValueChange = { name = it },
                    label = { Text("Scheme Name *") },
                    placeholder = { Text("e.g. Bank Scheme A, Daily Deposit 1") },
                    singleLine = true,
                    modifier = Modifier.fillMaxWidth()
                )

                if (isDuplicateName) {
                    Text(
                        text = "Warning: A scheme with this name already exists.",
                        style = MaterialTheme.typography.bodySmall,
                        color = MaterialTheme.colorScheme.error
                    )
                }

                OutlinedTextField(
                    value = amountStr,
                    onValueChange = { amountStr = it },
                    label = { Text("Usual Daily Deposit (₹) *") },
                    keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Decimal),
                    singleLine = true,
                    modifier = Modifier.fillMaxWidth()
                )
            }
        },
        confirmButton = {
            Button(
                onClick = {
                    if (isValid) {
                        onConfirm(name.trim(), amountPaise)
                    }
                },
                enabled = isValid
            ) {
                Text(if (initialScheme == null) "Create Scheme" else "Save Changes")
            }
        },
        dismissButton = {
            TextButton(onClick = onDismiss) {
                Text("Cancel")
            }
        }
    )
}

@Composable
fun SetOpeningBalanceDialog(
    scheme: PigmySchemeEntity,
    onDismiss: () -> Unit,
    onConfirm: (openingPaise: Long, date: Long?) -> Unit
) {
    var amountStr by remember { mutableStateOf(if (scheme.openingBalancePaise > 0)
(scheme.openingBalancePaise / 100).toString() else "0") }
    val amountPaise = CostCalculator.parseRupeesToPaise(amountStr) ?: 0L

    AlertDialog(
        onDismissRequest = onDismiss,
```

```
        title = { Text("Set Starting Savings: ${scheme.name}") },
        text = {
            Column(verticalArrangement = Arrangement.spacedBy(10.dp)) {
                Card(
                    colors = CardDefaults.cardColors(containerColor =
MaterialTheme.colorScheme.primaryContainer.copy(alpha = 0.6f)),
                    shape = RoundedCornerShape(8.dp)
                ) {
                    Text(
                        text = "• Confirmed savings accumulated with bank/agent before tracking in this
app.\n• This establishes starting savings balance only and will NOT create any cash transaction or count as
new deposit income.",
                        modifier = Modifier.padding(10.dp),
                        style = MaterialTheme.typography.bodySmall,
                        color = MaterialTheme.colorScheme.onPrimaryContainer
                    )
                }

                OutlinedTextField(
                    value = amountStr,
                    onValueChange = { amountStr = it },
                    label = { Text("Confirmed Starting Savings (₹) *") },
                    placeholder = { Text("0.00") },
                    keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Decimal),
                    singleLine = true,
                    modifier = Modifier.fillMaxWidth()
                )
            }
        },
        confirmButton = {
            Button(
                onClick = {
                    onConfirm(maxOf(0L, amountPaise), System.currentTimeMillis())
                }
            ) {
                Text("Save Opening Balance")
            }
        },
        dismissButton = {
            TextButton(onClick = onDismiss) {
                Text("Cancel")
            }
        }
    )
}

@Composable
fun RecordPigmyDepositDialog(
    scheme: PigmySchemeEntity,
    existingDeposits: List<PigmyDepositEntity>,
    onDismiss: () -> Unit,
    onConfirm: (amountPaise: Long, source: String, notes: String?, timestamp: Long) -> Unit
) {
    var amountStr by remember { mutableStateOf((scheme.usualDailyAmountPaise / 100).toString()) }
    var paymentSource by remember { mutableStateOf("CASH") }
    var notes by remember { mutableStateOf("") }
    var entryTimestamp by remember { mutableLongStateOf(System.currentTimeMillis()) }
    val context = LocalContext.current
    val cal = Calendar.getInstance().apply { timeInMillis = entryTimestamp }
    val dateStr = SimpleDateFormat("dd MMM yyyy, hh:mm a", Locale.getDefault()).format(Date(entryTimestamp))

    val amountPaise = CostCalculator.parseRupeesToPaise(amountStr) ?: 0L

    AlertDialog(
        onDismissRequest = onDismiss,
        title = {
            Row(verticalAlignment = Alignment.CenterVertically) {
                Icon(
                    painter = painterResource(id = R.drawable.ic_pigmy_coin),
                    contentDescription = null,
                    tint = Color(0xFFD97706),
                    modifier = Modifier.size(22.dp)
                )
                Spacer(modifier = Modifier.width(8.dp))
                Text("Deposit: ${scheme.name}")
            }
        },
        text = {
            Column(verticalArrangement = Arrangement.spacedBy(10.dp)) {
                OutlinedTextField(
                    value = amountStr,
                    onValueChange = { amountStr = it },
                    label = { Text("Deposit Amount (₹) *") },
                    keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Decimal),
                    singleLine = true,
                    modifier = Modifier.fillMaxWidth()
                )

```

```
                Row(
                    modifier = Modifier.fillMaxWidth(),
                    horizontalArrangement = Arrangement.SpaceBetween,
                    verticalAlignment = Alignment.CenterVertically
                ) {
                    Text("Date & Time:", style = MaterialTheme.typography.bodySmall, fontWeight =
FontWeight.SemiBold)
                    OutlinedButton(
                        onClick = {
                            DatePickerDialog(context, { _, y, m, d ->
                                val c = Calendar.getInstance().apply {
                                    timeInMillis = entryTimestamp
                                    set(Calendar.YEAR, y)
                                    set(Calendar.MONTH, m)
                                    set(Calendar.DAY_OF_MONTH, d)
                                }
                                TimePickerDialog(context, { _, h, min ->
                                    c.set(Calendar.HOUR_OF_DAY, h)
                                    c.set(Calendar.MINUTE, min)
                                    entryTimestamp = c.timeInMillis
                                }, cal.get(Calendar.HOUR_OF_DAY), cal.get(Calendar.MINUTE), false).show()
                            }, cal.get(Calendar.YEAR), cal.get(Calendar.MONTH),
cal.get(Calendar.DAY_OF_MONTH)).show()
                        },
                        shape = RoundedCornerShape(8.dp)
                    ) {
                        Icon(Icons.Filled.CalendarToday, contentDescription = null, modifier =
Modifier.size(14.dp))
                        Spacer(modifier = Modifier.width(4.dp))
                        Text(dateStr, fontSize = 11.5.sp)
                    }
                }

                Text("Payment Funding Source:", style = MaterialTheme.typography.bodySmall, fontWeight =
FontWeight.Bold)
                Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
                    FilterChip(
                        selected = paymentSource == "CASH",
                        onClick = { paymentSource = "CASH" },
                        label = { Text("Shop Cash (Outflow)") }
                    )
                    FilterChip(
                        selected = paymentSource == "BANK",
                        onClick = { paymentSource = "BANK" },
                        label = { Text("Direct / Bank") }
                    )
                }

                if (paymentSource == "CASH") {
                    Card(
                        colors = CardDefaults.cardColors(containerColor =
MaterialTheme.colorScheme.primaryContainer),
                        shape = RoundedCornerShape(8.dp)
                    ) {
                        Text(
                            text = "• Shop cash outflow will be automatically linked in Cash In/Out Tracker
as a Savings Transfer (does not reduce shop profit).",
                            modifier = Modifier.padding(10.dp),
                            style = MaterialTheme.typography.bodySmall,
                            color = MaterialTheme.colorScheme.onPrimaryContainer
                        )
                    }
                } else {
                    Card(
                        colors = CardDefaults.cardColors(containerColor =
MaterialTheme.colorScheme.surfaceVariant),
                        shape = RoundedCornerShape(8.dp)
                    ) {
                        Text(
                            text = "• Direct/Bank payment does not reduce physical shop cash.",
                            modifier = Modifier.padding(10.dp),
                            style = MaterialTheme.typography.bodySmall,
                            color = MaterialTheme.colorScheme.onSurfaceVariant
                        )
                    }
                }

                OutlinedTextField(
                    value = notes,
                    onValueChange = { notes = it },
                    label = { Text("Reference / Note (Optional)") },
                    placeholder = { Text("e.g. Receipt #, Agent name") },
                    singleLine = true,
                    modifier = Modifier.fillMaxWidth()
                )
            }
        },
        confirmButton = {
```

```
            Button(
                onClick = {
                    if (amountPaise > 0) {
                        onConfirm(amountPaise, paymentSource, notes.trim().ifBlank { null }, entryTimestamp)
                    }
                },
                enabled = amountPaise > 0
            ) {
                Text("Confirm Deposit")
            }
        },
        dismissButton = {
            TextButton(onClick = onDismiss) {
                Text("Cancel")
            }
        }
    )
}

@Composable
fun RecordPigmyWithdrawalDialog(
    scheme: PigmySchemeEntity,
    currentTrackedBalancePaise: Long,
    onDismiss: () -> Unit,
    onConfirm: (amountPaise: Long, destination: String, notes: String?, timestamp: Long) -> Unit
) {
    var amountStr by remember { mutableStateOf("") }
    var destination by remember { mutableStateOf("CASH") } // CASH, BANK
    var notes by remember { mutableStateOf("") }
    var entryTimestamp by remember { mutableLongStateOf(System.currentTimeMillis()) }
    val context = LocalContext.current
    val cal = Calendar.getInstance().apply { timeInMillis = entryTimestamp }
    val dateStr = SimpleDateFormat("dd MMM yyyy, hh:mm a", Locale.getDefault()).format(Date(entryTimestamp))

    val amountPaise = CostCalculator.parseRupeesToPaise(amountStr) ?: 0L
    val exceedsBalance = amountPaise > currentTrackedBalancePaise

    AlertDialog(
        onDismissRequest = onDismiss,
        title = {
            Row(verticalAlignment = Alignment.CenterVertically) {
                Icon(
                    painter = painterResource(id = R.drawable.ic_pigmy_coin),
                    contentDescription = null,
                    tint = Color(0xFFD97706),
                    modifier = Modifier.size(22.dp)
                )
                Spacer(modifier = Modifier.width(8.dp))
                Text("Withdraw: ${scheme.name}")
            }
        },
        text = {
            Column(verticalArrangement = Arrangement.spacedBy(10.dp)) {
                Text(
                    text = "Available Tracked Savings:
${CostCalculator.formatPaiseToRupees(currentTrackedBalancePaise)}",
                    style = MaterialTheme.typography.bodySmall.copy(fontWeight = FontWeight.Bold),
                    color = MaterialTheme.colorScheme.primary
                )

                OutlinedTextField(
                    value = amountStr,
                    onValueChange = { amountStr = it },
                    label = { Text("Withdrawal Amount (₹) *") },
                    placeholder = { Text("0.00") },
                    keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Decimal),
                    singleLine = true,
                    isError = exceedsBalance,
                    modifier = Modifier.fillMaxWidth()
                )

                if (exceedsBalance) {
                    Text(
                        text = "   Withdrawal (${CostCalculator.formatPaiseToRupees(amountPaise)}) exceeds
tracked savings (${CostCalculator.formatPaiseToRupees(currentTrackedBalancePaise)}). Please review history
or set confirmed opening balance.",
                        style = MaterialTheme.typography.bodySmall,
                        color = MaterialTheme.colorScheme.error,
                        fontWeight = FontWeight.SemiBold
                    )
                }

                Row(
                    modifier = Modifier.fillMaxWidth(),
                    horizontalArrangement = Arrangement.SpaceBetween,
                    verticalAlignment = Alignment.CenterVertically
                ) {
                    Text("Date & Time:", style = MaterialTheme.typography.bodySmall, fontWeight =
```

```
FontWeight.SemiBold)
                    OutlinedButton(
                        onClick = {
                            DatePickerDialog(context, { _, y, m, d ->
                                val c = Calendar.getInstance().apply {
                                    timeInMillis = entryTimestamp
                                    set(Calendar.YEAR, y)
                                    set(Calendar.MONTH, m)
                                    set(Calendar.DAY_OF_MONTH, d)
                                }
                                TimePickerDialog(context, { _, h, min ->
                                    c.set(Calendar.HOUR_OF_DAY, h)
                                    c.set(Calendar.MINUTE, min)
                                    entryTimestamp = c.timeInMillis
                                }, cal.get(Calendar.HOUR_OF_DAY), cal.get(Calendar.MINUTE), false).show()
                            }, cal.get(Calendar.YEAR), cal.get(Calendar.MONTH),
cal.get(Calendar.DAY_OF_MONTH)).show()
                        },
                        shape = RoundedCornerShape(8.dp)
                    ) {
                        Icon(Icons.Filled.CalendarToday, contentDescription = null, modifier =
Modifier.size(14.dp))
                        Spacer(modifier = Modifier.width(4.dp))
                        Text(dateStr, fontSize = 11.5.sp)
                    }
                }

                Text("Receipt Destination:", style = MaterialTheme.typography.bodySmall, fontWeight =
FontWeight.Bold)
                Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
                    FilterChip(
                        selected = destination == "CASH",
                        onClick = { destination = "CASH" },
                        label = { Text("Shop Cash (Inflow)") }
                    )
                    FilterChip(
                        selected = destination == "BANK",
                        onClick = { destination = "BANK" },
                        label = { Text("Bank Account") }
                    )
                }

                if (destination == "CASH") {
                    Card(
                        colors = CardDefaults.cardColors(containerColor = Color(0xFFFFF3E0)),
                        shape = RoundedCornerShape(8.dp)
                    ) {
                        Text(
                            text = "• Received into shop cash: automatically creates exactly one linked Cash
 In entry as returned savings (NOT sales revenue, NOT profit).",
                            modifier = Modifier.padding(10.dp),
                            style = MaterialTheme.typography.bodySmall,
                            color = Color(0xFFE65100)
                        )
                    }
                } else {
                    Card(
                        colors = CardDefaults.cardColors(containerColor =
MaterialTheme.colorScheme.surfaceVariant),
                        shape = RoundedCornerShape(8.dp)
                    ) {
                        Text(
                            text = "• Received into bank account: records savings reduction, does NOT
increase physical shop cash.",
                            modifier = Modifier.padding(10.dp),
                            style = MaterialTheme.typography.bodySmall,
                            color = MaterialTheme.colorScheme.onSurfaceVariant
                        )
                    }
                }

                OutlinedTextField(
                    value = notes,
                    onValueChange = { notes = it },
                    label = { Text("Reference / Note (Optional)") },
                    placeholder = { Text("e.g. Returned savings, Cheque #") },
                    singleLine = true,
                    modifier = Modifier.fillMaxWidth()
                )
            }
        },
        confirmButton = {
            Button(
                onClick = {
                    if (amountPaise > 0 && !exceedsBalance) {
                        onConfirm(amountPaise, destination, notes.trim().ifBlank { null }, entryTimestamp)
                    }
                },
                enabled = amountPaise > 0 && !exceedsBalance,
```

```
                colors = ButtonDefaults.buttonColors(containerColor = Color(0xFFD97706))
            ) {
                Text("Confirm Withdrawal")
            }
        },
        dismissButton = {
            TextButton(onClick = onDismiss) {
                Text("Cancel")
            }
        }
    )
}

@Composable
fun EditPigmyDepositDialog(
    deposit: PigmyDepositEntity,
    allSchemes: List<PigmySchemeEntity>,
    currentScheme: PigmySchemeEntity?,
    onDismiss: () -> Unit,
    onConfirm: (schemeId: String, amountPaise: Long, source: String, notes: String?, timestamp: Long) ->
Unit
) {
    var selectedSchemeId by remember { mutableStateOf(deposit.accountSchemeId) }
    var amountStr by remember {
mutableStateOf(CostCalculator.formatPaiseToRupees(deposit.amountPaise).replace("₹", "").trim()) }
    var paymentSource by remember { mutableStateOf(deposit.paymentSource) }
    var notes by remember { mutableStateOf(deposit.notes ?: "") }
    var entryTimestamp by remember { mutableLongStateOf(deposit.paidTimestamp) }
    val context = LocalContext.current
    val cal = Calendar.getInstance().apply { timeInMillis = entryTimestamp }
    val dateStr = SimpleDateFormat("dd MMM yyyy, hh:mm a", Locale.getDefault()).format(Date(entryTimestamp))

    val amountPaise = CostCalculator.parseRupeesToPaise(amountStr) ?: 0L

    AlertDialog(
        onDismissRequest = onDismiss,
        title = { Text("Edit Pigmy Deposit") },
        text = {
            Column(verticalArrangement = Arrangement.spacedBy(10.dp)) {
                Text("Select Scheme:", style = MaterialTheme.typography.bodySmall, fontWeight =
FontWeight.Bold)
                Row(horizontalArrangement = Arrangement.spacedBy(6.dp)) {
                    allSchemes.forEach { sc ->
                        FilterChip(
                            selected = selectedSchemeId == sc.id,
                            onClick = { selectedSchemeId = sc.id },
                            label = { Text(sc.name) }
                        )
                    }
                }

                OutlinedTextField(
                    value = amountStr,
                    onValueChange = { amountStr = it },
                    label = { Text("Deposit Amount (₹) *") },
                    keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Decimal),
                    singleLine = true,
                    modifier = Modifier.fillMaxWidth()
                )

                Row(
                    modifier = Modifier.fillMaxWidth(),
                    horizontalArrangement = Arrangement.SpaceBetween,
                    verticalAlignment = Alignment.CenterVertically
                ) {
                    Text("Date & Time:", style = MaterialTheme.typography.bodySmall, fontWeight =
FontWeight.SemiBold)
                    OutlinedButton(
                        onClick = {
                            DatePickerDialog(context, { _, y, m, d ->
                                val c = Calendar.getInstance().apply {
                                    timeInMillis = entryTimestamp
                                    set(Calendar.YEAR, y)
                                    set(Calendar.MONTH, m)
                                    set(Calendar.DAY_OF_MONTH, d)
                                }
                                TimePickerDialog(context, { _, h, min ->
                                    c.set(Calendar.HOUR_OF_DAY, h)
                                    c.set(Calendar.MINUTE, min)
                                    entryTimestamp = c.timeInMillis
                                }, cal.get(Calendar.HOUR_OF_DAY), cal.get(Calendar.MINUTE), false).show()
                            }, cal.get(Calendar.YEAR), cal.get(Calendar.MONTH),
cal.get(Calendar.DAY_OF_MONTH)).show()
                        },
                        shape = RoundedCornerShape(8.dp)
                    ) {
                        Icon(Icons.Filled.CalendarToday, contentDescription = null, modifier =
Modifier.size(14.dp))
```

```
                        Spacer(modifier = Modifier.width(4.dp))
                        Text(dateStr, fontSize = 11.5.sp)
                    }
                }

                Text("Payment Source:", style = MaterialTheme.typography.bodySmall, fontWeight =
FontWeight.Bold)
                Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
                    FilterChip(
                        selected = paymentSource == "CASH",
                        onClick = { paymentSource = "CASH" },
                        label = { Text("Shop Cash (Outflow)") }
                    )
                    FilterChip(
                        selected = paymentSource == "BANK",
                        onClick = { paymentSource = "BANK" },
                        label = { Text("Direct / Bank") }
                    )
                }

                Text(
                    text = "• Updating will synchronize the linked Cash Out savings transfer atomically.",
                    style = MaterialTheme.typography.bodySmall,
                    color = MaterialTheme.colorScheme.primary
                )

                OutlinedTextField(
                    value = notes,
                    onValueChange = { notes = it },
                    label = { Text("Reference / Note (Optional)") },
                    singleLine = true,
                    modifier = Modifier.fillMaxWidth()
                )
            }
        },
        confirmButton = {
            Button(
                onClick = {
                    if (amountPaise > 0) {
                        onConfirm(selectedSchemeId, amountPaise, paymentSource, notes.trim().ifBlank { null
}, entryTimestamp)
                    }
                },
                enabled = amountPaise > 0
            ) {
                Text("Save Changes")
            }
        },
        dismissButton = {
            TextButton(onClick = onDismiss) {
                Text("Cancel")
            }
        }
    )
}

@Composable
fun EditPigmyWithdrawalDialog(
    withdrawal: PigmyWithdrawalEntity,
    currentScheme: PigmySchemeEntity?,
    maxAvailablePaise: Long,
    onDismiss: () -> Unit,
    onConfirm: (amountPaise: Long, destination: String, notes: String?, timestamp: Long) -> Unit
) {
    var amountStr by remember {
mutableStateOf(CostCalculator.formatPaiseToRupees(withdrawal.amountPaise).replace("₹", "").trim()) }
    var destination by remember { mutableStateOf(withdrawal.destination) }
    var notes by remember { mutableStateOf(withdrawal.notes ?: "") }
    var entryTimestamp by remember { mutableLongStateOf(withdrawal.withdrawalTimestamp) }
    val context = LocalContext.current
    val cal = Calendar.getInstance().apply { timeInMillis = entryTimestamp }
    val dateStr = SimpleDateFormat("dd MMM yyyy, hh:mm a", Locale.getDefault()).format(Date(entryTimestamp))

    val amountPaise = CostCalculator.parseRupeesToPaise(amountStr) ?: 0L
    val exceedsBalance = amountPaise > maxAvailablePaise

    AlertDialog(
        onDismissRequest = onDismiss,
        title = { Text("Edit Pigmy Withdrawal (${currentScheme?.name ?: "Scheme"})") },
        text = {
            Column(verticalArrangement = Arrangement.spacedBy(10.dp)) {
                Text(
                    text = "Max Available Savings:
${CostCalculator.formatPaiseToRupees(maxAvailablePaise)}",
                    style = MaterialTheme.typography.bodySmall.copy(fontWeight = FontWeight.Bold),
                    color = MaterialTheme.colorScheme.primary
                )

```

```
                OutlinedTextField(
                    value = amountStr,
                    onValueChange = { amountStr = it },
                    label = { Text("Withdrawal Amount (₹) *") },
                    keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Decimal),
                    singleLine = true,
                    isError = exceedsBalance,
                    modifier = Modifier.fillMaxWidth()
                )

                if (exceedsBalance) {
                    Text(
                        text = "   Amount exceeds available tracked savings (${CostCalculator.formatPaiseToRupees(maxAvailablePaise)}).",
                        style = MaterialTheme.typography.bodySmall,
                        color = MaterialTheme.colorScheme.error
                    )
                }

                Row(
                    modifier = Modifier.fillMaxWidth(),
                    horizontalArrangement = Arrangement.SpaceBetween,
                    verticalAlignment = Alignment.CenterVertically
                ) {
                    Text("Date & Time:", style = MaterialTheme.typography.bodySmall, fontWeight = FontWeight.SemiBold)
                    OutlinedButton(
                        onClick = {
                            DatePickerDialog(context, { _, y, m, d ->
                                val c = Calendar.getInstance().apply {
                                    timeInMillis = entryTimestamp
                                    set(Calendar.YEAR, y)
                                    set(Calendar.MONTH, m)
                                    set(Calendar.DAY_OF_MONTH, d)
                                }
                                TimePickerDialog(context, { _, h, min ->
                                    c.set(Calendar.HOUR_OF_DAY, h)
                                    c.set(Calendar.MINUTE, min)
                                    entryTimestamp = c.timeInMillis
                                }, cal.get(Calendar.HOUR_OF_DAY), cal.get(Calendar.MINUTE), false).show()
                            }, cal.get(Calendar.YEAR), cal.get(Calendar.MONTH), cal.get(Calendar.DAY_OF_MONTH)).show()
                        },
                        shape = RoundedCornerShape(8.dp)
                    ) {
                        Icon(Icons.Filled.CalendarToday, contentDescription = null, modifier = Modifier.size(14.dp))
                        Spacer(modifier = Modifier.width(4.dp))
                        Text(dateStr, fontSize = 11.5.sp)
                    }
                }

                Text("Destination:", style = MaterialTheme.typography.bodySmall, fontWeight = FontWeight.Bold)
                Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
                    FilterChip(
                        selected = destination == "CASH",
                        onClick = { destination = "CASH" },
                        label = { Text("Shop Cash (Inflow)") }
                    )
                    FilterChip(
                        selected = destination == "BANK",
                        onClick = { destination = "BANK" },
                        label = { Text("Bank Account") }
                    )
                }

                Text(
                    text = "• Updating will synchronize the linked Cash In savings transfer atomically.",
                    style = MaterialTheme.typography.bodySmall,
                    color = MaterialTheme.colorScheme.primary
                )

                OutlinedTextField(
                    value = notes,
                    onValueChange = { notes = it },
                    label = { Text("Reference / Note (Optional)") },
                    singleLine = true,
                    modifier = Modifier.fillMaxWidth()
                )
            }
        },
        confirmButton = {
            Button(
                onClick = {
                    if (amountPaise > 0 && !exceedsBalance) {
                        onConfirm(amountPaise, destination, notes.trim().ifBlank { null }, entryTimestamp)
                    }
```

```
                },
                enabled = amountPaise > 0 && !exceedsBalance
            ) {
                Text("Save Changes")
            }
        },
        dismissButton = {
            TextButton(onClick = onDismiss) {
                Text("Cancel")
            }
        }
    )
}
```

app\src\main\java\com\alieducational\app\ui\reports\CashTrackerScreen.kt

```
package com.alieducational.app.ui.reports

import android.content.Context
import android.content.Intent
import android.graphics.Bitmap
import android.graphics.BitmapFactory
import android.net.Uri
import android.provider.OpenableColumns
import android.app.DatePickerDialog
import android.app.TimePickerDialog
import android.widget.Toast
import androidx.activity.compose.rememberLauncherForActivityResult
import androidx.activity.result.contract.ActivityResultContracts
import androidx.compose.foundation.Image
import androidx.compose.foundation.background
import androidx.compose.foundation.border
import androidx.compose.foundation.clickable
import androidx.compose.foundation.horizontalScroll
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.automirrored.filled.ArrowBack
import androidx.compose.material.icons.filled.AccessTime
import androidx.compose.material.icons.filled.Add
import androidx.compose.material.icons.filled.CalendarToday
import androidx.compose.material.icons.filled.CameraAlt
import androidx.compose.material.icons.filled.ChevronLeft
import androidx.compose.material.icons.filled.ChevronRight
import androidx.compose.material.icons.filled.Close
import androidx.compose.material.icons.filled.Delete
import androidx.compose.material.icons.filled.FileUpload
import androidx.compose.material.icons.filled.Image
import androidx.compose.material.icons.filled.PhotoLibrary
import androidx.compose.material.icons.filled.Remove
import androidx.compose.material.icons.filled.Share
import androidx.compose.material3.AlertDialog
import androidx.compose.material3.Button
import androidx.compose.material3.ButtonDefaults
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.CircularProgressIndicator
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.FilterChip
import androidx.compose.material3.FilterChipDefaults
import androidx.compose.material3.HorizontalDivider
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedButton
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.RadioButton
import androidx.compose.material3.ExposedDropdownMenuBox
import androidx.compose.material3.ExposedDropdownMenuDefaults
import androidx.compose.material3.DropdownMenuItem
import androidx.compose.material3.Surface
import androidx.compose.material3.Tab
import androidx.compose.material3.TabRow
import androidx.compose.material3.Text
import androidx.compose.material3.TextButton
import androidx.compose.runtime.Composable
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableIntStateOf
import androidx.compose.runtime.mutableLongStateOf
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.rememberCoroutineScope
import androidx.compose.runtime.setValue
```

```
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.asImageBitmap
import androidx.compose.ui.layout.ContentScale
import androidx.compose.foundation.layout.heightIn
import androidx.compose.material.icons.filled.DeleteOutline
import androidx.compose.ui.res.painterResource
import com.alieducational.app.R
import com.alieducational.app.data.local.entity.CashCategoryEntity
import com.alieducational.app.data.local.entity.PigmySchemeEntity
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.text.style.TextAlign
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.alieducational.app.data.local.entity.ShopExpenseEntity
import com.alieducational.app.data.service.CashTrackerImportPreview
import com.alieducational.app.domain.calculation.CostCalculator
import com.alieducational.app.ui.AliViewModel
import com.alieducational.app.util.AttachmentManager
import kotlinx.coroutines.launch
import java.io.File
import java.text.SimpleDateFormat
import java.util.Calendar
import java.util.Date
import java.util.Locale

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun CashTrackerScreen(
    viewModel: AliViewModel,
    onNavigateBack: () -> Unit,
    modifier: Modifier = Modifier
) {
    val allExpenses by viewModel.allExpenses.collectAsState()
    val activeCategories by viewModel.activeCashCategories.collectAsState()
    val allPigmySchemes by viewModel.allPigmySchemes.collectAsState()
    val activePigmySchemes = remember(allPigmySchemes) { allPigmySchemes.filter { !it.isArchived } }

    var showManageCategoriesDialog by remember { mutableStateOf(false) }
    var categoryToDelete by remember { mutableStateOf<CashCategoryEntity?>(null) }

    val context = LocalContext.current
    val coroutineScope = rememberCoroutineScope()

    var isImporting by remember { mutableStateOf(false) }
    var isExporting by remember { mutableStateOf(false) }
    var importPreview by remember { mutableStateOf<CashTrackerImportPreview?>(null) }

    val importLauncher = rememberLauncherForActivityResult(
        contract = ActivityResultContracts.OpenDocument()
    ) { uri: Uri? ->
        if (uri != null) {
            isImporting = true
            val fileName = queryFileName(context, uri)
            viewModel.parseCashTrackerExcel(uri, fileName) { preview ->
                isImporting = false
                importPreview = preview
            }
        }
    }

    // 0: All, 1: Daily, 2: Weekly, 3: Monthly, 4: Yearly
    var selectedTab by remember { mutableIntStateOf(3) } // Default to Monthly
    var anchorTimestamp by remember { mutableLongStateOf(System.currentTimeMillis()) }

    var selectedCategoryFilter by remember { mutableStateOf<String?>(null) } // null = All Categories

    var showAddDialog by remember { mutableStateOf(false) }
    var isIncomeMode by remember { mutableStateOf(true) }

    val widgetAction by viewModel.widgetActionTrigger.collectAsState()
    androidx.compose.runtime.LaunchedEffect(widgetAction) {
        when (widgetAction) {
            com.alieducational.app.widget.AliSlimBarWidgetProvider.ACTION_CASH_IN -> {
                isIncomeMode = true
                showAddDialog = true
                viewModel.consumeWidgetAction()
            }
            com.alieducational.app.widget.AliSlimBarWidgetProvider.ACTION_CASH_OUT -> {
                isIncomeMode = false
                showAddDialog = true
                viewModel.consumeWidgetAction()
            }
        }
```

```
    }
    var selectedItemForVoid by remember { mutableStateOf<ShopExpenseEntity?>(null) }
    var viewingPhotoFile by remember { mutableStateOf<File?>(null) }

    // Date calculations based on anchor timestamp and selected tab
    val (startDate, endDate, rangeText) = remember(selectedTab, anchorTimestamp) {
        val cal = Calendar.getInstance().apply { timeInMillis = anchorTimestamp }
        val dateFormat = SimpleDateFormat("dd-MMM-yyyy", Locale.US)

        when (selectedTab) {
            0 -> { // All
                Triple(0L, Long.MAX_VALUE, "All Transactions")
            }
            1 -> { // Daily
                cal.set(Calendar.HOUR_OF_DAY, 0)
                cal.set(Calendar.MINUTE, 0)
                cal.set(Calendar.SECOND, 0)
                cal.set(Calendar.MILLISECOND, 0)
                val start = cal.timeInMillis
                cal.set(Calendar.HOUR_OF_DAY, 23)
                cal.set(Calendar.MINUTE, 59)
                cal.set(Calendar.SECOND, 59)
                cal.set(Calendar.MILLISECOND, 999)
                val end = cal.timeInMillis
                Triple(start, end, dateFormat.format(Date(start)))
            }
            2 -> { // Weekly (Monday to Sunday)
                cal.firstDayOfWeek = Calendar.MONDAY
                cal.set(Calendar.DAY_OF_WEEK, Calendar.MONDAY)
                cal.set(Calendar.HOUR_OF_DAY, 0)
                cal.set(Calendar.MINUTE, 0)
                cal.set(Calendar.SECOND, 0)
                cal.set(Calendar.MILLISECOND, 0)
                val start = cal.timeInMillis
                cal.add(Calendar.DAY_OF_WEEK, 6)
                cal.set(Calendar.HOUR_OF_DAY, 23)
                cal.set(Calendar.MINUTE, 59)
                cal.set(Calendar.SECOND, 59)
                cal.set(Calendar.MILLISECOND, 999)
                val end = cal.timeInMillis
                Triple(start, end, "${dateFormat.format(Date(start))} -> ${dateFormat.format(Date(end))}")
            }
            3 -> { // Monthly (1st to last day)
                cal.set(Calendar.DAY_OF_MONTH, 1)
                cal.set(Calendar.HOUR_OF_DAY, 0)
                cal.set(Calendar.MINUTE, 0)
                cal.set(Calendar.SECOND, 0)
                cal.set(Calendar.MILLISECOND, 0)
                val start = cal.timeInMillis
                cal.set(Calendar.DAY_OF_MONTH, cal.getActualMaximum(Calendar.DAY_OF_MONTH))
                cal.set(Calendar.HOUR_OF_DAY, 23)
                cal.set(Calendar.MINUTE, 59)
                cal.set(Calendar.SECOND, 59)
                cal.set(Calendar.MILLISECOND, 999)
                val end = cal.timeInMillis
                Triple(start, end, "${dateFormat.format(Date(start))} -> ${dateFormat.format(Date(end))}")
            }
            else -> { // Yearly
                cal.set(Calendar.DAY_OF_YEAR, 1)
                cal.set(Calendar.HOUR_OF_DAY, 0)
                cal.set(Calendar.MINUTE, 0)
                cal.set(Calendar.SECOND, 0)
                cal.set(Calendar.MILLISECOND, 0)
                val start = cal.timeInMillis
                cal.set(Calendar.DAY_OF_YEAR, cal.getActualMaximum(Calendar.DAY_OF_YEAR))
                cal.set(Calendar.HOUR_OF_DAY, 23)
                cal.set(Calendar.MINUTE, 59)
                cal.set(Calendar.SECOND, 59)
                cal.set(Calendar.MILLISECOND, 999)
                val end = cal.timeInMillis

                Triple(start, end, "${dateFormat.format(Date(start))} -> ${dateFormat.format(Date(end))}")
            }
        }
    }

    // Filter transactions in active range and category filter
    val filteredItems = remember(allExpenses, startDate, endDate, selectedCategoryFilter) {
        allExpenses.filter { item ->
            val inDate = item.timestamp in startDate..endDate
            val matchesCategory = if (selectedCategoryFilter == null) {
                true
            } else {
                item.categoryId == selectedCategoryFilter || item.category.equals(selectedCategoryFilter,
ignoreCase = true)
            }
            inDate && matchesCategory
        }
    }
```

```
258
259     // Totals
260     val totalIncomePaise = remember(filteredItems) {
261         filteredItems.filter { it.isIncome }.sumOf { it.amountPaise }
262     }
263     val totalExpensePaise = remember(filteredItems) {
264         filteredItems.filter { !it.isIncome }.sumOf { it.amountPaise }
265     }
266     val netBalancePaise = totalIncomePaise - totalExpensePaise
267
268     // Group items by day (descending)
269     val groupedByDay = remember(filteredItems) {
270         val dayFormat = SimpleDateFormat("EEE, dd MMM yyyy", Locale.US)
271         filteredItems.groupBy { dayFormat.format(Date(it.timestamp)) }
272     }
273
274     val tabTitles = listOf("All", "Daily", "Weekly", "Monthly", "Yearly")
275
276     Surface(
277         modifier = modifier.fillMaxSize(),
278         color = MaterialTheme.colorScheme.background
279     ) {
280         Column(modifier = Modifier.fillMaxSize()) {
281             // Header
282             Row(
283                 modifier = Modifier
284                     .fillMaxWidth()
285                     .background(MaterialTheme.colorScheme.surface)
286                     .padding(horizontal = 8.dp, vertical = 6.dp),
287                 verticalAlignment = Alignment.CenterVertically
288             ) {
289                 IconButton(onClick = onNavigateBack) {
290                     Icon(
291                         imageVector = Icons.AutoMirrored.Filled.ArrowBack,
292                         contentDescription = "Back",
293                         tint = MaterialTheme.colorScheme.onSurface
294                     )
295                 }
296                 Spacer(modifier = Modifier.width(4.dp))
297                 Text(
298                     text = "Cash In & Out",
299                     style = MaterialTheme.typography.titleLarge.copy(fontWeight = FontWeight.Bold),
300                     color = MaterialTheme.colorScheme.primary,
301                     modifier = Modifier.weight(1f)
302                 )
303
304                 // Import Excel Button
305                 IconButton(
306                     onClick = {
307                         importLauncher.launch(
308                             arrayOf(
309                                 "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
310                                 "application/vnd.ms-excel",
311                                 "text/csv",
312                                 "*/*"
313                             )
314                         )
315                     },
316                     enabled = !isImporting && !isExporting
317                 ) {
318                     if (isImporting) {
319                         CircularProgressIndicator(
320                             modifier = Modifier.size(20.dp),
321                             color = MaterialTheme.colorScheme.primary,
322                             strokeWidth = 2.dp
323                         )
324                     } else {
325                         Icon(
326                             imageVector = Icons.Filled.FileUpload,
327                             contentDescription = "Import Excel",
328                             tint = MaterialTheme.colorScheme.primary
329                         )
330                     }
331                 }
332
333                 // Share / Export via WhatsApp Button
334                 IconButton(
335                     onClick = {
336                         val itemsToExport = if (filteredItems.isNotEmpty()) filteredItems else allExpenses
337                         if (itemsToExport.isEmpty()) {
338                             Toast.makeText(context, "No transactions to export", Toast.LENGTH_SHORT).show()
339                         } else {
340                             isExporting = true
341                             viewModel.exportCashTrackerExcel(itemsToExport, rangeText) { shareIntent ->
342                                 isExporting = false
343                                 try {
344                                     val chooser = Intent.createChooser(shareIntent, "Share Cash Book").apply
    {
```

```
                                    addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)
                                }
                                context.startActivity(chooser)
                            } catch (e: Exception) {
                                Toast.makeText(context, "Failed to share: ${e.message}",
Toast.LENGTH_SHORT).show()
                            }
                        }
                    }
                },
                enabled = !isImporting && !isExporting
            ) {
                if (isExporting) {
                    CircularProgressIndicator(
                        modifier = Modifier.size(20.dp),
                        color = MaterialTheme.colorScheme.primary,
                        strokeWidth = 2.dp
                    )
                } else {
                    Icon(
                        imageVector = Icons.Filled.Share,
                        contentDescription = "Share / Export (WhatsApp)",
                        tint = MaterialTheme.colorScheme.primary
                    )
                }
            }
        }

        // Period Tabs (All, Daily, Weekly, Monthly, Yearly)
        TabRow(
            selectedTabIndex = selectedTab,
            containerColor = MaterialTheme.colorScheme.surface,
            contentColor = MaterialTheme.colorScheme.primary
        ) {
            tabTitles.forEachIndexed { index, title ->
                Tab(
                    selected = selectedTab == index,
                    onClick = { selectedTab = index },
                    text = {
                        Text(
                            text = title,
                            fontWeight = if (selectedTab == index) FontWeight.Bold else
FontWeight.Normal,
                            color = if (selectedTab == index) MaterialTheme.colorScheme.primary else
MaterialTheme.colorScheme.onSurfaceVariant,
                            fontSize = 11.5.sp,
                            maxLines = 1,
                            softWrap = false
                        )
                    }
                )
            }
        }

        // Date Navigator Banner
        Row(
            modifier = Modifier
                .fillMaxWidth()
                .background(MaterialTheme.colorScheme.surfaceVariant.copy(alpha = 0.5f))
                .padding(horizontal = 12.dp, vertical = 4.dp),
            verticalAlignment = Alignment.CenterVertically,
            horizontalArrangement = Arrangement.SpaceBetween
        ) {
            IconButton(
                onClick = {
                    val cal = Calendar.getInstance().apply { timeInMillis = anchorTimestamp }
                    when (selectedTab) {
                        1 -> cal.add(Calendar.DAY_OF_YEAR, -1)
                        2 -> cal.add(Calendar.WEEK_OF_YEAR, -1)
                        3 -> cal.add(Calendar.MONTH, -1)
                        4 -> cal.add(Calendar.YEAR, -1)
                    }
                    anchorTimestamp = cal.timeInMillis
                },
                enabled = selectedTab != 0,
                modifier = Modifier.size(32.dp)
            ) {
                Icon(Icons.Filled.ChevronLeft, contentDescription = "Previous")
            }

            Text(
                text = rangeText,
                style = MaterialTheme.typography.bodyMedium.copy(fontWeight = FontWeight.SemiBold),
                color = MaterialTheme.colorScheme.onSurface
            )

            IconButton(
                onClick = {
```

```
                        val cal = Calendar.getInstance().apply { timeInMillis = anchorTimestamp }
                        when (selectedTab) {
                            1 -> cal.add(Calendar.DAY_OF_YEAR, 1)
                            2 -> cal.add(Calendar.WEEK_OF_YEAR, 1)
                            3 -> cal.add(Calendar.MONTH, 1)
                            4 -> cal.add(Calendar.YEAR, 1)
                        }
                        anchorTimestamp = cal.timeInMillis
                    },
                    enabled = selectedTab != 0,
                    modifier = Modifier.size(32.dp)
                ) {
                    Icon(Icons.Filled.ChevronRight, contentDescription = "Next")
                }
            }

            // Category Filter Scrollable Row
            Row(
                modifier = Modifier
                    .fillMaxWidth()
                    .background(MaterialTheme.colorScheme.surface)
                    .horizontalScroll(rememberScrollState())
                    .padding(horizontal = 12.dp, vertical = 4.dp),
                horizontalArrangement = Arrangement.spacedBy(6.dp),
                verticalAlignment = Alignment.CenterVertically
            ) {
                FilterChip(
                    selected = selectedCategoryFilter == null,
                    onClick = { selectedCategoryFilter = null },
                    label = { Text("All Categories (${allExpenses.size})", fontSize = 11.sp) }
                )

                activeCategories.filter { it.name != "Pigmy" }.forEach { cat ->
                    val count = allExpenses.count { it.categoryId == cat.id || it.category.equals(cat.name,
ignoreCase = true) }
                    FilterChip(
                        selected = selectedCategoryFilter == cat.id || selectedCategoryFilter == cat.name,
                        onClick = {
                            selectedCategoryFilter = if (selectedCategoryFilter == cat.id ||
selectedCategoryFilter == cat.name) null else cat.id
                        },
                        label = { Text("${cat.name} ($count)", fontSize = 11.sp) }
                    )
                }

                activePigmySchemes.forEach { scheme ->
                    val count = allExpenses.count { it.categoryId == scheme.id ||
it.category.equals(scheme.name, ignoreCase = true) }
                    FilterChip(
                        selected = selectedCategoryFilter == scheme.id || selectedCategoryFilter ==
scheme.name,
                        onClick = {
                            selectedCategoryFilter = if (selectedCategoryFilter == scheme.id ||
selectedCategoryFilter == scheme.name) null else scheme.id
                        },
                        leadingIcon = {
                            Icon(
                                painter = painterResource(R.drawable.ic_pigmy_coin),
                                contentDescription = null,
                                modifier = Modifier.size(13.dp),
                                tint = MaterialTheme.colorScheme.primary
                            )
                        },
                        label = { Text("${scheme.name} ($count)", fontSize = 11.sp) }
                    )
                }

                IconButton(
                    onClick = { showManageCategoriesDialog = true },
                    modifier = Modifier.size(28.dp)
                ) {
                    Icon(
                        Icons.Filled.DeleteOutline,
                        contentDescription = "Manage Categories",
                        modifier = Modifier.size(16.dp),
                        tint = MaterialTheme.colorScheme.onSurfaceVariant
                    )
                }
            }

            // Table Header: Date | Income | Expense
            Row(
                modifier = Modifier
                    .fillMaxWidth()
                    .background(MaterialTheme.colorScheme.surfaceVariant.copy(alpha = 0.2f))
                    .padding(horizontal = 16.dp, vertical = 6.dp),
                verticalAlignment = Alignment.CenterVertically
            ) {
                Text(
```

```
                    text = "Description / Details",
                    style = MaterialTheme.typography.labelSmall.copy(fontWeight = FontWeight.Bold),
                    color = MaterialTheme.colorScheme.onSurfaceVariant,
                    modifier = Modifier.weight(1.5f)
                )
                Text(
                    text = "Income",
                    style = MaterialTheme.typography.labelSmall.copy(fontWeight = FontWeight.Bold),
                    color = Color(0xFF2E7D32),
                    textAlign = TextAlign.End,
                    modifier = Modifier.weight(1f)
                )
                Text(
                    text = "Expense",
                    style = MaterialTheme.typography.labelSmall.copy(fontWeight = FontWeight.Bold),
                    color = Color(0xFFD32F2F),
                    textAlign = TextAlign.End,
                    modifier = Modifier.weight(1f)
                )
            }

            // Content List
            if (filteredItems.isEmpty()) {
                Box(
                    modifier = Modifier
                        .fillMaxWidth()
                        .weight(1f),
                    contentAlignment = Alignment.Center
                ) {
                    Column(horizontalAlignment = Alignment.CenterHorizontally) {
                        Text(
                            text = "No entries in this period",
                            style = MaterialTheme.typography.bodyMedium,
                            color = MaterialTheme.colorScheme.onSurfaceVariant
                        )
                        Spacer(modifier = Modifier.height(4.dp))
                        Text(
                            text = "Tap + Income or - Expense below to add",
                            style = MaterialTheme.typography.labelSmall,
                            color = MaterialTheme.colorScheme.onSurfaceVariant
                        )
                    }
                }
            } else {
                LazyColumn(
                    modifier = Modifier
                        .fillMaxWidth()
                        .weight(1f)
                ) {
                    val timeFormat = SimpleDateFormat("hh:mm a", Locale.US)

                    groupedByDay.forEach { (dayString, itemsOnDay) ->
                        // Date Header
                        item {
                            Surface(
                                color = MaterialTheme.colorScheme.surfaceVariant.copy(alpha = 0.3f),
                                modifier = Modifier.fillMaxWidth()
                            ) {
                                Text(
                                    text = dayString,
                                    style = MaterialTheme.typography.labelMedium.copy(fontWeight =
    FontWeight.Bold),
                                    color = MaterialTheme.colorScheme.onSurface,
                                    modifier = Modifier.padding(horizontal = 16.dp, vertical = 6.dp)
                                )
                            }
                        }

                        // Day's Entries
                        items(itemsOnDay) { entry ->
                            Row(
                                modifier = Modifier
                                    .fillMaxWidth()
                                    .clickable { selectedItemForVoid = entry }
                                    .padding(horizontal = 16.dp, vertical = 10.dp),
                                verticalAlignment = Alignment.CenterVertically
                            ) {
                                // Description, Category pill, Time & Photo Icon
                                Column(modifier = Modifier.weight(1.5f)) {
                                    Row(verticalAlignment = Alignment.CenterVertically) {
                                        // Category Pill
                                        Card(
                                            shape = RoundedCornerShape(4.dp),
                                            colors = CardDefaults.cardColors(
                                                containerColor = if (entry.isSavingsTransfer)
    Color(0xFFFFF3E0) else MaterialTheme.colorScheme.primaryContainer.copy(alpha = 0.6f)
                                            )
                                        ) {
```

```
                                        Text(
                                            text = entry.category,
                                            modifier = Modifier.padding(horizontal = 6.dp, vertical =
    2.dp),
                                            style = MaterialTheme.typography.labelSmall,
                                            color = if (entry.isSavingsTransfer) Color(0xFFE65100) else
    MaterialTheme.colorScheme.onPrimaryContainer,
                                            fontWeight = FontWeight.Bold,
                                            fontSize = 10.sp
                                        )
                                    }

                                    // Photo Attachment Indicator
                                    if (!entry.attachmentUri.isNullOrBlank()) {
                                        Spacer(modifier = Modifier.width(6.dp))
                                        Box(
                                            modifier = Modifier
                                                .size(20.dp)
                                                .background(Color(0xFFE0F2FE), RoundedCornerShape(4.dp))
                                                .clickable {
                                                    val f = AttachmentManager.getAttachmentFile(context,
     entry.attachmentUri)
                                                    if (f != null && f.exists()) {
                                                        viewingPhotoFile = f
                                                    } else {
                                                        Toast.makeText(context, "Attachment file not
    found", Toast.LENGTH_SHORT).show()
                                                    }
                                                },
                                            contentAlignment = Alignment.Center
                                        ) {
                                            Icon(
                                                Icons.Filled.Image,
                                                contentDescription = "View Photo",
                                                tint = Color(0xFF0284C7),
                                                modifier = Modifier.size(14.dp)
                                            )
                                        }
                                    }
                                }

                                Spacer(modifier = Modifier.height(2.dp))

                                val noteSuffix = if (!entry.notes.isNullOrBlank()) entry.notes else ""
                                if (!noteSuffix.isNullOrBlank()) {
                                    Text(
                                        text = noteSuffix,
                                        style = MaterialTheme.typography.bodyMedium.copy(fontWeight =
    FontWeight.SemiBold),
                                        color = MaterialTheme.colorScheme.onSurface
                                    )
                                }

                                Text(
                                    text = timeFormat.format(Date(entry.timestamp)),
                                    style = MaterialTheme.typography.labelSmall,
                                    color = MaterialTheme.colorScheme.onSurfaceVariant
                                )
                            }

                            // Income Column
                            Text(
                                text = if (entry.isIncome)
    CostCalculator.formatPaiseToRupees(entry.amountPaise) else "",
                                style = MaterialTheme.typography.bodyMedium.copy(fontWeight =
    FontWeight.Bold),
                                color = Color(0xFF2E7D32),
                                textAlign = TextAlign.End,
                                modifier = Modifier.weight(1f)
                            )

                            // Expense Column
                            Text(
                                text = if (!entry.isIncome)
    CostCalculator.formatPaiseToRupees(entry.amountPaise) else "",
                                style = MaterialTheme.typography.bodyMedium.copy(fontWeight =
    FontWeight.Bold),
                                color = Color(0xFFD32F2F),
                                textAlign = TextAlign.End,
                                modifier = Modifier.weight(1f)
                            )
                        }
                        HorizontalDivider(
                            color = MaterialTheme.colorScheme.outlineVariant.copy(alpha = 0.2f),
                            modifier = Modifier.padding(horizontal = 16.dp)
                        )
                    }
                }
```

```
            }
        }

        // Two Action Buttons: + Income & - Expense
        Row(
            modifier = Modifier
                .fillMaxWidth()
                .padding(horizontal = 12.dp, vertical = 8.dp),
            horizontalArrangement = Arrangement.spacedBy(10.dp)
        ) {
            Button(
                onClick = {
                    isIncomeMode = true
                    showAddDialog = true
                },
                modifier = Modifier
                    .weight(1f)
                    .height(48.dp),
                shape = RoundedCornerShape(8.dp),
                colors = ButtonDefaults.buttonColors(
                    containerColor = Color(0xFF2E7D32),
                    contentColor = Color.White
                )
            ) {
                Icon(Icons.Filled.Add, contentDescription = null, modifier = Modifier.size(18.dp))
                Spacer(modifier = Modifier.width(6.dp))
                Text("Income", fontSize = 16.sp, fontWeight = FontWeight.Bold)
            }

            Button(
                onClick = {
                    isIncomeMode = false
                    showAddDialog = true
                },
                modifier = Modifier
                    .weight(1f)
                    .height(48.dp),
                shape = RoundedCornerShape(8.dp),
                colors = ButtonDefaults.buttonColors(
                    containerColor = Color(0xFFD32F2F),
                    contentColor = Color.White
                )
            ) {
                Icon(Icons.Filled.Remove, contentDescription = null, modifier = Modifier.size(18.dp))
                Spacer(modifier = Modifier.width(6.dp))
                Text("Expense", fontSize = 16.sp, fontWeight = FontWeight.Bold)
            }
        }

        // Summary Footer
        Surface(
            color = MaterialTheme.colorScheme.surface,
            tonalElevation = 6.dp,
            modifier = Modifier
                .fillMaxWidth()
                .border(1.dp, MaterialTheme.colorScheme.outlineVariant)
        ) {
            Row(
                modifier = Modifier
                    .fillMaxWidth()
                    .padding(vertical = 8.dp),
                horizontalArrangement = Arrangement.SpaceEvenly
            ) {
                Column(
                    modifier = Modifier.weight(1f),
                    horizontalAlignment = Alignment.CenterHorizontally
                ) {
                    Text(
                        text = "Total Income",
                        style = MaterialTheme.typography.labelSmall,
                        color = Color(0xFF2E7D32)
                    )
                    Text(
                        text = CostCalculator.formatPaiseToRupees(totalIncomePaise),
                        style = MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.Bold),
                        color = Color(0xFF2E7D32)
                    )
                }

                Box(
                    modifier = Modifier
                        .width(1.dp)
                        .height(36.dp)
                        .background(MaterialTheme.colorScheme.outlineVariant)
                )

                Column(
                    modifier = Modifier.weight(1f),
```

```
                        horizontalAlignment = Alignment.CenterHorizontally
                    ) {
                        Text(
                            text = "Total Expense",
                            style = MaterialTheme.typography.labelSmall,
                            color = Color(0xFFD32F2F)
                        )
                        Text(
                            text = CostCalculator.formatPaiseToRupees(totalExpensePaise),
                            style = MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.Bold),
                            color = Color(0xFFD32F2F)
                        )
                    }

                    Box(
                        modifier = Modifier
                            .width(1.dp)
                            .height(36.dp)
                            .background(MaterialTheme.colorScheme.outlineVariant)
                    )

                    Column(
                        modifier = Modifier.weight(1f),
                        horizontalAlignment = Alignment.CenterHorizontally
                    ) {
                        Text(
                            text = "Balance",
                            style = MaterialTheme.typography.labelSmall,
                            color = if (netBalancePaise >= 0) Color(0xFF2E7D32) else Color(0xFFD32F2F)
                        )
                        Text(
                            text = CostCalculator.formatPaiseToRupees(netBalancePaise),
                            style = MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.Bold),
                            color = if (netBalancePaise >= 0) Color(0xFF2E7D32) else Color(0xFFD32F2F)
                        )
                    }
                }
            }
        }
    }

    // Add Income / Expense Dialog with Dedicated Category Selector & Photo Attachment
    if (showAddDialog) {
        var amountStr by remember { mutableStateOf("") }
        var description by remember { mutableStateOf("") }
        var paymentMode by remember { mutableStateOf("CASH") }
        var note by remember { mutableStateOf("") }
        var entryTimestamp by remember { mutableLongStateOf(System.currentTimeMillis()) }

        // Category selection
        val defaultCat = activeCategories.firstOrNull { it.name == "General" } ?:
activeCategories.firstOrNull()
        var selectedCatId by remember { mutableStateOf(defaultCat?.id ?: "") }
        var selectedCatName by remember { mutableStateOf(defaultCat?.name ?: "General") }

        val selectedPigmyScheme = remember(selectedCatId, selectedCatName, activePigmySchemes) {
            activePigmySchemes.firstOrNull { it.id == selectedCatId || it.name.equals(selectedCatName,
ignoreCase = true) }
        }

        var showCreateCategoryDialog by remember { mutableStateOf(false) }

        // Photo attachment
        var attachedPhotoUri by remember { mutableStateOf<Uri?>(null) }
        var tempCameraFile by remember { mutableStateOf<File?>(null) }

        val cameraLauncher = rememberLauncherForActivityResult(
            contract = ActivityResultContracts.TakePicture()
        ) { success ->
            if (success && tempCameraFile != null && tempCameraFile!!.exists() && tempCameraFile!!.length()
> 0) {
                attachedPhotoUri = Uri.fromFile(tempCameraFile)
            }
        }

        val galleryLauncher = rememberLauncherForActivityResult(
            contract = ActivityResultContracts.GetContent()
        ) { uri ->
            if (uri != null) {
                attachedPhotoUri = uri
            }
        }

        val dialogContext = LocalContext.current
        val cal = Calendar.getInstance().apply { timeInMillis = entryTimestamp }
        val entryDateStr = SimpleDateFormat("dd MMM yyyy", Locale.US).format(Date(entryTimestamp))
        val entryTimeStr = SimpleDateFormat("hh:mm a", Locale.US).format(Date(entryTimestamp))
        val isFuture = entryTimestamp > System.currentTimeMillis() + 60_000L
```

```
852
853         AlertDialog(
854             onDismissRequest = { showAddDialog = false },
855             title = {
856                 Text(
857                     text = if (isIncomeMode) "Add Income (Cash In)" else "Add Expense (Cash Out)",
858                     fontWeight = FontWeight.Bold,
859                     color = if (isIncomeMode) Color(0xFF2E7D32) else Color(0xFFD32F2F)
860                 )
861             },
862             text = {
863                 LazyColumn(
864                     verticalArrangement = Arrangement.spacedBy(10.dp),
865                     modifier = Modifier.fillMaxWidth()
866                 ) {
867                     item {
868                         // Category Selector
869                         Text(
870                             text = "Category",
871                             style = MaterialTheme.typography.labelMedium.copy(fontWeight =
    FontWeight.SemiBold),
872                             color = MaterialTheme.colorScheme.onSurfaceVariant
873                         )
874                         Row(
875                             modifier = Modifier
876                                 .fillMaxWidth()
877                                 .horizontalScroll(rememberScrollState()),
878                             horizontalArrangement = Arrangement.spacedBy(6.dp),
879                             verticalAlignment = Alignment.CenterVertically
880                         ) {
881                             activeCategories.filter { it.name != "Pigmy" }.forEach { cat ->
882                                 FilterChip(
883                                     selected = selectedPigmyScheme == null && (selectedCatId == cat.id ||
    selectedCatName == cat.name),
884                                     onClick = {
885                                         selectedCatId = cat.id
886                                         selectedCatName = cat.name
887                                     },
888                                     label = { Text(cat.name, fontSize = 11.5.sp) }
889                                 )
890                             }
891
892                             activePigmySchemes.forEach { scheme ->
893                                 FilterChip(
894                                     selected = selectedPigmyScheme?.id == scheme.id ||
    selectedPigmyScheme?.name == scheme.name,
895                                     onClick = {
896                                         selectedCatId = scheme.id
897                                         selectedCatName = scheme.name
898                                     },
899                                     leadingIcon = {
900                                         Icon(
901                                             painter = painterResource(R.drawable.ic_pigmy_coin),
902                                             contentDescription = null,
903                                             modifier = Modifier.size(14.dp),
904                                             tint = MaterialTheme.colorScheme.primary
905                                         )
906                                     },
907                                     label = { Text(scheme.name, fontSize = 11.5.sp) }
908                                 )
909                             }
910
911                             // Add New Category Chip / Action
912                             OutlinedButton(
913                                 onClick = { showCreateCategoryDialog = true },
914                                 shape = RoundedCornerShape(8.dp),
915                                 modifier = Modifier.height(32.dp)
916                             ) {
917                                 Icon(Icons.Filled.Add, contentDescription = null, modifier =
    Modifier.size(14.dp))
918                                 Spacer(modifier = Modifier.width(4.dp))
919                                 Text("New Category", fontSize = 11.sp)
920                             }
921
922                             // Remove / Manage Categories Button
923                             OutlinedButton(
924                                 onClick = { showManageCategoriesDialog = true },
925                                 shape = RoundedCornerShape(8.dp),
926                                 modifier = Modifier.height(32.dp),
927                                 colors = ButtonDefaults.outlinedButtonColors(contentColor =
    MaterialTheme.colorScheme.error)
928                             ) {
929                                 Icon(Icons.Filled.DeleteOutline, contentDescription = null, modifier =
    Modifier.size(14.dp))
930                                 Spacer(modifier = Modifier.width(4.dp))
931                                 Text("Remove...", fontSize = 11.sp)
932                             }
933                         }
```

```
                    }

                    if (selectedPigmyScheme != null) {
                        item {
                            Card(
                                colors = CardDefaults.cardColors(containerColor =
MaterialTheme.colorScheme.primaryContainer.copy(alpha = 0.4f)),
                                shape = RoundedCornerShape(8.dp),
                                modifier = Modifier.fillMaxWidth()
                            ) {
                                Row(
                                    modifier = Modifier.padding(8.dp),
                                    verticalAlignment = Alignment.CenterVertically
                                ) {
                                    Icon(
                                        painter = painterResource(R.drawable.ic_pigmy_coin),
                                        contentDescription = null,
                                        modifier = Modifier.size(18.dp),
                                        tint = MaterialTheme.colorScheme.primary
                                    )
                                    Spacer(modifier = Modifier.width(8.dp))
                                    Text(
                                        text = if (isIncomeMode) {
                                            "Pigmy Withdrawal: Will withdraw from
${selectedPigmyScheme.name} into shop cash."
                                        } else {
                                            "Pigmy Deposit: Will deposit into ${selectedPigmyScheme.name}
from shop cash."
                                        },
                                        style = MaterialTheme.typography.bodySmall,
                                        color = MaterialTheme.colorScheme.onSurface
                                    )
                                }
                            }
                        }
                    }

                    item {
                        // Date & Time pickers
                        Text(
                            text = "Transaction Date & Time",
                            style = MaterialTheme.typography.labelMedium.copy(fontWeight =
FontWeight.SemiBold),
                            color = MaterialTheme.colorScheme.onSurfaceVariant
                        )

                        Row(
                            modifier = Modifier.fillMaxWidth(),
                            horizontalArrangement = Arrangement.spacedBy(8.dp)
                        ) {
                            OutlinedButton(
                                onClick = {
                                    DatePickerDialog(
                                        dialogContext,
                                        { _, y, m, d ->
                                            val c = Calendar.getInstance().apply {
                                                timeInMillis = entryTimestamp
                                                set(Calendar.YEAR, y)
                                                set(Calendar.MONTH, m)
                                                set(Calendar.DAY_OF_MONTH, d)
                                            }
                                            entryTimestamp = c.timeInMillis
                                        },
                                        cal.get(Calendar.YEAR),
                                        cal.get(Calendar.MONTH),
                                        cal.get(Calendar.DAY_OF_MONTH)
                                    ).show()
                                },
                                shape = RoundedCornerShape(8.dp),
                                modifier = Modifier.weight(1.1f)
                            ) {
                                Icon(Icons.Filled.CalendarToday, contentDescription = null, modifier =
Modifier.size(14.dp))
                                Spacer(modifier = Modifier.width(4.dp))
                                Text(entryDateStr, fontSize = 12.sp, maxLines = 1)
                            }

                            OutlinedButton(
                                onClick = {
                                    TimePickerDialog(
                                        dialogContext,
                                        { _, hour, minute ->
                                            val c = Calendar.getInstance().apply {
                                                timeInMillis = entryTimestamp
                                                set(Calendar.HOUR_OF_DAY, hour)
                                                set(Calendar.MINUTE, minute)
                                            }
                                            entryTimestamp = c.timeInMillis
                                        },
                                        cal.get(Calendar.HOUR_OF_DAY),
```

```
                                        cal.get(Calendar.MINUTE),
                                        false
                                    ).show()
                                },
                                shape = RoundedCornerShape(8.dp),
                                modifier = Modifier.weight(0.9f)
                            ) {
                                Icon(Icons.Filled.AccessTime, contentDescription = null, modifier = Modifier.size(14.dp))
                                Spacer(modifier = Modifier.width(4.dp))
                                Text(entryTimeStr, fontSize = 12.sp, maxLines = 1)
                            }
                        }

                        if (isFuture) {
                            Card(
                                colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.tertiaryContainer.copy(alpha = 0.5f)),
                                shape = RoundedCornerShape(8.dp),
                                modifier = Modifier
                                    .fillMaxWidth()
                                    .padding(top = 4.dp)
                            ) {
                                Text(
                                    text = "   Notice: This entry has a future date/time. It will be recorded with this timestamp.",
                                    style = MaterialTheme.typography.bodySmall,
                                    color = MaterialTheme.colorScheme.onTertiaryContainer,
                                    modifier = Modifier.padding(8.dp)
                                )
                            }
                        }
                    }

                    item {
                        OutlinedTextField(
                            value = amountStr,
                            onValueChange = { amountStr = it },
                            label = { Text("Amount (₹) *") },
                            placeholder = { Text("0") },
                            keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Decimal),
                            singleLine = true,
                            modifier = Modifier.fillMaxWidth()
                        )
                    }

                    item {
                        OutlinedTextField(
                            value = description,
                            onValueChange = { description = it },
                            label = { Text("Description / Particulars (Optional)") },
                            placeholder = { Text(if (isIncomeMode) "e.g. Counter Sales, Customer Payment" else "e.g. Office Rent, Electricity Bill, Tea") },
                            singleLine = true,
                            modifier = Modifier.fillMaxWidth()
                        )
                    }

                    item {
                        // Payment Mode Chips
                        Text(
                            text = "Payment Mode",
                            style = MaterialTheme.typography.labelMedium.copy(fontWeight = FontWeight.SemiBold),
                            color = MaterialTheme.colorScheme.onSurfaceVariant
                        )
                        Row(
                            modifier = Modifier.fillMaxWidth(),
                            horizontalArrangement = Arrangement.spacedBy(6.dp)
                        ) {
                            listOf("CASH" to "Cash", "GPAY" to "GPAY / UPI", "BANK" to "Bank").forEach { (key, label) ->
                                FilterChip(
                                    selected = paymentMode == key,
                                    onClick = { paymentMode = key },
                                    label = { Text(label, fontSize = 12.sp) },
                                    modifier = Modifier.weight(1f)
                                )
                            }
                        }
                    }

                    item {
                        OutlinedTextField(
                            value = note,
                            onValueChange = { note = it },
                            label = { Text("Extra Note (Optional)") },
                            placeholder = { Text("Optional details / voucher #") },
```

```
                            singleLine = true,
                            modifier = Modifier.fillMaxWidth()
                        )
                    }

                    item {
                        // Optional Photo Attachment Section
                        Text(
                            text = "Receipt / Evidence Photo (Optional)",
                            style = MaterialTheme.typography.labelMedium.copy(fontWeight =
FontWeight.SemiBold),
                            color = MaterialTheme.colorScheme.onSurfaceVariant
                        )

                        if (attachedPhotoUri != null) {
                            // Show thumbnail with actions
                            Card(
                                shape = RoundedCornerShape(8.dp),
                                colors = CardDefaults.cardColors(containerColor =
MaterialTheme.colorScheme.surfaceVariant.copy(alpha = 0.5f)),
                                modifier = Modifier.fillMaxWidth()
                            ) {
                                Row(
                                    modifier = Modifier
                                        .fillMaxWidth()
                                        .padding(8.dp),
                                    verticalAlignment = Alignment.CenterVertically,
                                    horizontalArrangement = Arrangement.SpaceBetween
                                ) {
                                    Row(verticalAlignment = Alignment.CenterVertically) {
                                        Icon(
                                            Icons.Filled.Image,
                                            contentDescription = null,
                                            tint = MaterialTheme.colorScheme.primary,
                                            modifier = Modifier.size(28.dp)
                                        )
                                        Spacer(modifier = Modifier.width(8.dp))
                                        Column {
                                            Text("Photo Attached", fontWeight = FontWeight.SemiBold,
fontSize = 13.sp)
                                            Text("Ready to save", style =
MaterialTheme.typography.bodySmall, color = MaterialTheme.colorScheme.onSurfaceVariant)
                                        }
                                    }

                                    Row(verticalAlignment = Alignment.CenterVertically) {
                                        TextButton(onClick = {
                                            val (camUri, file) =
AttachmentManager.createTempCameraUri(dialogContext)
                                            tempCameraFile = file
                                            cameraLauncher.launch(camUri)
                                        }) {
                                            Text("Replace", fontSize = 11.sp)
                                        }
                                        IconButton(onClick = { attachedPhotoUri = null }) {
                                            Icon(Icons.Filled.Delete, contentDescription = "Remove", tint =
MaterialTheme.colorScheme.error)
                                        }
                                    }
                                }
                            }
                        } else {
                            Row(
                                modifier = Modifier.fillMaxWidth(),
                                horizontalArrangement = Arrangement.spacedBy(8.dp)
                            ) {
                                OutlinedButton(
                                    onClick = {
                                        val (camUri, file) =
AttachmentManager.createTempCameraUri(dialogContext)
                                        tempCameraFile = file
                                        cameraLauncher.launch(camUri)
                                    },
                                    shape = RoundedCornerShape(8.dp),
                                    modifier = Modifier.weight(1f)
                                ) {
                                    Icon(Icons.Filled.CameraAlt, contentDescription = null, modifier =
Modifier.size(16.dp))
                                    Spacer(modifier = Modifier.width(6.dp))
                                    Text("Take Photo", fontSize = 12.sp)
                                }

                                OutlinedButton(
                                    onClick = { galleryLauncher.launch("image/*") },
                                    shape = RoundedCornerShape(8.dp),
                                    modifier = Modifier.weight(1f)
                                ) {
                                    Icon(Icons.Filled.PhotoLibrary, contentDescription = null, modifier =
```

```
    Modifier.size(16.dp))
                                    Spacer(modifier = Modifier.width(6.dp))
                                    Text("Choose Photo", fontSize = 12.sp)
                                }
                            }
                        }
                    }
                },
                confirmButton = {
                    Button(
                        onClick = {
                            val amountPaise = CostCalculator.parseRupeesToPaise(amountStr) ?: 0L
                            if (amountPaise <= 0) return@Button

                            val effectiveDescription = if (description.isNotBlank()) description.trim() else
    selectedCatName

                            coroutineScope.launch {
                                var persistedPath: String? = null
                                if (attachedPhotoUri != null) {
                                    persistedPath = AttachmentManager.persistImageAttachment(context,
    attachedPhotoUri!!)
                                }

                                if (selectedPigmyScheme != null) {
                                    val pigmyNote = buildString {
                                        if (description.isNotBlank() && description.trim() !=
    selectedPigmyScheme.name) {
                                            append(description.trim())
                                        }
                                        if (note.isNotBlank()) {
                                            if (isNotEmpty()) append(" - ")
                                            append(note.trim())
                                        }
                                    }.ifBlank { null }

                                    if (isIncomeMode) {
                                        val destination = if (paymentMode == "CASH") "CASH" else "BANK"
                                        viewModel.recordPigmyWithdrawal(
                                            schemeId = selectedPigmyScheme.id,
                                            amountPaise = amountPaise,
                                            destination = destination,
                                            notes = pigmyNote,
                                            withdrawalTimestamp = entryTimestamp,
                                            attachmentUri = persistedPath,
                                            onDone = { showAddDialog = false },
                                            onError = { errMsg ->
                                                Toast.makeText(context, errMsg, Toast.LENGTH_LONG).show()
                                            }
                                        )
                                    } else {
                                        val paymentSource = if (paymentMode == "CASH") "CASH" else "BANK"
                                        viewModel.recordPigmyDeposit(
                                            schemeId = selectedPigmyScheme.id,
                                            amountPaise = amountPaise,
                                            paymentSource = paymentSource,
                                            notes = pigmyNote,
                                            paidTimestamp = entryTimestamp,
                                            attachmentUri = persistedPath,
                                            onDone = { showAddDialog = false }
                                        )
                                    }
                                } else {
                                    val paymentNote = buildString {
                                        append(paymentMode)
                                        if (note.isNotBlank()) append(" - ${note.trim()}")
                                        if (description.isNotBlank() && description.trim() != selectedCatName) {
                                            append(" (${description.trim()})")
                                        } else if (description.isBlank()) {
                                            append(" ($selectedCatName)")
                                        }
                                    }

                                    viewModel.recordShopExpense(
                                        category = selectedCatName,
                                        amountPaise = amountPaise,
                                        isIncome = isIncomeMode,
                                        notes = paymentNote,
                                        timestamp = entryTimestamp,
                                        categoryId = selectedCatId.ifBlank { null },
                                        attachmentUri = persistedPath,
                                        onDone = { showAddDialog = false }
                                    )
                                }
                            }
                        },
                        colors = ButtonDefaults.buttonColors(
```

```
                        containerColor = if (isIncomeMode) Color(0xFF2E7D32) else Color(0xFFD32F2F)
                    )
                ) {
                    Text("Save", fontWeight = FontWeight.Bold)
                }
            },
            dismissButton = {
                OutlinedButton(onClick = { showAddDialog = false }) {
                    Text("Cancel")
                }
            }
        )

        // Sub-dialog: Add New Category (Preserves parent form intact)
        if (showCreateCategoryDialog) {
            var newCatName by remember { mutableStateOf("") }
            var catError by remember { mutableStateOf<String?>(null) }

            AlertDialog(
                onDismissRequest = { showCreateCategoryDialog = false },
                title = { Text("New Category") },
                text = {
                    Column(verticalArrangement = Arrangement.spacedBy(8.dp)) {
                        Text("Create a new organizational category for cash entries.")
                        OutlinedTextField(
                            value = newCatName,
                            onValueChange = {
                                newCatName = it
                                catError = null
                            },
                            label = { Text("Category Name *") },
                            placeholder = { Text("e.g. Repairs, Shipping, Tea") },
                            singleLine = true,
                            isError = catError != null,
                            modifier = Modifier.fillMaxWidth()
                        )
                        if (catError != null) {
                            Text(catError!!, color = MaterialTheme.colorScheme.error, style =
MaterialTheme.typography.bodySmall)
                        }
                    }
                },
                confirmButton = {
                    Button(
                        onClick = {
                            val trimmed = newCatName.trim()
                            if (trimmed.isBlank()) {
                                catError = "Category name cannot be empty"
                                return@Button
                            }
                            if (activeCategories.any { it.name.equals(trimmed, ignoreCase = true) }) {
                                catError = "Category with this name already exists"
                                return@Button
                            }
                            viewModel.createCashCategory(trimmed) { newId ->
                                selectedCatId = newId
                                selectedCatName = trimmed
                                showCreateCategoryDialog = false
                            }
                        }
                    ) {
                        Text("Create & Select")
                    }
                },
                dismissButton = {
                    TextButton(onClick = { showCreateCategoryDialog = false }) {
                        Text("Cancel")
                    }
                }
            )
        }
    }

    // Dialog: Manage / Remove Categories
    if (showManageCategoriesDialog) {
        AlertDialog(
            onDismissRequest = {
                showManageCategoriesDialog = false
                categoryToDelete = null
            },
            title = {
                Row(
                    modifier = Modifier.fillMaxWidth(),
                    horizontalArrangement = Arrangement.SpaceBetween,
                    verticalAlignment = Alignment.CenterVertically
                ) {
                    Text("Manage Categories", fontWeight = FontWeight.Bold)
                    IconButton(onClick = {
```

```
                        showManageCategoriesDialog = false
                        categoryToDelete = null
                    }) {
                        Icon(Icons.Filled.Close, contentDescription = "Close")
                    }
                }
            },
            text = {
                Column(
                    modifier = Modifier
                        .fillMaxWidth()
                        .padding(vertical = 4.dp),
                    verticalArrangement = Arrangement.spacedBy(8.dp)
                ) {
                    Text(
                        text = "Remove categories you no longer need. Past transactions will keep their recorded category name.",
                        style = MaterialTheme.typography.bodySmall,
                        color = MaterialTheme.colorScheme.onSurfaceVariant
                    )

                    HorizontalDivider()

                    val removableCategories = activeCategories.filter { it.name != "Pigmy" }
                    if (removableCategories.isEmpty()) {
                        Text(
                            text = "No categories available to remove.",
                            style = MaterialTheme.typography.bodyMedium,
                            color = MaterialTheme.colorScheme.onSurfaceVariant
                        )
                    } else {
                        LazyColumn(
                            modifier = Modifier
                                .fillMaxWidth()
                                .heightIn(max = 300.dp),
                            verticalArrangement = Arrangement.spacedBy(6.dp)
                        ) {
                            items(removableCategories) { cat ->
                                val count = allExpenses.count { it.categoryId == cat.id || it.category.equals(cat.name, ignoreCase = true) }
                                Card(
                                    modifier = Modifier.fillMaxWidth(),
                                    colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surfaceVariant.copy(alpha = 0.4f)),
                                    shape = RoundedCornerShape(8.dp)
                                ) {
                                    Row(
                                        modifier = Modifier
                                            .fillMaxWidth()
                                            .padding(horizontal = 12.dp, vertical = 8.dp),
                                        verticalAlignment = Alignment.CenterVertically,
                                        horizontalArrangement = Arrangement.SpaceBetween
                                    ) {
                                        Column(modifier = Modifier.weight(1f)) {
                                            Text(
                                                text = cat.name,
                                                fontWeight = FontWeight.SemiBold,
                                                style = MaterialTheme.typography.bodyMedium
                                            )
                                            Text(
                                                text = if (count > 0) "$count transaction(s)" else "No transactions",
                                                style = MaterialTheme.typography.labelSmall,
                                                color = MaterialTheme.colorScheme.onSurfaceVariant
                                            )
                                        }

                                        IconButton(
                                            onClick = { categoryToDelete = cat },
                                            modifier = Modifier.size(36.dp)
                                        ) {
                                            Icon(
                                                Icons.Filled.DeleteOutline,
                                                contentDescription = "Remove ${cat.name}",
                                                tint = MaterialTheme.colorScheme.error
                                            )
                                        }
                                    }
                                }
                            }
                        }
                    }
                }
            },
            confirmButton = {
                Button(
                    onClick = {
                        showManageCategoriesDialog = false
```

```
                        categoryToDelete = null
                    }
                ) {
                    Text("Done")
                }
            }
        )
    }

    // Confirmation dialog for removing a specific category
    categoryToDelete?.let { cat ->
        val count = allExpenses.count { it.categoryId == cat.id || it.category.equals(cat.name, ignoreCase = true) }
        AlertDialog(
            onDismissRequest = { categoryToDelete = null },
            title = { Text("Remove Category?") },
            text = {
                Text(
                    "Are you sure you want to remove \"${cat.name}\"?\n\n" +
                    "It will no longer appear in category options for new transactions." +
                    if (count > 0) " Past records ($count transactions) will safely retain this name." else ""
                )
            },
            confirmButton = {
                Button(
                    onClick = {
                        viewModel.deleteCashCategory(cat.id) {
                            categoryToDelete = null
                        }
                    },
                    colors = ButtonDefaults.buttonColors(containerColor = MaterialTheme.colorScheme.error)
                ) {
                    Text("Remove", fontWeight = FontWeight.Bold)
                }
            },
            dismissButton = {
                OutlinedButton(onClick = { categoryToDelete = null }) {
                    Text("Cancel")
                }
            }
        )
    }

    // View Attachment Full Photo Dialog
    viewingPhotoFile?.let { file ->
        val bitmap = remember(file) {
            try {
                BitmapFactory.decodeFile(file.absolutePath)
            } catch (e: Exception) {
                null
            }
        }

        AlertDialog(
            onDismissRequest = { viewingPhotoFile = null },
            title = {
                Row(
                    modifier = Modifier.fillMaxWidth(),
                    horizontalArrangement = Arrangement.SpaceBetween,
                    verticalAlignment = Alignment.CenterVertically
                ) {
                    Text("Receipt Evidence Photo", fontWeight = FontWeight.Bold, fontSize = 16.sp)
                    IconButton(onClick = { viewingPhotoFile = null }) {
                        Icon(Icons.Filled.Close, contentDescription = "Close")
                    }
                }
            },
            text = {
                Column(
                    modifier = Modifier.fillMaxWidth(),
                    horizontalAlignment = Alignment.CenterHorizontally
                ) {
                    if (bitmap != null) {
                        Image(
                            bitmap = bitmap.asImageBitmap(),
                            contentDescription = "Receipt Photo",
                            contentScale = ContentScale.Fit,
                            modifier = Modifier
                                .fillMaxWidth()
                                .height(380.dp)
                                .clip(RoundedCornerShape(8.dp))
                        )
                    } else {
                        Text("Unable to render image file.", color = MaterialTheme.colorScheme.error)
                    }
                    Spacer(modifier = Modifier.height(6.dp))
                    Text(
                        text = "File: ${file.name} (${file.length() / 1024} KB)",
```

```
                        style = MaterialTheme.typography.labelSmall,
                        color = MaterialTheme.colorScheme.onSurfaceVariant
                    )
                }
            },
            confirmButton = {
                Button(onClick = { viewingPhotoFile = null }) {
                    Text("Close")
                }
            }
        )
    }

    // Void / Delete Entry Dialog
    selectedItemForVoid?.let { entry ->
        var voidReason by remember { mutableStateOf("Manual cancellation") }

        AlertDialog(
            onDismissRequest = { selectedItemForVoid = null },
            title = { Text("Cancel / Void Entry") },
            text = {
                Column(verticalArrangement = Arrangement.spacedBy(8.dp)) {
                    Text(
                        text = "Are you sure you want to void this entry?",
                        style = MaterialTheme.typography.bodyMedium
                    )
                    Text(
                        text = "${entry.category}: ${CostCalculator.formatPaiseToRupees(entry.amountPaise)} (${if (entry.isIncome) "Income" else "Expense"})",
                        fontWeight = FontWeight.Bold,
                        color = if (entry.isIncome) Color(0xFF2E7D32) else Color(0xFFD32F2F)
                    )
                    if (entry.isSavingsTransfer) {
                        Text(
                            text = "• Notice: This is a linked Pigmy savings transfer. Voiding here will maintain accounting integrity.",
                            style = MaterialTheme.typography.bodySmall,
                            color = Color(0xFFE65100)
                        )
                    }
                    if (!entry.attachmentUri.isNullOrBlank()) {
                        Text(
                            text = "• Attached photo evidence will be retained in history for audit purposes.",
                            style = MaterialTheme.typography.bodySmall,
                            color = MaterialTheme.colorScheme.onSurfaceVariant
                        )
                    }
                    OutlinedTextField(
                        value = voidReason,
                        onValueChange = { voidReason = it },
                        label = { Text("Reason for voiding") },
                        singleLine = true,
                        modifier = Modifier.fillMaxWidth()
                    )
                }
            },
            confirmButton = {
                Button(
                    onClick = {
                        viewModel.voidExpense(entry.id, voidReason.trim()) {
                            selectedItemForVoid = null
                        }
                    },
                    colors = ButtonDefaults.buttonColors(containerColor = MaterialTheme.colorScheme.error)
                ) {
                    Text("Void Entry")
                }
            },
            dismissButton = {
                OutlinedButton(onClick = { selectedItemForVoid = null }) {
                    Text("Keep Entry")
                }
            }
        )
    }

    // Excel Import Confirmation / Error Dialog
    importPreview?.let { preview ->
        if (preview.errorMessage != null) {
            AlertDialog(
                onDismissRequest = { importPreview = null },
                title = {
                    Text(
                        text = "Excel Import Notice",
                        fontWeight = FontWeight.Bold,
                        color = MaterialTheme.colorScheme.error
                    )
```

```
                },
                text = { Text(preview.errorMessage) },
                confirmButton = {
                    Button(onClick = { importPreview = null }) {
                        Text("OK")
                    }
                }
            )
        } else {
            AlertDialog(
                onDismissRequest = { importPreview = null },
                title = {
                    Text(
                        text = "Import Cash Entries",
                        fontWeight = FontWeight.Bold,
                        color = MaterialTheme.colorScheme.primary
                    )
                },
                text = {
                    Column(verticalArrangement = Arrangement.spacedBy(8.dp)) {
                        Text("File: ${preview.fileName}", fontWeight = FontWeight.SemiBold, fontSize =
13.sp)
                        Text("Total entries found: ${preview.items.size}", fontWeight = FontWeight.Bold)
                        HorizontalDivider(modifier = Modifier.padding(vertical = 4.dp))
                        Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement =
Arrangement.SpaceBetween) {
                            Text("Income (${preview.incomeCount}):", color = Color(0xFF2E7D32), fontWeight =
 FontWeight.SemiBold)
                            Text(CostCalculator.formatPaiseToRupees(preview.totalIncomePaise), color =
Color(0xFF2E7D32), fontWeight = FontWeight.Bold)
                        }
                        Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement =
Arrangement.SpaceBetween) {
                            Text("Expense (${preview.expenseCount}):", color = Color(0xFFD32F2F), fontWeight
 = FontWeight.SemiBold)
                            Text(CostCalculator.formatPaiseToRupees(preview.totalExpensePaise), color =
Color(0xFFD32F2F), fontWeight = FontWeight.Bold)
                        }
                        if (preview.skippedRows > 0) {
                            Text(
                                "Skipped ${preview.skippedRows} empty / zero-value rows",
                                style = MaterialTheme.typography.labelSmall,
                                color = MaterialTheme.colorScheme.onSurfaceVariant
                            )
                        }
                        Spacer(modifier = Modifier.height(4.dp))
                        Text(
                            "These entries will be recorded in your Cash In & Out register with their
respective categories.",
                            style = MaterialTheme.typography.bodySmall,
                            color = MaterialTheme.colorScheme.onSurfaceVariant
                        )
                    }
                },
                confirmButton = {
                    Button(
                        onClick = {
                            viewModel.confirmCashTrackerImport(preview.items) {
                                val count = preview.items.size
                                importPreview = null
                                Toast.makeText(context, "Imported $count entries successfully",
Toast.LENGTH_SHORT).show()
                            }
                        },
                        colors = ButtonDefaults.buttonColors(containerColor =
MaterialTheme.colorScheme.primary)
                    ) {
                        Text("Import (${preview.items.size})", fontWeight = FontWeight.Bold)
                    }
                },
                dismissButton = {
                    OutlinedButton(onClick = { importPreview = null }) {
                        Text("Cancel")
                    }
                }
            )
        }
    }
}

private fun queryFileName(context: Context, uri: Uri): String {
    var name = "cash_book.xlsx"
    if (uri.scheme == "content") {
        try {
            val cursor = context.contentResolver.query(uri, null, null, null, null)
            cursor?.use {
                if (it.moveToFirst()) {
                    val idx = it.getColumnIndex(OpenableColumns.DISPLAY_NAME)
```

```
                    if (idx >= 0) {
                        name = it.getString(idx)
                    }
                }
            }
        } catch (ignored: Exception) {}
    } else if (uri.path != null) {
        name = java.io.File(uri.path!!).name
    }
    return name
}


@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun CashTrackerExportPreviewDialog(
    allExpenses: List<ShopExpenseEntity>,
    activeCategories: List<CashCategoryEntity>,
    onDismiss: () -> Unit,
    onExportAndShare: (
        items: List<ShopExpenseEntity>,
        periodTitle: String,
        fromDate: Long?,
        toDate: Long?,
        summary: String,
        isShare: Boolean
    ) -> Unit
) {
    val context = LocalContext.current
    var selectedPeriod by remember { mutableStateOf("MONTH") } // TODAY, WEEK, MONTH, ALL, CUSTOM
    var selectedDirection by remember { mutableStateOf("ALL") } // ALL, IN, OUT
    var selectedCategories by remember { mutableStateOf<Set<String>>(emptySet()) } // empty = All

    val now = System.currentTimeMillis()
    var fromTimestamp by remember {
        val cal = Calendar.getInstance().apply {
            set(Calendar.DAY_OF_MONTH, 1)
            set(Calendar.HOUR_OF_DAY, 0)
            set(Calendar.MINUTE, 0)
            set(Calendar.SECOND, 0)
            set(Calendar.MILLISECOND, 0)
        }
        mutableLongStateOf(cal.timeInMillis)
    }
    var toTimestamp by remember {
        val cal = Calendar.getInstance().apply {
            set(Calendar.DAY_OF_MONTH, getActualMaximum(Calendar.DAY_OF_MONTH))
            set(Calendar.HOUR_OF_DAY, 23)
            set(Calendar.MINUTE, 59)
            set(Calendar.SECOND, 59)
            set(Calendar.MILLISECOND, 999)
        }
        mutableLongStateOf(cal.timeInMillis)
    }

    val (effectiveStart, effectiveEnd, rangeTitle) = remember(selectedPeriod, fromTimestamp, toTimestamp) {
        val cal = Calendar.getInstance()
        val fmt = SimpleDateFormat("dd MMM yyyy", Locale.US)
        when (selectedPeriod) {
            "TODAY" -> {
                cal.set(Calendar.HOUR_OF_DAY, 0); cal.set(Calendar.MINUTE, 0); cal.set(Calendar.SECOND, 0);
cal.set(Calendar.MILLISECOND, 0)
                val s = cal.timeInMillis
                cal.set(Calendar.HOUR_OF_DAY, 23); cal.set(Calendar.MINUTE, 59); cal.set(Calendar.SECOND,
59); cal.set(Calendar.MILLISECOND, 999)
                val e = cal.timeInMillis
                Triple(s, e, "Today (${fmt.format(Date(s))})")
            }
            "WEEK" -> {
                cal.firstDayOfWeek = Calendar.MONDAY
                cal.set(Calendar.DAY_OF_WEEK, Calendar.MONDAY)
                cal.set(Calendar.HOUR_OF_DAY, 0); cal.set(Calendar.MINUTE, 0); cal.set(Calendar.SECOND, 0);
cal.set(Calendar.MILLISECOND, 0)
                val s = cal.timeInMillis
                cal.add(Calendar.DAY_OF_WEEK, 6)
                cal.set(Calendar.HOUR_OF_DAY, 23); cal.set(Calendar.MINUTE, 59); cal.set(Calendar.SECOND,
59); cal.set(Calendar.MILLISECOND, 999)
                val e = cal.timeInMillis
                Triple(s, e, "This Week (${fmt.format(Date(s))} to ${fmt.format(Date(e))})")
            }
            "MONTH" -> {
                cal.set(Calendar.DAY_OF_MONTH, 1)
                cal.set(Calendar.HOUR_OF_DAY, 0); cal.set(Calendar.MINUTE, 0); cal.set(Calendar.SECOND, 0);
cal.set(Calendar.MILLISECOND, 0)
                val s = cal.timeInMillis
                cal.set(Calendar.DAY_OF_MONTH, cal.getActualMaximum(Calendar.DAY_OF_MONTH))
                cal.set(Calendar.HOUR_OF_DAY, 23); cal.set(Calendar.MINUTE, 59); cal.set(Calendar.SECOND,
59); cal.set(Calendar.MILLISECOND, 999)
```

```
                val e = cal.timeInMillis
                Triple(s, e, "This Month (${fmt.format(Date(s))} to ${fmt.format(Date(e))})")
            }
            "ALL" -> {
                Triple(0L, Long.MAX_VALUE, "All Time")
            }
            else -> { // CUSTOM
                Triple(fromTimestamp, toTimestamp, "${fmt.format(Date(fromTimestamp))} to
${fmt.format(Date(toTimestamp))}")
            }
        }
    }

    val matchingItems = remember(allExpenses, effectiveStart, effectiveEnd, selectedDirection,
selectedCategories) {
        allExpenses.filter { item ->
            val inDate = item.timestamp in effectiveStart..effectiveEnd
            val matchesDir = when (selectedDirection) {
                "IN" -> item.isIncome
                "OUT" -> !item.isIncome
                else -> true
            }
            val matchesCat = if (selectedCategories.isEmpty()) true else {
                selectedCategories.contains(item.category)
            }
            inDate && matchesDir && matchesCat
        }
    }

    val inflowPaise = remember(matchingItems) { matchingItems.filter { it.isIncome }.sumOf { it.amountPaise
} }
    val outflowPaise = remember(matchingItems) { matchingItems.filter { !it.isIncome }.sumOf {
it.amountPaise } }
    val netMovementPaise = inflowPaise - outflowPaise

    val filtersSummary = buildString {
        append("Period: $rangeTitle | Direction: $selectedDirection")
        if (selectedCategories.isEmpty()) append(" | Categories: All")
        else append(" | Categories: ${selectedCategories.joinToString(", ")}")
    }

    AlertDialog(
        onDismissRequest = onDismiss,
        title = {
            Row(verticalAlignment = Alignment.CenterVertically) {
                Icon(Icons.Filled.Share, contentDescription = null, tint =
MaterialTheme.colorScheme.primary)
                Spacer(modifier = Modifier.width(8.dp))
                Text("Export & Share Cash Book", fontWeight = FontWeight.Bold)
            }
        },
        text = {
            LazyColumn(
                modifier = Modifier.fillMaxWidth(),
                verticalArrangement = Arrangement.spacedBy(10.dp)
            ) {
                item {
                    Text("Quick Periods", style = MaterialTheme.typography.labelMedium.copy(fontWeight =
FontWeight.SemiBold))
                    Row(
                        modifier = Modifier.fillMaxWidth().horizontalScroll(rememberScrollState()),
                        horizontalArrangement = Arrangement.spacedBy(6.dp)
                    ) {
                        listOf("TODAY" to "Today", "WEEK" to "This Week", "MONTH" to "This Month", "ALL" to
"All Time", "CUSTOM" to "Custom").forEach { (code, label) ->
                            FilterChip(
                                selected = selectedPeriod == code,
                                onClick = { selectedPeriod = code },
                                label = { Text(label, fontSize = 11.5.sp) }
                            )
                        }
                    }
                }

                if (selectedPeriod == "CUSTOM") {
                    item {
                        val fmt = SimpleDateFormat("dd MMM yyyy", Locale.US)
                        Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement =
Arrangement.spacedBy(8.dp)) {
                            OutlinedButton(
                                onClick = {
                                    val c = Calendar.getInstance().apply { timeInMillis = fromTimestamp }
                                    DatePickerDialog(context, { _, y, m, d ->
                                        val cal = Calendar.getInstance().apply {
                                            set(y, m, d, 0, 0, 0)
                                            set(Calendar.MILLISECOND, 0)
                                        }
                                        fromTimestamp = cal.timeInMillis
```

```
                                    }, c.get(Calendar.YEAR), c.get(Calendar.MONTH),
    c.get(Calendar.DAY_OF_MONTH)).show()
                                },
                                shape = RoundedCornerShape(8.dp),
                                modifier = Modifier.weight(1f)
                            ) {
                                Text("From: ${fmt.format(Date(fromTimestamp))}", fontSize = 11.sp, maxLines
    = 1)
                            }

                            OutlinedButton(
                                onClick = {
                                    val c = Calendar.getInstance().apply { timeInMillis = toTimestamp }
                                    DatePickerDialog(context, { _, y, m, d ->
                                        val cal = Calendar.getInstance().apply {
                                            set(y, m, d, 23, 59, 59)
                                            set(Calendar.MILLISECOND, 999)
                                        }
                                        toTimestamp = cal.timeInMillis
                                    }, c.get(Calendar.YEAR), c.get(Calendar.MONTH),
    c.get(Calendar.DAY_OF_MONTH)).show()
                                },
                                shape = RoundedCornerShape(8.dp),
                                modifier = Modifier.weight(1f)
                            ) {
                                Text("To: ${fmt.format(Date(toTimestamp))}", fontSize = 11.sp, maxLines = 1)
                            }
                        }
                    }
                }

                item {
                    Text("Direction", style = MaterialTheme.typography.labelMedium.copy(fontWeight =
    FontWeight.SemiBold))
                    Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement =
    Arrangement.spacedBy(8.dp)) {
                        FilterChip(
                            selected = selectedDirection == "ALL",
                            onClick = { selectedDirection = "ALL" },
                            label = { Text("All") },
                            modifier = Modifier.weight(1f)
                        )
                        FilterChip(
                            selected = selectedDirection == "IN",
                            onClick = { selectedDirection = "IN" },
                            label = { Text("Cash In") },
                            modifier = Modifier.weight(1f)
                        )
                        FilterChip(
                            selected = selectedDirection == "OUT",
                            onClick = { selectedDirection = "OUT" },
                            label = { Text("Cash Out") },
                            modifier = Modifier.weight(1f)
                        )
                    }
                }

                item {
                    Text("Categories Filter", style = MaterialTheme.typography.labelMedium.copy(fontWeight =
     FontWeight.SemiBold))
                    Row(
                        modifier = Modifier.fillMaxWidth().horizontalScroll(rememberScrollState()),
                        horizontalArrangement = Arrangement.spacedBy(6.dp)
                    ) {
                        FilterChip(
                            selected = selectedCategories.isEmpty(),
                            onClick = { selectedCategories = emptySet() },
                            label = { Text("All Categories", fontSize = 11.5.sp) }
                        )
                        activeCategories.forEach { cat ->
                            val isSel = selectedCategories.contains(cat.name)
                            FilterChip(
                                selected = isSel,
                                onClick = {
                                    selectedCategories = if (isSel) selectedCategories - cat.name else
    selectedCategories + cat.name
                                },
                                label = { Text(cat.name, fontSize = 11.5.sp) }
                            )
                        }
                    }
                }

                item {
                    // Preview Card
                    Card(
                        colors = CardDefaults.cardColors(containerColor =
    MaterialTheme.colorScheme.surfaceVariant.copy(alpha = 0.5f)),
```

```
                        shape = RoundedCornerShape(10.dp),
                        modifier = Modifier.fillMaxWidth()
                    ) {
                        Column(modifier = Modifier.padding(12.dp), verticalArrangement =
Arrangement.spacedBy(4.dp)) {
                            Text("Export Preview", fontWeight = FontWeight.Bold, style =
MaterialTheme.typography.labelMedium, color = MaterialTheme.colorScheme.primary)
                            Text("Range: $rangeTitle", style = MaterialTheme.typography.bodySmall)
                            Text("Matching Active Records: ${matchingItems.size}", fontWeight =
FontWeight.SemiBold, style = MaterialTheme.typography.bodyMedium)
                            Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement =
Arrangement.SpaceBetween) {
                                Text("Inflow Total:", style = MaterialTheme.typography.bodySmall)
                                Text(CostCalculator.formatPaiseToRupees(inflowPaise), style =
MaterialTheme.typography.bodySmall.copy(fontWeight = FontWeight.Bold), color = Color(0xFF2E7D32))
                            }
                            Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement =
Arrangement.SpaceBetween) {
                                Text("Outflow Total:", style = MaterialTheme.typography.bodySmall)
                                Text(CostCalculator.formatPaiseToRupees(outflowPaise), style =
MaterialTheme.typography.bodySmall.copy(fontWeight = FontWeight.Bold), color = Color(0xFFD32F2F))
                            }
                            HorizontalDivider(modifier = Modifier.padding(vertical = 2.dp))
                            Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement =
Arrangement.SpaceBetween) {
                                Text("Selected Net Movement:", fontWeight = FontWeight.Bold, style =
MaterialTheme.typography.bodyMedium)
                                Text(
                                    CostCalculator.formatPaiseToRupees(netMovementPaise),
                                    fontWeight = FontWeight.Bold,
                                    style = MaterialTheme.typography.bodyMedium,
                                    color = if (netMovementPaise >= 0) Color(0xFF2E7D32) else
Color(0xFFD32F2F)
                                )
                            }
                        }
                    }
                }
            },
            confirmButton = {
                Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
                    Button(
                        onClick = {
                            val fmt = SimpleDateFormat("dd MMM yyyy", Locale.US)
                            val periodTitle = when (selectedPeriod) {
                                "TODAY" -> "Today"
                                "WEEK" -> "This Week"
                                "MONTH" -> "This Month"
                                "ALL" -> "All Time"
                                else -> "${fmt.format(Date(effectiveStart))} to
${fmt.format(Date(effectiveEnd))}"
                            }
                            onExportAndShare(
                                matchingItems,
                                periodTitle,
                                if (selectedPeriod == "ALL") null else effectiveStart,
                                if (selectedPeriod == "ALL") null else effectiveEnd,
                                filtersSummary,
                                true
                            )
                        },
                        colors = ButtonDefaults.buttonColors(containerColor = MaterialTheme.colorScheme.primary)
                    ) {
                        Icon(Icons.Filled.Share, contentDescription = null, modifier = Modifier.size(16.dp))
                        Spacer(modifier = Modifier.width(4.dp))
                        Text("Share Excel", fontWeight = FontWeight.Bold)
                    }
                }
            },
            dismissButton = {
                OutlinedButton(onClick = onDismiss) {
                    Text("Cancel")
                }
            }
        )
}
```

app\src\main\java\com\alieducational\app\ui\reports\ProfitScreen.kt

```
package com.alieducational.app.ui.reports

import android.app.DatePickerDialog
import androidx.activity.compose.BackHandler
import androidx.compose.foundation.Canvas
import androidx.compose.foundation.background
import androidx.compose.foundation.clickable
import androidx.compose.foundation.gestures.detectTapGestures
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.LazyRow
import androidx.compose.foundation.lazy.items
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.ArrowBack
import androidx.compose.material.icons.filled.CalendarMonth
import androidx.compose.material.icons.filled.CheckCircle
import androidx.compose.material.icons.filled.Info
import androidx.compose.material.icons.filled.TrendingDown
import androidx.compose.material.icons.filled.TrendingUp
import androidx.compose.material.icons.filled.Warning
import androidx.compose.material3.AlertDialog
import androidx.compose.material3.Button
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.FilterChip
import androidx.compose.material3.HorizontalDivider
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedButton
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Surface
import androidx.compose.material3.Tab
import androidx.compose.material3.TabRow
import androidx.compose.material3.Text
import androidx.compose.material3.TopAppBar
import androidx.compose.material3.TopAppBarDefaults
import androidx.compose.runtime.Composable
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.geometry.Offset
import androidx.compose.ui.geometry.Size
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.nativeCanvas
import androidx.compose.ui.input.pointer.pointerInput
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.alieducational.app.data.local.entity.ShopExpenseEntity
import com.alieducational.app.domain.calculation.CostCalculator
import com.alieducational.app.ui.AliViewModel
import java.text.SimpleDateFormat
import java.util.Calendar
import java.util.Date
import java.util.Locale

data class ProfitTrendBucket(
    val label: String,
    val startTime: Long,
    val endTime: Long,
    val salesPaise: Long,
    val cogsPaise: Long,
    val grossProfitPaise: Long
)
```

```
 83
 84 @OptIn(ExperimentalMaterial3Api::class)
 85 @Composable
 86 fun ProfitScreen(
 87     viewModel: AliViewModel,
 88     onNavigateBack: () -> Unit,
 89     modifier: Modifier = Modifier
 90 ) {
 91     BackHandler {
 92         onNavigateBack()
 93     }
 94
 95     val context = LocalContext.current
 96     val allProducts by viewModel.allProducts.collectAsState()
 97     val allSales by viewModel.allSales.collectAsState()
 98     val allSaleLines by viewModel.allSaleLines.collectAsState()
 99     val allExpenses by viewModel.allExpenses.collectAsState()
100     val deviceRole by viewModel.deviceRole.collectAsState()
101
102     val isOwner = deviceRole == "OWNER"
103     var activeTab by remember { mutableStateOf("PRODUCTS") } // PRODUCTS, SERVICES, COMBINED
104     var selectedPeriod by remember { mutableStateOf("All Time") } // Today, This Week, This Month, All Time,
    Custom
105
106     // Custom date bounds
107     var customStartDate by remember { mutableStateOf<Long?>(null) }
108     var customEndDate by remember { mutableStateOf<Long?>(null) }
109
110     // Calculate time boundaries
111     val now = System.currentTimeMillis()
112     val calendar = Calendar.getInstance()
113     val (startTime, endTime) = when (selectedPeriod) {
114         "Today" -> {
115             calendar.timeInMillis = now
116             calendar.set(Calendar.HOUR_OF_DAY, 0)
117             calendar.set(Calendar.MINUTE, 0)
118             calendar.set(Calendar.SECOND, 0)
119             calendar.set(Calendar.MILLISECOND, 0)
120             val s = calendar.timeInMillis
121             calendar.set(Calendar.HOUR_OF_DAY, 23)
122             calendar.set(Calendar.MINUTE, 59)
123             calendar.set(Calendar.SECOND, 59)
124             calendar.set(Calendar.MILLISECOND, 999)
125             Pair(s, calendar.timeInMillis)
126         }
127         "This Week" -> {
128             calendar.timeInMillis = now
129             calendar.set(Calendar.DAY_OF_WEEK, calendar.firstDayOfWeek)
130             calendar.set(Calendar.HOUR_OF_DAY, 0)
131             calendar.set(Calendar.MINUTE, 0)
132             calendar.set(Calendar.SECOND, 0)
133             calendar.set(Calendar.MILLISECOND, 0)
134             Pair(calendar.timeInMillis, now)
135         }
136         "This Month" -> {
137             calendar.timeInMillis = now
138             calendar.set(Calendar.DAY_OF_MONTH, 1)
139             calendar.set(Calendar.HOUR_OF_DAY, 0)
140             calendar.set(Calendar.MINUTE, 0)
141             calendar.set(Calendar.SECOND, 0)
142             calendar.set(Calendar.MILLISECOND, 0)
143             Pair(calendar.timeInMillis, now)
144         }
145         "Custom" -> {
146             val s = customStartDate ?: 0L
147             val e = customEndDate ?: Long.MAX_VALUE
148             Pair(s, e)
149         }
150         else -> Pair(0L, Long.MAX_VALUE) // All Time
151     }
152
153     val displayDateRangeText = remember(selectedPeriod, startTime, endTime) {
154         val df = SimpleDateFormat("dd MMM yyyy", Locale.US)
155         when (selectedPeriod) {
156             "Today" -> df.format(Date())
157             "This Week", "This Month", "Custom" -> {
158                 if (startTime == 0L && endTime == Long.MAX_VALUE) "All Time"
159                 else "${df.format(Date(startTime))} - ${df.format(Date(minOf(endTime, now)))}"
160             }
161             else -> "All recorded history"
162         }
163     }
164
165     // ── FINANCIAL AGGREGATIONS ──────────────────────────────
166     // 1. Sales & COGS in period (non-voided)
167     val periodSales = remember(allSales, startTime, endTime) {
168         allSales.filter { !it.isVoided && it.timestamp in startTime..endTime }
169     }
```

```
    val periodSaleIds = remember(periodSales) { periodSales.map { it.id }.toSet() }
    val periodSaleLines = remember(allSaleLines, periodSaleIds) {
        allSaleLines.filter { it.saleId in periodSaleIds }
    }

    // Products (retail) lines
    val periodProductLines = remember(periodSaleLines) {
        periodSaleLines.filter { it.lineType != "SERVICE" }
    }
    val productRevenuePaise = remember(periodProductLines) {
        periodProductLines.sumOf { it.lineNetPaise }
    }
    val productCogsPaise = remember(periodProductLines) {
        periodProductLines.sumOf { line ->
            line.quantityBase.toLong() * (line.unitCostSnapshotPaise ?: 0L)
        }
    }
    val productGrossProfitPaise = productRevenuePaise - productCogsPaise
    val productGrossMarginPct = if (productRevenuePaise > 0) {
        (productGrossProfitPaise.toDouble() / productRevenuePaise.toDouble()) * 100.0
    } else 0.0

    // Services lines
    val periodServiceLines = remember(periodSaleLines) {
        periodSaleLines.filter { it.lineType == "SERVICE" }
    }
    val serviceRevenuePaise = remember(periodServiceLines) {
        periodServiceLines.sumOf { it.lineNetPaise }
    }

    // 2. Canonical Expenses & Refunds in period
    // Non-voided, non-savings-transfer expenses
    val periodActiveExpenses = remember(allExpenses, startTime, endTime) {
        allExpenses.filter { !it.isVoided && !it.isSavingsTransfer && it.timestamp in startTime..endTime }
    }

    // Product expenses & refunds
    val productExpenses = remember(periodActiveExpenses) {
        periodActiveExpenses.filter { !it.isIncome && it.classification == "PRODUCTS" }
    }
    val grossProductExpensesPaise = remember(productExpenses) { productExpenses.sumOf { it.amountPaise } }
    val productRefunds = remember(periodActiveExpenses) {
        periodActiveExpenses.filter { it.isIncome && it.isExpenseRefund && it.classification == "PRODUCTS" }
    }
    val productRefundsPaise = remember(productRefunds) { productRefunds.sumOf { it.amountPaise } }
    val netProductExpensesPaise = grossProductExpensesPaise - productRefundsPaise
    val productResultAfterExpensesPaise = productGrossProfitPaise - netProductExpensesPaise

    // Service expenses & refunds
    val serviceExpenses = remember(periodActiveExpenses) {
        periodActiveExpenses.filter { !it.isIncome && (it.classification == "SERVICES" || it.category ==
"SERVICE_EXPENSE" || it.serviceTarget != null) }
    }
    val grossServiceExpensesPaise = remember(serviceExpenses) { serviceExpenses.sumOf { it.amountPaise } }
    val serviceRefunds = remember(periodActiveExpenses) {
        periodActiveExpenses.filter { it.isIncome && it.isExpenseRefund && (it.classification == "SERVICES"
|| it.category == "SERVICE_EXPENSE" || it.serviceTarget != null) }
    }
    val serviceRefundsPaise = remember(serviceRefunds) { serviceRefunds.sumOf { it.amountPaise } }
    val netServiceExpensesPaise = grossServiceExpensesPaise - serviceRefundsPaise
    val serviceResultAfterExpensesPaise = serviceRevenuePaise - netServiceExpensesPaise

    // Shared expenses & refunds
    val sharedExpenses = remember(periodActiveExpenses) {
        periodActiveExpenses.filter {
            !it.isIncome && it.classification != "PRODUCTS" && it.classification != "SERVICES" &&
                    it.category != "SERVICE_EXPENSE" && it.serviceTarget == null
        }
    }
    val grossSharedExpensesPaise = remember(sharedExpenses) { sharedExpenses.sumOf { it.amountPaise } }

    val sharedRefunds = remember(periodActiveExpenses) {
        periodActiveExpenses.filter {
            it.isIncome && it.isExpenseRefund && it.classification != "PRODUCTS" && it.classification !=
"SERVICES" &&
                    it.category != "SERVICE_EXPENSE" && it.serviceTarget == null
        }
    }
    val sharedRefundsPaise = remember(sharedRefunds) { sharedRefunds.sumOf { it.amountPaise } }
    val netSharedExpensesPaise = grossSharedExpensesPaise - sharedRefundsPaise

    // Combined Net Operating Result:
    // Product Gross Profit + Net Service Revenue - (Net Product Expenses + Net Service Expenses + Net
Shared Expenses)
    val totalNetOperatingExpensesPaise = netProductExpensesPaise + netServiceExpensesPaise +
netSharedExpensesPaise
    val combinedRecordedOperatingResultPaise = productGrossProfitPaise + serviceRevenuePaise -
totalNetOperatingExpensesPaise

```

```
    // Potential inventory gross profit from current unsold stock
    val expectedGrossProfitPaise = remember(allProducts) {
        allProducts.sumOf { p ->
            val potentialRevenue = p.currentSellableStock.toLong() * p.sellingPricePiecePaise
            val costValue = p.currentSellableStock.toLong() * p.movingAvgCostPaise
            potentialRevenue - costValue
        }
    }

    // ── BUCKETED TREND DATA (Section 6 Graph A) ──────────────────────
    val trendBuckets = remember(periodSales, periodSaleLines, startTime, endTime, selectedPeriod) {
        generateTrendBuckets(periodSales, periodSaleLines, startTime, endTime, selectedPeriod)
    }

    var selectedBucketDetail by remember { mutableStateOf<ProfitTrendBucket?>(null) }

    Scaffold(
        topBar = {
            TopAppBar(
                title = {
                    Column {
                        Text("Sales & Profit", fontWeight = FontWeight.Bold, fontSize = 18.sp)
                        Text(
                            displayDateRangeText,
                            style = MaterialTheme.typography.bodySmall,
                            color = MaterialTheme.colorScheme.onSurfaceVariant
                        )
                    }
                },
                navigationIcon = {
                    IconButton(onClick = onNavigateBack) {
                        Icon(Icons.Filled.ArrowBack, contentDescription = "Back")
                    }
                },
                colors = TopAppBarDefaults.topAppBarColors(
                    containerColor = MaterialTheme.colorScheme.surface,
                    titleContentColor = MaterialTheme.colorScheme.onSurface
                )
            )
        }
    ) { padding ->
        Surface(
            modifier = modifier
                .fillMaxSize()
                .padding(padding),
            color = MaterialTheme.colorScheme.background
        ) {
            LazyColumn(
                modifier = Modifier
                    .fillMaxSize()
                    .padding(horizontal = 16.dp),
                verticalArrangement = Arrangement.spacedBy(14.dp)
            ) {
                // Period Filter Chips Row
                item {
                    Spacer(modifier = Modifier.height(4.dp))
                    LazyRow(
                        horizontalArrangement = Arrangement.spacedBy(8.dp),
                        modifier = Modifier.fillMaxWidth()
                    ) {
                        val periods = listOf("Today", "This Week", "This Month", "All Time", "Custom")
                        items(periods) { period ->
                            FilterChip(
                                selected = selectedPeriod == period,
                                onClick = {
                                    selectedPeriod = period
                                    if (period == "Custom" && customStartDate == null) {
                                        val c = Calendar.getInstance()
                                        DatePickerDialog(
                                            context,
                                            { _, y, m, d ->
                                                val startCal = Calendar.getInstance().apply {
                                                    set(y, m, d, 0, 0, 0)
                                                    set(Calendar.MILLISECOND, 0)
                                                }
                                                customStartDate = startCal.timeInMillis

                                                DatePickerDialog(
                                                    context,
                                                    { _, ey, em, ed ->
                                                        val endCal = Calendar.getInstance().apply {
                                                            set(ey, em, ed, 23, 59, 59)
                                                            set(Calendar.MILLISECOND, 999)
                                                        }
                                                        customEndDate = endCal.timeInMillis
                                                    },
                                                    y, m, d
                                                ).show()
```

```
                                        },
                                        c.get(Calendar.YEAR),
                                        c.get(Calendar.MONTH),
                                        c.get(Calendar.DAY_OF_MONTH)
                                    ).show()
                                }
                            },
                            label = { Text(period, fontWeight = if (selectedPeriod == period)
FontWeight.Bold else FontWeight.Normal) }
                        )
                    }
                }
            }

            // Tab Switcher (Products, Services, Combined)
            if (isOwner) {
                item {
                    TabRow(
                        selectedTabIndex = when (activeTab) {
                            "PRODUCTS" -> 0
                            "SERVICES" -> 1
                            else -> 2
                        },
                        containerColor = MaterialTheme.colorScheme.surface,
                        contentColor = MaterialTheme.colorScheme.primary
                    ) {
                        Tab(
                            selected = activeTab == "PRODUCTS",
                            onClick = { activeTab = "PRODUCTS" },
                            text = { Text("Products", fontWeight = FontWeight.Bold, fontSize = 13.sp) }
                        )
                        Tab(
                            selected = activeTab == "SERVICES",
                            onClick = { activeTab = "SERVICES" },
                            text = { Text("Services", fontWeight = FontWeight.Bold, fontSize = 13.sp) }
                        )
                        Tab(
                            selected = activeTab == "COMBINED",
                            onClick = { activeTab = "COMBINED" },
                            text = { Text("Combined", fontWeight = FontWeight.Bold, fontSize = 13.sp) }
                        )
                    }
                }
            }

            // ── TAB CONTENT: PRODUCTS ──────────────────────────────────
            if (activeTab == "PRODUCTS" || !isOwner) {
                // 1. Prominent Gross Profit Card (with Gross Margin %)
                item {
                    val isGrossProfitPositive = productGrossProfitPaise >= 0
                    Card(
                        modifier = Modifier.fillMaxWidth(),
                        colors = CardDefaults.cardColors(
                            containerColor = if (isGrossProfitPositive) Color(0xFF1B3828) else
Color(0xFF3E1B1B)
                        ),
                        shape = RoundedCornerShape(16.dp)
                    ) {
                        Column(modifier = Modifier.padding(18.dp)) {
                            Row(
                                modifier = Modifier.fillMaxWidth(),
                                horizontalArrangement = Arrangement.SpaceBetween,
                                verticalAlignment = Alignment.CenterVertically
                            ) {
                                Row(verticalAlignment = Alignment.CenterVertically) {
                                    Icon(
                                        imageVector = if (isGrossProfitPositive) Icons.Filled.TrendingUp
 else Icons.Filled.TrendingDown,
                                        contentDescription = null,
                                        tint = if (isGrossProfitPositive) Color(0xFF75B890) else
Color(0xFFE57373),
                                        modifier = Modifier.size(24.dp)
                                    )
                                    Spacer(modifier = Modifier.width(8.dp))
                                    Text(
                                        text = "Product Gross Profit",
                                        style = MaterialTheme.typography.titleMedium.copy(fontWeight =
FontWeight.Bold),
                                        color = MaterialTheme.colorScheme.onSurface
                                    )
                                }
                                Text(
                                    text = if (productRevenuePaise > 0) {
                                        "Margin: ${String.format(Locale.US, "%.2f",
productGrossMarginPct)}%"
                                    } else "Margin: Not applicable",
                                    style = MaterialTheme.typography.labelLarge.copy(fontWeight =
FontWeight.Bold),
```

```
                                        color = if (isGrossProfitPositive) Color(0xFF75B890) else
Color(0xFFE57373)
                                    )
                                }

                                Spacer(modifier = Modifier.height(10.dp))
                                Text(
                                    text = CostCalculator.formatPaiseToRupees(productGrossProfitPaise),
                                    style = MaterialTheme.typography.headlineLarge.copy(fontWeight =
FontWeight.Bold),
                                    color = if (isGrossProfitPositive) Color(0xFF75B890) else
Color(0xFFE57373)
                                )

                                Spacer(modifier = Modifier.height(6.dp))
                                Text(
                                    text = "Net Sales
${CostCalculator.formatPaiseToRupees(productRevenuePaise)} minus COGS
${CostCalculator.formatPaiseToRupees(productCogsPaise)}",
                                    style = MaterialTheme.typography.bodySmall,
                                    color = MaterialTheme.colorScheme.onSurfaceVariant
                                )
                            }
                        }
                    }

                    // 2. Sales and COGS 2-Column Cards
                    item {
                        Row(
                            modifier = Modifier.fillMaxWidth(),
                            horizontalArrangement = Arrangement.spacedBy(10.dp)
                        ) {
                            Card(
                                modifier = Modifier.weight(1f),
                                colors = CardDefaults.cardColors(containerColor =
MaterialTheme.colorScheme.surface),
                                shape = RoundedCornerShape(12.dp)
                            ) {
                                Column(modifier = Modifier.padding(14.dp)) {
                                    Text("Product Sales", style = MaterialTheme.typography.bodySmall, color
= MaterialTheme.colorScheme.onSurfaceVariant)
                                    Spacer(modifier = Modifier.height(4.dp))
                                    Text(
                                        CostCalculator.formatPaiseToRupees(productRevenuePaise),
                                        style = MaterialTheme.typography.titleMedium.copy(fontWeight =
FontWeight.Bold),
                                        color = MaterialTheme.colorScheme.onSurface
                                    )
                                }
                            }

                            Card(
                                modifier = Modifier.weight(1f),
                                colors = CardDefaults.cardColors(containerColor =
MaterialTheme.colorScheme.surface),
                                shape = RoundedCornerShape(12.dp)
                            ) {
                                Column(modifier = Modifier.padding(14.dp)) {
                                    Text("Cost of Goods Sold", style = MaterialTheme.typography.bodySmall,
color = MaterialTheme.colorScheme.onSurfaceVariant)
                                    Spacer(modifier = Modifier.height(4.dp))
                                    Text(
                                        CostCalculator.formatPaiseToRupees(productCogsPaise),
                                        style = MaterialTheme.typography.titleMedium.copy(fontWeight =
FontWeight.Bold),
                                        color = MaterialTheme.colorScheme.onSurface
                                    )
                                }
                            }
                        }
                    }

                    // 3. Product Expenses and Result after Product Expenses
                    item {
                        Card(
                            modifier = Modifier.fillMaxWidth(),
                            colors = CardDefaults.cardColors(containerColor =
MaterialTheme.colorScheme.surface),
                            shape = RoundedCornerShape(12.dp)
                        ) {
                            Column(modifier = Modifier.padding(14.dp)) {
                                Row(
                                    modifier = Modifier.fillMaxWidth(),
                                    horizontalArrangement = Arrangement.SpaceBetween,
                                    verticalAlignment = Alignment.CenterVertically
                                ) {
                                    Column {
                                        Text(
                                            "Recorded Product Expenses",
```

```
                                        style = MaterialTheme.typography.titleSmall.copy(fontWeight =
FontWeight.Bold)
                                    )
                                    if (productRefundsPaise > 0) {
                                        Text(
                                            "Gross
${CostCalculator.formatPaiseToRupees(grossProductExpensesPaise)} - Refunds
${CostCalculator.formatPaiseToRupees(productRefundsPaise)}",
                                            style = MaterialTheme.typography.bodySmall,
                                            color = MaterialTheme.colorScheme.onSurfaceVariant
                                        )
                                    }
                                }
                                Text(
                                    CostCalculator.formatPaiseToRupees(netProductExpensesPaise),
                                    style = MaterialTheme.typography.titleMedium.copy(fontWeight =
FontWeight.Bold),
                                    color = Color(0xFFE57373)
                                )
                            }

                            Spacer(modifier = Modifier.height(8.dp))
                            HorizontalDivider()
                            Spacer(modifier = Modifier.height(8.dp))

                            Row(
                                modifier = Modifier.fillMaxWidth(),
                                horizontalArrangement = Arrangement.SpaceBetween,
                                verticalAlignment = Alignment.CenterVertically
                            ) {
                                Text(
                                    "Result After Product Expenses",
                                    style = MaterialTheme.typography.bodyMedium.copy(fontWeight =
FontWeight.Bold)
                                )
                                Text(
                                    CostCalculator.formatPaiseToRupees(productResultAfterExpensesPaise),
                                    style = MaterialTheme.typography.titleMedium.copy(fontWeight =
FontWeight.Bold),
                                    color = if (productResultAfterExpensesPaise >= 0) Color(0xFF75B890)
else Color(0xFFE57373)
                                )
                            }
                            Text(
                                "Excludes Shared shop expenses (see Combined Overview for whole-shop
result).",
                                style = MaterialTheme.typography.labelSmall,
                                color = MaterialTheme.colorScheme.onSurfaceVariant
                            )
                        }
                    }
                }

                // 4. Sales & Gross Profit Trend Graph (Section 6 A)
                item {
                    SalesProfitTrendChart(
                        buckets = trendBuckets,
                        onBucketClick = { bucket -> selectedBucketDetail = bucket }
                    )
                }

                // 5. Expense Breakdown Chart (Section 6 B)
                item {
                    ExpenseBreakdownChart(
                        expenses = productExpenses,
                        refunds = productRefunds,
                        title = "Product Expenses Breakdown"
                    )
                }

                // 6. Expected Gross Profit from Current Stock Card
                item {
                    Card(
                        modifier = Modifier.fillMaxWidth(),
                        colors = CardDefaults.cardColors(containerColor =
MaterialTheme.colorScheme.surface),
                        shape = RoundedCornerShape(12.dp)
                    ) {
                        Column(modifier = Modifier.padding(14.dp)) {
                            Row(
                                modifier = Modifier.fillMaxWidth(),
                                horizontalArrangement = Arrangement.SpaceBetween,
                                verticalAlignment = Alignment.CenterVertically
                            ) {
                                Text(
                                    "Expected Profit from Current Stock",
                                    style = MaterialTheme.typography.titleSmall.copy(fontWeight =
FontWeight.Bold)
```

```
                                    )
                                    Text(
                                        CostCalculator.formatPaiseToRupees(expectedGrossProfitPaise),
                                        style = MaterialTheme.typography.titleMedium.copy(fontWeight =
FontWeight.Bold),
                                        color = MaterialTheme.colorScheme.primary
                                    )
                                }
                                Spacer(modifier = Modifier.height(4.dp))
                                Text(
                                    "Projected gross profit on unsold warehouse inventory if sold at current
 retail price.",
                                    style = MaterialTheme.typography.bodySmall,
                                    color = MaterialTheme.colorScheme.onSurfaceVariant
                                )
                            }
                        }
                    }
                }

                // ── TAB CONTENT: SERVICES ──────────────────────────────────────
                if (activeTab == "SERVICES" && isOwner) {
                    item {
                        Card(
                            modifier = Modifier.fillMaxWidth(),
                            colors = CardDefaults.cardColors(
                                containerColor = if (serviceResultAfterExpensesPaise >= 0) Color(0xFF1B3828)
 else Color(0xFF3E1B1B)
                            ),
                            shape = RoundedCornerShape(16.dp)
                        ) {
                            Column(modifier = Modifier.padding(18.dp)) {
                                Text(
                                    "Service Revenue Minus Service Expenses",
                                    style = MaterialTheme.typography.titleMedium.copy(fontWeight =
FontWeight.Bold),
                                    color = MaterialTheme.colorScheme.onSurface
                                )
                                Spacer(modifier = Modifier.height(10.dp))
                                Text(
                                    CostCalculator.formatPaiseToRupees(serviceResultAfterExpensesPaise),
                                    style = MaterialTheme.typography.headlineLarge.copy(fontWeight =
FontWeight.Bold),
                                    color = if (serviceResultAfterExpensesPaise >= 0) Color(0xFF75B890) else
 Color(0xFFE57373)
                                )
                                Spacer(modifier = Modifier.height(6.dp))
                                Text(
                                    "Service Revenue
${CostCalculator.formatPaiseToRupees(serviceRevenuePaise)} minus Service Costs
${CostCalculator.formatPaiseToRupees(netServiceExpensesPaise)}",
                                    style = MaterialTheme.typography.bodySmall,
                                    color = MaterialTheme.colorScheme.onSurfaceVariant
                                )
                            }
                        }
                    }

                    item {
                        Row(
                            modifier = Modifier.fillMaxWidth(),
                            horizontalArrangement = Arrangement.spacedBy(10.dp)
                        ) {
                            Card(
                                modifier = Modifier.weight(1f),
                                colors = CardDefaults.cardColors(containerColor =
MaterialTheme.colorScheme.surface),
                                shape = RoundedCornerShape(12.dp)
                            ) {
                                Column(modifier = Modifier.padding(14.dp)) {
                                    Text("Service Revenue", style = MaterialTheme.typography.bodySmall,
color = MaterialTheme.colorScheme.onSurfaceVariant)
                                    Spacer(modifier = Modifier.height(4.dp))
                                    Text(
                                        CostCalculator.formatPaiseToRupees(serviceRevenuePaise),
                                        style = MaterialTheme.typography.titleMedium.copy(fontWeight =
FontWeight.Bold),
                                        color = MaterialTheme.colorScheme.onSurface
                                    )
                                }
                            }

                            Card(
                                modifier = Modifier.weight(1f),
                                colors = CardDefaults.cardColors(containerColor =
MaterialTheme.colorScheme.surface),
                                shape = RoundedCornerShape(12.dp)
                            ) {
```

```
                                Column(modifier = Modifier.padding(14.dp)) {
                                    Text("Service Costs", style = MaterialTheme.typography.bodySmall, color = MaterialTheme.colorScheme.onSurfaceVariant)
                                    Spacer(modifier = Modifier.height(4.dp))
                                    Text(
                                        CostCalculator.formatPaiseToRupees(netServiceExpensesPaise),
                                        style = MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.Bold),
                                        color = MaterialTheme.colorScheme.onSurface
                                    )
                                }
                            }
                        }
                    }

                    item {
                        ExpenseBreakdownChart(
                            expenses = serviceExpenses,
                            refunds = serviceRefunds,
                            title = "Services Expenses Breakdown"
                        )
                    }
                }

                // ── TAB CONTENT: COMBINED ──────────────────────────────────────
                if (activeTab == "COMBINED" && isOwner) {
                    item {
                        Card(
                            modifier = Modifier.fillMaxWidth(),
                            colors = CardDefaults.cardColors(
                                containerColor = if (combinedRecordedOperatingResultPaise >= 0) Color(0xFF1B3828) else Color(0xFF3E1B1B)
                            ),
                            shape = RoundedCornerShape(16.dp)
                        ) {
                            Column(modifier = Modifier.padding(18.dp)) {
                                Text(
                                    "Combined Recorded Operating Result",
                                    style = MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.Bold),
                                    color = MaterialTheme.colorScheme.onSurface
                                )
                                Spacer(modifier = Modifier.height(10.dp))
                                Text(
                                    CostCalculator.formatPaiseToRupees(combinedRecordedOperatingResultPaise),
                                    style = MaterialTheme.typography.headlineLarge.copy(fontWeight = FontWeight.Bold),
                                    color = if (combinedRecordedOperatingResultPaise >= 0) Color(0xFF75B890) else Color(0xFFE57373)
                                )
                                Spacer(modifier = Modifier.height(6.dp))
                                Text(
                                    "Product GP + Service Rev - All Net Operating Expenses (Products + Services + Shared)",
                                    style = MaterialTheme.typography.bodySmall,
                                    color = MaterialTheme.colorScheme.onSurfaceVariant
                                )
                            }
                        }
                    }

                    item {
                        Card(
                            modifier = Modifier.fillMaxWidth(),
                            colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surface),
                            shape = RoundedCornerShape(12.dp)
                        ) {
                            Column(
                                modifier = Modifier.padding(14.dp),
                                verticalArrangement = Arrangement.spacedBy(8.dp)
                            ) {
                                Text("Operating Summary Breakdown", fontWeight = FontWeight.Bold)

                                Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement = Arrangement.SpaceBetween) {
                                    Text("1. Product Gross Profit:", style = MaterialTheme.typography.bodySmall)
                                    Text(CostCalculator.formatPaiseToRupees(productGrossProfitPaise), style = MaterialTheme.typography.bodySmall, fontWeight = FontWeight.Bold, color = Color(0xFF75B890))
                                }

                                Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement = Arrangement.SpaceBetween) {
                                    Text("2. Service Revenue:", style = MaterialTheme.typography.bodySmall)
                                    Text(CostCalculator.formatPaiseToRupees(serviceRevenuePaise), style = MaterialTheme.typography.bodySmall, fontWeight = FontWeight.Bold, color = Color(0xFF75B890))
```

```
                                }

                                HorizontalDivider()

                                Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement =
Arrangement.SpaceBetween) {
                                    Text("3. Product Expenses (Net):", style =
MaterialTheme.typography.bodySmall)
                                    Text("-${CostCalculator.formatPaiseToRupees(netProductExpensesPaise)}",
style = MaterialTheme.typography.bodySmall, color = Color(0xFFE57373))
                                }

                                Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement =
Arrangement.SpaceBetween) {
                                    Text("4. Service Expenses (Net):", style =
MaterialTheme.typography.bodySmall)
                                    Text("-${CostCalculator.formatPaiseToRupees(netServiceExpensesPaise)}",
style = MaterialTheme.typography.bodySmall, color = Color(0xFFE57373))
                                }

                                Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement =
Arrangement.SpaceBetween) {
                                    Text("5. Shared Expenses (Net):", style =
MaterialTheme.typography.bodySmall)
                                    Text("-${CostCalculator.formatPaiseToRupees(netSharedExpensesPaise)}",
style = MaterialTheme.typography.bodySmall, color = Color(0xFFE57373))
                                }

                                HorizontalDivider()

                                Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement =
Arrangement.SpaceBetween) {
                                    Text("Net Operating Result:", style =
MaterialTheme.typography.bodyMedium.copy(fontWeight = FontWeight.Bold))
                                    Text(
CostCalculator.formatPaiseToRupees(combinedRecordedOperatingResultPaise),
                                        style = MaterialTheme.typography.bodyMedium.copy(fontWeight =
FontWeight.Bold),
                                        color = if (combinedRecordedOperatingResultPaise >= 0)
Color(0xFF75B890) else Color(0xFFE57373)
                                    )
                                }
                            }
                        }
                    }

                    item {
                        ExpenseBreakdownChart(
                            expenses = periodActiveExpenses.filter { !it.isIncome },
                            refunds = periodActiveExpenses.filter { it.isIncome && it.isExpenseRefund },
                            title = "All Shop Expenses Breakdown"
                        )
                    }
                }

                item {
                    Spacer(modifier = Modifier.height(24.dp))
                }
            }
        }
    }

    // Bucket Detail Dialog
    selectedBucketDetail?.let { bucket ->
        AlertDialog(
            onDismissRequest = { selectedBucketDetail = null },
            title = { Text(bucket.label, fontWeight = FontWeight.Bold) },
            text = {
                Column(verticalArrangement = Arrangement.spacedBy(6.dp)) {
                    Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement =
Arrangement.SpaceBetween) {
                        Text("Sales:")
                        Text(CostCalculator.formatPaiseToRupees(bucket.salesPaise), fontWeight =
FontWeight.Bold)
                    }
                    Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement =
Arrangement.SpaceBetween) {
                        Text("COGS:")
                        Text(CostCalculator.formatPaiseToRupees(bucket.cogsPaise), fontWeight =
FontWeight.Bold)
                    }
                    Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement =
Arrangement.SpaceBetween) {
                        Text("Gross Profit:")
                        Text(
                            CostCalculator.formatPaiseToRupees(bucket.grossProfitPaise),
                            fontWeight = FontWeight.Bold,
```

```
                            color = if (bucket.grossProfitPaise >= 0) Color(0xFF75B890) else Color(0xFFE57373)
                        )
                    }
                }
            },
            confirmButton = {
                Button(onClick = { selectedBucketDetail = null }) { Text("OK") }
            }
        )
    }
}

// ── TREND BUCKET GENERATOR ──────────────────────────────────────
fun generateTrendBuckets(
    sales: List<com.alieducational.app.data.local.entity.SaleEntity>,
    lines: List<com.alieducational.app.data.local.entity.SaleLineEntity>,
    startTime: Long,
    endTime: Long,
    period: String
): List<ProfitTrendBucket> {
    if (sales.isEmpty()) return emptyList()

    val saleLinesMap = lines.groupBy { it.saleId }

    if (period == "Today") {
        val totalSales = sales.sumOf { it.netTotalPaise }
        val productLines = lines.filter { it.lineType != "SERVICE" }
        val totalCogs = productLines.sumOf { it.quantityBase.toLong() * (it.unitCostSnapshotPaise ?: 0L) }
        return listOf(
            ProfitTrendBucket(
                label = "Today",
                startTime = startTime,
                endTime = endTime,
                salesPaise = totalSales,
                cogsPaise = totalCogs,
                grossProfitPaise = totalSales - totalCogs
            )
        )
    }

    // Group sales into days
    val df = SimpleDateFormat("dd MMM", Locale.US)
    val dayBuckets = mutableMapOf<String, MutableList<com.alieducational.app.data.local.entity.SaleEntity>>()

    sales.sortedBy { it.timestamp }.forEach { s ->
        val key = df.format(Date(s.timestamp))
        dayBuckets.getOrPut(key) { mutableListOf() }.add(s)
    }

    return dayBuckets.map { (dateStr, daySales) ->
        val daySaleIds = daySales.map { it.id }.toSet()
        val dayLines = lines.filter { it.saleId in daySaleIds && it.lineType != "SERVICE" }
        val sPaise = daySales.sumOf { it.netTotalPaise }
        val cPaise = dayLines.sumOf { it.quantityBase.toLong() * (it.unitCostSnapshotPaise ?: 0L) }
        ProfitTrendBucket(
            label = dateStr,
            startTime = daySales.minOf { it.timestamp },
            endTime = daySales.maxOf { it.timestamp },
            salesPaise = sPaise,
            cogsPaise = cPaise,
            grossProfitPaise = sPaise - cPaise
        )
    }
}

// ── TREND CHART COMPOSABLE ──────────────────────────────────────
@Composable
fun SalesProfitTrendChart(
    buckets: List<ProfitTrendBucket>,
    onBucketClick: (ProfitTrendBucket) -> Unit,
    modifier: Modifier = Modifier
) {
    Card(
        modifier = modifier.fillMaxWidth(),
        colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surface),
        shape = RoundedCornerShape(12.dp)
    ) {
        Column(modifier = Modifier.padding(14.dp)) {
            Row(
                modifier = Modifier.fillMaxWidth(),
                horizontalArrangement = Arrangement.SpaceBetween,
                verticalAlignment = Alignment.CenterVertically
            ) {
                Text("Sales & Gross Profit Trend", fontWeight = FontWeight.Bold)
                Row(horizontalArrangement = Arrangement.spacedBy(10.dp)) {
                    Row(verticalAlignment = Alignment.CenterVertically) {
```

```
                        Box(modifier = Modifier.size(8.dp).background(Color(0xFFE5A93C),
RoundedCornerShape(2.dp)))
                        Spacer(modifier = Modifier.width(4.dp))
                        Text("Sales", fontSize = 11.sp, color = MaterialTheme.colorScheme.onSurfaceVariant)
                    }
                    Row(verticalAlignment = Alignment.CenterVertically) {
                        Box(modifier = Modifier.size(8.dp).background(Color(0xFF75B890),
RoundedCornerShape(2.dp)))
                        Spacer(modifier = Modifier.width(4.dp))
                        Text("Gross Profit", fontSize = 11.sp, color =
MaterialTheme.colorScheme.onSurfaceVariant)
                    }
                }
            }

            Spacer(modifier = Modifier.height(12.dp))

            if (buckets.isEmpty()) {
                Box(
                    modifier = Modifier
                        .fillMaxWidth()
                        .height(120.dp),
                    contentAlignment = Alignment.Center
                ) {
                    Text("No transactions in this period", style = MaterialTheme.typography.bodySmall, color
 = MaterialTheme.colorScheme.onSurfaceVariant)
                }
            } else {
                Box(
                    modifier = Modifier
                        .fillMaxWidth()
                        .height(150.dp)
                ) {
                    Canvas(
                        modifier = Modifier
                            .fillMaxSize()
                            .pointerInput(buckets) {
                                detectTapGestures { offset ->
                                    val bucketWidth = size.width / buckets.size
                                    val index = (offset.x / bucketWidth).toInt().coerceIn(0, buckets.size -
1)
                                    onBucketClick(buckets[index])
                                }
                            }
                    ) {
                        val maxSales = buckets.maxOfOrNull { it.salesPaise }?.coerceAtLeast(1L) ?: 1L
                        val maxProfit = buckets.maxOfOrNull { it.grossProfitPaise }?.coerceAtLeast(1L) ?: 1L
                        val minProfit = buckets.minOfOrNull { it.grossProfitPaise }?.coerceAtMost(0L) ?: 0L

                        val overallMax = maxOf(maxSales, maxProfit)
                        val overallMin = minProfit

                        val totalSpan = (overallMax - overallMin).toFloat().coerceAtLeast(1f)
                        val zeroY = size.height * (overallMax.toFloat() / totalSpan)

                        // Draw Zero Baseline
                        drawLine(
                            color = Color.Gray.copy(alpha = 0.5f),
                            start = Offset(0f, zeroY),
                            end = Offset(size.width, zeroY),
                            strokeWidth = 1.dp.toPx()
                        )

                        val bucketWidth = size.width / buckets.size
                        val barWidth = (bucketWidth * 0.35f).coerceAtLeast(4f)

                        buckets.forEachIndexed { index, b ->
                            val centerX = index * bucketWidth + bucketWidth / 2f

                            // Sales Bar
                            val salesHeight = (b.salesPaise.toFloat() / totalSpan) * size.height
                            drawRect(
                                color = Color(0xFFE5A93C),
                                topLeft = Offset(centerX - barWidth - 2f, zeroY - salesHeight),
                                size = Size(barWidth, salesHeight)
                            )

                            // Profit Bar (positive upwards, negative downwards)
                            val profitHeight = (kotlin.math.abs(b.grossProfitPaise).toFloat() / totalSpan) *
 size.height
                            val profitTop = if (b.grossProfitPaise >= 0) zeroY - profitHeight else zeroY
                            val profitColor = if (b.grossProfitPaise >= 0) Color(0xFF75B890) else
Color(0xFFE57373)

                            drawRect(
                                color = profitColor,
                                topLeft = Offset(centerX + 2f, profitTop),
                                size = Size(barWidth, profitHeight)
                            )
```

```
                        }
                    }
                }

                // Bucket Labels Row
                Row(
                    modifier = Modifier.fillMaxWidth(),
                    horizontalArrangement = Arrangement.SpaceBetween
                ) {
                    val sampleLabels = if (buckets.size <= 5) buckets else listOf(buckets.first(),
    buckets[buckets.size / 2], buckets.last())
                    sampleLabels.forEach { b ->
                        Text(b.label, fontSize = 10.sp, color = MaterialTheme.colorScheme.onSurfaceVariant)
                    }
                }
            }
        }
    }
}

// ── EXPENSE BREAKDOWN CHART (Section 6 B) ──────────────────────────
@Composable
fun ExpenseBreakdownChart(
    expenses: List<ShopExpenseEntity>,
    refunds: List<ShopExpenseEntity>,
    title: String,
    modifier: Modifier = Modifier
) {
    Card(
        modifier = modifier.fillMaxWidth(),
        colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surface),
        shape = RoundedCornerShape(12.dp)
    ) {
        Column(modifier = Modifier.padding(14.dp)) {
            Text(title, fontWeight = FontWeight.Bold)
            Spacer(modifier = Modifier.height(10.dp))

            if (expenses.isEmpty()) {
                Text(
                    "No recorded expenses in this scope/period.",
                    style = MaterialTheme.typography.bodySmall,
                    color = MaterialTheme.colorScheme.onSurfaceVariant
                )
            } else {
                // Group by category, calculating net expense (expenses minus refunds)
                val catExpenses = expenses.groupBy { it.category }
                val catRefunds = refunds.groupBy { it.category }
                val allCats = (catExpenses.keys + catRefunds.keys).distinct()

                val categoryTotals = allCats.map { cat ->
                    val gross = catExpenses[cat]?.sumOf { it.amountPaise } ?: 0L
                    val ref = catRefunds[cat]?.sumOf { it.amountPaise } ?: 0L

                    cat to (gross - ref)
                }.sortedByDescending { it.second }

                val maxAmount = categoryTotals.maxOfOrNull { kotlin.math.abs(it.second) }?.coerceAtLeast(1L)
     ?: 1L

                Column(verticalArrangement = Arrangement.spacedBy(8.dp)) {
                    categoryTotals.forEach { (cat, netPaise) ->
                        val fraction = (kotlin.math.abs(netPaise).toFloat() /
    maxAmount.toFloat()).coerceIn(0.05f, 1f)
                        val isNegative = netPaise < 0

                        Column {
                            Row(
                                modifier = Modifier.fillMaxWidth(),
                                horizontalArrangement = Arrangement.SpaceBetween
                            ) {
                                Text(cat, fontSize = 12.sp, fontWeight = FontWeight.Medium)
                                Text(
                                    CostCalculator.formatPaiseToRupees(netPaise),
                                    fontSize = 12.sp,
                                    fontWeight = FontWeight.Bold,
                                    color = if (isNegative) Color(0xFF75B890) else
    MaterialTheme.colorScheme.onSurface
                                )
                            }
                            Spacer(modifier = Modifier.height(3.dp))
                            Box(
                                modifier = Modifier
                                    .fillMaxWidth()
                                    .height(6.dp)
                                    .background(MaterialTheme.colorScheme.surfaceVariant,
    RoundedCornerShape(3.dp))
                            ) {
                                Box(
                                    modifier = Modifier
```

```
                                        .fillMaxWidth(fraction)
                                        .height(6.dp)
                                        .background(
                                            if (isNegative) Color(0xFF75B890) else Color(0xFFE57373),
                                            RoundedCornerShape(3.dp)
                                        )
                                )
                            }
                        }
                    }
                }
            }
        }
    }
}
```

app\src\main\java\com\alieducational\app\ui\sales\RecordDailyTotalScreen.kt

```
package com.alieducational.app.ui.sales

import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.foundation.verticalScroll
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.ArrowBack
import androidx.compose.material.icons.filled.Info
import androidx.compose.material3.Button
import androidx.compose.material3.ButtonDefaults
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.RadioButton
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.material3.TopAppBar
import androidx.compose.material3.TopAppBarDefaults
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.unit.dp
import com.alieducational.app.ui.AliViewModel

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun RecordDailyTotalScreen(
    viewModel: AliViewModel,
    onNavigateBack: () -> Unit,
    modifier: Modifier = Modifier
) {
    var amountStr by remember { mutableStateOf("") }
    var isFullDayTotal by remember { mutableStateOf(false) } // true = full day, false = additional unrecorded
    var notes by remember { mutableStateOf("") }

    val amountPaise = ((amountStr.toDoubleOrNull() ?: 0.0) * 100).toLong()

    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text("Record Daily Monetary Total", fontWeight = FontWeight.Bold) },
                navigationIcon = {
                    IconButton(onClick = onNavigateBack) {
                        Icon(Icons.Filled.ArrowBack, contentDescription = "Back")
                    }
                },
                colors = TopAppBarDefaults.topAppBarColors(
                    containerColor = MaterialTheme.colorScheme.background,
                    titleContentColor = MaterialTheme.colorScheme.onBackground
                )
            )
        }
    ) { padding ->
        Surface(
            modifier = modifier
                .fillMaxSize()
                .padding(padding),
```

```
            color = MaterialTheme.colorScheme.background
        ) {
            Column(
                modifier = Modifier
                    .fillMaxSize()
                    .verticalScroll(rememberScrollState())
                    .padding(16.dp),
                verticalArrangement = Arrangement.spacedBy(14.dp)
            ) {
                // Info Banner
                Card(
                    modifier = Modifier.fillMaxWidth(),
                    colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surfaceVariant),
                    shape = RoundedCornerShape(12.dp)
                ) {
                    Row(
                        modifier = Modifier
                            .fillMaxWidth()
                            .padding(14.dp),
                        verticalAlignment = Alignment.CenterVertically
                    ) {
                        Icon(Icons.Filled.Info, contentDescription = null, tint = MaterialTheme.colorScheme.primary, modifier = Modifier.size(24.dp))
                        Spacer(modifier = Modifier.width(10.dp))
                        Text(
                            text = "For busy shop days when only total cash is known. Keeps sales accurate without inventing stock deductions.",
                            style = MaterialTheme.typography.bodySmall,
                            color = MaterialTheme.colorScheme.onSurfaceVariant
                        )
                    }
                }

                // Scope Selector Card
                Card(
                    modifier = Modifier.fillMaxWidth(),
                    colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surface),
                    shape = RoundedCornerShape(14.dp)
                ) {
                    Column(modifier = Modifier.padding(14.dp)) {
                        Text(
                            text = "Total Scope",
                            style = MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.Bold),
                            color = MaterialTheme.colorScheme.onSurface
                        )
                        Spacer(modifier = Modifier.height(8.dp))

                        Row(
                            modifier = Modifier
                                .fillMaxWidth()
                                .clickable { isFullDayTotal = false }
                                .padding(vertical = 4.dp),
                            verticalAlignment = Alignment.CenterVertically
                        ) {
                            RadioButton(selected = !isFullDayTotal, onClick = { isFullDayTotal = false })
                            Spacer(modifier = Modifier.width(8.dp))
                            Column {
                                Text("Additional Unrecorded Sales", fontWeight = FontWeight.Bold)
                                Text("Adds onto already recorded individual sales for today", style = MaterialTheme.typography.bodySmall, color = MaterialTheme.colorScheme.onSurfaceVariant)
                            }
                        }

                        Spacer(modifier = Modifier.height(4.dp))

                        Row(
                            modifier = Modifier
                                .fillMaxWidth()
                                .clickable { isFullDayTotal = true }
                                .padding(vertical = 4.dp),
                            verticalAlignment = Alignment.CenterVertically
                        ) {
                            RadioButton(selected = isFullDayTotal, onClick = { isFullDayTotal = true })
                            Spacer(modifier = Modifier.width(8.dp))
                            Column {
                                Text("Full Day's Net Total", fontWeight = FontWeight.Bold)
                                Text("The complete daily net sales amount", style = MaterialTheme.typography.bodySmall, color = MaterialTheme.colorScheme.onSurfaceVariant)
                            }
                        }
                    }
                }

                // Amount Input Card
                Card(
                    modifier = Modifier.fillMaxWidth(),
                    colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surface),
```

```
                    shape = RoundedCornerShape(14.dp)
                ) {
                    Column(modifier = Modifier.padding(14.dp)) {
                        Text(
                            text = "Amount Details",
                            style = MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.Bold),
                            color = MaterialTheme.colorScheme.onSurface
                        )
                        Spacer(modifier = Modifier.height(10.dp))

                        OutlinedTextField(
                            value = amountStr,
                            onValueChange = { amountStr = it },
                            label = { Text("Total Net Amount (₹) *") },
                            keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Decimal),
                            modifier = Modifier.fillMaxWidth(),
                            singleLine = true,
                            shape = RoundedCornerShape(10.dp)
                        )

                        Spacer(modifier = Modifier.height(8.dp))
                        Text(
                            text = "Status: Marked as 'Stock details needed' & 'Profit unavailable'. You can link items later.",
                            style = MaterialTheme.typography.bodySmall,
                            color = Color(0xFFE4B765)
                        )

                        Spacer(modifier = Modifier.height(10.dp))

                        OutlinedTextField(
                            value = notes,
                            onValueChange = { notes = it },
                            label = { Text("Notes (Optional)") },
                            modifier = Modifier.fillMaxWidth(),
                            singleLine = true,
                            shape = RoundedCornerShape(10.dp)
                        )
                    }
                }

                // Save Button
                Button(
                    onClick = {
                        if (amountPaise <= 0) return@Button
                        viewModel.recordDailyMonetaryTotal(
                            dateTimestamp = System.currentTimeMillis(),
                            amountPaise = amountPaise,
                            isFullDayTotal = isFullDayTotal,
                            notes = notes.trim().ifBlank { null },
                            onDone = onNavigateBack
                        )
                    },
                    enabled = amountPaise > 0,
                    modifier = Modifier
                        .fillMaxWidth()
                        .height(54.dp),
                    shape = RoundedCornerShape(12.dp),
                    colors = ButtonDefaults.buttonColors(
                        containerColor = MaterialTheme.colorScheme.primary,
                        contentColor = MaterialTheme.colorScheme.onPrimary
                    )
                ) {
                    Text(
                        text = "Save Daily Monetary Total",
                        style = MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.Bold)
                    )
                }

                Spacer(modifier = Modifier.height(24.dp))

            }
        }
    }
}
```

app\src\main\java\com\alieducational\app\ui\sales\RecordSaleScreen.kt

```
package com.alieducational.app.ui.sales

import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.itemsIndexed
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Add
import androidx.compose.material.icons.filled.ArrowBack
import androidx.compose.material.icons.filled.Delete
import androidx.compose.material.icons.filled.Share
import androidx.compose.material3.AlertDialog
import androidx.compose.material3.Button
import androidx.compose.material3.ButtonDefaults
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.DropdownMenuItem
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.ExposedDropdownMenuBox
import androidx.compose.material3.ExposedDropdownMenuDefaults
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedButton
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.RadioButton
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.material3.TopAppBar
import androidx.compose.material3.TopAppBarDefaults
import androidx.compose.runtime.Composable
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateListOf
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.alieducational.app.data.local.entity.CustomerEntity
import com.alieducational.app.data.local.entity.SaleEntity
import com.alieducational.app.data.repository.SaleItemParam
import com.alieducational.app.domain.calculation.CostCalculator
import com.alieducational.app.ui.AliViewModel

data class CartLineState(
    var productId: String = "",
    var productName: String = "",
    var packSize: Int = 1,
    var isPack: Boolean = false,
    var quantityStr: String = "1",
    var unitPriceStr: String = "0"
)

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun RecordSaleScreen(
    viewModel: AliViewModel,
    onNavigateBack: () -> Unit,
    modifier: Modifier = Modifier
) {
    val allProducts by viewModel.allProducts.collectAsState()
    val allCustomers by viewModel.allCustomers.collectAsState()

    val cart = remember { mutableStateListOf<CartLineState>() }
```

```

    var selectedCustomer by remember { mutableStateOf<CustomerEntity?>(null) }
    var customerExpanded by remember { mutableStateOf(false) }
    var newCustomerName by remember { mutableStateOf("") }
    var showNewCustomerInput by remember { mutableStateOf(false) }

    var discountMode by remember { mutableStateOf("NONE") } // "NONE", "PERCENT", "FINAL_PRICE"
    var discountPercentStr by remember { mutableStateOf("0") }
    var agreedFinalPriceStr by remember { mutableStateOf("") }

    var amountReceivedStr by remember { mutableStateOf("") }
    var paymentMethod by remember { mutableStateOf("CASH") }
    var notes by remember { mutableStateOf("") }

    var showShortPaymentDialog by remember { mutableStateOf(false) }
    var completedSale by remember { mutableStateOf<SaleEntity?>(null) }

    // Computations
    val lineItems = cart.mapNotNull { line ->
        val qty = line.quantityStr.toIntOrNull() ?: 0
        val baseQty = if (line.isPack) qty * line.packSize else qty
        val pricePaise = ((line.unitPriceStr.toDoubleOrNull() ?: 0.0) * 100).toLong()
        if (line.productId.isNotBlank() && baseQty > 0) {
            SaleItemParam(
                productId = line.productId,
                productName = line.productName,
                quantityBase = baseQty,
                isPack = line.isPack,
                unitSellingPricePaise = pricePaise
            )
        } else null
    }

    val grossTotalPaise = lineItems.sumOf { it.quantityBase.toLong() * it.unitSellingPricePaise }

    val discountValuePaise: Long = when (discountMode) {
        "PERCENT" -> {
            val pct = discountPercentStr.toDoubleOrNull() ?: 0.0
            ((grossTotalPaise * pct) / 100.0).toLong()
        }
        "FINAL_PRICE" -> {
            val finalPrice = ((agreedFinalPriceStr.toDoubleOrNull() ?: 0.0) * 100).toLong()
            (grossTotalPaise - finalPrice).coerceAtLeast(0L)
        }
        else -> 0L
    }

    val netTotalPaise = (grossTotalPaise - discountValuePaise).coerceAtLeast(0L)
    val computedDiscountPercent: Double = if (grossTotalPaise > 0) {
        (discountValuePaise.toDouble() / grossTotalPaise.toDouble()) * 100.0
    } else 0.0

    val amountReceivedPaise = if (amountReceivedStr.isBlank()) netTotalPaise
    else ((amountReceivedStr.toDoubleOrNull() ?: 0.0) * 100).toLong()

    val isShortPayment = amountReceivedPaise < netTotalPaise
    val balanceOwed = (netTotalPaise - amountReceivedPaise).coerceAtLeast(0L)

    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text("Record Sale", fontWeight = FontWeight.Bold) },
                navigationIcon = {
                    IconButton(onClick = onNavigateBack) {
                        Icon(Icons.Filled.ArrowBack, contentDescription = "Back")
                    }
                },
                colors = TopAppBarDefaults.topAppBarColors(
                    containerColor = MaterialTheme.colorScheme.background,
                    titleContentColor = MaterialTheme.colorScheme.onBackground
                )
            )
        }
    ) { padding ->
        Surface(
            modifier = modifier
                .fillMaxSize()
                .padding(padding),
            color = MaterialTheme.colorScheme.background
        ) {
            LazyColumn(
                modifier = Modifier
                    .fillMaxSize()
                    .padding(horizontal = 16.dp),
                verticalArrangement = Arrangement.spacedBy(14.dp)
            ) {
                // Customer Selector (Optional for Cash, Required for Credit)
                item {
```

```
                    Card(
                        modifier = Modifier.fillMaxWidth(),
                        colors = CardDefaults.cardColors(containerColor =
MaterialTheme.colorScheme.surface),
                        shape = RoundedCornerShape(14.dp)
                    ) {
                        Column(modifier = Modifier.padding(14.dp)) {
                            Row(
                                modifier = Modifier.fillMaxWidth(),
                                horizontalArrangement = Arrangement.SpaceBetween,
                                verticalAlignment = Alignment.CenterVertically
                            ) {
                                Text(
                                    text = "Customer (Optional for paid sales)",
                                    style = MaterialTheme.typography.titleMedium.copy(fontWeight =
FontWeight.Bold),
                                    color = MaterialTheme.colorScheme.onSurface
                                )
                                if (selectedCustomer != null) {
                                    Text(
                                        text = "Due:
${CostCalculator.formatPaiseToRupees(selectedCustomer!!.currentBalancePaise)}",
                                        style = MaterialTheme.typography.bodySmall.copy(fontWeight =
FontWeight.Bold),
                                        color = Color(0xFFE4B765)
                                    )
                                }
                            }

                            Spacer(modifier = Modifier.height(10.dp))

                            if (showNewCustomerInput) {
                                OutlinedTextField(
                                    value = newCustomerName,
                                    onValueChange = { newCustomerName = it },
                                    label = { Text("Customer Name") },
                                    modifier = Modifier.fillMaxWidth(),
                                    singleLine = true
                                )
                                Spacer(modifier = Modifier.height(8.dp))
                                Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
                                    Button(
                                        onClick = {
                                            if (newCustomerName.isNotBlank()) {
                                                val c = CustomerEntity(name = newCustomerName.trim())
                                                viewModel.addCustomer(c) {
                                                    selectedCustomer = c
                                                    showNewCustomerInput = false
                                                }
                                            }
                                        }
                                    ) { Text("Save Customer") }
                                    OutlinedButton(onClick = { showNewCustomerInput = false }) {
Text("Cancel") }
                                }
                            } else {
                                ExposedDropdownMenuBox(
                                    expanded = customerExpanded,
                                    onExpandedChange = { customerExpanded = !customerExpanded }
                                ) {
                                    OutlinedTextField(
                                        value = selectedCustomer?.name ?: "Walk-in Cash Customer (No
account)",
                                        onValueChange = {},
                                        readOnly = true,
                                        trailingIcon = { ExposedDropdownMenuDefaults.TrailingIcon(expanded =
 customerExpanded) },
                                        modifier = Modifier
                                            .fillMaxWidth()
                                            .menuAnchor(),
                                        shape = RoundedCornerShape(10.dp)
                                    )
                                    ExposedDropdownMenu(
                                        expanded = customerExpanded,
                                        onDismissRequest = { customerExpanded = false }
                                    ) {
                                        DropdownMenuItem(
                                            text = { Text("Walk-in Cash Customer (No account)") },
                                            onClick = {
                                                selectedCustomer = null
                                                customerExpanded = false
                                            }
                                        )
                                        allCustomers.forEach { customer ->
                                            DropdownMenuItem(
                                                text = { Text("${customer.name} (Balance:
${CostCalculator.formatPaiseToRupees(customer.currentBalancePaise)})") },
                                                onClick = {
```

```
                                                selectedCustomer = customer
                                                customerExpanded = false
                                            }
                                        )
                                    }
                                    DropdownMenuItem(
                                        text = { Text("+ Add New Customer", color =
MaterialTheme.colorScheme.primary, fontWeight = FontWeight.Bold) },
                                        onClick = {
                                            customerExpanded = false
                                            showNewCustomerInput = true
                                        }
                                    )
                                }
                            }
                        }
                    }
                }

                // Cart Items Section
                item {
                    Row(
                        modifier = Modifier.fillMaxWidth(),
                        horizontalArrangement = Arrangement.SpaceBetween,
                        verticalAlignment = Alignment.CenterVertically
                    ) {
                        Text(
                            text = "Sale Items (${cart.size})",
                            style = MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.Bold),
                            color = MaterialTheme.colorScheme.onBackground
                        )
                        Button(
                            onClick = { cart.add(CartLineState()) },
                            shape = RoundedCornerShape(8.dp),
                            contentPadding = ButtonDefaults.TextButtonContentPadding
                        ) {
                            Icon(Icons.Filled.Add, contentDescription = null, modifier =
Modifier.size(16.dp))
                            Spacer(modifier = Modifier.width(4.dp))
                            Text("Add Item", fontSize = 12.sp)
                        }
                    }
                }

                itemsIndexed(cart) { index, line ->
                    Card(
                        modifier = Modifier.fillMaxWidth(),
                        colors = CardDefaults.cardColors(containerColor =
MaterialTheme.colorScheme.surface),
                        shape = RoundedCornerShape(12.dp)
                    ) {
                        Column(modifier = Modifier.padding(12.dp)) {
                            Row(
                                modifier = Modifier.fillMaxWidth(),
                                horizontalArrangement = Arrangement.SpaceBetween,
                                verticalAlignment = Alignment.CenterVertically
                            ) {
                                Text(
                                    text = "Item #${index + 1}",
                                    style = MaterialTheme.typography.labelMedium,
                                    color = MaterialTheme.colorScheme.primary
                                )
                                IconButton(
                                    onClick = { cart.removeAt(index) },
                                    modifier = Modifier.size(24.dp)
                                ) {
                                    Icon(Icons.Filled.Delete, contentDescription = "Delete", tint =
Color(0xFFE57373))
                                }
                            }

                            Spacer(modifier = Modifier.height(8.dp))

                            // Product Dropdown
                            var prodMenuExpanded by remember { mutableStateOf(false) }
                            ExposedDropdownMenuBox(
                                expanded = prodMenuExpanded,
                                onExpandedChange = { prodMenuExpanded = !prodMenuExpanded }
                            ) {
                                OutlinedTextField(
                                    value = line.productName.ifBlank { "Select Product..." },
                                    onValueChange = {},
                                    readOnly = true,
                                    trailingIcon = { ExposedDropdownMenuDefaults.TrailingIcon(expanded =
prodMenuExpanded) },
                                    modifier = Modifier
                                        .fillMaxWidth()
```

```
                                        .menuAnchor(),
                                    shape = RoundedCornerShape(8.dp)
                                )
                                ExposedDropdownMenu(
                                    expanded = prodMenuExpanded,
                                    onDismissRequest = { prodMenuExpanded = false }
                                ) {
                                    allProducts.forEach { product ->
                                        DropdownMenuItem(
                                            text = {
                                                Text("${product.name} (Stock:
${product.currentSellableStock} pcs • ₹${product.sellingPricePiecePaise / 100.0}/pc)")
                                            },
                                            onClick = {
                                                line.productId = product.id
                                                line.productName = product.name
                                                line.packSize = product.packSize
                                                line.unitPriceStr = (product.sellingPricePiecePaise /
100.0).toString()
                                                prodMenuExpanded = false
                                            }
                                        )
                                    }
                                }
                            }

                            Spacer(modifier = Modifier.height(8.dp))

                            Row(
                                modifier = Modifier.fillMaxWidth(),
                                horizontalArrangement = Arrangement.spacedBy(8.dp)
                            ) {
                                OutlinedTextField(
                                    value = line.quantityStr,
                                    onValueChange = { line.quantityStr = it },
                                    label = { Text("Qty (Pieces)") },
                                    keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Number),
                                    modifier = Modifier.weight(1f),
                                    singleLine = true
                                )

                                OutlinedTextField(
                                    value = line.unitPriceStr,
                                    onValueChange = { line.unitPriceStr = it },
                                    label = { Text("Price (₹/pc)") },
                                    keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Decimal),
                                    modifier = Modifier.weight(1f),
                                    singleLine = true
                                )
                            }
                        }
                    }
                }

                // Wholesale Discount Card
                item {
                    Card(
                        modifier = Modifier.fillMaxWidth(),
                        colors = CardDefaults.cardColors(containerColor =
MaterialTheme.colorScheme.surface),
                        shape = RoundedCornerShape(14.dp)
                    ) {
                        Column(modifier = Modifier.padding(14.dp)) {
                            Text(
                                text = "Wholesale Discount",
                                style = MaterialTheme.typography.titleMedium.copy(fontWeight =
FontWeight.Bold),
                                color = MaterialTheme.colorScheme.onSurface
                            )
                            Spacer(modifier = Modifier.height(8.dp))

                            Row(
                                modifier = Modifier.fillMaxWidth(),
                                horizontalArrangement = Arrangement.spacedBy(8.dp)
                            ) {
                                Row(
                                    verticalAlignment = Alignment.CenterVertically,
                                    modifier = Modifier.clickable { discountMode = "NONE" }
                                ) {
                                    RadioButton(selected = discountMode == "NONE", onClick = { discountMode
= "NONE" })
                                    Text("None")
                                }
                                Row(
                                    verticalAlignment = Alignment.CenterVertically,
                                    modifier = Modifier.clickable { discountMode = "PERCENT" }
                                ) {
                                    RadioButton(selected = discountMode == "PERCENT", onClick = {
```

```
    discountMode = "PERCENT" })
                                    Text("Percent (%)")
                                }
                                Row(
                                    verticalAlignment = Alignment.CenterVertically,
                                    modifier = Modifier.clickable { discountMode = "FINAL_PRICE" }
                                ) {
                                    RadioButton(selected = discountMode == "FINAL_PRICE", onClick = {
    discountMode = "FINAL_PRICE" })
                                    Text("Final Price (₹)")
                                }
                            }

                            Spacer(modifier = Modifier.height(8.dp))

                            if (discountMode == "PERCENT") {
                                OutlinedTextField(
                                    value = discountPercentStr,
                                    onValueChange = { discountPercentStr = it },
                                    label = { Text("Discount Percentage (%)") },
                                    keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Decimal),
                                    modifier = Modifier.fillMaxWidth(),
                                    singleLine = true,
                                    shape = RoundedCornerShape(10.dp)
                                )
                            } else if (discountMode == "FINAL_PRICE") {
                                OutlinedTextField(
                                    value = agreedFinalPriceStr,
                                    onValueChange = { agreedFinalPriceStr = it },
                                    label = { Text("Agreed Final Bill Amount (₹)") },
                                    keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Decimal),
                                    modifier = Modifier.fillMaxWidth(),
                                    singleLine = true,
                                    shape = RoundedCornerShape(10.dp)
                                )
                            }

                            if (discountValuePaise > 0L) {
                                Spacer(modifier = Modifier.height(8.dp))
                                Text(
                                    text = "Discount:
    -${CostCalculator.formatPaiseToRupees(discountValuePaise)} (${String.format("%.1f",
    computedDiscountPercent)}%)",
                                    style = MaterialTheme.typography.bodySmall.copy(fontWeight =
    FontWeight.Bold),
                                    color = MaterialTheme.colorScheme.primary

                                )
                            }
                        }
                    }
                }

                // Payment Summary & Tender Card
                item {
                    Card(
                        modifier = Modifier.fillMaxWidth(),
                        colors = CardDefaults.cardColors(containerColor =
    MaterialTheme.colorScheme.surface),
                        shape = RoundedCornerShape(14.dp)
                    ) {
                        Column(modifier = Modifier.padding(14.dp)) {
                            Text(
                                text = "Payment & Tender",
                                style = MaterialTheme.typography.titleMedium.copy(fontWeight =
    FontWeight.Bold),
                                color = MaterialTheme.colorScheme.onSurface
                            )
                            Spacer(modifier = Modifier.height(10.dp))

                            Row(
                                modifier = Modifier.fillMaxWidth(),
                                horizontalArrangement = Arrangement.SpaceBetween
                            ) {
                                Text("Gross Subtotal:", color = MaterialTheme.colorScheme.onSurfaceVariant)
                                Text(CostCalculator.formatPaiseToRupees(grossTotalPaise), fontWeight =
    FontWeight.Bold)
                            }

                            if (discountValuePaise > 0) {
                                Spacer(modifier = Modifier.height(4.dp))
                                Row(
                                    modifier = Modifier.fillMaxWidth(),
                                    horizontalArrangement = Arrangement.SpaceBetween
                                ) {
                                    Text("Discount:", color = MaterialTheme.colorScheme.onSurfaceVariant)
                                    Text("-${CostCalculator.formatPaiseToRupees(discountValuePaise)}", color
     = MaterialTheme.colorScheme.primary)
                                }
                            }
```

```
497
498                             Spacer(modifier = Modifier.height(4.dp))
499                             Row(
500                                 modifier = Modifier.fillMaxWidth(),
501                                 horizontalArrangement = Arrangement.SpaceBetween
502                             ) {
503                                 Text("Final Net Bill:", fontWeight = FontWeight.Bold, fontSize = 16.sp)
504                                 Text(CostCalculator.formatPaiseToRupees(netTotalPaise), fontWeight =
    FontWeight.Bold, fontSize = 18.sp, color = MaterialTheme.colorScheme.primary)
505                             }
506
507                             Spacer(modifier = Modifier.height(12.dp))
508
509                             OutlinedTextField(
510                                 value = amountReceivedStr,
511                                 onValueChange = { amountReceivedStr = it },
512                                 label = { Text("Amount Received (₹)") },
513                                 placeholder = { Text(CostCalculator.formatPaiseToRupees(netTotalPaise)) },
514                                 keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Decimal),
515                                 modifier = Modifier.fillMaxWidth(),
516                                 singleLine = true,
517                                 shape = RoundedCornerShape(10.dp)
518                             )
519
520                             // Short Payment Notice
521                             if (isShortPayment) {
522                                 Spacer(modifier = Modifier.height(8.dp))
523                                 Text(
524                                     text = "Short payment by
    ${CostCalculator.formatPaiseToRupees(balanceOwed)}",
525                                     style = MaterialTheme.typography.bodySmall.copy(fontWeight =
    FontWeight.Bold),
526                                     color = Color(0xFFE4B765)
527                                 )
528                             }
529                         }
530                     }
531                 }
532
533                 // Save Sale Button
534                 item {
535                     Button(
536                         onClick = {
537                             if (lineItems.isEmpty()) return@Button
538
539                             if (isShortPayment) {
540                                 showShortPaymentDialog = true
541                             } else {
542                                 viewModel.recordSale(
543                                     customerId = selectedCustomer?.id,
544                                     customerName = selectedCustomer?.name,
545                                     items = lineItems,
546                                     discountType = discountMode,
547                                     discountInput = if (discountMode == "PERCENT")
    discountPercentStr.toDoubleOrNull() ?: 0.0 else (discountValuePaise / 100.0),
548                                     amountReceivedPaise = amountReceivedPaise,
549                                     paymentMethod = paymentMethod,
550                                     notes = notes.trim().ifBlank { null },
551                                     onSuccess = { sale -> completedSale = sale }
552                                 )
553                             }
554                         },
555                         enabled = lineItems.isNotEmpty(),
556                         modifier = Modifier
557                             .fillMaxWidth()
558                             .height(54.dp),
559                         shape = RoundedCornerShape(12.dp),
560                         colors = ButtonDefaults.buttonColors(
561                             containerColor = MaterialTheme.colorScheme.primary,
562                             contentColor = MaterialTheme.colorScheme.onPrimary
563                         )
564                     ) {
565                         Text(
566                             text = "Save Sale & Issue Receipt",
567                             style = MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.Bold)
568                         )
569                     }
570                     Spacer(modifier = Modifier.height(24.dp))
571                 }
572             }
573         }
574     }
575
576     // Short Payment Dialog (Spec requirement: Give discount vs Keep as credit)
577     if (showShortPaymentDialog) {
578         AlertDialog(
579             onDismissRequest = { showShortPaymentDialog = false },
580             title = { Text("Payment is Short") },
```

```
            text = {
                Column {
                    Text("Total due: ${CostCalculator.formatPaiseToRupees(netTotalPaise)}")
                    Text("Received: ${CostCalculator.formatPaiseToRupees(amountReceivedPaise)}")
                    Text(
                        "Remaining: ${CostCalculator.formatPaiseToRupees(balanceOwed)}",
                        fontWeight = FontWeight.Bold,
                        color = Color(0xFFE4B765)
                    )
                    Spacer(modifier = Modifier.height(12.dp))
                    Text("How would you like to handle the remaining
${CostCalculator.formatPaiseToRupees(balanceOwed)}?")
                }
            },
            confirmButton = {
                Button(
                    onClick = {
                        // Option 1: Give discount (adjusts final price so balance = 0)
                        val extraDiscount = balanceOwed
                        val totalDisc = discountValuePaise + extraDiscount
                        showShortPaymentDialog = false

                        viewModel.recordSale(
                            customerId = selectedCustomer?.id,
                            customerName = selectedCustomer?.name,
                            items = lineItems,
                            discountType = "FIXED",
                            discountInput = totalDisc / 100.0,
                            amountReceivedPaise = amountReceivedPaise,
                            paymentMethod = paymentMethod,
                            notes = "Short payment discounted. $notes",
                            onSuccess = { sale -> completedSale = sale }
                        )
                    }
                ) {
                    Text("Give Discount")
                }
            },
            dismissButton = {
                Button(
                    onClick = {
                        // Option 2: Keep remaining as credit
                        showShortPaymentDialog = false
                        viewModel.recordSale(
                            customerId = selectedCustomer?.id,
                            customerName = selectedCustomer?.name ?: "Credit Customer",
                            items = lineItems,
                            discountType = discountMode,
                            discountInput = if (discountMode == "PERCENT")
discountPercentStr.toDoubleOrNull() ?: 0.0 else (discountValuePaise / 100.0),
                            amountReceivedPaise = amountReceivedPaise,
                            paymentMethod = "CREDIT",
                            notes = "Credit sale balance ${CostCalculator.formatPaiseToRupees(balanceOwed)}.
 $notes",
                            onSuccess = { sale -> completedSale = sale }
                        )
                    },
                    colors = ButtonDefaults.buttonColors(containerColor =
MaterialTheme.colorScheme.surfaceVariant, contentColor = MaterialTheme.colorScheme.onSurface)
                ) {
                    Text("Keep as Credit")
                }
            }
        )
    }

    // Receipt Completed Dialog
    if (completedSale != null) {
        val sale = completedSale!!
        AlertDialog(
            onDismissRequest = {
                completedSale = null
                onNavigateBack()
            },
            title = { Text("Sale Saved! Receipt: ${sale.receiptNumber}") },
            text = {
                Column {
                    Text("Amount Paid: ${CostCalculator.formatPaiseToRupees(sale.amountReceivedPaise)}")
                    if (sale.remainingCreditPaise > 0) {
                        Text(
                            "Credit Balance:
${CostCalculator.formatPaiseToRupees(sale.remainingCreditPaise)}",
                            fontWeight = FontWeight.Bold,
                            color = Color(0xFFE4B765)
                        )
                    }
                    Spacer(modifier = Modifier.height(12.dp))
                    Text("Stock and account balances updated together offline.")
```

```
                }
            },
            confirmButton = {
                Button(
                    onClick = {
                        val text = viewModel.receiptService.generateReceiptText(sale, emptyList())
                        viewModel.receiptService.shareReceipt(text)
                    }
                ) {
                    Icon(Icons.Filled.Share, contentDescription = null, modifier = Modifier.size(18.dp))
                    Spacer(modifier = Modifier.width(6.dp))
                    Text("Share WhatsApp")
                }
            },
            dismissButton = {
                OutlinedButton(
                    onClick = {
                        completedSale = null
                        onNavigateBack()
                    }
                ) {
                    Text("Done")
                }
            }
        )
    }
}
```

app\src\main\java\com\alieducational\app\ui\settings\SettingsScreen.kt

```
package com.alieducational.app.ui.settings

import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.verticalScroll
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.ArrowBack
import androidx.compose.material.icons.filled.DarkMode
import androidx.compose.material.icons.filled.Devices
import androidx.compose.material.icons.filled.Fingerprint
import androidx.compose.material.icons.filled.Sync
import androidx.compose.material3.AlertDialog
import androidx.compose.material3.Button
import androidx.compose.material3.ButtonDefaults
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.FilterChip
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.RadioButton
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Surface
import androidx.compose.material3.Switch
import androidx.compose.material3.SwitchDefaults
import androidx.compose.material3.Text
import androidx.compose.material3.TextButton
import androidx.compose.material3.TopAppBar
import androidx.compose.material3.TopAppBarDefaults
import androidx.compose.runtime.Composable
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.alieducational.app.data.service.SyncConnectionState
import com.alieducational.app.data.service.TwoPhoneSyncService
import android.appwidget.AppWidgetManager
import android.content.ComponentName
import androidx.compose.material.icons.filled.AddCircleOutline
import androidx.compose.material.icons.filled.Widgets
import androidx.compose.ui.platform.LocalContext
import com.alieducational.app.widget.AliSlimBarWidgetProvider
import com.alieducational.app.ui.AliViewModel

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun SettingsScreen(
    viewModel: AliViewModel,
    onNavigateBack: () -> Unit,
    modifier: Modifier = Modifier
) {
    val isDarkTheme by viewModel.isDarkTheme.collectAsState()
    val isLockEnabled by viewModel.isLockEnabled.collectAsState()
    val lockTimeoutSeconds by viewModel.lockTimeoutSeconds.collectAsState()
    val deviceRole by viewModel.deviceRole.collectAsState()
    val pairedOwnerHost by viewModel.pairedOwnerHost.collectAsState()
    val syncState by viewModel.syncConnectionState.collectAsState()
    val pendingSyncCount by viewModel.pendingSyncCount.collectAsState()

    var showSwitchToBillingWarning by remember { mutableStateOf(false) }
    var hostAddressInput by remember(pairedOwnerHost) { mutableStateOf(pairedOwnerHost) }
```

```
    var syncResultMsg by remember { mutableStateOf<String?>(null) }

    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text("App Settings", fontWeight = FontWeight.Bold) },
                navigationIcon = {
                    IconButton(onClick = onNavigateBack) {
                        Icon(Icons.Filled.ArrowBack, contentDescription = "Back")
                    }
                },
                colors = TopAppBarDefaults.topAppBarColors(
                    containerColor = MaterialTheme.colorScheme.background,
                    titleContentColor = MaterialTheme.colorScheme.onBackground
                )
            )
        }
    ) { padding ->
        Surface(
            modifier = modifier
                .fillMaxSize()
                .padding(padding),
            color = MaterialTheme.colorScheme.background
        ) {
            Column(
                modifier = Modifier
                    .fillMaxSize()
                    .verticalScroll(rememberScrollState())
                    .padding(16.dp),
                verticalArrangement = Arrangement.spacedBy(14.dp)
            ) {
                // 1. Two-Phone Sync & Role Card
                Card(
                    modifier = Modifier.fillMaxWidth(),
                    colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surface),
                    shape = RoundedCornerShape(14.dp)
                ) {
                    Column(modifier = Modifier.padding(16.dp)) {
                        Row(verticalAlignment = Alignment.CenterVertically) {
                            Icon(Icons.Filled.Devices, contentDescription = null, tint =
    MaterialTheme.colorScheme.primary, modifier = Modifier.size(24.dp))
                            Spacer(modifier = Modifier.width(10.dp))
                            Column {
                                Text("Two-Phone Billing & Sync", style =
    MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.Bold))
                                Text("Share database over Wi-Fi / Hotspot", style =
    MaterialTheme.typography.bodySmall, color = MaterialTheme.colorScheme.onSurfaceVariant)
                            }
                        }

                        Spacer(modifier = Modifier.height(14.dp))
                        Text("Current Phone Role", style = MaterialTheme.typography.labelMedium, fontWeight
    = FontWeight.Bold)
                        Spacer(modifier = Modifier.height(6.dp))

                        Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement =
    Arrangement.spacedBy(10.dp)) {
                            FilterChip(
                                selected = deviceRole == "OWNER",
                                onClick = {
                                    if (deviceRole != "OWNER") {
                                        viewModel.setDeviceRole("OWNER")
                                    }
                                },
                                label = { Text("Owner (Main)") },
                                modifier = Modifier.weight(1f)
                            )
                            FilterChip(
                                selected = deviceRole == "BILLING",
                                onClick = {
                                    if (deviceRole != "BILLING") {
                                        showSwitchToBillingWarning = true
                                    }
                                },
                                label = { Text("Billing Device") },
                                modifier = Modifier.weight(1f)
                            )
                        }

                        Spacer(modifier = Modifier.height(14.dp))

                        if (deviceRole == "OWNER") {
                            // OWNER HOST CONTROLS
                            val localIp = remember { viewModel.twoPhoneSyncService.getLocalIpAddress() }
                            Card(
                                modifier = Modifier.fillMaxWidth(),
                                colors = CardDefaults.cardColors(containerColor =
    MaterialTheme.colorScheme.surfaceVariant),
```

```
                                shape = RoundedCornerShape(10.dp)
                            ) {
                                Column(modifier = Modifier.padding(12.dp)) {
                                    Text("Main Phone Host Info", fontWeight = FontWeight.Bold, fontSize =
13.sp)
                                    Spacer(modifier = Modifier.height(4.dp))
                                    Text("IP Address: $localIp (Port: 8080)", fontSize = 12.sp, color =
MaterialTheme.colorScheme.primary, fontWeight = FontWeight.SemiBold)
                                    Spacer(modifier = Modifier.height(6.dp))
                                    Text(
                                        "Keep this phone connected to the shop Wi-Fi or start its personal
hotspot. On the secondary phone, select 'Billing Device' and enter this IP address.",
                                        fontSize = 11.sp,
                                        color = MaterialTheme.colorScheme.onSurfaceVariant
                                    )
                                }
                            }

                            Spacer(modifier = Modifier.height(10.dp))
                            Row(
                                modifier = Modifier.fillMaxWidth(),
                                horizontalArrangement = Arrangement.SpaceBetween,
                                verticalAlignment = Alignment.CenterVertically
                            ) {
                                Text(
                                    text = if (syncState is SyncConnectionState.Hosting) "Server Active"
else "Server Inactive",
                                    fontWeight = FontWeight.Bold,
                                    fontSize = 13.sp,
                                    color = if (syncState is SyncConnectionState.Hosting) Color(0xFF75B890)
else MaterialTheme.colorScheme.onSurfaceVariant
                                )
                                Button(
                                    onClick = {
                                        if (syncState is SyncConnectionState.Hosting) {
                                            viewModel.stopSyncServer()
                                        } else {
                                            viewModel.startSyncServer(8080)
                                        }
                                    },
                                    colors = ButtonDefaults.buttonColors(
                                        containerColor = if (syncState is SyncConnectionState.Hosting)
MaterialTheme.colorScheme.error else MaterialTheme.colorScheme.primary
                                    ),
                                    shape = RoundedCornerShape(8.dp)
                                ) {
                                    Text(if (syncState is SyncConnectionState.Hosting) "Stop Server" else
"Start Server")
                                }
                            }
                        } else {
                            // BILLING DEVICE CONTROLS
                            OutlinedTextField(
                                value = hostAddressInput,
                                onValueChange = { hostAddressInput = it },
                                label = { Text("Owner Phone IP Address") },
                                placeholder = { Text("e.g. 192.168.1.5") },
                                singleLine = true,
                                modifier = Modifier.fillMaxWidth(),
                                shape = RoundedCornerShape(10.dp)
                            )

                            Spacer(modifier = Modifier.height(8.dp))
                            Row(
                                modifier = Modifier.fillMaxWidth(),
                                horizontalArrangement = Arrangement.spacedBy(8.dp)
                            ) {
                                Button(
                                    onClick = {
                                        viewModel.setPairedOwnerHost(hostAddressInput.trim())
                                    },
                                    modifier = Modifier.weight(1f),
                                    shape = RoundedCornerShape(8.dp),
                                    colors = ButtonDefaults.buttonColors(containerColor =
MaterialTheme.colorScheme.surfaceVariant, contentColor = MaterialTheme.colorScheme.onSurface)
                                ) {
                                    Text("Save IP")
                                }

                                Button(
                                    onClick = {
                                        val host = hostAddressInput.trim()
                                        if (host.isNotBlank()) {
                                            viewModel.setPairedOwnerHost(host)
                                            viewModel.syncWithMainPhone(host) { success ->
                                                syncResultMsg = if (success) "Sync successful!" else "Sync
failed. Check IP & Wi-Fi."
                                            }
```

```
                                    }
                                },
                                modifier = Modifier.weight(1f),
                                shape = RoundedCornerShape(8.dp),
                                colors = ButtonDefaults.buttonColors(containerColor = MaterialTheme.colorScheme.primary)
                            ) {
                                Icon(Icons.Filled.Sync, contentDescription = null, modifier = Modifier.size(16.dp))
                                Spacer(modifier = Modifier.width(4.dp))
                                Text("Sync Now")
                            }
                        }

                        if (syncResultMsg != null) {
                            Spacer(modifier = Modifier.height(6.dp))
                            Text(
                                text = syncResultMsg!!,
                                fontSize = 12.sp,
                                fontWeight = FontWeight.Bold,
                                color = if (syncResultMsg!!.contains("successful")) Color(0xFF75B890) else MaterialTheme.colorScheme.error
                            )
                        }

                        if (pendingSyncCount > 0) {
                            Spacer(modifier = Modifier.height(6.dp))
                            Text(
                                text = "$pendingSyncCount bills waiting to sync to Main Phone",
                                fontSize = 12.sp,
                                color = Color(0xFFE4B765)
                            )
                        }
                    }
                }
            }

            // 2. Theme Card
            Card(
                modifier = Modifier.fillMaxWidth(),
                colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surface),
                shape = RoundedCornerShape(14.dp)
            ) {
                Column(modifier = Modifier.padding(16.dp)) {
                    Row(verticalAlignment = Alignment.CenterVertically) {
                        Icon(Icons.Filled.DarkMode, contentDescription = null, tint = MaterialTheme.colorScheme.primary)
                        Spacer(modifier = Modifier.width(10.dp))
                        Text("Appearance Theme", style = MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.Bold))
                    }
                    Spacer(modifier = Modifier.height(12.dp))
                    Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement = Arrangement.spacedBy(12.dp)) {
                        FilterChip(
                            selected = isDarkTheme,
                            onClick = { viewModel.setTheme("dark") },
                            label = { Text("Dark Theme (Default)") },
                            modifier = Modifier.weight(1f)
                        )
                        FilterChip(
                            selected = !isDarkTheme,
                            onClick = { viewModel.setTheme("light") },
                            label = { Text("Light Theme") },
                            modifier = Modifier.weight(1f)
                        )
                    }
                }
            }

            // 3. App Lock Card
            Card(
                modifier = Modifier.fillMaxWidth(),
                colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surface),
                shape = RoundedCornerShape(14.dp)
            ) {
                Column(modifier = Modifier.padding(16.dp)) {
                    Row(
                        modifier = Modifier.fillMaxWidth(),
                        verticalAlignment = Alignment.CenterVertically,
                        horizontalArrangement = Arrangement.SpaceBetween
                    ) {
                        Row(verticalAlignment = Alignment.CenterVertically) {
                            Icon(Icons.Filled.Fingerprint, contentDescription = null, tint = MaterialTheme.colorScheme.primary, modifier = Modifier.size(26.dp))
                            Spacer(modifier = Modifier.width(10.dp))
                            Column {
                                Text("Device Lock Protection", style =
```

```
    MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.Bold))
325                                 Text("Fingerprint or device PIN/pattern", style =
    MaterialTheme.typography.bodySmall, color = MaterialTheme.colorScheme.onSurfaceVariant)
326                             }
327                         }
328                         Switch(
329                             checked = isLockEnabled,
330                             onCheckedChange = { viewModel.setLockEnabled(it) },
331                             colors = SwitchDefaults.colors(
332                                 checkedThumbColor = MaterialTheme.colorScheme.primary,
333                                 checkedTrackColor = MaterialTheme.colorScheme.primaryContainer
334                             )
335                         )
336                     }
337
338                     if (isLockEnabled) {
339                         Spacer(modifier = Modifier.height(16.dp))
340                         Text("Inactivity Relock Timeout", style =
    MaterialTheme.typography.bodyMedium.copy(fontWeight = FontWeight.Bold))
341                         Spacer(modifier = Modifier.height(8.dp))
342
343                         val timeoutOptions = listOf(
344                             30 to "30 Seconds",
345                             60 to "1 Minute",
346                             120 to "2 Minutes (Default)",
347                             300 to "5 Minutes",
348                             600 to "10 Minutes"
349                         )
350
351                         timeoutOptions.forEach { (seconds, label) ->
352                             Row(
353                                 modifier = Modifier
354                                     .fillMaxWidth()
355                                     .clickable { viewModel.setLockTimeout(seconds) }
356                                     .padding(vertical = 4.dp),
357                                 verticalAlignment = Alignment.CenterVertically
358                             ) {
359                                 RadioButton(
360                                     selected = lockTimeoutSeconds == seconds,
361                                     onClick = { viewModel.setLockTimeout(seconds) }
362                                 )
363                                 Spacer(modifier = Modifier.width(8.dp))
364                                 Text(label, style = MaterialTheme.typography.bodyMedium)
365                             }
366                         }
367                     }
368                 }
369             }
370
371             // 4. Home Screen Widget Card
372             val context = LocalContext.current
373             val canPinWidget = android.os.Build.VERSION.SDK_INT >= android.os.Build.VERSION_CODES.O &&
374                 (context.getSystemService(AppWidgetManager::class.java)?.isRequestPinAppWidgetSupported
    == true)
375
376             if (canPinWidget) {
377                 Card(
378                     modifier = Modifier.fillMaxWidth(),
379                     colors = CardDefaults.cardColors(containerColor =
    MaterialTheme.colorScheme.surface),
380                     shape = RoundedCornerShape(14.dp)
381                 ) {
382                     Column(modifier = Modifier.padding(16.dp)) {
383                         Row(verticalAlignment = Alignment.CenterVertically) {
384                             Icon(Icons.Filled.Widgets, contentDescription = null, tint =
    MaterialTheme.colorScheme.primary, modifier = Modifier.size(24.dp))
385                             Spacer(modifier = Modifier.width(10.dp))
386                             Column {
387                                 Text("Home Screen Widget", style =
    MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.Bold))
388                                 Text("Slim bar with Cash In, Cash Out, New Bill & Add Stock", style =
    MaterialTheme.typography.bodySmall, color = MaterialTheme.colorScheme.onSurfaceVariant)
389                             }
390                         }
391                         Spacer(modifier = Modifier.height(12.dp))
392                         Button(
393                             onClick = {
394                                 val appWidgetManager =
    context.getSystemService(AppWidgetManager::class.java)
395                                 val myProvider = ComponentName(context,
    AliSlimBarWidgetProvider::class.java)
396                                 appWidgetManager?.requestPinAppWidget(myProvider, null, null)
397                             },
398                             modifier = Modifier.fillMaxWidth(),
399                             shape = RoundedCornerShape(8.dp),
400                             colors = ButtonDefaults.buttonColors(containerColor =
    MaterialTheme.colorScheme.primary)
401                         ) {
```

```
                                Icon(Icons.Filled.AddCircleOutline, contentDescription = null, modifier =
    Modifier.size(18.dp))
                                Spacer(modifier = Modifier.width(8.dp))
                                Text("Add Widget to Home Screen")
                            }
                        }
                    }
                }

                Spacer(modifier = Modifier.height(24.dp))
            }
        }
    }

    if (showSwitchToBillingWarning) {
        AlertDialog(
            onDismissRequest = { showSwitchToBillingWarning = false },
            title = { Text("Switch to Billing Device Role?", fontWeight = FontWeight.Bold) },
            text = {
                Text("This phone will be set to Staff Billing mode. It will pull stock and products from the
     Owner phone and record bills with the 'AE-B1-' prefix. Financial and profit reports will be hidden on this
    device.")
            },
            confirmButton = {
                Button(
                    onClick = {
                        viewModel.setDeviceRole("BILLING")
                        showSwitchToBillingWarning = false
                    },
                    colors = ButtonDefaults.buttonColors(containerColor = MaterialTheme.colorScheme.primary)
                ) {
                    Text("Confirm Switch")
                }
            },
            dismissButton = {
                TextButton(onClick = { showSwitchToBillingWarning = false }) {
                    Text("Cancel")
                }
            }
        )
    }
}
```

app\src\main\java\com\alieducational\app\ui\setup\SetupScreen.kt

```
package com.alieducational.app.ui.setup

import androidx.compose.foundation.Image
import androidx.compose.foundation.background
import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.verticalScroll
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Check
import androidx.compose.material.icons.filled.Fingerprint
import androidx.compose.material.icons.filled.Language
import androidx.compose.material.icons.filled.Storefront
import androidx.compose.material3.Button
import androidx.compose.material3.ButtonDefaults
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.Icon
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.Surface
import androidx.compose.material3.Switch
import androidx.compose.material3.SwitchDefaults
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.layout.ContentScale
import androidx.compose.ui.res.painterResource
import androidx.compose.ui.res.stringResource
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.alieducational.app.R
import com.alieducational.app.ui.AliViewModel

@Composable
fun SetupScreen(
    viewModel: AliViewModel,
    onSetupComplete: () -> Unit,
    modifier: Modifier = Modifier
) {
    var shopName by remember { mutableStateOf("Ali Educational") }
    var selectedLanguage by remember { mutableStateOf("en") }
    var enableLock by remember { mutableStateOf(false) }

    Surface(
        modifier = modifier.fillMaxSize(),
        color = MaterialTheme.colorScheme.background
    ) {
        Column(
            modifier = Modifier
                .fillMaxSize()
                .verticalScroll(rememberScrollState())
                .padding(24.dp),
            horizontalAlignment = Alignment.CenterHorizontally
        ) {
            Spacer(modifier = Modifier.height(16.dp))

            Image(
                painter = painterResource(id = R.drawable.ali_logo_dark),
                contentDescription = "Ali Educational Logo",
                contentScale = ContentScale.Fit,
                modifier = Modifier
                    .width(180.dp)
                    .height(95.dp)
```

```
            )

            Spacer(modifier = Modifier.height(16.dp))

            Text(
                text = stringResource(R.string.setup_welcome),
                style = MaterialTheme.typography.headlineMedium.copy(fontWeight = FontWeight.Bold),
                color = MaterialTheme.colorScheme.onBackground
            )

            Text(
                text = stringResource(R.string.setup_subtitle),
                style = MaterialTheme.typography.bodyMedium,
                color = MaterialTheme.colorScheme.onSurfaceVariant
            )

            Spacer(modifier = Modifier.height(32.dp))

            // Shop Name Card (English interface by default; Urdu/Arabic Unicode supported in product names)
            Card(
                modifier = Modifier.fillMaxWidth(),
                colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surface),
                shape = RoundedCornerShape(16.dp)
            ) {
                Column(modifier = Modifier.padding(16.dp)) {
                    Row(verticalAlignment = Alignment.CenterVertically) {
                        Icon(
                            imageVector = Icons.Filled.Storefront,
                            contentDescription = null,
                            tint = MaterialTheme.colorScheme.primary
                        )
                        Spacer(modifier = Modifier.width(8.dp))
                        Text(
                            text = stringResource(R.string.setup_shop_name),
                            style = MaterialTheme.typography.titleMedium.copy(fontWeight =
    FontWeight.SemiBold),
                            color = MaterialTheme.colorScheme.onSurface
                        )
                    }

                    Spacer(modifier = Modifier.height(12.dp))

                    OutlinedTextField(
                        value = shopName,
                        onValueChange = { shopName = it },
                        modifier = Modifier.fillMaxWidth(),
                        singleLine = true,
                        shape = RoundedCornerShape(10.dp)
                    )
                }
            }

            Spacer(modifier = Modifier.height(16.dp))

            // 3. Biometric Lock Card
            Card(
                modifier = Modifier.fillMaxWidth(),
                colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surface),
                shape = RoundedCornerShape(16.dp)
            ) {
                Row(
                    modifier = Modifier
                        .fillMaxWidth()
                        .padding(16.dp),
                    verticalAlignment = Alignment.CenterVertically,
                    horizontalArrangement = Arrangement.SpaceBetween
                ) {
                    Row(
                        modifier = Modifier.weight(1f),
                        verticalAlignment = Alignment.CenterVertically
                    ) {
                        Icon(
                            imageVector = Icons.Filled.Fingerprint,
                            contentDescription = null,
                            tint = MaterialTheme.colorScheme.primary,
                            modifier = Modifier.size(32.dp)
                        )
                        Spacer(modifier = Modifier.width(12.dp))
                        Column {
                            Text(
                                text = stringResource(R.string.setup_enable_lock),
                                style = MaterialTheme.typography.bodyLarge.copy(fontWeight =
    FontWeight.SemiBold),
                                color = MaterialTheme.colorScheme.onSurface
                            )
                            Text(
                                text = stringResource(R.string.setup_enable_lock_desc),
                                style = MaterialTheme.typography.bodySmall,
```

```
                                color = MaterialTheme.colorScheme.onSurfaceVariant
                            )
                        }
                    }

                    Switch(
                        checked = enableLock,
                        onCheckedChange = { enableLock = it },
                        colors = SwitchDefaults.colors(
                            checkedThumbColor = MaterialTheme.colorScheme.primary,
                            checkedTrackColor = MaterialTheme.colorScheme.primaryContainer
                        )
                    )
                }
            }

            Spacer(modifier = Modifier.height(36.dp))

            // Complete Setup Button
            Button(
                onClick = {
                    viewModel.completeSetup(
                        shopName = shopName.ifBlank { "Ali Educational" },
                        language = selectedLanguage,
                        lockEnabled = enableLock
                    )
                    onSetupComplete()
                },
                modifier = Modifier
                    .fillMaxWidth()
                    .height(54.dp),
                shape = RoundedCornerShape(12.dp),
                colors = ButtonDefaults.buttonColors(
                    containerColor = MaterialTheme.colorScheme.primary,
                    contentColor = MaterialTheme.colorScheme.onPrimary
                )
            ) {
                Text(
                    text = stringResource(R.string.setup_get_started),
                    style = MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.Bold)
                )
            }

            Spacer(modifier = Modifier.height(24.dp))
        }
    }
}
```

app\src\main\java\com\alieducational\app\ui\stock\ImportCatalogScreen.kt

```
package com.alieducational.app.ui.stock

import android.net.Uri
import androidx.activity.compose.rememberLauncherForActivityResult
import androidx.activity.result.contract.ActivityResultContracts
import androidx.compose.foundation.BorderStroke
import androidx.compose.foundation.background
import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.itemsIndexed
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.ArrowBack
import androidx.compose.material.icons.filled.CheckCircle
import androidx.compose.material.icons.filled.ErrorOutline
import androidx.compose.material.icons.filled.FileOpen
import androidx.compose.material.icons.filled.Layers
import androidx.compose.material.icons.filled.Tune
import androidx.compose.material.icons.filled.UploadFile
import androidx.compose.material.icons.filled.Warning
import androidx.compose.material3.Button
import androidx.compose.material3.ButtonDefaults
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.CircularProgressIndicator
import androidx.compose.material3.DropdownMenuItem
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.ExposedDropdownMenuBox
import androidx.compose.material3.ExposedDropdownMenuDefaults
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.RadioButton
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.material3.TopAppBar
import androidx.compose.material3.TopAppBarDefaults
import androidx.compose.runtime.Composable
import androidx.compose.runtime.LaunchedEffect
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableIntStateOf
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.alieducational.app.data.service.ColumnMapping
import com.alieducational.app.ui.AliViewModel

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun ImportCatalogScreen(
    viewModel: AliViewModel,
    onNavigateBack: () -> Unit,
    modifier: Modifier = Modifier
) {
    val context = LocalContext.current
    var selectedUri by remember { mutableStateOf<Uri?>(null) }
    var selectedFileName by remember { mutableStateOf("") }
    var selectedSheetIndex by remember { mutableIntStateOf(0) }

    val previewData by viewModel.previewData.collectAsState()
    val importResult by viewModel.importResult.collectAsState()
```

```
    val isImporting by viewModel.isImporting.collectAsState()

    var nameColIndex by remember { mutableIntStateOf(0) }
    var categoryColIndex by remember { mutableIntStateOf(-1) }
    var brandColIndex by remember { mutableIntStateOf(-1) }
    var skuColIndex by remember { mutableIntStateOf(-1) }
    var barcodeColIndex by remember { mutableIntStateOf(-1) }
    var openingQtyColIndex by remember { mutableIntStateOf(-1) }
    var purchaseCostColIndex by remember { mutableIntStateOf(-1) }
    var sellingPieceColIndex by remember { mutableIntStateOf(-1) }
    var updateExisting by remember { mutableStateOf(false) }

    // Auto-detect headers whenever previewData changes
    LaunchedEffect(previewData) {
        val headers = previewData?.headers ?: return@LaunchedEffect
        headers.forEachIndexed { idx, rawHeader ->
            val h = rawHeader.lowercase().trim()
            when {
                nameColIndex <= 0 && (h.contains("name") || h.contains("item") || h.contains("product") || h.contains("title")) ->
                    nameColIndex = idx
                categoryColIndex == -1 && (h.contains("category") || h.contains("group") || h.contains("type")) ->
                    categoryColIndex = idx
                brandColIndex == -1 && (h.contains("brand") || h.contains("company") || h.contains("make")) ->
                    brandColIndex = idx
                skuColIndex == -1 && (h.contains("sku") || h.contains("code") || h.contains("item_code") || h.contains("product_code")) ->
                    skuColIndex = idx
                barcodeColIndex == -1 && (h.contains("barcode") || h.contains("upc") || h.contains("ean")) ->
                    barcodeColIndex = idx
                openingQtyColIndex == -1 && (h.contains("qty") || h.contains("quantity") || h.contains("stock") || h.contains("opening")) ->
                    openingQtyColIndex = idx
                purchaseCostColIndex == -1 && (h.contains("cost") || h.contains("purchase") || h.contains("buy")) ->
                    purchaseCostColIndex = idx
                sellingPieceColIndex == -1 && (h.contains("sell") || h.contains("mrp") || h.contains("rate") || h.contains("price")) ->
                    sellingPieceColIndex = idx
            }
        }
    }

    val filePicker = rememberLauncherForActivityResult(
        contract = ActivityResultContracts.OpenDocument()
    ) { uri ->
        if (uri != null) {
            selectedUri = uri
            val cursor = context.contentResolver.query(uri, null, null, null, null)
            val name = cursor?.use {
                if (it.moveToFirst()) {
                    val nameIndex = it.getColumnIndex(android.provider.OpenableColumns.DISPLAY_NAME)
                    if (nameIndex != -1) it.getString(nameIndex) else "spreadsheet"
                } else "spreadsheet"
            } ?: "spreadsheet"

            selectedFileName = name
            selectedSheetIndex = 0
            viewModel.previewSpreadsheet(uri, name, 0)
        }
    }

    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text("Import Product Catalog", fontWeight = FontWeight.Bold) },
                navigationIcon = {
                    IconButton(onClick = onNavigateBack) {
                        Icon(Icons.Filled.ArrowBack, contentDescription = "Back")
                    }
                },
                colors = TopAppBarDefaults.topAppBarColors(
                    containerColor = MaterialTheme.colorScheme.background,
                    titleContentColor = MaterialTheme.colorScheme.onBackground
                )
            )
        }
    ) { padding ->
        Surface(
            modifier = modifier
                .fillMaxSize()
                .padding(padding),
            color = MaterialTheme.colorScheme.background
        ) {
            LazyColumn(
```

```
                modifier = Modifier
                    .fillMaxSize()
                    .padding(horizontal = 16.dp),
                verticalArrangement = Arrangement.spacedBy(14.dp)
            ) {
                // File Selection Card
                item {
                    Spacer(modifier = Modifier.height(4.dp))
                    Card(
                        modifier = Modifier.fillMaxWidth(),
                        colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surface),
                        shape = RoundedCornerShape(14.dp)
                    ) {
                        Column(modifier = Modifier.padding(16.dp)) {
                            Text(
                                text = "Select Excel or CSV Spreadsheet",
                                style = MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.Bold),
                                color = MaterialTheme.colorScheme.onSurface
                            )
                            Spacer(modifier = Modifier.height(6.dp))

                            Text(
                                text = "Accepts modern Excel (.xlsx), legacy Excel (.xls), and (.csv) files. Safe and crash-proof for up to 3,000 products.",
                                style = MaterialTheme.typography.bodySmall,
                                color = MaterialTheme.colorScheme.onSurfaceVariant
                            )
                            Spacer(modifier = Modifier.height(14.dp))

                            Button(
                                onClick = {
                                    filePicker.launch(
                                        arrayOf(
                                            "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
                                            "application/vnd.ms-excel",
                                            "text/csv",
                                            "text/comma-separated-values",
                                            "*/*"
                                        )
                                    )
                                },
                                shape = RoundedCornerShape(10.dp)
                            ) {
                                Icon(Icons.Filled.FileOpen, contentDescription = null, modifier = Modifier.size(18.dp))
                                Spacer(modifier = Modifier.width(6.dp))
                                Text("Choose File from Phone")
                            }

                            if (selectedFileName.isNotBlank()) {
                                Spacer(modifier = Modifier.height(10.dp))
                                Text(
                                    text = "Selected File: $selectedFileName",
                                    style = MaterialTheme.typography.bodyMedium.copy(fontWeight = FontWeight.Bold),
                                    color = MaterialTheme.colorScheme.primary
                                )
                            }
                        }
                    }
                }

                // Error Message Card (if file invalid / encrypted / corrupt / cash ledger)
                if (previewData != null && previewData?.errorMessage != null) {
                    item {
                        val isCash = previewData?.isCashLedger == true
                        val cardBg = if (isCash) Color(0xFFF9A825).copy(alpha = 0.15f) else Color(0xFFE57373).copy(alpha = 0.15f)
                        val iconColor = if (isCash) Color(0xFFF9A825) else Color(0xFFE57373)
                        val titleText = if (isCash) "Cash In/Out Ledger Detected" else "Cannot Read Spreadsheet"

                        Card(
                            modifier = Modifier.fillMaxWidth(),
                            colors = CardDefaults.cardColors(containerColor = cardBg),
                            shape = RoundedCornerShape(12.dp)
                        ) {
                            Column(modifier = Modifier.padding(14.dp)) {
                                Row(verticalAlignment = Alignment.CenterVertically) {
                                    Icon(
                                        if (isCash) Icons.Filled.Warning else Icons.Filled.ErrorOutline,
                                        contentDescription = null,
                                        tint = iconColor
                                    )
                                    Spacer(modifier = Modifier.width(8.dp))
                                    Text(titleText, fontWeight = FontWeight.Bold, color = iconColor)
```

```
                                }
                                Spacer(modifier = Modifier.height(6.dp))
                                Text(
                                    text = previewData!!.errorMessage!!,
                                    style = MaterialTheme.typography.bodyMedium,
                                    color = MaterialTheme.colorScheme.onSurface
                                )
                                Spacer(modifier = Modifier.height(8.dp))
                                Text(
                                    text = if (isCash) "To import transactions from this file, please open the Cash In / Out Tracker from the Home screen."
                                    else "Tip: Ensure your file contains headers like 'Product Name', 'Category', 'Selling Price', 'Stock Qty', and is not password protected.",
                                    style = MaterialTheme.typography.bodySmall,
                                    color = MaterialTheme.colorScheme.onSurfaceVariant
                                )
                            }
                        }
                    }
                }

                // Multiple Worksheets Selector
                if (previewData != null && previewData!!.sheetNames.size > 1 && !previewData!!.isCashLedger) {
                    item {
                        Card(
                            modifier = Modifier.fillMaxWidth(),
                            colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surface),
                            shape = RoundedCornerShape(12.dp)
                        ) {
                            Column(modifier = Modifier.padding(14.dp)) {
                                Row(verticalAlignment = Alignment.CenterVertically) {
                                    Icon(Icons.Filled.Layers, contentDescription = null, tint = MaterialTheme.colorScheme.primary, modifier = Modifier.size(20.dp))
                                    Spacer(modifier = Modifier.width(8.dp))
                                    Text("Workbook Has Multiple Sheets", fontWeight = FontWeight.Bold)
                                }
                                Spacer(modifier = Modifier.height(6.dp))
                                Text("Select which worksheet contains your product catalog:", style = MaterialTheme.typography.bodySmall)
                                Spacer(modifier = Modifier.height(8.dp))

                                ColumnSelectorDropdown(
                                    label = "Active Sheet",
                                    options = previewData!!.sheetNames,
                                    selectedIndex = selectedSheetIndex,
                                    onSelect = { newIdx ->
                                        selectedSheetIndex = newIdx
                                        val uri = selectedUri
                                        if (uri != null) {
                                            viewModel.previewSpreadsheet(uri, selectedFileName, newIdx)
                                        }
                                    }
                                )
                            }
                        }
                    }
                }

                // Preview & Column Mapping Section (Only show if valid product stock spreadsheet)
                if (previewData != null && previewData?.headers?.isNotEmpty() == true && !previewData!!.isCashLedger) {
                    val preview = previewData!!
                    val headersWithNone = listOf("None / Do Not Import") + preview.headers

                    item {
                        Card(
                            modifier = Modifier.fillMaxWidth(),
                            colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surface),
                            shape = RoundedCornerShape(14.dp)
                        ) {
                            Column(modifier = Modifier.padding(16.dp)) {
                                Text(
                                    text = "Map Spreadsheet Columns",
                                    style = MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.Bold),
                                    color = MaterialTheme.colorScheme.onSurface
                                )
                                Spacer(modifier = Modifier.height(4.dp))
                                Text(
                                    text = "We automatically detected matching columns. Please confirm or adjust:",
                                    style = MaterialTheme.typography.bodySmall,
                                    color = MaterialTheme.colorScheme.onSurfaceVariant
                                )
                                Spacer(modifier = Modifier.height(10.dp))
```

```
                                ColumnSelectorDropdown(
                                    label = "Product Name *",
                                    options = headersWithNone,
                                    selectedIndex = nameColIndex + 1,
                                    onSelect = { nameColIndex = it - 1 }
                                )

                                ColumnSelectorDropdown(
                                    label = "Category",
                                    options = headersWithNone,
                                    selectedIndex = categoryColIndex + 1,
                                    onSelect = { categoryColIndex = it - 1 }
                                )

                                ColumnSelectorDropdown(
                                    label = "Brand",
                                    options = headersWithNone,
                                    selectedIndex = brandColIndex + 1,
                                    onSelect = { brandColIndex = it - 1 }
                                )

                                ColumnSelectorDropdown(
                                    label = "SKU / Item Code",
                                    options = headersWithNone,
                                    selectedIndex = skuColIndex + 1,
                                    onSelect = { skuColIndex = it - 1 }
                                )

                                ColumnSelectorDropdown(
                                    label = "Barcode",
                                    options = headersWithNone,
                                    selectedIndex = barcodeColIndex + 1,
                                    onSelect = { barcodeColIndex = it - 1 }
                                )

                                ColumnSelectorDropdown(
                                    label = "Opening Stock Quantity",
                                    options = headersWithNone,
                                    selectedIndex = openingQtyColIndex + 1,
                                    onSelect = { openingQtyColIndex = it - 1 }
                                )

                                ColumnSelectorDropdown(
                                    label = "Purchase Cost (₹)",
                                    options = headersWithNone,
                                    selectedIndex = purchaseCostColIndex + 1,
                                    onSelect = { purchaseCostColIndex = it - 1 }
                                )

                                ColumnSelectorDropdown(
                                    label = "Selling Price Piece (₹)",
                                    options = headersWithNone,
                                    selectedIndex = sellingPieceColIndex + 1,
                                    onSelect = { sellingPieceColIndex = it - 1 }
                                )
                            }
                        }
                    }

                    // Existing Products & Duplicates Policy Card
                    item {
                        Card(
                            modifier = Modifier.fillMaxWidth(),
                            colors = CardDefaults.cardColors(containerColor =
MaterialTheme.colorScheme.surface),
                            shape = RoundedCornerShape(14.dp)
                        ) {
                            Column(modifier = Modifier.padding(16.dp)) {
                                Row(verticalAlignment = Alignment.CenterVertically) {
                                    Icon(
                                        Icons.Filled.Tune,
                                        contentDescription = null,
                                        tint = MaterialTheme.colorScheme.primary,
                                        modifier = Modifier.size(20.dp)
                                    )
                                    Spacer(modifier = Modifier.width(8.dp))
                                    Text(
                                        text = "When Existing Item is Found",
                                        style = MaterialTheme.typography.titleMedium.copy(fontWeight =
FontWeight.Bold),
                                        color = MaterialTheme.colorScheme.onSurface
                                    )
                                }
                                Spacer(modifier = Modifier.height(6.dp))
                                Text(
                                    text = "Choose what happens if a spreadsheet row matches a product
already in your shop (by SKU, barcode, or name):",
```

```
                                    style = MaterialTheme.typography.bodySmall,
                                    color = MaterialTheme.colorScheme.onSurfaceVariant
                                )
                                Spacer(modifier = Modifier.height(12.dp))

                                // Option 1: Skip
                                Card(
                                    modifier = Modifier
                                        .fillMaxWidth()
                                        .clickable { updateExisting = false },
                                    colors = CardDefaults.cardColors(
                                        containerColor = if (!updateExisting)
MaterialTheme.colorScheme.primaryContainer.copy(alpha = 0.5f) else
MaterialTheme.colorScheme.surfaceVariant.copy(alpha = 0.4f)
                                    ),
                                    shape = RoundedCornerShape(10.dp),
                                    border = if (!updateExisting) BorderStroke(1.5.dp,
MaterialTheme.colorScheme.primary) else null
                                ) {
                                    Row(
                                        modifier = Modifier.padding(12.dp),
                                        verticalAlignment = Alignment.CenterVertically
                                    ) {
                                        RadioButton(
                                            selected = !updateExisting,
                                            onClick = { updateExisting = false }
                                        )
                                        Spacer(modifier = Modifier.width(8.dp))
                                        Column {
                                            Text("Skip existing items (Recommended)", fontWeight =
FontWeight.Bold, style = MaterialTheme.typography.bodyMedium)
                                            Text("Keep current catalog prices and stock untouched; only
import newly found products.", style = MaterialTheme.typography.bodySmall, color =
MaterialTheme.colorScheme.onSurfaceVariant)
                                        }
                                    }
                                }

                                Spacer(modifier = Modifier.height(8.dp))

                                // Option 2: Update
                                Card(
                                    modifier = Modifier
                                        .fillMaxWidth()
                                        .clickable { updateExisting = true },
                                    colors = CardDefaults.cardColors(
                                        containerColor = if (updateExisting)
MaterialTheme.colorScheme.primaryContainer.copy(alpha = 0.5f) else
MaterialTheme.colorScheme.surfaceVariant.copy(alpha = 0.4f)
                                    ),
                                    shape = RoundedCornerShape(10.dp),
                                    border = if (updateExisting) BorderStroke(1.5.dp,
MaterialTheme.colorScheme.primary) else null
                                ) {
                                    Row(
                                        modifier = Modifier.padding(12.dp),
                                        verticalAlignment = Alignment.CenterVertically
                                    ) {
                                        RadioButton(
                                            selected = updateExisting,
                                            onClick = { updateExisting = true }
                                        )
                                        Spacer(modifier = Modifier.width(8.dp))
                                        Column {
                                            Text("Update existing items", fontWeight = FontWeight.Bold,
style = MaterialTheme.typography.bodyMedium)
                                            Text("Update prices and moving cost; add incoming quantity to
current sellable stock.", style = MaterialTheme.typography.bodySmall, color =
MaterialTheme.colorScheme.onSurfaceVariant)
                                        }
                                    }
                                }
                            }
                        }
                    }

                    // Preview Rows
                    item {
                        Text(
                            text = "Sample Rows (${preview.previewRows.size} of
~${preview.totalEstimatedRows} total):",
                            style = MaterialTheme.typography.titleSmall.copy(fontWeight = FontWeight.Bold),
                            color = MaterialTheme.colorScheme.onBackground
                        )
                    }

                    itemsIndexed(preview.previewRows) { idx, row ->
                        Card(
                            modifier = Modifier.fillMaxWidth(),
```

```
                            colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surfaceVariant),
                            shape = RoundedCornerShape(8.dp)
                        ) {
                            Text(
                                text = "Row #${idx + 1}: ${row.joinToString(" | ")}",
                                style = MaterialTheme.typography.bodySmall,
                                modifier = Modifier.padding(10.dp)
                            )
                        }
                    }

                    // Execute Import Button with Loading State
                    item {
                        Button(
                            onClick = {
                                val uri = selectedUri ?: return@Button
                                val mapping = ColumnMapping(
                                    nameIndex = nameColIndex,
                                    categoryIndex = categoryColIndex,
                                    brandIndex = brandColIndex,
                                    skuIndex = skuColIndex,
                                    barcodeIndex = barcodeColIndex,
                                    openingQtyIndex = openingQtyColIndex,
                                    purchaseCostIndex = purchaseCostColIndex,
                                    sellingPricePieceIndex = sellingPieceColIndex
                                )
                                viewModel.executeImport(
                                    uri = uri,
                                    fileName = selectedFileName,
                                    sheetIndex = selectedSheetIndex,
                                    mapping = mapping,
                                    updateExisting = updateExisting
                                )
                            },
                            enabled = nameColIndex >= 0 && !isImporting,
                            modifier = Modifier
                                .fillMaxWidth()
                                .height(54.dp),
                            shape = RoundedCornerShape(12.dp),
                            colors = ButtonDefaults.buttonColors(
                                containerColor = MaterialTheme.colorScheme.primary,
                                contentColor = MaterialTheme.colorScheme.onPrimary
                            )
                        ) {
                            if (isImporting) {
                                CircularProgressIndicator(
                                    modifier = Modifier.size(24.dp),
                                    color = MaterialTheme.colorScheme.onPrimary,
                                    strokeWidth = 2.dp
                                )
                                Spacer(modifier = Modifier.width(10.dp))
                                Text("Importing Products...", fontWeight = FontWeight.Bold)
                            } else {
                                Icon(Icons.Filled.UploadFile, contentDescription = null)
                                Spacer(modifier = Modifier.width(8.dp))
                                Text("Start Catalog Import", fontWeight = FontWeight.Bold)
                            }
                        }
                    }
                }

                // Import Result Notice with Detailed Breakdown
                if (importResult != null) {
                    val res = importResult!!
                    item {
                        Card(
                            modifier = Modifier.fillMaxWidth(),
                            colors = CardDefaults.cardColors(
                                containerColor = if (res.isSuccess) Color(0xFF75B890).copy(alpha = 0.2f) else Color(0xFFE57373).copy(alpha = 0.2f)
                            ),
                            shape = RoundedCornerShape(12.dp)
                        ) {
                            Column(modifier = Modifier.padding(16.dp)) {
                                Row(verticalAlignment = Alignment.CenterVertically) {
                                    Icon(
                                        if (res.isSuccess) Icons.Filled.CheckCircle else Icons.Filled.Warning,
                                        contentDescription = null,
                                        tint = if (res.isSuccess) Color(0xFF75B890) else Color(0xFFE57373),
                                        modifier = Modifier.size(28.dp)
                                    )
                                    Spacer(modifier = Modifier.width(12.dp))
                                    Column {
                                        Text(
                                            if (res.isSuccess) "Import Completed!" else "Import Issue",
                                            fontWeight = FontWeight.Bold,
```

```
                                        color = MaterialTheme.colorScheme.onSurface
                                    )
                                    Text(
                                        "Total rows: ${res.totalRead}. New products imported: ${res.importedCount}.",
                                        style = MaterialTheme.typography.bodyMedium
                                    )
                                }
                            }

                            if (res.updatedCount > 0) {
                                Spacer(modifier = Modifier.height(8.dp))
                                Text(
                                    "Updated ${res.updatedCount} existing products with spreadsheet prices and stock:",
                                    style = MaterialTheme.typography.bodySmall.copy(fontWeight = FontWeight.Bold),
                                    color = Color(0xFF43A047)
                                )
                                res.updatedDetails.forEach { detail ->
                                    Text("• $detail", style = MaterialTheme.typography.bodySmall, color = MaterialTheme.colorScheme.onSurfaceVariant)
                                }
                            }

                            if (res.skippedDuplicatesCount > 0) {
                                Spacer(modifier = Modifier.height(8.dp))
                                Text(
                                    "Skipped ${res.skippedDuplicatesCount} existing products (catalog kept untouched):",
                                    style = MaterialTheme.typography.bodySmall.copy(fontWeight = FontWeight.Bold),
                                    color = Color(0xFFE4B765)
                                )
                                res.skippedDetails.forEach { detail ->
                                    Text("• $detail", style = MaterialTheme.typography.bodySmall, color = MaterialTheme.colorScheme.onSurfaceVariant)
                                }
                            }

                            if (res.errors.isNotEmpty()) {
                                Spacer(modifier = Modifier.height(8.dp))
                                Text(
                                    "Issues noted (${res.errors.size}):",
                                    style = MaterialTheme.typography.bodySmall.copy(fontWeight = FontWeight.Bold),
                                    color = Color(0xFFE57373)
                                )
                                res.errors.take(5).forEach { err ->
                                    Text("• $err", style = MaterialTheme.typography.bodySmall, color = MaterialTheme.colorScheme.onSurfaceVariant)
                                }
                            }
                        }
                    }
                }

                item { Spacer(modifier = Modifier.height(24.dp)) }
            }
        }
    }
}

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun ColumnSelectorDropdown(
    label: String,
    options: List<String>,
    selectedIndex: Int,
    onSelect: (Int) -> Unit
) {
    var expanded by remember { mutableStateOf(false) }
    val displayValue = if (selectedIndex in options.indices) options[selectedIndex] else "None / Do Not Import"

    Column(modifier = Modifier.padding(vertical = 4.dp)) {
        Text(
            text = label,
            style = MaterialTheme.typography.bodySmall.copy(fontWeight = FontWeight.Bold),
            color = MaterialTheme.colorScheme.onSurface
        )
        Spacer(modifier = Modifier.height(2.dp))
        ExposedDropdownMenuBox(
            expanded = expanded,
            onExpandedChange = { expanded = !expanded }
        ) {
            OutlinedTextField(
```

```
                value = displayValue,
                onValueChange = {},
                readOnly = true,
                trailingIcon = { ExposedDropdownMenuDefaults.TrailingIcon(expanded = expanded) },
                modifier = Modifier
                    .fillMaxWidth()
                    .menuAnchor(),
                shape = RoundedCornerShape(8.dp)
            )
            ExposedDropdownMenu(
                expanded = expanded,
                onDismissRequest = { expanded = false }
            ) {
                options.forEachIndexed { idx, opt ->
                    DropdownMenuItem(
                        text = { Text(opt) },
                        onClick = {
                            onSelect(idx)
                            expanded = false
                        }
                    )
                }
            }
        }
    }
}
```

app\src\main\java\com\alieducational\app\ui\stock\ReceiveStockScreen.kt

```
package com.alieducational.app.ui.stock

import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.itemsIndexed
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Add
import androidx.compose.material.icons.filled.ArrowBack
import androidx.compose.material.icons.filled.Delete
import androidx.compose.material3.Button
import androidx.compose.material3.ButtonDefaults
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.DropdownMenuItem
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.ExposedDropdownMenuBox
import androidx.compose.material3.ExposedDropdownMenuDefaults
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedButton
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.material3.TopAppBar
import androidx.compose.material3.TopAppBarDefaults
import androidx.compose.runtime.Composable
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateListOf
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.alieducational.app.data.local.entity.SupplierEntity
import com.alieducational.app.data.repository.PurchaseItemParam
import com.alieducational.app.domain.calculation.CostCalculator
import com.alieducational.app.domain.calculation.PurchaseLineInput
import com.alieducational.app.ui.AliViewModel

data class ReceiveLineState(
    var productId: String = "",
    var productName: String = "",
    var packSize: Int = 1,
    var isPack: Boolean = false,
    var quantityStr: String = "1",
    var unitPriceStr: String = "0"
)

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun ReceiveStockScreen(
    viewModel: AliViewModel,
    onNavigateBack: () -> Unit,
    modifier: Modifier = Modifier
) {
    val suppliers by viewModel.allSuppliers.collectAsState()
    val allProducts by viewModel.allProducts.collectAsState()

    var selectedSupplier by remember { mutableStateOf<SupplierEntity?>(null) }
    var supplierExpanded by remember { mutableStateOf(false) }
    var newSupplierName by remember { mutableStateOf("") }
    var showNewSupplierInput by remember { mutableStateOf(false) }

```

```
    var invoiceNumber by remember { mutableStateOf("") }
    var deliveryChargeStr by remember { mutableStateOf("0") }
    var deliveryPaidTo by remember { mutableStateOf("SUPPLIER") } // SUPPLIER, CARRIER, SETTLED
    var amountPaidStr by remember { mutableStateOf("0") }
    var paymentMethod by remember { mutableStateOf("CASH") }
    var notes by remember { mutableStateOf("") }

    val items = remember { mutableStateListOf<ReceiveLineState>() }

    // Computations
    val totalDeliveryPaise = ((deliveryChargeStr.toDoubleOrNull() ?: 0.0) * 100).toLong()
    val amountPaidPaise = ((amountPaidStr.toDoubleOrNull() ?: 0.0) * 100).toLong()

    val lineInputs = items.mapNotNull { line ->
        val qty = line.quantityStr.toIntOrNull() ?: 0
        val baseQty = if (line.isPack) qty * line.packSize else qty
        val pricePaise = ((line.unitPriceStr.toDoubleOrNull() ?: 0.0) * 100).toLong()
        if (line.productId.isNotBlank() && baseQty > 0) {
            PurchaseLineInput(id = line.productId, quantity = baseQty, unitPricePaise = pricePaise)
        } else null
    }

    val totalGoodsValuePaise = lineInputs.sumOf { it.totalValuePaise }
    val allocatedDelivery = CostCalculator.allocateDelivery(lineInputs, totalDeliveryPaise)
    val totalLandedCostPaise = totalGoodsValuePaise + totalDeliveryPaise

    val supplierDeliveryLiability = if (deliveryPaidTo == "SUPPLIER") totalDeliveryPaise else 0L
    val totalBilledBySupplier = totalGoodsValuePaise + supplierDeliveryLiability
    val remainingPayablePaise = (totalBilledBySupplier - amountPaidPaise).coerceAtLeast(0L)

    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text("Receive Goods & Stock", fontWeight = FontWeight.Bold) },
                navigationIcon = {
                    IconButton(onClick = onNavigateBack) {
                        Icon(Icons.Filled.ArrowBack, contentDescription = "Back")
                    }
                },
                colors = TopAppBarDefaults.topAppBarColors(
                    containerColor = MaterialTheme.colorScheme.background,
                    titleContentColor = MaterialTheme.colorScheme.onBackground
                )
            )
        }
    ) { padding ->
        Surface(
            modifier = modifier
                .fillMaxSize()
                .padding(padding),
            color = MaterialTheme.colorScheme.background
        ) {
            LazyColumn(
                modifier = Modifier
                    .fillMaxSize()
                    .padding(horizontal = 16.dp),
                verticalArrangement = Arrangement.spacedBy(14.dp)
            ) {
                // Supplier Selection Card
                item {
                    Card(
                        modifier = Modifier.fillMaxWidth(),
                        colors = CardDefaults.cardColors(containerColor =
    MaterialTheme.colorScheme.surface),
                        shape = RoundedCornerShape(14.dp)
                    ) {
                        Column(modifier = Modifier.padding(14.dp)) {
                            Text(
                                text = "Supplier Details",
                                style = MaterialTheme.typography.titleMedium.copy(fontWeight =
    FontWeight.Bold),
                                color = MaterialTheme.colorScheme.onSurface
                            )
                            Spacer(modifier = Modifier.height(10.dp))

                            if (showNewSupplierInput) {
                                OutlinedTextField(
                                    value = newSupplierName,
                                    onValueChange = { newSupplierName = it },
                                    label = { Text("New Supplier Name") },
                                    modifier = Modifier.fillMaxWidth(),
                                    singleLine = true
                                )
                                Spacer(modifier = Modifier.height(8.dp))
                                Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
                                    Button(
                                        onClick = {
                                            if (newSupplierName.isNotBlank()) {
```

```
                                            val s = SupplierEntity(name = newSupplierName.trim())
                                            viewModel.addSupplier(s) {
                                                selectedSupplier = s
                                                showNewSupplierInput = false
                                            }
                                        }
                                    }
                                ) { Text("Save Supplier") }
                                OutlinedButton(onClick = { showNewSupplierInput = false }) {
    Text("Cancel") }
                            }
                        } else {
                            ExposedDropdownMenuBox(
                                expanded = supplierExpanded,
                                onExpandedChange = { supplierExpanded = !supplierExpanded }
                            ) {
                                OutlinedTextField(
                                    value = selectedSupplier?.name ?: "Select Supplier...",
                                    onValueChange = {},
                                    readOnly = true,
                                    trailingIcon = { ExposedDropdownMenuDefaults.TrailingIcon(expanded =
     supplierExpanded) },
                                    modifier = Modifier
                                        .fillMaxWidth()
                                        .menuAnchor(),
                                    shape = RoundedCornerShape(10.dp)
                                )
                                ExposedDropdownMenu(
                                    expanded = supplierExpanded,
                                    onDismissRequest = { supplierExpanded = false }
                                ) {
                                    suppliers.forEach { supplier ->
                                        DropdownMenuItem(
                                            text = { Text(supplier.name) },
                                            onClick = {
                                                selectedSupplier = supplier
                                                supplierExpanded = false
                                            }
                                        )
                                    }
                                    DropdownMenuItem(
                                        text = { Text("+ Add New Supplier", color =
    MaterialTheme.colorScheme.primary, fontWeight = FontWeight.Bold) },
                                        onClick = {
                                            supplierExpanded = false
                                            showNewSupplierInput = true
                                        }
                                    )
                                }
                            }
                        }

                        Spacer(modifier = Modifier.height(8.dp))
                        OutlinedTextField(
                            value = invoiceNumber,
                            onValueChange = { invoiceNumber = it },
                            label = { Text("Invoice / Bill Number (Optional)") },
                            modifier = Modifier.fillMaxWidth(),
                            singleLine = true,
                            shape = RoundedCornerShape(10.dp)
                        )
                    }
                }
            }

            // Products Received
            item {
                Row(
                    modifier = Modifier.fillMaxWidth(),
                    horizontalArrangement = Arrangement.SpaceBetween,
                    verticalAlignment = Alignment.CenterVertically
                ) {
                    Text(
                        text = "Items Received (${items.size})",
                        style = MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.Bold),
                        color = MaterialTheme.colorScheme.onBackground
                    )
                    Button(
                        onClick = {
                            items.add(ReceiveLineState())
                        },
                        shape = RoundedCornerShape(8.dp),
                        contentPadding = ButtonDefaults.TextButtonContentPadding
                    ) {
                        Icon(Icons.Filled.Add, contentDescription = null, modifier =
    Modifier.size(16.dp))
                        Spacer(modifier = Modifier.width(4.dp))
                        Text("Add Line", fontSize = 12.sp)
```

```
                        }
                    }
                }

                itemsIndexed(items) { index, line ->
                    Card(
                        modifier = Modifier.fillMaxWidth(),
                        colors = CardDefaults.cardColors(containerColor =
MaterialTheme.colorScheme.surface),
                        shape = RoundedCornerShape(12.dp)
                    ) {
                        Column(modifier = Modifier.padding(12.dp)) {
                            Row(
                                modifier = Modifier.fillMaxWidth(),
                                horizontalArrangement = Arrangement.SpaceBetween,
                                verticalAlignment = Alignment.CenterVertically
                            ) {
                                Text(
                                    text = "Line #${index + 1}",
                                    style = MaterialTheme.typography.labelMedium,
                                    color = MaterialTheme.colorScheme.primary
                                )
                                IconButton(
                                    onClick = { items.removeAt(index) },
                                    modifier = Modifier.size(24.dp)
                                ) {
                                    Icon(Icons.Filled.Delete, contentDescription = "Delete", tint =
Color(0xFFE57373))
                                }
                            }

                            Spacer(modifier = Modifier.height(8.dp))

                            // Product Dropdown
                            var prodMenuExpanded by remember { mutableStateOf(false) }
                            ExposedDropdownMenuBox(
                                expanded = prodMenuExpanded,
                                onExpandedChange = { prodMenuExpanded = !prodMenuExpanded }
                            ) {
                                OutlinedTextField(
                                    value = line.productName.ifBlank { "Select Product..." },
                                    onValueChange = {},
                                    readOnly = true,
                                    trailingIcon = { ExposedDropdownMenuDefaults.TrailingIcon(expanded =
prodMenuExpanded) },
                                    modifier = Modifier
                                        .fillMaxWidth()
                                        .menuAnchor(),
                                    shape = RoundedCornerShape(8.dp)
                                )
                                ExposedDropdownMenu(
                                    expanded = prodMenuExpanded,
                                    onDismissRequest = { prodMenuExpanded = false }
                                ) {
                                    allProducts.forEach { product ->
                                        DropdownMenuItem(
                                            text = { Text("${product.name} (${product.category})") },
                                            onClick = {
                                                line.productId = product.id
                                                line.productName = product.name
                                                line.packSize = product.packSize
                                                line.unitPriceStr = (product.movingAvgCostPaise /
100.0).toString()
                                                prodMenuExpanded = false
                                            }
                                        )
                                    }
                                }
                            }

                            Spacer(modifier = Modifier.height(8.dp))

                            Row(
                                modifier = Modifier.fillMaxWidth(),
                                horizontalArrangement = Arrangement.spacedBy(8.dp)
                            ) {
                                OutlinedTextField(
                                    value = line.quantityStr,
                                    onValueChange = { line.quantityStr = it },
                                    label = { Text("Qty (Base)") },
                                    keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Number),
                                    modifier = Modifier.weight(1f),
                                    singleLine = true
                                )
                                OutlinedTextField(
                                    value = line.unitPriceStr,
                                    onValueChange = { line.unitPriceStr = it },
                                    label = { Text("Price (₹/pc)") },
```

```
                                    keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Decimal),
                                    modifier = Modifier.weight(1f),
                                    singleLine = true
                                )
                            }

                            // Show allocated delivery share for this line if delivery > 0
                            val alloc = allocatedDelivery.firstOrNull { it.lineId == line.productId }
                            if (alloc != null && totalDeliveryPaise > 0L) {
                                Spacer(modifier = Modifier.height(6.dp))
                                Row(
                                    modifier = Modifier.fillMaxWidth(),
                                    horizontalArrangement = Arrangement.SpaceBetween
                                ) {
                                    Text(
                                        text = "Allocated Delivery:
${CostCalculator.formatPaiseToRupees(alloc.allocatedDeliveryPaise)}",
                                        style = MaterialTheme.typography.bodySmall,
                                        color = MaterialTheme.colorScheme.primary
                                    )
                                    Text(
                                        text = "Effective Landed:
${CostCalculator.formatPaiseToRupees(alloc.effectiveUnitCostPaise)}/pc",
                                        style = MaterialTheme.typography.bodySmall.copy(fontWeight =
FontWeight.Bold),
                                        color = MaterialTheme.colorScheme.onSurface
                                    )
                                }
                            }

                            // Show profit after transport if selling price is set
                            val product = allProducts.firstOrNull { it.id == line.productId }
                            if (product != null && product.sellingPricePiecePaise > 0L) {
                                val landedCost = if (alloc != null && totalDeliveryPaise > 0L)
alloc.effectiveUnitCostPaise else (((line.unitPriceStr.toDoubleOrNull() ?: 0.0) * 100).toLong())
                                val profitPaise = product.sellingPricePiecePaise - landedCost
                                val margin = if (product.sellingPricePiecePaise > 0) (profitPaise.toDouble()
 / product.sellingPricePiecePaise.toDouble()) * 100.0 else 0.0
                                Spacer(modifier = Modifier.height(4.dp))
                                Row(
                                    modifier = Modifier.fillMaxWidth(),
                                    horizontalArrangement = Arrangement.SpaceBetween
                                ) {
                                    Text(
                                        text = "Selling:
${CostCalculator.formatPaiseToRupees(product.sellingPricePiecePaise)}/pc",
                                        style = MaterialTheme.typography.bodySmall,
                                        color = MaterialTheme.colorScheme.onSurfaceVariant
                                    )
                                    Text(
                                        text = "Profit after transport:
${CostCalculator.formatPaiseToRupees(profitPaise)} (${String.format(java.util.Locale.US, "%.1f", margin)}%
margin)",
                                        style = MaterialTheme.typography.bodySmall.copy(fontWeight =
FontWeight.Bold),
                                        color = if (profitPaise >= 0) Color(0xFF75B890) else
Color(0xFFE57373)
                                    )
                                }
                            }
                        }
                    }
                }

                // Delivery Charge Card
                item {
                    Card(
                        modifier = Modifier.fillMaxWidth(),
                        colors = CardDefaults.cardColors(containerColor =
MaterialTheme.colorScheme.surface),
                        shape = RoundedCornerShape(14.dp)
                    ) {
                        Column(modifier = Modifier.padding(14.dp)) {
                            Text(
                                text = "Delivery & Freight Allocation",
                                style = MaterialTheme.typography.titleMedium.copy(fontWeight =
FontWeight.Bold),
                                color = MaterialTheme.colorScheme.onSurface
                            )
                            Spacer(modifier = Modifier.height(10.dp))

                            OutlinedTextField(
                                value = deliveryChargeStr,
                                onValueChange = { deliveryChargeStr = it },
                                label = { Text("Total Delivery Charge (₹)") },
                                keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Decimal),
                                modifier = Modifier.fillMaxWidth(),
                                singleLine = true,
```

```
                                shape = RoundedCornerShape(10.dp)
                            )

                            Spacer(modifier = Modifier.height(8.dp))
                            Text(
                                text = "Distributed proportionally according to each item's purchase value.",
                                style = MaterialTheme.typography.bodySmall,
                                color = MaterialTheme.colorScheme.onSurfaceVariant
                            )

                            Spacer(modifier = Modifier.height(8.dp))
                            Row(verticalAlignment = Alignment.CenterVertically) {
                                Text(
                                    text = "Delivery owed to:",
                                    style = MaterialTheme.typography.bodyMedium,
                                    color = MaterialTheme.colorScheme.onSurface
                                )
                                Spacer(modifier = Modifier.width(8.dp))
                                OutlinedButton(
                                    onClick = {
                                        deliveryPaidTo = if (deliveryPaidTo == "SUPPLIER") "CARRIER" else "SUPPLIER"
                                    }
                                ) {
                                    Text(if (deliveryPaidTo == "SUPPLIER") "This Supplier" else "Carrier / Settled")
                                }
                            }
                        }
                    }
                }

                // Payment & Summary Card
                item {
                    Card(
                        modifier = Modifier.fillMaxWidth(),
                        colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surface),
                        shape = RoundedCornerShape(14.dp)
                    ) {
                        Column(modifier = Modifier.padding(14.dp)) {
                            Text(
                                text = "Bill Summary & Payment",
                                style = MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.Bold),
                                color = MaterialTheme.colorScheme.onSurface
                            )
                            Spacer(modifier = Modifier.height(10.dp))

                            Row(
                                modifier = Modifier.fillMaxWidth(),
                                horizontalArrangement = Arrangement.SpaceBetween
                            ) {
                                Text("Goods Value:", color = MaterialTheme.colorScheme.onSurfaceVariant)
                                Text(CostCalculator.formatPaiseToRupees(totalGoodsValuePaise), fontWeight = FontWeight.Bold)
                            }
                            Spacer(modifier = Modifier.height(4.dp))
                            Row(
                                modifier = Modifier.fillMaxWidth(),
                                horizontalArrangement = Arrangement.SpaceBetween
                            ) {
                                Text("Delivery Charge:", color = MaterialTheme.colorScheme.onSurfaceVariant)
                                Text(CostCalculator.formatPaiseToRupees(totalDeliveryPaise))
                            }
                            Spacer(modifier = Modifier.height(4.dp))
                            Row(
                                modifier = Modifier.fillMaxWidth(),
                                horizontalArrangement = Arrangement.SpaceBetween
                            ) {
                                Text("Total Landed Stock Cost:", fontWeight = FontWeight.Bold)
                                Text(CostCalculator.formatPaiseToRupees(totalLandedCostPaise), fontWeight = FontWeight.Bold, color = MaterialTheme.colorScheme.primary)
                            }

                            Spacer(modifier = Modifier.height(12.dp))

                            OutlinedTextField(
                                value = amountPaidStr,
                                onValueChange = { amountPaidStr = it },
                                label = { Text("Amount Paid Now (₹)") },
                                keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Decimal),
                                modifier = Modifier.fillMaxWidth(),
                                singleLine = true,
                                shape = RoundedCornerShape(10.dp)
                            )

                            Spacer(modifier = Modifier.height(8.dp))
```

```
                            Row(
                                modifier = Modifier.fillMaxWidth(),
                                horizontalArrangement = Arrangement.SpaceBetween
                            ) {
                                Text("Remaining Owed to Supplier:", fontWeight = FontWeight.Bold)
                                Text(
                                    CostCalculator.formatPaiseToRupees(remainingPayablePaise),
                                    fontWeight = FontWeight.Bold,
                                    color = if (remainingPayablePaise > 0) Color(0xFFE57373) else
    Color(0xFF75B890)
                                )
                            }
                        }
                    }

                    // Save Goods Received Button
                    item {
                        Button(
                            onClick = {
                                val supplier = selectedSupplier ?: return@Button
                                val validItems = items.mapNotNull { line ->
                                    val qty = line.quantityStr.toIntOrNull() ?: 0
                                    val baseQty = if (line.isPack) qty * line.packSize else qty
                                    val pricePaise = ((line.unitPriceStr.toDoubleOrNull() ?: 0.0) *
    100).toLong()
                                    if (line.productId.isNotBlank() && baseQty > 0) {
                                        PurchaseItemParam(
                                            productId = line.productId,
                                            quantityBase = baseQty,
                                            unitPurchasePricePaise = pricePaise
                                        )
                                    } else null
                                }

                                if (validItems.isEmpty()) return@Button

                                viewModel.receiveGoods(
                                    supplierId = supplier.id,
                                    invoiceNumber = invoiceNumber.trim().ifBlank { null },
                                    items = validItems,
                                    totalDeliveryPaise = totalDeliveryPaise,
                                    deliveryPaidTo = deliveryPaidTo,
                                    amountPaidPaise = amountPaidPaise,
                                    paymentMethod = paymentMethod,
                                    notes = notes.trim().ifBlank { null },
                                    onDone = onNavigateBack
                                )
                            },
                            enabled = selectedSupplier != null && lineInputs.isNotEmpty(),
                            modifier = Modifier
                                .fillMaxWidth()
                                .height(54.dp),
                            shape = RoundedCornerShape(12.dp),
                            colors = ButtonDefaults.buttonColors(
                                containerColor = MaterialTheme.colorScheme.primary,
                                contentColor = MaterialTheme.colorScheme.onPrimary
                            )
                        ) {
                            Text(
                                text = "Save Goods Received",
                                style = MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.Bold)
                            )
                        }
                        Spacer(modifier = Modifier.height(24.dp))
                    }
                }
            }
        }
    }
}
```

app\src\main\java\com\alieducational\app\ui\stock\RemoveStockScreen.kt

```
package com.alieducational.app.ui.stock

import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.foundation.verticalScroll
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.ArrowBack
import androidx.compose.material.icons.filled.Warning
import androidx.compose.material3.Button
import androidx.compose.material3.ButtonDefaults
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.DropdownMenuItem
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.ExposedDropdownMenuBox
import androidx.compose.material3.ExposedDropdownMenuDefaults
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.RadioButton
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.material3.TopAppBar
import androidx.compose.material3.TopAppBarDefaults
import androidx.compose.runtime.Composable
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.unit.dp
import com.alieducational.app.data.local.entity.ProductEntity
import com.alieducational.app.ui.AliViewModel

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun RemoveStockScreen(
    viewModel: AliViewModel,
    onNavigateBack: () -> Unit,
    onNavigateToRecordSale: () -> Unit,
    modifier: Modifier = Modifier
) {
    val allProducts by viewModel.allProducts.collectAsState()
    var selectedProduct by remember { mutableStateOf<ProductEntity?>(null) }
    var productExpanded by remember { mutableStateOf(false) }

    var isPack by remember { mutableStateOf(false) }
    var recipientNote by remember { mutableStateOf("") }
    var movementType by remember { mutableStateOf("DAMAGE") } // DAMAGE, SHOP_USE, FREE_ITEM, DONATION, RETURN, CORRECTION
    var quantityStr by remember { mutableStateOf("1") }
    var countedTotalStr by remember { mutableStateOf("") }
    var reason by remember { mutableStateOf("") }

    val rawQuantity = quantityStr.toIntOrNull() ?: 0
    val packSize = selectedProduct?.packSize ?: 1
    val baseQuantity = if (isPack) rawQuantity * packSize else rawQuantity
    val countedTotal = countedTotalStr.toIntOrNull()

    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text("Remove Stock / Adjustment", fontWeight = FontWeight.Bold) },
```

```
                navigationIcon = {
                    IconButton(onClick = onNavigateBack) {
                        Icon(Icons.Filled.ArrowBack, contentDescription = "Back")
                    }
                },
                colors = TopAppBarDefaults.topAppBarColors(
                    containerColor = MaterialTheme.colorScheme.background,
                    titleContentColor = MaterialTheme.colorScheme.onBackground
                )
            )
        }
    ) { padding ->
        Surface(
            modifier = modifier
                .fillMaxSize()
                .padding(padding),
            color = MaterialTheme.colorScheme.background
        ) {
            Column(
                modifier = Modifier
                    .fillMaxSize()
                    .verticalScroll(rememberScrollState())
                    .padding(16.dp),
                verticalArrangement = Arrangement.spacedBy(14.dp)
            ) {
                // Sale Shortcut banner
                Card(
                    modifier = Modifier
                        .fillMaxWidth()
                        .clickable { onNavigateToRecordSale() },
                    colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.primaryContainer),
                    shape = RoundedCornerShape(12.dp)
                ) {
                    Row(
                        modifier = Modifier
                            .fillMaxWidth()
                            .padding(14.dp),
                        verticalAlignment = Alignment.CenterVertically,
                        horizontalArrangement = Arrangement.SpaceBetween
                    ) {
                        Column {
                            Text(
                                text = "Selling goods to a customer?",
                                style = MaterialTheme.typography.titleSmall.copy(fontWeight = FontWeight.Bold),
                                color = MaterialTheme.colorScheme.onPrimaryContainer
                            )
                            Text(
                                text = "Use Record Sale to calculate profit & customer debt",
                                style = MaterialTheme.typography.bodySmall,
                                color = MaterialTheme.colorScheme.onPrimaryContainer.copy(alpha = 0.8f)
                            )
                        }
                        Button(onClick = onNavigateToRecordSale) {
                            Text("Go to Sale")
                        }
                    }
                }

                // Select Product
                Card(
                    modifier = Modifier.fillMaxWidth(),
                    colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surface),
                    shape = RoundedCornerShape(14.dp)
                ) {
                    Column(modifier = Modifier.padding(14.dp)) {
                        Text(
                            text = "Select Product",
                            style = MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.Bold),
                            color = MaterialTheme.colorScheme.onSurface
                        )
                        Spacer(modifier = Modifier.height(10.dp))

                        ExposedDropdownMenuBox(
                            expanded = productExpanded,
                            onExpandedChange = { productExpanded = !productExpanded }
                        ) {
                            OutlinedTextField(
                                value = selectedProduct?.name ?: "Select a product...",
                                onValueChange = {},
                                readOnly = true,
                                trailingIcon = { ExposedDropdownMenuDefaults.TrailingIcon(expanded = productExpanded) },
                                modifier = Modifier
                                    .fillMaxWidth()
                                    .menuAnchor(),
                                shape = RoundedCornerShape(10.dp)
```

```
                            )
                            ExposedDropdownMenu(
                                expanded = productExpanded,
                                onDismissRequest = { productExpanded = false }
                            ) {
                                allProducts.forEach { product ->
                                    DropdownMenuItem(
                                        text = { Text("${product.name} (Stock: ${product.currentSellableStock} pcs)") },
                                        onClick = {
                                            selectedProduct = product
                                            productExpanded = false
                                        }
                                    )
                                }
                            }
                        }

                        if (selectedProduct != null) {
                            Spacer(modifier = Modifier.height(8.dp))
                            Text(
                                text = "Current Sellable: ${selectedProduct!!.currentSellableStock} pcs • Damaged: ${selectedProduct!!.currentDamagedStock} pcs",
                                style = MaterialTheme.typography.bodySmall,
                                color = MaterialTheme.colorScheme.primary
                            )
                        }
                    }
                }

                // Select Reason / Type
                Card(
                    modifier = Modifier.fillMaxWidth(),
                    colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surface),
                    shape = RoundedCornerShape(14.dp)
                ) {
                    Column(modifier = Modifier.padding(14.dp)) {
                        Text(
                            text = "Reason for Removal / Adjustment",
                            style = MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.Bold),
                            color = MaterialTheme.colorScheme.onSurface
                        )
                        Spacer(modifier = Modifier.height(8.dp))

                        val options = listOf(
                            "FREE_ITEM" to "Giveaway (Free Item / Sample)",
                            "DONATION" to "Donation (Charity / Educational Gift)",
                            "DAMAGE" to "Damaged / Broken (Move to Damaged Stock)",
                            "SHOP_USE" to "Shop Use (e.g. pens, paper used in shop)",
                            "RETURN" to "Supplier Return",
                            "CORRECTION" to "Physical Count Counted Total Correction"
                        )

                        options.forEach { (type, label) ->
                            Row(
                                modifier = Modifier
                                    .fillMaxWidth()
                                    .clickable { movementType = type }
                                    .padding(vertical = 4.dp),
                                verticalAlignment = Alignment.CenterVertically
                            ) {
                                RadioButton(
                                    selected = movementType == type,
                                    onClick = { movementType = type }
                                )
                                Spacer(modifier = Modifier.width(8.dp))
                                Text(
                                    text = label,
                                    style = MaterialTheme.typography.bodyMedium,
                                    color = MaterialTheme.colorScheme.onSurface
                                )
                            }
                        }
                    }
                }

                // Quantity / Counted Total Inputs
                Card(
                    modifier = Modifier.fillMaxWidth(),
                    colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surface),
                    shape = RoundedCornerShape(14.dp)
                ) {
                    Column(modifier = Modifier.padding(14.dp)) {
                        val isGiveawayOrDonation = movementType == "FREE_ITEM" || movementType == "DONATION"

                        if (selectedProduct != null && selectedProduct!!.packSize > 1 && movementType != "CORRECTION") {
                            Row(
```

```
                                horizontalArrangement = Arrangement.spacedBy(8.dp),
                                modifier = Modifier.fillMaxWidth()
                            ) {
                                androidx.compose.material3.FilterChip(
                                    selected = !isPack,
                                    onClick = { isPack = false },
                                    label = { Text("Loose Pieces") }
                                )
                                androidx.compose.material3.FilterChip(
                                    selected = isPack,
                                    onClick = { isPack = true },
                                    label = { Text("Full Packs (${selectedProduct!!.packSize} pcs/pack)") }
                                )
                            }
                            Spacer(modifier = Modifier.height(8.dp))
                        }

                        if (movementType == "CORRECTION") {
                            OutlinedTextField(
                                value = countedTotalStr,
                                onValueChange = { countedTotalStr = it },
                                label = { Text("Actual Counted Total (pieces)") },
                                keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Number),
                                modifier = Modifier.fillMaxWidth(),
                                singleLine = true,
                                shape = RoundedCornerShape(10.dp)
                            )
                            if (selectedProduct != null && countedTotal != null) {
                                val diff = countedTotal - selectedProduct!!.currentSellableStock
                                Spacer(modifier = Modifier.height(6.dp))
                                Text(
                                    text = "Adjustment: ${if (diff > 0) "+$diff" else "$diff"} pcs",
                                    style = MaterialTheme.typography.bodySmall.copy(fontWeight =
    FontWeight.Bold),
                                    color = if (diff >= 0) Color(0xFF75B890) else Color(0xFFE57373)
                                )
                            }
                        } else {
                            val unitLabel = if (isPack) "packs" else "pieces"
                            OutlinedTextField(
                                value = quantityStr,
                                onValueChange = { quantityStr = it },
                                label = { Text("Quantity to Remove ($unitLabel)") },
                                keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Number),
                                modifier = Modifier.fillMaxWidth(),
                                singleLine = true,
                                shape = RoundedCornerShape(10.dp)
                            )

                            if (isPack && selectedProduct != null) {
                                Spacer(modifier = Modifier.height(4.dp))
                                Text(
                                    text = "$rawQuantity packs = $baseQuantity total pieces leaving stock",
                                    style = MaterialTheme.typography.bodySmall,
                                    color = MaterialTheme.colorScheme.primary
                                )
                            }

                            if (selectedProduct != null && baseQuantity >
    selectedProduct!!.currentSellableStock) {
                                Spacer(modifier = Modifier.height(6.dp))
                                Row(verticalAlignment = Alignment.CenterVertically) {
                                    Icon(Icons.Filled.Warning, contentDescription = null, tint =
    Color(0xFFE4B765), modifier = Modifier.size(16.dp))
                                    Spacer(modifier = Modifier.width(4.dp))
                                    Text(
                                        text = "Quantity ($baseQuantity pcs) exceeds current stock
    (${selectedProduct!!.currentSellableStock} pcs)",
                                        style = MaterialTheme.typography.bodySmall,
                                        color = Color(0xFFE4B765)
                                    )
                                }
                            }
                        }

                        if (isGiveawayOrDonation && selectedProduct != null) {
                            Spacer(modifier = Modifier.height(10.dp))
                            val costPaise = selectedProduct!!.movingAvgCostPaise
                            val totalCostPaise = costPaise * baseQuantity
                            Card(
                                colors = CardDefaults.cardColors(containerColor =
    MaterialTheme.colorScheme.surfaceVariant.copy(alpha = 0.5f)),
                                shape = RoundedCornerShape(8.dp)
                            ) {
                                Column(modifier = Modifier.padding(10.dp)) {
                                    Text(
                                        text = if (costPaise > 0) "Known unit cost:
    ${com.alieducational.app.domain.calculation.CostCalculator.formatPaiseToRupees(costPaise)}/pc • Total cost:
```

```
    ${com.alieducational.app.domain.calculation.CostCalculator.formatPaiseToRupees(totalCostPaise)}"
                                        else "Purchase cost: Unknown (Cost snapshot marked incomplete)",
                                        style = MaterialTheme.typography.bodySmall.copy(fontWeight =
    FontWeight.Bold),
                                        color = MaterialTheme.colorScheme.onSurface
                                    )
                                    Spacer(modifier = Modifier.height(4.dp))
                                    Text(
                                        text = "Reduces sellable stock. Creates zero sales revenue, bill
    payment or customer credit.",
                                        style = MaterialTheme.typography.bodySmall,
                                        color = MaterialTheme.colorScheme.onSurfaceVariant
                                    )
                                }
                            }

                            Spacer(modifier = Modifier.height(10.dp))
                            OutlinedTextField(
                                value = recipientNote,
                                onValueChange = { recipientNote = it },
                                label = { Text("Recipient / Purpose (Optional)") },
                                placeholder = { Text("e.g. Student aid, Madrasa, Sample for teacher") },
                                modifier = Modifier.fillMaxWidth(),
                                singleLine = true,
                                shape = RoundedCornerShape(10.dp)
                            )
                        }

                        Spacer(modifier = Modifier.height(10.dp))

                        OutlinedTextField(
                            value = reason,
                            onValueChange = { reason = it },
                            label = { Text("Notes / Reason (Optional)") },
                            modifier = Modifier.fillMaxWidth(),
                            singleLine = true,
                            shape = RoundedCornerShape(10.dp)
                        )
                    }
                }

                // Confirm Adjustment Button
                Button(
                    onClick = {
                        val product = selectedProduct ?: return@Button
                        if (movementType == "FREE_ITEM" || movementType == "DONATION") {
                            viewModel.recordProductGiveaway(
                                productId = product.id,
                                quantityBase = baseQuantity,
                                isPack = isPack,
                                packSizeSnapshot = packSize,
                                reason = movementType,
                                recipientNote = recipientNote.trim().ifBlank { reason.trim().ifBlank { null
    } },
                                onDone = onNavigateBack
                            )
                        } else {
                            val change = if (movementType == "CORRECTION") {
                                (countedTotal ?: product.currentSellableStock) -
    product.currentSellableStock
                            } else {
                                -baseQuantity
                            }

                            viewModel.recordStockAdjustment(
                                productId = product.id,
                                type = movementType,
                                quantityChange = change,
                                countedTotal = if (movementType == "CORRECTION") countedTotal else null,
                                reason = reason.trim().ifBlank { null },
                                onDone = onNavigateBack
                            )
                        }
                    },
                    enabled = selectedProduct != null && (if (movementType == "CORRECTION") countedTotal !=
    null else baseQuantity > 0),
                    modifier = Modifier
                        .fillMaxWidth()
                        .height(54.dp),
                    shape = RoundedCornerShape(12.dp),
                    colors = ButtonDefaults.buttonColors(
                        containerColor = MaterialTheme.colorScheme.primary,
                        contentColor = MaterialTheme.colorScheme.onPrimary
                    )
                ) {
                    Text(
                        text = "Confirm Stock Adjustment",
                        style = MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.Bold)
```

```
                    )
                }

                Spacer(modifier = Modifier.height(24.dp))
            }
        }
    }
}
```

app\src\main\java\com\alieducational\app\ui\stock\StockScreen.kt

```
package com.alieducational.app.ui.stock

import android.Manifest
import android.content.pm.PackageManager
import androidx.activity.compose.rememberLauncherForActivityResult
import androidx.activity.result.contract.ActivityResultContracts
import androidx.camera.core.CameraSelector
import androidx.camera.core.ImageAnalysis
import androidx.camera.core.Preview
import androidx.camera.lifecycle.ProcessCameraProvider
import androidx.camera.view.PreviewView
import androidx.compose.foundation.background
import androidx.compose.foundation.border
import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.LazyRow
import androidx.compose.foundation.lazy.items
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.foundation.verticalScroll
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Add
import androidx.compose.material.icons.filled.Clear
import androidx.compose.material.icons.filled.FileDownload
import androidx.compose.material.icons.filled.LocalShipping
import androidx.compose.material.icons.filled.QrCodeScanner
import androidx.compose.material.icons.filled.Receipt
import androidx.compose.material.icons.filled.Search
import androidx.compose.material.icons.filled.Warning
import androidx.compose.material3.AlertDialog
import androidx.compose.material3.Button
import androidx.compose.material3.ButtonDefaults
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.DropdownMenuItem
import androidx.compose.material3.HorizontalDivider
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.ExposedDropdownMenuBox
import androidx.compose.material3.ExposedDropdownMenuDefaults
import androidx.compose.material3.FilterChip
import androidx.compose.material3.FilterChipDefaults
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.MenuAnchorType
import androidx.compose.material3.OutlinedButton
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.material3.TextButton
import androidx.compose.runtime.Composable
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.compose.ui.viewinterop.AndroidView
import androidx.compose.ui.window.Dialog
import androidx.compose.ui.window.DialogProperties
import androidx.core.content.ContextCompat
import androidx.lifecycle.compose.LocalLifecycleOwner
```

```
import com.alieducational.app.data.local.entity.ProductEntity
import com.alieducational.app.data.local.entity.SupplierEntity
import com.alieducational.app.domain.calculation.CostCalculator
import com.alieducational.app.ui.AliViewModel
import com.alieducational.app.ui.billing.BarcodeAnalyzer
import java.util.Locale
import java.util.concurrent.Executors

@Composable
fun StockScreen(
    viewModel: AliViewModel,
    onNavigateToReceiveStock: () -> Unit,
    onNavigateToImport: () -> Unit,
    onNavigateToRemoveStock: () -> Unit,
    onNavigateBack: () -> Unit = {},
    modifier: Modifier = Modifier
) {
    androidx.activity.compose.BackHandler {
        onNavigateBack()
    }
    val allProducts by viewModel.allProducts.collectAsState()
    val allSuppliers by viewModel.allSuppliers.collectAsState()
    var searchQuery by remember { mutableStateOf("") }
    var selectedFilter by remember { mutableStateOf("All") } // "All", "Low Stock"
    var showAddDialog by remember { mutableStateOf(false) }
    var selectedProductForEdit by remember { mutableStateOf<ProductEntity?>(null) }

    val widgetAction by viewModel.widgetActionTrigger.collectAsState()
    androidx.compose.runtime.LaunchedEffect(widgetAction) {
        if (widgetAction == com.alieducational.app.widget.AliSlimBarWidgetProvider.ACTION_ADD_STOCK) {
            showAddDialog = true
            viewModel.consumeWidgetAction()
        }
    }

    val filteredProducts = allProducts.filter { product ->
        val matchesSearch = searchQuery.isBlank() ||
                product.name.contains(searchQuery, ignoreCase = true) ||
                (product.barcode?.contains(searchQuery, ignoreCase = true) == true) ||
                (product.sku?.contains(searchQuery, ignoreCase = true) == true) ||
                (product.brand?.contains(searchQuery, ignoreCase = true) == true) ||
                product.category.contains(searchQuery, ignoreCase = true)

        val matchesFilter = when (selectedFilter) {
            "Low Stock" -> product.currentSellableStock <= product.lowStockThreshold
            else -> true
        }

        matchesSearch && matchesFilter
    }

    Surface(
        modifier = modifier.fillMaxSize(),
        color = MaterialTheme.colorScheme.background
    ) {
        Column(
            modifier = Modifier
                .fillMaxSize()
                .padding(horizontal = 16.dp)
        ) {
            Spacer(modifier = Modifier.height(12.dp))

            // Integrated Top Action Buttons: Add Stock & Product + Import
            Row(
                modifier = Modifier.fillMaxWidth(),
                horizontalArrangement = Arrangement.spacedBy(10.dp)
            ) {
                Button(
                    onClick = { showAddDialog = true },
                    modifier = Modifier.weight(1.2f),
                    shape = RoundedCornerShape(10.dp),
                    colors = ButtonDefaults.buttonColors(
                        containerColor = MaterialTheme.colorScheme.primary,
                        contentColor = MaterialTheme.colorScheme.onPrimary
                    )
                ) {
                    Icon(Icons.Filled.Add, contentDescription = null, modifier = Modifier.size(18.dp))
                    Spacer(modifier = Modifier.width(6.dp))
                    Text(text = "Add Stock & Product", fontSize = 13.sp, fontWeight = FontWeight.Bold)
                }

                OutlinedButton(
                    onClick = onNavigateToImport,
                    modifier = Modifier.weight(0.9f),
                    shape = RoundedCornerShape(10.dp)
                ) {
                    Icon(Icons.Filled.FileDownload, contentDescription = null, modifier =
Modifier.size(18.dp))
```

```
                    Spacer(modifier = Modifier.width(6.dp))
                    Text(text = "Import Excel", fontSize = 12.sp, fontWeight = FontWeight.SemiBold)
                }
            }

            Spacer(modifier = Modifier.height(12.dp))

            // Search Bar
            OutlinedTextField(
                value = searchQuery,
                onValueChange = { searchQuery = it },
                modifier = Modifier.fillMaxWidth(),
                placeholder = { Text("Search product name, barcode, brand...") },
                leadingIcon = { Icon(Icons.Filled.Search, contentDescription = "Search") },
                trailingIcon = {
                    if (searchQuery.isNotEmpty()) {
                        IconButton(onClick = { searchQuery = "" }) {
                            Icon(Icons.Filled.Clear, contentDescription = "Clear")
                        }
                    }
                },
                singleLine = true,
                shape = RoundedCornerShape(12.dp)
            )

            Spacer(modifier = Modifier.height(8.dp))

            // Filter Chips
            LazyRow(
                horizontalArrangement = Arrangement.spacedBy(8.dp),
                modifier = Modifier.fillMaxWidth()
            ) {
                item {
                    FilterChip(
                        selected = selectedFilter == "All",
                        onClick = { selectedFilter = "All" },
                        label = { Text("All (${allProducts.size})") },
                        colors = FilterChipDefaults.filterChipColors(
                            selectedContainerColor = MaterialTheme.colorScheme.primaryContainer,
                            selectedLabelColor = MaterialTheme.colorScheme.onPrimaryContainer
                        )
                    )
                }
                item {
                    val lowStockCount = allProducts.count { it.currentSellableStock <= it.lowStockThreshold }
                    FilterChip(
                        selected = selectedFilter == "Low Stock",
                        onClick = { selectedFilter = "Low Stock" },
                        label = { Text("Low Stock ($lowStockCount)") },
                        colors = FilterChipDefaults.filterChipColors(
                            selectedContainerColor = MaterialTheme.colorScheme.errorContainer,
                            selectedLabelColor = MaterialTheme.colorScheme.onErrorContainer
                        )
                    )
                }
            }

            Spacer(modifier = Modifier.height(8.dp))

            // Product List
            if (filteredProducts.isEmpty()) {
                Box(
                    modifier = Modifier
                        .fillMaxWidth()
                        .weight(1f),
                    contentAlignment = Alignment.Center
                ) {
                    Column(horizontalAlignment = Alignment.CenterHorizontally) {
                        Text(
                            text = if (searchQuery.isBlank()) "No products in inventory" else "No matching products found",
                            style = MaterialTheme.typography.bodyLarge,
                            color = MaterialTheme.colorScheme.onSurfaceVariant
                        )
                        Spacer(modifier = Modifier.height(8.dp))
                        Text(
                            text = "Tap 'Add Stock & Product' above to create one",
                            style = MaterialTheme.typography.bodySmall,
                            color = MaterialTheme.colorScheme.onSurfaceVariant
                        )
                    }
                }
            } else {
                LazyColumn(
                    modifier = Modifier
                        .fillMaxWidth()
                        .weight(1f),
```

```
                    verticalArrangement = Arrangement.spacedBy(10.dp)
                ) {
                    items(filteredProducts) { product ->
                        ProductItemCard(
                            product = product,
                            onClick = { selectedProductForEdit = product }
                        )
                    }
                    item { Spacer(modifier = Modifier.height(72.dp)) }
                }
            }
        }
    }

    // Integrated Add / Edit Product & Stock Dialog
    if (showAddDialog || selectedProductForEdit != null) {
        IntegratedAddStockProductDialog(
            existingProduct = selectedProductForEdit,
            suppliers = allSuppliers,
            onAddSupplier = { newSup, onCreated ->
                viewModel.addSupplier(newSup) {
                    onCreated(newSup)
                }
            },
            onDismiss = {
                showAddDialog = false
                selectedProductForEdit = null
            },
            onSave = { existingId, name, barcode, category, brand, packSize, boxesBrought, loosePieces,
costPaise, sellingPaise, wholesalePaise, lowStock, supplierName, billNo, transportPaise, isPaidNow,
billPaymentMethod ->
                viewModel.addOrUpdateProductWithStock(
                    existingId = existingId,
                    name = name,
                    barcode = barcode,
                    category = category,
                    brand = brand,
                    packSize = packSize,
                    boxesBrought = boxesBrought,
                    loosePiecesBrought = loosePieces,
                    costPerPiecePaise = costPaise,
                    sellingPricePiecePaise = sellingPaise,
                    wholesalePricePiecePaise = wholesalePaise,
                    lowStockThreshold = lowStock,
                    supplierName = supplierName,
                    billNo = billNo,
                    transportChargesPaise = transportPaise,
                    isPaidNow = isPaidNow,
                    paymentMethod = billPaymentMethod
                )
                showAddDialog = false
                selectedProductForEdit = null
            }
        )
    }
}

@Composable
fun ProductItemCard(
    product: ProductEntity,
    onClick: () -> Unit
) {
    val isLowStock = product.currentSellableStock <= product.lowStockThreshold
    val marginPct = if (product.sellingPricePiecePaise > 0L && product.sellingPricePiecePaise >
product.movingAvgCostPaise) {
        ((product.sellingPricePiecePaise - product.movingAvgCostPaise).toDouble() /
product.sellingPricePiecePaise.toDouble()) * 100.0
    } else 0.0

    Card(
        modifier = Modifier
            .fillMaxWidth()
            .clickable(onClick = onClick),
        shape = RoundedCornerShape(12.dp),
        colors = CardDefaults.cardColors(containerColor =
MaterialTheme.colorScheme.surfaceVariant.copy(alpha = 0.4f))
    ) {
        Column(
            modifier = Modifier
                .fillMaxWidth()
                .padding(14.dp)
        ) {
            Row(
                modifier = Modifier.fillMaxWidth(),
                horizontalArrangement = Arrangement.SpaceBetween,
                verticalAlignment = Alignment.CenterVertically
            ) {
                Column(modifier = Modifier.weight(1f)) {
```

```
                    Text(
                        text = product.name,
                        style = MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.Bold),
                        color = MaterialTheme.colorScheme.onSurface
                    )
                    val subtitle = buildList {
                        if (!product.brand.isNullOrBlank()) add(product.brand)
                        add(product.category)
                        if (!product.barcode.isNullOrBlank()) add("Barcode: ${product.barcode}")
                        else if (!product.sku.isNullOrBlank()) add("SKU: ${product.sku}")
                    }.joinToString(" • ")
                    Text(
                        text = subtitle,
                        style = MaterialTheme.typography.bodySmall,
                        color = MaterialTheme.colorScheme.onSurfaceVariant
                    )
                }

                Column(horizontalAlignment = Alignment.End) {
                    Text(
                        text = "${product.currentSellableStock} pcs",
                        style = MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.Bold),
                        color = if (isLowStock) MaterialTheme.colorScheme.error else
MaterialTheme.colorScheme.onSurface
                    )
                    if (isLowStock) {
                        Row(verticalAlignment = Alignment.CenterVertically) {
                            Icon(
                                Icons.Filled.Warning,
                                contentDescription = null,
                                modifier = Modifier.size(12.dp),
                                tint = Color(0xFFE4B765)
                            )
                            Spacer(modifier = Modifier.width(3.dp))
                            Text(
                                text = "Low stock",
                                style = MaterialTheme.typography.labelSmall,
                                color = Color(0xFFE4B765)
                            )
                        }
                    }
                }
            }

            Spacer(modifier = Modifier.height(10.dp))

            // Pricing, Wholesale and Margin details
            Row(
                modifier = Modifier.fillMaxWidth(),
                horizontalArrangement = Arrangement.SpaceBetween,
                verticalAlignment = Alignment.CenterVertically
            ) {
                Column {
                    Text(
                        text = "Selling:
${CostCalculator.formatPaiseToRupees(product.sellingPricePiecePaise)}/pc",
                        style = MaterialTheme.typography.bodySmall.copy(fontWeight = FontWeight.Medium),
                        color = MaterialTheme.colorScheme.onSurface
                    )
                    if (product.wholesalePricePiecePaise > 0L) {
                        Text(
                            text = "Wholesale:
${CostCalculator.formatPaiseToRupees(product.wholesalePricePiecePaise)}/pc",
                            style = MaterialTheme.typography.labelSmall,
                            color = MaterialTheme.colorScheme.primary
                        )
                    }
                }

                Row(verticalAlignment = Alignment.CenterVertically, horizontalArrangement =
Arrangement.spacedBy(8.dp)) {
                    if (marginPct > 0) {
                        Box(
                            modifier = Modifier
                                .background(Color(0xFF75B890).copy(alpha = 0.2f), RoundedCornerShape(6.dp))
                                .padding(horizontal = 6.dp, vertical = 2.dp)
                        ) {
                            Text(
                                text = String.format(Locale.US, "%.1f%% Margin", marginPct),
                                style = MaterialTheme.typography.labelSmall.copy(fontWeight =
FontWeight.Bold),
                                color = Color(0xFF75B890)
                            )
                        }
                    }
                    Text(
                        text = "Cost: ${CostCalculator.formatPaiseToRupees(product.movingAvgCostPaise)}",
                        style = MaterialTheme.typography.bodySmall,
```

```
                        color = MaterialTheme.colorScheme.onSurfaceVariant
                    )
                }
            }
        }
    }
}

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun IntegratedAddStockProductDialog(
    existingProduct: ProductEntity?,
    suppliers: List<SupplierEntity>,
    onAddSupplier: (SupplierEntity, (SupplierEntity) -> Unit) -> Unit = { _, _ -> },
    onDismiss: () -> Unit,
    onSave: (
        existingId: String?,
        name: String,
        barcode: String?,
        category: String,
        brand: String?,
        packSize: Int,
        boxesBrought: Int,
        loosePieces: Int,
        costPaise: Long,
        sellingPaise: Long,
        wholesalePaise: Long,
        lowStock: Int,
        supplierName: String?,
        billNo: String?,
        transportPaise: Long,
        isPaidNow: Boolean,
        paymentMethod: String
    ) -> Unit
) {
    val context = LocalContext.current
    val lifecycleOwner = LocalLifecycleOwner.current

    var supplierName by remember { mutableStateOf("") }
    var selectedSupplier by remember { mutableStateOf<SupplierEntity?>(null) }
    var supplierMenuExpanded by remember { mutableStateOf(false) }
    var showAddSupplierInlineDialog by remember { mutableStateOf(false) }
    var billNo by remember { mutableStateOf("") }
    var transportChargesStr by remember { mutableStateOf("0") }

    var name by remember { mutableStateOf(existingProduct?.name ?: "") }
    var barcode by remember { mutableStateOf(existingProduct?.barcode ?: "") }
    var category by remember { mutableStateOf(existingProduct?.category ?: "Stationery") }
    var brand by remember { mutableStateOf(existingProduct?.brand ?: "") }

    var piecesStr by remember {
        mutableStateOf(if (existingProduct != null) existingProduct.currentSellableStock.toString() else "0")
    }

    var costPerPieceStr by remember {
        mutableStateOf(if (existingProduct != null && existingProduct.movingAvgCostPaise > 0L) (existingProduct.movingAvgCostPaise / 100.0).toString() else "0")
    }

    var sellingPieceStr by remember {
        mutableStateOf(if (existingProduct != null && existingProduct.sellingPricePiecePaise > 0L) (existingProduct.sellingPricePiecePaise / 100.0).toString() else "0")
    }

    var wholesalePieceStr by remember {
        mutableStateOf(if (existingProduct != null && existingProduct.wholesalePricePiecePaise > 0L) (existingProduct.wholesalePricePiecePaise / 100.0).toString() else "")
    }

    var marginPercentStr by remember {
        val cost = (existingProduct?.movingAvgCostPaise ?: 0L) / 100.0
        val selling = (existingProduct?.sellingPricePiecePaise ?: 0L) / 100.0
        val initialMargin = if (selling > 0 && selling > cost) ((selling - cost) / selling) * 100.0 else 0.0
        mutableStateOf(if (initialMargin > 0) String.format(Locale.US, "%.1f", initialMargin) else "0")
    }

    var lowStockStr by remember { mutableStateOf(existingProduct?.lowStockThreshold?.toString() ?: "5") }

    var isPaidNow by remember { mutableStateOf(false) }
    var billPaymentMethod by remember { mutableStateOf("CASH") }

    // Camera Barcode Scanner State
    var showBarcodeScanner by remember { mutableStateOf(false) }
    var hasCameraPermission by remember {
        mutableStateOf(
            ContextCompat.checkSelfPermission(context, Manifest.permission.CAMERA) == PackageManager.PERMISSION_GRANTED
        )
```

```
    }
    val permissionLauncher = rememberLauncherForActivityResult(
        contract = ActivityResultContracts.RequestPermission()
    ) { granted ->
        hasCameraPermission = granted
        if (granted) {
            showBarcodeScanner = true
        }
    }

    // Computations
    val packSize = existingProduct?.packSize ?: 1
    val totalPiecesAdded = piecesStr.replace(",", "").trim().toIntOrNull() ?: 0
    val boxesBrought = 0
    val loosePieces = totalPiecesAdded

    val baseCost = costPerPieceStr.replace(",", "").trim().toDoubleOrNull() ?: 0.0
    val transportTotal = transportChargesStr.replace(",", "").trim().toDoubleOrNull() ?: 0.0
    val transportPerPiece = if (totalPiecesAdded > 0 && transportTotal > 0.0) {
        transportTotal / totalPiecesAdded
    } else 0.0
    val landedCost = baseCost + transportPerPiece

    val selling = sellingPieceStr.replace(",", "").trim().toDoubleOrNull() ?: 0.0
    val profitBeforeTransport = selling - baseCost
    val profitAfterTransport = selling - landedCost

    val calculatedMarginPct = if (selling > 0.0) {
        (profitAfterTransport / selling) * 100.0
    } else 0.0

    AlertDialog(
        onDismissRequest = onDismiss,
        properties = DialogProperties(dismissOnClickOutside = false),
        title = {
            Text(if (existingProduct != null) "Edit Product & Stock" else "Add Stock & Product", fontWeight
= FontWeight.Bold)
        },
        text = {
            Column(
                modifier = Modifier
                    .fillMaxWidth()
                    .verticalScroll(rememberScrollState()),
                verticalArrangement = Arrangement.spacedBy(8.dp)
            ) {
                // 1. Product Name *
                OutlinedTextField(
                    value = name,
                    onValueChange = { name = it },
                    label = { Text("Product Name *") },
                    placeholder = { Text("Product Name") },
                    singleLine = true,
                    modifier = Modifier.fillMaxWidth()
                )

                // 2. Barcode (Optional) with working Camera Scanner button
                OutlinedTextField(
                    value = barcode,
                    onValueChange = { barcode = it },
                    label = { Text("Barcode (Optional)") },
                    placeholder = { Text("Scan or enter barcode") },
                    trailingIcon = {
                        IconButton(
                            onClick = {
                                if (hasCameraPermission) {
                                    showBarcodeScanner = true
                                } else {
                                    permissionLauncher.launch(Manifest.permission.CAMERA)
                                }
                            }
                        ) {
                            Icon(
                                imageVector = Icons.Filled.QrCodeScanner,
                                contentDescription = "Scan Barcode",
                                tint = MaterialTheme.colorScheme.primary
                            )
                        }
                    },
                    singleLine = true,
                    modifier = Modifier.fillMaxWidth()
                )

                // 3. Category & Brand
                Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement = Arrangement.spacedBy(8.dp))
{
                    OutlinedTextField(
                        value = category,
                        onValueChange = { category = it },
```

```
                        label = { Text("Category") },
                        singleLine = true,
                        modifier = Modifier.weight(1f)
                    )
                    OutlinedTextField(
                        value = brand,
                        onValueChange = { brand = it },
                        label = { Text("Brand") },
                        singleLine = true,
                        modifier = Modifier.weight(1f)
                    )
                }

                // 4. Pcs & Low Stock Alert
                Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement = Arrangement.spacedBy(8.dp))
    {
                    OutlinedTextField(
                        value = piecesStr,
                        onValueChange = { piecesStr = it },
                        label = { Text("Pcs") },
                        placeholder = { Text("0") },
                        keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Number),
                        singleLine = true,
                        modifier = Modifier.weight(1f)
                    )
                    OutlinedTextField(
                        value = lowStockStr,
                        onValueChange = { lowStockStr = it },
                        label = { Text("Low Stock Alert") },
                        placeholder = { Text("5") },
                        keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Number),
                        singleLine = true,
                        modifier = Modifier.weight(1f)
                    )
                }

                // 5. Cost & Selling Price
                Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement = Arrangement.spacedBy(8.dp))
    {
                    OutlinedTextField(
                        value = costPerPieceStr,
                        onValueChange = {
                            costPerPieceStr = it
                            val c = it.toDoubleOrNull() ?: 0.0
                            val effCost = c + transportPerPiece
                            val s = sellingPieceStr.toDoubleOrNull() ?: 0.0
                            if (s > 0 && s > effCost) {
                                marginPercentStr = String.format(Locale.US, "%.1f", ((s - effCost) / s) *
    100.0)
                            }
                        },
                        label = { Text("Base Cost (₹/pc) *") },
                        keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Decimal),
                        singleLine = true,
                        modifier = Modifier.weight(1f)
                    )

                    OutlinedTextField(
                        value = sellingPieceStr,
                        onValueChange = {
                            sellingPieceStr = it
                            val s = it.toDoubleOrNull() ?: 0.0
                            if (s > 0 && s > landedCost) {
                                marginPercentStr = String.format(Locale.US, "%.1f", ((s - landedCost) / s) *
     100.0)
                            }
                        },
                        label = { Text("Selling (₹/pc) *") },
                        keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Decimal),
                        singleLine = true,
                        modifier = Modifier.weight(1f)
                    )
                }

                // 6. Margin (%) & Wholesale (₹/pc)
                Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement = Arrangement.spacedBy(8.dp))
    {
                    OutlinedTextField(
                        value = marginPercentStr,
                        onValueChange = {
                            marginPercentStr = it
                            val m = it.toDoubleOrNull() ?: 0.0
                            if (landedCost > 0 && m < 100.0 && m > 0.0) {
                                val calculatedSelling = landedCost / (1.0 - (m / 100.0))
                                sellingPieceStr = String.format(Locale.US, "%.2f", calculatedSelling)
                            }
                        },
                        label = { Text("Margin (%)") },
```

```
                        keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Decimal),
                        singleLine = true,
                        modifier = Modifier.weight(1f)
                    )

                    OutlinedTextField(
                        value = wholesalePieceStr,
                        onValueChange = { wholesalePieceStr = it },
                        label = { Text("Wholesale (₹/pc)") },
                        placeholder = { Text("Optional") },
                        keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Decimal),
                        singleLine = true,
                        modifier = Modifier.weight(1f)
                    )
                }

                // 7. Supplier Selector (Full Width)
                Column(modifier = Modifier.fillMaxWidth()) {
                    Text(
                        text = "Supplier",
                        style = MaterialTheme.typography.labelMedium.copy(fontWeight = FontWeight.SemiBold),
                        color = MaterialTheme.colorScheme.onSurfaceVariant
                    )
                    Spacer(modifier = Modifier.height(4.dp))
                    ExposedDropdownMenuBox(
                        expanded = supplierMenuExpanded,
                        onExpandedChange = { supplierMenuExpanded = !supplierMenuExpanded },
                        modifier = Modifier.fillMaxWidth()
                    ) {
                        OutlinedTextField(
                            value = supplierName,
                            onValueChange = {
                                supplierName = it
                                supplierMenuExpanded = true
                            },
                            label = { Text("Search or select supplier") },
                            placeholder = { Text("Select supplier from directory") },
                            trailingIcon = { ExposedDropdownMenuDefaults.TrailingIcon(expanded =
supplierMenuExpanded) },
                            singleLine = true,
                            modifier = Modifier
                                .fillMaxWidth()
                                .menuAnchor()
                        )
                        val matchingSuppliers = remember(suppliers, supplierName) {
                            if (supplierName.isBlank()) suppliers
                            else suppliers.filter { it.name.contains(supplierName, ignoreCase = true) }
                        }
                        ExposedDropdownMenu(
                            expanded = supplierMenuExpanded,
                            onDismissRequest = { supplierMenuExpanded = false }
                        ) {
                            DropdownMenuItem(
                                text = {
                                    Row(verticalAlignment = Alignment.CenterVertically) {
                                        Icon(Icons.Filled.Add, contentDescription = null, tint =
MaterialTheme.colorScheme.primary, modifier = Modifier.size(16.dp))
                                        Spacer(modifier = Modifier.width(6.dp))
                                        Text("+ Add new supplier", fontWeight = FontWeight.Bold, color =
MaterialTheme.colorScheme.primary)
                                    }
                                },
                                onClick = {
                                    supplierMenuExpanded = false
                                    showAddSupplierInlineDialog = true
                                }
                            )
                            HorizontalDivider()
                            matchingSuppliers.forEach { sup ->
                                DropdownMenuItem(
                                    text = {
                                        Column {
                                            Text(sup.name, fontWeight = FontWeight.SemiBold)
                                            if (!sup.phone.isNullOrBlank()) {
                                                Text(sup.phone, style = MaterialTheme.typography.bodySmall,
color = MaterialTheme.colorScheme.onSurfaceVariant)
                                            }
                                        }
                                    },
                                    onClick = {
                                        selectedSupplier = sup
                                        supplierName = sup.name
                                        supplierMenuExpanded = false
                                    }
                                )
                            }
                        }
                    }
```

```
                }

                // Bill No. (Dedicated Row)
                OutlinedTextField(
                    value = billNo,
                    onValueChange = { billNo = it },
                    label = { Text("Bill / Invoice No.") },
                    placeholder = { Text("Optional invoice number") },
                    leadingIcon = { Icon(Icons.Filled.Receipt, contentDescription = null, modifier =
Modifier.size(16.dp)) },
                    singleLine = true,
                    modifier = Modifier.fillMaxWidth()
                )

                // 8. Transport Charges
                OutlinedTextField(
                    value = transportChargesStr,
                    onValueChange = { transportChargesStr = it },
                    label = { Text("Transport Charges (₹)") },
                    placeholder = { Text("0") },
                    leadingIcon = { Icon(Icons.Filled.LocalShipping, contentDescription = null, modifier =
Modifier.size(18.dp)) },
                    keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Decimal),
                    singleLine = true,
                    modifier = Modifier.fillMaxWidth()
                )

                // Supplier Bill Payment Status
                if (supplierName.isNotBlank() || billNo.isNotBlank()) {
                    Column(
                        modifier = Modifier
                            .fillMaxWidth()
                            .background(MaterialTheme.colorScheme.surfaceVariant.copy(alpha = 0.35f),
RoundedCornerShape(8.dp))
                            .padding(8.dp),
                        verticalArrangement = Arrangement.spacedBy(6.dp)
                    ) {
                        Text(
                            text = "Supplier Bill Payment",
                            style = MaterialTheme.typography.labelMedium.copy(fontWeight = FontWeight.Bold),
                            color = MaterialTheme.colorScheme.primary
                        )
                        Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement =
Arrangement.spacedBy(8.dp)) {
                            FilterChip(
                                selected = !isPaidNow,
                                onClick = { isPaidNow = false },
                                label = { Text("Unpaid (Add to Dues)") },
                                modifier = Modifier.weight(1f)
                            )
                            FilterChip(
                                selected = isPaidNow,
                                onClick = { isPaidNow = true },
                                label = { Text("Paid Now") },
                                modifier = Modifier.weight(1f)
                            )
                        }
                        if (isPaidNow) {
                            Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement =
Arrangement.spacedBy(8.dp)) {
                                FilterChip(
                                    selected = billPaymentMethod == "CASH",
                                    onClick = { billPaymentMethod = "CASH" },
                                    label = { Text("Cash") },
                                    modifier = Modifier.weight(1f)
                                )
                                FilterChip(
                                    selected = billPaymentMethod == "UPI",
                                    onClick = { billPaymentMethod = "UPI" },
                                    label = { Text("UPI / GPay") },
                                    modifier = Modifier.weight(1f)
                                )
                                FilterChip(
                                    selected = billPaymentMethod == "BANK",
                                    onClick = { billPaymentMethod = "BANK" },
                                    label = { Text("Bank") },
                                    modifier = Modifier.weight(1f)
                                )
                            }
                        }
                    }
                }

                // 9. Profit After Transport Breakdown
                Card(
                    modifier = Modifier.fillMaxWidth(),
                    colors = CardDefaults.cardColors(containerColor =
MaterialTheme.colorScheme.surfaceVariant.copy(alpha = 0.5f)),
```

```
                    shape = RoundedCornerShape(8.dp)
                ) {
                    Column(modifier = Modifier.padding(10.dp), verticalArrangement =
Arrangement.spacedBy(4.dp)) {
                        Text(
                            text = "Profit after transport",
                            style = MaterialTheme.typography.labelMedium.copy(fontWeight = FontWeight.Bold),
                            color = MaterialTheme.colorScheme.primary
                        )

                        if (selling > 0) {
                            if (transportTotal > 0.0 && totalPiecesAdded > 0) {
                                Text(
                                    text = "Profit after transport: ₹${String.format(Locale.US, "%.2f",
profitAfterTransport)}/pc (${String.format(Locale.US, "%.1f", calculatedMarginPct)}% margin)",
                                    style = MaterialTheme.typography.bodySmall.copy(fontWeight =
FontWeight.Bold),
                                    color = if (profitAfterTransport >= 0) Color(0xFF75B890) else
Color(0xFFE57373)
                                )
                                Text(
                                    text = "Before transport was: ₹${String.format(Locale.US, "%.2f",
profitBeforeTransport)}/pc • Landed Cost: ₹${String.format(Locale.US, "%.2f", landedCost)}/pc",
                                    style = MaterialTheme.typography.labelSmall,
                                    color = MaterialTheme.colorScheme.onSurfaceVariant
                                )
                            } else {
                                Text(
                                    text = "Profit per piece: ₹${String.format(Locale.US, "%.2f",
profitAfterTransport)} (${String.format(Locale.US, "%.1f", calculatedMarginPct)}% margin)",
                                    style = MaterialTheme.typography.bodySmall.copy(fontWeight =
FontWeight.SemiBold),
                                    color = if (profitAfterTransport >= 0) Color(0xFF75B890) else
Color(0xFFE57373)
                                )
                            }
                        }
                    }
                }
            }
        },
        confirmButton = {
            Button(
                onClick = {
                    if (name.isBlank()) return@Button
                    val costPaise = ((baseCost * 100)).toLong()
                    val transportPaise = ((transportTotal * 100)).toLong()
                    val sellingPaise = ((selling * 100)).toLong()
                    val wholesalePaise = ((wholesalePieceStr.replace(",", "").trim().toDoubleOrNull() ?:
0.0) * 100).toLong()
                    val lowStock = lowStockStr.replace(",", "").trim().toIntOrNull() ?: 5

                    onSave(
                        existingProduct?.id,
                        name.trim(),
                        barcode.trim().ifBlank { null },
                        category.trim().ifBlank { "Stationery" },
                        brand.trim().ifBlank { null },
                        packSize,
                        boxesBrought,
                        loosePieces,
                        costPaise,
                        sellingPaise,
                        wholesalePaise,
                        lowStock,
                        supplierName.trim().ifBlank { null },
                        billNo.trim().ifBlank { null },
                        transportPaise,
                        isPaidNow,
                        billPaymentMethod
                    )
                }
            ) {
                Text("Save")
            }
        },
        dismissButton = {
            OutlinedButton(onClick = onDismiss) {
                Text("Cancel")
            }
        }
    )

    // Camera Barcode Scanner Dialog
    if (showBarcodeScanner) {
        Dialog(onDismissRequest = { showBarcodeScanner = false }) {
            Card(
                modifier = Modifier
```

```
                    .fillMaxWidth()
                    .padding(8.dp),
                shape = RoundedCornerShape(16.dp),
                colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surface)
            ) {
                Column(
                    modifier = Modifier.padding(16.dp),
                    horizontalAlignment = Alignment.CenterHorizontally,
                    verticalArrangement = Arrangement.spacedBy(12.dp)
                ) {
                    Text(
                        text = "Scan Barcode",
                        style = MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.Bold)
                    )
                    Box(
                        modifier = Modifier
                            .fillMaxWidth()
                            .height(240.dp)
                            .clip(RoundedCornerShape(12.dp))
                            .background(Color.Black)
                    ) {
                        AndroidView(
                            factory = { ctx ->
                                val previewView = PreviewView(ctx)
                                val cameraProviderFuture = ProcessCameraProvider.getInstance(ctx)
                                val cameraExecutor = Executors.newSingleThreadExecutor()

                                cameraProviderFuture.addListener({
                                    val cameraProvider = cameraProviderFuture.get()
                                    val preview = Preview.Builder().build().also {
                                        it.surfaceProvider = previewView.surfaceProvider
                                    }
                                    val imageAnalyzer = ImageAnalysis.Builder()
                                        .setBackpressureStrategy(ImageAnalysis.STRATEGY_KEEP_ONLY_LATEST)
                                        .build()
                                        .also { analysis ->
                                            analysis.setAnalyzer(
                                                cameraExecutor,
                                                BarcodeAnalyzer(
                                                    onBarcodeDetected = { scannedCode, _ ->
                                                        barcode = scannedCode
                                                        showBarcodeScanner = false
                                                    }
                                                )
                                            )

                                        }
                                    val cameraSelector = CameraSelector.DEFAULT_BACK_CAMERA
                                    try {
                                        cameraProvider.unbindAll()
                                        cameraProvider.bindToLifecycle(
                                            lifecycleOwner,
                                            cameraSelector,
                                            preview,
                                            imageAnalyzer
                                        )
                                    } catch (e: Exception) {}
                                }, ContextCompat.getMainExecutor(ctx))
                                previewView
                            },
                            modifier = Modifier.fillMaxSize()
                        )
                        Box(
                            modifier = Modifier
                                .size(width = 200.dp, height = 110.dp)
                                .align(Alignment.Center)
                                .border(2.dp, MaterialTheme.colorScheme.primary, RoundedCornerShape(8.dp))
                        )
                    }
                    Button(
                        onClick = { showBarcodeScanner = false },
                        colors = ButtonDefaults.buttonColors(containerColor =
    MaterialTheme.colorScheme.surfaceVariant)
                    ) {
                        Text("Cancel", color = MaterialTheme.colorScheme.onSurfaceVariant)
                    }
                }
            }
        }
    }
}
```

app\src\main\java\com\alieducational\app\ui\theme\Color.kt

```
 1  package com.alieducational.app.ui.theme
 2
 3  import androidx.compose.ui.graphics.Color
 4
 5  // ── Dark Theme Colors ──
 6  val DarkBackground = Color(0xFF111411)
 7  val DarkSurface = Color(0xFF1C211D)
 8  val DarkOnSurface = Color(0xFFF6F2E9)
 9  val DarkOnSurfaceVariant = Color(0xFFB9C0B8)
10  val DarkOutline = Color(0xFF354038)
11
12  // ── Light Theme Colors (Targeting >= 4.5:1 text contrast) ──
13  val LightBackground = Color(0xFFF6F7F5)
14  val LightSurface = Color(0xFFFFFFFF)
15  val LightOnSurface = Color(0xFF141915)
16  val LightOnSurfaceVariant = Color(0xFF47524A)
17  val LightOutline = Color(0xFFCCD4CD)
18
19  // ── Brand / Accent Colors ──
20  val GoldDark = Color(0xFFD9AC46)
21  val GoldLight = Color(0xFF8A6014) // Darker gold for WCAG AA compliance with white
22
23  // ── Semantic High-Contrast Colors ──
24  val PositiveGreenDark = Color(0xFF75B890)
25  val PositiveGreenLight = Color(0xFF1B5E20) // WCAG 8.6:1 contrast with white
26  val NegativeRedDark = Color(0xFFE57373)
27  val NegativeRedLight = Color(0xFFC62828) // WCAG 5.9:1 contrast with white
28  val WarningOrangeDark = Color(0xFFE4B765)
29  val WarningOrangeLight = Color(0xFFB76E00) // WCAG 4.7:1 contrast with white
```

app\src\main\java\com\alieducational\app\ui\theme\Theme.kt

```
package com.alieducational.app.ui.theme

import android.app.Activity
import androidx.compose.foundation.isSystemInDarkTheme
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.darkColorScheme
import androidx.compose.material3.lightColorScheme
import androidx.compose.runtime.Composable
import androidx.compose.runtime.SideEffect
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.toArgb
import androidx.compose.ui.platform.LocalView
import androidx.core.view.WindowCompat

private val ErrorColorDark = Color(0xFFCF6679)
private val ErrorColorLight = Color(0xFFBA1A1A)

private val DarkColorScheme = darkColorScheme(
    primary = GoldDark,
    onPrimary = DarkBackground,
    primaryContainer = GoldDark.copy(alpha = 0.2f),
    onPrimaryContainer = GoldDark,
    secondary = GoldDark,
    onSecondary = DarkBackground,
    secondaryContainer = GoldDark.copy(alpha = 0.15f),
    onSecondaryContainer = GoldDark,
    tertiary = PositiveGreenDark,
    onTertiary = DarkBackground,
    background = DarkBackground,
    onBackground = DarkOnSurface,
    surface = DarkSurface,
    onSurface = DarkOnSurface,
    surfaceVariant = DarkSurface,
    onSurfaceVariant = DarkOnSurfaceVariant,
    outline = DarkOutline,
    outlineVariant = DarkOutline.copy(alpha = 0.5f),
    error = ErrorColorDark,
    onError = DarkBackground,
)

private val LightColorScheme = lightColorScheme(
    primary = GoldLight,
    onPrimary = Color.White,
    primaryContainer = Color(0xFFF4EAD7),
    onPrimaryContainer = Color(0xFF5D400A),
    secondary = GoldLight,
    onSecondary = Color.White,
    secondaryContainer = Color(0xFFF4EAD7),
    onSecondaryContainer = Color(0xFF5D400A),
    tertiary = PositiveGreenLight,
    onTertiary = Color.White,
    background = LightBackground,
    onBackground = LightOnSurface,
    surface = LightSurface,
    onSurface = LightOnSurface,
    surfaceVariant = LightBackground,
    onSurfaceVariant = LightOnSurfaceVariant,
    outline = LightOutline,
    outlineVariant = LightOutline.copy(alpha = 0.5f),
    error = ErrorColorLight,
    onError = Color.White,
)

@Composable
fun AliEducationalTheme(
    darkTheme: Boolean = isSystemInDarkTheme(),
    content: @Composable () -> Unit,
) {
    val colorScheme = if (darkTheme) DarkColorScheme else LightColorScheme

    val view = LocalView.current
    if (!view.isInEditMode) {
        SideEffect {
            val window = (view.context as Activity).window
            @Suppress("DEPRECATION")
            window.statusBarColor = colorScheme.background.toArgb()
            WindowCompat.getInsetsController(window, view).isAppearanceLightStatusBars = !darkTheme
        }
    }

    MaterialTheme(
        colorScheme = colorScheme,
```

```
83          typography = AliTypography,
84          content = content,
85      )
86  }
```

app\src\main\java\com\alieducational\app\ui\theme\Type.kt

```
package com.alieducational.app.ui.theme

import androidx.compose.material3.Typography
import androidx.compose.ui.text.TextStyle
import androidx.compose.ui.text.font.FontFamily
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.sp

val NotoSans = FontFamily.Default
val NotoNastaliqUrdu = FontFamily.Default

val AliTypography = Typography(
    displayLarge = TextStyle(
        fontFamily = NotoSans,
        fontWeight = FontWeight.Bold,
        fontSize = 32.sp,
        lineHeight = 40.sp,
    ),
    displayMedium = TextStyle(
        fontFamily = NotoSans,
        fontWeight = FontWeight.Bold,
        fontSize = 28.sp,
        lineHeight = 36.sp,
    ),
    displaySmall = TextStyle(
        fontFamily = NotoSans,
        fontWeight = FontWeight.SemiBold,
        fontSize = 24.sp,
        lineHeight = 32.sp,
    ),
    headlineLarge = TextStyle(
        fontFamily = NotoSans,
        fontWeight = FontWeight.Bold,
        fontSize = 28.sp,
        lineHeight = 36.sp,
    ),
    headlineMedium = TextStyle(
        fontFamily = NotoSans,
        fontWeight = FontWeight.SemiBold,
        fontSize = 24.sp,
        lineHeight = 32.sp,
    ),
    headlineSmall = TextStyle(
        fontFamily = NotoSans,
        fontWeight = FontWeight.SemiBold,
        fontSize = 20.sp,
        lineHeight = 28.sp,
    ),
    titleLarge = TextStyle(
        fontFamily = NotoSans,
        fontWeight = FontWeight.SemiBold,
        fontSize = 22.sp,
        lineHeight = 28.sp,
    ),
    titleMedium = TextStyle(
        fontFamily = NotoSans,
        fontWeight = FontWeight.Medium,
        fontSize = 18.sp,
        lineHeight = 24.sp,
    ),
    titleSmall = TextStyle(
        fontFamily = NotoSans,
        fontWeight = FontWeight.Medium,
        fontSize = 14.sp,
        lineHeight = 20.sp,
    ),
    bodyLarge = TextStyle(
        fontFamily = NotoSans,
        fontWeight = FontWeight.Normal,
        fontSize = 16.sp,
        lineHeight = 24.sp,
    ),
    bodyMedium = TextStyle(
        fontFamily = NotoSans,
        fontWeight = FontWeight.Normal,
        fontSize = 14.sp,
        lineHeight = 20.sp,
    ),
    bodySmall = TextStyle(
        fontFamily = NotoSans,
        fontWeight = FontWeight.Normal,
        fontSize = 12.sp,
        lineHeight = 16.sp,
    ),
    labelLarge = TextStyle(
```

```
        fontFamily = NotoSans,
        fontWeight = FontWeight.Medium,
        fontSize = 14.sp,
        lineHeight = 20.sp,
    ),
    labelMedium = TextStyle(
        fontFamily = NotoSans,
        fontWeight = FontWeight.Medium,
        fontSize = 12.sp,
        lineHeight = 16.sp,
    ),
    labelSmall = TextStyle(
        fontFamily = NotoSans,
        fontWeight = FontWeight.Medium,
        fontSize = 10.sp,
        lineHeight = 14.sp,
    ),
)
```

app\src\main\java\com\alieducational\app\util\AttachmentManager.kt

```
package com.alieducational.app.util

import android.content.Context
import android.graphics.Bitmap
import android.graphics.BitmapFactory
import android.graphics.Matrix
import android.media.ExifInterface
import android.net.Uri
import androidx.core.content.FileProvider
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.withContext
import java.io.File
import java.io.FileOutputStream
import java.io.InputStream
import java.util.UUID

object AttachmentManager {

    private const val ATTACHMENTS_DIR_NAME = "attachments"
    private const val MAX_DIMENSION = 1920
    private const val JPEG_QUALITY = 85

    fun getAttachmentsDir(context: Context): File {
        val dir = File(context.filesDir, ATTACHMENTS_DIR_NAME)
        if (!dir.exists()) {
            dir.mkdirs()
        }
        return dir
    }

    /**
     * Resolves a stored attachmentUri (which may be a relative path, filename, or absolute path)
     * to a valid File inside durable app storage, ensuring restored backups work on any device.
     */
    fun getAttachmentFile(context: Context, pathOrUri: String?): File? {
        if (pathOrUri.isNullOrBlank()) return null

        val fileName = File(pathOrUri).name
        val durableFile = File(getAttachmentsDir(context), fileName)
        if (durableFile.exists() && durableFile.length() > 0) {
            return durableFile
        }

        val directFile = File(pathOrUri)
        if (directFile.exists() && directFile.length() > 0) {
            return directFile
        }


        return null
    }

    /**
     * Creates a temporary file in cache and returns a FileProvider URI suitable for camera capture.
     */
    fun createTempCameraUri(context: Context): Pair<Uri, File> {
        val tempDir = File(context.cacheDir, "camera_temp")
        if (!tempDir.exists()) tempDir.mkdirs()

        val tempFile = File(tempDir, "capture_${System.currentTimeMillis()}.jpg")
        val uri = FileProvider.getUriForFile(
            context,
            "${context.packageName}.fileprovider",
            tempFile
        )
        return Pair(uri, tempFile)
    }

    /**
     * Copies and normalizes an image from a content/file URI into durable app-managed attachment storage,
     * correcting EXIF orientation and keeping file size lightweight while retaining readability.
     * Returns the relative path "attachments/attachment_<UUID>.jpg".
     */
    suspend fun persistImageAttachment(context: Context, sourceUri: Uri): String? =
withContext(Dispatchers.IO) {
        try {
            val attachmentsDir = getAttachmentsDir(context)
            val outputFileName = "attachment_${UUID.randomUUID()}.jpg"
            val outputFile = File(attachmentsDir, outputFileName)

            // 1. Check EXIF orientation
            var orientation = ExifInterface.ORIENTATION_NORMAL
            try {
```

```
                context.contentResolver.openInputStream(sourceUri)?.use { stream ->
                    val exif = ExifInterface(stream)
                    orientation = exif.getAttributeInt(
                        ExifInterface.TAG_ORIENTATION,
                        ExifInterface.ORIENTATION_NORMAL
                    )
                }
            } catch (ignored: Exception) {}

            // 2. Decode bitmap bounds first to scale down if necessary
            val options = BitmapFactory.Options().apply { inJustDecodeBounds = true }
            context.contentResolver.openInputStream(sourceUri)?.use { stream ->
                BitmapFactory.decodeStream(stream, null, options)
            }

            var inSampleSize = 1
            val origWidth = options.outWidth
            val origHeight = options.outHeight

            if (origWidth > MAX_DIMENSION || origHeight > MAX_DIMENSION) {
                val halfHeight = origHeight / 2
                val halfWidth = origWidth / 2
                while ((halfHeight / inSampleSize) >= MAX_DIMENSION && (halfWidth / inSampleSize) >= MAX_DIMENSION) {
                    inSampleSize *= 2
                }
            }

            // 3. Decode bitmap
            val decodeOptions = BitmapFactory.Options().apply {
                this.inSampleSize = inSampleSize
            }
            val decodedBitmap: Bitmap? = context.contentResolver.openInputStream(sourceUri)?.use { stream ->
                BitmapFactory.decodeStream(stream, null, decodeOptions)
            }
            val originalBitmap: Bitmap = decodedBitmap ?: return@withContext null

            // 4. Rotate bitmap if required by EXIF
            val rotationDegrees = when (orientation) {
                ExifInterface.ORIENTATION_ROTATE_90 -> 90f
                ExifInterface.ORIENTATION_ROTATE_180 -> 180f
                ExifInterface.ORIENTATION_ROTATE_270 -> 270f
                else -> 0f
            }

            val finalBitmap: Bitmap = if (rotationDegrees != 0f) {
                val matrix = Matrix().apply { postRotate(rotationDegrees) }
                val rotated = Bitmap.createBitmap(
                    originalBitmap,
                    0, 0,
                    originalBitmap.width,
                    originalBitmap.height,
                    matrix,
                    true
                )
                if (rotated != originalBitmap) originalBitmap.recycle()
                rotated
            } else {
                originalBitmap
            }

            // 5. Compress to durable JPEG file
            FileOutputStream(outputFile).use { out ->
                finalBitmap.compress(Bitmap.CompressFormat.JPEG, JPEG_QUALITY, out)
            }
            finalBitmap.recycle()

            "attachments/$outputFileName"
        } catch (e: Exception) {
            e.printStackTrace()
            null
        }
    }

    /**
     * Safely deletes an attachment file from storage.
     */
    fun deleteAttachment(context: Context, pathOrUri: String?) {
        if (pathOrUri.isNullOrBlank()) return
        try {
            val file = getAttachmentFile(context, pathOrUri)
            if (file != null && file.exists()) {
                file.delete()
            }
        } catch (ignored: Exception) {}
    }
}
```

app\src\main\java\com\alieducational\app\util\BiometricAuthManager.kt

```kotlin
package com.alieducational.app.util

import android.content.Context
import androidx.biometric.BiometricManager
import androidx.biometric.BiometricManager.Authenticators.BIOMETRIC_STRONG
import androidx.biometric.BiometricManager.Authenticators.DEVICE_CREDENTIAL
import androidx.biometric.BiometricPrompt
import androidx.core.content.ContextCompat
import androidx.fragment.app.FragmentActivity
import dagger.hilt.android.qualifiers.ApplicationContext
import javax.inject.Inject
import javax.inject.Singleton

@Singleton
class BiometricAuthManager @Inject constructor(
    @ApplicationContext private val context: Context
) {

    fun canAuthenticate(): Boolean {
        val biometricManager = BiometricManager.from(context)
        val canAuth = biometricManager.canAuthenticate(BIOMETRIC_STRONG or DEVICE_CREDENTIAL)
        return canAuth == BiometricManager.BIOMETRIC_SUCCESS
    }

    fun promptBiometric(
        activity: FragmentActivity,
        title: String = "Unlock Ali Educational",
        subtitle: String = "Confirm your fingerprint or device credential",
        onSuccess: () -> Unit,
        onError: (String) -> Unit
    ) {
        val executor = ContextCompat.getMainExecutor(activity)
        val prompt = BiometricPrompt(
            activity,
            executor,
            object : BiometricPrompt.AuthenticationCallback() {
                override fun onAuthenticationSucceeded(result: BiometricPrompt.AuthenticationResult) {
                    super.onAuthenticationSucceeded(result)
                    onSuccess()
                }

                override fun onAuthenticationError(errorCode: Int, errString: CharSequence) {
                    super.onAuthenticationError(errorCode, errString)
                    onError(errString.toString())
                }

                override fun onAuthenticationFailed() {
                    super.onAuthenticationFailed()
                    onError("Authentication failed. Please try again.")
                }
            }
        )

        val promptInfo = BiometricPrompt.PromptInfo.Builder()
            .setTitle(title)
            .setSubtitle(subtitle)
            .setAllowedAuthenticators(BIOMETRIC_STRONG or DEVICE_CREDENTIAL)
            .build()

        prompt.authenticate(promptInfo)
    }
}
```

app\src\main\java\com\alieducational\app\widget\AliSlimBarWidgetProvider.kt

```kotlin
package com.alieducational.app.widget

import android.app.PendingIntent
import android.appwidget.AppWidgetManager
import android.appwidget.AppWidgetProvider
import android.content.ComponentName
import android.content.Context
import android.content.Intent
import android.net.Uri
import android.os.Bundle
import android.widget.RemoteViews
import com.alieducational.app.MainActivity
import com.alieducational.app.R

class AliSlimBarWidgetProvider : AppWidgetProvider() {

    override fun onUpdate(
        context: Context,
        appWidgetManager: AppWidgetManager,
        appWidgetIds: IntArray
    ) {
        for (appWidgetId in appWidgetIds) {
            updateAppWidget(context, appWidgetManager, appWidgetId)
        }
    }

    override fun onAppWidgetOptionsChanged(
        context: Context,
        appWidgetManager: AppWidgetManager,
        appWidgetId: Int,
        newOptions: Bundle?
    ) {
        updateAppWidget(context, appWidgetManager, appWidgetId)
        super.onAppWidgetOptionsChanged(context, appWidgetManager, appWidgetId, newOptions)
    }

    override fun onReceive(context: Context, intent: Intent) {
        super.onReceive(context, intent)
        updateAllWidgets(context)
    }

    override fun onEnabled(context: Context) {
        super.onEnabled(context)
        updateAllWidgets(context)
    }

    companion object {
        const val ACTION_WIDGET_CLICK = "com.alieducational.app.action.WIDGET_CLICK"
        const val EXTRA_WIDGET_ACTION = "extra_widget_action"

        const val ACTION_CASH_IN = "CASH_IN"
        const val ACTION_CASH_OUT = "CASH_OUT"
        const val ACTION_NEW_BILL = "NEW_BILL"
        const val ACTION_ADD_STOCK = "ADD_STOCK"
        const val ACTION_HOME = "HOME"

        fun updateAllWidgets(context: Context) {
            try {
                val appWidgetManager = AppWidgetManager.getInstance(context) ?: return
                val myProvider = ComponentName(context, AliSlimBarWidgetProvider::class.java)
                val ids = appWidgetManager.getAppWidgetIds(myProvider)
                if (ids != null && ids.isNotEmpty()) {
                    for (id in ids) {
                        updateAppWidget(context, appWidgetManager, id)
                    }
                }
            } catch (_: Exception) {
            }
        }

        fun updateAppWidget(
            context: Context,
            appWidgetManager: AppWidgetManager,
            appWidgetId: Int
        ) {
            val views = RemoteViews(context.packageName, R.layout.widget_slim_bar)

            // Setup pending intents for each of the 4 actions + home title
            views.setOnClickPendingIntent(
                R.id.widget_title,
                createActionPendingIntent(context, ACTION_HOME, 100)
            )
```

```
            views.setOnClickPendingIntent(
                R.id.widget_action_cash_in,
                createActionPendingIntent(context, ACTION_CASH_IN, 101)
            )
            views.setOnClickPendingIntent(
                R.id.widget_action_cash_out,
                createActionPendingIntent(context, ACTION_CASH_OUT, 102)
            )
            views.setOnClickPendingIntent(
                R.id.widget_action_new_bill,
                createActionPendingIntent(context, ACTION_NEW_BILL, 103)
            )
            views.setOnClickPendingIntent(
                R.id.widget_action_add_stock,
                createActionPendingIntent(context, ACTION_ADD_STOCK, 104)
            )

            appWidgetManager.updateAppWidget(appWidgetId, views)
        }

        private fun createActionPendingIntent(
            context: Context,
            action: String,
            requestCode: Int
        ): PendingIntent {
            val intent = Intent(context, MainActivity::class.java).apply {
                this.action = ACTION_WIDGET_CLICK
                putExtra(EXTRA_WIDGET_ACTION, action)
                // Set unique data URI so OS doesn't collapse distinct shortcut pending intents
                data = Uri.parse("aliedu://widget/${action.lowercase()}")
                flags = Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_SINGLE_TOP or
    Intent.FLAG_ACTIVITY_CLEAR_TOP
            }

            return PendingIntent.getActivity(
                context,
                requestCode,
                intent,
                PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE
            )
        }
    }
}
```

app\src\main\res\drawable\bg_widget_action_item.xml

```
<?xml version="1.0" encoding="utf-8"?>
<selector xmlns:android="http://schemas.android.com/apk/res/android">
    <item android:state_pressed="true">
        <shape android:shape="rectangle">
            <corners android:radius="10dp" />
            <solid android:color="#33D9AC46" />
        </shape>
    </item>
    <item>
        <shape android:shape="rectangle">
            <corners android:radius="10dp" />
            <solid android:color="@android:color/transparent" />
        </shape>
    </item>
</selector>
```

app\src\main\res\drawable\bg_widget_slim_bar.xml

```
<?xml version="1.0" encoding="utf-8"?>
<shape xmlns:android="http://schemas.android.com/apk/res/android"
    android:shape="rectangle">
    <solid android:color="#1A201C" />
    <corners android:radius="20dp" />
    <stroke android:width="1dp" android:color="#2E3A33" />
</shape>
```

app\src\main\res\drawable\ic_pigmy_coin.xml

```
<vector xmlns:android="http://schemas.android.com/apk/res/android"
    android:width="24dp"
    android:height="24dp"
    android:viewportWidth="24"
    android:viewportHeight="24">
    <!-- Coin Outer Rim Circle -->
    <path
        android:fillColor="#FFD97706"
        android:pathData="M12,2C6.48,2 2,6.48 2,12s4.48,10 10,10 10,-4.48 10,-10S17.52,2 12,2zM12,20.5c-4.69,0 -8.5,-3.81 -8.5,-8.5S7.31,3.5 12,3.5s8.5,3.81 8.5,8.5 -3.81,8.5 -8.5,8.5z"/>
    <!-- Inner Coin Background / Fill -->
    <path
        android:fillColor="#FFF59E0B"
        android:fillAlpha="0.15"
        android:pathData="M12,3.5c-4.69,0 -8.5,3.81 -8.5,8.5s3.81,8.5 8.5,8.5 8.5,-3.81 8.5,-8.5 -3.81,-8.5 -8.5,-8.5z"/>
    <!-- Inner Coin Ring -->
    <path
        android:fillColor="#FFD97706"
        android:fillAlpha="0.4"
        android:pathData="M12,5c-3.87,0 -7,3.13 -7,7s3.13,7 7,7 7,-3.13 7,-7 -3.13,-7 -7,-7zM12,18.2c-3.42,0 -6.2,-2.78 -6.2,-6.2s2.78,-6.2 6.2,-6.2 6.2,2.78 6.2,6.2 -2.78,6.2 -6.2,6.2z"/>
    <!-- Central Bitcoin B Symbol with Vertical Double Strokes -->
    <path
        android:fillColor="#FFD97706"
        android:pathData="M14.5,10.2c0.6,-0.4 0.9,-1.1 0.8,-1.8 -0.1,-1.2 -1.1,-2 -2.3,-2H11.5V5h-1.2v1.4H9V5H7.8v1.4H6.5v1.2h1.3v8.8H6.5v1.2h1.3V19H9v-1.4h1.3V19h1.2v-1.4h1.7c1.4,0 2.5,-1.1 2.5,-2.5 0,-1 -0.5,-1.8 -1.2,-2.2zM9.1,7.6h3.6c0.7,0 1.2,0.5 1.2,1.2 0,0.7 -0.5,1.2 -1.2,1.2H9.1V7.6zM13,16.4H9.1v-2.8H13c0.8,0 1.4,0.6 1.4,1.4 0,0.8 -0.6,1.4 -1.4,1.4z"/>
</vector>
```

app\src\main\res\drawable\ic_widget_add_stock.xml

```
<vector xmlns:android="http://schemas.android.com/apk/res/android"
    android:width="32dp"
    android:height="32dp"
    android:viewportWidth="24"
    android:viewportHeight="24">
    <!-- Isometric Box Top Face (Gold) -->
    <path
        android:fillColor="#FBC02D"
        android:pathData="M11,2 L18,5.5 L11,9 L4,5.5 Z" />
    <!-- Isometric Box Left Face (Gold) -->
    <path
        android:fillColor="#E5A93C"
        android:pathData="M4,6.5 L10.5,10 V18.5 L4,15 Z" />
    <!-- Isometric Box Right Face (Gold) -->
    <path
        android:fillColor="#FBC02D"
        android:pathData="M11.5,10 L18,6.5 V12 L13,14.5 V17.5 L11.5,16.5 Z" />
    <!-- Plus Circle Badge (Gold) -->
    <path
        android:fillColor="#FBC02D"
        android:pathData="M17,12.5 C19.5,12.5 21.5,14.5 21.5,17 C21.5,19.5 19.5,21.5 17,21.5 C14.5,21.5
12.5,19.5 12.5,17 C12.5,14.5 14.5,12.5 17,12.5 Z" />
    <!-- Plus Symbol in Badge (Dark) -->
    <path
        android:fillColor="#1C211D"
        android:pathData="M14.5,16.2 H19.5 V17.8 H14.5 Z M16.2,14.5 H17.8 V19.5 H16.2 Z" />
</vector>
```

app\src\main\res\drawable\ic_widget_cash_in.xml

```
<vector xmlns:android="http://schemas.android.com/apk/res/android"
    android:width="32dp"
    android:height="32dp"
    android:viewportWidth="24"
    android:viewportHeight="24">
    <!-- Upright Down Arrow (Green) -->
    <path
        android:fillColor="#4CAF50"
        android:pathData="M12,16.5 L6,10.5 H10 V3 H14 V10.5 H18 Z" />
    <!-- Horizontal Baseline Tray -->
    <path
        android:fillColor="#4CAF50"
        android:pathData="M4.5,19 H19.5 C20.3,19 21,19.7 21,20.5 C21,21.3 20.3,22 19.5,22 H4.5 C3.7,22
3,21.3 3,20.5 C3,19.7 3.7,19 4.5,19 Z" />
</vector>
```

app\src\main\res\drawable\ic_widget_cash_out.xml

```
<vector xmlns:android="http://schemas.android.com/apk/res/android"
    android:width="32dp"
    android:height="32dp"
    android:viewportWidth="24"
    android:viewportHeight="24">
    <!-- Upright Up Arrow (Orange) -->
    <path
        android:fillColor="#FF7043"
        android:pathData="M12,3.5 L18,9.5 H14 V17 H10 V9.5 H6 Z" />
    <!-- Horizontal Baseline Tray -->
    <path
        android:fillColor="#FF7043"
        android:pathData="M4.5,19 H19.5 C20.3,19 21,19.7 21,20.5 C21,21.3 20.3,22 19.5,22 H4.5 C3.7,22
3,21.3 3,20.5 C3,19.7 3.7,19 4.5,19 Z" />
</vector>
```

app\src\main\res\drawable\ic_widget_new_bill.xml

```
<vector xmlns:android="http://schemas.android.com/apk/res/android"
    android:width="32dp"
    android:height="32dp"
    android:viewportWidth="24"
    android:viewportHeight="24">
    <!-- Receipt Body (Gold) -->
    <path
        android:fillColor="#FBC02D"
        android:pathData="M5,2 C4.4,2 4,2.5 4,3.2 V19.5 L6,18 L8,19.5 L10,18 L12,19.5 V14.5 H16 C16.6,14.5 17,14.1 17,13.5 V3.2 C17,2.5 16.6,2 16,2 Z" />
    <!-- Receipt Lines Cutout (Dark) -->
    <path
        android:fillColor="#1C211D"
        android:pathData="M6.5,5.5 H14.5 V7 H6.5 Z M6.5,8.5 H12.5 V10 H6.5 Z" />
    <!-- Plus Circle Badge (Gold) -->
    <path
        android:fillColor="#FBC02D"
        android:pathData="M17,12.5 C19.5,12.5 21.5,14.5 21.5,17 C21.5,19.5 19.5,21.5 17,21.5 C14.5,21.5 12.5,19.5 12.5,17 C12.5,14.5 14.5,12.5 17,12.5 Z" />
    <!-- Plus Symbol in Badge (Dark) -->
    <path
        android:fillColor="#1C211D"
        android:pathData="M14.5,16.2 H19.5 V17.8 H14.5 Z M16.2,14.5 H17.8 V19.5 H16.2 Z" />
</vector>
```

app\src\main\res\layout\widget_slim_bar.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/widget_root"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@drawable/bg_widget_slim_bar"
    android:orientation="vertical"
    android:paddingStart="10dp"
    android:paddingEnd="10dp"
    android:paddingTop="8dp"
    android:paddingBottom="8dp">

    <!-- Prominent Light Heading matching Reference -->
    <TextView
        android:id="@+id/widget_title"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="@string/app_name"
        android:textColor="#FFFFFF"
        android:textSize="14sp"
        android:textStyle="bold"
        android:paddingStart="6dp"
        android:paddingEnd="6dp"
        android:paddingBottom="6dp"
        android:singleLine="true"
        android:clickable="true"
        android:focusable="true"
        android:contentDescription="@string/app_name" />

    <!-- Subtle Horizontal Divider Line matching Reference -->
    <ImageView
        android:layout_width="match_parent"
        android:layout_height="1dp"
        android:layout_marginStart="4dp"
        android:layout_marginEnd="4dp"
        android:layout_marginBottom="6dp"
        android:background="#2E3A33"
        android:importantForAccessibility="no" />

    <!-- 4 Equal Action Columns with Vertical Dividers -->
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        android:baselineAligned="false"
        android:gravity="center_vertical"
        android:orientation="horizontal">

        <!-- Action 1: Cash In -->
        <LinearLayout
            android:id="@+id/widget_action_cash_in"
            android:layout_width="0dp"
            android:layout_height="match_parent"
            android:layout_weight="1"
            android:background="@drawable/bg_widget_action_item"
            android:clickable="true"
            android:focusable="true"
            android:gravity="center"
            android:orientation="vertical"
            android:paddingStart="2dp"
            android:paddingEnd="2dp"
            android:contentDescription="@string/widget_action_cash_in">

            <ImageView
                android:layout_width="32dp"
                android:layout_height="32dp"
                android:src="@drawable/ic_widget_cash_in"
                android:importantForAccessibility="no" />

            <TextView
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:layout_marginTop="3dp"
                android:text="@string/widget_action_cash_in"
                android:textColor="#FFFFFF"
                android:textSize="13sp"
                android:textStyle="bold"
                android:singleLine="true"
                android:lines="1"
                android:ellipsize="none"
                android:importantForAccessibility="no" />
        </LinearLayout>

        <!-- Vertical Divider 1 -->
        <ImageView
```

```
            android:layout_width="1dp"
            android:layout_height="32dp"
            android:layout_gravity="center_vertical"
            android:background="#2E3A33"
            android:importantForAccessibility="no" />

        <!-- Action 2: Cash Out -->
        <LinearLayout
            android:id="@+id/widget_action_cash_out"
            android:layout_width="0dp"
            android:layout_height="match_parent"
            android:layout_weight="1"
            android:background="@drawable/bg_widget_action_item"
            android:clickable="true"
            android:focusable="true"
            android:gravity="center"
            android:orientation="vertical"
            android:paddingStart="2dp"
            android:paddingEnd="2dp"
            android:contentDescription="@string/widget_action_cash_out">

            <ImageView
                android:layout_width="32dp"
                android:layout_height="32dp"
                android:src="@drawable/ic_widget_cash_out"
                android:importantForAccessibility="no" />

            <TextView
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:layout_marginTop="3dp"
                android:text="@string/widget_action_cash_out"
                android:textColor="#FFFFFF"
                android:textSize="13sp"
                android:textStyle="bold"
                android:singleLine="true"
                android:lines="1"
                android:ellipsize="none"
                android:importantForAccessibility="no" />
        </LinearLayout>

        <!-- Vertical Divider 2 -->
        <ImageView
            android:layout_width="1dp"
            android:layout_height="32dp"
            android:layout_gravity="center_vertical"
            android:background="#2E3A33"
            android:importantForAccessibility="no" />

        <!-- Action 3: New Bill -->
        <LinearLayout
            android:id="@+id/widget_action_new_bill"
            android:layout_width="0dp"
            android:layout_height="match_parent"
            android:layout_weight="1"
            android:background="@drawable/bg_widget_action_item"
            android:clickable="true"
            android:focusable="true"
            android:gravity="center"
            android:orientation="vertical"
            android:paddingStart="2dp"
            android:paddingEnd="2dp"
            android:contentDescription="@string/widget_action_new_bill">

            <ImageView
                android:layout_width="32dp"
                android:layout_height="32dp"
                android:src="@drawable/ic_widget_new_bill"
                android:importantForAccessibility="no" />

            <TextView
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:layout_marginTop="3dp"
                android:text="@string/widget_action_new_bill"
                android:textColor="#FFFFFF"
                android:textSize="13sp"
                android:textStyle="bold"
                android:singleLine="true"
                android:lines="1"
                android:ellipsize="none"
                android:importantForAccessibility="no" />
        </LinearLayout>

        <!-- Vertical Divider 3 -->
        <ImageView
            android:layout_width="1dp"
            android:layout_height="32dp"
```

```
            android:layout_gravity="center_vertical"
            android:background="#2E3A33"
            android:importantForAccessibility="no" />

        <!-- Action 4: Add Stock -->
        <LinearLayout
            android:id="@+id/widget_action_add_stock"
            android:layout_width="0dp"
            android:layout_height="match_parent"
            android:layout_weight="1"
            android:background="@drawable/bg_widget_action_item"
            android:clickable="true"
            android:focusable="true"
            android:gravity="center"
            android:orientation="vertical"
            android:paddingStart="2dp"
            android:paddingEnd="2dp"
            android:contentDescription="@string/widget_action_add_stock">

            <ImageView
                android:layout_width="32dp"
                android:layout_height="32dp"
                android:src="@drawable/ic_widget_add_stock"
                android:importantForAccessibility="no" />

            <TextView
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:layout_marginTop="3dp"
                android:text="@string/widget_action_add_stock"
                android:textColor="#FFFFFF"
                android:textSize="13sp"
                android:textStyle="bold"
                android:singleLine="true"
                android:lines="1"
                android:ellipsize="none"
                android:importantForAccessibility="no" />
        </LinearLayout>

    </LinearLayout>

</LinearLayout>
```

app\src\main\res\mipmap-anydpi-v26\ic_launcher.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<adaptive-icon xmlns:android="http://schemas.android.com/apk/res/android">
    <background android:drawable="@color/ic_launcher_background" />
    <foreground android:drawable="@drawable/ic_launcher_foreground" />
    <monochrome android:drawable="@drawable/ic_launcher_foreground" />
</adaptive-icon>
```

app\src\main\res\mipmap-anydpi-v26\ic_launcher_round.xml

```
<?xml version="1.0" encoding="utf-8"?>
<adaptive-icon xmlns:android="http://schemas.android.com/apk/res/android">
    <background android:drawable="@color/ic_launcher_background" />
    <foreground android:drawable="@drawable/ic_launcher_foreground" />
    <monochrome android:drawable="@drawable/ic_launcher_foreground" />
</adaptive-icon>
```

app\src\main\res\values\colors.xml

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <resources>
3      <color name="ic_launcher_background">#111411</color>
4  </resources>
```

app\src\main\res\values\strings.xml

```
<resources>
    <string name="app_name">Ali Educational</string>
    <string name="nav_home">Home</string>
    <string name="nav_stock">Stock</string>
    <string name="nav_billing">Billing</string>
    <string name="nav_accounts">Accounts</string>
    <string name="nav_more">More</string>

    <string name="action_new_bill">New bill</string>
    <string name="action_add_stock">Add stock</string>
    <string name="action_remove_stock">Remove stock</string>
    <string name="action_record_payment">Record payment</string>
    <string name="action_record_sale">Record sale</string>

    <string name="common_cancel">Cancel</string>

    <string name="unlock_title">Ali Educational</string>
    <string name="unlock_subtitle">Stationery Shop Management</string>
    <string name="unlock_prompt">Confirm your fingerprint or device credential</string>
    <string name="unlock_button">Unlock App</string>
    <string name="unlock_failed">Authentication failed. Please try again.</string>
    <string name="unlock_no_credential">No screen lock set on device. Please set a PIN or fingerprint in device settings.</string>

    <string name="setup_welcome">Welcome to Ali Educational</string>
    <string name="setup_title">Welcome to Ali Educational</string>
    <string name="setup_continue">Continue</string>
    <string name="setup_subtitle">Fast, offline stock &amp;amp; accounts management</string>
    <string name="setup_shop_name">Shop Name</string>
    <string name="setup_default_shop_name">Ali Educational</string>
    <string name="setup_select_language">Select Language</string>
    <string name="setup_lang_english">English</string>
    <string name="setup_lang_urdu">     (Urdu)</string>
    <string name="setup_enable_lock">Enable App Lock (Fingerprint / PIN)</string>
    <string name="setup_enable_lock_desc">Requires device fingerprint or PIN to open the app</string>
    <string name="setup_get_started">Get Started</string>

    <string name="stock_title">Stock</string>
    <string name="accounts_title">Accounts</string>
    <string name="more_title">More</string>
    <string name="home_stock_value">Stock Value</string>
    <string name="home_low_stock">Low Stock Items</string>
    <string name="home_we_owe">We Owe Suppliers</string>
    <string name="home_customers_owe">Customers Owe Us</string>
    <string name="home_recent_activity">Recent Activity</string>
    <string name="home_backup_status">Last backup: Never</string>
    <string name="home_backup_now">Back up now</string>
    <string name="home_stationery_subtitle">Stationery &amp;amp; Education</string>
    <string name="home_record_sale_desc">Quick item sale with discounts &amp;amp; receipts</string>
    <string name="home_no_activity">No stock activity recorded yet. Add your first stock to get started.</string>
    <string name="home_stored_safely">Stored safely on this phone</string>
    <string name="home_last_backup_prefix">Last backup: %1$s</string>
    <string name="home_items_count">items %1$d</string>
    <string name="home_reports_and_tools">Reports &amp;amp; Tools</string>
    <string name="home_profit_analysis">Gross Profit &amp;amp; Margins</string>
    <string name="home_backup_restore">Backup &amp;amp; Restore</string>
    <string name="home_settings">Settings</string>
    <string name="home_expenses">Shop Expenses</string>

    <!-- Billing Strings -->
    <string name="billing_title">Billing</string>
    <string name="billing_header_subtitle">Counter Sale</string>
    <string name="billing_offline_badge">Offline</string>
    <string name="billing_offline_ready">100% Offline</string>
    <string name="billing_new_bill_title">New Bill</string>
    <string name="billing_new_bill_desc">Scan or select items to create a counter sale</string>
    <string name="billing_scan_barcode">Scan barcode</string>
    <string name="billing_scan_with_camera">Scan with camera</string>
    <string name="billing_search_placeholder">Search product name, brand, SKU...</string>
    <string name="billing_filter_all">All</string>
    <string name="billing_filter_favourites">Favourites</string>
    <string name="billing_filter_category">Category</string>
    <string name="billing_in_this_bill">In this bill</string>
    <string name="billing_in_this_bill_title">In this bill</string>
    <string name="billing_items_summary">%1$d items · %2$s</string>
    <string name="billing_review_pill">Review %1$d items · %2$s →</string>
    <string name="billing_review_action">Review %1$d items</string>
    <string name="billing_add_btn">Add</string>
    <string name="billing_add_again">Add again</string>
    <string name="billing_undo">Undo</string>
    <string name="billing_piece">Piece</string>
    <string name="billing_pack">Pack</string>
    <string name="billing_held_bills">Held bills</string>
    <string name="billing_held_tag">held</string>
```

```
    <string name="billing_hold_bill">Hold bill</string>
    <string name="billing_resume">Resume</string>
    <string name="billing_discard">Discard</string>
    <string name="billing_no_items">No items in draft</string>
    <string name="billing_no_items_desc">Scan a barcode, tap favourites, or search above to add items</string>
    <string name="billing_stock_tag">Stock: %1$d</string>
    <string name="billing_unknown_barcode_title">Unknown Barcode</string>
    <string name="billing_unknown_barcode_desc">Barcode %1$s is not linked to any product.</string>
    <string name="billing_link_existing">Link to Existing Product</string>
    <string name="billing_create_new">Create New Product</string>

    <!-- Scanner Strings -->
    <string name="scanner_title">Scan Barcode</string>
    <string name="scanner_guidance">Align product barcode within frame</string>
    <string name="scanner_permission_needed">Camera permission is required to scan manufacturer barcodes.</string>
    <string name="scanner_grant_permission">Grant Camera Permission</string>
    <string name="scanner_torch">Flashlight</string>
    <string name="scanner_added_label">Added</string>
    <string name="scanner_undo">Undo</string>
    <string name="scanner_debounce_hint">Hold barcode away to scan next item</string>
    <string name="scanner_add_again">Add again</string>
    <string name="scanner_view_bill">View bill</string>
    <string name="scanner_no_barcode_fallback">No barcode? Search or browse items</string>
    <string name="scanner_unknown_title">Unknown Barcode</string>
    <string name="scanner_scanned_code">Scanned Code</string>
    <string name="scanner_unknown_prompt">This barcode is not linked to any product. Would you like to link it now?</string>
    <string name="scanner_select_product_to_link">Select product to link</string>
    <string name="scanner_link_btn">Link Barcode</string>

    <!-- Review Screen Strings -->
    <string name="review_title">Review Bill</string>
    <string name="review_customer">Customer</string>
    <string name="review_walk_in">Walk-in Customer (Cash)</string>
    <string name="review_select_customer">Select / Change Customer</string>
    <string name="billing_existing_customer">Existing Customer</string>
    <string name="billing_new_customer">+ New Customer</string>
    <string name="billing_search_customer">Search name or phone...</string>
    <string name="billing_customer_name">Customer Name *</string>
    <string name="billing_customer_phone">Phone Number (Optional)</string>
    <string name="billing_customer_notes">Address / Notes (Optional)</string>
    <string name="billing_save_select_customer">Save &amp;amp; Select Customer</string>
    <string name="billing_customer_khata_due">Owes: %1$s</string>
    <string name="billing_customer_khata_clear">Clear (₹0)</string>
    <string name="review_items">Bill Items</string>
    <string name="review_add_item">+ Scan or add another item</string>
    <string name="review_subtotal">Subtotal</string>
    <string name="review_discount">Discount</string>
    <string name="review_add_discount">Add discount →</string>
    <string name="review_remove_discount">Remove</string>
    <string name="review_net_total">Net Bill Total</string>
    <string name="review_notice">Stock reduces when the bill is saved. Items in this draft have not been deducted.</string>
    <string name="review_continue_payment">Continue to payment →</string>
    <string name="review_discount_title">Whole-Bill Discount</string>
    <string name="review_discount_percent">Percentage (%)</string>
    <string name="review_discount_fixed">Fixed Amount (₹)</string>
    <string name="review_apply">Apply</string>
    <string name="review_cancel">Cancel</string>

    <!-- Payment Screen Strings -->
    <string name="payment_title">Payment &amp;amp; Change</string>
    <string name="payment_summary">Bill Summary</string>
    <string name="payment_method">Payment Method</string>
    <string name="payment_cash">Cash</string>
    <string name="payment_upi">UPI</string>
    <string name="payment_credit">Customer Credit / Khata</string>
    <string name="payment_other">Other</string>
    <string name="payment_cash_received">Cash Received (₹)</string>
    <string name="payment_exact">Exact</string>
    <string name="payment_change_to_give">Change to give: %1$s</string>
    <string name="payment_short_amount">Short by %1$s</string>
    <string name="payment_treat_as_discount">Treat difference as discount</string>
    <string name="payment_treat_as_credit">Add difference to customer khata</string>
    <string name="payment_credit_req_customer">Customer account required for credit payment</string>
    <string name="payment_receipt_action">Receipt after saving</string>
    <string name="payment_action_none">No action (quick save)</string>
    <string name="payment_action_preview">Preview receipt</string>
    <string name="payment_action_print">Print receipt</string>
    <string name="payment_action_share">Share on WhatsApp / PDF</string>
    <string name="payment_save_bill">Save bill · %1$s</string>

    <!-- Receipt Screen Strings -->
    <string name="receipt_saved_title">Bill Saved Successfully!</string>
    <string name="receipt_reference">Receipt #</string>
    <string name="receipt_date">Date &amp;amp; Time</string>
```

```
    <string name="receipt_items_total">Total Items</string>
    <string name="receipt_gross_total">Gross Total</string>
    <string name="receipt_discount_given">Discount Given</string>
    <string name="receipt_payment_mode">Payment Mode</string>
    <string name="receipt_items_purchased">Purchased Items</string>
    <string name="receipt_net_paid">Net Total Paid</string>
    <string name="receipt_cash_tendered">Cash Tendered</string>
    <string name="receipt_change_given">Change Given</string>
    <string name="receipt_new_bill">New Bill</string>
    <string name="receipt_done">Done / Return to Home</string>
    <string name="receipt_share_button">Share Receipt</string>
    <string name="receipt_print_button">Print Receipt</string>

    <!-- Held Bills Strings -->
    <string name="held_bills_title">Held Bills (Parked Drafts)</string>
    <string name="held_bills_empty">No held bills at the moment</string>
    <string name="held_bills_resume">Resume Draft</string>
    <string name="held_bills_discard">Discard</string>
    <string name="held_bill_prompt_note">Hold Bill Note (Optional)</string>
    <string name="held_bill_confirm_hold">Hold Current Bill</string>

    <!-- Sale Void & Return Strings -->
    <string name="receipt_void_button">Cancel / Void Bill</string>
    <string name="receipt_void_title">Cancel Bill &amp;amp; Return Items</string>
    <string name="receipt_void_warning">This will cancel this bill, refund the customer, and update inventory.</string>
    <string name="receipt_void_refund_payout">Refund Payout to Customer</string>
    <string name="receipt_void_stock_question">Returned Stock Condition:</string>
    <string name="receipt_void_condition_sellable">Sellable (Restore to stock)</string>
    <string name="receipt_void_condition_damaged">Damaged (Move to damaged)</string>
    <string name="receipt_void_reason_label">Reason for cancellation (optional)</string>
    <string name="receipt_void_confirm">Confirm Cancellation &amp;amp; Refund</string>
    <string name="receipt_voided_badge">VOIDED &amp;amp; REFUNDED</string>

    <!-- Account Payment & Receipt Voucher Strings -->
    <string name="voucher_title">Payment Receipt Voucher</string>
    <string name="voucher_recorded_success">Payment Recorded Successfully!</string>
    <string name="voucher_amount_paid">Amount Paid</string>
    <string name="voucher_payment_mode">Payment Mode</string>
    <string name="voucher_remaining_balance">Remaining Outstanding Balance</string>
    <string name="voucher_share_button">Share Payment Voucher</string>
    <string name="voucher_print_button">Print Voucher</string>
    <string name="voucher_done_button">Done</string>

    <!-- Widget Strings -->
    <string name="widget_slim_bar_name">Ali Educational Quick Actions</string>
    <string name="widget_slim_bar_description">Quick actions for Cash In, Cash Out, New Bill, and Add Stock</string>
    <string name="widget_action_cash_in">Cash In</string>
    <string name="widget_action_cash_out">Cash Out</string>
    <string name="widget_action_new_bill">New Bill</string>
    <string name="widget_action_add_stock">Add Stock</string>
</resources>
```

app\src\main\res\values\themes.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <style name="Theme.AliEducational" parent="android:Theme.Material.NoActionBar">
        <item name="android:statusBarColor">@android:color/transparent</item>
        <item name="android:navigationBarColor">@android:color/transparent</item>
    </style>
</resources>
```

app\src\main\res\values-ur\strings.xml

```
<resources>
    <string name="app_name">             </string>
    <string name="nav_home">   </string>
    <string name="nav_stock">     </string>
    <string name="nav_billing">    </string>
    <string name="nav_accounts">     </string>
    <string name="nav_more">    </string>

    <string name="action_new_bill">      </string>
    <string name="action_add_stock">              </string>
    <string name="action_remove_stock">            </string>
    <string name="action_record_payment">                 </string>
    <string name="action_record_sale">              </string>

    <string name="common_cancel">     </string>

    <string name="unlock_title">             </string>
    <string name="unlock_subtitle">                    </string>
    <string name="unlock_prompt">                                      </string>
    <string name="unlock_button">          </string>
    <string name="unlock_failed">                                     </string>
    <string name="unlock_no_credential">
          </string>

    <string name="setup_welcome">                           </string>
    <string name="setup_title">                           </string>
    <string name="setup_continue">         </string>
    <string name="setup_subtitle">                                      </string>
    <string name="setup_shop_name">           </string>
    <string name="setup_default_shop_name">             </string>
    <string name="setup_select_language">               </string>
    <string name="setup_lang_english">English</string>
    <string name="setup_lang_urdu">    </string>
    <string name="setup_enable_lock">                (          /   )</string>
    <string name="setup_enable_lock_desc">                                          </string>
    <string name="setup_get_started">         </string>

    <string name="stock_title">     </string>
    <string name="accounts_title">     </string>
    <string name="more_title">    </string>
    <string name="home_stock_value">                 </string>
    <string name="home_low_stock">                   </string>
    <string name="home_we_owe">                           </string>
    <string name="home_customers_owe">                          </string>
    <string name="home_recent_activity">              </string>
    <string name="home_backup_status">           :          </string>
    <string name="home_backup_now">               </string>
    <string name="home_stationery_subtitle">                </string>
    <string name="home_record_sale_desc">                                              </string>
    <string name="home_no_activity">
     </string>
    <string name="home_stored_safely">                            </string>
    <string name="home_last_backup_prefix">           : %1$s</string>
    <string name="home_items_count">%1$d      </string>
    <string name="home_reports_and_tools">                </string>
    <string name="home_profit_analysis">                      </string>
    <string name="home_backup_restore">                </string>
    <string name="home_settings">       </string>
    <string name="home_expenses">                </string>

    <!-- Billing Strings -->
    <string name="billing_title">    </string>
    <string name="billing_header_subtitle">          </string>
    <string name="billing_offline_badge">       </string>
    <string name="billing_offline_ready">100%        </string>
    <string name="billing_new_bill_title">      </string>
    <string name="billing_new_bill_desc">                                                                  </string>
    <string name="billing_scan_barcode">                 </string>
    <string name="billing_scan_with_camera">                    </string>
    <string name="billing_search_placeholder">                                        ...</string>
    <string name="billing_filter_all">    </string>
    <string name="billing_filter_favourites">       </string>
    <string name="billing_filter_category">       </string>
    <string name="billing_in_this_bill">         </string>
    <string name="billing_in_this_bill_title">         </string>
    <string name="billing_items_summary">%1$d       · %2$s</string>
    <string name="billing_review_pill">          %1$d       · %2$s ←</string>
    <string name="billing_review_action">           (%1$d      )</string>
    <string name="billing_add_btn">          </string>
    <string name="billing_add_again">                 </string>
    <string name="billing_undo">           </string>
    <string name="billing_piece">   </string>
    <string name="billing_pack">    /    </string>
    <string name="billing_held_bills">            </string>
    <string name="billing_held_tag">     </string>
```

```
    <string name="billing_hold_bill">           </string>
    <string name="billing_resume">         </string>
    <string name="billing_discard">        </string>
    <string name="billing_no_items">                            </string>
    <string name="billing_no_items_desc">
    </string>
    <string name="billing_stock_tag">     : %1$d</string>
    <string name="billing_unknown_barcode_title">             </string>
    <string name="billing_unknown_barcode_desc">       %1$s                              </string>
    <string name="billing_link_existing">                     </string>
    <string name="billing_create_new">                </string>

    <!-- Scanner Strings -->
    <string name="scanner_title">                 </string>
    <string name="scanner_guidance">                                    </string>
    <string name="scanner_permission_needed">
   </string>
    <string name="scanner_grant_permission">                 </string>
    <string name="scanner_torch">    </string>
    <string name="scanner_added_label">            </string>
    <string name="scanner_undo">        </string>
    <string name="scanner_debounce_hint">                                                                  </string>
    <string name="scanner_add_again">               </string>
    <string name="scanner_view_bill">         </string>
    <string name="scanner_no_barcode_fallback">                                        </string>
    <string name="scanner_unknown_title">              </string>
    <string name="scanner_scanned_code">             </string>
    <string name="scanner_unknown_prompt">
     </string>
    <string name="scanner_select_product_to_link">                 </string>
    <string name="scanner_link_btn">            </string>

    <!-- Review Screen Strings -->
    <string name="review_title">           </string>
    <string name="review_customer">    </string>
    <string name="review_walk_in">         (   )</string>
    <string name="review_select_customer">                       </string>
    <string name="billing_existing_customer">           </string>
    <string name="billing_new_customer">+         </string>
    <string name="billing_search_customer">                      ...</string>
    <string name="billing_customer_name">            *</string>
    <string name="billing_customer_phone">         (       )</string>
    <string name="billing_customer_notes">    /      (       )</string>
    <string name="billing_save_select_customer">                          </string>
    <string name="billing_customer_khata_due">          : %1$s</string>
    <string name="billing_customer_khata_clear">       (₹0)</string>
    <string name="review_items">           </string>
    <string name="review_add_item">+                             </string>
    <string name="review_subtotal">      </string>
    <string name="review_discount">     </string>
    <string name="review_add_discount">               ←</string>
    <string name="review_remove_discount">      </string>
    <string name="review_net_total">                     </string>
    <string name="review_notice">
                </string>
    <string name="review_continue_payment">                    ←</string>
    <string name="review_discount_title">               </string>
    <string name="review_discount_percent">     (%)</string>
    <string name="review_discount_fixed">          (₹)</string>
    <string name="review_apply">         </string>
    <string name="review_cancel">     </string>

    <!-- Payment Screen Strings -->
    <string name="payment_title">                    </string>
    <string name="payment_summary">           </string>
    <string name="payment_method">             </string>
    <string name="payment_cash">   </string>
    <string name="payment_upi">         </string>
    <string name="payment_credit">              /      </string>
    <string name="payment_other">    </string>
    <string name="payment_cash_received">                (₹)</string>
    <string name="payment_exact">        </string>
    <string name="payment_change_to_give">                       : %1$s</string>
    <string name="payment_short_amount">      : %1$s</string>
    <string name="payment_treat_as_discount">                      </string>
    <string name="payment_treat_as_credit">                                  </string>
    <string name="payment_credit_req_customer">
  </string>
    <string name="payment_receipt_action">                     </string>
    <string name="payment_action_none">             (          )</string>
    <string name="payment_action_preview">           </string>
    <string name="payment_action_print">             </string>
    <string name="payment_action_share">                              </string>
    <string name="payment_save_bill">              · %1$s</string>

    <!-- Receipt Screen Strings -->
    <string name="receipt_saved_title">                          !</string>
    <string name="receipt_reference">         </string>
```

```
    <string name="receipt_date">           </string>
    <string name="receipt_items_total">        </string>
    <string name="receipt_gross_total">           </string>
    <string name="receipt_discount_given">            </string>
    <string name="receipt_payment_mode">             </string>
    <string name="receipt_items_purchased">                 </string>
    <string name="receipt_net_paid">            </string>
    <string name="receipt_cash_tendered">               </string>
    <string name="receipt_change_given">                  </string>
    <string name="receipt_new_bill">      </string>
    <string name="receipt_done">     /             </string>
    <string name="receipt_share_button">              </string>
    <string name="receipt_print_button">              </string>

    <!-- Held Bills Strings -->
    <string name="held_bills_title">                </string>
    <string name="held_bills_empty">                                  </string>
    <string name="held_bills_resume">            </string>
    <string name="held_bills_discard">        </string>
    <string name="held_bill_prompt_note">             (       )</string>
    <string name="held_bill_confirm_hold">                    </string>

    <!-- Sale Void & Return Strings -->
    <string name="receipt_void_button">              /      </string>
    <string name="receipt_void_title">                                 </string>
    <string name="receipt_void_warning">
      </string>
    <string name="receipt_void_refund_payout">                 </string>
    <string name="receipt_void_stock_question">                           :</string>
    <string name="receipt_void_condition_sellable">           (                        )</string>
    <string name="receipt_void_condition_damaged">     /              (                       )</string>
    <string name="receipt_void_reason_label">              (       )</string>
    <string name="receipt_void_confirm">                                  </string>
    <string name="receipt_voided_badge">                      </string>

    <!-- Account Payment & Receipt Voucher Strings -->
    <string name="voucher_title">                   (     )</string>
    <string name="voucher_recorded_success">                              !</string>
    <string name="voucher_amount_paid">           </string>
    <string name="voucher_payment_mode">             </string>
    <string name="voucher_remaining_balance">                    </string>
    <string name="voucher_share_button">               </string>
    <string name="voucher_print_button">               </string>
    <string name="voucher_done_button">    </string>
</resources>
```

app\src\main\res\xml\ali_slim_widget_info.xml

```
<?xml version="1.0" encoding="utf-8"?>
<appwidget-provider xmlns:android="http://schemas.android.com/apk/res/android"
    android:minWidth="250dp"
    android:minHeight="70dp"
    android:minResizeWidth="220dp"
    android:minResizeHeight="56dp"
    android:targetCellWidth="4"
    android:targetCellHeight="1"
    android:updatePeriodMillis="0"
    android:initialLayout="@layout/widget_slim_bar"
    android:previewLayout="@layout/widget_slim_bar"
    android:resizeMode="horizontal|vertical"
    android:widgetCategory="home_screen"
    android:description="@string/widget_slim_bar_description" />
```

app\src\main\res\xml\data_extraction_rules.xml

```
<?xml version="1.0" encoding="utf-8"?>
<data-extraction-rules>
    <cloud-backup>
        <exclude domain="root" />
        <exclude domain="file" />
        <exclude domain="database" />
        <exclude domain="sharedpref" />
    </cloud-backup>
    <device-transfer>
        <include domain="root" />
        <include domain="file" />
        <include domain="database" />
        <include domain="sharedpref" />
    </device-transfer>
</data-extraction-rules>
```

app\src\main\res\xml\file_paths.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<paths xmlns:android="http://schemas.android.com/apk/res/android">
    <external-path name="external_storage" path="." />
    <external-files-path name="external_files" path="." />
    <files-path name="internal_files" path="." />
    <cache-path name="cache_files" path="." />
</paths>
```

app\src\test\java\com\alieducational\app\data\MigrationSqlTest.kt

```kotlin
package com.alieducational.app.data

import androidx.sqlite.db.SupportSQLiteDatabase
import com.alieducational.app.data.local.AppDatabase
import org.junit.Assert.assertTrue
import org.junit.Test
import java.lang.reflect.InvocationHandler
import java.lang.reflect.Method
import java.lang.reflect.Proxy

class MigrationSqlTest {

    @Test
    fun testMigration4To5StatementsRecorded() {
        val statements = mutableListOf<String>()

        val handler = InvocationHandler { _, method: Method, args: Array<out Any>? ->
            if (method.name == "execSQL" && args != null && args.isNotEmpty()) {
                statements.add(args[0] as String)
            }
            null
        }

        val proxyDb = Proxy.newProxyInstance(
            SupportSQLiteDatabase::class.java.classLoader,
            arrayOf(SupportSQLiteDatabase::class.java),
            handler
        ) as SupportSQLiteDatabase

        AppDatabase.MIGRATION_4_5.migrate(proxyDb)

        // Verify alter tables
        assertTrue(statements.any { it.contains("ALTER TABLE sales ADD COLUMN originDeviceId") })
        assertTrue(statements.any { it.contains("ALTER TABLE sales ADD COLUMN syncStatus") })
        assertTrue(statements.any { it.contains("ALTER TABLE sale_lines ADD COLUMN lineType") })
        assertTrue(statements.any { it.contains("ALTER TABLE sale_lines ADD COLUMN serviceId") })
        assertTrue(statements.any { it.contains("ALTER TABLE sale_lines ADD COLUMN serviceName") })
        assertTrue(statements.any { it.contains("ALTER TABLE sale_lines ADD COLUMN serviceOption") })
        assertTrue(statements.any { it.contains("ALTER TABLE sale_lines ADD COLUMN unitLabel") })
        assertTrue(statements.any { it.contains("ALTER TABLE shop_expenses ADD COLUMN expenseCategory") })
        assertTrue(statements.any { it.contains("ALTER TABLE shop_expenses ADD COLUMN serviceTarget") })
        assertTrue(statements.any { it.contains("ALTER TABLE shop_expenses ADD COLUMN attachmentUri") })
        assertTrue(statements.any { it.contains("ALTER TABLE shop_expenses ADD COLUMN isSavingsTransfer") })

        // Verify new tables
        assertTrue(statements.any { it.contains("CREATE TABLE IF NOT EXISTS `pigmy_deposits`") })
        assertTrue(statements.any { it.contains("CREATE TABLE IF NOT EXISTS `product_giveaways`") })
        assertTrue(statements.any { it.contains("CREATE TABLE IF NOT EXISTS `service_definitions`") })
        assertTrue(statements.any { it.contains("CREATE TABLE IF NOT EXISTS `sync_events`") })
    }
}
```

app\src\test\java\com\alieducational\app\data\service\BackupPackageAndReconciliationTest.kt

```
package com.alieducational.app.data.service

import com.alieducational.app.data.local.entity.AccountPaymentEntity
import com.alieducational.app.data.local.entity.CashCategoryEntity
import com.alieducational.app.data.local.entity.CustomerEntity
import com.alieducational.app.data.local.entity.PigmyDepositEntity
import com.alieducational.app.data.local.entity.PigmySchemeEntity
import com.alieducational.app.data.local.entity.PigmyWithdrawalEntity
import com.alieducational.app.data.local.entity.ProductEntity
import com.alieducational.app.data.local.entity.PurchaseEntity
import com.alieducational.app.data.local.entity.PurchaseLineEntity
import com.alieducational.app.data.local.entity.SaleEntity
import com.alieducational.app.data.local.entity.SaleLineEntity
import com.alieducational.app.data.local.entity.ServiceDefinitionEntity
import com.alieducational.app.data.local.entity.ShopExpenseEntity
import com.alieducational.app.data.local.entity.StockMovementEntity
import com.alieducational.app.data.local.entity.SupplierEntity
import org.apache.poi.xssf.usermodel.XSSFWorkbook
import org.junit.Assert.assertEquals
import org.junit.Assert.assertNotNull
import org.junit.Assert.assertTrue
import org.junit.Rule
import org.junit.Test
import org.junit.rules.TemporaryFolder
import java.io.File
import java.io.FileInputStream
import java.io.FileOutputStream
import java.util.zip.ZipEntry
import java.util.zip.ZipInputStream
import java.util.zip.ZipOutputStream

class BackupPackageAndReconciliationTest {

    @get:Rule
    val tempFolder = TemporaryFolder()

    @Test
    fun testExcelGeneratorGeneratesAll11SheetsWithNumericData() {
        val generator = ShopRecordsExcelGenerator()
        val outFile = tempFolder.newFile("test_shop_records.xlsx")

        val snapshotData = ShopSnapshotData(
            snapshotId = "SNAP_2026-11-01_20-24-23",
            timestamp = 1761999863000L,
            shopName = "Ali Educational",
            products = listOf(
                ProductEntity(
                    id = "PROD-001",
                    name = "Classmate Notebook 200p /      ",
                    category = "Notebooks",
                    brand = "Classmate",
                    sku = "CN-200",
                    barcode = "8901234567890",
                    baseUnit = "piece",
                    packSize = 10,
                    currentSellableStock = 50,
                    movingAvgCostPaise = 3500L, // ₹35.00
                    sellingPricePiecePaise = 5000L, // ₹50.00
                    sellingPricePackPaise = 48000L, // ₹480.00
                    lowStockThreshold = 10
                )
            ),
            movements = listOf(
                StockMovementEntity(
                    id = "MOV-001",
                    productId = "PROD-001",
                    type = "RECEIVE",
                    quantityChangeBase = 50,
                    unitCostPaise = 3500L,
                    reason = "Initial Stock"
                )
            ),
            purchases = emptyList(),
            purchaseLines = emptyList(),
            sales = listOf(
                SaleEntity(
                    id = "SALE-001",
                    receiptNumber = "AE-M-20261101-0001",
                    customerId = "CUST-001",
                    customerName = "Ahmed Walk-in",
                    paymentMethod = "CASH",
                    grossTotalPaise = 10000L,
```

```
                    discountValuePaise = 1000L,
                    netTotalPaise = 9000L, // ₹90.00
                    amountReceivedPaise = 9000L,
                    remainingCreditPaise = 0L
                )
            ),
            saleLines = listOf(
                SaleLineEntity(
                    id = "SL-001",
                    saleId = "SALE-001",
                    productId = "PROD-001",
                    productName = "Classmate Notebook 200p /      ",
                    lineType = "PRODUCT",
                    quantityBase = 2,
                    unitSellingPricePaise = 5000L,
                    unitCostSnapshotPaise = 3500L,
                    lineGrossPaise = 10000L,
                    lineNetPaise = 9000L,
                    lineProfitPaise = 2000L
                )
            ),
            accountPayments = listOf(
                AccountPaymentEntity(
                    id = "PAY-001",
                    accountType = "CUSTOMER",
                    accountId = "CUST-001",
                    amountPaise = 5000L,
                    paymentMethod = "UPI",
                    notes = "Advance cleared"
                )
            ),
            customers = listOf(
                CustomerEntity(
                    id = "CUST-001",
                    name = "Ahmed Walk-in",
                    phone = "9876543210",
                    currentBalancePaise = 0L
                )
            ),
            suppliers = listOf(
                SupplierEntity(
                    id = "SUPP-001",
                    name = "Star Paper Distributors",
                    phone = "9123456780",
                    currentBalancePaise = 150000L // ₹1,500.00
                )
            ),
            cashExpenses = listOf(
                ShopExpenseEntity(
                    id = "EXP-001",
                    isIncome = false,
                    amountPaise = 20000L,
                    category = "Tea & Refreshment",
                    notes = "Evening staff tea",
                    isSavingsTransfer = false
                ),
                ShopExpenseEntity(
                    id = "INC-001",
                    isIncome = true,
                    amountPaise = 50000L,
                    category = "Other Income",
                    notes = "Paper scrap sold",
                    isSavingsTransfer = false
                )
            ),
            pigmySchemes = listOf(
                PigmySchemeEntity(
                    id = "SCHEME-001",
                    name = "Daily 100 Savings",
                    usualDailyAmountPaise = 10000L
                )
            ),
            pigmyDeposits = listOf(
                PigmyDepositEntity(
                    id = "DEP-001",
                    accountSchemeId = "SCHEME-001",
                    amountPaise = 10000L,
                    paymentSource = "CASH",
                    notes = "Deposit by Musid"
                )
            ),
            pigmyWithdrawals = emptyList(),
            services = listOf(
                ServiceDefinitionEntity(
                    id = "SVC-001",
                    name = "Photocopying B&W",
                    unitLabel = "copies"
                )
```

```
            ),
            cashCategories = listOf(
                CashCategoryEntity(id = "CAT-001", name = "Tea & Refreshment"),
                CashCategoryEntity(id = "CAT-002", name = "Other Income")
            ),
            photoCount = 2
        )

        // Generate the workbook
        generator.generateWorkbook(snapshotData, outFile)
        assertTrue("Workbook file must exist", outFile.exists())
        assertTrue("Workbook size must be > 1000 bytes", outFile.length() > 1000)

        // Inspect the generated workbook with Apache POI
        FileInputStream(outFile).use { fis ->
            val wb = XSSFWorkbook(fis)
            assertEquals("Must have exactly 11 sheets", 11, wb.numberOfSheets)

            val sheetNames = (0 until wb.numberOfSheets).map { wb.getSheetName(it) }
            assertTrue(sheetNames.contains("Summary"))
            assertTrue(sheetNames.contains("Products and Stock"))
            assertTrue(sheetNames.contains("Stock Movements"))
            assertTrue(sheetNames.contains("Bills and Items"))
            assertTrue(sheetNames.contains("Payments and Dues"))
            assertTrue(sheetNames.contains("Customers and Suppliers"))
            assertTrue(sheetNames.contains("Cash In Out"))
            assertTrue(sheetNames.contains("Expenses and Refunds"))
            assertTrue(sheetNames.contains("Services"))
            assertTrue(sheetNames.contains("Pigmy Savings"))
            assertTrue(sheetNames.contains("Categories"))

            // Verify Products and Stock sheet has our notebook with exact Urdu name preserved
            val prodSheet = wb.getSheet("Products and Stock")
            val dataRow = prodSheet.getRow(1)
            assertNotNull(dataRow)
            assertEquals("PROD-001", dataRow.getCell(0).stringCellValue)
            assertEquals("Classmate Notebook 200p /      ", dataRow.getCell(2).stringCellValue)
            assertEquals(50.0, dataRow.getCell(7).numericCellValue, 0.001) // Sellable stock units
            assertEquals(35.0, dataRow.getCell(8).numericCellValue, 0.001) // Cost ₹35.00
            assertEquals(1750.0, dataRow.getCell(11).numericCellValue, 0.001) // Valuation ₹1750.00

            wb.close()
        }
    }

    @Test
    fun testOuterPackageBundleAndAutoDetectInnerCompleteZip() {
        // Create synthetic Complete_Backup.zip
        val completeBackupFile = tempFolder.newFile("Complete_Backup.zip")
        ZipOutputStream(FileOutputStream(completeBackupFile)).use { zos ->
            zos.putNextEntry(ZipEntry("ali_educational.db"))
            zos.write("SQLite format 3\u0000fake-db-bytes".toByteArray(Charsets.US_ASCII))
            zos.closeEntry()

            zos.putNextEntry(ZipEntry("manifest.json"))
            zos.write("{\"formatVersion\":1,\"schemaVersion\":8}".toByteArray(Charsets.UTF_8))
            zos.closeEntry()
        }

        // Create outer package ZIP
        val packageFile = tempFolder.newFile("Ali_Educational_Backup_Package_2026-11-01_20-24-23.zip")
        ZipOutputStream(FileOutputStream(packageFile)).use { zos ->
            // Put Complete_Backup.zip inside
            FileInputStream(completeBackupFile).use { fis ->
                zos.putNextEntry(ZipEntry("Complete_Backup.zip"))
                fis.copyTo(zos)
                zos.closeEntry()
            }

            // Put Shop_Records.xlsx inside
            zos.putNextEntry(ZipEntry("Shop_Records.xlsx"))
            zos.write("dummy excel bytes".toByteArray())
            zos.closeEntry()

            // Put Read_Me.txt inside
            zos.putNextEntry(ZipEntry("Read_Me.txt"))
            zos.write("ReadMe instructions".toByteArray())
            zos.closeEntry()
        }

        // Verify that scanning packageFile finds Complete_Backup.zip
        var foundCompleteZip = false
        var foundExcel = false
        var foundReadMe = false

        ZipInputStream(FileInputStream(packageFile)).use { zis ->
            var entry: ZipEntry?
            while (zis.nextEntry.also { entry = it } != null) {
```

```
                when (entry!!.name) {
                    "Complete_Backup.zip" -> foundCompleteZip = true
                    "Shop_Records.xlsx" -> foundExcel = true
                    "Read_Me.txt" -> foundReadMe = true
                }
                zis.closeEntry()
            }
        }

        assertTrue("Must contain Complete_Backup.zip", foundCompleteZip)
        assertTrue("Must contain Shop_Records.xlsx", foundExcel)
        assertTrue("Must contain Read_Me.txt", foundReadMe)
    }

    @Test
    fun testZipSlipPathTraversalRejection() {
        val maliciousFile = tempFolder.newFile("malicious.zip")
        ZipOutputStream(FileOutputStream(maliciousFile)).use { zos ->
            zos.putNextEntry(ZipEntry("../../../etc/passwd"))
            zos.write("malicious payload".toByteArray())
            zos.closeEntry()
        }

        var threwSecurityException = false
        try {
            ZipInputStream(FileInputStream(maliciousFile)).use { zis ->
                var entry: ZipEntry?
                while (zis.nextEntry.also { entry = it } != null) {
                    val entryName = entry!!.name
                    if (entryName.contains("..") || entryName.startsWith("/") || entryName.startsWith("\\"))
    {
                        throw SecurityException("Malicious path detected: $entryName")
                    }
                    zis.closeEntry()
                }
            }
        } catch (e: SecurityException) {
            threwSecurityException = true
        }

        assertTrue("Path traversal entry must be rejected with SecurityException", threwSecurityException)
    }
}
```

app\src\test\java\com\alieducational\app\domain\calculation\CostCalculatorTest.kt

```
package com.alieducational.app.domain.calculation

import org.junit.Assert.assertEquals
import org.junit.Assert.assertNull
import org.junit.Test

class CostCalculatorTest {

    @Test
    fun testCost01_ProportionalDeliveryAllocation() {
        // Spec COST-01 / CORE-05:
        // 20 pens at ₹10 = ₹200 (20000 paise)
        // 10 notebooks at ₹30 = ₹300 (30000 paise)
        // Delivery = ₹50 (5000 paise)
        // Total stock cost = ₹550 (55000 paise)
        // Expected: Pens get ₹20 (2000 paise), notebooks get ₹30 (3000 paise)
        // Pens unit cost = ₹11 (1100 paise), Notebooks unit cost = ₹33 (3300 paise)
        val lines = listOf(
            PurchaseLineInput(id = "pens", quantity = 20, unitPricePaise = 1000L),
            PurchaseLineInput(id = "notebooks", quantity = 10, unitPricePaise = 3000L)
        )
        val results = CostCalculator.allocateDelivery(lines, totalDeliveryPaise = 5000L)

        assertEquals(2, results.size)

        val penResult = results.first { it.lineId == "pens" }
        assertEquals(20000L, penResult.goodsValuePaise)
        assertEquals(2000L, penResult.allocatedDeliveryPaise)
        assertEquals(22000L, penResult.totalLandedCostPaise)
        assertEquals(1100L, penResult.effectiveUnitCostPaise) // ₹11

        val nbResult = results.first { it.lineId == "notebooks" }
        assertEquals(30000L, nbResult.goodsValuePaise)
        assertEquals(3000L, nbResult.allocatedDeliveryPaise)
        assertEquals(33000L, nbResult.totalLandedCostPaise)
        assertEquals(3300L, nbResult.effectiveUnitCostPaise) // ₹33

        // Total delivery allocated must equal total delivery charge exactly
        assertEquals(5000L, penResult.allocatedDeliveryPaise + nbResult.allocatedDeliveryPaise)
    }

    @Test
    fun testCost02_MovingWeightedAverage() {
        // Spec COST-02:
        // 10 pieces cost ₹100 (10000 paise, avg ₹10 = 1000 paise)
        // Receive 10 with goods ₹140 (14000 paise) + delivery ₹20 (2000 paise) = total landed ₹160 (16000 paise)
        // Combined quantity = 20, stock value = ₹260 (26000 paise), moving average = ₹13 (1300 paise)
        val newAvg = CostCalculator.calculateMovingWeightedAverage(
            existingStockQuantity = 10,
            existingAverageCostPaise = 1000L,
            incomingQuantity = 10,
            incomingTotalLandedCostPaise = 16000L
        )
        assertEquals(1300L, newAvg)
    }

    @Test
    fun testCost03_SaleWithDiscountAndProfit() {
        // Spec COST-03 / CORE-06:
        // Sell 5 from COST-02 for ₹100 gross (selling price ₹20 = 2000 paise each)
        // 10% discount = ₹10 (1000 paise) -> net sale = ₹90 (9000 paise)
        // Cost of 5 items at ₹13 (1300 paise) = ₹65 (6500 paise)
        // Actual gross profit = ₹90 - ₹65 = ₹25 (2500 paise)
        val lines = listOf(
            SaleLineInput(
                id = "item1",
                quantity = 5,
                unitSellingPricePaise = 2000L,
                unitCostPaise = 1300L
            )
        )
        val results = CostCalculator.allocateWholeSaleDiscount(lines, totalDiscountPaise = 1000L)

        assertEquals(1, results.size)
        val result = results[0]
        assertEquals(10000L, result.grossValuePaise)
        assertEquals(1000L, result.allocatedDiscountPaise)
        assertEquals(9000L, result.netValuePaise)
        assertEquals(2500L, result.actualProfitPaise) // ₹25 gross profit
    }

```

```
    @Test
    fun testCost04_HistoricalSaleNotAffectedByLaterReceipts() {
        // Spec COST-04:
        // Initial stock: 15 pieces at ₹13 (1300 paise)
        // Receive 10 more at ₹18 (1800 paise) landed cost
        // New avg = (15 * 1300 + 10 * 1800) / 25 = (19500 + 18000) / 25 = 37500 / 25 = 1500 paise (₹15)
        val updatedAvg = CostCalculator.calculateMovingWeightedAverage(
            existingStockQuantity = 15,
            existingAverageCostPaise = 1300L,
            incomingQuantity = 10,
            incomingTotalLandedCostPaise = 18000L
        )
        assertEquals(1500L, updatedAvg)
    }

    @Test
    fun testCore01AndFinal01_BoxesAndPieces() {
        // Spec FINAL-01: Stock is 23 pieces, 10 per box
        // Result: 2 boxes + 3 pieces
        val (boxes, pieces) = CostCalculator.formatBoxesAndPieces(23, 10)
        assertEquals(2, boxes)
        assertEquals(3, pieces)
    }

    @Test
    fun testZeroGoodsValueDeliveryAllocation() {
        // Edge case: All lines are zero-value (e.g. promotional free samples)
        // Delivery = ₹30 (3000 paise) across 2 lines -> ₹15 each
        val lines = listOf(
            PurchaseLineInput(id = "sample1", quantity = 5, unitPricePaise = 0L),
            PurchaseLineInput(id = "sample2", quantity = 5, unitPricePaise = 0L)
        )
        val results = CostCalculator.allocateDelivery(lines, totalDeliveryPaise = 3000L)
        assertEquals(1500L, results[0].allocatedDeliveryPaise)
        assertEquals(1500L, results[1].allocatedDeliveryPaise)
        assertEquals(3000L, results.sumOf { it.allocatedDeliveryPaise })
    }

    @Test
    fun testMissingCostProfitIsUnavailable() {
        // Spec CORE-12: When cost is unknown (null), profit must be null, not zero!
        val lines = listOf(
            SaleLineInput(
                id = "item_no_cost",
                quantity = 1,
                unitSellingPricePaise = 5000L,
                unitCostPaise = null
            )
        )
        val results = CostCalculator.allocateWholeSaleDiscount(lines, totalDiscountPaise = 0L)
        assertNull(results[0].actualProfitPaise)
    }

    @Test
    fun testCurrencyFormatting() {
        assertEquals("₹100.50", CostCalculator.formatPaiseToRupees(10050L))
        assertEquals("₹0.00", CostCalculator.formatPaiseToRupees(0L))
        assertEquals("-₹540.74", CostCalculator.formatPaiseToRupees(-54074L))
    }

    @Test
    fun testIndianDigitGrouping_Section23() {
        // Section 23 table:
        // 999 rupees -> ₹999.00
        assertEquals("₹999.00", CostCalculator.formatPaiseToRupees(99900L))
        // 1000 rupees -> ₹1,000.00
        assertEquals("₹1,000.00", CostCalculator.formatPaiseToRupees(100000L))
        // 12235 rupees -> ₹12,235.00
        assertEquals("₹12,235.00", CostCalculator.formatPaiseToRupees(1223500L))
        // 100000 rupees -> ₹1,00,000.00
        assertEquals("₹1,00,000.00", CostCalculator.formatPaiseToRupees(10000000L))
        // 1234567.50 rupees -> ₹12,34,567.50
        assertEquals("₹12,34,567.50", CostCalculator.formatPaiseToRupees(123456750L))
        // Negative 12197.72 rupees -> -₹12,197.72
        assertEquals("-₹12,197.72", CostCalculator.formatPaiseToRupees(-1219772L))
        // 123456 pieces -> 1,23,456 pcs
        assertEquals("1,23,456 pcs", CostCalculator.formatQuantityWithGrouping(123456, "pcs"))
    }
}
```

app\src\test\java\com\alieducational\app\service\CashTrackerExcelServiceTest.kt

```
package com.alieducational.app.service

import com.alieducational.app.data.service.CashTrackerExcelService
import org.apache.poi.xssf.usermodel.XSSFWorkbook
import org.junit.Assert.assertEquals
import org.junit.Assert.assertNotNull
import org.junit.Assert.assertNull
import org.junit.Assert.assertTrue
import org.junit.Before
import org.junit.Test

class CashTrackerExcelServiceTest {

    private lateinit var service: CashTrackerExcelService

    @Before
    fun setup() {
        service = CashTrackerExcelService()
    }

    @Test
    fun testParseWorkbook_StandardUserFormat() {
        val workbook = XSSFWorkbook()
        val sheet = workbook.createSheet("Cash In Out")

        // Row 0: Headers (Standard format selected by user)
        val headerRow = sheet.createRow(0)
        headerRow.createCell(0).setCellValue("Date")
        headerRow.createCell(1).setCellValue("Description")
        headerRow.createCell(2).setCellValue("Cash In (Income)")
        headerRow.createCell(3).setCellValue("Cash Out (Expense)")
        headerRow.createCell(4).setCellValue("Payment Mode")

        // Row 1: Income entry
        val r1 = sheet.createRow(1)
        r1.createCell(0).setCellValue("31-Oct-2026")
        r1.createCell(1).setCellValue("Sales Collection")
        r1.createCell(2).setCellValue(14084.0)
        r1.createCell(3).setCellValue(0.0)
        r1.createCell(4).setCellValue("CASH")

        // Row 2: Expense entry
        val r2 = sheet.createRow(2)
        r2.createCell(0).setCellValue("31-Oct-2026")
        r2.createCell(1).setCellValue("Shop Rent")
        r2.createCell(2).setCellValue(0.0)
        r2.createCell(3).setCellValue(2000.0)
        r2.createCell(4).setCellValue("BANK")

        // Row 3: Expense with Currency formatted string
        val r3 = sheet.createRow(3)
        r3.createCell(0).setCellValue("31-Oct-2026")
        r3.createCell(1).setCellValue("Pigmy - 4")
        r3.createCell(2).setCellValue("")
        r3.createCell(3).setCellValue("₹ 1,000.00")
        r3.createCell(4).setCellValue("GPAY")

        // Row 4: Empty row
        val r4 = sheet.createRow(4)
        r4.createCell(0).setCellValue("")

        val preview = service.parseWorkbook(workbook, "test_cash.xlsx")

        assertNull(preview.errorMessage)
        assertEquals(3, preview.items.size)
        assertEquals(1, preview.incomeCount)
        assertEquals(2, preview.expenseCount)
        assertEquals(1408400L, preview.totalIncomePaise) // 14,084.00 * 100
        assertEquals(300000L, preview.totalExpensePaise) // (2,000 + 1,000) * 100

        // Check individual items
        val salesItem = preview.items[0]
        assertEquals("Sales Collection", salesItem.category)
        assertTrue(salesItem.isIncome)
        assertEquals(1408400L, salesItem.amountPaise)
        assertEquals("CASH", salesItem.notes)

        val rentItem = preview.items[1]
        assertEquals("Shop Rent", rentItem.category)
        assertTrue(!rentItem.isIncome)
        assertEquals(200000L, rentItem.amountPaise)
        assertEquals("BANK", rentItem.notes)
```

```
83
84          val pigmyItem = preview.items[2]
85          assertEquals("Pigmy - 4", pigmyItem.category)
86          assertTrue(!pigmyItem.isIncome)
87          assertEquals(100000L, pigmyItem.amountPaise)
88          assertEquals("GPAY", pigmyItem.notes)
89
90          workbook.close()
91      }
92
93      @Test
94      fun testParseRealExcelFiles() {
95          val file29 = java.io.File("C:/Users/Musid/Downloads/Ali Educational 29-Oct-2026.xls")
96          if (file29.exists()) {
97              val wb = org.apache.poi.ss.usermodel.WorkbookFactory.create(file29)
98              println("File 29 sheets: " + (0 until wb.numberOfSheets).map { wb.getSheetName(it) })
99              val preview29 = service.parseWorkbook(wb, file29.name)
100             println("29-Oct items: ${preview29.items.size}, income: ${preview29.incomeCount}
    (₹${preview29.totalIncomePaise/100}), expense: ${preview29.expenseCount}
    (₹${preview29.totalExpensePaise/100}), err: ${preview29.errorMessage}")
101             wb.close()
102             assertNull(preview29.errorMessage)
103             assertTrue(preview29.items.isNotEmpty())
104             // Verify no item has a blank category/description
105             val blankDescItems = preview29.items.filter { it.category.isBlank() }
106             assertEquals("All items should have a non-blank description/category", 0, blankDescItems.size)
107         }
108
109         val file31 = java.io.File("C:/Users/Musid/Downloads/Ali Educational 31-Oct-2026.xls")
110         if (file31.exists()) {
111             val wb = org.apache.poi.ss.usermodel.WorkbookFactory.create(file31)
112             val preview31 = service.parseWorkbook(wb, file31.name)
113             println("31-Oct items: ${preview31.items.size}, income: ${preview31.incomeCount}
    (₹${preview31.totalIncomePaise/100}), expense: ${preview31.expenseCount}
    (₹${preview31.totalExpensePaise/100}), err: ${preview31.errorMessage}")
114             wb.close()
115             assertNull(preview31.errorMessage)
116             assertTrue(preview31.items.isNotEmpty())
117         }
118     }
119 }
```

app\src\test\java\com\alieducational\app\service\FullSyntheticFixtureLifecycleTest.kt

```
package com.alieducational.app.service

import com.alieducational.app.data.local.entity.ShopExpenseEntity
import com.alieducational.app.data.service.CashTrackerExcelService
import com.alieducational.app.domain.calculation.CostCalculator
import org.apache.poi.ss.usermodel.WorkbookFactory
import org.junit.Assert.assertEquals
import org.junit.Assert.assertFalse
import org.junit.Assert.assertNotNull
import org.junit.Assert.assertNull
import org.junit.Assert.assertTrue
import org.junit.Test
import java.io.File
import java.io.FileInputStream
import java.io.FileOutputStream
import java.util.zip.ZipEntry
import java.util.zip.ZipInputStream
import java.util.zip.ZipOutputStream

class FullSyntheticFixtureLifecycleTest {

    private val fixturesDir = File("C:/Users/Musid/.gemini/antigravity/brain/d92e10cb-d589-4a63-9db5-449f958ba78f/scratch")

    @Test
    fun testF6_SyntheticStockFixtureImportAndTotals() {
        val stockFile = File(fixturesDir, "backup_stock_fixture.xlsx")
        assertTrue("Fixture file backup_stock_fixture.xlsx must exist", stockFile.exists())

        val wb = WorkbookFactory.create(stockFile)
        val sheet = wb.getSheetAt(0)
        assertNotNull("Sheet must exist", sheet)

        // Headers check
        val headerRow = sheet.getRow(0)
        assertEquals("SKU / Barcode", headerRow.getCell(0).stringCellValue)
        assertEquals("Product Name", headerRow.getCell(1).stringCellValue)

        data class TestProduct(
            val sku: String,
            val name: String,
            val openingPieces: Int,
            val packSize: Int,
            val unitCostPaise: Long,
            val pieceSellingPricePaise: Long,
            val packSellingPricePaise: Long
        )

        val products = mutableListOf<TestProduct>()
        for (r in 1..sheet.lastRowNum) {
            val row = sheet.getRow(r) ?: continue
            val sku = row.getCell(0).stringCellValue.trim()
            val name = row.getCell(1).stringCellValue.trim()
            val packSize = row.getCell(5).numericCellValue.toInt()
            val pieces = row.getCell(6).numericCellValue.toInt()
            val costPaise = (row.getCell(7).numericCellValue * 100).toLong()
            val piecePricePaise = (row.getCell(8).numericCellValue * 100).toLong()
            val packPricePaise = (row.getCell(9).numericCellValue * 100).toLong()

            products.add(
                TestProduct(sku, name, pieces, packSize, costPaise, piecePricePaise, packPricePaise)
            )
        }
        wb.close()

        assertEquals("Must parse 3 products", 3, products.size)

        // 1. Test Notebook
        val notebook = products.find { it.name == "Test Notebook" }
        assertNotNull("Notebook found", notebook)
        assertEquals("004521", notebook!!.sku)
        assertEquals(20, notebook.openingPieces)
        assertEquals(1, notebook.packSize)
        assertEquals(5000L, notebook.unitCostPaise) // ₹50.00
        assertEquals(8000L, notebook.pieceSellingPricePaise) // ₹80.00

        // 2. Test Blue Pen
        val pen = products.find { it.name == "Test Blue Pen" }
        assertNotNull("Pen found", pen)
        assertEquals("009870", pen!!.sku)
        assertEquals(100, pen.openingPieces)
        assertEquals(10, pen.packSize)
```

```
        assertEquals(500L, pen.unitCostPaise) // ₹5.00
        assertEquals(800L, pen.pieceSellingPricePaise) // ₹8.00
        assertEquals(7500L, pen.packSellingPricePaise) // ₹75.00

        // 3. Test Urdu-name product —
        val urduItem = products.find { it.name.contains("   ") }
        assertNotNull("Urdu item found", urduItem)
        assertEquals("001234", urduItem!!.sku)
        assertEquals(12, urduItem.openingPieces)
        assertEquals(1025L, urduItem.unitCostPaise) // ₹10.25
        assertEquals(1550L, urduItem.pieceSellingPricePaise) // ₹15.50

        // Total opening pieces = 132
        val totalOpeningPieces = products.sumOf { it.openingPieces }
        assertEquals("Expected opening pieces: 132", 132, totalOpeningPieces)

        // Known inventory cost = ₹1,623.00 (162,300 paise)
        val totalInventoryCostPaise = products.sumOf { it.openingPieces * it.unitCostPaise }
        assertEquals("Expected opening inventory cost: 1623.00 Rs", 162300L, totalInventoryCostPaise)
    }

    @Test
    fun testF6_SyntheticCashLedgerImportAndTotals() {
        val cashFile = File(fixturesDir, "cash_ledger_fixture.xlsx")
        assertTrue("Fixture file cash_ledger_fixture.xlsx must exist", cashFile.exists())

        val wb = WorkbookFactory.create(cashFile)
        val service = CashTrackerExcelService()
        val preview = service.parseWorkbook(wb, cashFile.name)
        wb.close()

        assertNull("No parse errors expected: ${preview.errorMessage}", preview.errorMessage)

        // Opening balance and totals rows must NOT become transactions
        assertEquals("Expected transaction count 2 (opening & totals rows excluded)", 2, preview.items.size)
        assertEquals(1, preview.incomeCount)
        assertEquals(1, preview.expenseCount)

        // Dated income ₹1,000.25
        val incomeItem = preview.items.find { it.isIncome }
        assertNotNull("Income item present", incomeItem)
        assertEquals(100025L, incomeItem!!.amountPaise)
        assertEquals(100025L, preview.totalIncomePaise)

        // Dated outflow ₹250.50
        val expenseItem = preview.items.find { !it.isIncome }
        assertNotNull("Expense item present", expenseItem)
        assertEquals(25050L, expenseItem!!.amountPaise)
        assertEquals(25050L, preview.totalExpensePaise)

        // Net movement = ₹749.75
        val netMovementPaise = preview.totalIncomePaise - preview.totalExpensePaise
        assertEquals("Expected net movement ₹749.75", 74975L, netMovementPaise)
    }

    @Test
    fun testF6_Sale_PigmyDeposit_Giveaway_InventoryLifecycle() {
        // Initial state from fixture:
        var notebookStock = 20
        var penStock = 100
        var urduItemStock = 12
        val notebookCostPaise = 5000L
        val penCostPaise = 500L
        val urduCostPaise = 1025L

        assertEquals(132, notebookStock + penStock + urduItemStock)
        assertEquals(162300L, (notebookStock * notebookCostPaise) + (penStock * penCostPaise) +
(urduItemStock * urduCostPaise))

        // Step 4: Product sale of 2 Test Notebooks at ₹80 each, paid ₹200 cash
        val saleQty = 2
        val salePricePaise = 8000L
        val grossTotalPaise = saleQty * salePricePaise // ₹160.00
        val paidCashPaise = 20000L // ₹200.00 tendered
        val changeGivenPaise = paidCashPaise - grossTotalPaise // ₹40.00 change

        notebookStock -= saleQty
        val cogsPaise = saleQty * notebookCostPaise // ₹100.00
        val grossProfitPaise = grossTotalPaise - cogsPaise // ₹60.00

        assertEquals(16000L, grossTotalPaise)
        assertEquals(4000L, changeGivenPaise)
        assertEquals(18, notebookStock)
        assertEquals(10000L, cogsPaise)
        assertEquals(6000L, grossProfitPaise)

        // Step 5: Record confirmed ₹100 cash pigmy deposit and giveaway of 1 Test Blue Pen
        val pigmyDepositPaise = 10000L // ₹100.00
```

```
        val pigmyOutflow = ShopExpenseEntity(
            id = "pigmy-outflow-1",
            category = "SAVINGS_TRANSFER",
            amountPaise = pigmyDepositPaise,
            isIncome = false,
            isSavingsTransfer = true
        )
        // Pigmy outflow is linked and excluded from operating profit
        assertTrue(pigmyOutflow.isSavingsTransfer)
        assertFalse(pigmyOutflow.isIncome)

        // Giveaway of 1 Test Blue Pen
        val giveawayQty = 1
        penStock -= giveawayQty
        assertEquals(99, penStock)

        // Inventory verification after sale and giveaway:
        val remainingTotalPieces = notebookStock + penStock + urduItemStock
        assertEquals("Expected inventory pieces after sale & giveaway: 129", 129, remainingTotalPieces)

        val remainingValuePaise = (notebookStock * notebookCostPaise) + (penStock * penCostPaise) +
(urduItemStock * urduCostPaise)
        // 18 * 50 = 900; 99 * 5 = 495; 12 * 10.25 = 123 -> Total = 1,518.00 (151,800 paise)
        assertEquals("Expected inventory known value: ₹1,518.00", 151800L, remainingValuePaise)
    }

    @Test
    fun testF6_BackupArchiveCreationValidationAndManifestIntegrity() {
        val tempDir = File(fixturesDir, "test_backup_temp").apply { mkdirs() }
        val dummyDb = File(tempDir, "ali_educational.db")

        // Create a mock SQLite header file
        FileOutputStream(dummyDb).use { fos ->
            val sqliteHeader = "SQLite format 3\u0000".toByteArray(Charsets.US_ASCII)
            fos.write(sqliteHeader)
            // Fill with dummy db payload
            fos.write(ByteArray(1024))
        }

        val backupZip = File(tempDir, "test_backup.zip")

        // Create ZIP with DB + manifest
        ZipOutputStream(FileOutputStream(backupZip)).use { zos ->
            // 1. ali_educational.db
            FileInputStream(dummyDb).use { fis ->
                zos.putNextEntry(ZipEntry("ali_educational.db"))
                fis.copyTo(zos)
                zos.closeEntry()
            }

            // 2. manifest.json
            val manifest = """
                {
                    "formatVersion": 1,
                    "app": "Ali Educational",
                    "packageName": "com.alieducational.app",
                    "schemaVersion": 4,
                    "dbSizeBytes": ${dummyDb.length()},
                    "timestamp": 1730416800000,
                    "piecesTotal": 129,
                    "inventoryValuePaise": 151800
                }
            """.trimIndent()
            zos.putNextEntry(ZipEntry("manifest.json"))
            zos.write(manifest.toByteArray(Charsets.UTF_8))
            zos.closeEntry()
        }

        assertTrue(backupZip.exists())
        assertTrue(backupZip.length() > 0)

        // Validate archive integrity
        var foundDb = false
        var validHeader = false
        var foundManifest = false

        ZipInputStream(FileInputStream(backupZip)).use { zis ->
            var entry: ZipEntry?
            while (zis.nextEntry.also { entry = it } != null) {
                when (entry!!.name) {
                    "ali_educational.db" -> {
                        foundDb = true
                        val header = ByteArray(16)
                        val read = zis.read(header, 0, 16)
                        if (read == 16 && String(header, Charsets.US_ASCII).startsWith("SQLite format 3")) {
                            validHeader = true
                        }
                    }
```

```
                    "manifest.json" -> {
                        foundManifest = true
                        val content = zis.bufferedReader(Charsets.UTF_8).readText()
                        assertTrue(content.contains("\"piecesTotal\": 129"))
                        assertTrue(content.contains("\"inventoryValuePaise\": 151800"))
                    }
                }
                zis.closeEntry()
            }
        }

        assertTrue("ali_educational.db found in zip", foundDb)
        assertTrue("Valid SQLite header verified in zip", validHeader)
        assertTrue("manifest.json found and verified in zip", foundManifest)

        // Clean up
        dummyDb.delete()
        backupZip.delete()
        tempDir.delete()
    }
}
```

app\src\test\java\com\alieducational\app\service\GoldenScenarioAndSection25Test.kt

```
package com.alieducational.app.service

import com.alieducational.app.data.local.entity.ShopExpenseEntity
import com.alieducational.app.domain.calculation.CostCalculator
import com.alieducational.app.domain.calculation.PurchaseLineInput
import com.alieducational.app.domain.calculation.SaleLineInput
import org.junit.Assert.assertEquals
import org.junit.Assert.assertTrue
import org.junit.Test
import java.util.Calendar

class GoldenScenarioAndSection25Test {

    @Test
    fun testGoldenScenario_Section20() {
        // G01-G14 Golden scenario from ANTIGRAVITY_LAYOUT_GRAPHS_SUPPLIERS_AND_SHOP_EXPENSES.md Section 20
        var physicalCashPaise = 1000000L // ₹10,000.00
        var bankBalancePaise = 0L

        // G01: Supplier A, Customer A added in directory (0 balance effect)
        var supplierADuePaise = 0L
        var customerADuePaise = 0L
        var notebookStockQty = 0
        var notebookAvgCostPaise = 0L

        // G02: Receive 100 notebooks for ₹1,000; pay supplier ₹600
        val g02GoodsCostPaise = 100000L // ₹1,000.00
        val g02PaidPaise = 60000L // ₹600.00
        val g02UnpaidPaise = g02GoodsCostPaise - g02PaidPaise // ₹400.00
        notebookStockQty += 100
        supplierADuePaise += g02UnpaidPaise
        physicalCashPaise -= g02PaidPaise
        assertEquals(940000L, physicalCashPaise) // ₹9,400.00
        assertEquals(40000L, supplierADuePaise) // ₹400.00

        // G03: Delivery ₹100 capitalized into this receipt
        val deliveryPaise = 10000L // ₹100.00
        physicalCashPaise -= deliveryPaise
        val totalLandedGoodsCost = g02GoodsCostPaise + deliveryPaise // ₹1,100.00
        notebookAvgCostPaise = totalLandedGoodsCost / notebookStockQty // ₹11.00 per piece (1100 paise)
        assertEquals(930000L, physicalCashPaise) // ₹9,300.00
        assertEquals(1100L, notebookAvgCostPaise) // ₹11.00 unit cost
        assertEquals(40000L, supplierADuePaise) // supplier due still ₹400

        // G04: Bill Customer A for 10 notebooks at ₹20 each; apply ₹20 whole-bill discount; collect ₹100
        // Gross: 10 * 20 = 200; Discount: 20; Net: 180; COGS: 10 * 11 = 110; Profit: 70
        val g04GrossPaise = 20000L
        val g04DiscountPaise = 2000L
        val g04NetPaise = g04GrossPaise - g04DiscountPaise // 18000L (₹180.00)
        val g04CogsPaise = 10 * notebookAvgCostPaise // 11000L (₹110.00)
        val g04GrossProfitPaise = g04NetPaise - g04CogsPaise // 7000L (₹70.00)
        val g04CollectedPaise = 10000L // ₹100.00
        val g04DuePaise = g04NetPaise - g04CollectedPaise // 8000L (₹80.00)
        notebookStockQty -= 10
        customerADuePaise += g04DuePaise
        physicalCashPaise += g04CollectedPaise
        assertEquals(90, notebookStockQty)
        assertEquals(940000L, physicalCashPaise) // ₹9,400.00
        assertEquals(8000L, customerADuePaise) // ₹80.00

        // G05: Service bill: 5 photocopies at ₹10 each, fully paid
        val g05ServiceRevenuePaise = 5000L // ₹50.00
        physicalCashPaise += g05ServiceRevenuePaise
        assertEquals(945000L, physicalCashPaise) // ₹9,450.00

        // G06: Receive ₹30 repayment against Customer A's bill
        val g06RepaymentPaise = 3000L // ₹30.00
        customerADuePaise -= g06RepaymentPaise
        physicalCashPaise += g06RepaymentPaise
        assertEquals(5000L, customerADuePaise) // ₹50.00 due
        assertEquals(948000L, physicalCashPaise) // ₹9,480.00

        // G07: Pay Supplier A remaining ₹400
        val g07SupplierPaymentPaise = 40000L // ₹400.00
        supplierADuePaise -= g07SupplierPaymentPaise
        physicalCashPaise -= g07SupplierPaymentPaise
        assertEquals(0L, supplierADuePaise) // ₹0.00 due
        assertEquals(908000L, physicalCashPaise) // ₹9,080.00

        // G08: Shop expense: electricity ₹120, Shared
        val g08ExpSharedPaise = 12000L
        physicalCashPaise -= g08ExpSharedPaise
```

```
        assertEquals(896000L, physicalCashPaise) // ₹8,960.00

        // G09: Shop expense: paper ₹80, Services
        val g09ExpServicesPaise = 8000L
        physicalCashPaise -= g09ExpServicesPaise
        assertEquals(888000L, physicalCashPaise) // ₹8,880.00

        // G10: Shop expense: shelving repair ₹40, Products
        val g10ExpProductsPaise = 4000L
        physicalCashPaise -= g10ExpProductsPaise
        assertEquals(884000L, physicalCashPaise) // ₹8,840.00

        // G11: Linked electricity expense refund: ₹20
        val g11RefundSharedPaise = 2000L
        physicalCashPaise += g11RefundSharedPaise
        assertEquals(886000L, physicalCashPaise) // ₹8,860.00

        // G12: Confirm ₹300 Pigmy deposit
        var pigmyTrackedSavingsPaise = 0L
        val g12PigmyDepositPaise = 30000L
        pigmyTrackedSavingsPaise += g12PigmyDepositPaise
        physicalCashPaise -= g12PigmyDepositPaise
        assertEquals(30000L, pigmyTrackedSavingsPaise) // ₹300.00
        assertEquals(856000L, physicalCashPaise) // ₹8,560.00

        // G13: Withdraw ₹100 Pigmy savings into shop cash
        val g13PigmyWithdrawalPaise = 10000L
        pigmyTrackedSavingsPaise -= g13PigmyWithdrawalPaise
        physicalCashPaise += g13PigmyWithdrawalPaise
        assertEquals(20000L, pigmyTrackedSavingsPaise) // ₹200.00
        assertEquals(866000L, physicalCashPaise) // ₹8,660.00

        // G14: Return 2 saleable notebooks from G04, refund ₹36 in cash
        // Net discounted return: 2 * 18 = 36; Cost reversal: 2 * 11 = 22
        val g14ReturnNetPaise = 3600L // ₹36.00
        val g14ReturnCogsPaise = 2200L // ₹22.00
        notebookStockQty += 2
        physicalCashPaise -= g14ReturnNetPaise
        assertEquals(92, notebookStockQty)
        assertEquals(862400L, physicalCashPaise) // ₹8,624.00

        // Net financial summaries
        val netProductSalesPaise = g04NetPaise - g14ReturnNetPaise // 18000 - 3600 = 14400 (₹144.00)
        val netProductCogsPaise = g04CogsPaise - g14ReturnCogsPaise // 11000 - 2200 = 8800 (₹88.00)
        val productGrossProfitPaise = netProductSalesPaise - netProductCogsPaise // 5600 (₹56.00)
        val netServiceRevenuePaise = g05ServiceRevenuePaise // 5000 (₹50.00)
        val combinedNetBilledRevenuePaise = netProductSalesPaise + netServiceRevenuePaise // 19400 (₹194.00)

        val grossOperatingExpensesPaise = g08ExpSharedPaise + g09ExpServicesPaise + g10ExpProductsPaise //
12000 + 8000 + 4000 = 24000 (₹240.00)
        val linkedExpenseRefundsPaise = g11RefundSharedPaise // 2000 (₹20.00)
        val netOperatingExpensesPaise = grossOperatingExpensesPaise - linkedExpenseRefundsPaise // 22000
(₹220.00)

        val netProductsExpensesPaise = g10ExpProductsPaise // 4000 (₹40.00)
        val netServicesExpensesPaise = g09ExpServicesPaise // 8000 (₹80.00)
        val netSharedExpensesPaise = g08ExpSharedPaise - g11RefundSharedPaise // 12000 - 2000 = 10000
(₹100.00)

        val productResultAfterExpensesPaise = productGrossProfitPaise - netProductsExpensesPaise // 5600 -
4000 = 1600 (₹16.00)
        val serviceResultAfterExpensesPaise = netServiceRevenuePaise - netServicesExpensesPaise // 5000 -
8000 = -3000 (-₹30.00)
        val combinedRecordedOperatingResultPaise = productGrossProfitPaise + netServiceRevenuePaise -
netOperatingExpensesPaise // 5600 + 5000 - 22000 = -11400 (-₹114.00)

        val remainingInventoryCostPaise = notebookStockQty * notebookAvgCostPaise // 92 * 1100 = 101200
(₹1,012.00)

        // Exact reconciliation assertions matching Section 20 table
        assertEquals("Sellable notebooks", 92, notebookStockQty)
        assertEquals("Remaining notebook inventory cost", 101200L, remainingInventoryCostPaise)
        assertEquals("Net product sales", 14400L, netProductSalesPaise)
        assertEquals("Product COGS", 8800L, netProductCogsPaise)
        assertEquals("Product gross profit", 5600L, productGrossProfitPaise)
        assertEquals("Net service revenue", 5000L, netServiceRevenuePaise)
        assertEquals("Combined net billed revenue", 19400L, combinedNetBilledRevenuePaise)
        assertEquals("Gross operating expenses", 24000L, grossOperatingExpensesPaise)
        assertEquals("Linked expense refunds", 2000L, linkedExpenseRefundsPaise)
        assertEquals("Net operating expenses", 22000L, netOperatingExpensesPaise)
        assertEquals("Net Products expenses", 4000L, netProductsExpensesPaise)
        assertEquals("Net Services expenses", 8000L, netServicesExpensesPaise)
        assertEquals("Net Shared expenses", 10000L, netSharedExpensesPaise)
        assertEquals("Product result after Products expenses", 1600L, productResultAfterExpensesPaise)
        assertEquals("Service revenue minus Services expenses", -3000L, serviceResultAfterExpensesPaise)
        assertEquals("Combined recorded operating result", -11400L, combinedRecordedOperatingResultPaise)
        assertEquals("Physical cash", 862400L, physicalCashPaise)
        assertEquals("Bank balance", 0L, bankBalancePaise)
```

```
164         assertEquals("Customer A remaining due", 5000L, customerADuePaise)
165         assertEquals("Supplier A due", 0L, supplierADuePaise)
166         assertEquals("Pigmy tracked savings", 20000L, pigmyTrackedSavingsPaise)
167
168         // Gross margin calculation: 56.00 / 144.00 = 38.89%
169         val marginPct = (productGrossProfitPaise.toDouble() / netProductSalesPaise.toDouble()) * 100.0
170         assertEquals("Gross margin ~38.89%", 38.89, marginPct, 0.01)
171
172         // Independent balance identity check (Section 20):
173         // Cash (8,624) + Inventory (1,012) + Receivables (50) + Pigmy (200) - Payables (0) = 9,886
174         // Equal to Opening Cash (10,000) + Combined Result (-114) = 9,886
175         val endingTotalAssets = physicalCashPaise + remainingInventoryCostPaise + customerADuePaise +
    pigmyTrackedSavingsPaise - supplierADuePaise
176         val expectedAssetsFromOperations = 1000000L + combinedRecordedOperatingResultPaise
177         assertEquals(988600L, endingTotalAssets)
178         assertEquals(expectedAssetsFromOperations, endingTotalAssets)
179     }
180
181     @Test
182     fun testSection25_IndependentExportFixture() {
183         // Section 25 Independent Export Fixture
184         val cal = Calendar.getInstance()
185         fun makeTime(day: Int): Long {
186             cal.set(2026, Calendar.NOVEMBER, day, 12, 0, 0)
187             cal.set(Calendar.MILLISECOND, 0)
188             return cal.timeInMillis
189         }
190
191         val e1 = ShopExpenseEntity(id = "E1", category = "Electricity", amountPaise = 100000L, isIncome =
    false, timestamp = makeTime(1), classification = "SHARED")
192         val e2 = ShopExpenseEntity(id = "E2", category = "Paper", amountPaise = 20000L, isIncome = false,
    timestamp = makeTime(2), classification = "SERVICES")
193         val e3 = ShopExpenseEntity(id = "E3", category = "Electricity", amountPaise = 10000L, isIncome =
    true, timestamp = makeTime(3), isExpenseRefund = true, linkedExpenseId = "E1", classification = "SHARED")
194         val e4 = ShopExpenseEntity(id = "E4", category = "Other", amountPaise = 50000L, isIncome = true,
    timestamp = makeTime(4), classification = "SHARED")
195         val e5 = ShopExpenseEntity(id = "E5", category = "Electricity", amountPaise = 30000L, isIncome =
    false, timestamp = makeTime(5), classification = "SHARED")
196
197         val allItems = listOf(e1, e2, e3, e4, e5)
198
199         cal.set(2026, Calendar.NOVEMBER, 1, 0, 0, 0)
200         cal.set(Calendar.MILLISECOND, 0)
201         val rangeStart = cal.timeInMillis
202
203         cal.set(2026, Calendar.NOVEMBER, 3, 23, 59, 59)
204         cal.set(Calendar.MILLISECOND, 999)
205         val rangeEnd = cal.timeInMillis
206
207         // 1. Select 1-3 Nov and Electricity: 2 rows (E1, E3), inflow ₹100, outflow ₹1,000, net -₹900
208         val filter1 = allItems.filter { it.timestamp in rangeStart..rangeEnd && it.category == "Electricity"
     }
209         assertEquals(2, filter1.size)
210         val inflow1 = filter1.filter { it.isIncome }.sumOf { it.amountPaise }
211         val outflow1 = filter1.filter { !it.isIncome }.sumOf { it.amountPaise }
212         val net1 = inflow1 - outflow1
213         assertEquals(10000L, inflow1) // ₹100.00
214         assertEquals(100000L, outflow1) // ₹1,000.00
215         assertEquals(-90000L, net1) // -₹900.00
216
217         // 2. Select 1-3 Nov and Electricity + Paper: 3 rows (E1, E2, E3), inflow ₹100, outflow ₹1,200, net
    -₹1,100
218         val filter2 = allItems.filter { it.timestamp in rangeStart..rangeEnd && (it.category ==
    "Electricity" || it.category == "Paper") }
219         assertEquals(3, filter2.size)
220         val inflow2 = filter2.filter { it.isIncome }.sumOf { it.amountPaise }
221         val outflow2 = filter2.filter { !it.isIncome }.sumOf { it.amountPaise }
222         val net2 = inflow2 - outflow2
223         assertEquals(10000L, inflow2) // ₹100.00
224         assertEquals(120000L, outflow2) // ₹1,200.00
225         assertEquals(-110000L, net2) // -₹1,100.00
226
227         // 3. Select 1-3 Nov, Electricity and Out only: 1 row (E1), outflow ₹1,000, net -₹1,000
228         val filter3 = allItems.filter { it.timestamp in rangeStart..rangeEnd && it.category == "Electricity"
     && !it.isIncome }
229         assertEquals(1, filter3.size)
230         val inflow3 = filter3.filter { it.isIncome }.sumOf { it.amountPaise }
231         val outflow3 = filter3.filter { !it.isIncome }.sumOf { it.amountPaise }
232         val net3 = inflow3 - outflow3
233         assertEquals(0L, inflow3)
234         assertEquals(100000L, outflow3) // ₹1,000.00
235         assertEquals(-100000L, net3) // -₹1,000.00
236
237         // Invariants: E4 (4 Nov) and E5 (5 Nov) never appear in 1-3 Nov selection
238         assertTrue(filter1.none { it.id == "E4" || it.id == "E5" })
239         assertTrue(filter2.none { it.id == "E4" || it.id == "E5" })
240         assertTrue(filter3.none { it.id == "E4" || it.id == "E5" })
241     }
```

```
242
243     @Test
244     fun testIndianNumberGroupingValues() {
245         // Section 23 table:
246         assertEquals("₹999.00", CostCalculator.formatPaiseToRupees(99900L))
247         assertEquals("₹1,000.00", CostCalculator.formatPaiseToRupees(100000L))
248         assertEquals("₹12,235.00", CostCalculator.formatPaiseToRupees(1223500L))
249         assertEquals("₹1,00,000.00", CostCalculator.formatPaiseToRupees(10000000L))
250         assertEquals("₹12,34,567.50", CostCalculator.formatPaiseToRupees(123456750L))
251         assertEquals("-₹12,197.72", CostCalculator.formatPaiseToRupees(-1219772L))
252         assertEquals("1,23,456 pcs", CostCalculator.formatQuantityWithGrouping(123456, "pcs"))
253     }
254 }
```

app\src\test\java\com\alieducational\app\service\ImportExportRobustnessTest.kt

```
package com.alieducational.app.service

import org.apache.poi.ss.usermodel.WorkbookFactory
import org.apache.poi.xssf.usermodel.XSSFWorkbook
import org.junit.Assert.assertEquals
import org.junit.Assert.assertNotNull
import org.junit.Assert.assertTrue
import org.junit.Test
import java.io.ByteArrayInputStream
import java.io.ByteArrayOutputStream

class ImportExportRobustnessTest {

    @Test
    fun testCurrencyParsing_AllVariants() {
        val testCases = mapOf(
            "₹150" to 15000L,
            "Rs. 25.50" to 2550L,
            "1,250.00" to 125000L,
            " ₹ 99.99 " to 9999L,
            "0" to 0L,
            "" to 0L,
            "invalid" to 0L
        )

        for ((input, expectedPaise) in testCases) {
            val clean = input.replace("₹", "").replace("Rs.", "", ignoreCase = true).replace("Rs", "",
ignoreCase = true).replace(",", "").trim()
            val valueDouble = clean.toDoubleOrNull() ?: 0.0
            val actualPaise = (valueDouble * 100).toLong()
            assertEquals("Failed for $input", expectedPaise, actualPaise)
        }
    }

    @Test
    fun testCsvParsing_HandlesQuotesAndCommasCleanly() {
        val sampleLine = "\"Dux Geometry Box, Deluxe\",Geometry,Dux,DUX-01,\"25\",\"120.00\",\"150.00\""
        val result = mutableListOf<String>()
        var curVal = StringBuilder()
        var inQuotes = false

        for (c in sampleLine) {
            if (c == '\"') {
                inQuotes = !inQuotes
            } else if (c == ',' && !inQuotes) {
                result.add(curVal.toString().trim())
                curVal = StringBuilder()
            } else {
                curVal.append(c)
            }
        }
        result.add(curVal.toString().trim())

        assertEquals(7, result.size)
        assertEquals("Dux Geometry Box, Deluxe", result[0])
        assertEquals("Geometry", result[1])
        assertEquals("Dux", result[2])
        assertEquals("DUX-01", result[3])
        assertEquals("25", result[4])
        assertEquals("120.00", result[5])
        assertEquals("150.00", result[6])
    }

    @Test
    fun testMultiSheetWorkbook_GeneratesAndReadsSheetNames() {
        val workbook = XSSFWorkbook()
        val sheet1 = workbook.createSheet("Primary Catalog")
        val sheet2 = workbook.createSheet("Secondary Wholesaler")

        val r1 = sheet1.createRow(0)
        r1.createCell(0).setCellValue("Product Name")
        r1.createCell(1).setCellValue("Price")

        val r2 = sheet2.createRow(0)
        r2.createCell(0).setCellValue("Wholesale Item")
        r2.createCell(1).setCellValue("Box Rate")

        val out = ByteArrayOutputStream()
        workbook.write(out)
        workbook.close()

        val inStream = ByteArrayInputStream(out.toByteArray())
```

```
        val readWb = WorkbookFactory.create(inStream)
        assertEquals(2, readWb.numberOfSheets)
        assertEquals("Primary Catalog", readWb.getSheetName(0))
        assertEquals("Secondary Wholesaler", readWb.getSheetName(1))
        readWb.close()
    }

    @Test
    fun testCorruptWorkbook_HandledGracefullyWithoutCrashing() {
        val corruptBytes = "NOT AN EXCEL FILE CONTENT".toByteArray()
        val inStream = ByteArrayInputStream(corruptBytes)
        var caughtException = false
        var friendlyMessage: String? = null

        try {
            WorkbookFactory.create(inStream)
        } catch (e: Exception) {
            caughtException = true
            val msg = e.localizedMessage ?: e.javaClass.simpleName
            friendlyMessage = when {
                msg.contains("password", ignoreCase = true) -> "Password protected"
                else -> "The file format is not a recognized Excel spreadsheet. Please ensure it is a valid .xlsx, .xls, or .csv file."
            }
        }

        assertTrue("Should have caught corrupt file exception", caughtException)
        assertNotNull(friendlyMessage)
        assertTrue(friendlyMessage!!.contains("not a recognized Excel spreadsheet"))
    }

    @Test
    fun testCashLedgerDetection_RealReferenceFiles() {
        val file29 = java.io.File("C:/Users/Musid/Downloads/Ali Educational 29-Oct-2026.xls")
        if (file29.exists()) {
            val wb = WorkbookFactory.create(file29)
            val sheet = wb.getSheetAt(0)
            val headers = mutableListOf<String>()
            // Find header on row 5
            for (r in 0..minOf(10, sheet.lastRowNum)) {
                val row = sheet.getRow(r) ?: continue
                val rowVals = (0 until row.lastCellNum).map { c ->
                    val cell = row.getCell(c) ?: return@map ""
                    try { cell.stringCellValue?.trim() ?: "" } catch (_: Exception) { "" }
                }
                if (rowVals.any { it.contains("Income", ignoreCase = true) }) {
                    headers.addAll(rowVals)
                    break
                }
            }
            wb.close()

            assertTrue("Should find header with Income in 29-Oct file", headers.isNotEmpty())
            val hLower = headers.map { it.lowercase() }
            val isCash = hLower.any { it.contains("income") || it.contains("expense") } &&
                    hLower.any { it.contains("balance") || it.contains("notes") || it.contains("date") }
            assertTrue("29-Oct file should be detected as a cash ledger", isCash)
        }
    }
}
```

app\src\test\java\com\alieducational\app\service\NewFeaturesReconciliationTest.kt

```
package com.alieducational.app.service

import com.alieducational.app.data.local.entity.ShopExpenseEntity
import com.alieducational.app.domain.calculation.CostCalculator
import com.alieducational.app.domain.calculation.SaleLineInput
import com.alieducational.app.ui.BillingDraftItem
import org.junit.Assert.assertEquals
import org.junit.Assert.assertFalse
import org.junit.Assert.assertTrue
import org.junit.Test
import java.util.UUID

class NewFeaturesReconciliationTest {

    @Test
    fun testDiscountAllocationWithUniqueLineIds() {
        // Two distinct lines of the SAME product: piece line and pack line
        val line1 = SaleLineInput(
            id = "line-1",
            quantity = 2,
            unitSellingPricePaise = 5000L, // 2 * 50 = 100 Rs = 10,000 paise
            unitCostPaise = 3000L
        )
        val line2 = SaleLineInput(
            id = "line-2",
            quantity = 1,
            unitSellingPricePaise = 45000L, // 1 * 450 = 450 Rs = 45,000 paise
            unitCostPaise = 25000L
        )

        val lines = listOf(line1, line2)
        val wholeBillDiscount = 5500L // 55 Rs (10% of 550 Rs)

        val results = CostCalculator.allocateWholeSaleDiscount(lines, wholeBillDiscount)

        assertEquals("Both lines must have allocated discounts", 2, results.size)
        val resultMap = results.associateBy { it.lineId }
        assertTrue("Line 1 must exist in result map", resultMap.containsKey("line-1"))
        assertTrue("Line 2 must exist in result map", resultMap.containsKey("line-2"))

        val totalAllocated = results.sumOf { it.allocatedDiscountPaise }
        assertEquals("Total allocated discount must equal whole-bill discount", wholeBillDiscount,
totalAllocated)
        assertEquals("Line 1 discount is 10%", 1000L, resultMap["line-1"]?.allocatedDiscountPaise)
        assertEquals("Line 2 discount is 10%", 4500L, resultMap["line-2"]?.allocatedDiscountPaise)
    }

    @Test
    fun testPigmySavingsExcludedFromOperatingExpenses() {
        val regularExpense = ShopExpenseEntity(
            id = "exp-1",
            category = "ELECTRICITY",
            amountPaise = 250000L,
            isIncome = false,
            isSavingsTransfer = false
        )
        val pigmyTransfer = ShopExpenseEntity(
            id = "pigmy-transfer-1",
            category = "SAVINGS_TRANSFER",
            amountPaise = 100000L,
            isIncome = false,
            isSavingsTransfer = true
        )
        val serviceExpense = ShopExpenseEntity(
            id = "svc-exp-1",
            category = "SERVICE_EXPENSE",
            amountPaise = 50000L,
            isIncome = false,
            isSavingsTransfer = false
        )

        val allExpenses = listOf(regularExpense, pigmyTransfer, serviceExpense)

        // Filter operating expenses per Phase 6 & Phase 8 rules:
        val operatingExpenses = allExpenses.filter {
            !it.isVoided && !it.isIncome && !it.isSavingsTransfer && it.category != "SERVICE_EXPENSE"
        }

        assertEquals("Only regular operating expense must be included", 1, operatingExpenses.size)
        assertEquals("Regular expense amount", 250000L, operatingExpenses.sumOf { it.amountPaise })
        assertFalse("Pigmy transfer must NOT be in operating expenses", operatingExpenses.any {
it.isSavingsTransfer })
```

```
    }

    @Test
    fun testServiceBillingCalculationAndStockIsolation() {
        val printingJob = BillingDraftItem(
            lineId = UUID.randomUUID().toString(),
            productId = "SERVICE_PRINTING",
            productName = "Printing — A4 B&W",
            unitType = "SERVICE",
            quantity = 50, // 50 pages
            unitPricePaise = 300L, // 3 Rs per page = 150 Rs
            lineType = "SERVICE",
            serviceOption = "A4 B&W",
            unitLabel = "pages"
        )

        assertEquals("Total pieces for service line must be 0", 0, printingJob.totalPieces)
        assertEquals("Line gross paise must be quantity * rate", 15000L, printingJob.lineGrossPaise)
        assertEquals("Line type is SERVICE", "SERVICE", printingJob.lineType)
        assertEquals("Print mode is A4 B&W", "A4 B&W", printingJob.serviceOption)
    }

    @Test
    fun testReceiptPrefixSeriesLogic() {
        val ownerPrefix = "AE-M-"
        val billingPrefix = "AE-B1-"
        val dateStr = "20261101"
        val count = 5
        val seqStr = String.format("%04d", count)

        val ownerReceipt = "$ownerPrefix$dateStr-$seqStr"
        val billingReceipt = "$billingPrefix$dateStr-$seqStr"

        assertTrue(ownerReceipt.startsWith("AE-M-"))
        assertTrue(billingReceipt.startsWith("AE-B1-"))
        assertEquals("AE-B1-20261101-0005", billingReceipt)
    }
}
```

app\src\test\java\com\alieducational\app\service\PigmySchemesAndReceiptComprehensiveTest.kt

```
 1  package com.alieducational.app.service
 2
 3  import com.alieducational.app.data.local.entity.PigmyDepositEntity
 4  import com.alieducational.app.data.local.entity.PigmySchemeEntity
 5  import com.alieducational.app.data.local.entity.SaleEntity
 6  import com.alieducational.app.data.local.entity.SaleLineEntity
 7  import com.alieducational.app.data.local.entity.ShopExpenseEntity
 8  import com.alieducational.app.data.service.ReceiptService
 9  import com.alieducational.app.domain.calculation.CostCalculator
10  import org.junit.Assert.assertEquals
11  import org.junit.Assert.assertFalse
12  import org.junit.Assert.assertNotNull
13  import org.junit.Assert.assertTrue
14  import org.junit.Test
15  import java.io.File
16  import java.io.FileInputStream
17  import java.io.FileOutputStream
18  import java.util.UUID
19  import java.util.zip.ZipEntry
20  import java.util.zip.ZipInputStream
21  import java.util.zip.ZipOutputStream
22
23  class PigmySchemesAndReceiptComprehensiveTest {
24
25      private val receiptService = ReceiptService()
26
27      @Test
28      fun testPigmySchemes_InitialDefaultsAndExpandableSchemesLifecycle() {
29          // 1. Initial 3 schemes (A, B, C)
30          val initialSchemes = mutableListOf(
31              PigmySchemeEntity(
32                  id = "scheme-A",
33                  name = "Pigmy Scheme 1",
34                  usualDailyAmountPaise = 10000L, // ₹100.00
35                  isArchived = false
36              ),
37              PigmySchemeEntity(
38                  id = "scheme-B",
39                  name = "Pigmy Scheme 2",
40                  usualDailyAmountPaise = 20000L, // ₹200.00
41                  isArchived = false
42              ),
43              PigmySchemeEntity(
44                  id = "scheme-C",
45                  name = "Pigmy Scheme 3",
46                  usualDailyAmountPaise = 30000L, // ₹300.00
47                  isArchived = false
48              )
49          )
50          assertEquals("Initial schemes count must be 3", 3, initialSchemes.size)
51
52          // 2. Record synthetic cash deposits: A: ₹100, B: ₹200, C: ₹300
53          val deposits = mutableListOf<PigmyDepositEntity>()
54          val expenses = mutableListOf<ShopExpenseEntity>()
55
56          fun recordDeposit(schemeId: String, amountPaise: Long, paymentSource: String, notes: String?):
    String {
57              val depositId = UUID.randomUUID().toString()
58              var linkedExpenseId: String? = null
59              val scheme = initialSchemes.find { it.id == schemeId }
60
61              if (paymentSource == "CASH") {
62                  linkedExpenseId = UUID.randomUUID().toString()
63                  val expense = ShopExpenseEntity(
64                      id = linkedExpenseId,
65                      category = "SAVINGS_TRANSFER",
66                      amountPaise = amountPaise,
67                      isIncome = false,
68                      notes = "Pigmy Deposit (${scheme?.name}): ${notes ?: ""}".trim(),
69                      isSavingsTransfer = true,
70                      timestamp = System.currentTimeMillis()
71                  )
72                  expenses.add(expense)
73              }
74
75              val deposit = PigmyDepositEntity(
76                  id = depositId,
77                  accountSchemeId = schemeId,
78                  amountPaise = amountPaise,
79                  paymentSource = paymentSource,
80                  notes = notes,
81                  linkedExpenseId = linkedExpenseId,
```

```
                isVoided = false
            )
            deposits.add(deposit)
            return depositId
        }

        val depAId = recordDeposit("scheme-A", 10000L, "CASH", "Deposit A")
        val depBId = recordDeposit("scheme-B", 20000L, "CASH", "Deposit B")
        val depCId = recordDeposit("scheme-C", 30000L, "CASH", "Deposit C")

        // Assert individual and combined totals
        val totalA = deposits.filter { it.accountSchemeId == "scheme-A" && !it.isVoided }.sumOf { it.amountPaise }
        val totalB = deposits.filter { it.accountSchemeId == "scheme-B" && !it.isVoided }.sumOf { it.amountPaise }
        val totalC = deposits.filter { it.accountSchemeId == "scheme-C" && !it.isVoided }.sumOf { it.amountPaise }
        val combinedTotal1 = deposits.filter { !it.isVoided }.sumOf { it.amountPaise }

        assertEquals(10000L, totalA)
        assertEquals(20000L, totalB)
        assertEquals(30000L, totalC)
        assertEquals("Combined recorded deposits must be ₹600.00 (60000 paise)", 60000L, combinedTotal1)

        // Assert exactly 3 active savings transfers created in shop cash
        val activeSavingsTransfers1 = expenses.filter { it.isSavingsTransfer && !it.isVoided }
        assertEquals("Must have exactly 3 active savings transfers", 3, activeSavingsTransfers1.size)
        assertEquals(60000L, activeSavingsTransfers1.sumOf { it.amountPaise })

        // 3. Edit deposit B from ₹200 to ₹250
        val depBIndex = deposits.indexOfFirst { it.id == depBId }
        val oldDepB = deposits[depBIndex]
        deposits[depBIndex] = oldDepB.copy(amountPaise = 25000L)

        // Synchronize linked expense
        val expBIndex = expenses.indexOfFirst { it.id == oldDepB.linkedExpenseId }
        expenses[expBIndex] = expenses[expBIndex].copy(amountPaise = 25000L)

        val combinedTotal2 = deposits.filter { !it.isVoided }.sumOf { it.amountPaise }
        assertEquals("Combined recorded deposits after edit B must be ₹650.00 (65000 paise)", 65000L, combinedTotal2)
        assertEquals("Active savings transfers must remain 3 without duplicates", 3, expenses.filter { it.isSavingsTransfer && !it.isVoided }.size)
        assertEquals(65000L, expenses.filter { it.isSavingsTransfer && !it.isVoided }.sumOf { it.amountPaise })

        // 4. Cancel deposit C
        val depCIndex = deposits.indexOfFirst { it.id == depCId }
        val oldDepC = deposits[depCIndex]
        deposits[depCIndex] = oldDepC.copy(isVoided = true, voidReason = "Customer cancellation")

        // Reverse linked expense
        val expCIndex = expenses.indexOfFirst { it.id == oldDepC.linkedExpenseId }
        expenses[expCIndex] = expenses[expCIndex].copy(isVoided = true, voidReason = "Voided with Pigmy deposit")

        val combinedTotal3 = deposits.filter { !it.isVoided }.sumOf { it.amountPaise }
        assertEquals("Active combined deposits after cancel C must be ₹350.00 (35000 paise)", 35000L, combinedTotal3)
        assertEquals("Active savings transfers must now be 2", 2, expenses.filter { it.isSavingsTransfer && !it.isVoided }.size)
        assertEquals(35000L, expenses.filter { it.isSavingsTransfer && !it.isVoided }.sumOf { it.amountPaise })

        // 5. Direct / Bank funding test: bank payment must NOT create cash outflow
        val depBankId = recordDeposit("scheme-A", 5000L, "BANK", "Direct Bank Payment")
        val combinedTotal4 = deposits.filter { !it.isVoided }.sumOf { it.amountPaise }
        assertEquals("Combined deposits after bank deposit must be ₹400.00 (40000 paise)", 40000L, combinedTotal4)
        assertEquals("Active cash savings transfers must remain 2 (bank deposit does not reduce physical cash)", 2, expenses.filter { it.isSavingsTransfer && !it.isVoided }.size)

        // 6. Profit Neutrality Test:
        // Operating expenses exclude savings transfers
        val operatingExpenses = expenses.filter { !it.isVoided && !it.isIncome && !it.isSavingsTransfer }
        assertEquals("Operating expenses must be 0; savings transfers do NOT reduce product profit", 0, operatingExpenses.size)

        // 7. Expandability test: Add Scheme D and Scheme E
        val schemeD = PigmySchemeEntity(id = "scheme-D", name = "Scheme D", usualDailyAmountPaise = 50000L, isArchived = false)
        val schemeE = PigmySchemeEntity(id = "scheme-E", name = "Scheme E", usualDailyAmountPaise = 100000L, isArchived = false)
        initialSchemes.add(schemeD)
        initialSchemes.add(schemeE)
        assertEquals("Total schemes count must now be 5", 5, initialSchemes.size)

        // Archive Scheme D
```

```
        val dIdx = initialSchemes.indexOfFirst { it.id == "scheme-D" }
        initialSchemes[dIdx] = initialSchemes[dIdx].copy(isArchived = true)
        val activeSchemes = initialSchemes.filter { !it.isArchived }
        assertEquals("Active schemes count should be 4", 4, activeSchemes.size)
        assertFalse("Archived scheme D should not appear in active schemes list", activeSchemes.any { it.id == "scheme-D" })

        // Reactivate Scheme D
        initialSchemes[dIdx] = initialSchemes[dIdx].copy(isArchived = false)
        assertEquals("Active schemes count restored to 5", 5, initialSchemes.filter { !it.isArchived }.size)
    }

    @Test
    fun testReceiptCleanup_AbsenceOfGstNoticeAndRetentionFooter() {
        val sale = SaleEntity(
            id = "sale-test-1",
            receiptNumber = "AE-M-20261101-1036",
            timestamp = 1793517000000L,
            grossTotalPaise = 200L,
            discountPercent = 50.0,
            discountValuePaise = 100L,
            netTotalPaise = 100L,
            amountReceivedPaise = 100L,
            changeGivenPaise = 0L,
            paymentMethod = "CASH",
            customerName = "Walk-in Customer"
        )

        val lines = listOf(
            SaleLineEntity(
                id = "line-1",
                saleId = "sale-test-1",
                productId = null,
                productName = "Photocopying",
                quantityBase = 1,
                unitSellingPricePaise = 200L,
                unitCostSnapshotPaise = null,
                lineGrossPaise = 200L,
                lineDiscountAllocatedPaise = 100L,
                lineNetPaise = 100L,
                lineProfitPaise = null,
                unitLabel = "copies"
            ),
            SaleLineEntity(
                id = "line-2",
                saleId = "sale-test-1",
                productId = "prod-urdu",
                productName = "        ",
                quantityBase = 2,
                unitSellingPricePaise = 1500L,
                unitCostSnapshotPaise = 1000L,
                lineGrossPaise = 3000L,
                lineDiscountAllocatedPaise = 0L,
                lineNetPaise = 3000L,
                lineProfitPaise = 1000L,
                unitLabel = "pc"
            )
        )

        val summaryText = receiptService.generateReceiptSummaryText(sale, lines)

        // 1. Assert "Not GST Registered" is strictly ABSENT
        assertFalse("Summary text must not contain 'Not GST Registered'", summaryText.contains("Not GST Registered", ignoreCase = true))

        // 2. Assert customer retention instruction is strictly ABSENT
        assertFalse("Summary text must not contain 'retain'", summaryText.contains("retain", ignoreCase = true))

        // 3. Assert necessary shop header and footer remain present
        assertTrue("Must contain 'Ali Educational'", summaryText.contains("Ali Educational"))
        assertTrue("Must contain 'Stationery & Educational Supplies'", summaryText.contains("Stationery & Educational Supplies"))
        assertTrue("Must contain receipt number", summaryText.contains("AE-M-20261101-1036"))
        assertTrue("Must preserve Urdu product name", summaryText.contains("        "))
        assertTrue("Must contain 'Thank you for visiting Ali Educational!'", summaryText.contains("Thank you for visiting Ali Educational!"))
    }

    @Test
    fun testBackupAndRestore_WithFiveSchemesAndLinks() {
        val tempDir = File(System.getProperty("java.io.tmpdir"), "pigmy_backup_test_${System.currentTimeMillis()}")
        tempDir.mkdirs()

        try {
            val dbFile = File(tempDir, "ali_educational.db")
            // Create simulated SQLite format 3 database header
```

```
            FileOutputStream(dbFile).use { out ->
                out.write("SQLite format 3\u0000".toByteArray(Charsets.US_ASCII))
                out.write(ByteArray(1024)) // padding
            }

            val zipFile = File(tempDir, "ali_backup_test.zip")
            ZipOutputStream(FileOutputStream(zipFile)).use { zipOut ->
                // Add DB
                zipOut.putNextEntry(ZipEntry("ali_educational.db"))
                FileInputStream(dbFile).use { it.copyTo(zipOut) }
                zipOut.closeEntry()

                // Add Manifest
                val manifest = """
                    {
                        "app": "Ali Educational",
                        "schemaVersion": 6,
                        "schemesCount": 5,
                        "timestamp": ${System.currentTimeMillis()}
                    }
                """.trimIndent()
                zipOut.putNextEntry(ZipEntry("manifest.json"))
                zipOut.write(manifest.toByteArray(Charsets.UTF_8))
                zipOut.closeEntry()
            }

            assertTrue("Zip backup must exist", zipFile.exists())
            assertTrue("Zip backup size must be non-zero", zipFile.length() > 50)

            // Simulate restore into a disposable database
            val restoredDbFile = File(tempDir, "restored_ali_educational.db")
            var restoredManifest: String? = null

            ZipInputStream(FileInputStream(zipFile)).use { zipIn ->
                var entry: ZipEntry?
                while (zipIn.nextEntry.also { entry = it } != null) {
                    if (entry!!.name == "ali_educational.db") {
                        FileOutputStream(restoredDbFile).use { out ->
                            zipIn.copyTo(out)
                        }
                    } else if (entry!!.name == "manifest.json") {
                        restoredManifest = zipIn.bufferedReader().readText()
                    }
                    zipIn.closeEntry()
                }
            }

            assertTrue("Restored database must exist", restoredDbFile.exists())
            assertEquals("Restored database size must match original", dbFile.length(),
restoredDbFile.length())
            assertNotNull("Manifest must be restored", restoredManifest)
            assertTrue("Restored manifest must have schemaVersion 6",
restoredManifest!!.contains("\"schemaVersion\": 6"))
        } finally {
            tempDir.deleteRecursively()
        }
    }
}
```

app\src\test\java\com\alieducational\app\service\PigmyWithdrawalsAndCategoriesTest.kt

```
package com.alieducational.app.service

import androidx.sqlite.db.SupportSQLiteDatabase
import com.alieducational.app.data.local.AppDatabase
import com.alieducational.app.data.local.entity.CashCategoryEntity
import com.alieducational.app.data.local.entity.PigmyDepositEntity
import com.alieducational.app.data.local.entity.PigmySchemeEntity
import com.alieducational.app.data.local.entity.PigmyWithdrawalEntity
import com.alieducational.app.data.local.entity.ShopExpenseEntity
import com.alieducational.app.data.service.CashTrackerExcelService
import org.apache.poi.xssf.usermodel.XSSFWorkbook
import org.junit.Assert.assertEquals
import org.junit.Assert.assertFalse
import org.junit.Assert.assertNotNull
import org.junit.Assert.assertNull
import org.junit.Assert.assertTrue
import org.junit.Test
import java.io.File
import java.io.FileInputStream
import java.io.FileOutputStream
import java.lang.reflect.InvocationHandler
import java.lang.reflect.Method
import java.lang.reflect.Proxy
import java.util.UUID
import java.util.zip.ZipEntry
import java.util.zip.ZipInputStream
import java.util.zip.ZipOutputStream

class PigmyWithdrawalsAndCategoriesTest {

    @Test
    fun testMigration6To7_StatementsRecorded() {
        val statements = mutableListOf<String>()

        val handler = InvocationHandler { _, method: Method, args: Array<out Any>? ->
            if (method.name == "execSQL" && args != null && args.isNotEmpty()) {
                statements.add(args[0] as String)
            }
            null
        }

        val proxyDb = Proxy.newProxyInstance(
            SupportSQLiteDatabase::class.java.classLoader,
            arrayOf(SupportSQLiteDatabase::class.java),
            handler
        ) as SupportSQLiteDatabase

        AppDatabase.MIGRATION_6_7.migrate(proxyDb)

        assertTrue("Must create cash_categories table",
            statements.any { it.contains("CREATE TABLE IF NOT EXISTS `cash_categories`") })
        assertTrue("Must populate default cash categories",
            statements.any { it.contains("INSERT OR IGNORE INTO `cash_categories`") })
        assertTrue("Must alter shop_expenses to add categoryId",
            statements.any { it.contains("ALTER TABLE shop_expenses ADD COLUMN categoryId") })
        assertTrue("Must create pigmy_withdrawals table",
            statements.any { it.contains("CREATE TABLE IF NOT EXISTS `pigmy_withdrawals`") })
        assertTrue("Must alter pigmy_schemes to add openingBalancePaise",
            statements.any { it.contains("ALTER TABLE pigmy_schemes ADD COLUMN openingBalancePaise") })
        assertTrue("Must alter pigmy_schemes to add openingBalanceDate",
            statements.any { it.contains("ALTER TABLE pigmy_schemes ADD COLUMN openingBalanceDate") })
    }

    @Test
    fun testPigmyTrackedBalanceCalculations_WithWithdrawalsAndOpeningBalance() {
        val schemeId = "scheme_alpha"
        val openingBalance = 500000L // ₹5,000.00 confirmed starting balance

        val deposits = listOf(
            PigmyDepositEntity(
                id = "dep_1",
                accountSchemeId = schemeId,
                amountPaise = 100000L, // ₹1,000.00
                paymentSource = "CASH",
                linkedExpenseId = "exp_out_1",
                isVoided = false
            ),
            PigmyDepositEntity(
                id = "dep_2",
                accountSchemeId = schemeId,
                amountPaise = 50000L, // ₹500.00
                paymentSource = "BANK",
```

```
                linkedExpenseId = null,
                isVoided = false
            ),
            PigmyDepositEntity(
                id = "dep_voided",
                accountSchemeId = schemeId,
                amountPaise = 20000L, // ₹200.00 (Cancelled)
                paymentSource = "CASH",
                linkedExpenseId = "exp_out_voided",
                isVoided = true
            )
        )

        val withdrawals = listOf(
            PigmyWithdrawalEntity(
                id = "with_1",
                accountSchemeId = schemeId,
                amountPaise = 200000L, // ₹2,000.00 to Shop Cash
                destination = "CASH",
                linkedExpenseId = "exp_in_1",
                isVoided = false
            ),
            PigmyWithdrawalEntity(
                id = "with_2",
                accountSchemeId = schemeId,
                amountPaise = 100000L, // ₹1,000.00 to Bank Account
                destination = "BANK",
                linkedExpenseId = null,
                isVoided = false
            ),
            PigmyWithdrawalEntity(
                id = "with_voided",
                accountSchemeId = schemeId,
                amountPaise = 50000L, // ₹500.00 (Cancelled)
                destination = "CASH",
                linkedExpenseId = "exp_in_voided",
                isVoided = true
            )
        )

        // 1. Check active totals
        val activeDepositsTotal = deposits.filter { !it.isVoided }.sumOf { it.amountPaise }
        val activeWithdrawalsTotal = withdrawals.filter { !it.isVoided }.sumOf { it.amountPaise }
        assertEquals(150000L, activeDepositsTotal) // ₹1,500.00
        assertEquals(300000L, activeWithdrawalsTotal) // ₹3,000.00

        // 2. Tracked Balance = Opening Balance + Active Deposits - Active Withdrawals
        val trackedBalance = openingBalance + activeDepositsTotal - activeWithdrawalsTotal
        assertEquals(350000L, trackedBalance) // ₹5,000 + ₹1,500 - ₹3,000 = ₹3,500.00

        // 3. Validation: Withdrawal of ₹4,000 exceeds available ₹3,500
        val requestedWithdrawal = 400000L
        val exceedsBalance = requestedWithdrawal > trackedBalance
        assertTrue("Requested withdrawal above tracked funds must be flagged", exceedsBalance)
    }

    @Test
    fun testCashInSavingsTransfer_ClassifiedAsTransferNotSalesOrProfit() {
        val withdrawal = PigmyWithdrawalEntity(
            id = "with_101",
            accountSchemeId = "DEFAULT_PIGMY",
            amountPaise = 300000L, // ₹3,000.00
            destination = "CASH",
            linkedExpenseId = "exp_in_101"
        )

        // Linked Cash In entry generated into shop_expenses
        val linkedCashIn = ShopExpenseEntity(
            id = "exp_in_101",
            category = "Pigmy",
            amountPaise = withdrawal.amountPaise,
            isIncome = true, // Cash In
            notes = "Pigmy withdrawal — Pigmy Scheme 1",
            isSavingsTransfer = true, // Must be marked as savings transfer!
            categoryId = "cat_pigmy"
        )

        // Ensure isSavingsTransfer is true
        assertTrue(linkedCashIn.isSavingsTransfer)
        assertTrue(linkedCashIn.isIncome)
        assertEquals("Pigmy", linkedCashIn.category)

        // When calculating operating profit or sales, isSavingsTransfer entries must be excluded!
        val operatingSales = listOf<ShopExpenseEntity>().filter { !it.isSavingsTransfer && it.isIncome }
        assertEquals(0, operatingSales.size)
    }

    @Test
```

```
    fun testExcelImportAndExport_WithCategoryAndAttachmentColumns() {
        val service = CashTrackerExcelService()
        val testWorkbook = XSSFWorkbook()
        val sheet = testWorkbook.createSheet("Cash In Out")

        // Create headers with Category and Attachment
        val headerRow = sheet.createRow(0)
        headerRow.createCell(0).setCellValue("Date")
        headerRow.createCell(1).setCellValue("Category")
        headerRow.createCell(2).setCellValue("Description")
        headerRow.createCell(3).setCellValue("Payment Mode")
        headerRow.createCell(4).setCellValue("Attachment")
        headerRow.createCell(5).setCellValue("Cash In (Income)")
        headerRow.createCell(6).setCellValue("Cash Out (Expense)")

        // Row 1: Categorized expense with attachment
        val r1 = sheet.createRow(1)
        r1.createCell(0).setCellValue("01-Nov-2026")
        r1.createCell(1).setCellValue("Rent")
        r1.createCell(2).setCellValue("Shop Rent Nov 2026")
        r1.createCell(3).setCellValue("BANK")
        r1.createCell(4).setCellValue("Yes")
        r1.createCell(5).setCellValue(0.0)
        r1.createCell(6).setCellValue(2000.0)

        // Row 2: Pigmy withdrawal savings transfer
        val r2 = sheet.createRow(2)
        r2.createCell(0).setCellValue("01-Nov-2026")
        r2.createCell(1).setCellValue("Pigmy")
        r2.createCell(2).setCellValue("Pigmy withdrawal — Scheme 1")
        r2.createCell(3).setCellValue("CASH")
        r2.createCell(4).setCellValue("No")
        r2.createCell(5).setCellValue(3000.0)
        r2.createCell(6).setCellValue(0.0)

        val preview = service.parseWorkbook(testWorkbook, "categorized_test.xlsx")
        assertNull(preview.errorMessage)
        assertEquals(2, preview.items.size)

        val rentItem = preview.items[0]
        assertEquals("Rent", rentItem.category)
        assertFalse(rentItem.isIncome)
        assertEquals(200000L, rentItem.amountPaise)

        val pigmyItem = preview.items[1]
        assertEquals("Pigmy", pigmyItem.category)
        assertTrue(pigmyItem.isIncome)
        assertEquals(300000L, pigmyItem.amountPaise)

        testWorkbook.close()
    }

    @Test
    fun testBackupAndRestore_ZipsAndUnpacksAttachmentsDurableStorage() {
        val tempDir = File(System.getProperty("java.io.tmpdir"), "test_backup_" + UUID.randomUUID())
        tempDir.mkdirs()

        val filesDir = File(tempDir, "files")
        val attachmentsDir = File(filesDir, "attachments")
        attachmentsDir.mkdirs()

        // Create a synthetic attachment photo
        val samplePhoto = File(attachmentsDir, "attachment_test_123.jpg")
        samplePhoto.writeBytes(byteArrayOf(0xFF.toByte(), 0xD8.toByte(), 0xFF.toByte(), 0xE0.toByte(), 0x12,
0x34))

        // Create synthetic backup zip containing db, manifest, and attachment
        val backupZip = File(tempDir, "backup_with_photo.zip")
        ZipOutputStream(FileOutputStream(backupZip)).use { zipOut ->
            // SQLite dummy header
            zipOut.putNextEntry(ZipEntry("ali_educational.db"))
            zipOut.write("SQLite format 3\u0000".toByteArray(Charsets.US_ASCII))
            zipOut.closeEntry()

            // Manifest
            zipOut.putNextEntry(ZipEntry("manifest.json"))
            zipOut.write("{\"app\":\"Ali Educational\"}".toByteArray(Charsets.UTF_8))
            zipOut.closeEntry()

            // Attachment entry
            zipOut.putNextEntry(ZipEntry("attachments/${samplePhoto.name}"))
            FileInputStream(samplePhoto).use { it.copyTo(zipOut) }
            zipOut.closeEntry()
        }

        assertTrue("Backup ZIP must exist and be non-empty", backupZip.exists() && backupZip.length() > 0)

        // Restore into a separate target directory to verify independence
```

```
        val restoredTargetDir = File(tempDir, "restored_installation")
        val restoredAttachmentsDir = File(restoredTargetDir, "attachments")

        ZipInputStream(FileInputStream(backupZip)).use { zipIn ->
            var entry: ZipEntry?
            while (zipIn.nextEntry.also { entry = it } != null) {
                if (entry!!.name.startsWith("attachments/") && !entry!!.isDirectory) {
                    restoredAttachmentsDir.mkdirs()
                    val destFile = File(restoredAttachmentsDir, File(entry!!.name).name)
                    FileOutputStream(destFile).use { out -> zipIn.copyTo(out) }
                }
                zipIn.closeEntry()
            }
        }

        val restoredPhoto = File(restoredAttachmentsDir, "attachment_test_123.jpg")
        assertTrue("Restored photo must exist in target installation", restoredPhoto.exists())
        assertEquals(samplePhoto.length(), restoredPhoto.length())

        // Cleanup
        tempDir.deleteRecursively()
    }

    @Test
    fun testTwoWaySync_CashOutPigmyDeposit_And_CashInPigmyWithdrawal() {
        val schemeId = "scheme_1"
        val schemeName = "Pigmy Scheme 1"

        // 1. Simulating Cash Out recorded under Pigmy Scheme 1
        val cashOutExpenseId = UUID.randomUUID().toString()
        val depositId = UUID.randomUUID().toString()
        val depositAmountPaise = 50000L // ₹500.00

        val linkedCashOutExpense = ShopExpenseEntity(
            id = cashOutExpenseId,
            category = schemeName,
            categoryId = schemeId,
            amountPaise = depositAmountPaise,
            isIncome = false,
            isSavingsTransfer = true,
            notes = "Pigmy Deposit ($schemeName)"
        )
        val linkedDeposit = PigmyDepositEntity(
            id = depositId,
            accountSchemeId = schemeId,
            amountPaise = depositAmountPaise,
            paymentSource = "CASH",
            linkedExpenseId = cashOutExpenseId,
            isVoided = false
        )

        assertEquals(schemeName, linkedCashOutExpense.category)
        assertEquals(schemeId, linkedCashOutExpense.categoryId)
        assertFalse(linkedCashOutExpense.isIncome)
        assertTrue(linkedCashOutExpense.isSavingsTransfer)
        assertEquals(linkedCashOutExpense.id, linkedDeposit.linkedExpenseId)

        // 2. Simulating Cash In recorded under Pigmy Scheme 1
        val cashInExpenseId = UUID.randomUUID().toString()
        val withdrawalId = UUID.randomUUID().toString()
        val withdrawalAmountPaise = 30000L // ₹300.00

        val linkedCashInExpense = ShopExpenseEntity(
            id = cashInExpenseId,
            category = schemeName,
            categoryId = schemeId,
            amountPaise = withdrawalAmountPaise,
            isIncome = true,
            isSavingsTransfer = true,
            notes = "Pigmy withdrawal — $schemeName"
        )
        val linkedWithdrawal = PigmyWithdrawalEntity(
            id = withdrawalId,
            accountSchemeId = schemeId,
            amountPaise = withdrawalAmountPaise,
            destination = "CASH",
            linkedExpenseId = cashInExpenseId,
            isVoided = false
        )

        assertEquals(schemeName, linkedCashInExpense.category)
        assertEquals(schemeId, linkedCashInExpense.categoryId)
        assertTrue(linkedCashInExpense.isIncome)
        assertTrue(linkedCashInExpense.isSavingsTransfer)
        assertEquals(linkedCashInExpense.id, linkedWithdrawal.linkedExpenseId)

        // 3. Two-way balance tracking
        val netSchemeSavings = linkedDeposit.amountPaise - linkedWithdrawal.amountPaise
```

```
        assertEquals(20000L, netSchemeSavings) // ₹500 - ₹300 = ₹200 net

        // 4. Two-way voiding verification
        val voidedExpense = linkedCashOutExpense.copy(isVoided = true, voidReason = "Voided by user")
        val voidedDeposit = linkedDeposit.copy(isVoided = true, voidReason = "Voided by user")
        assertTrue(voidedExpense.isVoided)
        assertTrue(voidedDeposit.isVoided)
    }

    @Test
    fun testCategoryDeletion_SafelyRemovesCategory_PreservingPastExpenses() {
        // Given existing categories
        val cat1 = CashCategoryEntity(id = "cat_tea", name = "Tea & Snacks", isSystem = false, isArchived =
    false)
        val cat2 = CashCategoryEntity(id = "cat_rent", name = "Rent", isSystem = false, isArchived = false)
        val categories = mutableListOf(cat1, cat2)

        // Past expense recorded under Tea & Snacks
        val pastExpense = ShopExpenseEntity(
            id = "exp_tea_1",
            category = "Tea & Snacks",
            categoryId = "cat_tea",
            amountPaise = 15000L,
            isIncome = false,
            notes = "CASH (Tea & Snacks)"
        )

        // When user removes "Tea & Snacks" category
        categories.removeIf { it.id == "cat_tea" }

        // Category is no longer in active selectable categories
        assertEquals(1, categories.size)
        assertEquals("cat_rent", categories[0].id)
        assertFalse(categories.any { it.name == "Tea & Snacks" })

        // Past expense record remains completely intact with original category name
        assertEquals("Tea & Snacks", pastExpense.category)
        assertEquals("cat_tea", pastExpense.categoryId)
        assertEquals(15000L, pastExpense.amountPaise)
    }

    @Test
    fun testOptionalDescription_DefaultsToCategoryNameWhenBlank() {
        val selectedCategory = "Office Rent"
        val userEnteredDescription = "   " // blank

        val effectiveDescription = if (userEnteredDescription.isNotBlank()) {
            userEnteredDescription.trim()
        } else {
            selectedCategory
        }

        assertEquals("Office Rent", effectiveDescription)

        val paymentNote = buildString {
            append("CASH")
            if (effectiveDescription != selectedCategory || userEnteredDescription.isNotBlank()) {
                append(" ($effectiveDescription)")
            } else {
                append(" ($selectedCategory)")
            }
        }

        assertEquals("CASH (Office Rent)", paymentNote)
    }
}
```

app\src\test\java\com\alieducational\app\service\ReceiptServiceTest.kt

```
package com.alieducational.app.service

import com.alieducational.app.data.local.entity.SaleEntity
import com.alieducational.app.data.local.entity.SaleLineEntity
import com.alieducational.app.data.service.ReceiptService
import org.junit.Assert.assertFalse
import org.junit.Assert.assertTrue
import org.junit.Test

class ReceiptServiceTest {

    @Test
    fun testGenerateReceiptSummaryText_SuppliedPhotocopyingExample() {
        // Supplied Photocopying example: 1 copy @ ₹2.00, 50% discount = ₹1.00
        val sale = SaleEntity(
            receiptNumber = "AE-M-20261101-8261",
            customerName = "Walk-in Customer",
            grossTotalPaise = 200L, // ₹2.00
            discountType = "PERCENT",
            discountValuePaise = 100L, // ₹1.00
            discountPercent = 50.0,
            netTotalPaise = 100L, // ₹1.00
            amountReceivedPaise = 100L, // ₹1.00
            changeGivenPaise = 0L,
            remainingCreditPaise = 0L,
            paymentMethod = "CASH",
            timestamp = 1762002120000L
        )

        val lines = listOf(
            SaleLineEntity(
                saleId = sale.id,
                productId = null,
                productName = "Photocopying",
                quantityBase = 1,
                unitSellingPricePaise = 200L,
                unitCostSnapshotPaise = null,
                lineGrossPaise = 200L,
                lineDiscountAllocatedPaise = 100L,
                lineNetPaise = 100L,
                lineProfitPaise = null,
                lineType = "SERVICE",
                serviceId = "svc_copy",
                serviceName = "Photocopying",
                unitLabel = "copies"
            )
        )

        val service = ReceiptService()
        val text = service.generateReceiptSummaryText(sale, lines)

        // Verify discount shown ONCE:
        // Item row: 1 copy @ ₹2.00 = ₹2.00 (gross)
        assertTrue(text.contains("Photocopying: 1 copy @ ₹2.00 = ₹2.00"))
        assertTrue(text.contains("Subtotal: ₹2.00"))
        assertTrue(text.contains("Discount (50%): -₹1.00"))
        assertTrue(text.contains("TOTAL PAYABLE: ₹1.00"))
        assertTrue(text.contains("Amount paid: ₹1.00"))
        assertTrue(text.contains("Balance due: ₹0.00"))
        assertTrue(text.contains("Status: PAID"))

        // Assert strictly absent
        assertFalse(text.contains("Not GST registered"))
        assertFalse(text.contains("Please retain"))
    }

    @Test
    fun testGenerateReceiptSummaryText_PartialPaymentCreditSale() {
        val sale = SaleEntity(
            receiptNumber = "AE-M-20261101-9002",
            customerName = "Rashid Khan",
            grossTotalPaise = 25000L, // ₹250.00
            discountType = "NONE",
            discountValuePaise = 0L,
            discountPercent = 0.0,
            netTotalPaise = 25000L, // ₹250.00
            amountReceivedPaise = 10000L, // ₹100.00
            changeGivenPaise = 0L,
            remainingCreditPaise = 15000L, // ₹150.00 balance due
            paymentMethod = "CREDIT",
            timestamp = 1762002120000L
        )
```

```
83
84          val lines = listOf(
85              SaleLineEntity(
86                  saleId = sale.id,
87                  productId = "prod_pen",
88                  productName = "Parker Vector Pen (   )",
89                  quantityBase = 1,
90                  isPack = true,
91                  unitSellingPricePaise = 25000L,
92                  unitCostSnapshotPaise = 18000L,
93                  lineGrossPaise = 25000L,
94                  lineDiscountAllocatedPaise = 0L,
95                  lineNetPaise = 25000L,
96                  lineProfitPaise = 7000L
97              )
98          )
99
100         val service = ReceiptService()
101         val text = service.generateReceiptSummaryText(sale, lines)
102
103         assertTrue(text.contains("Receipt No: AE-M-20261101-9002"))
104         assertTrue(text.contains("Customer: Rashid Khan"))
105         assertTrue(text.contains("Payment: Credit"))
106         assertTrue(text.contains("Parker Vector Pen (   )"))
107         assertTrue(text.contains("Subtotal: ₹250.00"))
108         assertTrue(text.contains("TOTAL PAYABLE: ₹250.00"))
109         assertTrue(text.contains("Amount paid: ₹100.00"))
110         assertTrue(text.contains("Balance due: ₹150.00"))
111         assertTrue(text.contains("Status: PARTIAL"))
112     }
113
114     @Test
115     fun testGenerateReceiptSummaryText_CashChangeSale() {
116         val sale = SaleEntity(
117             receiptNumber = "AE-M-20261101-9003",
118             customerName = "Zaid Ali",
119             grossTotalPaise = 17000L, // ₹170.00
120             discountType = "NONE",
121             discountValuePaise = 0L,
122             discountPercent = 0.0,
123             netTotalPaise = 17000L, // ₹170.00
124             amountReceivedPaise = 20000L, // ₹200.00 tendered
125             changeGivenPaise = 3000L, // ₹30.00 change
126             remainingCreditPaise = 0L,
127             paymentMethod = "CASH",
128             timestamp = 1762002120000L
129         )
130
131         val lines = listOf(
132             SaleLineEntity(
133                 saleId = sale.id,
134                 productId = "prod_reg",
135                 productName = "Classmate Register 200 Pgs",
136                 quantityBase = 2,
137                 isPack = false,
138                 unitSellingPricePaise = 8500L,
139                 unitCostSnapshotPaise = 6000L,
140                 lineGrossPaise = 17000L,
141                 lineDiscountAllocatedPaise = 0L,
142                 lineNetPaise = 17000L,
143                 lineProfitPaise = 5000L
144             )
145         )
146
147         val service = ReceiptService()
148         val text = service.generateReceiptSummaryText(sale, lines)
149
150         assertTrue(text.contains("Subtotal: ₹170.00"))
151         assertTrue(text.contains("TOTAL PAYABLE: ₹170.00"))
152         assertTrue(text.contains("Amount paid: ₹200.00"))
153         assertTrue(text.contains("Change: ₹30.00"))
154         assertTrue(text.contains("Balance due: ₹0.00"))
155         assertTrue(text.contains("Status: PAID"))
156     }
157 }
```

app\src\test\java\com\alieducational\app\widget\AliSlimBarWidgetTest.kt

```kotlin
package com.alieducational.app.widget

import android.net.Uri
import org.junit.Assert.assertEquals
import org.junit.Assert.assertNotNull
import org.junit.Assert.assertTrue
import org.junit.Test

class AliSlimBarWidgetTest {

    @Test
    fun testWidgetActionConstants_Section02() {
        // Required exact labels & action identities
        assertEquals("CASH_IN", AliSlimBarWidgetProvider.ACTION_CASH_IN)
        assertEquals("CASH_OUT", AliSlimBarWidgetProvider.ACTION_CASH_OUT)
        assertEquals("NEW_BILL", AliSlimBarWidgetProvider.ACTION_NEW_BILL)
        assertEquals("ADD_STOCK", AliSlimBarWidgetProvider.ACTION_ADD_STOCK)
        assertEquals("HOME", AliSlimBarWidgetProvider.ACTION_HOME)
        assertEquals("com.alieducational.app.action.WIDGET_CLICK",
AliSlimBarWidgetProvider.ACTION_WIDGET_CLICK)
        assertEquals("extra_widget_action", AliSlimBarWidgetProvider.EXTRA_WIDGET_ACTION)
    }

    @Test
    fun testDistinctUrisPreventIntentCoalescing_Section04() {
        val actions = listOf(
            AliSlimBarWidgetProvider.ACTION_CASH_IN,
            AliSlimBarWidgetProvider.ACTION_CASH_OUT,
            AliSlimBarWidgetProvider.ACTION_NEW_BILL,
            AliSlimBarWidgetProvider.ACTION_ADD_STOCK,
            AliSlimBarWidgetProvider.ACTION_HOME
        )

        val uris = actions.map { "aliedu://widget/${it.lowercase()}" }.toSet()
        // Exactly 5 unique URIs so Android PendingIntent never collapses or crosses intents
        assertEquals(5, uris.size)
        assertTrue(uris.contains("aliedu://widget/cash_in"))
        assertTrue(uris.contains("aliedu://widget/cash_out"))
        assertTrue(uris.contains("aliedu://widget/new_bill"))
        assertTrue(uris.contains("aliedu://widget/add_stock"))
        assertTrue(uris.contains("aliedu://widget/home"))
    }

    @Test
    fun testWidgetActionRoutingSafety_Section03() {
        // Tapping shortcuts must never auto-commit transactions or alter balances
        val simulatedDbBalancePaise = 1000000L // ₹10,000.00
        val simulatedStockQty = 100

        val widgetTappedActions = listOf(
            AliSlimBarWidgetProvider.ACTION_CASH_IN,
            AliSlimBarWidgetProvider.ACTION_CASH_OUT,
            AliSlimBarWidgetProvider.ACTION_NEW_BILL,
            AliSlimBarWidgetProvider.ACTION_ADD_STOCK
        )

        // Simulating repeated widget taps without form submission
        for (action in widgetTappedActions) {
            assertNotNull(action)
            // Balance and stock must remain completely untouched
            assertEquals(1000000L, simulatedDbBalancePaise)
            assertEquals(100, simulatedStockQty)
        }
    }
}
```

app\build.gradle.kts

```
import java.io.FileInputStream
import java.util.Properties

plugins {
    alias(libs.plugins.android.application)
    alias(libs.plugins.kotlin.android)
    alias(libs.plugins.kotlin.compose)
    alias(libs.plugins.ksp)
    alias(libs.plugins.hilt)
    alias(libs.plugins.room)
}

android {
    namespace = "com.alieducational.app"
    compileSdk = 36

    defaultConfig {
        applicationId = "com.alieducational.app"
        minSdk = 26
        targetSdk = 36
        versionCode = 12
        versionName = "1.0.12"

        testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"
    }

    val keystorePropsFile = rootProject.file("keystore.properties")
    val keystoreProperties = Properties()
    if (keystorePropsFile.exists()) {
        keystoreProperties.load(FileInputStream(keystorePropsFile))
    }

    signingConfigs {
        create("release") {
            if (keystorePropsFile.exists()) {
                val storeFilePath = keystoreProperties.getProperty("storeFile")
                storeFile = if (storeFilePath.startsWith("../"))
rootProject.file(storeFilePath.removePrefix("../")) else rootProject.file(storeFilePath)
                storePassword = keystoreProperties.getProperty("storePassword")
                keyAlias = keystoreProperties.getProperty("keyAlias")
                keyPassword = keystoreProperties.getProperty("keyPassword")
                enableV1Signing = true
                enableV2Signing = true
                enableV3Signing = true
                enableV4Signing = true
            }
        }
    }

    buildTypes {
        release {
            isMinifyEnabled = true
            isShrinkResources = true
            signingConfig = signingConfigs.getByName("release")
            proguardFiles(
                getDefaultProguardFile("proguard-android-optimize.txt"),
                "proguard-rules.pro"
            )
        }
        debug {
            isMinifyEnabled = false
            applicationIdSuffix = ".debug"
            versionNameSuffix = "-debug"
        }
    }

    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_17
        targetCompatibility = JavaVersion.VERSION_17
    }

    kotlinOptions {
        jvmTarget = "17"
    }

    buildFeatures {
        compose = true
        buildConfig = true
    }

    // Room schema directory for migration testing
    room {
        schemaDirectory("$projectDir/schemas")
    }

```

```
    packaging {
        resources {
            // Exclude duplicate files from Apache POI and other libs
            excludes += "/META-INF/{AL2.0,LGPL2.1}"
            excludes += "/META-INF/DEPENDENCIES"
            excludes += "/META-INF/LICENSE"
            excludes += "/META-INF/LICENSE.txt"
            excludes += "/META-INF/license.txt"
            excludes += "/META-INF/NOTICE"
            excludes += "/META-INF/NOTICE.txt"
            excludes += "/META-INF/notice.txt"
            excludes += "/META-INF/ASL2.0"
            excludes += "/META-INF/*.kotlin_module"
        }
    }
}

dependencies {
    // ── AndroidX Core ──────────────────────────────────────
    implementation(libs.androidx.core.ktx)
    implementation(libs.androidx.appcompat)
    implementation(libs.androidx.activity.compose)
    implementation(libs.androidx.lifecycle.runtime.ktx)
    implementation(libs.androidx.lifecycle.viewmodel.compose)
    implementation(libs.androidx.lifecycle.runtime.compose)
    implementation(libs.androidx.datastore.preferences)

    // ── Jetpack Compose (BOM-managed) ──────────────────────
    implementation(platform(libs.compose.bom))
    implementation(libs.compose.ui)
    implementation(libs.compose.ui.graphics)
    implementation(libs.compose.ui.tooling.preview)
    implementation(libs.compose.material3)
    implementation(libs.compose.material.icons.extended)
    debugImplementation(libs.compose.ui.tooling)
    debugImplementation(libs.compose.ui.test.manifest)

    // ── Navigation ─────────────────────────────────────────
    implementation(libs.androidx.navigation.compose)

    // ── Biometric ──────────────────────────────────────────
    implementation(libs.androidx.biometric)

    // ── Room ───────────────────────────────────────────────
    implementation(libs.room.runtime)
    implementation(libs.room.ktx)
    ksp(libs.room.compiler)

    // ── Hilt ───────────────────────────────────────────────
    implementation(libs.hilt.android)
    ksp(libs.hilt.android.compiler)
    implementation(libs.hilt.navigation.compose)

    // ── Apache POI (Excel import/export) ───────────────────
    implementation(libs.poi)
    implementation(libs.poi.ooxml)

    // ── Accompanist ────────────────────────────────────────
    implementation(libs.accompanist.systemuicontroller)

    // ── CameraX & Barcode (100% Offline) ───────────────────
    implementation(libs.androidx.camera.core)
    implementation(libs.androidx.camera.camera2)
    implementation(libs.androidx.camera.lifecycle)
    implementation(libs.androidx.camera.view)
    implementation(libs.zxing.core)

    // ── Testing ────────────────────────────────────────────
    testImplementation(libs.junit)
    androidTestImplementation(libs.androidx.junit)
    androidTestImplementation(libs.androidx.espresso.core)
    androidTestImplementation(platform(libs.compose.bom))
    androidTestImplementation(libs.compose.ui.test.junit4)
}
```

build.gradle.kts

```
1  // Top-level build file for Ali Educational Inventory App
2  plugins {
3      alias(libs.plugins.android.application) apply false
4      alias(libs.plugins.kotlin.android) apply false
5      alias(libs.plugins.kotlin.compose) apply false
6      alias(libs.plugins.ksp) apply false
7      alias(libs.plugins.hilt) apply false
8      alias(libs.plugins.room) apply false
9  }
```

settings.gradle.kts

```
pluginManagement {
    repositories {
        google()
        mavenCentral()
        gradlePluginPortal()
    }
}

dependencyResolutionManagement {
    repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
    repositories {
        google()
        mavenCentral()
    }
}

rootProject.name = "AliEducational"
include(":app")
```

gradle\libs.versions.toml

```
[versions]
# Core
agp = "8.7.3"
kotlin = "2.0.21"
ksp = "2.0.21-1.0.28"

# AndroidX
coreKtx = "1.15.0"
appcompat = "1.7.0"
activityCompose = "1.9.3"
lifecycleRuntimeKtx = "2.8.7"
lifecycleViewmodelCompose = "2.8.7"
navigationCompose = "2.8.5"
biometric = "1.2.0-alpha05"
datastorePreferences = "1.1.1"

# Compose
composeBom = "2025.01.01"

# Room
room = "2.6.1"

# Hilt
hilt = "2.51.1"
hiltNavigationCompose = "1.2.0"

# Apache POI (Excel import)
poi = "5.2.5"

# Accompanist
accompanistSystemUiController = "0.34.0"

# Camera & Barcode
camerax = "1.4.1"
zxing = "3.5.3"

# Testing
junit = "4.13.2"
junitExt = "1.2.1"
espressoCore = "3.6.1"

[libraries]
# AndroidX Core
androidx-core-ktx = { group = "androidx.core", name = "core-ktx", version.ref = "coreKtx" }
androidx-appcompat = { group = "androidx.appcompat", name = "appcompat", version.ref = "appcompat" }
androidx-activity-compose = { group = "androidx.activity", name = "activity-compose", version.ref = "activityCompose" }
androidx-lifecycle-runtime-ktx = { group = "androidx.lifecycle", name = "lifecycle-runtime-ktx", version.ref = "lifecycleRuntimeKtx" }
androidx-lifecycle-viewmodel-compose = { group = "androidx.lifecycle", name = "lifecycle-viewmodel-compose", version.ref = "lifecycleViewmodelCompose" }
androidx-lifecycle-runtime-compose = { group = "androidx.lifecycle", name = "lifecycle-runtime-compose", version.ref = "lifecycleRuntimeKtx" }
androidx-navigation-compose = { group = "androidx.navigation", name = "navigation-compose", version.ref = "navigationCompose" }
androidx-biometric = { group = "androidx.biometric", name = "biometric", version.ref = "biometric" }
androidx-datastore-preferences = { group = "androidx.datastore", name = "datastore-preferences", version.ref = "datastorePreferences" }

# Compose BOM
compose-bom = { group = "androidx.compose", name = "compose-bom", version.ref = "composeBom" }
compose-ui = { group = "androidx.compose.ui", name = "ui" }
compose-ui-graphics = { group = "androidx.compose.ui", name = "ui-graphics" }
compose-ui-tooling = { group = "androidx.compose.ui", name = "ui-tooling" }
compose-ui-tooling-preview = { group = "androidx.compose.ui", name = "ui-tooling-preview" }
compose-ui-test-manifest = { group = "androidx.compose.ui", name = "ui-test-manifest" }
compose-ui-test-junit4 = { group = "androidx.compose.ui", name = "ui-test-junit4" }
compose-material3 = { group = "androidx.compose.material3", name = "material3" }
compose-material-icons-extended = { group = "androidx.compose.material", name = "material-icons-extended" }

# Room
room-runtime = { group = "androidx.room", name = "room-runtime", version.ref = "room" }
room-ktx = { group = "androidx.room", name = "room-ktx", version.ref = "room" }
room-compiler = { group = "androidx.room", name = "room-compiler", version.ref = "room" }

# Hilt
hilt-android = { group = "com.google.dagger", name = "hilt-android", version.ref = "hilt" }
hilt-android-compiler = { group = "com.google.dagger", name = "hilt-compiler", version.ref = "hilt" }
hilt-navigation-compose = { group = "androidx.hilt", name = "hilt-navigation-compose", version.ref = "hiltNavigationCompose" }

# Apache POI
poi = { group = "org.apache.poi", name = "poi", version.ref = "poi" }
poi-ooxml = { group = "org.apache.poi", name = "poi-ooxml", version.ref = "poi" }

```

```
# Accompanist
accompanist-systemuicontroller = { group = "com.google.accompanist", name = "accompanist-systemuicontroller", version.ref = "accompanistSystemUiController" }

# Camera & Barcode
androidx-camera-core = { group = "androidx.camera", name = "camera-core", version.ref = "camerax" }
androidx-camera-camera2 = { group = "androidx.camera", name = "camera-camera2", version.ref = "camerax" }
androidx-camera-lifecycle = { group = "androidx.camera", name = "camera-lifecycle", version.ref = "camerax" }
androidx-camera-view = { group = "androidx.camera", name = "camera-view", version.ref = "camerax" }
zxing-core = { group = "com.google.zxing", name = "core", version.ref = "zxing" }

# Testing
junit = { group = "junit", name = "junit", version.ref = "junit" }
androidx-junit = { group = "androidx.test.ext", name = "junit", version.ref = "junitExt" }
androidx-espresso-core = { group = "androidx.test.espresso", name = "espresso-core", version.ref = "espressoCore" }

[plugins]
android-application = { id = "com.android.application", version.ref = "agp" }
kotlin-android = { id = "org.jetbrains.kotlin.android", version.ref = "kotlin" }
kotlin-compose = { id = "org.jetbrains.kotlin.plugin.compose", version.ref = "kotlin" }
ksp = { id = "com.google.devtools.ksp", version.ref = "ksp" }
hilt = { id = "com.google.dagger.hilt.android", version.ref = "hilt" }
room = { id = "androidx.room", version.ref = "room" }
```

app\proguard-rules.pro

```
# ─── Ali Educational ProGuard Rules ──────────────────────────

# ── Room ──────────────────────────────────────────────────────
# Keep Room-generated Impl classes
-keep class * extends androidx.room.RoomDatabase { *; }
-keep @androidx.room.Entity class * { *; }
-keep @androidx.room.Dao interface * { *; }
-dontwarn androidx.room.paging.**

# ── Hilt / Dagger ─────────────────────────────────────────────
-dontwarn dagger.hilt.internal.**
-keep class dagger.hilt.** { *; }
-keep class javax.inject.** { *; }
-keep class * extends dagger.hilt.android.internal.managers.** { *; }
-keepclassmembers class * {
    @dagger.hilt.android.lifecycle.HiltViewModel *;
}

# ── Apache POI ────────────────────────────────────────────────
# POI uses reflection heavily; keep all POI classes
-keep class org.apache.poi.** { *; }
-keep class org.apache.xmlbeans.** { *; }
-keep interface org.apache.xmlbeans.** { *; }
-keep class schemaorg_apache_xmlbeans.** { *; }
-keep class org.openxmlformats.** { *; }
-keep class com.microsoft.schemas.** { *; }
-keep class org.apache.logging.** { *; }
-keep class org.apache.commons.** { *; }
-dontwarn org.apache.poi.**
-dontwarn org.apache.xmlbeans.**
-dontwarn org.openxmlformats.**
-dontwarn org.apache.logging.**
-dontwarn org.apache.commons.**
-dontwarn javax.xml.stream.**
-dontwarn org.apache.batik.**
-dontwarn java.awt.**
-dontwarn com.microsoft.schemas.**
-dontwarn com.graphbuilder.**
-dontwarn org.etsi.uri.**
-dontwarn org.bouncycastle.**

# ── Kotlin Serialization (if used later) ──────────────────────
-keepattributes *Annotation*, InnerClasses
-dontnote kotlinx.serialization.**

# ── Compose ───────────────────────────────────────────────────
# Compose is handled by default rules, but keep stability annotations
-keepclassmembers class * {
    @androidx.compose.runtime.Composable <methods>;
}

# ── General ───────────────────────────────────────────────────
-keepattributes Signature
-keepattributes *Annotation*
-keepattributes SourceFile,LineNumberTable
-renamesourcefileattribute SourceFile
```